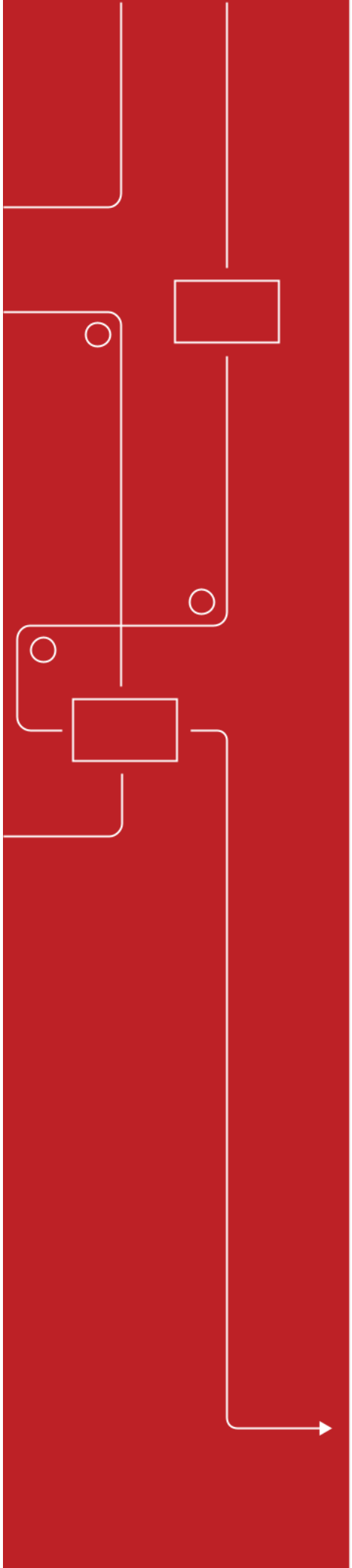


Red Hat
Training and Certification

Developing Advanced Automation with Red Hat Ansible Automation Platform

Red Hat Ansible Automation Platform 2.5 AU374

Student Workbook

Edition 2

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Table of Contents

Developing Advanced Automation with Red Hat Ansible Automation Platform

Document Conventions

Admonitions

Inclusive Language

Introduction

Developing Advanced Automation with Red Hat Ansible Automation Platform

Orientation to the Classroom Environment

Performing Lab Exercises

Viewing the Student Guide

Chapter 1. Developing Playbooks with Ansible Automation Platform 2

1.1. Red Hat Ansible Automation Platform 2

1.2. Quiz Red Hat Ansible Automation Platform 2

1.3. Running Playbooks with Automation Content Navigator

1.4. Guided Exercise Running Playbooks with Automation Content Navigator

1.5. Managing Ansible Project Materials Using Git

1.6. Guided Exercise Managing Ansible Project Materials Using Git

1.7. Implementing Recommended Ansible Practices

1.8. Guided Exercise Implementing Recommended Ansible Practices

1.9. Lab Developing Playbooks with Ansible Automation Platform

1.10. Summary

Chapter 2. Managing Ansible Content Collections and Automation Execution Environments

2.1. Reusing Content from Ansible Content Collections

2.2. Guided Exercise Reusing Content from Ansible Content Collections

2.3. Finding and Installing Ansible Content Collections

2.4. Guided Exercise Finding and Installing Ansible Content Collections

2.5. Selecting an Automation Execution Environment

2.6. Guided Exercise Selecting an Automation Execution Environment

2.7. Lab Managing Ansible Content Collections and Automation Execution Environments

2.8. Summary

Chapter 3. Running Playbooks with Automation Controller

3.1. Explaining the Automation Controller Architecture

3.2. Quiz Explaining the Automation Controller Architecture
3.3. Creating Automation Controller Resources
3.4. Guided Exercise Creating Automation Controller Resources
3.5. Lab Running Playbooks with Automation Controller
3.6. Summary
Chapter 4. Working with Ansible Configuration Settings
4.1. Examining the Ansible Configuration with Automation Content Navigator
4.2. Guided Exercise Examining the Ansible Configuration with Automation Content Navigator
4.3. Configuring Automation Content Navigator
4.4. Guided Exercise Configuring Automation Content Navigator
4.5. Developing Content in Visual Studio Code
4.6. Guided Exercise Developing Content in Visual Studio Code
4.7. Lab Working with Ansible Configuration Settings
4.8. Summary
Chapter 5. Managing Inventories
5.1. Managing Dynamic Inventories
5.2. Guided Exercise Managing Dynamic Inventories
5.3. Writing YAML Inventory Files
5.4. Guided Exercise Writing YAML Inventory Files
5.5. Managing Variables
5.6. Guided Exercise Managing Variables for Inventory Hosts and Groups
5.7. Lab Managing Inventories
5.8. Summary
Chapter 6. Managing Task Execution
6.1. Controlling Privilege Escalation
6.2. Guided Exercise Controlling Privilege Escalation
6.3. Controlling Task Execution
6.4. Guided Exercise Controlling Task Execution
6.5. Running Selected Tasks
6.6. Guided Exercise Running Selected Tasks
6.7. Optimizing Execution for Speed
6.8. Guided Exercise Optimizing Execution for Speed
6.9. Lab Managing Task Execution

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

6.10. Summary

Chapter 7. Transforming Data with Filters and Plug-ins

7.1. Processing Variables Using Filters

7.2. Guided Exercise Processing Variables Using Filters

7.3. Templating External Data Using Lookups

7.4. Guided Exercise Templating External Data Using Lookups

7.5. Implementing Advanced Loops

7.6. Guided Exercise Implementing Advanced Loops

7.7. Using Filters to Work with Network Addresses

7.8. Guided Exercise Using Filters to Work with Network Addresses

7.9. Lab Transforming Data with Filters and Plug-ins

7.10. Summary

Chapter 8. Coordinating Rolling Updates

8.1. Delegating Tasks and Facts

8.2. Guided Exercise Delegating Tasks and Facts

8.3. Configuring Parallelism

8.4. Guided Exercise Configuring Parallelism

8.5. Managing Rolling Updates

8.6. Guided Exercise Managing Rolling Updates

8.7. Lab Coordinating Rolling Updates

8.8. Summary

Chapter 9. Creating Ansible Content Collections and Automation Execution Environments

9.1. Writing Ansible Content Collections

9.2. Guided Exercise Writing Ansible Content Collections

9.3. Building a Custom Automation Execution Environment

9.4. Guided Exercise Building a Custom Automation Execution Environment

9.5. Validating a Custom Automation Execution Environment

9.6. Guided Exercise Validating a Custom Automation Execution Environment

9.7. Using Custom Ansible Content Collections and Automation Execution Environments in Automation Controller

9.8. Guided Exercise Using Custom Ansible Content Collections and Automation Execution Environments in Automation Controller

9.9. Lab Creating Ansible Content Collections and Automation Execution Environments

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

9.10. Summary

Chapter 10. Comprehensive Review

10.1. Summary of Learning Objectives

10.2. Lab Managing Inventory Variables to Use with Automation Content Navigator

10.3. Lab Optimizing a Playbook for Large-scale Use

10.4. Lab Creating and Using Ansible Content Collections and Automation Execution Environments

10.5. Summary

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Developing Advanced Automation with Red Hat Ansible Automation Platform

Red Hat Ansible Automation Platform 2.5 AU374

Edition 2

Publication date 2025-08-01

Authors: Mike Kelly, Michael Phillips, Alejandra Ramírez, Dallas Spohn, Morgan Weetman

Course Architects: Steve Bonneville

DevOps Engineers: Travis Michette

Editors: David O'Brien

Contributors: Pat Sharp, Roberto Velázquez Barajas, Sajith Eyamkuzhy Sugathan, Wasim Raja, Rajat Agrawal

© 2025 Red Hat, Inc.

The contents of this course and all its modules and related materials, including handouts to audience members, are © 2025 Red Hat, Inc.

No part of this publication may be stored in a retrieval system, transmitted or reproduced in any way, including, but not limited to, photocopy, photograph, magnetic, electronic or other record, without the prior written permission of Red Hat, Inc.

This instructional program, including all material provided herein, is supplied without any guarantees from Red Hat, Inc. Red Hat, Inc. assumes no liability for damages or legal action arising from the use or misuse of contents or details contained herein.

If you believe Red Hat training materials are being used, copied, or otherwise improperly distributed, please send email to training@redhat.com or phone toll-free (USA) +1 (866) 626-2994 or +1 (919) 754-3700.

Red Hat, Red Hat Enterprise Linux, the Red Hat logo, JBoss, OpenShift, Fedora, Hibernate, Ansible, RHCA, RHCE, RHCSA, Ceph, and Gluster are trademarks or registered trademarks of Red Hat, Inc. or its subsidiaries in the United States and other countries.

Linux® is the registered trademark of Linus Torvalds in the United States and other countries.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Java® is a registered trademark of Oracle American, Inc. and/or its affiliates.

XFS® is a registered trademark of Hewlett Packard Enterprise Development LP or its subsidiaries in the United States and/or other countries.

MySQL® is a registered trademark of MySQL AB in the United States, the European Union and other countries.

Node.js® is a trademark of Joyent. Red Hat is not formally related to or endorsed by the official Joyent Node.js open source or commercial project.

The OpenStack word mark and the Square O Design, together or apart, are trademarks or registered trademarks of OpenStack Foundation, in the United States and other countries and are used with the OpenStack Foundation's permission. Red Hat, Inc. is not affiliated with, endorsed by, or sponsored by the OpenStack Foundation or the OpenStack community.

All other trademarks are the property of their respective owners.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Document Conventions

This section describes various conventions and practices that are used throughout all Red Hat Training courses.

Admonitions

Red Hat Training courses use the following admonitions:

References
References describe where to find external documentation that is relevant to a subject.

Note
Notes are tips, shortcuts, or alternative approaches to the task at hand. Ignoring a note should have no negative consequences, but you might miss out on something that makes your life easier.

Important
Important sections provide details of information that is easily missed: configuration changes that apply only to the current session, or services that need restarting before an update applies. Ignoring these admonitions will not cause data loss, but might cause irritation and frustration.

Warning
Do not ignore warnings. Ignoring these admonitions will most likely cause data loss.

Inclusive Language

Red Hat Training is currently reviewing its use of language in various areas to help remove any potentially offensive terms. This is an ongoing process and requires alignment with the products and services that are covered in Red Hat Training courses. Red Hat appreciates your patience during this process.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Introduction

Developing Advanced Automation with Red Hat Ansible Automation Platform

Developing Advanced Automation with Red Hat Ansible Automation Platform (AU374) is designed for automation content developers to use container-focused tools from Red Hat Ansible Automation Platform to efficiently develop automation that can be managed by the automation controller.

Course Objectives

- Use the capabilities of automation content navigator to develop Ansible Playbooks.
- Apply recommended practices for effective and efficient automation with Ansible.
- Use advanced features of Ansible to work with data, including filters and plug-ins.
- Perform automation operations as rolling updates.
- Create automation execution environments to bundle and distribute the dependencies needed by automation code.

Audience

- This course is designed for users who create recommended design patterns and operate automation practices at scale, including the following roles:
  - DevOps engineers
  - Linux system administrators
  - Developers
  - Other IT professionals with some experience using Red Hat Ansible Automation Platform to automate, provision, configure, and deploy applications and services in a Linux environment

Prerequisites

- Red Hat Certified Engineer (RHCE) certification (EX294) on Red Hat Enterprise Linux 9 or later, or equivalent Ansible experience.

Orientation to the Classroom Environment

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

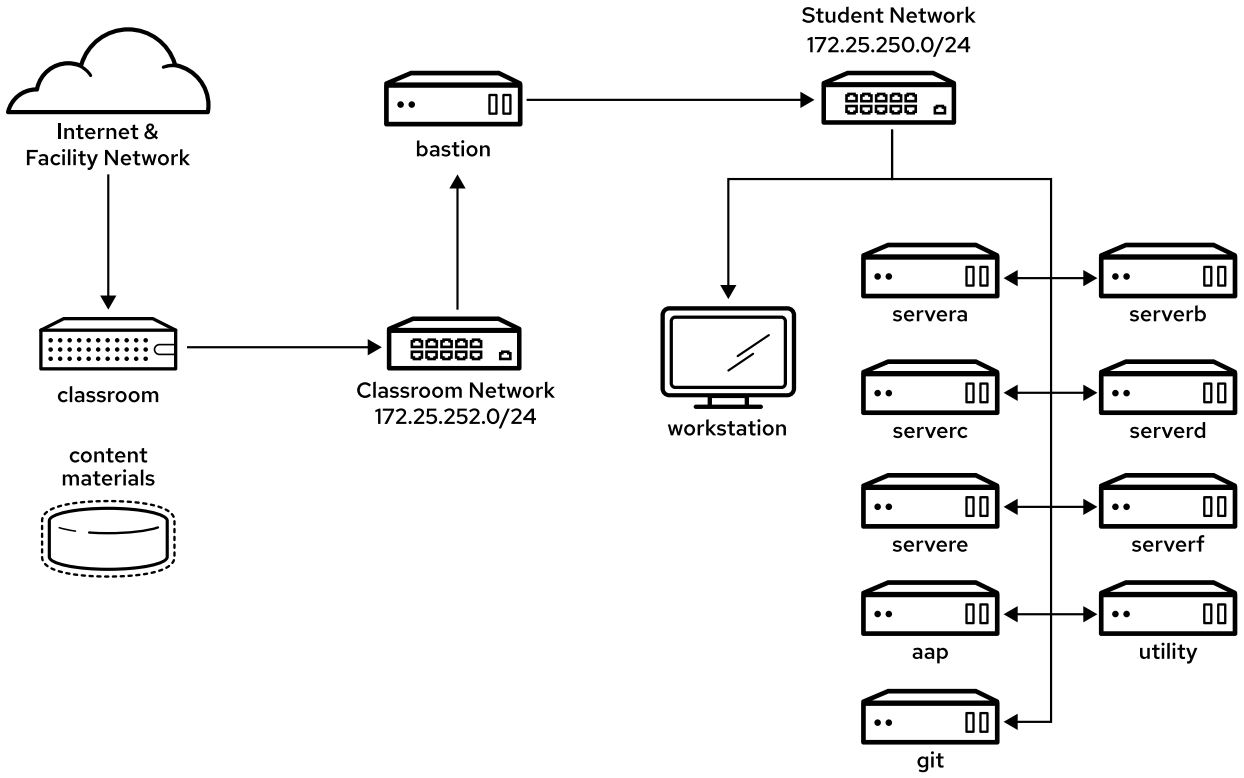


Figure 1. Classroom environment

In this course, the main computer system used for hands-on learning activities is the `workstation` machine. Other machines are also used by students for these activities: `servera`, `serverb`, `serverc`, `serverd`, `servere`, `serverf`, `aap`, `git`, and `utility`. All machines are in the `lab.example.com` DNS domain.

All student computer systems have a standard user account, `student`, with `student` as the password. The `root` password on all student systems is `redhat`.

Table 1. Classroom Machines

Machine name	IP addresses	Role
`bastion.lab.example.com`	`172.25.250.254`	Router linking VMs to central servers
`workstation.lab.example.com`	`172.25.250.9`	Graphical workstation used for system administration
`utility.lab.example.com`	`172.25.250.220`	Server hosting course-specific support services
`git.lab.example.com`	`172.25.250.5`	GitLab server hosting course repositories

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Machine name	IP addresses	Role
servera.lab.example.com	172.25.250.10	Managed server "A"
serverb.lab.example.com	172.25.250.11	Managed server "B"
serverc.lab.example.com	172.25.250.12	Managed server "C"
serverd.lab.example.com	172.25.250.13	Managed server "D"
servere.lab.example.com	172.25.250.14	Managed server "E"
serverf.lab.example.com	172.25.250.15	Managed server "F"
aap.lab.example.com	172.25.250.30	Ansible Automation Platform

The bastion system acts as a router between the network that connects the student machines and the classroom network. If bastion is down, other student machines might not function properly or might even hang during boot.

The classroom system should always be running for proper use of the lab environment. The http://content.example.com URL is an alias to the classroom system and this URL provides software used in some hands-on activities.

The https://aap.lab.example.com URL provides access to the web-based user interface of the Ansible Automation Platform unified UI which provides access to resources such as private automation hub and automation controller. You can log in as the admin user, with redhat as the password.

Connecting to Managed Servers for Ansible

You write and run Ansible Playbooks to automate tasks in a number of hands-on activities. To automate your managed hosts, they have been preconfigured to allow easier access from workstation (your Ansible control node).

- The devops user on your managed servers has been preconfigured to allow key-based SSH authentication from the student account on workstation.
- To simplify privilege escalation, the devops user is also allowed to use sudo to run any command as root without entering a password. You do not need to directly log in to your managed servers as root.

Your workstation system also has Ansible Core and automation content navigator preinstalled for automation activities. Often, an automation activity also provides default ansible.cfg and ansible-navigator.yml files,

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

an inventory, and possibly partially completed playbooks or other materials.

Controlling Your Systems

In an instructor-led training classroom, you are assigned a physical computer (`foundation`*X*`.ilt.example.com`), which accesses the virtual machines that run on that host. You are automatically logged in to the physical machine as the `kiosk` user, by using `redhat` as the password. The classroom systems that you work with are virtual machines that run on that host.

On `foundation`*X*, you use the `rht-vmctl` and `rht-vmview` commands to work with the virtual machines. Run the commands in the following table as the `kiosk` user on `foundation`*X*. You can use these commands with any of the virtual machines.

Table 2. `rht-vmctl` Commands

Action	Command
Start the virtual machine named `server`.	`rht-vmctl start server`
Display the system console for the virtual machine named `server`, to log in and work on that system.	`rht-vmview view server`
Reset the virtual machine named `server` to its initial course state and restart it.	`rht-vmctl reset server`

At the start of a lab exercise, if the "reset your `servera`" instruction appears, then run the `rht-vmctl reset servera` command on the `foundation`*X* system as the `kiosk` user. Likewise, if the "reset your workstation" instruction appears, then run the `rht-vmctl reset workstation` command on `foundation`*X* as the `kiosk` user.

Performing Lab Exercises

There are four types of lab activities that you might see in this course.

- A *guided exercise* is a hands-on practice exercise that follows a presentation section. It walks you through a procedure to perform, step-by-step.
- A *quiz* is typically used when checking knowledge-based learning, or when a hands-on activity is impractical for some other reason.
- An *end-of-chapter lab* is a gradable hands-on activity to help you check your learning. You are provided with a set of high-level steps to perform, based on the guided exercises in that chapter, but the steps do not walk you

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

through every command. You are also given a solution that provides a step-by-step walk-through.

- A *comprehensive review lab* is used at the end of the course. It is also a gradable hands-on activity, but it might cover content from throughout the entire course. You are provided with a specification that details what you need to accomplish in the activity, but not the specific steps to do so. Again, you are given a solution that provides a step-by-step walk-through that meets the specification.

To prepare your lab environment at the start of each hands-on activity, run the `lab start` command with an activity name specified by the activity's instructions. Likewise, at the end of each hands-on activity, run the `lab finish` command with that same activity name to clean up after the activity. Each hands-on activity has a unique name within a course.

The syntax for running an exercise script is:

```
[student@workstation ~]$ lab action exercise
```

The *action* is a choice of `start`, `grade`, or `finish`. All exercises support `start` and `finish` actions. Only end-of-chapter labs and comprehensive review labs support the `grade` action.

start

The `start` action verifies the required resources to begin an exercise. It might include configuring settings, creating resources, ensuring prerequisite services are running, and verifying necessary outcomes from previous exercises. You can take an exercise at any time, even without taking the preceding exercises.

grade

For gradable activities, the `grade` action directs the `lab` command to evaluate your work, and displays a list of grading criteria with a PASS or FAIL status for each. To achieve a PASS status for all criteria, fix any failures and rerun the `grade` action.

finish

The `finish` action cleans up resources configured during the exercise. You can take an exercise as many times as you want.

The `lab` command supports tab completion. For example, to list all exercises that you can start, enter `lab start` and press the Tab key twice.

Viewing the Student Guide

Commands and code blocks in the student guide are best viewed when your preferred viewer displays at least 80 characters per line. If you adjust your viewer to display fewer than 80 characters per line, then commands and code blocks might wrap at unexpected locations. An unexpected line break might lead to confusion and to running a command incorrectly or to introducing a mistake in a file.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

The following line is 80 characters long. For the best viewing experience, adjust your viewer so that the following line does not wrap. If you are using a web browser, then you might adjust the zoom percentage. For an EPUB viewer, you might adjust the orientation and the font size.

```
================================================================================
```

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Chapter 1.
Developing Playbooks with Ansible Automation Platform 2

Goal

Develop Ansible Playbooks with Red Hat Ansible Automation Platform 2 following recommended practices.

Sections

- Red Hat Ansible Automation Platform 2 (and Quiz)
- Running Playbooks with Automation Content Navigator (and Guided Exercise)
- Managing Ansible Project Materials Using Git (and Guided Exercise)
- Implementing Recommended Ansible Practices (and Guided Exercise)

Lab

- Developing Playbooks with Ansible Automation Platform

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

1.1. Red Hat Ansible Automation Platform 2

Objectives

- Describe the architecture of Red Hat Ansible Automation Platform 2 and how its new features help with Ansible automation development.

Orientation to Red Hat Ansible Automation Platform 2

Red Hat Ansible Automation Platform 2 is the next evolution in automation from Red Hat. It is a distributed, modular solution, featuring tools, services, and capabilities that offer a whole level of customization and control.

Ansible Automation Platform 2 delivers an elevated automation experience that expands the boundaries of what is possible for your enterprise, and it can help you achieve goals across many departments.

- Automate for velocity and speed up business outcomes.
- Automate for collaboration and orchestrate across teams.
- Automate for growth and innovate at scale.

To install the latest available release, Ansible Automation Platform 2.5, you can choose between three methods:

Containerized

Ideal for rapid scaling, because it provides sufficient flexibility and simplifies dependency management. It is available from the Red Hat Customer portal.

RPM

Convenient for traditional Red Hat Enterprise Linux environments. It is available from the Red Hat Customer portal.

OpenShift operator

Provides seamless integration with OpenShift. It is available through OperatorHub in Red Hat OpenShift Container Platform.

For more details on the system requirements for containerized, RPM, and OpenShift operator installations, refer to the links in the References section.

Ansible Automation Platform 2 Components

Red Hat Ansible Automation Platform 2 includes a number of distinct components that together provide a complete and integrated set of automation tools and resources.

Ansible Core

Red Hat Ansible Automation Platform 2.5 provides Ansible Core 2.16 in the `ansible-core` RPM package. Ansible Core provides the fundamental functionality used to run Ansible Playbooks:

- Defines the automation language that is used to write Ansible Playbooks in YAML text files

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

- Provides the functions such as loops, conditionals, and other Ansible imperatives needed for automation code
- Provides the framework and basic command-line tools to drive automation

Ansible Content Collections

Historically, Ansible provided many modules as part of the core package; an approach referred to in the Ansible community as "batteries included". However, with the success and rapid growth of Ansible, the number of modules included with Ansible grew exponentially. This led to certain challenges with support, especially because users sometimes wanted to use earlier or later versions of modules that were packaged with a particular version of Ansible.

The upstream developers decided to reorganize most modules into separate *Ansible Content Collections*, made up of related modules, roles, and plug-ins that are supported by the same group of developers. Ansible Core itself is limited to a small set of modules provided by the `ansible.builtin` Ansible Content Collection, which is always part of Ansible Core.

Ansible Content Collections are versioned. This provides users with the flexibility to select different versions of collections, or different sets of collections, based on their needs. It also provides developers with the ability to update their modules on a separate cadence from Ansible itself.

Ansible Content Collections are published in *namespaces*. Namespaces can be a company, a brand, or an organization responsible for managing the collection. Certain collections, such as *Red Hat Ansible Certified Content Collections*, are officially supported by Red Hat and its partners through Ansible Automation Platform.

Automation Execution Environments

An *automation execution environment* is a container image that contains Ansible Core, Ansible Content Collections, and any Python libraries, executables, or other dependencies needed to run your playbook. Each automation execution environment is built on a Red Hat Universal Base Image (UBI).

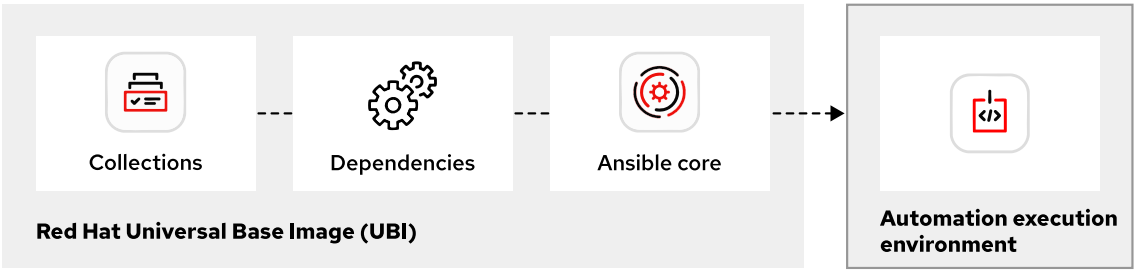


Figure 2. Automation execution environment content

The Red Hat Ecosystem Catalog (https://catalog.redhat.com/search?searchType=containers&build_categories_list=Automation%20execution%20environment&p=1) includes prebuilt and supported automation execution environments provided by Red Hat. For example, the following automation execution environments are hosted under the `ansible-automation-platform-25` namespace:

Red Hat minimal automation execution environments

- `ansible-automation-platform-25/ee-minimal-rhel8:latest`

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

- `ansible-automation-platform-25/ee-minimal-rhel9:latest`

Red Hat supported automation execution environments

- `ansible-automation-platform-25/ee-supported-rhel8:latest`
- `ansible-automation-platform-25/ee-supported-rhel9:latest`

Using automation execution environments makes it easier for you to provide a complete working environment for your automation code, from development to production.

Execution Environment Builder

Provided by Ansible Automation Platform 2, *execution environment builder* (`ansible-builder`) is a command-line tool that allows you to create your own custom automation execution environments.

After the automation execution environment is built, you can push the container image to a centralized container repository such as private automation hub.

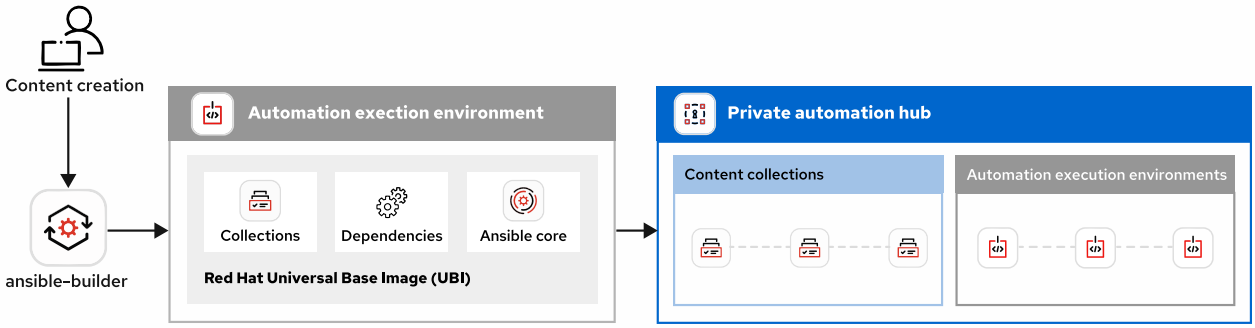


Figure 3. User experience: Adapting execution environments to your needs

Automation Content Navigator

Ansible Automation Platform 2 provides a top-level tool, *automation content navigator* (`ansible-navigator`), to develop, test, and run Ansible Playbooks. This tool replaces and extends the functionality of several earlier command-line utilities, including `ansible-playbook`, `ansible-inventory`, and `ansible-config`.

In addition, automation content navigator separates the control node, where you run Ansible, from the automation execution environment containing all the necessary dependencies for the tasks being executed. This separation makes it easier to create a complete working environment for your automation code, from development to production.

When you run a playbook with `ansible-navigator`, you can select an automation execution environment for it to use to run that playbook. When your code is working, you can provide the playbook and the automation execution environment to automation controller and know that automation controller has everything it needs to correctly run your playbook.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

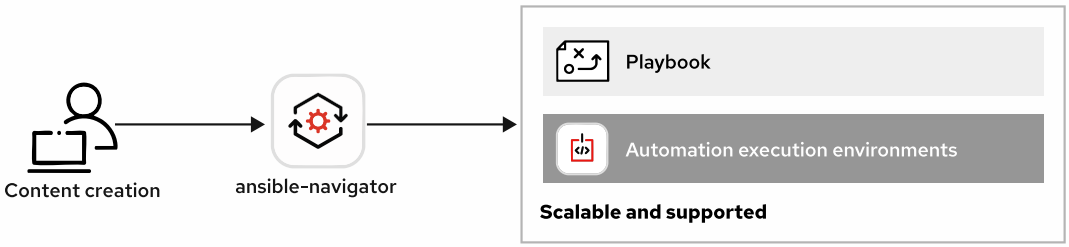


Figure 4. Develop, test, and deploy Ansible content by using automation content navigator

You can use `ansible-navigator` also to inspect the local environment, troubleshoot runtime issues, and review and replay previous runs.

Unified User Interface

Starting with Ansible Automation Platform 2.5, Automation controller, Event-Driven Ansible controller, and private automation hub, are all integrated in a single *unified user interface (unified UI)*.

The unified UI allows you to have all your automation tools in one place, making it easier to collaborate among teams across your organization, such as operators, developers, and platform administrators.

The unified UI provides you with the following features:

- Central point of control to run your enterprise automation code
- Web UI and REST API that can be used to configure, run, and evaluate your automation jobs
- Role-based access control (RBAC)
- Centralized logging
- Credential management
- Ansible analytics integration

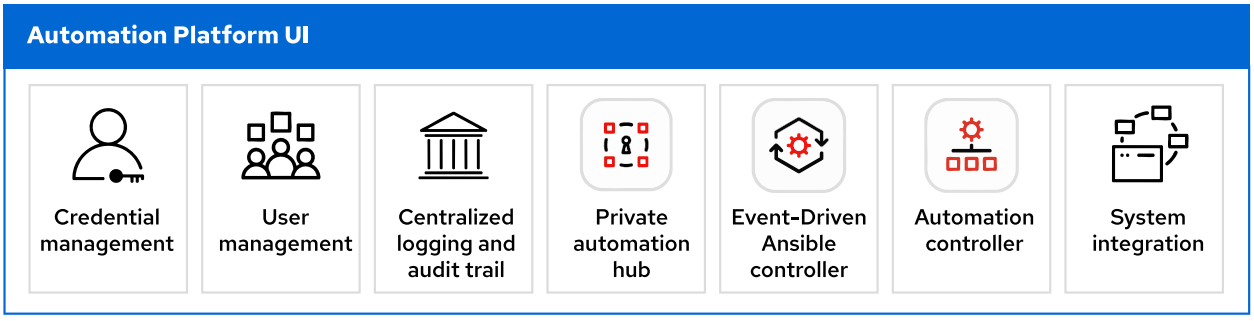


Figure 5. Ansible Automation Platform Unified UI

Automation execution (automation controller)

Automation execution refers to automation controller. It is the component of Ansible Automation Platform that provides a central point of control to run your enterprise automation code.

Automation execution provides you with access to the Ansible controller resources, such as inventories, credentials, templates, projects, jobs, and so on.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Automation decisions (Event-Driven Ansible controller)

Automation decisions refers to Event-Driven Ansible controller, which provides the event-handling capability needed to automate tasks that respond to changing conditions in your organization.

Automation decisions supports simplified event routing, highly available clusters, horizontal scaling, and other improvements.

Automation content (private automation hub)

Automation content refers to private automation hub. It provides a centralized place for the organization's Ansible developers to obtain automation content. It also offers a container registry that you can use for distributing your automation execution environments.

Automation analytics

Automation analytics helps to provide better insight into the performance of your automation infrastructure. You can use automation analytics to analyze what modules, playbooks, and workflows you most often use, as well as measure the success of jobs and tasks. By using automation analytics, you can calculate the return on investment (ROI) on your organization.

Before Ansible Automation Platform 2.5, automation analytics only existed as a resource outside the integrated platform experience given by the unified UI.

Hosted Services

The public service at `https://console.redhat.com` provides some additional hosted services, such as Ansible automation hub, Insights for Ansible Automation Platform, and even automation analytics.

Ansible automation hub

Ansible automation hub provides you with a way to manage and distribute automation content. The following certified collections are just a few of the collections that you can get from Ansible automation hub:

- `ansible.hub`
- `ansible.eda`
- `ansible.controller`
- `ansible.platform`

Insights for Ansible Automation Platform

Insights for Ansible Automation Platform helps you understand your automation use across the business and establish common approaches and practices to automation workflows. You can use Insights for Ansible Automation Platform, for example, to evaluate the positive impact of automation on your organization.

Red Hat Ansible Automation Platform 2 Architecture

The following diagram shows the components of a Red Hat Ansible Automation Platform 2 deployment alongside the personas of some people who would use them.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

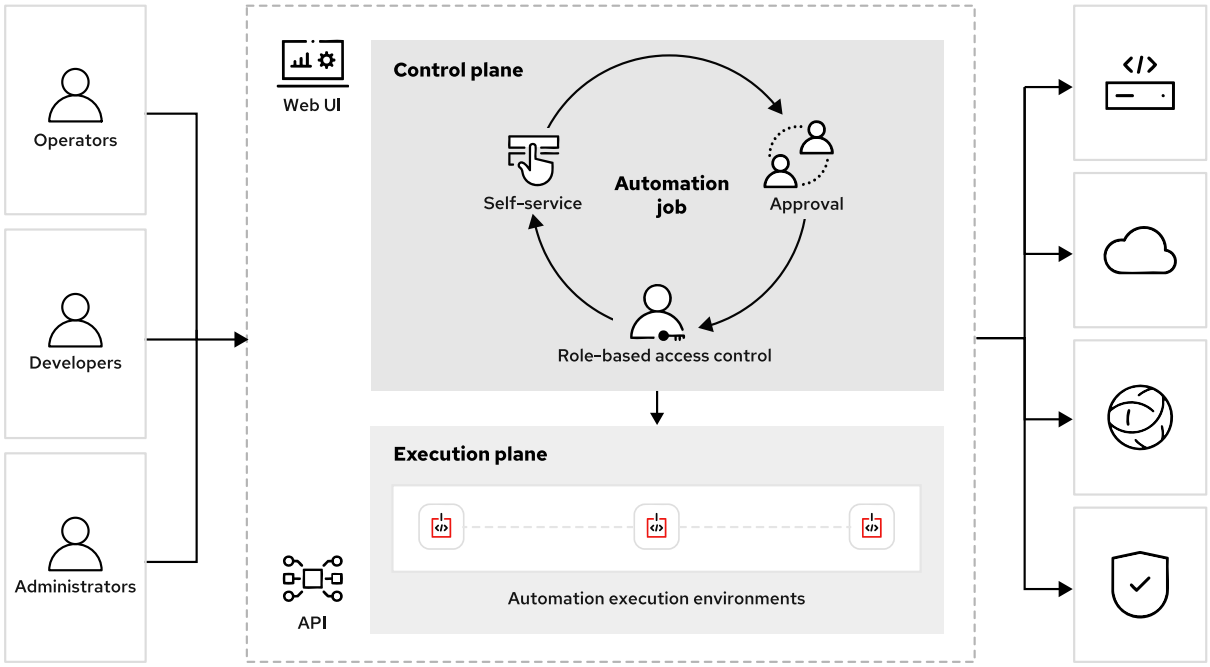


Figure 6. Architecture of Red Hat Ansible Automation Platform 2

Operators, developers, and platform administrators from your organization access Ansible Automation Platform either via the web UI or through the API. Depending on the role you assign, and the privileges you grant, these users can create projects, launch workflow job templates, schedule jobs, and so on. Access to Ansible Automation Platform is governed by enterprise RBAC support and configured approval processes, ensuring that users only have access to required resources.

See the References section for information about more advanced infrastructure topologies or reference architectures provided by Red Hat.

The platform supports deployment in physical, virtual, cloud, and container environments, offering flexible design options that enable you to automate across data centers, hybrid cloud, and edge environments.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

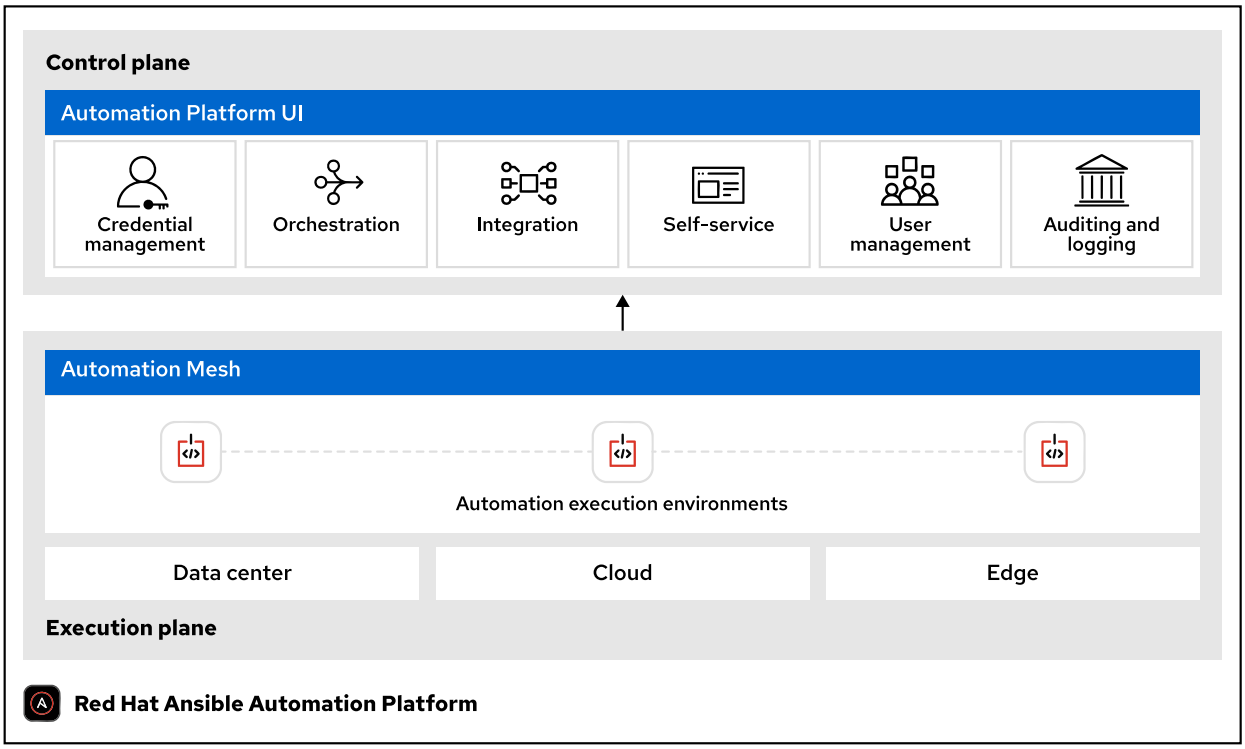


Figure 7. Ansible Automation Platform distributed architecture

References

Red Hat Ansible Automation Platform

Getting started with Ansible Automation Platform

Tested Deployment Models

Planning your Installation: System Requirements

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

1.2. Quiz
Red Hat Ansible Automation Platform 2

Choose the correct answers to the following questions:

1. Which component replaces commands such as `ansible-playbook` and `ansible-doc`?

- [ ] **a.** Automation hub
- [ ] **b.** Ansible Core
- [ ] **c.** Automation content navigator
- [ ] **d.** Ansible Playbook editor

2. Which service provides automation content officially supported by Red Hat?

- [ ] **a.** Ansible automation hub
- [ ] **b.** Red Hat Enterprise Ansible Cloud
- [ ] **c.** Ansible Galaxy
- [ ] **d.** Ansible content platform

3. Automation content navigator separates which two functions to make it a complete working environment?

- [ ] **a.** Network management and host management
- [ ] **b.** Content plane and configuration plane
- [ ] **c.** Management network and production network
- [ ] **d.** Control node and automation execution environment

4. Which three of the following components are integrated in the unified UI?

- [ ] **a.** Red Hat Insights for Red Hat Ansible Automation Platform
- [ ] **b.** Automation controller
- [ ] **c.** Event-Driven Ansible
- [ ] **d.** Private automation hub

5. Which Ansible Content Collection is always included with `ansible-core`?

- [ ] **a.** `community.general`
- [ ] **b.** `ansible.builtin`
- [ ] **c.** `ansible.utils`
- [ ] **d.** `ansible.posix`
- [ ] **e.** `kubernetes.core`
- [ ] **f.** `ansible.netcommon`

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Solution

1. Which component replaces commands such as `ansible-playbook` and `ansible-doc`?
   - ✗ **a.** Automation hub
   - ✗ **b.** Ansible Core
   - ✓ **c. Automation content navigator**
   - ✗ **d.** Ansible Playbook editor
2. Which service provides automation content officially supported by Red Hat?
   - ✓ **a. Ansible automation hub**
   - ✗ **b.** Red Hat Enterprise Ansible Cloud
   - ✗ **c.** Ansible Galaxy
   - ✗ **d.** Ansible content platform
3. Automation content navigator separates which two functions to make it a complete working environment?
   - ✗ **a.** Network management and host management
   - ✗ **b.** Content plane and configuration plane
   - ✗ **c.** Management network and production network
   - ✓ **d. Control node and automation execution environment**
4. Which three of the following components are integrated in the unified UI?
   - ✗ **a.** Red Hat Insights for Red Hat Ansible Automation Platform
   - ✓ **b. Automation controller**
   - ✓ **c. Event-Driven Ansible**
   - ✓ **d. Private automation hub**
5. Which Ansible Content Collection is always included with `ansible-core`?
   - ✗ **a.** `community.general`
   - ✓ **b.** `ansible.builtin`
   - ✗ **c.** `ansible.utils`
   - ✗ **d.** `ansible.posix`
   - ✗ **e.** `kubernetes.core`
   - ✗ **f.** `ansible.netcommon`

1.3. Running Playbooks with Automation Content Navigator

Objectives

- Install automation content navigator and use it to run an existing playbook with a supported automation execution environment.

Automation Content Navigator

Automation content navigator (`ansible-navigator`) is a tool introduced in Red Hat Ansible Automation Platform 2 that combines the features formerly provided by `ansible-playbook`, `ansible-inventory`, `ansible-config`, and `ansible-doc` in one top-level interface.

Automation content navigator offers an interactive mode with a text-based user interface (TUI), and can also be run using the `--mode stdout` (or `-m stdout`) option to provide output in the original format used by the earlier tools.

For example, to run a playbook with `ansible-playbook`, you might enter the following command:

```
[user@host ~]$ ansible-playbook playbook.yml -i inventory
```

You can use the `ansible-navigator` command to do the same thing and get output from the playbook in the same format, as follows:

```
[user@host ~]$ ansible-navigator run playbook.yml -i inventory -m stdout
```

However, if you omit the `-m stdout` option for the `ansible-navigator` command, it starts an interactive mode that summarizes the output of the playbook run in real time. This enables you to investigate the results in a TUI after the run completes.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

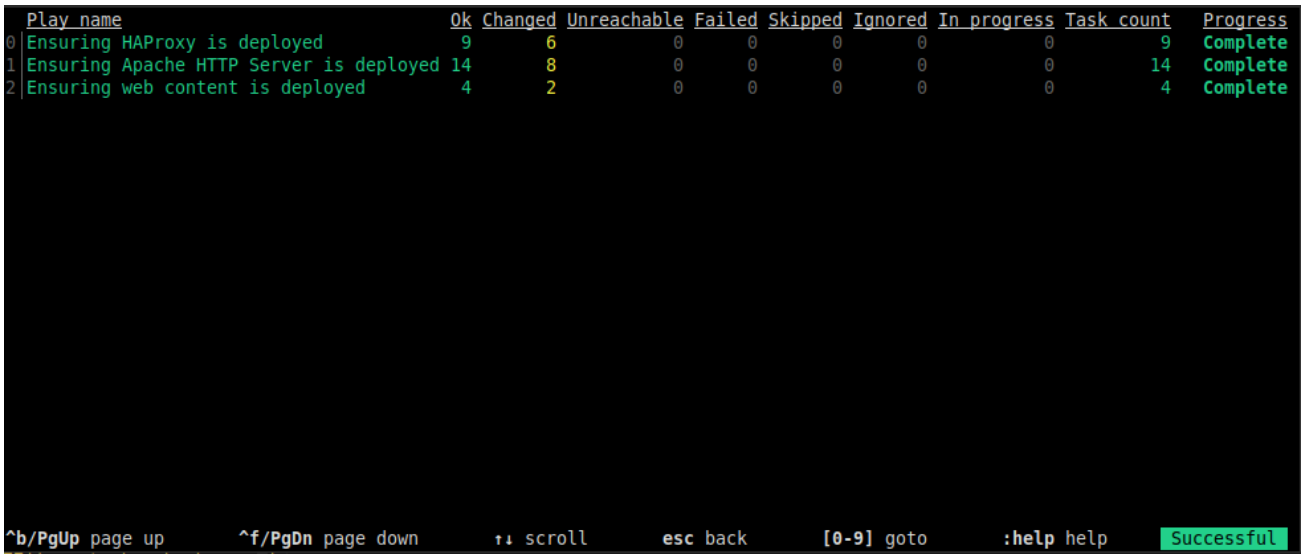


Figure 8. Output of a playbook run with automation content navigator

In the preceding image, you can see that the `ansible-navigator` command ran a playbook containing three plays:

- The first play, "Ensuring HAProxy is deployed", ran nine tasks, six of which made changes.
- The second play, "Ensuring Apache HTTP Server is deployed", ran 14 tasks, eight of which made changes.
- The third play, "Ensuring web content is deployed", ran four tasks, two of which made changes.

You can get more information about the tasks that were run by the first play by pressing `0`, or by typing `:0` and then pressing `Enter` in `ansible-navigator`:



Figure 9. Details of a play run with automation content navigator

You can also get detailed information about particular tasks. In the preceding example, if you press 2 or type `:2` and then press `Enter`, then details of the "Ensure HAProxy is started and enabled" task are displayed, and you can use the arrow keys to scroll through the results:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

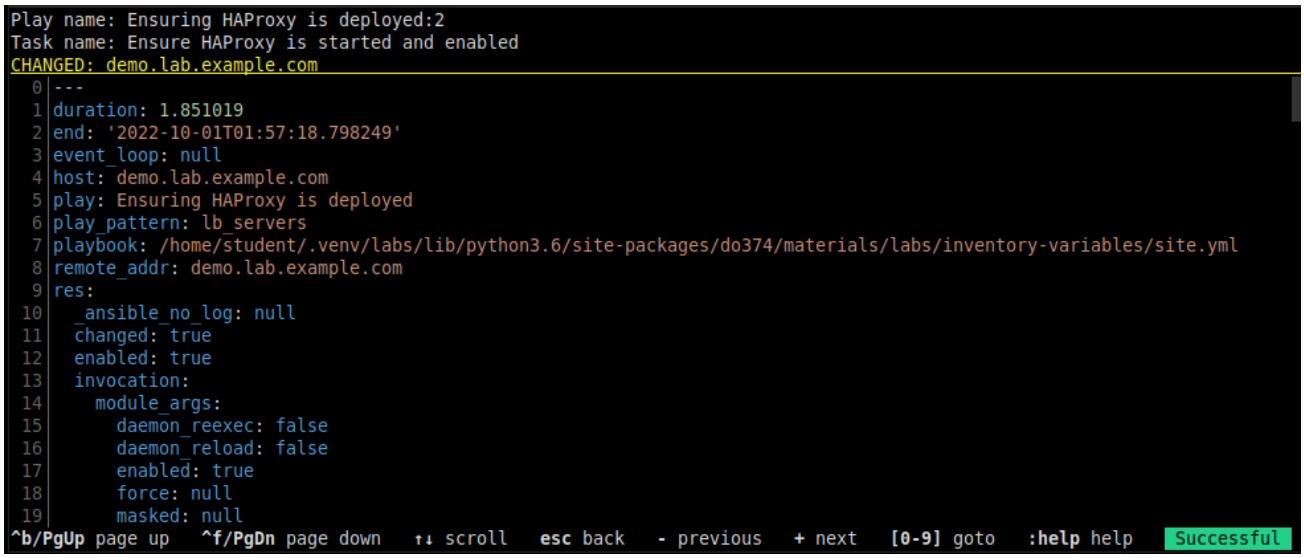


Figure 10. Details of a task in a play run with automation content navigator

To go back to previous pages or exit from `ansible-navigator` at the top-level page, press Esc.

Improving Portability with Automation Execution Environments

One feature of Red Hat Ansible Automation Platform 2 is support for *automation execution environments*. An automation execution environment is a container image that includes Ansible Content Collections, their software dependencies, and a minimal Ansible engine that can run your playbooks and interact with Ansible. Both `ansible-navigator` and automation controller (formerly known as Red Hat Ansible Tower) use automation execution environments to simplify the development, testing, and deployment of your automation code in a consistent Ansible environment.

Red Hat provides several supported automation execution environments consisting of slightly different Ansible deployments. You can also create your own customized automation execution environments.

With an automation execution environment, you can avoid creating multiple Python virtual environments on automation controller. Red Hat Ansible Tower required multiple Python virtual environments to support different combinations of Ansible, modules, Python libraries, and other software dependencies. This was hard to develop, hard to deploy, and hard to manage. Red Hat Ansible Automation Platform 2 uses multiple automation execution environments rather than multiple Python virtual environments. In Ansible Automation Platform, you can create a custom automation execution environment, test it with `ansible-navigator`, and distribute it to automation controller as a container image.

When you use the `ansible-navigator` command to run a playbook, and do not specify a particular automation execution environment, `ansible-navigator` automatically attempts to pull the default automation execution environment from `registry.redhat.io` if it is not already present on the system. For Red Hat Ansible Automation Platform 2.5, this default environment includes Ansible Core 2.16 and a standard set of Red Hat Ansible Certified Content Collections.

By default, the `ansible-navigator` command in Ansible Automation Platform 2.5 attempts to download and use the container image available at `registry.redhat.io/ansible-automation-platform-25/ee-`

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

supported-rhel8:latest if the automation execution environment is not specified in some other way.

> **Important**
> For ansible-navigator to pull the container image for the default automation execution environment (or any other execution environment) from the container registry on registry.redhat.io, it needs to authenticate. For this to succeed, you must be logged in to registry.redhat.io and have a valid subscription to Ansible Automation Platform.
>
> Use the podman login registry.redhat.io command with your Customer Portal login credentials before running the ansible-navigator command. You only need to do this one time per session.
>
> If the container image has already been downloaded, then ansible-navigator can use that without needing to pull the container image again.
>
> In this course, the classroom might not have internet access in some delivery modes. However, you can also download the supported container images in this course from a container registry on aap.lab.example.com. You can use the podman login aap.lab.example.com command to authenticate in the classroom environment using admin as the username and redhat as the password. After authenticating, you can manually download the available automation execution environments. The hands-on activities provide detailed instructions.

Selecting Alternative Automation Execution Environments

When you run automation content navigator, you can use the --eei option to select a specific container image to use as the automation execution environment, such as the ee-supported-rhel8 container image. Container images are also published with tags so that you can select a specific version of the container image. For example, you could specify the ee-supported-rhel8:latest container image. The latest tag indicates that you want the latest version of that image.

You can also use the --pull-policy (--pp) option to control how ansible-navigator downloads container images. When the value of this option is set to missing, automation content navigator only downloads the container image if the image does not exist on the local system.

Installing Automation Content Navigator

You only need to install ansible-navigator on your control node. You do not need to install ansible-navigator on managed hosts.

You need a valid Red Hat Ansible Automation Platform subscription to install automation content navigator on your control node. If you are not yet using the product, trial entitlements are available through www.redhat.com and developer entitlements are available through the Red Hat Developer program.

If you have activated Simple Content Access for your organization, then you do not need to attach the entitlement to your system. The installation process is as follows:

- Register your system using Red Hat Subscription Manager:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[root@host ~]# subscription-manager register
```

- Enable the Red Hat Ansible Automation Platform 2 repository:

```
[root@host ~]# subscription-manager repos \
--enable ansible-automation-platform-2.5-for-rhel-9-x86_64-rpms
```

> **Note**
> You do not need to run these `subscription-manager` commands in the classroom lab environment.

- Install the `ansible-navigator` RPM package:

```
[root@host ~]# dnf install ansible-navigator
```

Configuring Authentication to Managed Hosts

Automation content navigator needs to be able to log in to managed hosts and gain superuser privileges on those hosts. The easiest way to implement this is by using SSH key-based authentication to an account that allows privilege escalation through `sudo` without a password.

Preparing SSH Key-based Authentication

The initial part of the configuration of your managed hosts is the same as the procedure you use for the `ansible-playbook` command:

- Set a `remote_user` directive to the name of the user account you plan to use on the managed hosts in the `[defaults]` section of your Ansible configuration file on the control node.
- Use the `ssh-keygen` command to generate an SSH key pair for the user account that you use to run `ansible-navigator`.
- Install the public key of that key pair in the `~/.ssh/authorized_keys` file for the `remote_user` account on each managed host. You can use the `ssh-copy-id` *`user@host`* command to do this, where *`user`* is the remote user and *`host`* is one of the managed hosts.
- On the managed hosts, configure `sudo` to allow the remote user you specified to run all commands as any user without a password.

Running Automation Content Navigator

Use the `ansible-navigator` command to run automation content navigator. If you run the command with no arguments, or as `ansible-navigator welcome`, it starts in interactive mode. This displays a help page describing the subcommands provided by the tool.

Many earlier Ansible commands can be replaced with an automation content navigator subcommand.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Earlier Ansible command	Automation content navigator subcommand
`ansible-config`	`ansible-navigator config`
`ansible-doc`	`ansible-navigator doc`
`ansible-inventory`	`ansible-navigator inventory`
`ansible-playbook`	`ansible-navigator run`

Automation content navigator also provides additional functionality. You can run most commands either from the command line or from an interactive automation content navigator session, but be aware that some commands do not support the `-m stdout` option.

The following table briefly describes the available subcommands.

Subcommand	Description
`collections`	Get information about installed collections.
`config`	Examine the current Ansible configuration.
`doc`	Examine the Ansible documentation for a plug-in.
`help`	Display detailed help for `ansible-navigator`.
`images`	Examine an execution environment.
`inventory`	Explore an inventory.
`log`	Review the current log file.
`open`	Open the current page in a text editor.
`replay`	Replay a playbook artifact.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Subcommand	Description
`run`	Run a playbook.

The help page displayed by `ansible-navigator welcome` is similar to the preceding table. When running a subcommand in an interactive session, type `:` to start the command, such as `:config`, and then press `Enter`.

The following sections introduce some `ansible-navigator` subcommands.

Running Playbooks

You can run playbooks using automation content navigator from the command line with the `ansible-navigator run` command or interactively with the `:run` subcommand. The `ansible-navigator run` command supports most of the same options as the `ansible-playbook` command.

When using the default `interactive` mode (rather than `stdout` mode), you can interactively examine playbook output, as previously discussed. For example, you might display details for a failed play or for a specific task in a play that failed.

Reviewing Previous Playbook Runs

Automation content navigator provides a replay feature that displays the output of a previous playbook run. If playbook artifacts are enabled (the default), then the `ansible-navigator run` command generates an *artifact file*, which use names such as `site-artifact-2025-03-25T20:09:57.939910+00:00.json` or similar. The first part of the file name refers to the playbook name, and so `site` is used for a run using the `site.yml` playbook. The second part of the file name is the date and time with time zone offset from UTC (`+00:00` indicates that the time was recorded in UTC).

You can review a previous playbook run by using either the `ansible-navigator replay` command from the command line or the `:replay` subcommand in interactive mode. Most replay files can be safely ignored or even deleted. If you plan to save a replay file, then consider renaming it so that you can identify it.

One advantage of the replay file is that if the playbook fails then you can share the replay file with another developer or a support team. They can proceed with troubleshooting without needing to have access to all the files in the project directory, such as playbooks, roles, inventory files, variable files, and so on.

> **Note**
> Automation content navigator can prompt for passwords, but only if playbook artifacts are disabled. Configuration settings for automation content navigator, including the ability to disable playbook artifacts, are discussed later in the course.

Reading Documentation

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Automation content navigator provides access to documentation for plug-ins, such as modules, lookups, and callbacks, much like the `ansible-doc` command.

For example, you can run `ansible-navigator doc ansible.posix.firewalld` from the command line to display documentation for the `ansible.posix.firewalld` module. You can do the same thing by running the `:doc ansible.posix.firewalld` subcommand from within an interactive automation content navigator session.

Getting Help

Automation content navigator provides multiple ways to get help. If you are running `ansible-navigator` in interactive mode, you can type `:help` and then press `Enter` to open a help page.

Alternatively, add the `--help` (`-h`) option to the `ansible-navigator` command. Using this option displays command syntax and describes command options.

Important

Using the `--help` option does not always display *all* options. For example, running the `ansible-navigator run --help` command displays the main options and then includes the following message:

```
...output omitted...

Note: 'ansible-navigator run' additionally supports the same parameters as the
'ansible-playbook' command. For more information about these, try 'ansible-
navigator run --help-playbook --mode stdout'
```

Provided you have logged in to the classroom's container registry, the following command displays additional options:

```
[student@workstation ~]$ ansible-navigator run --help-playbook --mode stdout \
--eei ee-supported-rhel8:latest
...output omitted...
```

References

Using Content Navigator

Ansible Navigator Documentation

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

1.4. Guided Exercise
Running Playbooks with Automation Content Navigator

Use automation content navigator to run a playbook and review the results of its execution.

Outcomes

- Use automation content navigator to run a playbook.

Prerequisites

As the `student` user on the `workstation` machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
[student@workstation ~]$ lab start develop-navigator
```

Instructions

1. Install the `ansible-navigator` package:

```
[student@workstation ~]$ sudo dnf install ansible-navigator
[sudo] password for student: student
...output omitted...
Is this ok [y/N]: y
...output omitted...
```

2. Review the exercise playbook.

   2.1. Change to the `/home/student/develop-navigator` directory:

```
[student@workstation ~]$ cd ~/develop-navigator/
[student@workstation develop-navigator]$
```

   2.2. Use the `tree` command to view the files in the directory:

```
[student@workstation develop-navigator]$ tree
.
├── ansible.cfg
├── intranet.yml
└── inventory

0 directories, 3 files
```

   2.3. Review the `intranet.yml` playbook:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
---
- name: Enable intranet services
  hosts: servera.lab.example.com
  become: true
  gather_facts: false
  tasks:
    - name: Install software
      ansible.builtin.dnf:
        name:
          - httpd
          - firewalld
        state: present

    - name: Deploy index.html
      ansible.builtin.copy:
        content: |
          Welcome to the example.com intranet!
        dest: /var/www/html/index.html
        owner: root
        group: root
        mode: "0644"

    - name: Start and enable firewalld
      ansible.builtin.service:
        name: firewalld
        enabled: true
        state: started

    - name: Allow firewall access
      ansible.posix.firewalld:
        service: http
        permanent: true
        state: enabled
        immediate: true

    - name: Start and enable httpd
      ansible.builtin.service:
        name: httpd
        enabled: true
        state: started

- name: Test intranet web server
  hosts: workstation.lab.example.com
  become: false
  gather_facts: false
  tasks:
```

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
    - name: Test access to intranet web server
      ansible.builtin.uri:
        url: http://servera.lab.example.com
        return_content: true
        status_code: 200
```

3. Run the `intranet.yml` playbook using automation content navigator.

   3.1. Automation content navigator needs to pull the container image for its automation execution environment to your workstation from an automation hub or other container registry.

   Use the `podman login` command to log in to the container registry at `aap.lab.example.com`. Use `admin` as the username and `redhat` as the password.

> **Note**
> If you did not have a private automation hub, but had internet access, you could use your Customer Portal account to log in to the automation hub hosted by Red Hat at `registry.redhat.io` instead.

```
[student@workstation develop-navigator]$ podman login aap.lab.example.com
Username: admin
Password: redhat
Login Succeeded!
```

   3.2. Use the `ansible-navigator` command to run the `intranet.yml` playbook. The following command specifies the automation execution environment image to use and displays output in the `stdout` mode:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation develop-navigator]$ ansible-navigator run intranet.yml
\
--eei aap.lab.example.com/ee-supported-rhel8 --mode stdout
------------------------------------------------------------------------------
----
Execution environment image and pull policy overview
------------------------------------------------------------------------------
----
Execution environment image name:     aap.lab.example.com/ee-supported-
rhel8:...
Execution environment image tag:      latest
Execution environment pull arguments: None
Execution environment pull policy:    tag
Execution environment pull needed:    True
------------------------------------------------------------------------------
----
Updating the execution environment
------------------------------------------------------------------------------
----
Running the command: podman pull aap.lab.example.com/ee-supported-
rhel8:latest
Trying to pull aap.lab.example.com/ee-supported-rhel8:latest...
...output omitted...

PLAY [Enable intranet services]
************************************************

TASK [Install software]
************************************************************
changed: [servera.lab.example.com]

TASK [Deploy index.html]
************************************************************
changed: [servera.lab.example.com]

TASK [Start and enable firewalld]
*********************************************
ok: [servera.lab.example.com]

TASK [Allow firewall access]
*****************************************************
changed: [servera.lab.example.com]

TASK [Start and enable httpd]
*************************************************
changed: [servera.lab.example.com]
```


For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
PLAY [Test intranet web server]
*********************************************

TASK [Test access to intranet web server]
************************************
ok: [workstation.lab.example.com]

PLAY RECAP
*********************************************************************
servera.lab.example.com    : ok=5    changed=4    unreachable=0    failed=0
...
workstation.lab.example.com : ok=1    changed=0    unreachable=0    failed=0
...
```

4. Using the `aap.lab.example.com/ee-supported-rhel8` automation execution environment image, run the `intranet.yml` playbook interactively from within automation content navigator.

   4.1. Use the `ansible-navigator` command to launch automation content navigator:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation develop-navigator]$ ansible-navigator \
--eei aap.lab.example.com/ee-supported-rhel8
```

```
 0|Welcome
 1|
_____________________________________________________________________________
_
 2|
 3|Some things you can try from here:
 4|- :collections                              Explore available collections
 5|- :config                                   Explore the current ansible
conf...
 6|- :doc <plugin>                            Review documentation for a
modul...
 7|- :help                                     Show the main help page
 8|- :images                                   Explore execution environment
im...
 9|- :inventory -i <inventory>                 Explore an inventory
10|- :log                                      Review the application log
11|- :lint <file or directory>                 Lint Ansible/YAML files
(experim...
12|- :open                                     Open current page in the editor
13|- :replay                                   Explore a previous run using a
p...
14|- :run <playbook> -i <inventory>           Run a playbook in interactive
mode
15|- :settings                                 Review the current ansible-
navig...
16|- :quit                                     Quit the application
17|
18|happy automating,
19|
20|-winston
```

4.2. Type `:run intranet.yml` and then press **Enter** to interactively run the `intranet.yml` playbook.

```
  Play name Ok Changed ... Failed Skipped Ignored ... Task count  Progress
0|Enable in  5       0 ...      0       0       0 ...          5  Complete
1|Test intr  1       0 ...      0       0       0 ...          1  Complete

^f/PgUp page up    ^b/PgDn page down  ↑↓ scroll    esc back  ... SUCCESSFUL
```

4.3. Type `0` to display details for the `Enable intranet services` play.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
  Result  Host       Number  Changed  Task                          ...
0|Ok      servera...      0  False    Install software              ...
1|Ok      servera...      1  False    Deploy index.html             ...
2|Ok      servera...      2  False    Start and enable firewalld    ...
3|Ok      servera...      3  False    Allow firewall access         ...
4|Ok      servera...      4  False    Start and enable httpd        ...

^f/PgUp page up    ^b/PgDn page down  ↑↓ scroll    esc back  ... SUCCESSFUL
```

4.4. Press Esc to return to the main playbook summary page, and then type 1 to display details for the `Test intranet web server` play.

```
  Result  Host          Number  Changed  Task
...
0|Ok      workstation...     0  False    Test access to intranet web server
...

^f/PgUp page up     ^b/PgDn page down     ↑↓ scroll    esc back  ...
SUCCESSFUL
```

Type `:q` and then press **Enter** to exit the `ansible-navigator` command.

Finish

On the `workstation` machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish develop-navigator
```


For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

1.5. Managing Ansible Project Materials Using Git

Objectives

- Create and manage Ansible Playbooks in a Git repository, following recommended practices.

Overview of Git

Git is a *distributed version control system (DVCS)* that enables you to manage changes to files in a project collaboratively. Using Git provides many benefits:

- You can review and restore earlier versions of files.
- You can compare two versions of the same file to identify changes.
- You can record a log of who made what changes, and when those changes were made.
- You can work with multiple users to modify files, resolve conflicting changes, and merge the changes.

Using Git, you can start by *cloning* an existing shared project from a *remote repository*. Cloning a project creates a complete copy of the original remote repository as a *local repository*. This local copy has the entire history of the files in Git, and not just the latest snapshot of the project files.

You make edits to files in a *working tree*, which is a checkout of a single snapshot of the project. Next, you place the modified files in a staging area, ready to commit the changes to the local repository. At this point, no changes have been made to the shared remote repository.

When you are ready to share the completed changes, you *commit* the changes and then *push* the changes to the remote repository.

Likewise, when you are ready to update the local repository with the latest changes from the remote repository, you *pull* the changes, which fetches them from the remote repository and merges them into the local repository.

To use Git effectively, you must be aware of the possible states of files in the working tree: *modified*, *staged*, *committed*, or *clean (unmodified)*.

State	Description
Modified	You edited the copy of the file in the working tree and it is different from the latest version in the repository.
Staged	You added the modified file to a list of changed files to commit as a set, but the staged files have not been committed yet.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

State	Description
Committed	You committed the modified file to the local repository.
Clean	You have not modified the file since the last commit.

> **Note**
> The presentation in this section assumes that you are using Git from the command line of a Bash shell. A number of programming editors and IDEs have integrated Git support, and although the configuration and UI details might differ, they provide a different front end to the same workflow.

Initial Git Configuration

Because Git users often modify projects with multiple contributors, Git records the user's name and email address on each of their commits. You can define these values at a project level, but you can also set global defaults for a user. The `git config` command controls these settings.

Using `git config` with the `--global` option manages the default settings for all Git projects to which you contribute by saving the settings in your `~/.gitconfig` file.

```
[user@host ~]$ git config --global user.name 'Peter Shadowman'
[user@host ~]$ git config --global user.email peter@host.example.com
```

If you use the Bash shell, then another useful option is to configure your prompt to automatically update to indicate the status of your working tree. The easiest way to do this is to use the `git-prompt.sh` script that ships with the `git` package.

To modify your shell prompt in this way, add the following lines to your `~/.bashrc` file:

```
source /usr/share/git-core/contrib/completion/git-prompt.sh
export GIT_PS1_SHOWDIRTYSTATE=true
export GIT_PS1_SHOWUNTRACKEDFILES=true
export PS1='[\u@\h \W$(declare -F __git_ps1 &>/dev/null && __git_ps1 " (%s)")]\$ '
```

If your current directory is in a Git working tree, then the name of the current Git branch for the working tree is displayed in parentheses. If you have untracked, modified, or staged files in your working tree, then the prompt could indicate one of the following meanings:

- `(branch *)` means that you have modified a tracked file.
- `(branch +)` means that you have modified a tracked file and used `git add` to stage it.
- `(branch %)` means that you have untracked files in your tree.
- Combinations of markers are possible, such as `(branch *+)`.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Starting the Git Workflow

When you first start working on an existing project, you *clone* a remote repository to create a local repository in a subdirectory of the current directory.

This process creates a working tree of files ready for revisions. Because the working tree is unmodified, it is initially in a clean state. A clean state means the working tree does not contain any modified, staged, or new files.

To clone a repository from the command line, use the `git clone` command, followed by the URL that specifies the location of the remote repository that you want to clone.

For example, the following command clones the `project.git` repository at `git.lab.example.com` by connecting using the SSH protocol and authenticating as the `git` user:

```
[user@host ~]$ git clone git@git.lab.example.com:project.git
```

As you work, you create files and modify existing files in the working tree. This changes the working tree to a *dirty* state. The `git status` command displays detailed information about which files in the working tree are modified but unstaged, untracked (new), or staged for the next commit.

The `git add` command stages files, preparing them to be committed. Only files that are staged to the staging area are saved to the repository on the next commit.

To add all changed files in the current directory and subdirectories, use the `git add .` command. To add all changed files in every directory in the entire working tree, use the `git add -A` command.

> **Note**
> The `git add .` and `git add -A` commands can simultaneously add multiple files to the staging area.
>
> If you are working on multiple changes at the same time, then you might want to add each file separately so that you can organize the files into different commits for better tracking of changes.

The `git rm` command removes a file from the working directory and also stages its removal from the repository on the next commit.

The `git reset` command removes a file from the staging area that has been added for the next commit. This command has no effect on the file's contents in the working tree.

The `git commit` command commits the staged files to the local Git repository. You must provide a log message that explains why you are saving the current set of staged files. Failure to provide a message aborts the commit. Log messages do not have to be long, but they should be meaningful so that they are useful.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Note

Meaningful and concise commit messages are the key to maintaining a clear history for an Ansible project. Different approaches exist for writing good commit messages, but most of these approaches agree on the following three points:

- The first line should be a short summary of the reason for the commit. This summary is usually fewer than 50 characters.
- A blank line follows the first line, and then the rest of the message must explain all the details, description of what was done, and the reasons for the commit.
- If available, then add references to an issue or feature tracker. These references expand the commit message with additional text, related people, or history.

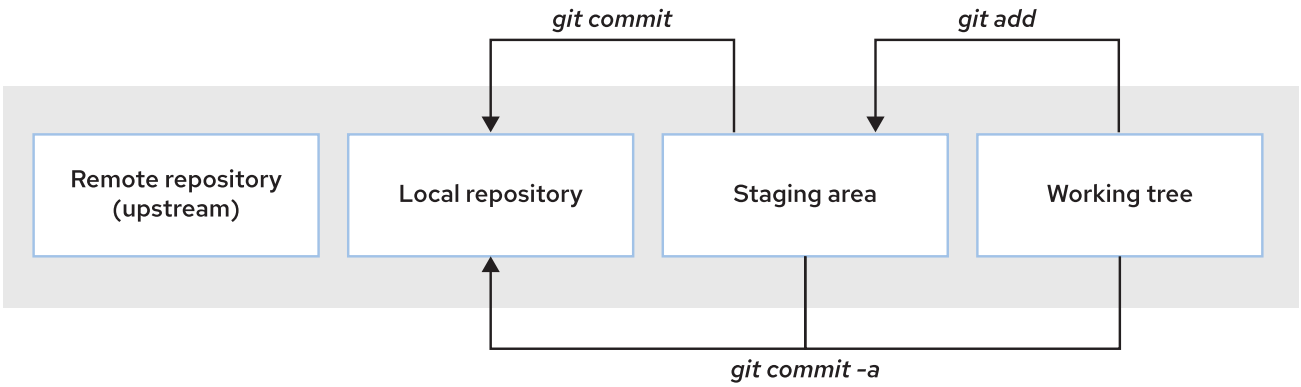


Figure 11. Git subcommands used to add and update local repository content

Important

The `git commit` command by itself does not automatically commit changed files in the working tree.

The `git commit -a` command stages and commits all the *modified* files in one step. However, that command does not include any *untracked* (newly created) files in the directory. When you add a new file, you must explicitly run the `git add` command to stage the new file.

Synchronizing Changes

From the command line, the `git push` command uploads commits you made in your local repository to the remote repository. One common way to coordinate work with Git is for all developers to push their work to the same, shared, remote repository.

Before Git pushes can work, you must define the default push method to use. The following command sets the default push method to the `simple` method. This is the safest option for beginners.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[user@host ~]$ git config --global push.default simple
```

Likewise, the `git pull` command downloads changes to your local repository from the remote repository. Use the `git pull` command to perform the following tasks:

- Fetch commits from the remote repository.
- Add those commits to the local directory.
- Merge changes into the files in the working tree.

Run the `git pull` command often to stay current with the latest merged changes that others are making to the project in the remote repository. Keep in mind that if multiple developers work in different branches, you receive their updates after their changes are reviewed, approved, and merged.

> **Note**
> An alternative approach is to use the `git fetch` command to download the changes from the remote repository to your local repository. You then use the `git merge` command to merge the changes in the tracking branch to your current branch.
>
> Later in this section, you learn more about Git branches.

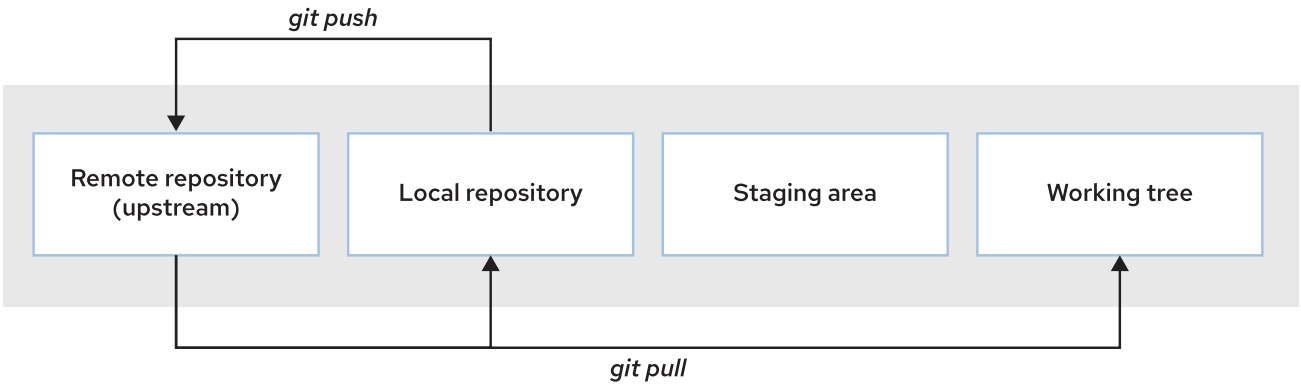


Figure 12. Git subcommands that interact with a remote repository

Inspecting the Git Log

Part of the point of a version control system is to track a history of commits. A commit hash identifies each commit.

The `git log` command displays the commit log messages with the associated ID hashes for each commit.

The `git show` *`commit-hash`* command shows what was in the change set for a particular commit hash. You do not need to use the entire hash with this command; only enough of it to uniquely identify a particular commit in the repository. These hashes can also be used to revert to earlier commits or otherwise explore the version control system's history.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Table 3. Git Quick Reference

Command	Description
`git clone` *URL*	Clone an existing Git project from the remote repository at *URL* into the current directory.
`git status`	Display the status of files in the working tree.
`git add` *file*	Stage a new or changed file for the next commit.
`git rm` *file*	Stage removal of a file for the next commit.
`git reset`	Unstage files that have been staged for the next commit.
`git commit`	Commit the staged files to the local repository with a descriptive message.
`git push`	Push changes in the local repository to the remote repository.
`git pull`	Fetch updates from the remote repository to the local repository and merge them into the working tree.
`git revert` *commit-hash*	Create a commit, undoing the changes in the referenced commit. You can use the commit hash that identifies the commit, although there are other ways to reference a commit.
`git init`	Create a project.
`git log`	Display the commit log messages.
`git show` *commit-hash*	Shows what was in the change set for a particular commit hash.

Working with Branches and References

Branches enable you to change your project, working on new development, without altering your main branch. Then, when you are satisfied with your changes, you can *merge* the changes in your branch to the main branch of development, integrating the two branches.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

A branch is a pointer to a particular commit in your commit tree. Each commit contains the changes it made and information about how to reference it and what its relationship is to other commits in the commit tree. The information in a commit includes:

- A unique ID in the form of a 40-character hexadecimal string. This ID is the Secure Hash Algorithm 1 (SHA-1) hash of the contents of the commit.
- A list of repository files that it changed, and the exact changes to each of them.
- The ID of the parent commit that defines the status of the repository before applying the commit in question.
- Information about the author and the creator (or *committer*) for the commit in the commit data.
- A list of *references*. A reference is like a named pointer to the commit. The most common references are *tags* and *branches*.

The `git commit` command generates a new commit with all changes added to the stage with the `git add` command. You can display a Git repository as a commit graph, and branches as references to commits.

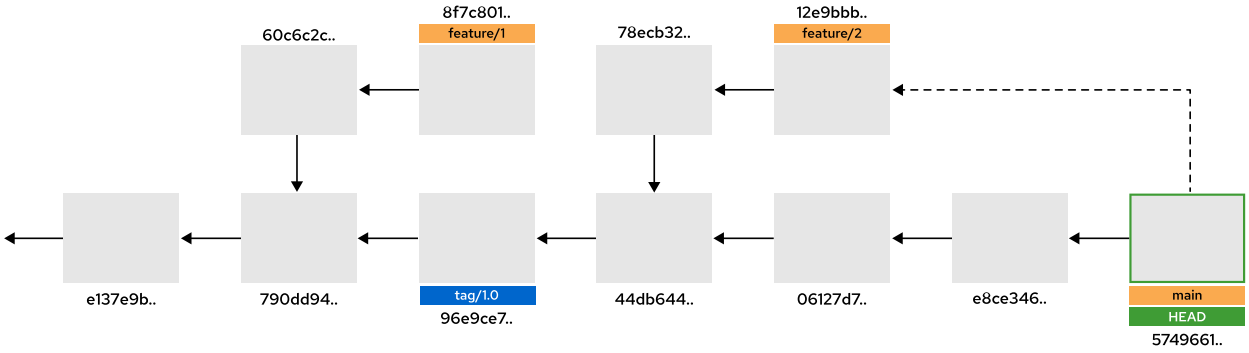


Figure 13. Sample Git repository

The preceding figure shows a sample Git repository containing 11 commits. The most recent commits are to the right; the arrows point to earlier commits. It has three branches (`main`, `feature/1`, and `feature/2`) and a single tag (`tag/1.0`).

The HEAD reference is the current commit in the local working tree. If you make a change in your working tree, stage it with the `git add` command, and commit it with the `git commit` command. A new commit is then created with the most recent commit as its parent, and HEAD moves to point to the new commit.

Creating Branches

Different branches in Git enable different work streams to evolve in parallel on the same Git repository. Commits for each work stream append only to that branch.

Use the `git branch` command to create a branch from the current HEAD commit. This command creates a reference for the new branch, but it does not set the current HEAD to this branch. Use the `git checkout` command to move the HEAD to the appropriate branch.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

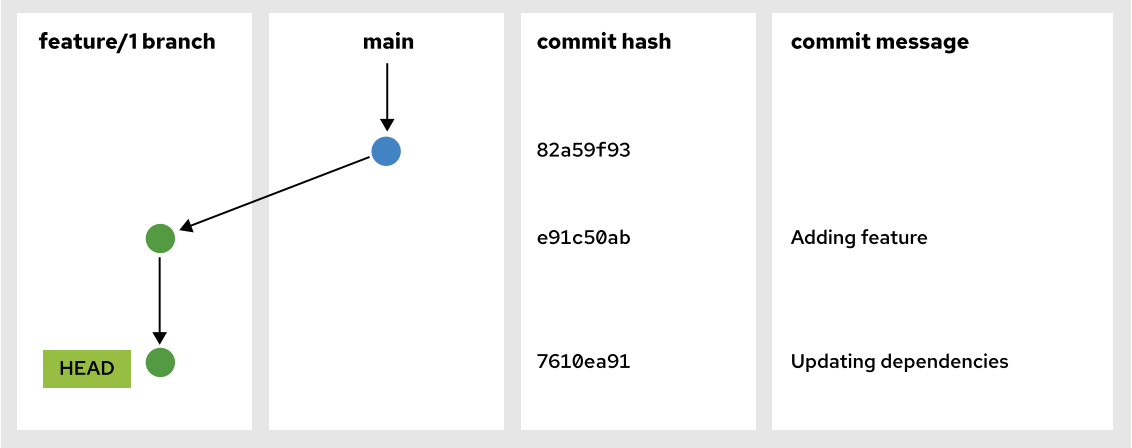


Figure 14. Commits to a branch of a Git repository.

In the preceding figure, the most recent commit for the `main` branch (and HEAD at that time) was commit 82a59f93, which occurred at some point in the past. We are assuming you ran the `git branch feature/1` command, creating a branch named `feature/1`. Then, you ran `git checkout feature/1` to indicate that future commits should be made to that branch. Finally, you staged and committed two commits to the `feature/1` branch, commits e91c50ab and 7610ea91. After running these commands, the HEAD was at commit 7610ea91, and the new commits were added to the `feature/1` branch.

You can create a branch and switch to it in a single step by running the `git checkout` command with the `-b` option:

```
[user@host ~]$ git checkout -b feature/1
Switched to a new branch 'feature/1'
```

Merging Branches

When work is complete on a branch, you can *merge* the branch with the original branch. This enables you to work in parallel on new features and bug fixes, leaving the main branch free of incomplete or untested work.

When you use a branch for feature development, you can commit and share your code often without impacting the stability of code in the `main` branch.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

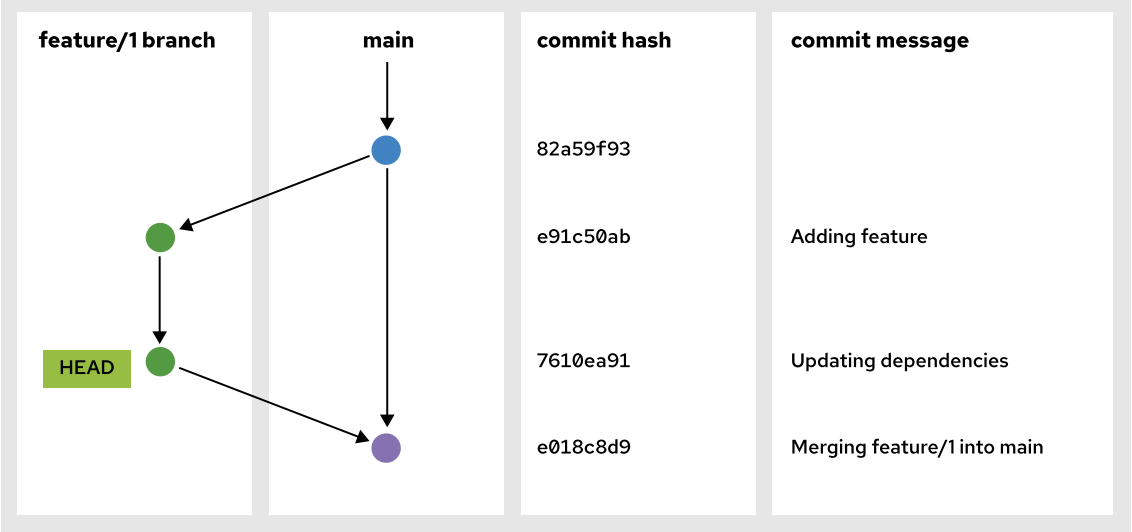


Figure 15. Merging a branch into the main branch.

In the preceding figure, it was determined that all changes from `feature/1` should be merged into the `main` branch. That is, after ensuring that the code in the `feature/1` branch is complete, tested, and reviewed, you are ready to merge the branch into another branch, such as the `main` branch. *Merging* is the process of combining the commit histories from two separate branches into a single branch.

Sometimes, changes on more than one branch cannot merge automatically because each branch makes changes to the same parts of the same files. This situation is called a *merge conflict* and you need to manually edit the affected files to resolve it.

> **Important**
> Some Git servers, such as GitHub, GitLab, and Bitbucket, allow you to protect branches from unwanted changes. As a best practice you might protect certain branches on the remote repository, such as the `main` branch, to avoid merging unwanted changes to them from other branches created by developers.
>
> These servers also often implement a feature called *pull requests* or *merge requests*. This provides a mechanism that you or other code reviewers can use to review and approve requests before the branch can be merged with the protected branch on the remote server.

Merge strategies, conflict resolution, and protection of branches, are out of scope for this course.

Pushing Branches to Remote Repositories

Initially, any branches that you create only exist in your local repository. If you want to share them with users of the original remote repository that you cloned, then you must push them to the repository.

The most common way to do this is to use the `git push` command with the `--set-upstream` (or `-u`) option to create a branch on the remote repository. Your current local branch then tracks that branch.

For example, to push your new `feature/1` branch to your remote repository, run the following commands:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[user@host ~]$ git checkout feature/1
Switched to branch 'feature/1'
[user@host ~]$ git push --set-upstream origin feature/1
```

The remote `origin` repository normally refers to the Git repository that you originally cloned. This command indicates that you want to push the current branch to the `origin` repository, creating `feature/1`. When the Git reviewers approve the push request, Git creates the `feature/1` branch in the remote `origin` repository.

Running the `git pull` command on this branch pulls and merges any commits from the remote repository into your local copy of the branch.

Structuring Ansible Projects in Git

Each Ansible project should have its own Git repository. The structure of the files in that repository should follow the directory layout recommended at http://docs.ansible.com/ansible/latest/tips_tricks/sample_setup.html.

For example, the directory structure might appear as follows:

```
site.yml                # main playbook
webservers.yml          # playbook for webserver tier
dbservers.yml           # playbook for dbserver tier

collections/            # holds Ansible Content Collections in directories
  requirements.yml      # specifies collections needed by this project

roles/
  common/               # this hierarchy represents a "role"
    tasks/              #
      main.yml          # tasks file can include smaller files if warranted
    handlers/           #
      main.yml          # handlers file
    templates/          # files for use with the template resource
      ntp.conf.j2       # templates end in .j2
    files/              #
      bar.txt           # files for use with the copy resource
      myscript.sh       # script files for use with the script resource
    vars/               #
      main.yml          # variables associated with this role
    defaults/           #
      main.yml          # default lower priority variables for this role
    meta/               #
      main.yml          # role dependencies
  ...additional roles...
```

Ansible Roles and Ansible Content Collections

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Ansible Roles and Ansible Content Collections are ways that you can reuse code from other Ansible projects.

You can include roles and Ansible Content Collections in your Ansible project's Git repository. In this case, each collection is generally stored in subdirectories of the `collections/` directory, and each role is stored in subdirectories of the `roles/` directory.

There might also be a `requirements.yml` file in either or both of those directories that specifies collections or roles that are used by this project.

A role or collection can have its own Git repository. That way, instead of keeping a copy of the role or collection in your Ansible project's repository, automation controller can retrieve it directly from the relevant role or collection repository when it runs playbooks that belong to the project.

To set this up, create a `roles/requirements.yml` file to define the required roles and a `collections/requirements.yml` file to define the required collections. Then use the `ansible-galaxy` command to install the required roles and collections.

> **Note**
> Automation controller automatically updates the project with roles and collections based on the `roles/requirements.yml` and `collections/requirements.yml` files included in the Ansible project.

Configuring Git to Ignore Files

Not every file and directory in your Ansible project directory should be tracked by Git. You can configure Git to ignore files and directories that should not be tracked or committed.

To configure Git to ignore specific files and paths in your project, add a `.gitignore` file at the top of your Ansible project directory.

Each line in a `.gitignore` file represents a pattern to match, and that determines whether Git ignores a file in the repository. Generally, lines that match file names indicate files that Git ignores. Lines that start with an exclamation mark (!) indicate files that Git should not ignore, even if they were matched by a previous pattern. You can use blank lines to improve readability, and any line that starts with a number sign (#) is a comment.

Details on the pattern matching format for this file are documented in the `gitignore`(5) man page under "PATTERN FORMAT". Pay particular attention to the distinction between single asterisks (*) and double asterisks (**) in these pattern matching lines. (Double asterisks can match multiple directory levels.)

For example, if you use `requirements.yml` files with the `ansible-galaxy` command to populate the project's `roles` and `collections` directories, then you should configure Git to ignore the populated content.

A sample `.gitignore` file might contain the following content:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
roles/**
!roles/requirements.yml
collections/ansible_collections
```

References

`gittutorial`(7), `git`(1), and `gitignore`(5) man pages

Git

Pro Git by Scott Chacon and Ben Straub (free book)

Set Up Git

Learning Git Branching

Sample Directory Layout

How to Write a Git Commit Message

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

1.6. Guided Exercise
Managing Ansible Project Materials Using Git

Clone an existing Git repository that contains an Ansible Playbook, edit files in that repository, commit the changes to your local repository, and push those changes to the original repository.

Outcomes

- Use basic Git commands to manage both new and modified files stored in an existing Git repository.

Prerequisites

As the `student` user on the `workstation` machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
[student@workstation ~]$ lab start develop-git
```

Instructions

1. Clone the `https://git.lab.example.com/student/develop-git.git` repository into the `/home/student/git-repos` directory and then create a branch for this exercise.

   1.1. From a terminal, create the `/home/student/git-repos` directory if it does not already exist, and then change into it:

   ```
   [student@workstation ~]$ mkdir -p ~/git-repos/
   [student@workstation ~]$ cd ~/git-repos/
   [student@workstation git-repos]$
   ```

   1.2. Clone the `https://git.lab.example.com/student/develop-git.git` repository and then change to the cloned repository:

   ```
   [student@workstation git-repos]$ git clone \
   https://git.lab.example.com/student/develop-git.git
   Cloning into 'develop-git'...
   ...output omitted...
   [student@workstation git-repos]$ cd develop-git
   [student@workstation develop-git]$
   ```

   1.3. Create the `exercise` branch and check it out:

```
[student@workstation develop-git]$ git checkout -b exercise
Switched to a new branch 'exercise'
```

2. Configure the global Git settings for the `student` user to specify a username, an email address, and a default push method. Additionally, specify the username to use when connecting to `https://git.lab.example.com` and allow Git to remember your password for two hours.

   2.1. Use the `git config` command to customize Git globally:

```
[student@workstation develop-git]$ git config --global user.name 'Student
User'
[student@workstation develop-git]$ git config --global \
user.email student@lab.example.com
[student@workstation develop-git]$ git config --global push.default simple
[student@workstation develop-git]$ git config --global \
credential.https://git.lab.example.com.username student
[student@workstation develop-git]$ git config --global \
credential.helper cache --timeout=7200
```

   2.2. Verify the global configuration settings:

```
[student@workstation develop-git]$ git config --global -l
user.name=Student User
user.email=student@lab.example.com
push.default=simple
credential.https://git.lab.example.com.username=student
credential.helper=cache
```

3. Review the files in the project directory:

```
[student@workstation develop-git]$ tree
.
├── apache-setup.yml
└── templates
    ├── httpd.conf.j2
    └── index.html.j2

1 directory, 3 files
```

   - The `apache-setup.yml` file is the playbook.
   - The `httpd.conf.j2` template customizes the Apache server configuration.
   - The `index.html.j2` template generates a basic index page for the server.

4. Modify the `apache-setup.yml` playbook to target the `web_servers` host group.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

4.1. Create the `inventory` file in the project directory with the following content:

```
[web_servers]
serverd
```

4.2. Create an Ansible configuration file in the project directory that uses the `inventory` file and connects to remote hosts as the `devops` user.

The `ansible.cfg` file should consist of the following content:

```
[defaults]
inventory = inventory
remote_user = devops
```

4.3. Modify the `apache-setup.yml` playbook to target the `web_servers` host group. The top of the modified playbook contains the following content:

```
---
- name: Install web servers with WSGI interface
  hosts: web_servers
  become: true
  vars:
    httpd_packages:
      - httpd
      - python3-mod_wsgi
    apache_test_message: This is a test message
    apache_max_keep_alive_requests: 115
...output omitted...
```

5. Run the `apache-setup.yml` playbook using automation content navigator and then verify the content displayed by the web server.

5.1. Use the `podman login` command to log in to the container registry at `aap.lab.example.com`.

When prompted, enter `admin` as the username and `redhat` as the password.

This step simulates a customer logging in to `registry.redhat.io` and ensures that the specified automation execution environment is pulled down to the local system if it does not already exist:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation develop-git]$ podman login aap.lab.example.com
Username: admin
Password: redhat
Login Succeeded!
```

5.2. Using the `aap.lab.example.com/ee-supported-rhel8:latest` automation execution environment, run the `apache-setup.yml` playbook:

```
[student@workstation develop-git]$ ansible-navigator run apache-setup.yml \
--eei aap.lab.example.com/ee-supported-rhel8:latest

  Play name                    Ok  Changed ... Failed ... Task count
Progress
0|Install web servers with ...  7        6 ...      0 ...          7
Complete

^f/PgUp page up    ^b/PgDn page down    ↑↓ scroll    esc back  ...
Successful
```

After the playbook completes, press Esc to exit the `ansible-navigator` command.

5.3. Use the `curl` command to review the content displayed by the web server:

```
[student@workstation develop-git]$ curl serverd
This is a test message RedHat 9.4 <br>
Current Host: serverd <br>
Server list: <br>
serverd <br>
```

6. Review the status of the local Git repository. Configure the local Git repository so that it ignores specific files and file patterns. Commit your changes to the local repository and then push your changes to the remote repository.

6.1. Review the current status of the local Git repository. In addition to your changes, notice that the `ansible-navigator` command created the `ansible-navigator.log` file and one artifact file for each time that you used the `ansible-navigator` command to run a playbook.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation develop-git]$ git status
On branch exercise
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   apache-setup.yml

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        ansible-navigator.log
        ansible.cfg
        apache-setup-artifact-2025-06-30T13:21:57.541805+00:00.json
        inventory

no changes added to commit (use "git add" and/or "git commit -a")
```

6.2. Create a `.gitignore` file to ignore files and directories created by the `ansible-navigator` command. The `.gitignore` file contains the following content:

```
ansible-navigator.log
*-artifact-*
```

6.3. Review the status of the local Git repository. Files and directories that should not be committed are now ignored.

```
[student@workstation develop-git]$ git status
On branch exercise
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   apache-setup.yml

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore
        ansible.cfg
        inventory

no changes added to commit (use "git add" and/or "git commit -a")
```

6.4. Use the `git add` command to stage all changes for the next commit:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation develop-git]$ git add .
```

6.5. Verify that all files are staged:

```
[student@workstation develop-git]$ git status
On branch exercise
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   .gitignore
        new file:   ansible.cfg
        modified:   apache-setup.yml
        new file:   inventory
```

6.6. Commit the changes with the message: `Target the web_servers group`

```
[student@workstation develop-git]$ git commit -m "Target the web_servers
group"
[exercise b38604e] Target the web_servers group
 4 files changed, 8 insertions(+), 1 deletion(-)
 create mode 100644 .gitignore
 create mode 100644 ansible.cfg
 create mode 100644 inventory
```

6.7. Push the locally committed changes to the remote server. If prompted, use `Stud3nt123` as the password:

```
[student@workstation develop-git]$ git push -u origin exercise
Password for 'https://student@git.lab.example.com': Stud3nt123
...output omitted...
To https://git.lab.example.com/student/develop-git.git
 * [new branch]      exercise -> exercise
Branch 'exercise' set up to track remote branch 'exercise' from 'origin'.
```

> **Important**
>
> In a more realistic scenario, you would also create a merge request or a pull request so that changes could be incorporated into the `main` branch.

Finish

On the `workstation` machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish develop-git
```


For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

1.7. Implementing Recommended Ansible Practices

Objectives

- Demonstrate and describe recommended practices for developing and maintaining effective Ansible automation solutions.

The Effectiveness of Ansible

As you work with more advanced features and larger, more complex projects, it becomes more difficult to manage and maintain Ansible Playbooks, or to use them effectively.

This course introduces a number of advanced features of Red Hat Ansible Automation Platform. Using Ansible effectively is not just about features or tools, but about practices and organization.

To paraphrase Ansible developer Jeff Geerling, using Ansible effectively relies on three key practices:

- Keeping things simple
- Staying organized
- Testing often

Keeping Things Simple

One of the strengths of Ansible is its simplicity. Simple playbooks are easier to use, modify, and understand.

Keeping Your Playbooks Readable

Keep your playbooks well commented and easy to read. Use vertical white space and comments liberally. Always give plays and tasks meaningful names that make clear what the play or task is doing. These practices help document the playbook and make it easier to troubleshoot a failed playbook run.

YAML is not a programming language. It is good at expressing a list of tasks or items, or a set of key-value pairs. If you are struggling to express a complex control structure or conditional in your Ansible Playbook, then consider approaching the problem differently. Templates and Jinja2 filters provide features to process data in a variable, which might be a better approach to your problem.

Use native YAML syntax, not the "folded" syntax. For example, the following example is not a recommended format:

```
- name: Postfix is installed and updated
  ansible.builtin.dnf: name=postfix state=latest
  notify: restart_postfix

- name: Postfix is running
  ansible.builtin.service: name=postfix state=started
```

The following syntax is easier for most people to read:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
  - name: Postfix is installed and updated
    ansible.builtin.dnf:
      name: postfix
      state: latest
    notify: update_postfix

  - name: Postfix is running
    ansible.builtin.service:
      name: postfix
      state: started
```

Use Existing Modules

When writing a new playbook, start with a basic playbook and, if possible, a static inventory. Use `ansible.builtin.debug` tasks as stubs as you build your design. When your playbook functions as expected, break up your playbook into smaller, logical components by using imports and includes.

Use special-purpose modules included with Ansible when you can, instead of `ansible.builtin.command`, `ansible.builtin.shell`, `ansible.builtin.raw`, or other similar modules. It is easier to make your playbook idempotent and easier to maintain if you use the modules designed for a specific task.

Many modules have a default state or other variables that control what they do. For example, the `ansible.builtin.dnf` module currently assumes that the package you name should usually be present (installed). However, you should explicitly specify what state you want. Doing so makes it easier to read the playbook, and protects you against changes in the module's default behavior in later versions of Ansible.

Adhering to a Standard Style

Consider having a standard style or set of conventions that your team follows when writing Ansible projects. For example, how many spaces do you indent? How do you want vertical white space used? How should tasks, plays, roles, and variables be named? What should get commented on and how? Having a consistent standard can help improve maintainability and readability.

Staying Organized

Well-organized standards can help with maintainability, troubleshooting, and auditing. Take advantage of Ansible organization features, such as roles, so that you can reuse them.

Following Conventions for Naming Variables

Variable naming can be significant because Ansible has a reasonably flat namespace. Use descriptive variables, such as `apache_tls_port` rather than a less explanatory variable such as p. In roles, it is a good practice to prefix role variables with the role name.

If the name of your role is `myapp` then prefix your variables with `myapp_` so that you can identify them from variables in other roles and the playbook.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Variable names should clarify contents. A name such as `apache_max_keepalive` clearly explains the meaning of the associated values. Prefix roles and group variables with the name of the role or group to which the variable belongs. `apache_port_number` is more error-resistant than `port_number`.

Standardizing the Project Structure

Use a consistent pattern when structuring the files of your Ansible project on a file system. The following is a good example:

```
.
├── dbservers.yml
├── inventories/
│   ├── prod/
│   │   ├── group_vars/
│   │   ├── host_vars/
│   │   └── inventory/
│   └── stage/
│       ├── group_vars/
│       ├── host_vars/
│       └── inventory/
├── roles/
│   └── std_server/
├── site.yml
├── storage.yml
└── webservers.yml
```

The `site.yml` file is the main playbook, which includes or imports playbooks that perform specific tasks: `dbservers.yml`, `storage.yml`, and `webservers.yml`. Each role is in a subdirectory in the `roles` directory, such as `std_server`. Two static inventories in the `inventories/prod` and `inventories/stage` directories provide separate inventory variable files so that you can select different sets of servers by changing the inventory that you use.

One of the playbook structure benefits is that you can divide up your extensive playbook into smaller files to make it more readable. Those smaller playbooks can contain plays for a specific purpose that you can run independently.

Using Dynamic Inventories

Use dynamic inventories whenever possible. Dynamic inventories enable central management of your hosts and groups from one primary source of truth. They also ensure that you have an updated list of hosts and groups before you run your playbooks. Dynamic inventories are especially powerful when used with cloud providers, containers, and virtual machine management systems. Those systems might already have inventory information in a form that Ansible can consume.

If you cannot use dynamic inventories, then other tools can help you construct groups or extra information. For example, you can use the `group_by` module to generate group membership based on a fact. That group membership is valid for the rest of the playbook.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
- name: Generate dynamic groups
  hosts: all
  tasks:
    - name: Generate dynamic groups based on architecture
      ansible.builtin.group_by:
        key: arch_{{ ansible_facts['architecture'] }}

- name: Configure x86_64 systems
  hosts: arch_x86_64
  tasks:
    - name: First task for x86_64 configuration
...output omitted...
```

Taking Advantage of Groups

Hosts can be members of many groups. Consider dividing your hosts into different categories based on different characteristics:

Geographical

Differentiate hosts from different regions, countries, continents, or data centers.

Environmental

Differentiate hosts dedicated to different stages of the software lifecycle, including development, staging, testing, and production.

Sites or services

Group hosts that offer or link to a subset of functions, such as a specific website, an application, or a subset of features.

Important

Remember that hosts inherit variables from all groups of which they are members. If two groups have different settings for the same variable, and a host is a member of both, then the value that is used is the last one loaded.

If there are differences in settings between two different groups that might be used at the same time, then take special care to determine how those variables should be set.

Using Roles and Ansible Content Collections for Reusable Content

Roles and collections help you keep your playbooks simple and enable you to save work by reusing standard code across projects. If you write your own roles and collections, keep them focused on a particular purpose or function, just like you do with playbooks. Make roles generic and configurable through variables so that you do not need to edit them when you use them with a different set of playbooks.

Use the `ansible-galaxy` command to initialize the directory hierarchy for your role or collection and provide initial template files. This makes it easier to share your content through private automation hub or on the community

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Ansible Galaxy website.

Take advantage of Red Hat Ansible Certified Content Collections. These are supported with your Ansible Automation Platform subscription, and provide many useful functions and features.

You can also examine the roles provided by the community through Ansible Galaxy. Be aware that these roles have varying levels of quality, so choose the ones that you use carefully.

Keep your roles in the `roles` subdirectory of your project, and your collections in the `collections` subdirectory of your project. (Your collections might contain roles, of course, and those should stay in their collection's directory.) Use the `ansible-galaxy` command to get roles from automation hub or a separate Git repository automatically.

Running Playbooks Centrally

To control access to your systems and audit Ansible activity, consider using a dedicated control node from which all Ansible Playbooks are run. Ideally, you should use automation controller.

System administrators should still have their own accounts on the system, as well as credentials to connect to managed hosts and escalate privileges, if needed. When a system administrator leaves, their SSH key can be removed from the managed hosts' `authorized_keys` files and their `sudo` command privileges revoked, without impacting other administrators.

Consider using automation controller as this central host because it is included with your Red Hat Ansible Automation Platform subscription. Automation controller provides features that make it easier to control access to credentials, control playbook execution, simplify automation for users who are not comfortable with the Linux command line, as well as audit and track playbook runs. Later in this course, you learn about using automation controller. However, even if you do not use automation controller, using a central control node can be beneficial.

Building Automation Execution Environments

Create a custom automation execution environment if you often need to use specific Ansible Content Collections, especially if those collections have Python dependencies or system executables that are not included in the supported automation execution environments.

However, if you can use an existing automation execution environment to run your playbooks, you should consider doing that. Reusing automation execution environments can reduce maintenance effort and the number of execution environments that you must manage.

Performing Regular Testing

Test your playbooks and your tasks often during the development process, when the tasks run, and after the playbooks are in use.

Testing the Results of Tasks

If you need to confirm that a task succeeded, then verify the result of the task rather than trusting the return code of the module. You can verify tasks in a number of ways, depending on the module involved.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
  - name: Start web server
    ansible.builtin.service:
      name: httpd
      state: started

  - name: Check web site from web server
    ansible.builtin.uri:
      url: http://{{ ansible_fqdn }}
      return_content: true
    register: example_webpage
    failed_when: example_webpage.status != 200
```

Using the Block and Rescue Directives to Recover or Roll Back

In playbooks, you can use *blocks* to group tasks that should be executed as a single unit. It is a good practice to add a name to each block to document the purpose of the block.

Blocks also enable error handling using the `block` and `rescue` directives for the grouped tasks. If any task in a `block` section fails, then the playbook runs tasks in the `rescue` section.

The following example shows how to recover a web server by using blocks:

```
...output omitted...

  tasks:
    - name: Verify web server status and recover in case of failure
      block: (1)
        - name: Check web site from web server
          ansible.builtin.uri:
            url: http://{{ ansible_fqdn }}
            return_content: true
          register: example_webpage
          failed_when: example_webpage['status'] != 200

      rescue: (2)
        - name: Restart web server
          ansible.builtin.service:
            name: httpd
            state: restarted
```

(1) The `block` section defines the main tasks to run.
(2) The `rescue` section defines the tasks to run if any task in the `block` section fails.

Developing Playbooks with the Latest Ansible Version

Even if you are not using the latest version of Ansible Core in production, you should routinely test your playbooks against the latest version of Ansible Core. This test helps you to avoid issues as Ansible modules and features evolve.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Automation execution environments can help make this easier by providing the current version of Ansible Core that you can use to test your playbooks. If your playbooks print warnings or deprecation messages when they run, then you should pay attention to them and make adjustments. If a feature in Ansible Core is being deprecated or is changing, then the project provides deprecation notices for four minor releases before the feature gets removed or altered.

To prepare for changes and future updates to the upstream Ansible distribution, read the Ansible Porting Guides at https://docs.ansible.com/ansible/devel/porting_guides/porting_guides.html.

Making Use of Ansible Lint

To help you follow good practices, Red Hat Ansible Automation Platform 2.5 provides a tool called Ansible Lint, which uses a set of predefined rules to look for possible issues with your playbook.

Use the `ansible-navigator lint` command to review a file for common errors and issues:

```
[user@host ~]$ ansible-navigator lint playbook.yml
```

Not all the issues that Ansible Lint reports break your playbook, but a reported issue might indicate the presence of a more serious error. The output of the `ansible-navigator lint` command shows the issue found, the line where the issue was found, and the rule that allowed Ansible Lint to find the issue.

For example, assume that you have the following `web_status.yml` playbook:

```
---
- name: Web server status
  hosts: web_servers
  become: yes
  tasks:
    - name: Check web site from web server
      ansible.builtin.uri:
        url: http://{{ ansible_fqdn }}
        return_content: true
      register: example_webpage
      failed_when: example_webpage['status'] != 200

    - name: Report status
      debug:
         msg: >-
          "The web site status was:
          {{ example_webpage['status'] }}"
```

Use the `ansible-navigator lint` command to validate it:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[user@host ~]$ ansible-navigator lint web_status.yml -m stdout
yaml[truthy]: Truthy value should be one of [false, true]
web_status.yml.yml:4 (1)

yaml[trailing-spaces]: Trailing spaces
web_status.yml:8 (2)

fqcn[action-core]: Use FQCN for builtin module actions (debug).
web_status.yml:13 Use `ansible.builtin.debug` or `ansible.legacy.debug` instead. (3)

yaml[indentation]: Wrong indentation: expected 8 but found 9
web_status.yml:15 (4)

yaml[empty-lines]: Too many blank lines (2 > 0)
web_status.yml:19 (5)
```

(1) Line 4 of the playbook (`become: yes`). The issue was detected by the `yaml[truthy]` rule. Ansible Lint indicates that you need to use `true` or `false` instead of the current `yes` value in the playbook.

(2) Line 8 of the playbook (`url: http://{{ ansible_fqdn }}`). The issue was detected by the `yaml[trailing-spaces]` rule. Ansible Lint indicates an issue related to trailing spaces. Although they are not visible, blank spaces exist at the end of the line. It is not a problem with the playbook directly, but many developers prefer not to have trailing white space in files stored in version control to avoid unnecessary differences as they edit the files.

(3) Line 13 of the playbook (`debug:`). The issue was detected by the `fqcn[action-core]` rule. Ansible Lint indicates that you should use the use of the *Fully Qualified Collection Name (FQCN)* for the module name on that task. You should use `ansible.builtin.debug` instead of just debug.

(4) Line 15 of the playbook (`msg: >-`). The issue was detected by the `yaml[indentation]` rule. Ansible Lint indicates incorrect indentation. It expected 8 spaces but found 9. You need to decrease the indentation by one space.

(5) The playbook ends with one or more blank lines, detected by the `yaml[empty-lines]` rule. Again, this is not a problem with the playbook directly, but it is good practice to remove this last blank line.

References

Ansible Documentation: Ansible Tips and Tricks

Ansible Porting Guides

Blocks

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

1.8. Guided Exercise
Implementing Recommended Ansible Practices

Apply recommended practices to an existing Ansible automation project.

Outcomes

- Set up a web server on `servera.lab.example.com` and `serverb.lab.example.com`.
- Set up a database server on `serverc.lab.example.com` and `serverd.lab.example.com`.
- Identify points of improvement on Ansible projects.
- Implement recommended practices.

Prerequisites

As the `student` user on the `workstation` machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
[student@workstation ~]$ lab start develop-recommend
```

Instructions

1. Review the exercise playbook.

   1.1. Change to the `/home/student/develop-recommend` directory:

   ```
   [student@workstation ~]$ cd ~/develop-recommend/
   [student@workstation develop-recommend]$
   ```

   1.2. Use the `tree` command to view the files in the directory:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation develop-recommend]$ tree
.
├── ansible.cfg
├── deploy.yml
├── group_vars
│   └── webservers
├── inventory
├── solutions
│   └── deploy.yml
└── templates
    └── index.html.j2

3 directories, 6 files
```

> **Note**
> The `/home/student/develop-recommend/solutions` directory contains the modified `deploy.yml` playbook that you can use for comparison.

1.3. Review the `deploy.yml` playbook.

Notice that the playbook is made up of three plays. The first play deploys a web application on managed hosts in the `webservers` group. The second play installs the MariaDB database server on the `serverc.lab.example.com` and `serverd.lab.example.com` hosts. The third play tests the web application on managed hosts in the `webservers` group.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
---
- name: Deploy a web application
  hosts: webservers
  become: true
  vars:
    service: httpd
    fw_service: http
    package:
      - httpd
  tasks:
    - name: Install the webserver
      ansible.builtin.dnf: name={{ package }}

    - name: Deploy web content
      ansible.builtin.template:
        src: templates/index.html.j2
        dest: "{{ web_dest }}"

    - name: Start the webserver
      ansible.builtin.service:
        name: "{{ service }}"
        state: started
        enabled: true

    - name: Enable the service in the firewall
      ansible.posix.firewalld:
          service: "{{ fw_service }}"
          state: enabled
          permanent: true
          immediate: true

- hosts:
    - serverc.lab.example.com
    - serverd.lab.example.com
  become: true
  vars:
    service: mariadb
    fw_service: mysql
    package:
      - mariadb-server
  tasks:
    - name: Install the database server
      ansible.builtin.dnf:
        name: "{{ package }}"

    - name: Start the database server
```

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
      ansible.builtin.shell:
        cmd: systemctl enable --now {{ service }}

    - name: Enable the service in the firewall
      ansible.posix.firewalld:
        service: "{{ fw_service }}"
        state: enabled
        permanent: true
        immediate: true

- name: Test web application
  hosts: webservers
  tasks:
    #TODO: Add test
```

2. Create a name for the second play and replace its list of hosts with a group, so that it follows recommended practices.

   2.1. In the inventory file, create the `dbservers` group consisting of the `serverc.lab.example.com` and `serverd.lab.example.com` hosts:

```
...output omitted...

[webservers]
servera.lab.example.com
serverb.lab.example.com

[dbservers]
serverc.lab.example.com
serverd.lab.example.com
```

   2.2. In the `deploy.yml` playbook, change the second play to run against the `dbservers` group, and give that unnamed play the `Deploy database servers` name:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
...output omitted...

- name: Deploy database servers
  hosts: dbservers
  become: true
  vars:
    service: mariadb
    fw_service: mysql
    package:
      - mariadb-server
...output omitted...
```

3. Update the variables used in the first play of the `deploy.yml` playbook so that they are defined and named following recommended practices.

   3.1. Move the `service`, `fw_service`, and `package` variables to the `~/develop-recommend/group_vars/webservers` group variable file. Rename them to `web_service`, `web_fw_service`, and `web_package`, respectively.

   The `~/develop-recommend/group_vars/webservers` file should contain the following content:

```
---
web_dest: /var/www/html/index.html
web_service: httpd
web_fw_service: http
web_package:
  - httpd
```

   3.2. Remove the `vars` section and the definitions of the `service`, `fw_service`, and `package` variables from the first play in the `deploy.yml` playbook.

   The top of the modified playbook should consist of the following content:

```
---
- name: Deploy a web application
  hosts: webservers
  become: true
  tasks:
    - name: Install the webserver
      ansible.builtin.dnf: name={{ package }}

...output omitted...
```

   3.3. Update the variable names used in the play in the following tasks:

   - `Install the webserver`

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

- Start the webserver
- Enable the service in the firewall

```
...output omitted...
  tasks:
    - name: Install the webserver
      ansible.builtin.dnf: name={{ web_package }}

    - name: Deploy web content
      ansible.builtin.template:
        src: templates/index.html.j2
        dest: "{{ web_dest }}"

    - name: Start the webserver
      ansible.builtin.service:
        name: "{{ web_service }}"
        state: started
        enabled: true

    - name: Enable the service in the firewall
      ansible.posix.firewalld:
          service: "{{ web_fw_service }}"
          state: enabled
          permanent: true
          immediate: true

...output omitted...
```

4. Update the variables used in the second play of the `deploy.yml` playbook so that they are defined and named following recommended practices.

   4.1. Create the `~/develop-recommend/group_vars/dbservers` file and move the `service`, `fw_service`, and `package` variables to this file. Rename the variables to `db_service`, `db_fw_service`, and `db_package`, respectively.

   The `~/develop-recommend/group_vars/dbservers` file should contain the following content:

```
---
db_service: mariadb
db_fw_service: mysql
db_package:
  - mariadb-server
```

   4.2. Remove the `vars` section and the definitions of the `service`, `fw_service`, and `package` variables from the second play in the `deploy.yml` playbook.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

The top of the modified play should consist of the following content:

```
...output omitted...

- name: Deploy database servers
  hosts: dbservers
  become: true
  tasks:
...output omitted...
```

4.3. Update the variable names used in the following tasks:

- `Install the database server`
- `Start the database server`
- `Enable the service in the firewall`

```
...output omitted...
  tasks:
    - name: Install the database server
      ansible.builtin.dnf:
        name: "{{ db_package }}"

    - name: Start the database server
      ansible.builtin.shell:
        cmd: systemctl enable --now {{ db_service }}

    - name: Enable the service in the firewall
      ansible.posix.firewalld:
        service: "{{ db_fw_service }}"
        state: enabled
        permanent: true
        immediate: true

...output omitted...
```

5. The third play in the `deploy.yml` playbook has no tasks. Add `block` and `rescue` directives to that play to recover from a failure on the hosts in the `webservers` group.

5.1. Create a `block` directive that checks the web application, and a `rescue` directive that restarts the web service if the web application is unavailable:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
...output omitted...

- name: Test web application
  hosts: webservers
  tasks:
    - name: Web application test
      block:
        - name: Check web application
          ansible.builtin.uri:
            url: http://{{ item }}
          loop: "{{ groups['webservers'] }}"

      rescue:
        - name: Restart web server
          ansible.builtin.service:
            name: "{{ web_service }}"
            state: restarted
```

6. Run Ansible Lint on the `deploy.yml` playbook and fix any detected issues.

6.1. Use the `ansible-navigator lint` command to detect potential problems in the `deploy.yml` playbook:

```
[student@workstation develop-recommend]$ ansible-navigator lint \
deploy.yml --eei aap.lab.example.com/ee-supported-rhel8 --pp missing
```

6.2. Six major issues are detected in the `deploy.yml` playbook:

```
  Severity Message                                                ... Line
0|Major    Avoid using free-form when calling module actions.    ...    6
1|Major    File permissions unset or incorrect.                 ...    9
2|Major    Wrong indentation: expected 8 but found 10           ...   22
3|Major    systemctl used in place of systemd module            ...   35
4|Major    Use shell only when shell functionality is required. ...   35
5|Major    Commands should not change things if nothing needs   ...   35


^b/PgUp page up  ^f/PgDn page down  ↑↓ scroll  esc  ...         Issues 6
```

Press ESC to exit from the `ansible-navigator` command.

6.3. The `Install the webserver` task in the first play of the `deploy.yml` playbook uses free-form when calling the `ansible.builtin.dnf` module. Change the task to use native YAML syntax:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
...output omitted...
  tasks:
    - name: Install the webserver
      ansible.builtin.dnf:
        name: "{{ web_package }}"

...output omitted...
```

6.4. Ensure that the file created in the `Deploy web content` task is owned by the `root` user and group, and has permissions of `0644`:

```
...output omitted...
    - name: Deploy web content
      ansible.builtin.template:
        src: templates/index.html.j2
        dest: "{{ web_dest }}"
        owner: root
        group: root
        mode: "0644"

...output omitted...
```

6.5. The `Enable the service in the firewall` task in the first play uses indentation that is not standard with the rest of the playbook. Update the indentation in the task:

```
...output omitted...

    - name: Enable the service in the firewall
      ansible.posix.firewalld:
        service: "{{ web_fw_service }}"
        state: enabled
        permanent: true
        immediate: true

...output omitted...
```

6.6. The `Start the database server` task uses the `ansible.builtin.shell` module to run the `systemctl` command to enable and start a service. Change this task to use the `ansible.builtin.service` module instead:

The updated task should now consist of the following content:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
    - name: Start the database server
      ansible.builtin.service:
        name: "{{ db_service }}"
        state: started
        enabled: true
```

6.7. Verify that the detected issues were fixed by using the `ansible-navigator lint` command again:

```
[student@workstation develop-recommend]$ ansible-navigator lint \
deploy.yml --eei aap.lab.example.com/ee-supported-rhel8 --pp missing
```

6.8. This time, Ansible Lint does not detect any issue:

```
                Success
                ──────────────────────────────────────────

                Congratulations, no lint issues found!
                ──────────────────────────────────────────
                                                      Ok
```

Press `Enter` to exit from the `ansible-navigator` command.

7. Review your playbook. The completed `~/develop-recommend/deploy.yml` playbook should consist of the following content:

> **Note**
> The `/home/student/develop-recommend/solutions` directory contains the modified `deploy.yml` playbook that you can use for comparison.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
---
- name: Deploy a web application
  hosts: webservers
  become: true
  tasks:
    - name: Install the webserver
      ansible.builtin.dnf:
        name: "{{ web_package }}"

    - name: Deploy web content
      ansible.builtin.template:
        src: templates/index.html.j2
        dest: "{{ web_dest }}"
        owner: root
        group: root
        mode: "0644"

    - name: Start the webserver
      ansible.builtin.service:
        name: "{{ web_service }}"
        state: started
        enabled: true

    - name: Enable the service in the firewall
      ansible.posix.firewalld:
        service: "{{ web_fw_service }}"
        state: enabled
        permanent: true
        immediate: true

- name: Deploy database servers
  hosts: dbservers
  become: true
  tasks:
    - name: Install the database server
      ansible.builtin.dnf:
        name: "{{ db_package }}"

    - name: Start the database server
      ansible.builtin.service:
        name: "{{ db_service }}"
        state: started
        enabled: true

    - name: Enable the service in the firewall
      ansible.posix.firewalld:
```

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
        service: "{{ db_fw_service }}"
        state: enabled
        permanent: true
        immediate: true

- name: Test web application
  hosts: webservers
  tasks:
    - name: Web application test
      block:
        - name: Check web application
          ansible.builtin.uri:
            url: http://{{ item }}
          loop: "{{ groups['webservers'] }}"

      rescue:
        - name: Restart web server
          ansible.builtin.service:
            name: "{{ web_service }}"
            state: restarted
```

8. Run the `deploy.yml` playbook using automation content navigator:

```
[student@workstation develop-recommend]$ ansible-navigator run \
deploy.yml --eei aap.lab.example.com/ee-supported-rhel8 --pp missing

  Play name                Ok Changed ... Failed ... Task count  Progress
0|Deploy a web application  10       8 ...      0 ...         10  Complete
1|Deploy database servers    8       6 ...      0 ...          8  Complete
2|Test web application       4       0 ...      0 ...          4  Complete

^b/PgUp page up  ^f/PgDn page down  ↑↓ scroll  esc ...         Successful
```

After the playbook completes, press Esc to exit the `ansible-navigator` command.

Finish

On the `workstation` machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish develop-recommend
```

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

1.9. Lab
Developing Playbooks with Ansible Automation Platform

Improve an existing Ansible Playbook to follow recommended practices, commit the changes to its Git repository, and test it with automation content navigator.

Outcomes

- Implement style guidelines for task and variable names in roles and playbooks.
- Commit changes to a Git repository.

Prerequisites

As the `student` user on the `workstation` machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available.

This command initializes the remote Git repository at `https://git.lab.example.com/student/develop-review.git`. The Git repository contains a `site.yml` playbook that uses a role to configure the number of seconds that the Grand Unified Bootloader (GRUB) waits before proceeding with the boot process. You can push changes to this repository by using `Stud3nt123` as the Git password.

```
[student@workstation ~]$ lab start develop-review
```

Instructions

1. Clone the `https://git.lab.example.com/student/develop-review.git` repository into the `/home/student/git-repos` directory and then create a branch named `exercise`.
2. Use Ansible Lint to identify points of improvement on the `site.yml` playbook. Use the `ee-supported-rhel8` automation execution environment available from `aap.lab.example.com`.
3. Implement recommended practices in the Ansible project, based on your findings with Ansible Lint. If you need to use a license, use the BSD license.
4. Run the `~/git-repos/develop-review/site.yml` playbook. If you updated your project correctly, then the playbook sets the persistent and temporary GRUB timeout values for the `servera.lab.example.com` load balancer.
5. Push your changes to the `exercise` branch of the remote Git repository.

Evaluation

As the `student` user on the `workstation` machine, use the `lab` command to grade your work. Correct any reported failures. Make sure to commit and push any additional changes to the remote Git repository and then rerun the command until successful.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation ~]$ lab grade develop-review
```

Finish

As the `student` user on the `workstation` machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish develop-review
```

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Solution

Improve an existing Ansible Playbook to follow recommended practices, commit the changes to its Git repository, and test it with automation content navigator.

Outcomes

- Implement style guidelines for task and variable names in roles and playbooks.
- Commit changes to a Git repository.

Prerequisites

As the `student` user on the `workstation` machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available.

This command initializes the remote Git repository at `https://git.lab.example.com/student/develop-review.git`. The Git repository contains a `site.yml` playbook that uses a role to configure the number of seconds that the Grand Unified Bootloader (GRUB) waits before proceeding with the boot process. You can push changes to this repository by using `Stud3nt123` as the Git password.

```
[student@workstation ~]$ lab start develop-review
```

Instructions

1. Clone the `https://git.lab.example.com/student/develop-review.git` repository into the `/home/student/git-repos` directory and then create a branch named `exercise`.

   1.1. From a terminal, create the `/home/student/git-repos` directory if it does not already exist, and then change into it:

   ```
   [student@workstation ~]$ mkdir -p ~/git-repos/
   [student@workstation ~]$ cd ~/git-repos/
   [student@workstation git-repos]$
   ```

   1.2. Clone the `https://git.lab.example.com/student/develop-review.git` repository and then change to the cloned repository:

   ```
   [student@workstation git-repos]$ git clone \
   https://git.lab.example.com/student/develop-review.git
   Cloning into 'develop-review'...
   ...output omitted...
   [student@workstation git-repos]$ cd develop-review
   [student@workstation develop-review]$
   ```

   1.3. Create the `exercise` branch and check it out:

```
[student@workstation develop-review]$ git checkout -b exercise
Switched to a new branch 'exercise'
```

2. Use Ansible Lint to identify points of improvement on the `site.yml` playbook. Use the `ee-supported-rhel8` automation execution environment available from `aap.lab.example.com`.

   2.1. Use the `ansible-navigator lint` command to detect potential problems in the `site.yml` playbook:

```
[student@workstation develop-review]$ ansible-navigator lint \
site.yml --eei aap.lab.example.com/ee-supported-rhel8 --pp missing
```

   2.2. Seven major issues are detected in the `deploy.yml` playbook:

```
  Severity Message                                              ... Line
0|Major    $ 'timeout' does not match any of the regexes: '^(?  ...    1
1|Major    Variables names must not be Ansible reserved names.  ...    2
2|Major    Variables names from within roles should use grub_   ...    3
3|Major    Should change default metadata: license              ...    1
4|Major    All tasks should be named.                           ...    2
5|Major    All tasks should be named.                           ...    8
6|Major    All plays should be named.                           ...    2

^b/PgUp page up  ^f/PgDn page down  ↑↓ scroll  esc  ...         Issues 7
```

   To see details of any detected issue, enter its corresponding number. Type `:q` and then press `Enter` to exit the `ansible-navigator` command.

3. Implement recommended practices in the Ansible project, based on your findings with Ansible Lint. If you need to use a license, use the BSD license.
   - Ensure that plays and tasks have names associated with them.
   - Change the `timeout` and `persistent` role variables names to use the `grub_` prefix.
   - Update the default metadata license for the `grub` role to use the BSD license.

   3.1. Edit the `site.yml` playbook to include a name for the play:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
---
- name: Sets the GRUB timeout
  hosts:
    - lb_servers
    - web_servers
  become: true
  roles:
    - role: grub
```

3.2. Edit the `roles/grub/tasks/main.yml` file to add a name to each task:

```
---
- name: Sets persistent GRUB timeout
  ansible.builtin.lineinfile:
    path: /etc/default/grub
    regexp: "^GRUB_TIMEOUT="
    line: "GRUB_TIMEOUT={{ timeout | int }}"
  when: persistent

- name: Sets temporary GRUB timeout
  ansible.builtin.lineinfile:
    path: /boot/grub2/grub.cfg
    regexp: "^set timeout="
    line: "set timeout={{ timeout | int }}"
```

3.3. Edit the `roles/grub/defaults/main.yml` file to add the `grub_` prefix:

```
---
grub_timeout: 0
grub_persistent: true
```

3.4. Edit the `group_vars/lb_servers.yml` to add the `grub_` prefix:

```
---
grub_timeout: 30
grub_persistent: true
```

3.5. Update the `roles/grub/tasks/main.yml` file to use the new variable names:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
---
- name: Sets persistent GRUB timeout
  ansible.builtin.lineinfile:
    path: /etc/default/grub
    regexp: "^GRUB_TIMEOUT="
    line: "GRUB_TIMEOUT={{ grub_timeout | int }}"
  when: grub_persistent

- name: Sets temporary GRUB timeout
  ansible.builtin.lineinfile:
    path: /boot/grub2/grub.cfg
    regexp: "^set timeout="
    line: "set timeout={{ grub_timeout | int }}"
```

3.6. *(Optional)* Update the `roles/grub/README.md` file to change the role and variable names.

The `README.md` file provides information on how to use the role, but the file is not used by the role itself. The grading script does not evaluate this file.

```
...output omitted...
* `grub_timeout`: An integer used to set the GRUB timeout (in seconds). If a
user does not interrupt the boot process within this timeout, then the
system uses the default kernel with the default boot options. Defaults to a
value of `0`.
* `grub_persistent`: A boolean that controls whether a change to the GRUB
timeout is persistent or temporary. A persistent change also updates the
`/etc/default/grub` file so that the new timeout will be applied, even if
students install a new kernel. Defaults to a value of `true`.

Example Playbook
----------------

    - name: Sets the GRUB timeout
      hosts: all
      become: true
      roles:
        - role: grub
          grub_timeout: 15
          grub_persistent: true
...output omitted...
```

3.7. Update the `roles/grub/meta/main.yml` file to establish a license for the role.

Although not choosing a license for the role is not a problem that breaks your project, it is good practice to do it.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
...output omitted...
  # Choose a valid license ID from https://spdx.org - some suggested
licenses:
  # - BSD-3-Clause (default)
  # - MIT
  # - GPL-2.0-or-later
  # - GPL-3.0-only
  # - Apache-2.0
  # - CC-BY-4.0
  license: BSD

...output omitted...
```

3.8. Verify that the detected issues were fixed by using the `ansible-navigator lint` command again:

```
[student@workstation develop-review]$ ansible-navigator lint \
site.yml --eei aap.lab.example.com/ee-supported-rhel8 --pp missing
```

3.9. This time, Ansible Lint does not detect any issue:

```
                Success
                ──────────────────────────────────────────

                Congratulations, no lint issues found!
                ──────────────────────────────────────────
                                                      Ok
```

Press `Enter` to exit from the `ansible-navigator` command.

4. Run the `~/git-repos/develop-review/site.yml` playbook. If you updated your project correctly, then the playbook sets the persistent and temporary GRUB timeout values for the `servera.lab.example.com` load balancer.

4.1. Run the `site.yml` playbook using the `ee-supported-rhel8` automation execution environment. Verify that the playbook has no errors:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation develop-review]$ ansible-navigator run site.yml \
--eei aap.lab.example.com/ee-supported-rhel8 --pp missing -m stdout
...output omitted...
PLAY RECAP
*********************************************************************
servera.lab.example.com    : ok=3    changed=2    unreachable=0    failed=0
...
serverb.lab.example.com    : ok=3    changed=0    unreachable=0    failed=0
...
serverc.lab.example.com    : ok=3    changed=0    unreachable=0    failed=0
...
```

5. Push your changes to the `exercise` branch of the remote Git repository.

5.1. Display changes made to the local Git repository:

```
[student@workstation develop-review]$ git status
On branch exercise
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   group_vars/lb_servers.yml
        modified:   roles/grub/README.md
        modified:   roles/grub/defaults/main.yml
        modified:   roles/grub/meta/main.yml
        modified:   roles/grub/tasks/main.yml
        modified:   site.yml

no changes added to commit (use "git add" and/or "git commit -a")
```

5.2. Use the `git add` command to stage all the modified files for the next commit:

```
[student@workstation develop-review]$ git add .
...output omitted...
```

5.3. Commit the staged files to the local Git repository:

```
[student@workstation develop-review]$ git commit -m 'Changes for exercise'
...output omitted...
```

5.4. Push the changes from the local Git repository to the `exercise` branch on the remote Git repository. If prompted, use `Stud3nt123` as the Git password:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation develop-review]$ git push -u origin exercise
Password for 'https://student@git.lab.example.com': Stud3nt123
...output omitted...
```

5.5. Verify that your changes have been pushed to the `exercise` branch:

```
[student@workstation develop-review]$ git status
On branch exercise
Your branch is up to date with 'origin/exercise'.

nothing to commit, working tree clean
```

Evaluation

As the `student` user on the `workstation` machine, use the `lab` command to grade your work. Correct any reported failures. Make sure to commit and push any additional changes to the remote Git repository and then rerun the command until successful.

```
[student@workstation ~]$ lab grade develop-review
```

Finish

As the `student` user on the `workstation` machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish develop-review
```

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

1.10. Summary

- Red Hat Ansible Automation Platform 2 simplifies the development and use of Ansible Playbooks at scale, by introducing new features such as Ansible Content Collections and automation execution environments.
- Starting with Ansible Automation Platform 2.5, automation controller, Event-Driven Ansible, and automation hub are all integrated into a new unified UI, making it easier to collaborate with your team and with teams across your organization.
- Automation content navigator (`ansible-navigator`) extends and can replace many previous Ansible commands, including `ansible-playbook`.
- An Ansible Content Collection provides a set of related Ansible modules, roles, and plug-ins.
- Automation execution environments are container images that contain a specific version of Ansible Core, Ansible Content Collections, and any Python libraries, executables, or other dependencies needed to run playbooks.
- You should manage your automation content in a version control system, such as Git, which enables you to track and control changes to your automation code.
- Ansible Lint is a tool that uses rules to detect issues in a playbook. You can run this tool inside an automation execution environment by running the `ansible-navigator lint` command.
- Implementing recommended practices facilitates collaboration and reduces potential problems.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Chapter 2.
Managing Ansible Content Collections and Automation Execution Environments

Goal

Run playbooks that use Ansible Content Collections not included in `ansible-core`, either by using an existing automation execution environment or by downloading the required collections.

Sections

- Reusing Content from Ansible Content Collections (and Guided Exercise)
- Finding and Installing Ansible Content Collections (and Guided Exercise)
- Selecting an Automation Execution Environment (and Guided Exercise)

Lab

- Managing Ansible Content Collections and Automation Execution Environments

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

2.1. Reusing Content from Ansible Content Collections

Objectives

- Describe how Ansible Content Collections are used to distribute modules and plug-ins, and create plays that use content from them.

Defining Ansible Content Collections

Ansible Content Collections are a distribution format for Ansible content. A collection provides a set of related modules, roles, and plug-ins that you can download to your control node and then use in your playbooks.

For example:

- The `redhat.insights` collection groups modules and roles that you can use to register a system with Red Hat Insights for Red Hat Enterprise Linux.
- The `cisco.ios` collection groups modules and plug-ins that manage Cisco IOS network appliances. The Cisco company supports and maintains that collection.
- The `community.crypto` collection provides modules that create SSL/TLS certificates.

You can use collections to separate core Ansible code updates from updates to collection modules and plug-ins. Vendors and developers can then maintain and distribute their collections at their own pace, independently of Ansible releases. You can develop your own collections to provide custom roles and modules to your teams.

Collections also give you more flexibility. By using collections, you can install only the content you need instead of installing all supported modules. You can also select a specific version of a collection (possibly an earlier or a later one) or choose between a version of a collection supported by Red Hat and one provided by the community.

Upstream Ansible unbundled most modules from the core Ansible code in Ansible Base 2.10 and Ansible Core 2.11 and placed them in collections. Red Hat Ansible Automation Platform 2.5 provides automation execution environments based on Ansible Core 2.16 that inherit this feature.

Organizing Ansible Content Collections in Namespaces

To make it easier to specify collections and their contents by name, collection names are organized into *namespaces*. Vendors, partners, developers, and content creators can use namespaces to assign unique names to their collections without conflicting with other developers.

The namespace is the first part of a collection name. For example, all the collections that the Ansible community maintain are in the `community` namespace, and have names such as `community.crypto`, `community.postgresql`, and `community.rabbitmq`. Collections that Red Hat maintains and supports use the `redhat` namespace, and have names such as `redhat.openshift_virtualization`, `redhat.satellite`, and `redhat.insights`. Red Hat also supports and maintains collections in the `ansible` namespace, such as the `ansible.controller`, `ansible.posix`, and `ansible.utils` collections.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Names of namespaces are limited to ASCII lowercase letters, numbers, and underscores, must be at least two characters long, and must not start with an underscore.

Using Ansible Content Collections

The automation execution environments that Red Hat provides already include some collections. You can install additional collections on your local system or create a custom automation execution environment that incorporates these collections. Other sections of the course present those subjects in more detail.

Accessing Ansible Content Collection Documentation

Use the `ansible-navigator collections` command to list the collections available in automation execution environments. The following output shows the collections available in the `ee-supported-rhel8` automation execution environment:

```
   Name                 Version  Shadowed  Type       Path
 0|amazon.aws           7.4.0    False     contained  /usr/share/ansible/...
 1|ansible.builtin      2.16.14  False     contained  /usr/lib/python3.11...
 2|ansible.controller   4.6.3    False     contained  /usr/share/ansible/...
...output omitted...
 9|ansible.scm          3.0.0    False     contained  /usr/share/ansible/...
10|ansible.security     3.0.0    False     contained  /usr/share/ansible/...
11|ansible.snmp         3.0.0    False     contained  /usr/share/ansible/...
...output omitted...
```

To list the modules and plug-ins for a collection, type the associated collection number. If the collection number is greater than nine, type a colon before the collection number, for example, `:11`, for line 11, and then press `Enter`. The following output shows the modules available in the `ansible.snmp` collection:

```
  Ansible.snmp  Type        Added  Deprecated  Description
0|get           module      1.0.0  False       Perform an SNMP get against...
1|set           module      1.0.0  False       Perform an SNMP set against...
2|v1            connection  1.0.0  False       Make SNMP v1 connections to...
3|v2c           connection  1.0.0  False       Make SNMP v2c connections t...
4|v3_usm        connection  1.0.0  False       Make SNMP v3 user-based sec...
5|walk          module      1.0.0  False       Perform an SNMP walk agains...
```

Enter the module number to access its documentation. The `ansible-navigator collections` command renders the documentation in the YAML format by default. You can use the `--mode stdout` (or `-m stdout`) option of the `ansible-navigator doc` command to render the documentation in plain text. For example, the following command displays the documentation of the `ansible.snmp.get` module:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[user@host ~]$ ansible-navigator doc ansible.snmp.get --mode stdout
> ANSIBLE.SNMP.GET
(/usr/share/ansible/collections/ansible_collections/ansible/snmp/plugins/modules/ge
t.py)

        Perform an SNMP get against a remote device for one or more OIDs

ADDED IN: version 1.0.0 of ansible.snmp

  * note: This module has a corresponding action plugin.

OPTIONS (= is mandatory):

- best_guess
        This setting controls how return value oids are parsed.
        Setting to 0 causes a regular lookup.
        Setting to 1 causes a regular expression match (defined as -Ib in snmpcmd).
        Setting to 2 causes a random access lookup (defined as -IR in snmpcmd).
        choices: [0, 1, 2]
        default: 0
        type: int
...output omitted...
```

Using Ansible Content Collections in Playbooks

To use a module or a role from a collection, refer to it with its *fully qualified collection name (FQCN)*. For example, use `redhat.insights.insights_register` to refer to the `insights_register` module.

The play in the following playbook includes a task that calls the `insights_register` module:

```
---
- name: Register new systems
  hosts: db.example.com
  become: true
  tasks:
    - name: Ensure the new system is registered with Red Hat Insights
      redhat.insights.insights_register:
        state: present
        force_reregister: true
```

The play in the following playbook uses the `organizations` role from the `redhat.satellite` collection:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
---
- name: Add the test organizations to Red Hat Satellite
  hosts: host.example.com
  become: true
  tasks:
    - name: Ensure the organizations exist
      ansible.builtin.include_role:
        name: redhat.satellite.organizations
      vars:
        satellite_server_url: https://sat.example.com
        satellite_username: admin
        satellite_password: Sup3r53cr3t
        satellite_organizations:
          - name: test1
            label: tst1
            state: present
            description: Test organization 1
          - name: test2
            label: tst2
            state: present
            description: Test organization 2
```

The play in the following playbook uses the k8s lookup plug-in from the `kubernetes.core` collection:

```
---
- name: Fetch the namespaces from Kubernetes
  hosts: host.example.com
  tasks:
    - name: Retrieve the namespaces as a list
      ansible.builtin.set_fact:
        ns: "{{ lookup('kubernetes.core.k8s', kind='Namespace') }}"
```

Finding Ansible Content Collections

If you plan to update legacy playbooks that use modules moved to collections, then you need to identify in which collection these modules are now available.

The `ansible-core` package provides the `ansible_builtin_runtime.yml` file, which maps earlier module names from Ansible 2.9 to their new fully qualified collection names (FQCNs). Use the following command to locate that file on your system:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[user@host ~]$ rpm -ql ansible-core | grep ansible_builtin_runtime.yml
/usr/lib/python3.11/site-packages/ansible/config/ansible_builtin_runtime.yml
```

Note

You can also find the `ansible_builtin_runtime.yml` file at `https://github.com/ansible/ansible/blob/devel/lib/ansible/config/ansible_builtin_runtime.yml`.

The following extract from that file shows that the `acl` module is now part of the `ansible.posix` collection, which uses `ansible.posix.acl` as its FQCN:

```
...output omitted...
    acl:
      redirect: ansible.posix.acl
    synchronize:
      redirect: ansible.posix.synchronize
    at:
      redirect: ansible.posix.at
    authorized_key:
      redirect: ansible.posix.authorized_key
    mount:
      redirect: ansible.posix.mount
...output omitted...
```

This redirection mechanism is also used by many Ansible Content Collections to translate short names to FQCNs, so that playbooks written for Ansible 2.9 (Red Hat Ansible Automation Platform 1.2) require less immediate migration work.

Important

Red Hat currently provides the `registry.redhat.io/ansible-automation-platform/ee-29-rhel8` automation execution environment as a temporary solution for migrating playbooks that do not use Ansible Content Collections. That automation execution environment provides Ansible 2.9.

The `ee-29-rhel8` automation execution environment is no longer included in the latest bundled installation archive files. That automation execution environment is scheduled to be deprecated in Ansible Automation Platform 2.6.

Using the Built-in Ansible Content Collection

Ansible always includes a special collection named `ansible.builtin`. This collection includes a set of common modules, such as `copy`, `template`, `file`, `dnf`, `command`, and `service`.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Although you can use the short names of these modules in your playbooks, Red Hat recommends that you use the FQCN notation to prevent conflicts with collections that might use the same module names.

The following playbook uses the FQCN notation for the `dnf`, `copy`, and `service` modules:

```
---
- name: Install and start Apache HTTPD
  hosts: web
  become: true
  gather_facts: false
  tasks:
    - name: Ensure the httpd package is present
      ansible.builtin.dnf:
        name: httpd
        state: present

    - name: Ensure the index.html file is present
      ansible.builtin.copy:
        src: files/index.html
        dest: /var/www/html/index.html
        owner: root
        group: root
        mode: "0644"

    - name: Ensure the httpd service is started
      ansible.builtin.service:
        name: httpd
        state: started
        enabled: true
```

References

Knowledgebase: "Where Did a Module Go During the Ansible 2.9 to 2.10 Content Migration?"

Using Ansible Collections - Ansible Documentation

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

2.2. Guided Exercise
Reusing Content from Ansible Content Collections

Write a playbook that uses modules from an Ansible Content Collection.

Outcomes

- Identify the Ansible Content Collection that provides a specific module.
- Create a task that specifies a module by its fully qualified collection name (FQCN).

Prerequisites

As the `student` user on the `workstation` machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
[student@workstation ~]$ lab start manage-usage
```

Instructions

1. Change to the `/home/student/manage-usage` directory and attempt to run the `basic-web.yml` playbook by using the `ansible-playbook` command.

   1.1. Change to the `/home/student/manage-usage` directory and list the contents of the directory:

   ```
   [student@workstation ~]$ cd ~/manage-usage/
   [student@workstation manage-usage]$ ls
   ansible.cfg  basic-web.yml  inventory
   ```

   1.2. Try to run the `basic-web.yml` playbook. The attempt fails with an error message.

   ```
   [student@workstation manage-usage]$ ansible-playbook basic-web.yml
   ERROR! couldn't resolve module/action 'firewalld'. This often indicates a
   misspelling, missing collection, or incorrect module path.
   ...output omitted...
   ```

   1.3. Try to find the `firewalld` module in the default modules installed by the `ansible-core` package. The command does not return any output, indicating that the module is not found.

```
[student@workstation manage-usage]$ ansible-doc -l | grep firewalld
```

> **Important**
>
> The `ansible-playbook` command provided by Red Hat Ansible Automation Platform 2 is based on the Ansible Core 2.16 package. It does not include any Ansible Content Collections other than `ansible.builtin`, which is why the preceding two substeps fail. Your control node also does not have any additional Ansible Content Collections installed. The `firewalld` module was moved into a separate collection after Ansible 2.9.

2. Use automation content navigator with the `aap.lab.example.com/ee-supported-rhel8` automation execution environment to identify the Ansible Content Collection that provides the `firewalld` module.

   2.1. Use the `podman login` command to log in to the container registry at `aap.lab.example.com`. Log in as `admin` with `redhat` as the password. This step simulates a customer logging in to `registry.redhat.io` and ensures that the specified automation execution environment is pulled down to the local system if it does not already exist.

```
[student@workstation manage-usage]$ podman login aap.lab.example.com
Username: admin
Password: redhat
Login Succeeded!
```

   2.2. List the module documentation available in the `aap.lab.example.com/ee-supported-rhel8` automation execution environment. The colon at the end of the output indicates that the output fills more than one screen. You can scroll through the rest of the output by using the arrow keys or the space bar.

```
[student@workstation manage-usage]$ ansible-navigator doc -l \
--eei aap.lab.example.com/ee-supported-rhel8 --pp missing -m stdout
amazon.aws.autoscaling_group        Create or delete AWS AutoScaling Groups
...
amazon.aws.autoscaling_group_info   Gather information about EC2 Auto
Scalin...
amazon.aws.aws_az_info              Gather information about availability
zo...
...output omitted...
amazon.aws.ec2_vpc_igw_info         Gather information about internet
gatewa...
amazon.aws.ec2_vpc_nat_gateway      Manage AWS VPC NAT Gateways
:
```

   2.3. Type `/firewalld` and then press `Enter` to search the module documentation for `firewalld`.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
amazon.aws.autoscaling_group        Create or delete AWS AutoScaling Groups
...
amazon.aws.autoscaling_group_info   Gather information about EC2 Auto
Scalin...
amazon.aws.aws_az_info              Gather information about availability
zo...
...output omitted...
/firewalld
```

The `ansible.posix` content collection provides the `firewalld` module. The fully qualified collection name for the `firewalld` module is `ansible.posix.firewalld`.

```
ansible.posix.firewalld         Manage arbitrary ports/services with
firewalld
ansible.posix.firewalld_info    Gather information about firewalld
...output omitted...
```

Press `q` to exit the `ansible-navigator` command.

3. Update the `basic-web.yml` playbook to use the fully qualified collection names for each module. The `ansible.posix` collection provides the `firewalld` module. The `ansible.builtin` collection provides the other modules. The modified `basic-web.yml` playbook should consist of the following content:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
---
- name: Configure a basic web server
  hosts: servera.lab.example.com
  become: true
  gather_facts: true
  tasks:
    - name: Install software
      ansible.builtin.dnf:
        name:
          - httpd
          - firewalld
        state: latest

    - name: Start and enable services
      ansible.builtin.service:
        name: "{{ item }}"
        state: started
        enabled: true
      loop:
        - httpd
        - firewalld

    - name: Open firewall access
      ansible.posix.firewalld:
        service: http
        immediate: true
        permanent: true
        state: enabled

    - name: Configure simple index.html
      ansible.builtin.copy:
        content: >
          Hello world from {{ ansible_facts['fqdn'] }}.
        dest: /var/www/html/index.html
        owner: root
        group: root
        mode: "0644"
```

4. Use the `aap.lab.example.com/ee-supported-rhel8` automation execution environment to verify that the `basic-web.yml` playbook completes successfully.

   4.1. Run the `basic-web.yml` playbook:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation manage-usage]$ ansible-navigator run basic-web.yml \
--eei aap.lab.example.com/ee-supported-rhel8 --pp missing

  Play name                     Ok  Changed ... Failed ... Task count
Progress
0│Configure a basic web server   5        4 ...      0 ...          5
Complete

^f/PgUp page up     ^b/PgDn page down     ↑↓ scroll    esc back  ...
Successful
```

When the `Progress` column shows `Successful`, press `Esc` to exit from the `ansible-navigator` command.

4.2. Use the `curl` command to display web content on `servera.lab.example.com`.

```
[student@workstation manage-usage]$ curl servera.lab.example.com
Hello world from servera.lab.example.com.
```

Finish

On the `workstation` machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish manage-usage
```

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

2.3. Finding and Installing Ansible Content Collections

Objectives

- Search automation hub for Ansible Content Collections, and install them from the command line by name or by using a `requirements.yml` file.

Sources for Ansible Content Collections

Two sources provide Ansible Content Collections: automation hub, which is officially supported by Red Hat; and Ansible Galaxy, which is provided by the open source community.

Automation hub

Automation hub hosts Red Hat Certified Ansible Content Collections. Red Hat and its partners review, maintain, update, and fully support these collections. For example, the `redhat.openshift_virtualization`, `redhat.satellite`, `redhat.insights`, and `cisco.ios` collections are available on this platform.

You need a valid Red Hat Ansible Automation Platform subscription to access automation hub. Use the automation hub web UI at https://console.redhat.com/ansible/automation-hub/ to list and access the collections.

Ansible Galaxy

Ansible Galaxy is a public site that hosts unsupported collections submitted by various Ansible developers and users. For example, the `community.crypto`, `community.postgresql`, and `community.rabbitmq` collections are all available from that platform.

Use the Ansible Galaxy web UI at https://galaxy.ansible.com/ to search it for collections.

In addition to these two distribution platforms, you can also deploy a private automation hub on-premise to store your custom collections or to provide a curated list of collections to your teams. You can also retrieve collections directly from their Git repositories.

Finding Collections on Automation Hub

Use the automation hub web UI at https://console.redhat.com/ansible/automation-hub/ to browse its Ansible Content Collections.

Go to **Automation Hub > Collections** to list the collections. Use the search menu to adjust how the filter works, and then filter collections by keyword, repository, namespace, tag, or sign state.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

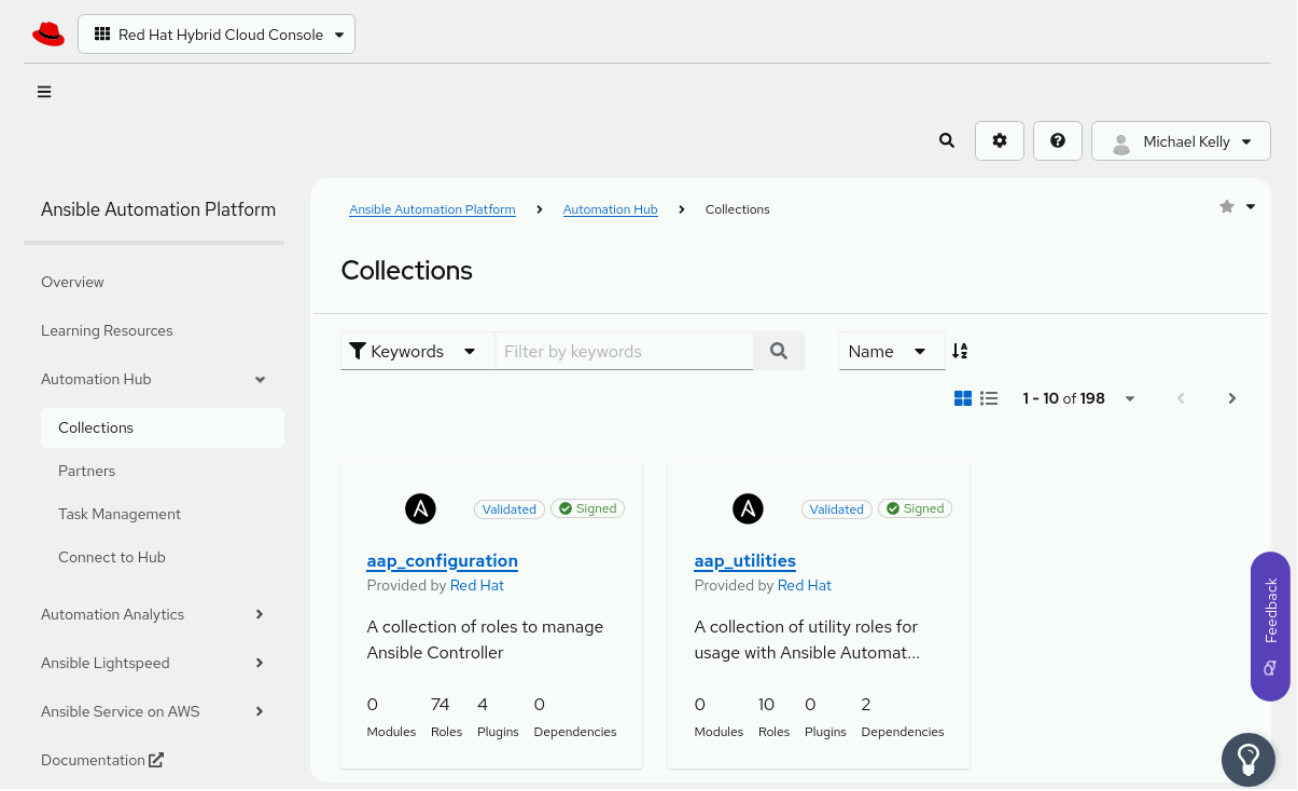


Figure 16. Listing collections in automation hub

You can also go to **Automation Hub > Partners** to browse the collections by partner.

Installing Ansible Content Collections

Your automation execution environment might not include the Ansible Content Collection that you want to use in your playbook. If your collection is not included, then you must make it available.

One method is to install the collection in the directory containing your Ansible Playbook before running the `ansible-navigator` command. Normally you create a `collections` subdirectory in that directory and put your collection in that subdirectory.

Installing Collections from the Command Line

Use the `ansible-galaxy` command to download collections from a number of possible sources, including Ansible Galaxy.

The following example uses the `ansible-galaxy` command with the `collection` argument to download and then install the `community.crypto` collection on the local system:

```
[user@host ~]$ ansible-galaxy collection install community.crypto
```

By default, the `ansible-galaxy` command tries to download the collection from Ansible Galaxy. Configuring `ansible-galaxy` to use automation hub or a private automation hub to get collections is covered later in this section.

The `ansible-galaxy` command can also install a collection from a local or a remote `.tar` file or a Git repository.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[user@host ~]$ ansible-galaxy collection install /tmp/community-dns-3.2.2.tar.gz
[user@host ~]$ ansible-galaxy collection install \
http://www.example.com/redhat-insights-1.3.0.tar.gz
[user@host ~]$ ansible-galaxy collection install \
git@github.com:ansible-collections/community.mysql.git
```

When Ansible runs a playbook, it looks for collections in a `collections` directory in the same directory as the playbook. If it cannot find the required collection in that location, then it uses the `collections_path` directive from the `ansible.cfg` configuration file. That directive specifies a colon-separated list of paths on the system where Ansible looks for installed collections.

> **Important**
>
> If you modify the `collections_path` directive in an Ansible project's `ansible.cfg` file, it is important that you keep `/usr/share/ansible/collections` as one of the directories containing collections.
>
> The Ansible configuration file for the project is used by the automation execution environment, and the collections included in the execution environment are in the `/usr/share/ansible/collections` directory inside the container.

By default, the `ansible-galaxy` command installs the collections in the first directory that the `collections_path` directive defines.

The default value for `collections_path` is `~/.ansible/collections:/usr/share/ansible/collections`. Therefore, the `ansible-galaxy` command installs collections in the `~/.ansible/collections` directory by default.

To install a collection in a different directory, use the `--collections-path` (or `-p`) option:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[user@host ~]$ ansible-galaxy collection install community.postgresql \
-p ~/myproject/collections/
```

Important

The `ansible-galaxy` command was originally written for `ansible-playbook` users, for whom the control node and the automation execution environment are the same system. The `ansible-galaxy` command installed files in the `/usr/share/ansible/collections` directory on the control node so that the `ansible-playbook` command could use them.

Red Hat Ansible Automation Platform 2 separates automation execution environments from the control node so that they run in containers. These automation execution environments have limited access to the control node's file systems. When you run the playbook in the container, the `collections_path` directive uses the `/usr/share/ansible/collections` directory inside the container, not on the control node. Therefore, if you install your collections in the `~/.ansible/collections` or `/usr/share/ansible/collections` directories on the control node, the automation execution environment cannot use them.

However, automation execution environments do have access to the directory that hosts the playbook that they are running. If you install your collections in a `./collections` directory in the same directory as your playbook, then the automation execution environment can access them.

Installing Collections with a Requirements File

The `collections/requirements.yml` file is a list of the collections needed to run playbooks in your Ansible project, just like you can create a `roles/requirements.yml` file to list the roles that you need to install. By adding a `collections/requirements.yml` file in your Ansible project, your team members can immediately identify the required collections, and they can use `ansible-galaxy` to manually install them. Automation controller detects the `collections/requirements.yml` file and automatically installs the collections when it synchronizes a project.

The following `collections/requirements.yml` file lists several collections to install. You can target a specific version of an Ansible Content Collection or provide local or remote `.tar` files or Git repositories as the source of the collection.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
---
collections:
  - name: community.crypto

  - name: ansible.posix
    version: 1.2.0

  - name: /tmp/community-dns-3.2.2.tar.gz

  - name: http://www.example.com/redhat-insights-1.3.0.tar.gz

  - name: git@github.com:ansible-collections/community.mysql.git
```

You can process that file with the `ansible-galaxy` command to install those collections. Use the `--requirements-file` (or `-r`) option to provide the `requirements.yml` file to the command:

```
[user@host ~]$ ansible-galaxy collection install -r requirements.yml
```

Listing Installed Collections

The `ansible-navigator collections` command starts an interactive text-based user interface (TUI) that lists the collections installed in the current automation execution environment.

```
   Name               Version      Shadowed Type      Path
 0|amazon.aws         7.4.0        False    contained /usr/share/ansible/col...
 1|ansible.builtin    2.16.14      False    contained /usr/lib/python3.11/si...
 2|ansible.controller 4.6.3        False    contained /usr/share/ansible/col...
 3|ansible.eda        2.3.0        False    contained /usr/share/ansible/col...
 4|ansible.hub        1.0.0        False    contained /usr/share/ansible/col...
 5|ansible.netcommon  7.1.0        False    contained /usr/share/ansible/col...
 6|ansible.network    4.0.0        False    contained /usr/share/ansible/col...
 7|ansible.platform   2.5.20241218 False    contained /usr/share/ansible/col...
 8|ansible.posix      1.5.4        False    contained /usr/share/ansible/col...
...output omitted...
```

If you enter the line number of a particular collection, `ansible-navigator` displays detailed information about the content of that collection in the TUI.

For example, if `ansible-navigator` displays the preceding output, and you type 8, then the following output is displayed:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
   Ansible.posix     Type     Added Deprecated Description
 0|acl              module   1.0.0 False      Set and retrieve file ACL inf...
 1|at               module   1.0.0 False      Schedule the execution of a c...
 2|authorized_key   module   1.0.0 False      Adds or removes an SSH author...
 3|cgroup_perf_recap callback None  False      Profiles system activity of t...
...output omitted...
```

From that page, you could type 1 to get documentation and usage examples for the `ansible.posix.at` module:

```
Image: ansible.posix.at
Description: Schedule the execution of a command or script file via the at c...
 0|---
 1|additional_information: {}
 2|collection_info:
 3|  authors:
 4|  - Ansible (github.com/ansible)
 5|  dependencies: {}
 6|  description: Ansible Collection targeting POSIX and POSIX-ish platforms
...output omitted...
```

You can also use the `ansible-galaxy collection list` command to list the collections available in the directories specified by your current `collections_path` Ansible configuration directive on your control node. The output does not list collections that you might have installed in directories not specified by that directive.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[user@host ~]$ ansible-galaxy collection list

# /home/user/.ansible/collections/ansible_collections
Collection       Version
---------------- ------
ansible.posix    2.0.0
community.crypto 2.26.0
community.dns    3.2.2
community.mysql  3.13.0
redhat.insights  1.3.0

# /usr/share/ansible/collections/ansible_collections
Collection               Version
------------------------ -------
redhat.rhel_system_roles 1.23.0
```

Important

The `ansible-galaxy collection list` command lists the Ansible Content Collections installed in the directories specified by the `collections_path` directive on the *control node*. This might include collections installed in your Ansible project directory, if that location is listed in your `collections_path` directive.

Use the `ansible-navigator collections` command to list the Ansible Content Collections in your current automation execution environment, including collections installed in the directories listed in your `collections_path` directive.

Configuring Collection Sources

By default, the `ansible-galaxy` command uses Ansible Galaxy at `https://galaxy.ansible.com/` to download collections. You can configure additional distribution platforms in the `ansible.cfg` configuration file.

Installing Collections from Automation Hub

For the `ansible-galaxy` command to also use automation hub, add the following directives to the `ansible.cfg` file:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
...output omitted...
[galaxy]
server_list = hub_certified, hub_validated, galaxy (1)

[galaxy_server.hub_certified]
url=https://console.redhat.com/api/automation-hub/content/published/ (2)
auth_url=https://sso.redhat.com/auth/realms/redhat-external/protocol/openid-
connect/token (3)
token=eyJh...Jf0o (4)

[galaxy_server.hub_validated]
url=https://console.redhat.com/api/automation-hub/content/validated/ (5)
auth_url=https://sso.redhat.com/auth/realms/redhat-external/protocol/openid-
connect/token
token=eyJh...Jf0o

[galaxy_server.galaxy]
url=https://galaxy.ansible.com/
```

(1) Lists all the repositories that the `ansible-galaxy` command must use, in order. For each name that you define, add a `[galaxy_server.`*name*`]` section to provide the connection parameters. Because automation hub might not provide all the collections that your playbooks need, you can add Ansible Galaxy in the last position as a fallback. This way, if the collection is not available in automation hub, then the `ansible-galaxy` command uses Ansible Galaxy to retrieve it.

(2) Provides the URL for certified content from automation hub.

(3) Provides the URL for authentication.

(4) Specifies the authentication token associated with your account. Required to access automation hub.

(5) Provides the URL for validated content from automation hub.

To retrieve the preceding URLs and generate a token, log in to the automation hub web UI. Go to **Automation Hub > Connect to Hub** and then click **Load token**.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

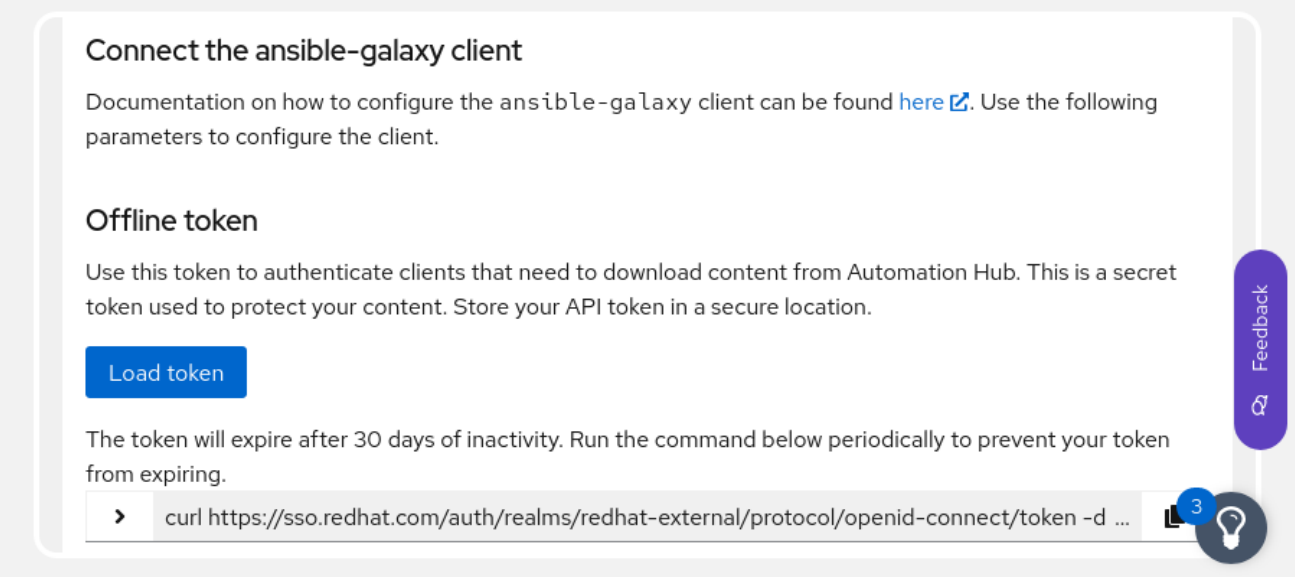


Figure 17. Generating a token

Instead of a token, you can use the `username` and `password` parameters to provide your customer portal username and password.

```
...output omitted...
[galaxy_server.hub_certified]
url=https://console.redhat.com/api/automation-hub/content/published/
username=operator1
password=Sup3r53cR3t
...output omitted...
```

You might not want to expose your credentials or your token in the `ansible.cfg` file because the file could potentially get committed using version control. In that case, remove the authentication parameters from the `ansible.cfg` file and define them as environment variables:

```
ANSIBLE_GALAXY_SERVER_SERVERID_KEY_=value
```

SERVERID

: Server identifier in uppercase. The server identifier is the name you use in the `server_list` parameter and in the name of the `[galaxy_server.serverid]` section.

KEY

: Key name in uppercase. For example, use TOKEN for the `token` key. The following example provides the `token` key as an environment variable:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[user@host ~]$ cat ansible.cfg
...output omitted...
[galaxy_server.hub_certified]
url=https://console.redhat.com/api/automation-hub/content/published/
auth_url=https://sso.redhat.com/auth/realms/redhat-external/protocol/openid-
connect/token
[user@host ~]$ export \
ANSIBLE_GALAXY_SERVER_HUB_CERTIFIED_TOKEN='eyJh...Jf0o'
[user@host ~]$ ansible-galaxy collection install redhat.insights
```

> **Note**
> If you develop and test playbooks on your local system, then you can define these ANSIBLE_GALAXY_SERVER_* variables in your ~/.bashrc file. If you do so, then protect the file because the variables contain your authentication credentials or your authentication token.

Installing Collections from Private Automation Hub

Configuring ansible-galaxy to get content from a private automation hub is similar to how it is configured to use automation hub. The following example includes a new section that configures ansible-galaxy to use a private automation hub installed on aap.example.com:

```
...output omitted...
[galaxy]
server_list = my_hub, automation_hub, galaxy

[galaxy_server.my_hub]
url=https://aap.example.com/api/galaxy/content/rh-certified/
token=e8e4...b0c2

[galaxy_server.automation_hub]
url=https://console.redhat.com/api/automation-hub/
auth_url=https://sso.redhat.com/auth/realms/redhat-external/protocol/openid-
connect/token

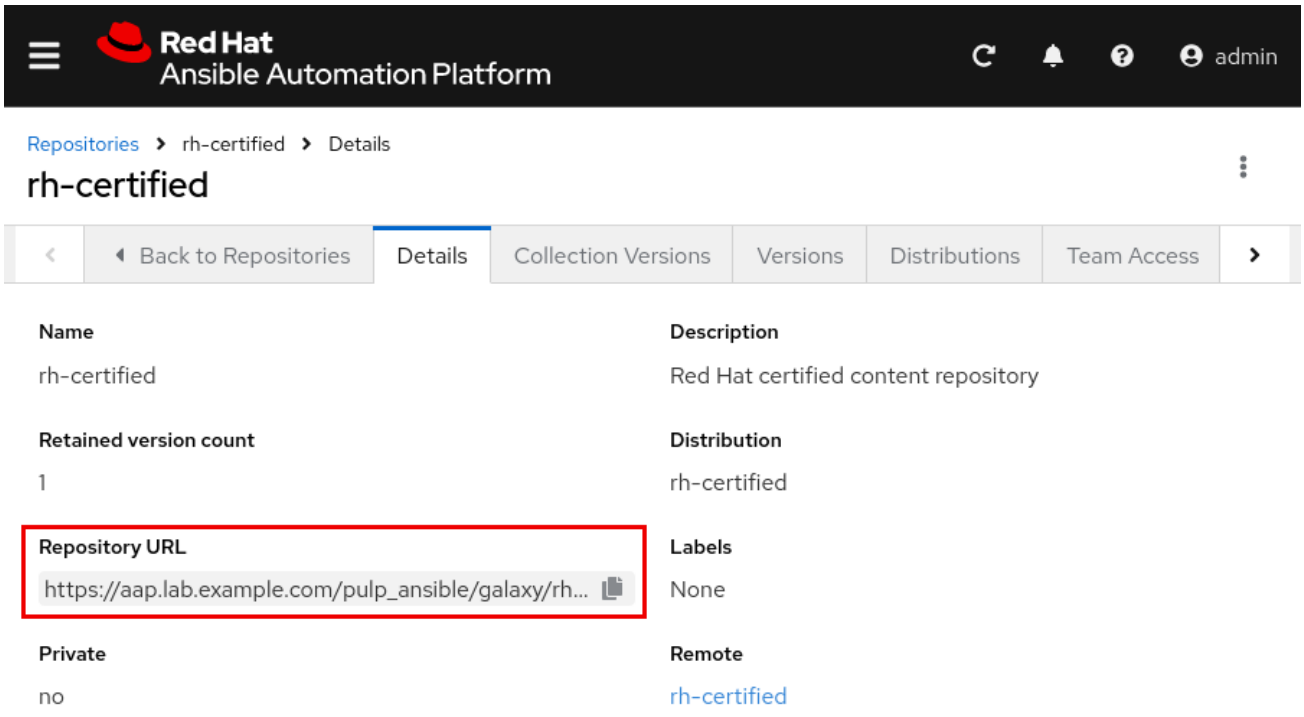
[galaxy_server.galaxy]
url=https://galaxy.ansible.com/
```

Notice that you do not need the auth_url directive for private automation hub.

Use the Ansible Automation Platform unified UI to retrieve the URL and generate the token.

Go to **Automation Content > Repositories** to list the collection repositories. If you click the link for a repository name, then the **Details** tab displays the repository URL.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use



Private automation hub typically contains at least the following repositories:

`community`
: The `community` repository uses the collections that you have synchronized from Ansible Galaxy.

`published`
: The `published` repository uses custom collections that you have installed and approved.

`rh-certified`
: The `rh-certified` repository uses the certified collections that you have synchronized from automation hub.

`validated`
: The `validated` repository uses the validated collections that you have synchronized from automation hub.

To generate a token, go to **Automation Content > API Token** and then click **Generate token**.

Installing and deploying private automation hub is beyond the scope of this course.

References
Using Ansible Collections - Ansible Documentation

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

2.4. Guided Exercise
Finding and Installing Ansible Content Collections

Configure `ansible-galaxy` and use it to install Ansible Content Collections.

Outcomes

- Configure various collection sources on a system to install collections.
- Install collections with the `ansible-galaxy collection` command.
- Install multiple collections using a `requirements.yml` file.

Prerequisites

As the `student` user on the `workstation` machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
[student@workstation ~]$ lab start manage-finding
```

Instructions

1. Go to `https://aap.lab.example.com` and log in to Ansible Automation Platform as `student` using `student` as the password.

2. Browse the Ansible Content Collections available from the unified UI. Access the details for one of the collections that Red Hat provides and supports, `ansible.netcommon`.

   2.1. Go to **Automation Content > Collections** and then search for the `ansible.netcommon` collection by entering `netcommon` in the search field. When the search completes, click the `netcommon` link.

   The **Details** tab provides information about the collection including the repository which contains the collection.

   2.2. Click the **Install** tab. Notice that the UI displays the command that you can use to install the collection. This tab also contains a link to download the collection archive file.

   2.3. Click the **Documentation** tab to access information about the modules and plug-ins available in this collection.

3. On the `workstation` machine, configure the Ansible project in the `/home/student/manage-finding` directory so that the `ansible-galaxy` command retrieves collections from the classroom's private automation hub. Enable access to the `rh-certified` and `community` repositories.

   3.1. Change to the `/home/student/manage-finding` directory:

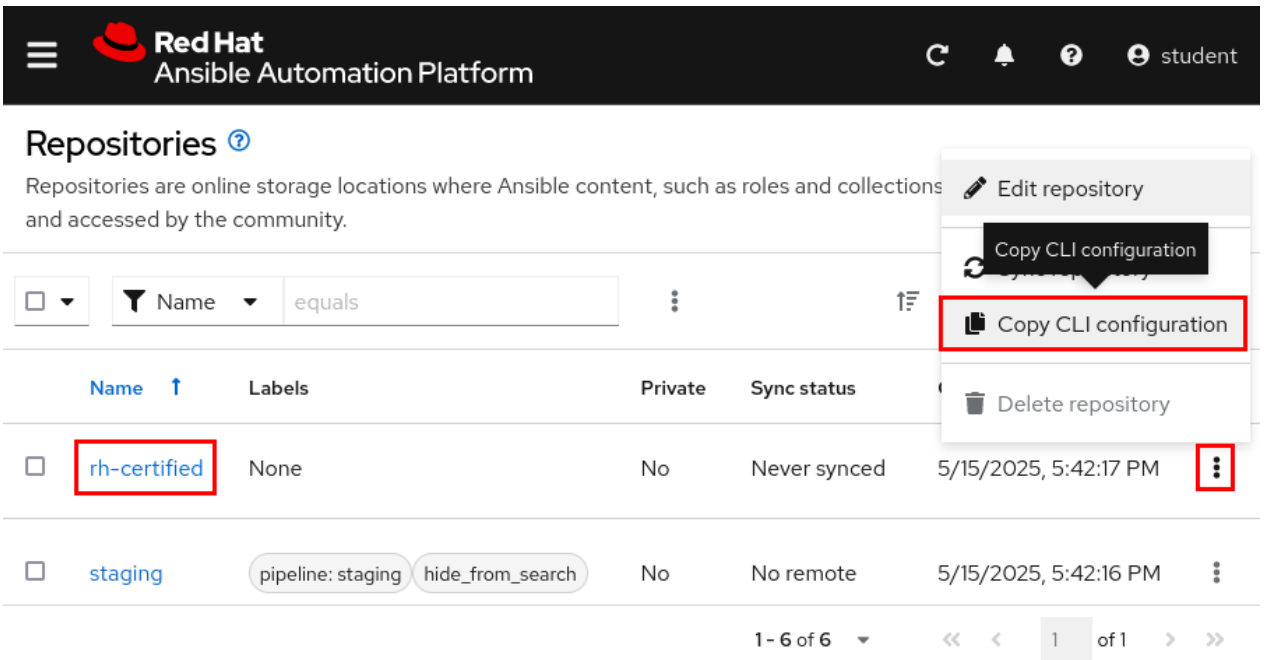
For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation ~]$ cd ~/manage-finding/
[student@workstation manage-finding]$
```

> **Important**
>
> You need to run the `ansible-galaxy` commands in this exercise from the `/home/student/manage-finding` directory, which contains the Ansible project. This helps ensure that you use the correct configuration file and install Ansible Content Collections to the correct location.

3.2. From the unified UI, go to **Automation Content > Repositories**. This page has the parameters that you need for configuring the `ansible.cfg` file.

From the `rh-certified` row, click the menu icon at the end of the row and select **Copy CLI configuration**.



Do not close your web browser window.

3.3. Modify the `ansible.cfg` file to append the lines that you copied to the clipboard in the preceding step. The modified `ansible.cfg` file now contains the following content:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[defaults]
inventory = inventory
remote_user = devops

[galaxy]
server_list = rh-certified

[galaxy_server.rh-certified]
url=http://localhost:5001/api/galaxy/content/rh-certified/
token=<put your token here>
```

3.4. Return to the unified UI. From the `community` row, click the menu icon at the end of the row and select **Copy CLI configuration**.

Do not close your web browser window.

3.5. Modify the `ansible.cfg` file to append the lines that you copied to the clipboard in the preceding step. The modified `ansible.cfg` file now contains the following content:

```
[defaults]
inventory = inventory
remote_user = devops

[galaxy]
server_list = rh-certified

[galaxy_server.rh-certified]
url=http://localhost:5001/api/galaxy/content/rh-certified/
token=<put your token here>

[galaxy]
server_list = community

[galaxy_server.community]
url=http://localhost:5001/api/galaxy/content/community/
token=<put your token here>
```

> **Important**
> You added a second `[galaxy]` section to the file, but the file must only contain one `[galaxy]` section. You correct that problem in the next step.

3.6. Update the `ansible.cfg` file so that it only contains one `[galaxy]` section. Update the first `[galaxy]` section so that it lists both repositories, `rh-certified` and `community`. Remove the second `[galaxy]` section. The modified `ansible.cfg` file now contains the following content:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[defaults]
inventory = inventory
remote_user = devops

[galaxy]
server_list = rh-certified, community

[galaxy_server.rh-certified]
url=http://localhost:5001/api/galaxy/content/rh-certified/
token=<put your token here>

[galaxy_server.community]
url=http://localhost:5001/api/galaxy/content/community/
token=<put your token here>
```

3.7. Update the `ansible.cfg` file so that both `url` lines point to the correct URL for the classroom private automation hub. Change `http://localhost:5001` to `https://aap.lab.example.com` in both locations. The modified `ansible.cfg` file now contains the following content:

```
[defaults]
inventory = inventory
remote_user = devops

[galaxy]
server_list = rh-certified, community

[galaxy_server.rh-certified]
url=https://aap.lab.example.com/api/galaxy/content/rh-certified/
token=<put your token here>

[galaxy_server.community]
url=https://aap.lab.example.com/api/galaxy/content/community/
token=<put your token here>
```

3.8. Return to the unified UI. Go to **Automation Content > API Token** and then click **Generate token**. Copy the API token.

3.9. Using the copied token, update both `token` lines in the `ansible.cfg` file. Your token is different from the one displayed in this example. Save and close the file when done:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[defaults]
inventory = inventory
remote_user = devops

[galaxy]
server_list = rh-certified, community

[galaxy_server.rh-certified]
url=https://aap.lab.example.com/api/galaxy/content/rh-certified/
token=24699d01b598f4599b66008dba994585122d4170

[galaxy_server.community]
url=https://aap.lab.example.com/api/galaxy/content/community/
token=24699d01b598f4599b66008dba994585122d4170
```

4. Create a `collections` directory in the project directory, and then install the `ansible.netcommon` collection into the new directory.

   4.1. Create the `collections` directory:

```
[student@workstation manage-finding]$ mkdir collections
```

   4.2. Use the `ansible-galaxy` command to install the `ansible.netcommon` collection into the `collections` directory:

```
[student@workstation manage-finding]$ ansible-galaxy collection install \
ansible.netcommon -p collections/
...output omitted...
ansible.netcommon:7.1.0 was installed successfully
...output omitted...
ansible.utils:5.1.2 was installed successfully
```

> **Note**
> You can safely ignore the warning message related to collections path. You solve that issue when you add the `collections_path` directive in the next step.

The preceding output indicates that the `ansible-galaxy` command downloaded the collection from `https://aap.lab.example.com`, the classroom private automation hub.

Notice that the `ansible.utils` collection is installed automatically as a dependency of the `ansible.netcommon` collection.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

4.3. Modify the `ansible.cfg` file so that it explicitly looks for collections in the project `collections` directory. The top of the modified file now contains the following content:

```
[defaults]
inventory = inventory
remote_user = devops
collections_path = ./collections:/usr/share/ansible/collections
...output omitted...
```

4.4. In the `/home/student/manage-finding` directory, use the `ansible-galaxy` command to list the collections installed on the control node that are specified by the current `collections_path` directive:

```
[student@workstation manage-finding]$ ansible-galaxy collection list

# /home/student/manage-finding/collections/ansible_collections
Collection               Version
------------------------ -------
ansible.netcommon        7.1.0
ansible.utils            5.1.2

# /usr/share/ansible/collections/ansible_collections
Collection               Version
------------------------ -------
redhat.rhel_system_roles 1.23.0
```

> **Note**
> Although other Ansible Content Collections might be installed on the system, the `ansible-galaxy` command only lists collections in the directories specified by the `collections_path` directive. The versions of the `ansible.netcommon`, `ansible.utils`, and `redhat.rhel_system_roles` collections installed on your system might be different from the versions displayed here.

5. Install version 3.0.1 of the `redhat.openshift` collection. Because you updated the project's `ansible.cfg` file to look first for collections in the `./collections` directory, you do not need to use the `-p` option.

```
[student@workstation manage-finding]$ ansible-galaxy collection install \
redhat.openshift:3.0.1
...output omitted...
redhat.openshift:3.0.1 was installed successfully
...output omitted...
kubernetes.core:3.0.1 was installed successfully
```

6. Install collections specified in a requirements file. Include the existing `ansible.netcommon` and `redhat.openshift` collections, and then add two additional collections, `community.crypto` and

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

`ansible.controller`.

6.1. Create the `collections/requirements.yml` file, and add the following content:

```
---
collections:
  - name: ansible.netcommon

  - name: redhat.openshift
    version: 3.0.1

  - name: community.crypto
    version: 2.24.0

  - name: ansible.controller
    version: 4.6.3
```

6.2. Install the collections by using the `ansible-galaxy` command. Add the `-r` option to specify the location of the requirements file.

The command skips installing any collection that is already installed.

```
[student@workstation manage-finding]$ ansible-galaxy collection install \
-r collections/requirements.yml
...output omitted...
```

6.3. List the collections installed on the control node:

```
[student@workstation manage-finding]$ ansible-galaxy collection list

# /home/student/manage-finding/collections/ansible_collections
Collection               Version
------------------------ -------
ansible.controller       4.6.3
ansible.netcommon        7.1.0
ansible.utils            5.1.2
community.crypto         2.24.0
kubernetes.core          3.0.1
redhat.openshift         3.0.1

# /usr/share/ansible/collections/ansible_collections
Collection               Version
------------------------ -------
redhat.rhel_system_roles 1.23.0
```

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Finish

On the `workstation` machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish manage-finding
```

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

2.5. Selecting an Automation Execution Environment

Objectives

- Identify the automation execution environments provided by Red Hat and select the correct one for your use case.

Automation Execution Environments

An *automation execution environment* is a container image that includes Ansible Content Collections, their software dependencies, and a minimal Ansible engine that can run your playbooks.

By using an automation execution environment, you can use the same portable environment to develop your Ansible Playbooks on one system and run them on another. This streamlines the development process and helps to ensure predictable, reproducible results.

The automation execution environment is where your Ansible Playbook runs. You normally use a tool such as `ansible-navigator` to run a playbook, but the playbook runs inside the container rather than directly on your system.

An automation execution environment consists of the following components:

- Ansible Core
- Ansible Content Collections to supplement Ansible Core
- Python and system packages that are required by Ansible Core or by the included collections
- Ansible Runner to run your playbooks

Automation execution environments are prepared by the execution environment builder (`ansible-builder`), and can be pushed to any container registry.

> **Note**
> A subsequent chapter covers how to create custom automation execution environments.

Selecting a Supported Automation Execution Environment

Red Hat Ansible Automation Platform 2.5 provides four prebuilt automation execution environments:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Short name	Container image	Description
Minimal RHEL 8	`ee-minimal-rhel8`	A minimal automation execution environment built on RHEL 8, based on Ansible Core 2.16, that only includes the `ansible.builtin` Ansible Content Collection.
Supported RHEL 8	`ee-supported-rhel8`	An automation execution environment built on RHEL 8, based on Ansible Core 2.16, that includes Red Hat Ansible Certified Content Collections and their software dependencies.
Minimal RHEL 9	`ee-minimal-rhel9`	A minimal automation execution environment built on RHEL 9, based on Ansible Core 2.16, that only includes the `ansible.builtin` Ansible Content Collection.
Supported RHEL 9	`ee-supported-rhel9`	An automation execution environment built on RHEL 9, based on Ansible Core 2.16, that includes Red Hat Ansible Certified Content Collections and their software dependencies.

The minimal automation execution environments only include the `ansible.builtin` Ansible Content Collection. You usually use them as a starting point for building custom automation execution environments.

The supported automation execution environments include some collections that Red Hat supports. Automation content navigator and automation controller use the supported automation execution environment by default. Use this automation execution environment with your new playbooks.

The automation execution environments based on RHEL 8 and RHEL 9 perform the same functionality, but the version of Python and OS packages required by your Ansible Content Collections as dependencies might require one version of RHEL over the other.

Red Hat hosts these four automation execution environments in its container registry at `registry.redhat.io`, under the `ansible-automation-platform-25` namespace:

- `registry.redhat.io/ansible-automation-platform-25/ee-minimal-rhel8:latest`
- `registry.redhat.io/ansible-automation-platform-25/ee-supported-rhel8:latest`
- `registry.redhat.io/ansible-automation-platform-25/ee-minimal-rhel9:latest`

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

- `registry.redhat.io/ansible-automation-platform-25/ee-supported-rhel9:latest`

You can browse the registry catalog at https://catalog.redhat.com/search?gs&q=ansible-automation-platform-25.

> **Note**
> There is another prebuilt automation execution environment named `ee-29-rhel8` that is used for compatibility with Ansible 2.9. Use this automation execution environment when you want to run legacy playbooks written for Ansible 2.9 (or Red Hat Ansible Automation Platform 1.2) that have not yet been migrated to Red Hat Ansible Automation Platform 2.
>
> Red Hat hosts this automation execution environment in its container registry at `registry.redhat.io`, under the `ansible-automation-platform` namespace.

Inspecting Automation Execution Environments

Use automation content navigator to list and inspect the automation execution environments available on your system. To do so, run the `ansible-navigator images` command:

```
[user@host ~]$ ansible-navigator images
  Image                Tag     Execution environment    Created         Size
0|ee-minimal-rhel8     latest  True                     4 months ago    341 MB
1|ee-supported-rhel8   latest  True                     4 months ago    2.24 GB


^f/PgUp page up ^b/PgDn page down ↑↓ scroll esc back [0-9] goto :help help
```

You can type the number of the automation execution environment that you want to inspect. For example, press `1` to review the details of the supported automation execution environment:

```
  Image: ee-supported-rhel8:latest Description
0|Image information               Information collected from image inspection
1|General information             OS and python version information
2|Ansible version and collections  Information about ansible and ansible collecti
3|Python packages                  Information about python and python packages
4|Operating system packages        Information about operating system packages
5|Everything                       All image information


^f/PgUp page up ^b/PgDn page down ↑↓ scroll esc back [0-9] goto :help help
```

You can view further details on a specific aspect of the automation execution environment by selecting the associated number. In the preceding example, if you press `2` (or type `:2` and then press `Enter`) you get the list of the available collections in that automation execution environment.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Using Automation Execution Environments with Automation Content Navigator

The `ansible-navigator run` command runs your playbooks in an automation execution environment.

This command uses the supported automation execution environment from `registry.redhat.io` by default, but you can use the `--eei` option to specify a different environment. The following example shows how to run a playbook using the minimal automation execution environment:

```
[user@host ~]$ ansible-navigator run playbook.yml \
--eei registry.redhat.io/ansible-automation-platform-25/ee-minimal-rhel8
```

If the container image is not already available on your system, `ansible-navigator` tries to pull it from the container registry. Use the `podman login` command to ensure that you are authenticated to the registry. Provide your Customer Portal credentials to authenticate to the registry:

```
[user@host ~]$ podman login registry.redhat.io
...output omitted..
```

You can control how `ansible-navigator` pulls container images by using the `--pull-policy` (or `--pp`) option with one of the following arguments:

Option	Description
`always`	Always pulls the image.
`missing`	Only pulls the image if it is not already available locally.
`never`	Never pulls the image.
`tag`	Pulls the image if the image tag is `latest` or if it is not already available locally (the default).

> **Note**
> Instead of using the `--eei` option, you can create an `ansible-navigator.yml` configuration file to define the automation execution environment to use by default. A later chapter describes that configuration file in more detail.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

References

Automation Execution Environments

Creating and Using Execution Environments

Getting Started With Execution Environments

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

2.6. Guided Exercise
Selecting an Automation Execution Environment

Ensure that the automation execution environments you need are installed on your development workstation and use them to run playbooks.

Outcomes

- Identify the automation execution environment that includes a specific Ansible Content Collection.
- Run a playbook using that automation execution environment.

Prerequisites

As the `student` user on the `workstation` machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
[student@workstation ~]$ lab start manage-selecting
```

Instructions

1. Identify the Ansible Content Collections required to run the prebuilt playbook.

   1.1. Change to the `/home/student/manage-selecting/` directory:

   ```
   [student@workstation ~]$ cd ~/manage-selecting
   [student@workstation manage-selecting]$
   ```

   1.2. Review the `acl_info.yml` playbook. Notice that the playbook requires the following modules to run:

   - `ansible.posix.acl`
   - `ansible.builtin.debug`

   ```
   ...output omitted...
     tasks:
       - name: Obtain the ACLs for /webserver_data directory
         ansible.posix.acl:
           path: /webserver_data/
         register: acl_info

       - name: Print the ACLs obtained for /webserver_data directory
         ansible.builtin.debug:
           var: acl_info['acl']
   ```

   Whereas the `ansible.builtin.debug` module is part of the `ansible.builtin` collection that is included in the `ansible-core` RPM package, the `ansible.posix.acl` module is part of the `ansible.posix` Ansible Content Collection.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

2. Find the automation execution environment that includes the `ansible.posix` Ansible Content Collection.

2.1. Run the `podman login` command to log in to the container registry at `aap.lab.example.com`. Use `admin` as the username and `redhat` as the password:

```
[student@workstation manage-selecting]$ podman login aap.lab.example.com
Username: admin
Password: redhat
Login Succeeded!
```

This step simulates logging in to `registry.redhat.io`, but uses the private automation hub in the classroom environment. That private automation hub includes the `ee-minimal-rhel8` and `ee-supported-rhel8` automation execution environments provided by Red Hat Ansible Automation Platform 2.

2.2. Use the `ansible-navigator images` command to download and inspect the `aap.lab.example.com/ee-minimal-rhel8` automation execution environment:

```
[student@workstation manage-selecting]$ ansible-navigator images \
--eei aap.lab.example.com/ee-minimal-rhel8
...output omitted...
```

2.3. Automation content navigator displays output that lists the `ee-minimal-rhel8` automation execution environment:

```
  Image              Tag     Execution environment  Created      Size
0|ee-minimal-rhel8   latest  True                   4 months ago 341 MB


^b/PgUp page   up ^f/PgDn page down   ↑↓ scroll   esc back   [0-9] go...
```

2.4. Type `0` to inspect this automation execution environment, and then type 2 to select the `Ansible version and collections` option:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
  Image: ee-minimal-rhel8:latest (primary) Description
0|Image information                       Information collected from...
1|General information                     OS and python version info...
2|Ansible version and collections         Information about ansible ...
3|Python packages                         Information about python a...
4|Operating system packages               Information about operatin...
5|Everything                              All image information


^b/PgUp page   up ^f/PgDn page down   ↑↓ scroll   esc back   [0-9] go...
```

2.5. After a few seconds, the output indicates that the minimal automation execution environment does not contain any additional Ansible Content Collections.

```
Image: ee-minimal-rhel8:latest (primary) (Information about ansible and
ansible collections)
0|---
1|ansible:
2|  collections:
3|    details: {}
4|  version:
5|    details: ansible [core 2.16.14]


^b/PgUp page   up ^f/PgDn page down    ↑↓ scroll   esc back   :help help
```

Type `:q` and then press `Enter` to exit the `ansible-navigator` command.

The `ee-minimal-rhel8` automation execution environment is not useful for running the `acl_info.yml` playbook.

2.6. Use the `ansible-navigator images` command to download and inspect the `aap.lab.example.com/ee-supported-rhel8` automation execution environment:

```
[student@workstation manage-selecting]$ ansible-navigator images \
--eei aap.lab.example.com/ee-supported-rhel8
...output omitted...
```

2.7. Automation content navigator displays the `ee-supported-rhel8` automation execution environment in the second row of the image list. Type 1 to inspect that automation execution environment:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
  Image              Tag     Execution environment  Created      Size
0|ee-minimal-rhel8   latest  True                   4 months ago 341 MB
1|ee-supported-rhel8 latest  True                   4 months ago 2.24 GB


^b/PgUp page   up ^f/PgDn page down   ↑↓ scroll   esc back   [0-9] go...
```

2.8. Type 2 to select the `Ansible version and collections` option.

> **Note**
> Wait for automation content navigator to collect the image details. This action can take up to five minutes to complete.

```
            Working on it...
            ──────────────────────────────────────────────────
            Collecting image details, this may take a minute...
            ──────────────────────────────────────────────────
```

2.9. The resulting output indicates that the `ee-supported-rhel8` automation execution environment contains the `ansible.posix` Ansible Content Collection:

```
Image: ee-supported-rhel8:latest (primary) (Information about ansible and
...
 0|---
 1|ansible:
 2| collections:
 3|   details:
 4|     amazon.aws: 7.4.0
 5|     ansible.controller: 4.6.3
 6|     ansible.eda: 2.3.0
 7|     ansible.hub: 1.0.0
 8|     ansible.netcommon: 7.1.0
 9|     ansible.network: 4.0.0
10|     ansible.platform: 2.5.20241218
11|     ansible.posix: 1.5.4
12|     ansible.scm: 3.0.0
...output omitted...
```

Type `:q` and then press `Enter` to exit the `ansible-navigator` command.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

The `ee-supported-rhel8` automation execution environment contains the `ansible.posix` Ansible Content Collection that you need. You can run this playbook using this automation execution environment.

3. Run the `acl_info.yml` playbook.

   3.1. Demonstrate that the `aap.lab.example.com/ee-minimal-rhel8` automation execution environment does not work with the `acl_info.yml` playbook because it does not include the `ansible.posix` collection.

   Use the `ansible-navigator` command to run the `acl_info.yml` playbook. As expected, the playbook cannot resolve the `ansible.posix.acl` module, causing the playbook to fail.

```
[student@workstation manage-selecting]$ ansible-navigator run acl_info.yml \
-m stdout --eei aap.lab.example.com/ee-minimal-rhel8 --pp missing
ERROR! couldn't resolve module/action 'ansible.posix.acl'. This often
indicates a misspelling, missing collection, or incorrect module path.

The error appears to be in '/home/student/manage-selecting/acl_info.yml':
line 7, column 7, but may be elsewhere in the file depending on the exact
syntax problem.

The offending line appears to be:

  tasks:
    - name: Obtain the ACLs for /webserver_data directory
      ^ here
Please review the log for errors.
```

   3.2. Use the `ansible-navigator` command to run the `acl_info.yml` playbook using the `aap.lab.example.com/ee-supported-rhel8` automation execution environment. This automation execution environment contains the `ansible.posix` collection and the command succeeds.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation manage-selecting]$ ansible-navigator run acl_info.yml \
-m stdout --eei aap.lab.example.com/ee-supported-rhel8 --pp missing

PLAY [Retrieving ACLs using the ansible.posix collection]
**********************

TASK [Obtain the ACLs for /webserver_data directory]
***************************
ok: [serverc.lab.example.com]
ok: [serverb.lab.example.com]

TASK [Print the ACLs obtained for /webserver_data directory]
*******************
ok: [serverc.lab.example.com] => {
    "acl_info['acl']": [
        "user::rwx",
        "group::r-x",
        "other::r-x",
        "default:user::rwx",
        "default:group::r-x",
        "default:mask::rwx",
        "default:other::r-x"
    ]
}
ok: [serverb.lab.example.com] => {
    "acl_info['acl']": [
        "user::rwx",
        "group::r-x",
        "other::r-x",
        "default:user::rwx",
        "default:group::r-x",
        "default:mask::rwx",
        "default:other::r-x"
    ]
}

PLAY RECAP
*********************************************************************
serverb.lab.example.com    : ok=2    changed=0    unreachable=0    failed=0
...
serverc.lab.example.com    : ok=2    changed=0    unreachable=0    failed=0
...
```

Finish

On the `workstation` machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation ~]$ lab finish manage-selecting
```

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

2.7. Lab
Managing Ansible Content Collections and Automation Execution Environments

Run a playbook that needs an Ansible Content Collection in a particular automation execution environment, and another playbook that needs an Ansible Content Collection that is not provided by an automation execution environment.

Outcomes

- Identify Ansible Content Collections inside automation execution environment images.
- Run a playbook that requires a collection from an automation execution environment.
- Install a collection from automation hub.
- Run a playbook that requires a collection from automation hub.

Prerequisites

As the `student` user on the `workstation` machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
[student@workstation ~]$ lab start manage-review
```

Instructions

1. Change to the `/home/student/manage-review` directory and review the `cockpit.yml` playbook. Make note of the Ansible Content Collections that the playbook requires.

2. Use the `ansible-navigator` command to inspect the automation execution environment images available on your local system. Determine which one you need to run the `cockpit.yml` playbook.

3. Run the `cockpit.yml` playbook by using the `ansible-navigator` command with the correct automation execution environment image.

4. Access the web console at `https://servera.lab.example.com:9090` to confirm that the playbook correctly configured the web console. Accept the insecure TLS certificate and then log in as the `student` user. Use `student` as the password.

5. Review the `create_samples_db.yml` playbook. Make note of the collections that it requires. Find those collections in private automation hub.

   You can access private automation hub in the unified UI at `https://aap.lab.example.com`. Use `student` as the username and `student` as the password.

6. Configure the `/home/student/manage-review/ansible.cfg` file so that the `ansible-galaxy` command can install collections from the private automation hub `community` repository.

7. Create the `/home/student/manage-review/collections` directory. In this directory, install any collections that the `create_samples_db.yml` playbook requires.

8. Use the `ansible-navigator` command to run the `create_samples_db.yml` playbook.

9. Confirm that the playbook correctly configured a MariaDB database on the `serverb` machine. You can confirm that the playbook created the `samples` database by running the `sudo mysql -e "SHOW DATABASES"` command. The password for the `student` user is `student`.

Evaluation

As the `student` user on the `workstation` machine, use the `lab` command to grade your work. Correct any reported failures and rerun the command until successful.

```
[student@workstation ~]$ lab grade manage-review
```

Finish

As the `student` user on the `workstation` machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish manage-review
```

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Solution

Run a playbook that needs an Ansible Content Collection in a particular automation execution environment, and another playbook that needs an Ansible Content Collection that is not provided by an automation execution environment.

Outcomes

- Identify Ansible Content Collections inside automation execution environment images.
- Run a playbook that requires a collection from an automation execution environment.
- Install a collection from automation hub.
- Run a playbook that requires a collection from automation hub.

Prerequisites

As the `student` user on the `workstation` machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
[student@workstation ~]$ lab start manage-review
```

Instructions

1. Change to the `/home/student/manage-review` directory and review the `cockpit.yml` playbook. Make note of the Ansible Content Collections that the playbook requires.

   1.1. Change to the `/home/student/manage-review` directory:

   ```
   [student@workstation ~]$ cd ~/manage-review/
   [student@workstation manage-review]$
   ```

   1.2. Review the `cockpit.yml` playbook:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
---
- name: Deploy cockpit
  hosts: servera.lab.example.com
  become: true

  tasks:
    - name: Ensure cockpit is installed
      ansible.builtin.dnf:
        name: cockpit
        state: present

    - name: Ensure the cockpit service is started and enabled
      ansible.builtin.service:
        name: cockpit
        state: started
        enabled: true

    - name: Ensure cockpit network traffic is allowed
      ansible.posix.firewalld:
        service: cockpit
        permanent: true
        state: enabled
        immediate: true
```

Notice that the `ansible.posix.firewalld` module requires the `ansible.posix` collection.

2. Use the `ansible-navigator` command to inspect the automation execution environment images available on your local system. Determine which one you need to run the `cockpit.yml` playbook.

   2.1. List the available automation execution environment images:

```
[student@workstation manage-review]$ ansible-navigator images
  Image                Tag     Execution environment  Created        Size
0|ee-minimal-rhel8     latest  True                   4 months ago   341 MB
1|ee-supported-rhel8   latest  True                   4 months ago   2.24 GB
```

   Type 1 to select the `ee-supported-rhel8` image.

   2.2. The text-based interface (TUI) displays information about the `ee-supported-rhel8` image.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
  Image: ee-supported-rhel8:latest  Description
0|Image information                 Information collected from image
inspection
1|General information               OS and python version information
2|Ansible version and collections   Information about ansible and ansible
...
3|Python packages                   Information about python and python
packages
4|Operating system packages         Information about operating system
packages
5|Everything                        All image information
```

Type 2 to view the collections available in the `ee-supported-rhel8` image.

2.3. Notice that the `ee-supported-rhel8` image provides the `ansible.posix` collection that you need to run the `cockpit.yml` playbook.

```
Image: ee-supported-rhel8:latest (Information about ansible and ansible
collections)
 0|---
 1|ansible:
 2|  collections:
 3|    details:
 4|      amazon.aws: 7.4.0
 5|      ansible.controller: 4.6.3
 6|      ansible.eda: 2.3.0
 7|      ansible.hub: 1.0.0
 8|      ansible.netcommon: 7.1.0
 9|      ansible.network: 4.0.0
10|      ansible.platform: 2.5.20241218
11|      ansible.posix: 1.5.4
12|      ansible.scm: 3.0.0
13|      ansible.security: 3.0.0
...output omitted...
```

> **Note**
> It might take a few minutes to display the list of collections.

Type `:q` and then press `Enter` to exit the `ansible-navigator` command.

3. Run the `cockpit.yml` playbook by using the `ansible-navigator` command with the correct automation execution environment image.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

3.1. Use the `ansible-navigator run` command with the `ee-supported-rhel8` image to run the `cockpit.yml` playbook:

```
[student@workstation manage-review]$ ansible-navigator run \
--eei aap.lab.example.com/ee-supported-rhel8 cockpit.yml
...output omitted...
```

Press `Esc` to exit the `ansible-navigator` command after the playbook run is complete.

4. Access the web console at `https://servera.lab.example.com:9090` to confirm that the playbook correctly configured the web console. Accept the insecure TLS certificate and then log in as the `student` user. Use `student` as the password.

4.1. Open a web browser on the `workstation` machine and go to `https://servera.lab.example.com:9090`.

4.2. Log in with `student` as the username and `student` as the password. The playbook correctly configured the web console if the login process is successful. Log out of the web console.

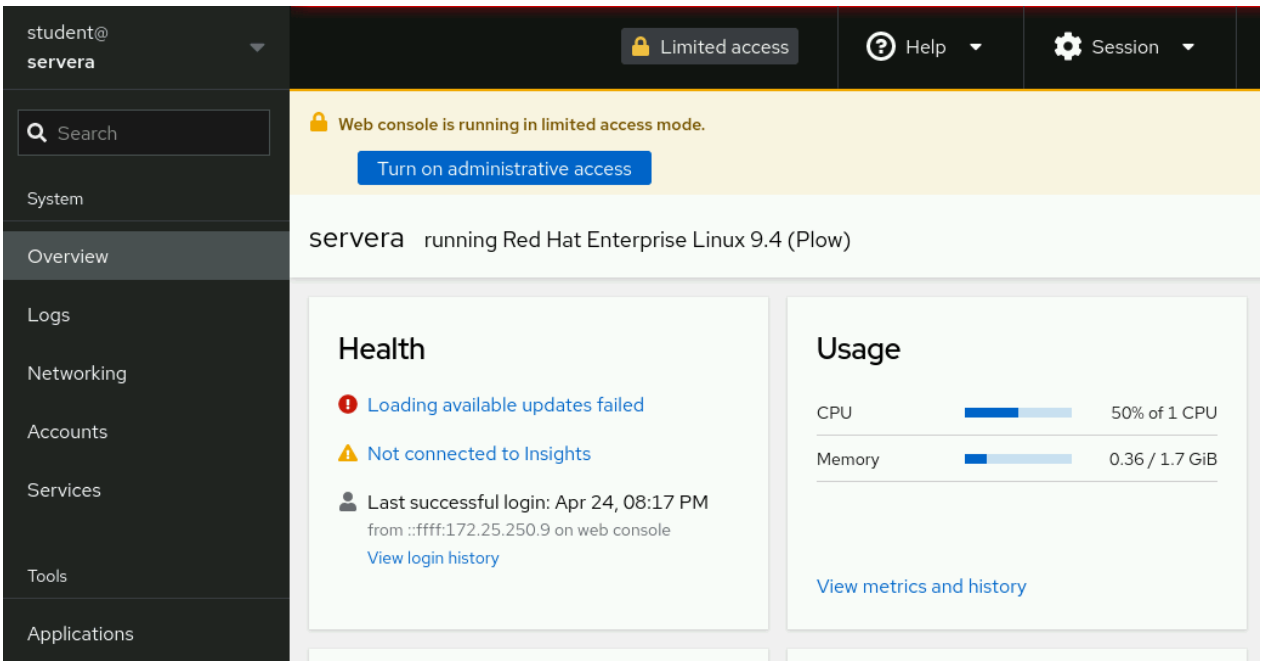


Figure 21. Accessing the Overview page

5. Review the `create_samples_db.yml` playbook. Make note of the collections that it requires. Find those collections in private automation hub.

You can access private automation hub in the unified UI at `https://aap.lab.example.com`. Use `student` as the username and `student` as the password.

5.1. Review the `create_samples_db.yml` playbook:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
---
- name: Create samples database
  hosts: serverb.lab.example.com
  become: true

  tasks:
    - name: Ensure MariaDB is installed
      ansible.builtin.dnf:
        name:
          - mariadb
          - mariadb-server
        state: present

    - name: Ensure the MariaDB service is started and enabled
      ansible.builtin.service:
        name: mariadb
        state: started
        enabled: true

    - name: Ensure the samples database exists
      community.mysql.mysql_db:
        name: samples
        state: present
```

Notice that the playbook requires the `community.mysql` collection.

5.2. Go to `https://aap.lab.example.com` and log in to Ansible Automation Platform as `student` using `student` as the password.

Go to **Automation Content > Collections** and search for `mysql`.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

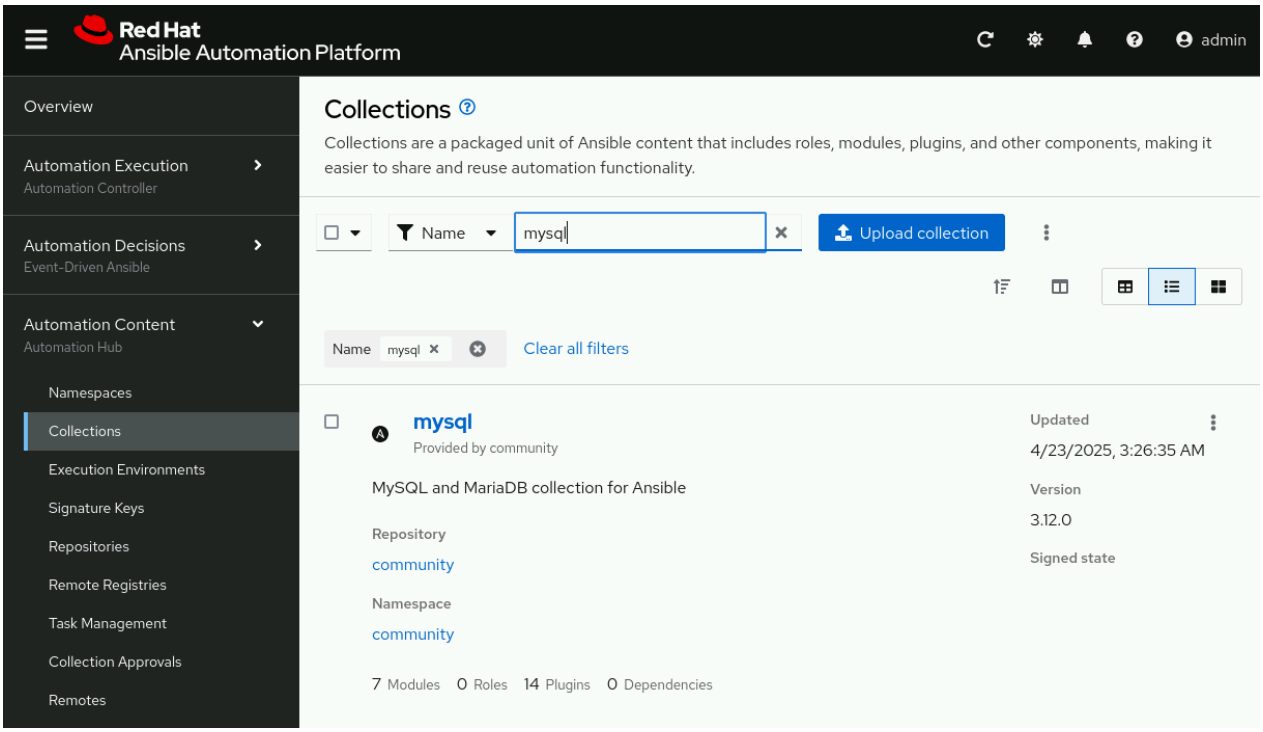


Figure 22. Searching collections

5.3. Click the `mysql` collection and then click the **Install** tab to obtain the `ansible-galaxy` installation command.

6. Configure the `/home/student/manage-review/ansible.cfg` file so that the `ansible-galaxy` command can install collections from the private automation hub `community` repository.

6.1. From the unified UI, go to **Automation Content > Repositories**. Click the menu icon at the end of the row for the `community` repository and select **Copy CLI configuration**.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

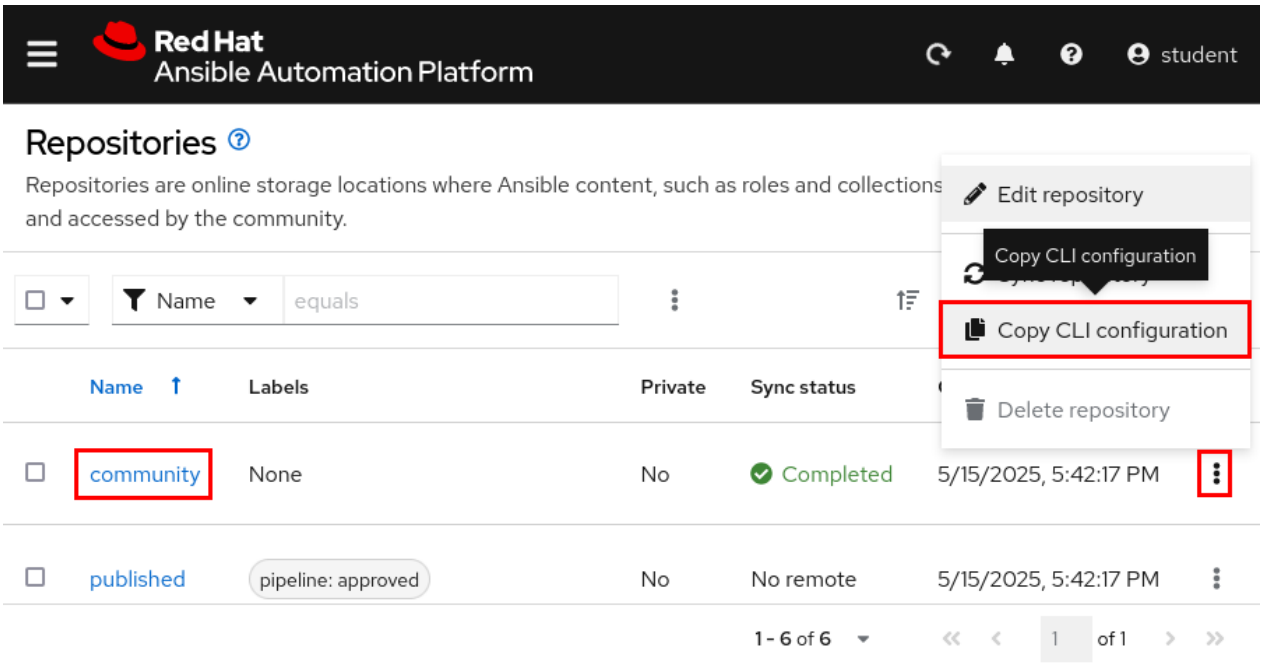


Figure 23. Listing the private automation hub repositories

6.2. Add the repository configurations to the `/home/student/manage-review/ansible.cfg` file:

```
[defaults]
inventory = ./inventory
remote_user = devops

[galaxy]
server_list = community

[galaxy_server.community]
url=http://localhost:5001/api/galaxy/content/community/
token=<put your token here>
```

6.3. Update the `ansible.cfg` file so that the `url` line points to the correct URL for the classroom private automation hub:

```
...output omitted...

[galaxy_server.community]
url=https://aap.lab.example.com/api/galaxy/content/community/
token=<put your token here>
```

6.4. From the unified UI, go to **Automation Content > API Token** and then click **Generate token**.

6.5. Edit the `/home/student/manage-review/ansible.cfg` file and then paste the token from the clipboard as the value for the token parameters. Your token is different from the one displayed in the following example.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[defaults]
inventory = ./inventory
remote_user = devops

[galaxy]
server_list = community

[galaxy_server.community]
url=https://aap.lab.example.com/api/galaxy/content/community/
token=c7087504efef551cf1173093988d89604f899490
```

7. Create the `/home/student/manage-review/collections` directory. In this directory, install any collections that the `create_samples_db.yml` playbook requires.

   7.1. Create the `/home/student/manage-review/collections` directory:

```
[student@workstation manage-review]$ mkdir collections
```

   7.2. Install the `community.mysql` collection into the `/home/student/manage-review/collections` directory:

```
[student@workstation manage-review]$ ansible-galaxy collection install \
community.mysql -p collections/
...output omitted...
```

> **Note**
> You can safely ignore the warning message.

   7.3. List the installed collections in the `/home/student/manage-review/collections` directory:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation manage-review]$ ansible-galaxy collection list \
-p collections/

# /home/student/manage-review/collections/ansible_collections
Collection               Version
------------------------ -------
community.mysql          3.12.0

# /usr/share/ansible/collections/ansible_collections
Collection               Version
------------------------ -------
redhat.rhel_system_roles 1.23.0
```

8. Use the `ansible-navigator` command to run the `create_samples_db.yml` playbook.

   8.1. Use the `ansible-navigator run` command to run the playbook:

```
[student@workstation manage-review]$ ansible-navigator run
create_samples_db.yml
...output omitted...
```

   Press Esc to exit the `ansible-navigator` command after the playbook run is complete.

9. Confirm that the playbook correctly configured a MariaDB database on the `serverb` machine. You can confirm that the playbook created the `samples` database by running the `sudo mysql -e "SHOW DATABASES"` command. The password for the `student` user is `student`.

   9.1. Connect to `serverb` as the `student` user:

```
[student@workstation manage-review]$ ssh serverb
...output omitted...
[student@serverb ~]$
```

   9.2. Confirm that the `samples` database exists:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@serverb ~]$ sudo mysql -e "SHOW DATABASES"
[sudo] password for student: student
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| samples            |
+--------------------+
```

9.3. Log out of the `serverb` machine:

```
[student@serverb ~]$ logout
Connection to serverb closed.
[student@workstation manage-review]$
```

Evaluation

As the `student` user on the `workstation` machine, use the `lab` command to grade your work. Correct any reported failures and rerun the command until successful.

```
[student@workstation ~]$ lab grade manage-review
```

Finish

As the `student` user on the `workstation` machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish manage-review
```

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

2.8. Summary

- You can use the `ansible-navigator doc --mode stdout` command to access the module documentation provided by Ansible Content Collections that are available to your automation execution environment.
- You can use the `ansible-navigator images` command to inspect automation execution environments and to list the collections and other resources and collection dependencies provided by an automation execution environment's container image.
- Ansible Playbooks should use fully qualified collection names (FQCNs) to refer to modules, roles, and plug-ins provided by Ansible Content Collections.
- The `ansible-galaxy collection install` command installs Ansible Content Collections on the local system. The `--collections-path` (or `-p`) option specifies the installation directory.
- You can use the `collections/requirements.yml` file to list the Ansible Content Collections that are required for the project.
- Automation execution environments can access the Ansible Content Collections that are installed in the `collections/` subdirectory in the directory that contains the playbook.
- The `ee-supported-rhel8` automation execution environment is used by default by automation content navigator and automation controller, and includes selected Red Hat Certified Ansible Content Collections in addition to the `ansible.builtin` collection.
- The `ee-minimal-rhel8` automation execution environment only provides the `ansible.builtin` Ansible Content Collection, but you can also use Ansible Content Collections from your project's `collections/` directory.
- The `ee-supported-rhel9` and `ee-minimal-rhel9` automation execution environments can be used if your collections have dependencies for more recent OS package versions.
- You can use the `ee-29-rhel8` automation execution environment for legacy playbooks that require Ansible 2.9.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Chapter 3.
Running Playbooks with Automation Controller

Goal

Explain what automation controller is and demonstrate how to use it to run playbooks that you developed with automation content navigator.

Sections

- Explaining the Automation Controller Architecture (and Quiz)
- Creating Automation Controller Resources (and Guided Exercise)

Lab

- Running Playbooks with Automation Controller

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

3.1. Explaining the Automation Controller Architecture

Objectives

- Describe the architecture and use cases of the automation controller component of Red Hat Ansible Automation Platform.

Automation Controller Architecture

Red Hat Ansible Automation Platform 2.5 includes several components, one of which is called automation controller. This component used to be known as Red Hat Ansible Tower in some earlier versions of Ansible Automation Platform.

The introduction of automation execution environments in Ansible Automation Platform 2 decoupled the automation controller control plane from its execution environment. Nodes in the control plane manage and delegate automation jobs to nodes in the execution plane.

Red Hat Ansible Tower 3.8 and earlier tightly coupled the execution environment to the system that ran Ansible Tower. Sometimes, this tight coupling required you to manage the dependencies of various modules needed to run Ansible Playbooks. If two playbooks required different environments, then you needed to create one or more Python virtual environments on the Ansible Tower system to manage the different dependencies and requirements. Some enterprises ended up with tens or hundreds of Python virtual environments.

Automation controller improves this architecture significantly. Instead of using the system executables and Python installations or virtual environments, automation controller uses automation execution environments. These environments are container images that you can pull from a central container registry, install on automation controller, and manage through the unified UI.

If needed, then you can create custom automation execution environments. After confirming that a custom automation execution environment works with your automation code, you can publish the container image to a container registry and then make the new or updated automation execution environment available to automation controller.

Using the same automation execution environment helps ensure that automation code runs consistently on both your system and in automation controller.

Automation Mesh

Red Hat Ansible Automation Platform 2 separates the *control plane*, which starts automation jobs, from the *execution plane*, which runs automation jobs. This separation enables you to run Ansible automation on a large scale and with improved reliability, redundancy, and efficiency.

You can have multiple control nodes that schedule jobs cooperatively on execution nodes, spreading out the load across multiple systems. You can also add execution nodes to scale up your capacity to run jobs.

The control nodes communicate with the execution nodes by using automation mesh.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

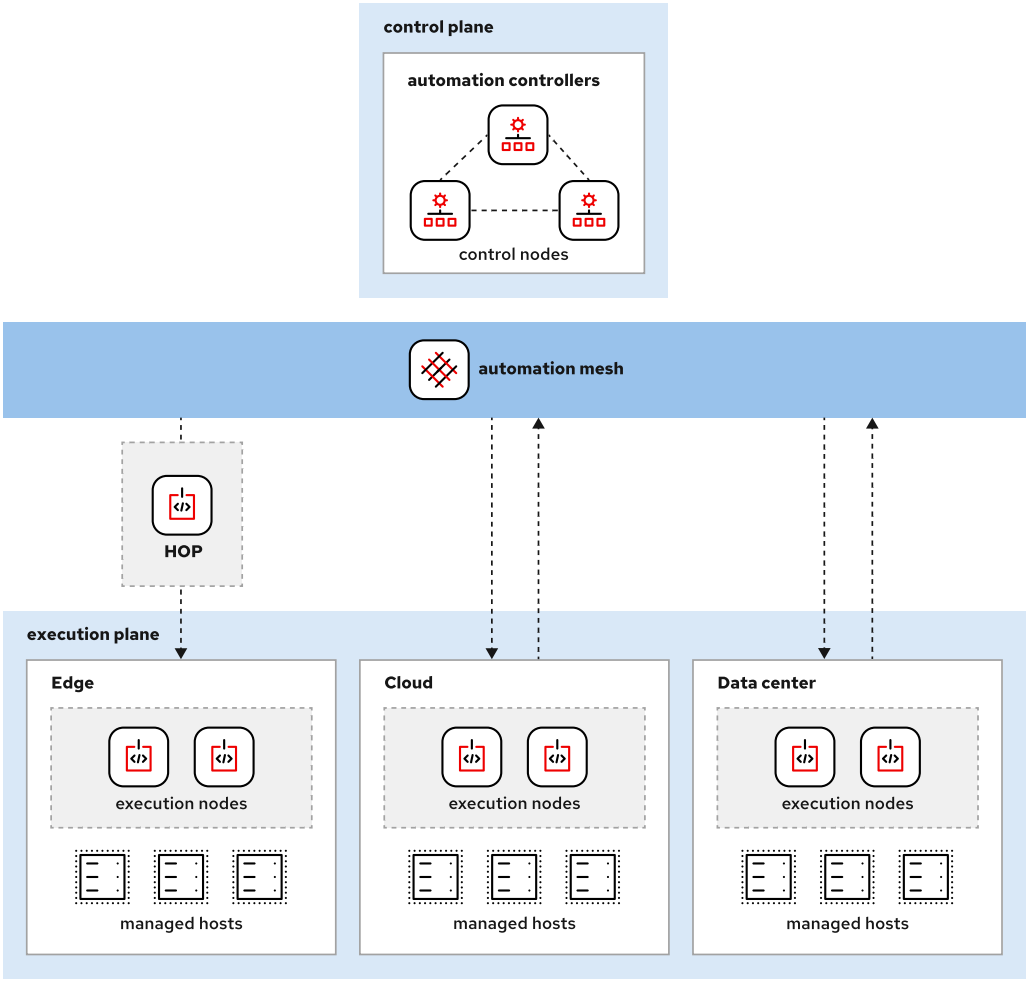


Figure 24. Execution and control plane

Automation Controller Features

Automation controller offers many features for controlling, securing, and managing Ansible in an enterprise environment, such as:

Visual dashboard

The unified UI displays a dashboard that provides a summary of an enterprise's entire Ansible environment. You can use the visual dashboard to review the current status of hosts and inventories, and the results of recent job executions.

Role-based Access Control (RBAC)

Ansible Automation Platform uses a role-based access control (RBAC) system, which maintains security and streamlines user access management. Organizations, teams, and users are shared across the platform and you can assign roles to provide access to automation controller resources, such as templates, projects, and inventories.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Graphical inventory management

You can use the unified UI to create inventories and then add host groups and hosts to the inventories. Alternatively, you can update inventories from an external inventory source such as public cloud providers, local virtualization environments, an organization's custom configuration management database (CMDB), or content in a Git repository.

Task manager and job scheduling

Use automation controller to schedule playbook execution and updates from external data sources either on a one-time or recurring basis. This means that routine tasks can run unattended, and this is especially useful for tasks such as backup routines, which are ideally executed during operational off-hours.

Real-time and historical job status reporting

When you initiate a playbook run in automation controller, the unified UI displays the playbook output and execution results in real time. The results of previously executed jobs and scheduled job runs are also available from automation controller for auditing or review.

User-triggered automation

Users with appropriate permissions can launch job templates or workflow job templates with a single click of an icon.

Credential management

Automation controller centrally manages authentication credentials. This means that you can run Ansible plays on managed hosts, synchronize information from dynamic inventory sources, and import Ansible project content from version control systems.

Automation controller encrypts the passwords or keys provided so that automation controller users cannot retrieve them. Even if a user has privileges to use or edit a credential, automation controller does not expose the current values of the credential passwords or keys.

Centralized logging and auditing

Automation controller logs all playbook and remote command execution. This provides the ability to audit when each job was executed and by whom. In addition, automation controller can integrate its log data into third-party logging aggregation solutions, such as Splunk and Sumologic.

Integrated notifications

Automation controller can notify you when job executions start and whether the job succeeds or fails. Automation controller can deliver notifications to many applications, including email, Grafana, IRC, Mattermost, Slack, Rocket.Chat, Twilio, and Webhooks.

Multiplaybook workflows

Complex operations often involve the serial execution of multiple playbooks. Automation controller multiplaybook workflows can chain together many playbooks to ease the implementation of complex routines involving provisioning, configuration, deployment, and orchestration. An intuitive workflow editor also helps model multiplaybook workflows.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Browsable RESTful API

The automation controller's RESTful API exposes every automation controller feature available through a web browser. The browsable format of the API makes it self-documenting and lets you look up information on how to use the API.

References

Using Automation Execution

Red Hat Ansible Automation Platform 2.5 Documentation

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

3.2. Quiz
Explaining the Automation Controller Architecture

Choose the correct answers to the following questions:

1. What is the primary role of automation controller in Red Hat Ansible Automation Platform 2.5?
   - [ ] **a.** Managing user desktops across an enterprise
   - [ ] **b.** Serving as the central control plane for orchestrating automation workflows
   - [ ] **c.** Providing access to Red Hat training materials
   - [ ] **d.** Executing Ansible Playbooks without state tracking
2. Which statement about automation controller is accurate?
   - [ ] **a.** Automation controller is not included with Red Hat Ansible Automation Platform 2.
   - [ ] **b.** Automation controller is a modular application that has decoupled the control plane from the execution plane by using containerization.
   - [ ] **c.** Automation controller is a command-line tool that you use to manage Ansible Playbooks in Git repositories.
   - [ ] **d.** Automation controller is a cloud-based application accessed through `console.redhat.com`.
3. Which feature is not part of automation controller?
   - [ ] **a.** A wizard for creating playbooks
   - [ ] **b.** Logs that record the results and output of playbook execution
   - [ ] **c.** Role-based access control (RBAC)
   - [ ] **d.** A RESTful API
4. Which statement is true about how automation controller manages credentials?
   - [ ] **a.** Users who can use credentials stored in automation controller can read the secrets stored in them.
   - [ ] **b.** Credentials cannot be used to authenticate to a version control system to import Ansible project content.
   - [ ] **c.** Automation controller encrypts the secrets stored in credentials so that automation controller users can use them but not read them.
   - [ ] **d.** Automation controller cannot manage credentials.
5. Which statement about improvements to the automation controller architecture is true?
   - [ ] **a.** You can create playbooks in the web interface.
   - [ ] **b.** You can use Python virtual environments to manage playbook dependencies.
   - [ ] **c.** You can use automation execution environments to contain playbook dependencies, separating the control plane from the execution plane.
   - [ ] **d.** You can identify failed deployments and automatically launch jobs by using AI.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

6. Which feature allows automation controller to securely manage API tokens, SSH keys, and secrets?

- [ ] **a.** REST API
- [ ] **b.** Execution environments
- [ ] **c.** Credentials
- [ ] **d.** Inventory templates

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Solution

1. What is the primary role of automation controller in Red Hat Ansible Automation Platform 2.5?
   - ✗ **a.** Managing user desktops across an enterprise
   - ✓ **b. Serving as the central control plane for orchestrating automation workflows**
   - ✗ **c.** Providing access to Red Hat training materials
   - ✗ **d.** Executing Ansible Playbooks without state tracking
2. Which statement about automation controller is accurate?
   - ✗ **a.** Automation controller is not included with Red Hat Ansible Automation Platform 2.
   - ✓ **b. Automation controller is a modular application that has decoupled the control plane from the execution plane by using containerization.**
   - ✗ **c.** Automation controller is a command-line tool that you use to manage Ansible Playbooks in Git repositories.
   - ✗ **d.** Automation controller is a cloud-based application accessed through `console.redhat.com`.
3. Which feature is not part of automation controller?
   - ✓ **a. A wizard for creating playbooks**
   - ✗ **b.** Logs that record the results and output of playbook execution
   - ✗ **c.** Role-based access control (RBAC)
   - ✗ **d.** A RESTful API
4. Which statement is true about how automation controller manages credentials?
   - ✗ **a.** Users who can use credentials stored in automation controller can read the secrets stored in them.
   - ✗ **b.** Credentials cannot be used to authenticate to a version control system to import Ansible project content.
   - ✓ **c. Automation controller encrypts the secrets stored in credentials so that automation controller users can use them but not read them.**
   - ✗ **d.** Automation controller cannot manage credentials.
5. Which statement about improvements to the automation controller architecture is true?
   - ✗ **a.** You can create playbooks in the web interface.
   - ✗ **b.** You can use Python virtual environments to manage playbook dependencies.
   - ✓ **c. You can use automation execution environments to contain playbook dependencies, separating the control plane from the execution plane.**
   - ✗ **d.** You can identify failed deployments and automatically launch jobs by using AI.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

6. Which feature allows automation controller to securely manage API tokens, SSH keys, and secrets?

- ✗ **a.** REST API
- ✗ **b.** Execution environments
- ✓ **c.** **Credentials**
- ✗ **d.** Inventory templates

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

3.3. Creating Automation Controller Resources

Objectives

- Navigate and describe the Automation Execution component of the unified UI, and successfully launch a job using a job template, project, credential, and inventory.

Automation Controller Resources

As in earlier versions of Ansible Automation Platform, automation controller provides a centralized location for organizations to run their Ansible Playbooks. You can manage these resources in the Automation Execution section of the unified UI.

Several resources must exist before you can create a job template to run a playbook in automation controller.

Typically, these resources include:

- A machine credential used to connect to the managed hosts
- A source control credential used to download and synchronize remote content, such as from a Git repository
- A project that specifies the location of content, such as playbooks
- An inventory with at least one host

Although automation controller provides many features, this course focuses on creating the basic resources needed to launch a job template. Detailed information about individual resources, including options and assigning permissions to those resources, is outside the scope of this course.

Creating Credential Resources

Automation controller accesses many resources that might require separate credentials. The following list briefly describes some credential types and their intended purpose. Descriptions of additional credential types and their intended purpose can be found in the reference links.

Ansible Galaxy/automation hub API token

Create this credential resource so that automation controller can download content collections and roles from Ansible Galaxy, automation hub, and private automation hub.

After creating the credential, you must enable the credential for an organization. During this process, you can specify the precedence by ordering the credentials.

Container registry

Create this type of credential if you need to authenticate before you can pull down a container image from a container registry, such as `registry.redhat.io` or a private automation hub.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

GitHub personal access token

GitHub no longer supports password authentication to synchronize projects using the HTTPS protocol. To continue using HTTPS (rather than SSH), create and use a GitHub personal access token, and then create an automation controller credential that uses the personal access token.

Machine

Automation controller can use this credential type to access and apply changes to managed hosts. This credential specifies a username and either a password or the content of the user's SSH private key. If the credential should allow privilege escalation, specify the privilege escalation username (typically `root`) and specify a privilege escalation password if necessary.

> **Note**
> In organizations that mandate against storing passwords, you can configure automation controller to prompt for passwords when using the credential.

Source control

Automation controller can use this credential type to synchronize project resources from a remote repository. Specify a username and either a password or an SSH private key.

Vault

Use vault credentials to decrypt files that have been encrypted with Ansible vault.

Listing Credentials

Go to **Automation Execution > Infrastructure > Credentials** to list credentials. Automation controller displays the name and credential type for each credential.

Click the name of any credential to display the details.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

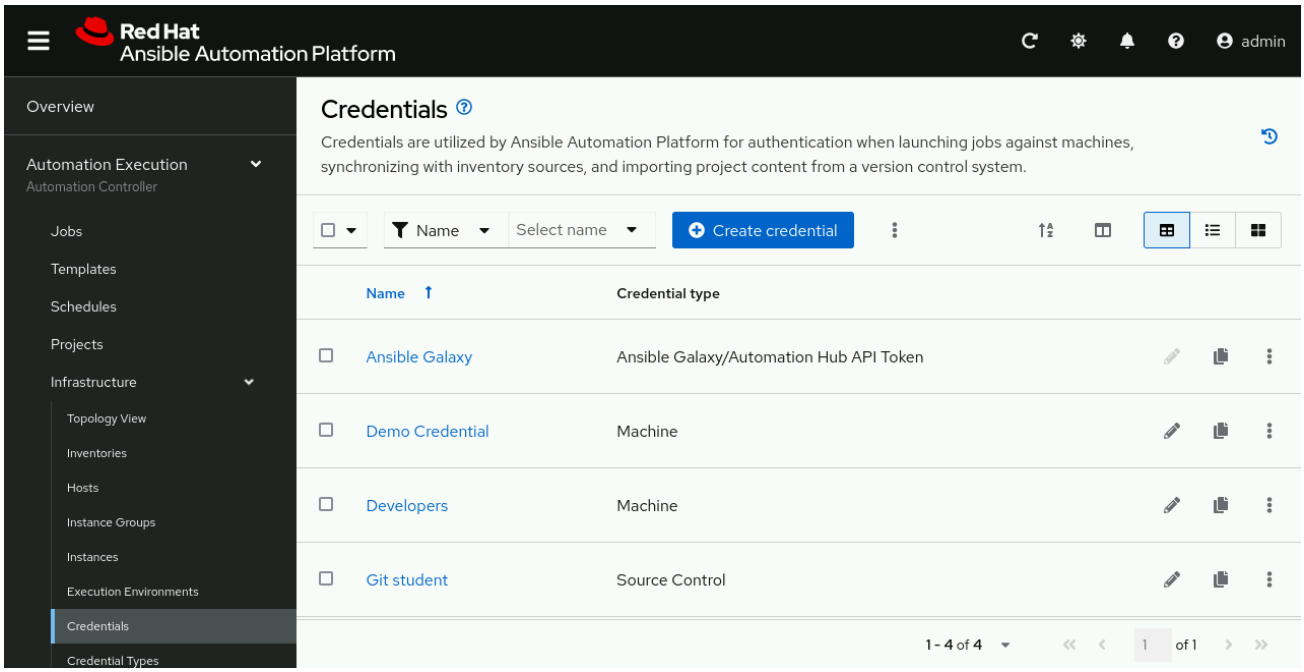


Figure 25. List of existing credentials

Creating a Machine Credential

To create a machine credential, go to **Automation Execution > Infrastructure > Credentials** and click **Create credential**. Specify a name for the credential and choose the `Machine` credential type. Specify additional settings and then click **Create credential**.

Setting	Description
Username	Automation controller connects to the managed hosts as this user (similar to the `remote_user` setting in an `ansible.cfg` file).
Password	The password associated with the user. Enter the password, or prompt for the password when the credential is used. A password or SSH private key is required. Using a password does not work if a managed host prevents password authentication over SSH.
SSH Private Key	This is the associated private key for a public SSH key that has already been copied to the managed host. A password or SSH private key is required.
Private Key Passphrase	If the private SSH key is password protected, then enter the passphrase, or prompt for the passphrase when the credential is used.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Setting	Description
Privilege Escalation Method	The method used for privilege escalation (the equivalent of the `become_method` setting in an `ansible.cfg` file).
Privilege Escalation Username	The user to become for tasks that require privilege escalation (the equivalent of the `become_user` setting in an `ansible.cfg` file).
Privilege Escalation Password	If the user requires a password for privilege escalation, then enter the password, or prompt for the password when the credential is used.

Creating a Source Control Credential

To create a source control credential, go to **Automation Execution > Infrastructure > Credentials** and click **Create credential**. Specify a name for the credential and choose the `Source Control` credential type.

Setting	Description
Username	Automation controller connects to the source control repository as this user.
Password	The password associated with the user. The source control repository might disable password authentication. In that case, use the SCM private key.
SCM Private Key	This is the associated private key for a public SSH key that has already been copied to the managed host.
Private Key Passphrase	If the private SCM key is password-protected, then enter the passphrase.

Creating Project Resources

Automation controller uses project resources to provide access to Ansible Playbooks. If a project uses source control, such as Git or Subversion, then automation controller synchronizes content from a remote repository.

Go to **Automation Execution > Projects** to list existing projects. Click the **>** icon to the left of any project name to expand project information. Synchronize projects that use source control from the `Projects` page.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Use the `Revision` column to assess if your project is current. For a Git repository, the revision string matches a commit hash.

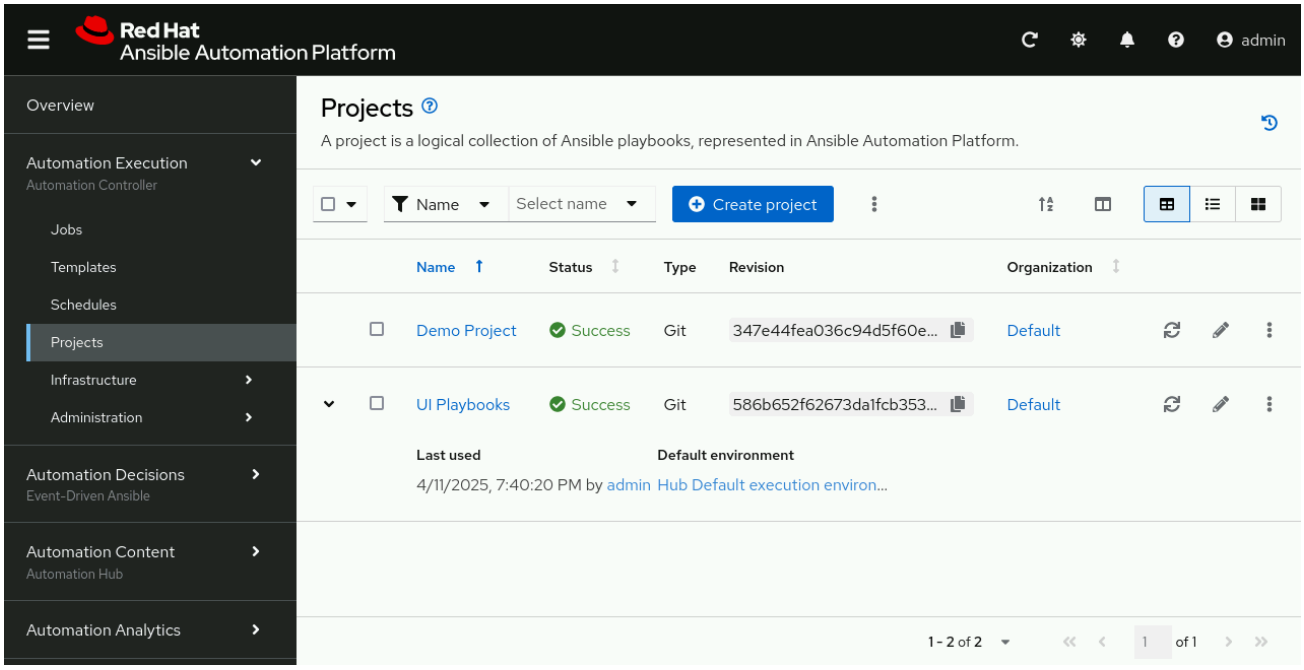


Figure 26. List of existing projects

To create a project, go to **Automation Execution > Projects** and click **Create project**.

Projects can specify a default execution environment to use for job templates. If not specified, then the project defaults to the defined default execution environment for automation controller. Job templates can choose a different execution environment.

Choose a source control credential type, such as Git, specify additional settings, and then click **Create project**.

Setting	Description
Source control URL	The URL used to clone the remote repository.
Source control branch/tag/commit	*(Optional)* A specific branch, tag, or commit for the repository. Use the `Allow branch override` option so that job templates can use a different branch, tag, or commit.
Source control credential	An *existing* credential that can be used to synchronize the remote repository.

Automation controller automatically attempts to synchronize new projects. A project synchronization can fail for many reasons, including:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

- The project specifies an incorrect source control URL.
- The project fails to specify, or specifies the wrong source control credential.
- The source control credential specifies the wrong username, password, or source control management (SCM) private key.
- The project cannot install required collections or roles.

Creating Inventory Resources

Similar to a local inventory file or directory, you can create inventory resources within automation controller. Inventories can contain groups of hosts as well as ungrouped hosts. You can configure variables that apply to the entire inventory (similar to the `all` group), to a specific group (similar to using the `groups_vars` directory), and to a specific host (similar to using the `host_vars` directory). When creating a job template, you must specify an inventory resource to use. This is similar to specifying the `--inventory` (`-i`) option to the `ansible-navigator` command.

Automation controller can contain many inventories, both static and dynamic. Inventories that contain dynamic content display the status of the most recent inventory synchronization attempt. Static inventories display `Disabled` in the `Status` column.

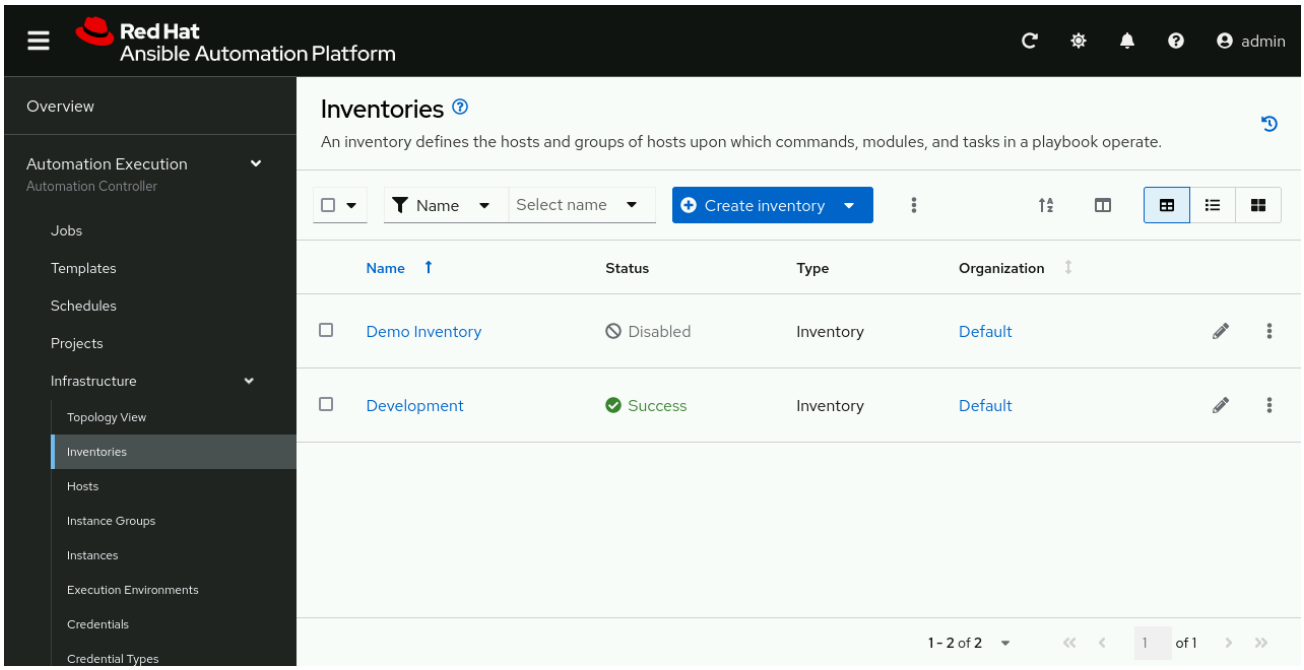


Figure 27. List of existing inventories

To create an inventory, go to **Automation Execution > Infrastructure > Inventories** and click **Create inventory > Create inventory**. Specify a name for the inventory and click **Create inventory**.

After creating the inventory, populate the inventory with groups and hosts. You can manually add the groups and hosts, or you can automatically populate them by using an existing inventory file in the project repository.

Manually Adding Groups and Hosts

To manually add a group, go to the **Groups** tab and then click **Create group**. Enter a name for the group and then click **Create group**.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

To manually add an ungrouped host, go to the **Hosts** tab for the inventory and click **Create host**. Enter a name for the host (often the fully qualified domain name for the host) and then click **Create host**. You can add an ungrouped host to a group at any time.

To manually add a host to a group, go to the **Groups** tab for the inventory, click the group name, and then click the **Hosts** tab for the group. You can either associate an existing host within the inventory or add a new host to the group. When adding a new host, enter a name for the host (often the fully qualified domain name for the host) and then click **Create host**.

The breadcrumb navigation displays the inventory name and the group name. In the following example, the `serverb.lab.example.com` host is being added to the `web` group in the `Development` inventory.

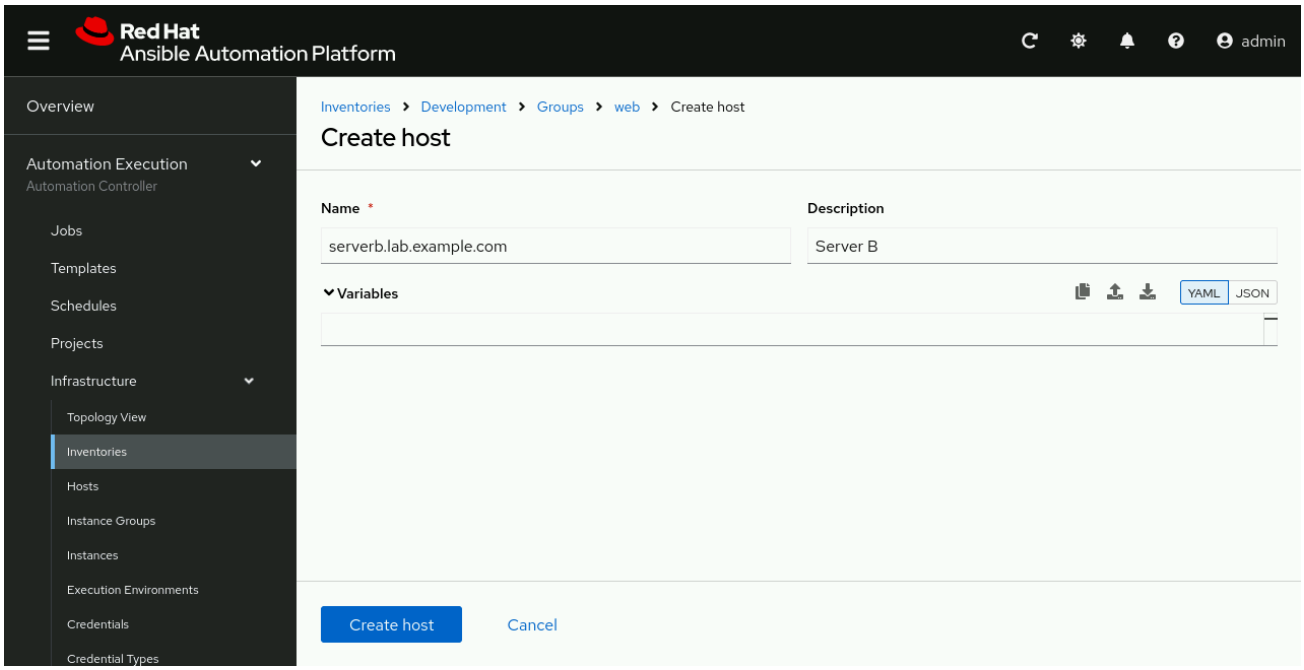


Figure 28. Adding a new host to a group

Populating Groups and Hosts by Using a Project Inventory File

If a project already contains an inventory file, then you can use the file to automatically populate groups and hosts within automation controller.

Within an existing inventory, go to the **Sources** tab and then click **Create source**. Enter a name for the source and then select `Sourced from a Project` from the `Source` menu. Choose an existing project, select an inventory file that exists within the project, and then click **Create source**.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

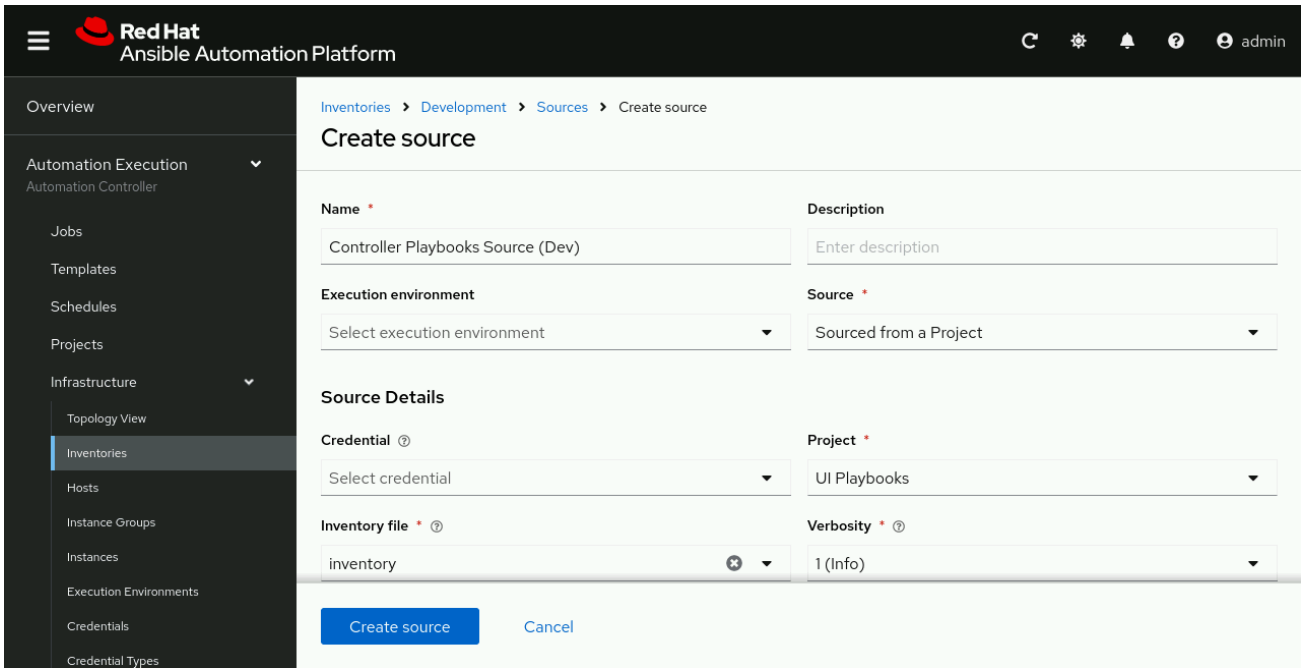


Figure 29. Adding a new inventory source from a project

Within an existing inventory, go to the **Sources** tab and then click the **Launch inventory update** icon for the source. A successful inventory update creates the hosts and groups defined in the selected inventory file.

Creating Job Template Resources

A job template is the equivalent of a specific way of running a playbook. Among other things, a job template includes an inventory and might specify variables to override, or tags to either use or skip.

Got to **Automation Execution > Templates** to list existing templates. Expand template information by clicking the **>** icon to the left of any template name. Initiate a new job run by clicking the **Launch Template** icon.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

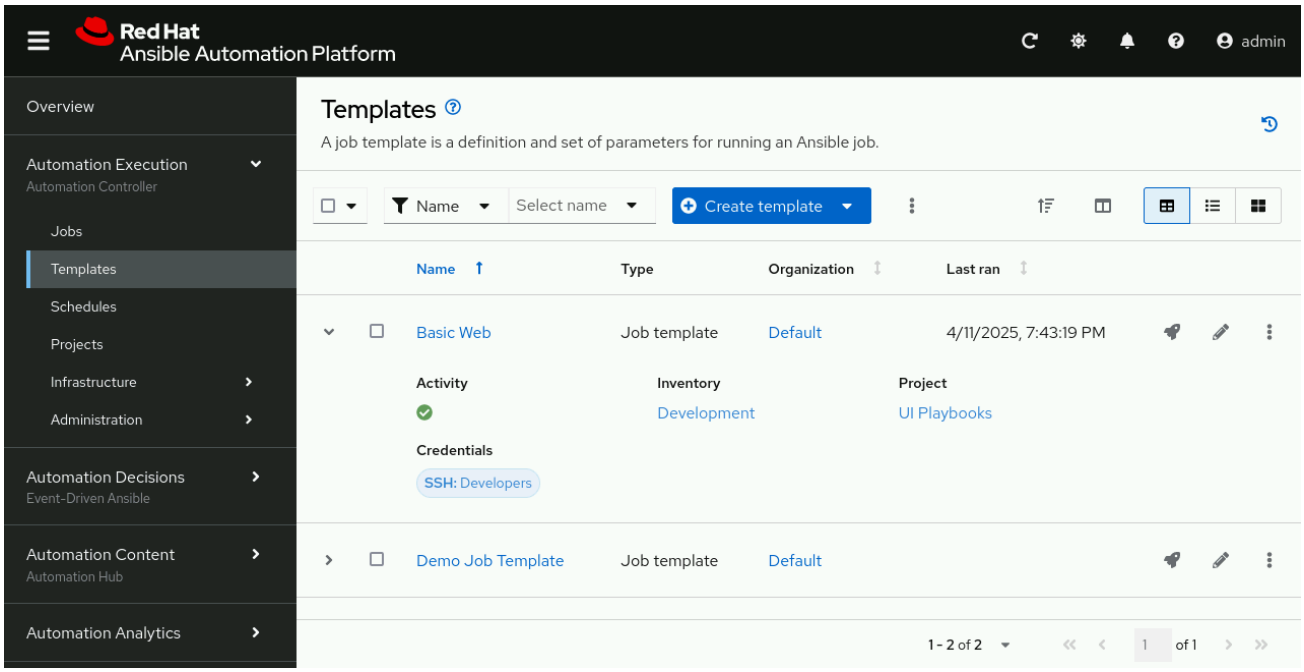


Figure 30. List of existing job templates

To create a job template, go to **Automation Execution > Templates** and then click **Create template > Create job template**. At a minimum, enter a name, select a project, and select a playbook. You can either choose an existing inventory or prompt for an inventory when the job template is launched. Job templates should specify necessary credentials, such as the machine credential that automation controller needs to connect to the managed hosts.

> **Important**
> Organizations that forbid storing credentials, such as passwords or passphrases, should configure job templates to prompt for credentials. Any job template that prompts for passwords (or other settings) must be run *interactively*.

Job templates include several additional optional fields. Many of these options have corresponding command-line options that can be used when running a playbook with the `ansible-navigator` command. You can specify these options as part of the job template, or be prompted for customizations when the job template is launched.

Job template setting or option	Equivalent command-line option
Inventory	`--inventory (-i)`
Execution Environment	`--execution-environment-image (--eei)`
Extra variables	`--extra-vars (-e)`

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Job template setting or option	Equivalent command-line option
Forks	`--forks (-f)`
Limit	`--limit (-l)`
Verbosity	`--verbose (-v)`
Job Tags	`--tags (-t)`
Skip Tags	`--skip-tags`
Privilege Escalation	`--become (-b)`

Launching and Reviewing Jobs

When you launch a job template, automation controller uses an automation execution environment to run the playbook using all the supplied information. If you configured the job template to prompt for information, such as an inventory, credentials, or variables, then automation controller prompts you for this information. Similarly, if the credentials used by your job template prompt for passwords or passphrases, then automation controller prompts you for this information. Job templates that prompt for any information must be run interactively.

Go to **Automation Execution › Jobs** to review job output. In addition to the output of playbook runs, the **Automation Execution › Jobs** page displays inventory synchronization jobs and source control update jobs. Each job contains an ID to show the order in which automation controller ran the job.

Expand job information by clicking the › icon to the left of any job name. Expanded details include who launched the job, which inventory was used, and which execution environment was used. Click a job name to see the full job output.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

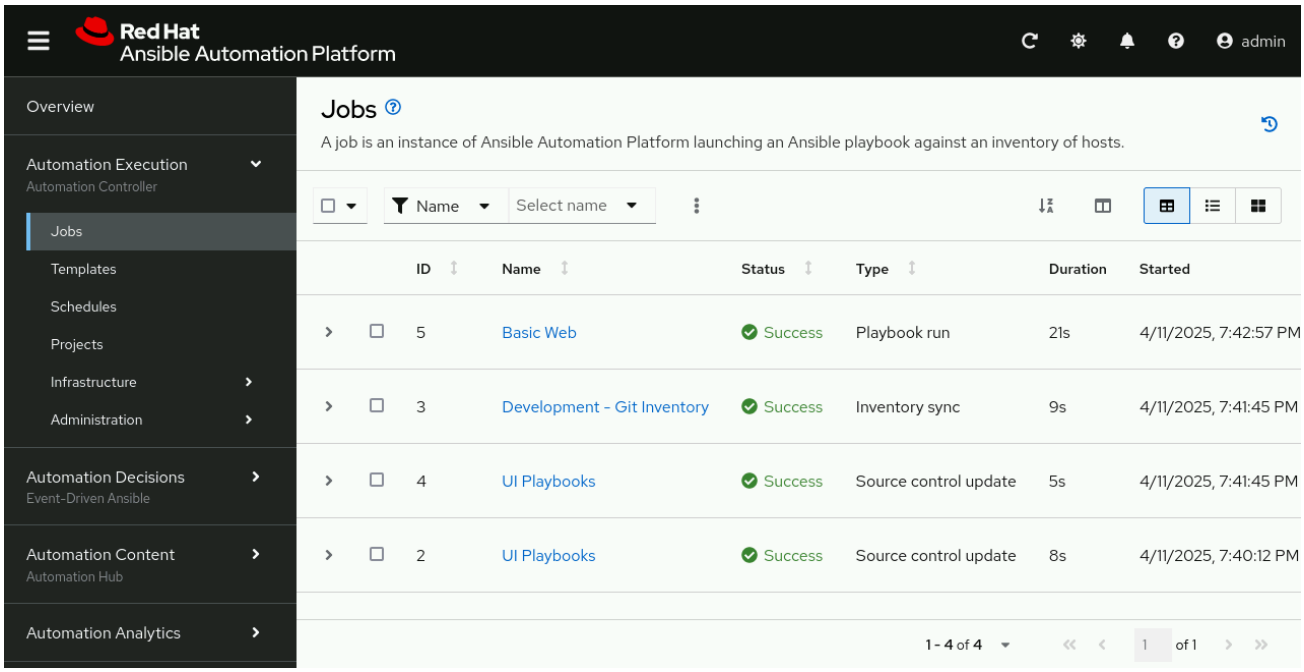


Figure 31. List of jobs

References

Using Automation Execution

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

3.4. Guided Exercise
Creating Automation Controller Resources

Create automation controller resources by using the Automation Execution component of the unified UI, and then launch a job.

Outcomes

- Create source control and machine credentials.
- Create a project.
- Create an inventory that uses an inventory file from the project.
- Create a job template.
- Launch a job template and review the output generated by the job.

Prerequisites

As the `student` user on the `workstation` machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
[student@workstation ~]$ lab start ui-playbooks
```

Instructions

1. Go to `https://aap.lab.example.com` and log in to Ansible Automation Platform as `admin` using `redhat` as the password.

2. To access content from a Git repository, automation controller might require a *source control credential*. Create a source control credential named `Git student`. Configure this credential to use authentication information for the `git.lab.example.com` server.

   2.1. From the unified UI, go to **Automation Execution > Infrastructure > Credentials** and click **Create credential**. Create the credential with the following settings and then click **Create credential**.

Field	Value
Name	`Git student`
Organization	`Default`
Credential type	`Source Control`

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Field	Value
Username	`student`
Password	*(empty)*
SCM Private Key	Content of the `/home/student/.ssh/gitlab_rsa` file.

> **Note**
> If you choose to browse for the file, then right-click anywhere in the directory navigation and select **Show Hidden Files**. With this option enabled you can see the `.ssh` directory in the `/home/student` directory.

3. To run the playbook against each managed host, automation controller must have a *machine credential* that contains authentication information. Create a machine credential named `Developers` that contains authentication information for the managed hosts in the classroom environment.

   3.1. Go to **Automation Execution > Infrastructure > Credentials** and click **Create credential**. Create the credential with the following settings and then click **Create credential**.

Field	Value
Name	`Developers`
Organization	`Default`
Credential type	`Machine`
Username	`devops`
Password	*(empty)*
SSH Private Key	Content of the `/home/student/.ssh/lab_rsa` file.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

> **Note**
> If you choose to browse for the file, then right-click anywhere in the directory navigation and select **Show Hidden Files**. This lets you see the `.ssh` directory in the `/home/student` directory.

4. Create a project named `UI Playbooks`. This project points to a Git repository that contains a playbook and an inventory file. Configure the project to specify a default automation execution environment.

   4.1. Go to **Automation Execution > Projects** and then click **Create project**. Create the project with the following settings and then click **Create project**.

Field	Value
Name	`UI Playbooks`
Organization	`Default`
Execution environment	`Hub Default execution environment`
Source control type	`Git`
Source control URL	`git@git.lab.example.com:student/ui-playbooks.git`
Source control credential	`Git student`
Options	`Allow branch override`

   After you click **Create project**, the unified UI immediately attempts to synchronize the project. The project synchronization should succeed.

> **Important**
> If synchronization fails, ensure that the project specifies the correct source control URL and credentials and then attempt the project synchronization again.

5. Create an inventory for this exercise named `Development` using the `inventory` file in the project repository.

   Managing your inventory file in a Git repository enables you to benefit from storing your changes in a version control system. You can also update the inventory by editing the file rather than manually updating inventory information in the unified UI.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

5.1. Go to **Automation Execution > Infrastructure > Inventories** and click **Create inventory > Create inventory**.

5.2. Create an inventory with the following settings and then click **Create inventory**.

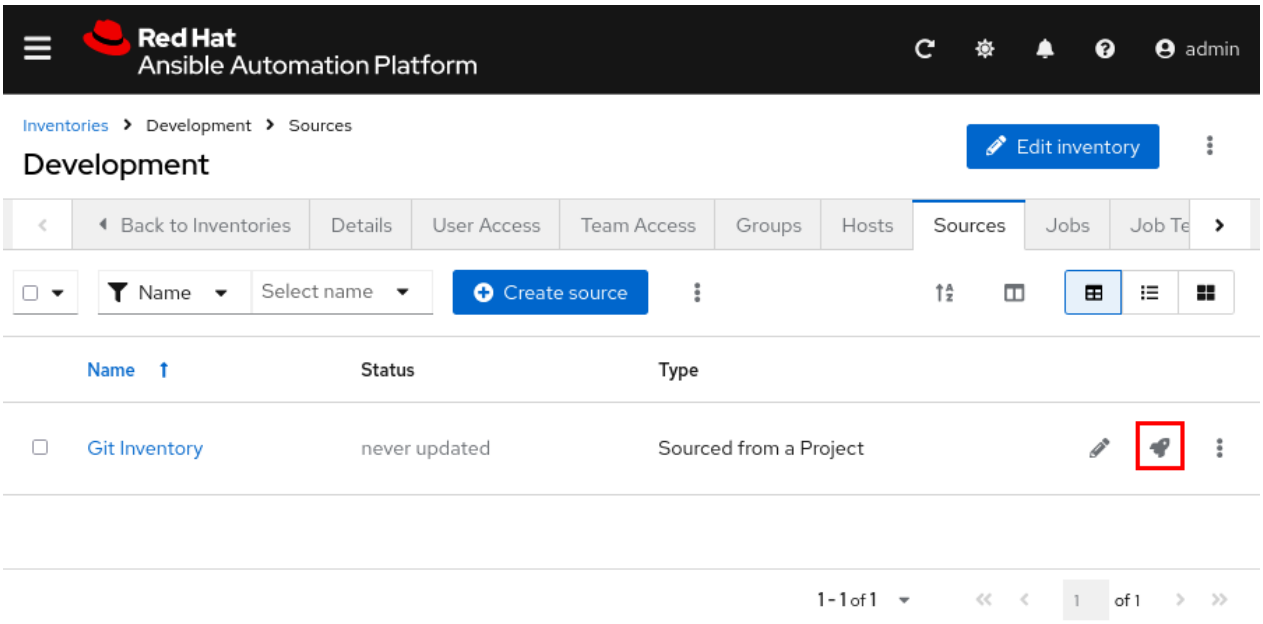
Field	Value
Name	Development
Organization	Default

5.3. From the **Sources** tab, click **Create source**. Create an inventory source with the following settings and then click **Create source**.

Field	Value
Name	Git Inventory
Source	Sourced from a Project
Project	UI Playbooks
Inventory file	inventory
Options	Update on launch
Cache timeout (seconds)	600

5.4. Click the **Back to Inventory Sources** tab and then click the **Launch inventory update** icon for the **Git Inventory** source.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use



The status updates to indicate that the synchronization succeeded.

5.5. Click the **Hosts** tab. The inventory source imported the `serverd.lab.example.com` host.

5.6. Click the **Groups** tab. The inventory source imported the web group.

5.7. Click **web** and then click the **Hosts** tab. The web group contains the `serverd.lab.example.com` host.

6. Create a job template named `Basic Web`. Configure the job template to run the `basic-web.yml` playbook using the default automation execution environment for the project. The playbook ensures that a web server and its content are deployed on hosts in the web inventory host group.

6.1. Go to **Automation Execution > Templates** and then click **Create template > Create job template**. Create the job template with the following settings and then click **Create job template**.

Field	Value
Name	`Basic Web`
Inventory	`Development`
Project	`UI Playbooks`
Playbook	`basic-web.yml`
Credentials	`Developers | Machine`

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

6.2. Click **Launch template** to run the job template. The resulting job successfully runs the playbook.

> **Note**
> You can safely ignore the deprecation warning about the ANSIBLE_COLLECTIONS_PATHS option.

6.3. Go to **Automation Execution > Jobs** and look for a numbered job with Basic Web in the name. By default, the unified UI lists the most recent jobs at the top. The Basic Web job displays a status of Success.

6.4. *(Optional)* Click the name of the Basic Web job to review the job output. This is the same output that you initially saw when you launched the job.

6.5. Verify that the job successfully deployed a web server on serverd.lab.example.com. From a terminal window, use the curl command to access web content at http://serverd.lab.example.com.

```
[student@workstation ~]$ curl http://serverd.lab.example.com
Hello world from serverd.lab.example.com.
```

Finish

On the workstation machine, use the lab command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish ui-playbooks
```

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

3.5. Lab
Running Playbooks with Automation Controller

Launch a job that uses a specific automation execution environment to run a playbook, and then review the job results.

Outcomes

- Create automation controller resources, such as credentials, projects, and inventories.
- Create a job template and then launch a job from that job template.

Prerequisites

As the `student` user on the `workstation` machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
[student@workstation ~]$ lab start ui-review
```

Instructions

Create all resources from the Ansible Automation Platform unified UI at `https://aap.lab.example.com`. Log in as `admin` using `redhat` as the password.

1. Create a source control credential named `Git Review`. The credential must belong to the `Default` organization. The GitLab user for the credential is `student`. Use the private key stored in the `/home/student/.ssh/gitlab_rsa` file on the `workstation` machine as the source control management (SCM) private key.

2. Create a machine credential named `Machine Review`. The credential must belong to the `Default` organization. Configure this credential for the `devops` user. Use the SSH private key stored in the `/home/student/.ssh/lab_rsa` file.

3. Create an inventory named `Inventory Review`. The inventory must belong to the `Default` organization. Add the `servera.lab.example.com` host to this inventory.

4. Create a project named `Project Review`. The project must belong to the `Default` organization. This project must use the `git@git.lab.example.com:student/ui-review.git` repository and use the `Git Review` source control credential to authenticate to the repository.

5. Create a job template named `Template Review`. Configure the job template to use the `Inventory Review` inventory, the `Project Review` project, the `webserver.yml` playbook, and the `Machine Review` machine credential. Configure the job template to use the `Hub Default execution environment` automation execution environment.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Launch a job that uses the `Template Review` job template. If you completed the exercise correctly, then navigating to `http://servera.lab.example.com` displays the following message:

```
Successful playbook run from the unified UI.
```

Evaluation

As the `student` user on the `workstation` machine, use the `lab` command to grade your work. Correct any reported failures and rerun the command until successful.

```
[student@workstation ~]$ lab grade ui-review
```

Finish

As the `student` user on the `workstation` machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish ui-review
```

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Solution

Launch a job that uses a specific automation execution environment to run a playbook, and then review the job results.

Outcomes

- Create automation controller resources, such as credentials, projects, and inventories.
- Create a job template and then launch a job from that job template.

Prerequisites

As the `student` user on the `workstation` machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
[student@workstation ~]$ lab start ui-review
```

Instructions

Create all resources from the Ansible Automation Platform unified UI at `https://aap.lab.example.com`. Log in as `admin` using `redhat` as the password.

1. Create a source control credential named `Git Review`. The credential must belong to the `Default` organization. The GitLab user for the credential is `student`. Use the private key stored in the `/home/student/.ssh/gitlab_rsa` file on the `workstation` machine as the source control management (SCM) private key.

   1.1. Go to `https://aap.lab.example.com` and log in to Ansible Automation Platform as `admin` using `redhat` as the password.

   1.2. From the unified UI, go to **Automation Execution › Infrastructure › Credentials** and click **Create credential**. Create the credential with the following settings and then click **Create credential**.

Field	Value
Name	`Git Review`
Organization	`Default`
Credential type	`Source Control`
Username	`student`

Field	Value
Password	*(empty)*
SCM Private Key	Content of the `/home/student/.ssh/gitlab_rsa` file.

> **Note**
> If you choose to browse for the file, then right-click anywhere in the directory navigation area and select **Show Hidden Files**. Viewing hidden files enables you to view the `.ssh` directory in the `/home/student` directory.

2. Create a machine credential named `Machine Review`. The credential must belong to the `Default` organization. Configure this credential for the `devops` user. Use the SSH private key stored in the `/home/student/.ssh/lab_rsa` file.

   2.1. Go to **Automation Execution > Infrastructure > Credentials** and click **Create credential**. Create the credential with the following settings and then click **Create credential**.

Field	Value
Name	`Machine Review`
Organization	`Default`
Credential type	`Machine`
Username	`devops`
Password	*(empty)*
SSH Private Key	Content of the `/home/student/.ssh/lab_rsa` file.

3. Create an inventory named `Inventory Review`. The inventory must belong to the `Default` organization. Add the `servera.lab.example.com` host to this inventory.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

3.1. Go to **Automation Execution > Infrastructure > Inventories** and click **Create inventory > Create inventory**.

3.2. Create an inventory with the following settings and then click **Create inventory**.

Field	Value
Name	`Inventory Review`
Organization	`Default`

3.3. Click the **Hosts** tab and then click **Create host**. Add the `servera.lab.example.com` host, and then click **Create host**.

4. Create a project named `Project Review`. The project must belong to the `Default` organization. This project must use the `git@git.lab.example.com:student/ui-review.git` repository and use the `Git Review` source control credential to authenticate to the repository.

4.1. Go to **Automation Execution > Projects** and then click **Create project**. Create the project with the following settings and then click **Create project**.

Field	Value
Name	`Project Review`
Organization	`Default`
Source control type	`Git`
Source control URL	`git@git.lab.example.com:student/ui-review.git`
Source control credential	`Git Review`

After you click **Create project**, the unified UI immediately attempts to synchronize the project. The project synchronization should succeed.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

> **Important**
> If the project synchronization fails, then ensure that the project specifies the correct source control URL and credential and then attempt the project synchronization again.

5. Create a job template named `Template Review`. Configure the job template to use the `Inventory Review` inventory, the `Project Review` project, the `webserver.yml` playbook, and the `Machine Review` machine credential. Configure the job template to use the `Hub Default execution environment` automation execution environment.

   Launch a job that uses the `Template Review` job template. If you completed the exercise correctly, then navigating to `http://servera.lab.example.com` displays the following message:

   ```
   Successful playbook run from the unified UI.
   ```

   5.1. Go to **Automation Execution > Templates** and then click **Create template > Create job template**. Create the job template with the following settings and then click **Create job template**.

Field	Value
Name	`Template Review`
Inventory	`Inventory Review`
Project	`Project Review`
Playbook	`webserver.yml`
Execution environment	`Hub Default execution environment`
Credentials	`Machine Review | Machine`

   5.2. Click **Launch template** to create a job from the job template. The resulting job runs the playbook.

> **Important**
> If the job fails, then ensure that the job template specifies the correct inventory and machine credential. Ensure that the machine credential uses the correct values and then launch a new job from the job template.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

5.3. Click the **Details** tab. Notice that the job displays `Success` as its status.

5.4. Verify that the job updated the web content. Either use a web browser to go to `http://servera.lab.example.com` or run the following `curl` command:

```
[student@workstation ~]$ curl http://servera.lab.example.com
Successful playbook run from the unified UI.
```

Evaluation

As the `student` user on the `workstation` machine, use the `lab` command to grade your work. Correct any reported failures and rerun the command until successful.

```
[student@workstation ~]$ lab grade ui-review
```

Finish

As the `student` user on the `workstation` machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish ui-review
```

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

3.6. Summary

- Automation controller provides a centralized location that you can use to run your Ansible automation and review the results of automation runs.
- An automation controller *project* specifies the location of a Git repository that stores Ansible code and, if needed, a source control credential to authenticate to the Git repository.
- An automation controller *job template* specifies the inventory, machine credentials, automation execution environment, project, and playbook to use to run your Ansible code on managed hosts.
- You can use automation controller in the unified UI to launch automation jobs from a job template and to review the results of jobs.
- Using `ansible-navigator` to test Ansible Playbooks in an automation execution environment can help streamline the process of setting up those playbooks to run successfully on automation controller.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Chapter 4.
Working with Ansible Configuration Settings

Goal

Examine and adjust the configuration of Ansible and automation content navigator to simplify development and to troubleshoot issues.

Sections

- Examining the Ansible Configuration with Automation Content Navigator (and Guided Exercise)
- Configuring Automation Content Navigator (and Guided Exercise)
- Developing Content in Visual Studio Code (and Guided Exercise)

Lab

- Working with Ansible Configuration Settings

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

4.1. Examining the Ansible Configuration with Automation Content Navigator

Objectives

- Browse the current Ansible configuration by using automation content navigator.

Inspecting the Ansible Configuration in Interactive Mode

The `ansible-navigator config` command displays the current Ansible configuration used by the `ansible-navigator run` command in interactive mode. You can also use `ansible-navigator config` to determine the source of current `ansible-navigator` configuration settings.

Knowing the source of `ansible-navigator` configuration settings can help you determine why a particular setting in your current Ansible configuration has a specific value.

For example, configuration settings can come from these sources:

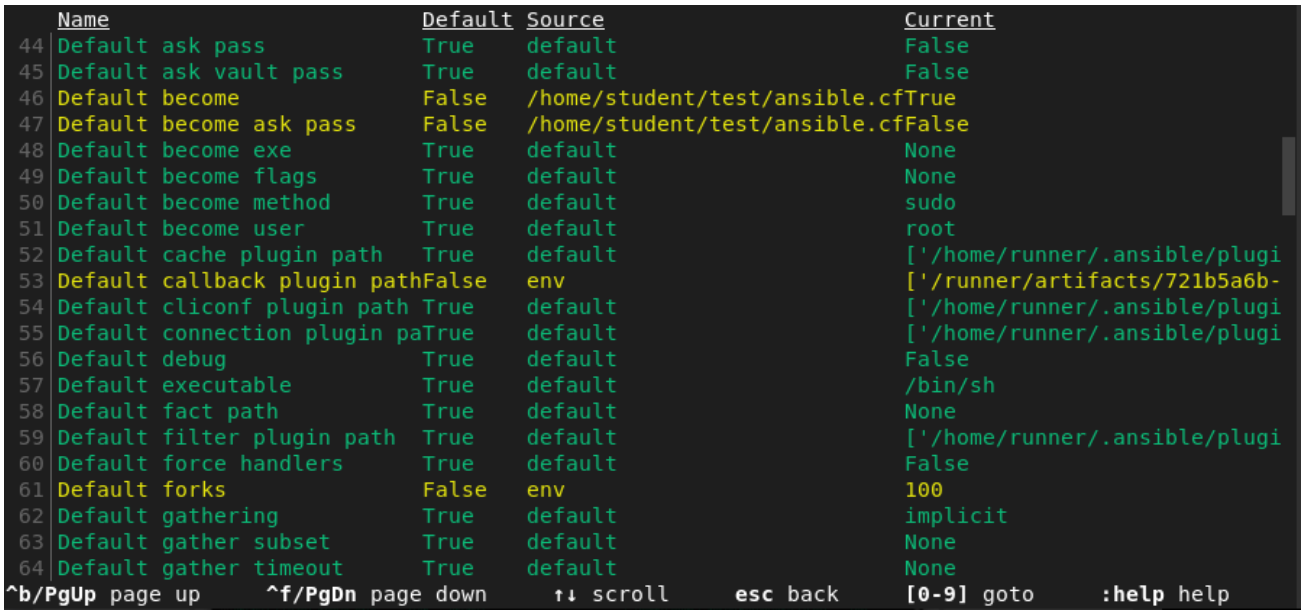
- Specific environment variables
- A configuration file specified by the `ANSIBLE_CONFIG` environment variable
- Hard-coded default values

You can use the `ansible-navigator config` command to examine your configuration, retrieve the current and default configuration settings, and access documentation for each configuration parameter.

The `ansible-navigator config` command is useful for troubleshooting configuration issues. For example, this command can help you locate the issue when a configuration parameter does not work as expected or when Ansible does not take a parameter into account. It also displays the actual value that Ansible uses for each parameter and from which source it retrieves that value, configuration file, or environment variable.

The following screenshot shows the output of the `ansible-navigator config` command. Each line describes an Ansible configuration parameter.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use



```
    Name                          Default Source                      Current
 44 Default ask pass              True    default                     False
 45 Default ask vault pass        True    default                     False
 46 Default become                False   /home/student/test/ansible.cfTrue
 47 Default become ask pass       False   /home/student/test/ansible.cfFalse
 48 Default become exe            True    default                     None
 49 Default become flags          True    default                     None
 50 Default become method         True    default                     sudo
 51 Default become user           True    default                     root
 52 Default cache plugin path     True    default                     ['/home/runner/.ansible/plugi
 53 Default callback plugin pathFalse     env                         ['/runner/artifacts/721b5a6b-
 54 Default cliconf plugin path   True    default                     ['/home/runner/.ansible/plugi
 55 Default connection plugin paTrue      default                     ['/home/runner/.ansible/plugi
 56 Default debug                 True    default                     False
 57 Default executable            True    default                     /bin/sh
 58 Default fact path             True    default                     None
 59 Default filter plugin path    True    default                     ['/home/runner/.ansible/plugi
 60 Default force handlers        True    default                     False
 61 Default forks                 False   env                         100
 62 Default gathering             True    default                     implicit
 63 Default gather subset         True    default                     None
 64 Default gather timeout        True    default                     None
^b/PgUp page up     ^f/PgDn page down     ↑↓ scroll     esc back     [0-9] goto     :help help
```

The columns provide information about each parameter:

Name

The Name column displays the internal names that Ansible uses for parameters. These names are not the same as the names that you use in the `ansible.cfg` configuration file. For example, the `Default become` parameter corresponds to the become parameter in the `ansible.cfg` configuration file. You can display that name mapping by typing the number of the parameter you are interested in, prefixed by a colon if that number is greater than 9.

Default

The `Default` column indicates whether the parameter is using its default value. The column displays `False` when you override the default value of a parameter by explicitly setting the value in an `ansible.cfg` configuration file or through an environment variable. To help you identify changed parameters, the `ansible-navigator` command displays default parameter settings in green and those that have changed from the defaults in yellow.

Source

When a parameter is not using its default value, the `Source` column indicates how the parameter has been set. The `Source` column gives the full path to the configuration file from which Ansible retrieved the parameter's value. If the column displays the `env` keyword, then an environment variable set that parameter's value.

Current

The `Current` column displays the actual value of the parameter.

Searching for Specific Configuration Parameters

The filter function of the `ansible-navigator` command enables you to search for specific parameters. Type `:filter` (or `:f`) followed by a regular expression to perform a search in the command output. For example, to locate the `Default forks` parameter, you could use the `:f forks` filter command.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

This function is not limited to searching for parameter names. You can search for any pattern in the `ansible-navigator` output. For example, you can use the `:f ansible.cfg` filter command to list all the parameters defined in an `ansible.cfg` configuration file. Remember that regular expressions are case-sensitive.

Accessing Parameter Details

To get the details of a parameter, type its number, prefixed by a colon if that number is greater than 9. This example displays the details of the `Default forks` parameter:

```
Default forks (current: 100)  (default: 5)
 0|---
 1|current_config_file: None
 2|current_value: 100 (1)
 3|default: false
 4|default_value: 5 (2)
 5|description: Maximum number of forks Ansible will use to run tasks on target
hosts.
 6|env:
 7|- name: ANSIBLE_FORKS (3)
 8|ini:
 9|- key: forks (4)
10|  section: defaults
11|name: Default forks
12|option: DEFAULT_FORKS
13|source: env
14|type: integer
15|via: ANSIBLE_FORKS
```

(1) The actual parameter value is `100`.
(2) The parameter's default value is 5. Ansible only uses that default value if you do not override it in the `ansible.cfg` configuration file or through the environment variable.
(3) The name of the environment variable that you can use to set the parameter value is `ANSIBLE_FORKS`.
(4) You can also set the parameter's value in an `ansible.cfg` configuration file by setting the `forks` parameter in the `[defaults]` section.

Inspecting the Local Configuration

By default, the `ansible-navigator` command uses an automation execution environment to perform its tasks. If your Ansible project does not provide the `ansible.cfg` configuration file, then the `ansible-navigator` command uses the `/etc/ansible/ansible.cfg` file that the automation execution environment supplies.

The command does not use the `/etc/ansible/ansible.cfg` or the `~/.ansible.cfg` configuration files from the local system when you use automation execution environments. To process those local files, use the `--ee false` option to prevent the command from using automation execution environments.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Important

Because you use `ansible-navigator` to run your playbook with an automation execution environment instead of the local control node environment, you usually inspect the automation execution environment configuration.

Inspecting the Ansible Configuration in Standard Output Mode

In addition to its interactive mode, the `ansible-navigator config` command can list the configuration parameters on the standard output and then exit. Add the `--mode stdout` (or `-m stdout`) option to the command to use that mode.

- The `list` subcommand lists all the Ansible configuration parameters. The resulting output is a static list that describes each parameter. The command does not report the current parameter values.
- The `dump` subcommand lists all the Ansible configuration parameters and their current values.
- The `view` subcommand displays the contents of the `ansible.cfg` configuration file that Ansible is using. This subcommand is helpful to confirm that Ansible is using the expected configuration file.

This example lists the configuration parameters and their current values:

```
[user@host ansible-project]$ ansible-navigator config dump -m stdout
ACTION_WARNINGS(default) = True
AGNOSTIC_BECOME_PROMPT(default) = True
...output omitted...
DEFAULT_ASK_VAULT_PASS(default) = False
DEFAULT_BECOME(/home/user/ansible-project/ansible.cfg) = True
DEFAULT_BECOME_ASK_PASS(/home/user/ansible-project/ansible.cfg) = False
DEFAULT_BECOME_EXE(default) = None
...output omitted...
DEFAULT_FORCE_HANDLERS(default) = False
DEFAULT_FORKS(env: ANSIBLE_FORKS) = 100
DEFAULT_GATHERING(default) = implicit
...output omitted...
```

References

Review Your Ansible Configuration with Automation Content Navigator

Configuring Ansible - Ansible Documentation

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

4.2. Guided Exercise
Examining the Ansible Configuration with Automation Content Navigator

Browse the current configuration settings for running Ansible Playbooks by using automation content navigator.

Outcomes

- Use the `ansible-navigator` command to review your current Ansible configuration.
- Compare your custom configuration to default Ansible settings.

Prerequisites

As the `student` user on the `workstation` machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
[student@workstation ~]$ lab start config-ansible
```

Instructions

1. Use automation content navigator to browse the current configuration and determine which settings use their default values.

   1.1. Run the following `ansible-navigator config` command. Add the `--eei` option to specify the automation execution environment image, and add the `--pp` option to specify if the command should attempt to download (or pull) the image. In this example, the command only attempts to download the image if it does not already exist on the local system.

   ```
   [student@workstation ~]$ ansible-navigator config \
   --eei aap.lab.example.com/ee-supported-rhel8 --pp missing
   ```

   You can explore the current configuration settings for automation content navigator.

   ```
     Name                                  Default    Source     Current
   0|Action warnings                       True       default    True
   1|Agnostic become prompt                True       default    True
   2|Allow world readable tmpfiles         True       default    False
   3|Ansible connection path               True       default    None
   ...output omitted...
   ```

   1.2. Search for the `Default host list` option. Type `:f host` and then press Enter.

```
  Name                                Default    Source     Current
0|Default host list                  True       default
['/etc/ansible/hosts']
1|Display skipped hosts              True       default    True
...output omitted...
```

Notice that line `0` in your terminal output is green and the value in the `Default` column is `True`, indicating that it is a default setting.

1.3. Press `0` to view more details for the `Default host list` option:

```
Default host list (current/default: ['/etc/ansible/hosts'])
 0|---
 1|current_config_file: None
 2|current_value:
 3|- /etc/ansible/hosts
 4|default: true
 5|default_value: /etc/ansible/hosts
 6|description: Comma separated list of Ansible inventory sources
 7|env:
 8|- name: ANSIBLE_INVENTORY
 9|expand_relative_paths: true
10|ini:
11|- key: inventory
12|  section: defaults
13|name: Default host list
14|option: DEFAULT_HOST_LIST
15|source: default
16|type: pathlist
17|via: default
18|yaml:
19|  key: defaults.inventory
```

Exit from `ansible-navigator` by pressing `Esc` twice, or type `:q` and then press `Enter`.

2. Customize your current Ansible configuration and use automation content navigator to identify some changes that you made to the default settings. Ansible configuration settings specified within an `ansible.cfg` file in your current directory take precedence over the default Ansible configuration settings.

2.1. Change to the `/home/student/config-ansible` directory and review the `ansible.cfg` configuration file:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation ~]$ cd ~/config-ansible
[student@workstation config-ansible]$ cat ansible.cfg
[defaults]
inventory=inventory
remote_user=devops
```

2.2. Run the following `ansible-navigator config` command:

```
[student@workstation config-ansible]$ ansible-navigator config \
--eei aap.lab.example.com/ee-supported-rhel8 --pp missing
```

2.3. Search for the `Default host list` option again. Type `:f host` and then press `Enter`.

```
  Name                  Default Source                Current
0|Default host list     False   /home/student/conf...
['/home/student/conf...
1|Display skipped hosts True    default               True
...output omitted...
```

Notice that line `0` in your terminal output is yellow and the value in the `Default` column is `False`, indicating that it is not a default setting. This time the `Default host list` option has the following displayed values:

Field	Value
Default	False
Source	/home/student/config-ansible/ansible.cfg
Current	['/home/student/config-ansible/inventory']

2.4. Press `0` to view more details for the `Default host list` option in the custom configuration:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
Default host list (current: ['/home/student/config-ansible/inventory']) ...
 0|---
 1|current_config_file: /home/student/config-ansible/ansible.cfg
 2|current_value:
 3|- /home/student/config-ansible/inventory
 4|default: false
 5|default_value: /etc/ansible/hosts
 6|description: Comma separated list of Ansible inventory sources
 7|env:
 8|- name: ANSIBLE_INVENTORY
 9|expand_relative_paths: true
10|ini:
11|- key: inventory
12|  section: defaults
13|name: Default host list
14|option: DEFAULT_HOST_LIST
15|source: /home/student/config-ansible/ansible.cfg
16|type: pathlist
17|via: /home/student/config-ansible/ansible.cfg
18|yaml:
19|  key: defaults.inventory
```

Exit from the `ansible-navigator` command by typing `:q` and then pressing `Enter`.

3. Use automation content navigator to search for a specific configuration setting, determine if the setting is customized, and discover the source of any customizations.

   3.1. Run the following `ansible-navigator config` command:

```
[student@workstation config-ansible]$ ansible-navigator config \
--eei aap.lab.example.com/ee-supported-rhel8 --pp missing
```

   Type `:f ask` and then press `Enter` to search for the `Default ask pass` setting.

```
  Name                                  Default    Source     Current
0|Default ask pass                      True       default    False
1|Default ask vault pass                True       default    False
...output omitted...
```

   Notice that line `0` in your terminal output is green and the value in the `Default` column is `True`. This means that the displayed values are the default settings for the `Default ask pass` option.

   Type `:q` and then press `Enter` to exit from the `ansible-navigator` command.

   3.2. In the `ansible.cfg` file, add the `ask_pass=true` line to the `[defaults]` section. The resulting `ansible.cfg` file has the following content:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[defaults]
inventory=inventory
remote_user=devops
ask_pass=true
```

3.3. Run the `ansible-navigator config` command again:

```
[student@workstation config-ansible]$ ansible-navigator config \
--eei aap.lab.example.com/ee-supported-rhel8 --pp missing
```

Type `:f ask` and then press `Enter` to search for the `Default ask pass` setting.

```
  Name                          Default Source
Current
0|Default ask pass              False   /home/student/config-ansibl... True
1|Default ask vault pass        True    default                        False
...output omitted...
```

Notice that line `0` in your terminal output is yellow and the value in the `Default` column is `False`. The displayed values correspond to the custom configuration for the `Default ask pass` option.

Exit the `ansible-navigator` command by pressing `Esc`.

3.4. Change the value of the `ask_pass` setting in the `ansible.cfg` file to `false`. The resulting `ansible.cfg` file should read as follows:

```
[defaults]
inventory=inventory
remote_user=devops
ask_pass=false
```

3.5. Run the `ansible-navigator config` command again:

```
[student@workstation config-ansible]$ ansible-navigator config \
--eei aap.lab.example.com/ee-supported-rhel8 --pp missing
```

Type `:f ask` and then press `Enter` to search for the `Default ask pass` setting.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
  Name                          Default Source
Current
0|Default ask pass              False   /home/student/config-ansibl... False
1|Default ask vault pass        True    default                        False
...output omitted...
```

The `Default ask pass` setting has a value of `False` if it is not overridden. Although this value matches the value configured in your `ansible.cfg` file, line `0` in your terminal output remains yellow and the value in the `Default` column is `False`. The `ansible.cfg` file in your current directory overrides the `Default ask pass` setting, even if the file uses the same value as the default value for the setting. The displayed values correspond to the custom configuration for the `Default ask pass` option:

Field	Value
Default	False
Source	/home/student/config-ansible/ansible.cfg
Current	False

Exit the `ansible-navigator` command by pressing `Esc`.

Finish

On the `workstation` machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish config-ansible
```

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

4.3. Configuring Automation Content Navigator

Objectives

- Change configuration settings for automation content navigator with its configuration file, and determine where the configuration file is located.

Format of the Settings File

You can create a configuration file (or *settings* file) for `ansible-navigator` to override the default values of its configuration settings. A configuration file is useful, for example, if you want to use a different automation execution environment from the default one, and you do not want to enter the correct `--eei` option every time you run the `ansible-navigator` command.

- The settings file can be in JSON or YAML format.
- For settings in JSON format, the extension must be `.json`.
- For settings in YAML format, the extension must be `.yml` or `.yaml`.

This course focuses on the YAML-formatted configuration file.

Locating the Settings File

Automation content navigator looks for a settings file in the following order and uses the first file that it finds:

- The file defined by the `ANSIBLE_NAVIGATOR_CONFIG` environment variable
- The `ansible-navigator.yml` file in your current Ansible project directory
- The `~/.ansible-navigator.yml` file (in your home directory) Notice that it has a "dot" at the start of its file name.

Each project can have its own automation content navigator settings file. Common configuration parameters can be names of automation execution environment images required for the project, Ansible configuration parameters such as forks, log levels, artifact creation, and so on.

> **Important**
> The project directory and home directory can only contain one settings file each. Having an `ansible-navigator.yml` file and an `ansible-navigator.yaml` (or `ansible-navigator.json` file) in the same directory results in an error.

Selecting a Settings File to Use

If you have an `ansible-navigator.yml` file in your Ansible project directory, it is stored in version control with the rest of your project, and overrides any configuration file in the user's home directory. This also means that other users of that project use that same configuration file.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

If you have a `~/.ansible-navigator.yml` file, it is used only if there is no other configuration file available. You can use it to specify default settings to use if the Ansible project you are working with has no preferred configuration settings.

You should only specify an automation content navigator configuration file with the `ANSIBLE_NAVIGATOR_CONFIG` environment variable if you want to override all other configuration files.

Generating a Settings File

You can use the `ansible-navigator settings` command to generate settings files. If you run that command with the `--sample` option, then the command outputs a sample `ansible-navigator` configuration file in YAML format. If you run that command with the `--effective` option instead, then the command outputs a configuration file that matches the current effective configuration for `ansible-navigator`. In either case, you could redirect the output of the command into a file to save it, edit it, and use it.

The `--sample` option generates output where most lines begin with a number sign (#), indicating that they are commented out. This output also contains comments that describe the settings. Many comments include samples and display valid setting values. If you want to use a commented-out line, then to preserve the correct indentation of the YAML file, you must remove the number sign at the beginning of the line *and* one blank space. Removing the additional space conforms to the best practice of using two spaces for indentation and ensures that the uncommented lines match the existing indentation of the `logging` top-level setting.

The `--effective` option generates more condensed output and does not include any comments. Many options that you can pass to the command, such as the `--eei` and `--pp` options are configured in the output. When running this command, the `mode` key always has a value of `stdout`. If you do not need this value, then you can change the value after generating the sample file. You can delete any of the lines that you do not want to explicitly configure, but if you remove a parent key, then you must remove all its child keys. Because the command correctly indents all the output, this method generates a working configuration that you can immediately use.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Important

If you redirect the output of the `ansible-navigator settings` command into a file named `ansible-navigator.yml` in the current directory, then the `ansible-navigator` command attempts to use the configuration file and ultimately fails. Instead, redirect the command output to a file in a different directory, such as to the `/tmp/ansible-navigator.yml` file, or to a file with a different name in the current directory, such as `sample.yml`.

After the command completes, you can copy or move the generated file to the desired directory with the correct name.

```
[user@host ~]$ ansible-navigator settings --effective \
--eei aap.lab.example.com/ee-supported-rhel8 \
--pp missing > /tmp/ansible-navigator.yml
```

The generated file starts with an `ansible-navigator` key, which contains subkeys representing the top-level configuration categories and settings. Top-level settings include `ansible` for Ansible-specific settings, and `execution-environment` for the automation execution environment settings that `ansible-navigator` uses.

```
---
ansible-navigator:
...output omitted...
  execution-environment:
    container-engine: podman
    enabled: true
    image: aap.lab.example.com/ee-supported-rhel8:latest
    pull:
      policy: missing
...output omitted...
```

Setting a Default Automation Execution Environment

Use the `execution-environment` key to set default values for a number of the options that `ansible-navigator` accepts to control automation execution environments.

`image`

This option specifies the container image to use for the automation execution environment, and sets the default for the `--execution-environment-image` (or `--eei`) option.

`pull`

This option specifies when and how to pull the container image. Configuring the `policy` key is the equivalent to using the `--pull-policy` (or `--pp`) option. You can also pass additional arguments, such as disabling TLS verification.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

`enabled`

This option specifies whether to use an automation execution environment or not and defaults to `true`.

For example, consider the following `ansible-navigator` command:

```
[user@host ~]$ ansible-navigator run site.yml --eei ee-29-rhel8 --pp always
```

You can eliminate the `--eei` and `--pp` options if you create an `ansible-navigator.yml` file that contains the following content, in the same directory as the `site.yml` file:

```
---
ansible-navigator:
  execution-environment:
    image: ee-29-rhel8:latest
    pull:
      policy: always
```

By creating a file with those settings, your `ansible-navigator` commands no longer require the `--eei` or `--pp` options:

```
[user@host ~]$ ansible-navigator run site.yml
```

Setting the Default Mode to Standard Output

The `mode` key takes the same options as the `--mode` (`-m`) option, and defaults to `interactive` mode. If you prefer to work in `stdout` mode, you can specify this with an `ansible-navigator.yml` file that has the following content:

```
---
ansible-navigator:
  mode: stdout
```

With this configuration, you need to specify the `-m interactive` option explicitly if you want to run in interactive mode.

Disabling Playbook Artifacts

The basic design of `ansible-navigator` assumes that you cannot provide interactive input to the playbook while it is running. This makes sense because that is also true when you run a playbook in automation controller.

Automation content navigator also records *playbook artifact* files for each run of the playbook. These files record information about the playbook run and can be used to review the results of the run when it completes, to help troubleshoot issues, or be kept for compliance purposes. You can review the contents of these files with the `ansible-navigator replay` *`filename`* command. However, the contents of these files might contain sensitive information about the playbook run, especially if you are providing authentication-related input to the playbook.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

If you often run playbooks with interactive password prompts or playbooks that use modules that prompt you for input, you can disable the generation of playbook artifacts with a configuration file such as the following example:

```
---
ansible-navigator:
  playbook-artifact:
    enable: false
```

Alternatively, you can temporarily disable playbook artifacts by running the `ansible-navigator run` command with the `--pae false` option.

An Example Settings File

The following `ansible-navigator.yml` file configures some common settings:

```
---
ansible-navigator:
  ansible:
    config:
      path: ./ansible.cfg

  editor: (1)
    command: /bin/emacs

  execution-environment: (2)
    image: aap.lab.example.com/ee-supported-rhel8:latest
    pull:
      policy: missing

  playbook-artifact: (3)
    enable: false
```

(1) You can configure `ansible-navigator` to use the text editor of your choice.
(2) By default, the `ansible-navigator` command looks for the `registry.redhat.io/ansible-automation-platform-25/ee-supported-rhel8:latest` image. You can configure it to use a specific automation execution environment by defining the `execution-environment` section in the settings file.
(3) To enable the `ansible-navigator` command to prompt for passwords, you can disable playbook artifacts.

More information about the available options and how to use them is available in the references at the end of this section.

References
Using Content Navigator

ansible-navigator Settings

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

4.4. Guided Exercise
Configuring Automation Content Navigator

Change settings for automation content navigator by creating and editing its configuration file.

Outcomes

- Configure automation content navigator by using an `ansible-navigator.yml` file.
- Identify the sequence of locations that automation content navigator checks for its configuration file.
- Configure automation content navigator to prompt the user for an Ansible Vault password.

Prerequisites

As the `student` user on the `workstation` machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
[student@workstation ~]$ lab start config-navigator
```

Instructions

1. Review the Ansible configuration files, inventories, and the playbook in the `/home/student/config-navigator` directory. Modify the Ansible configuration files to use the `secret-pass` vault password file.

   1.1. Change to the `/home/student/config-navigator` directory:

   ```
   [student@workstation ~]$ cd ~/config-navigator/
   [student@workstation config-navigator]$
   ```

   1.2. Display the contents of the `ansible1.cfg` and `inventory1` files. Notice how the `ansible1.cfg` configuration file sets the inventory location to `inventory1`.

   ```
   [defaults]
   inventory = inventory1
   ```

   The `inventory1` file defines the `devservers` host group with `serverb.lab.example.com` as the only managed host.

   ```
   [devservers]
   serverb.lab.example.com
   ```

   1.3. Display the contents of the `ansible2.cfg` and `inventory2` files. Notice how the `ansible2.cfg` configuration file sets the inventory location to `inventory2`.

```
[defaults]
inventory = inventory2
```

The `inventory2` file defines the `devservers` host group with `servera.lab.example.com` as the only managed host.

```
[devservers]
servera.lab.example.com
```

1.4. Display the contents of the `create_users.yml` playbook:

```
---
- name: Create user accounts in devservers
  hosts: devservers
  become: true
  remote_user: devops
  gather_facts: false
  vars_files:
    - secret.yml

  tasks:
    - name: Creating user from secret.yml
      ansible.builtin.user:
        name: "{{ username }}"
        password: "{{ pwhash }}"
```

This playbook uses the variables defined in the `secret.yml` encrypted file.

1.5. Use the `ansible-vault view` command to display the contents of the `secret.yml` file. Use the password in the `secret-pass` file to decrypt the `secret.yml` file:

```
[student@workstation config-navigator]$ cat secret-pass
redhat

[student@workstation config-navigator]$ ansible-vault view secret.yml \
--vault-password-file=secret-pass
username: ansibleuser
pwhash: $6$j...xhP1
```

1.6. In the `ansible1.cfg` file, specify `secret-pass` as the vault password file. The `ansible1.cfg` file should now consist of the following content:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[defaults]
inventory = inventory1
vault_password_file = secret-pass
```

2. Create the `ansible-navigator.yml` settings file and configure it so that `ansible-navigator` uses the `ansible1.cfg` Ansible configuration file, the `aap.lab.example.com/ee-supported-rhel8:latest` automation execution environment, and standard output mode.

   2.1. Use the `ansible-navigator settings --effective` command to generate the `sample.yml` settings file. You can use this file as a starting point to configure automation content navigator.

```
[student@workstation config-navigator]$ ansible-navigator settings --
effective \
-m stdout --eei aap.lab.example.com/ee-supported-rhel8:latest \
--pp never > sample.yml
```

> **Important**
> When you redirect the output of the `ansible-navigator settings --effective` command to a file in the current working directory, you must redirect it to a file with a name other than `ansible-navigator.yml` or the command fails.

   2.2. Rename the `sample.yml` file to `ansible-navigator.yml`.

```
[student@workstation config-navigator]$ mv -v sample.yml ansible-
navigator.yml
renamed 'sample.yml' -> 'ansible-navigator.yml'
```

   2.3. Open the `ansible-navigator.yml` settings file in a text editor, and set the `ANSIBLE_CONFIG` environment variable to `ansible1.cfg` by editing the `environment-variables` subsection of the `execution-environment` section:

```
...output omitted...
  execution-environment:
    container-engine: podman
    enabled: true
    environment-variables:
      set:
        ANSIBLE_CONFIG: /home/student/config-navigator/ansible1.cfg
...output omitted...
```

   2.4. Examine the `ansible-navigator.yml` settings file and notice that the `ansible-navigator` command set the `image` key using the value you specified with the `--eei` option:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
...output omitted...
    image: aap.lab.example.com/ee-supported-rhel8:latest
...output omitted...
```

2.5. Examine the `ansible-navigator.yml` settings file and notice that the `ansible-navigator` command set the `policy` key using the value you specified with the `--pp` option:

```
...output omitted...
    pull:
      policy: never
...output omitted...
```

The updated `ansible-navigator.yml` file now contains the following content:

```
---
ansible-navigator:
...output omitted...
  execution-environment:
    container-engine: podman
    enabled: true
    environment-variables:
      set:
        ANSIBLE_CONFIG: /home/student/config-navigator/ansible1.cfg
    image: aap.lab.example.com/ee-supported-rhel8:latest
    pull:
      policy: never
...output omitted...
```

> **Important**
>
> Be careful with indentation of the lines in the `ansible-navigator.yml` file. The preceding example shows correct indentation for each line.

3. Create a similar `~/.ansible-navigator.yml` settings file, but set the `ANSIBLE_CONFIG` environment variable to `ansible2.cfg`.

Run the `create_users.yml` playbook and confirm that the configuration file in the directory for its project has precedence over the `~/.ansible-navigator.yml` file in your home directory.

3.1. Copy the existing automation content navigator settings file to `~/.ansible-navigator.yml`.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation config-navigator]$ cp ansible-navigator.yml \
~/.ansible-navigator.yml
```

3.2. Edit the new `~/.ansible-navigator.yml` settings file. Set the `ANSIBLE_CONFIG` environment variable to `ansible2.cfg`. The updated `~/.ansible-navigator.yml` file now contains the following content:

```
---
ansible-navigator:
...output omitted...
  execution-environment:
    container-engine: podman
    enabled: true
    environment-variables:
      set:
        ANSIBLE_CONFIG: /home/student/config-navigator/ansible2.cfg
    image: aap.lab.example.com/ee-supported-rhel8:latest
    pull:
      policy: never
...output omitted...
```

3.3. Run the `create_users.yml` playbook by using the `ansible-navigator` command:

```
[student@workstation config-navigator]$ ansible-navigator run
create_users.yml

PLAY [Create user accounts in devservers]
**************************************

TASK [Creating user from secret.yml]
*****************************************
changed: [serverb.lab.example.com]

PLAY RECAP
*********************************************************************
serverb.lab.example.com    : ok=1    changed=1    unreachable=0    failed=0
...
```

> **Note**
>
> When running commands, you do not need to specify any settings that are already defined in the automation content navigator settings file.
>
> The fact that the playbook ran against `serverb` implies that the `./ansible-navigator.yml` file has precedence over the `~/.ansible-navigator.yml` file.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

4. Verify that you can use `~/.ansible-navigator.yml` as the settings file for automation content navigator when there is no configuration file in the local project directory, or when you specify it as the value of the `ANSIBLE_NAVIGATOR_CONFIG` environment variable.

   4.1. Back up the `ansible-navigator.yml` file by moving it to `backup-an.yml`. Run the `create_users.yml` playbook by using the `ansible-navigator` command with the `--pae false` and `--ask-vault-pass` options.

   This time, the playbook runs against the `servera` server, because there is no settings file in the project directory and automation content navigator found the settings file in your home directory.

```
[student@workstation config-navigator]$ mv ansible-navigator.yml backup-
an.yml
[student@workstation config-navigator]$ ansible-navigator run
create_users.yml \
--pae false --ask-vault-pass
Vault password: redhat

PLAY [Create user accounts in devservers]
*************************************

TASK [Creating user from secret.yml]
*****************************************
changed: [servera.lab.example.com]

PLAY RECAP
*********************************************************************
servera.lab.example.com    : ok=1    changed=1    unreachable=0    failed=0
...
```

> **Note**
> The `--ask-vault-pass` option is used because the `vault_password_file` option is not specified in the `ansible2.cfg` file. This option also requires the use of the `--pae false` option to temporarily disable playbook artifacts.

   4.2. Restore the local automation content navigator settings file and set the `ANSIBLE_NAVIGATOR_CONFIG` environment variable to `~/.ansible-navigator.yml`.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation config-navigator]$ mv backup-an.yml ansible-
navigator.yml
[student@workstation config-navigator]$ export \
ANSIBLE_NAVIGATOR_CONFIG=~/.ansible-navigator.yml
```

4.3. Run the `create_users.yml` playbook by using the `ansible-navigator` command. As expected, the playbook runs against the `servera` server, even though the local `./ansible-navigator.yml` settings file is present, because the `ANSIBLE_NAVIGATOR_CONFIG` environment variable takes precedence.

```
[student@workstation config-navigator]$ ansible-navigator run
create_users.yml \
--pae false --ask-vault-pass
Vault password: redhat

PLAY [Create user accounts in devservers]
*************************************

TASK [Creating user from secret.yml]
*****************************************
ok: [servera.lab.example.com]

PLAY RECAP
*********************************************************************
servera.lab.example.com    : ok=1    changed=0    unreachable=0    failed=0
...
```

4.4. Unset the `ANSIBLE_NAVIGATOR_CONFIG` environment variable so that the project's `./ansible-navigator.yml` settings file is used for the remaining exercises:

```
[student@workstation config-navigator]$ unset ANSIBLE_NAVIGATOR_CONFIG
```

5. You might prefer to enter the password to decrypt the `secret.yml` file as user input when you run the `create_users.yml` playbook instead of using the `--pae false` option every time. Prepare the Ansible and automation content navigator settings files to prompt for the Ansible Vault password.

5.1. Remove the line for `vault_password_file` in the `ansible1.cfg` file. The `ansible1.cfg` file should now consist of the following content:

```
[defaults]
inventory = inventory1
```

5.2. In the `ansible-navigator.yml` file, disable creating playbook artifacts by setting the `enable` key to `false`:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
---
...output omitted...
  mode: stdout
  playbook-artifact:
    enable: false
    save-as: '{playbook_dir}/{playbook_name}-artifact-{time_stamp}.json'
...output omitted...
```

> **Important**
>
> Disabling playbook artifact creation enables you to run the playbook interactively. Running interactively is useful when you want to use options such as `--ask-vault-pass` on the command line.
>
> However, disabling artifact creation removes the ability to run `ansible-navigator replay`.

5.3. Run the `create_users.yml` playbook using the `ansible-navigator` command. This time, specify the `--ask-vault-pass` option to prompt for the Ansible Vault password:

```
[student@workstation config-navigator]$ ansible-navigator run
create_users.yml \
--ask-vault-pass
Vault password: redhat

PLAY [Create user accounts in devservers]
*************************************

TASK [Creating user from secret.yml]
*****************************************
ok: [serverb.lab.example.com]

PLAY RECAP
*********************************************************************
serverb.lab.example.com    : ok=1    changed=0    unreachable=0    failed=0
...
```

Finish

On the `workstation` machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish config-navigator
```


For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

4.5. Developing Content in Visual Studio Code

Objectives

- Use Visual Studio Code to develop and run automation content and synchronize changes with a remote Git repository.

Using Visual Studio Code to Create Automation Content

Ansible uses text files and you can create these files using any text editor. This course provides Visual Studio Code (VS Code), available from https://code.visualstudio.com, as an option for creating your automation content.

Overview of the VS Code Interface

The VS Code interface contains five primary components:

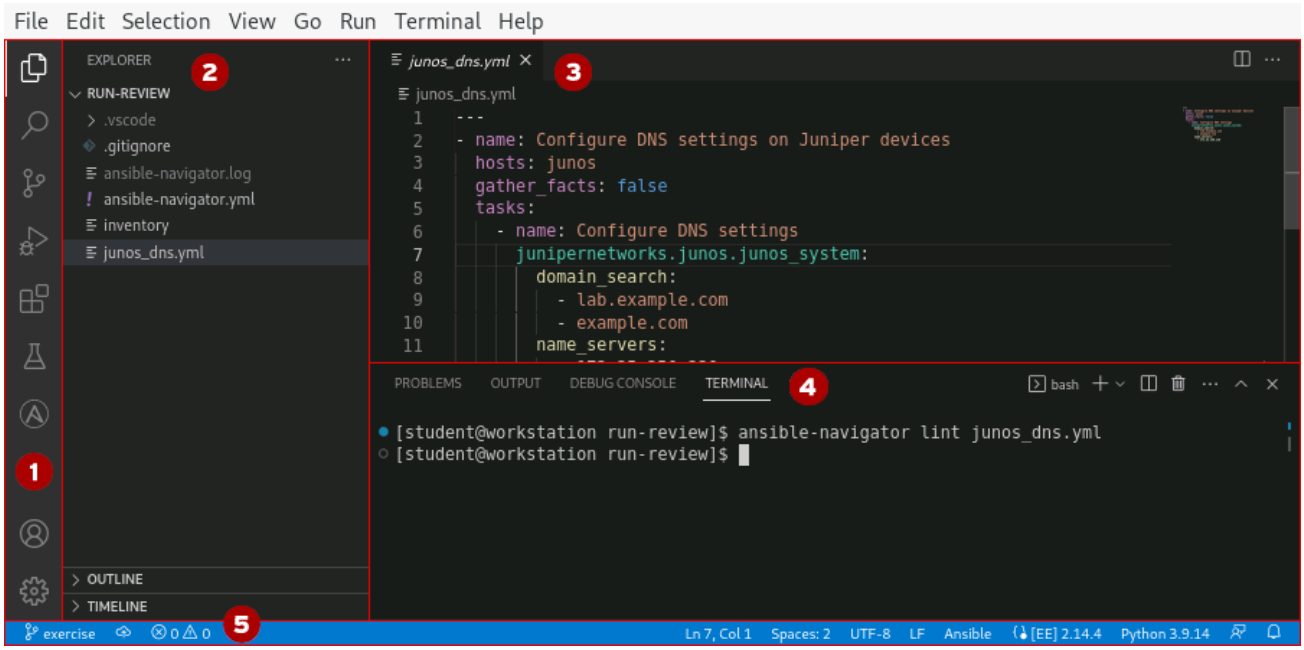


Figure 32. The VS Code user interface

1. **Activity Bar**

   At the far left, the Activity Bar contains shortcuts to change the view of the Side Bar. By default, the Activity Bar contains shortcuts for the Explorer, Search, Source Control, Run and Debug, and Extensions views.

   You can also access these activity views from the View menu. You can expand or collapse the Side Bar by clicking a shortcut icon more than one time.

2. **Side Bar**

   Located immediately to the right of the Activity Bar, the Side Bar area displays the selected Activity view, such as the Explorer view.

3. **Editor Groups**

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

The upper right region of VS Code contains one or more groups of editors. By default, there is only one editor group. Any active editor takes up the entire region of the editor group.

Click the **Split Editor Right** icon in the upper right to create a second editor group. With two editor groups, you can edit two different files side by side.

4. **Panels**

   Located below the editors, individual panels provide different output or debugging information for activities in VS Code. VS Code includes the following panels:

   - PROBLEMS
   - OUTPUT
   - DEBUG CONSOLE
   - TERMINAL
   - PORTS

   Additional panels, such as the ANSIBLE panel, might also be available.

5. **Status Bar**

   At the bottom, the Status Bar area provides information about the open project and files as you edit.

Using the VS Code Integrated Terminal

VS Code integrates the terminal from your operating system, enabling you to execute terminal commands in VS Code. For example, you might use the `ansible-navigator run` command to run an Ansible Playbook in the integrated terminal. To view the integrated terminal, click **View > Terminal**.

If you need more space to view the terminal, then you can modify its size by dragging the border between the Editor Groups and Panels, or by clicking the **Maximize Panel Size** icon (^).

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

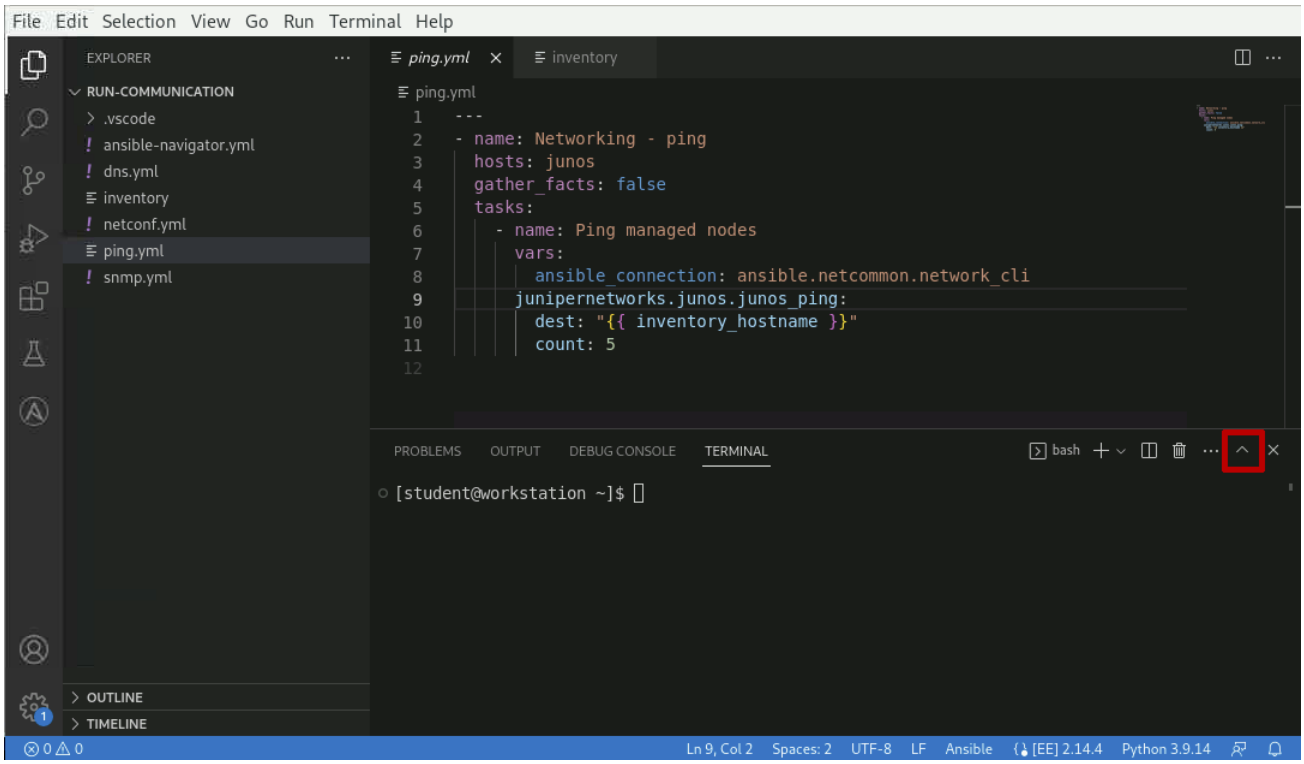


Figure 33. Maximizing a VS Code terminal

If you prefer to open a terminal in the **Editor Groups** location by default, then click **File › Preferences › Settings**, search for `Terminal > Integrated: Default Location`, and then select **editor** from the menu.

Configuring VS Code

You can customize VS Code by modifying its various settings and by using extensions.

Configuring Extensions

VS Code provides an extension framework to encourage the integration of software development capabilities. Any user or organization can contribute extensions. After an extension is developed, it is advertised and published on the VS Code marketplace.

The Ansible (`redhat.ansible`) and YAML (`redhat.vscode-yaml`) VS Code extensions have been installed in the classroom environment. You can search for VS Code extensions at `https://marketplace.visualstudio.com`, or manage extensions by navigating to **View › Extensions** within VS Code.

> **Important**
> Although Red Hat provides some VS Code extensions, VS Code is a third-party tool and is not supported by Red Hat.

You can configure the Ansible extension from within VS Code by selecting **View › Extensions** and then clicking the **Ansible** extension. Click the **Manage** icon and then click **Settings**.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

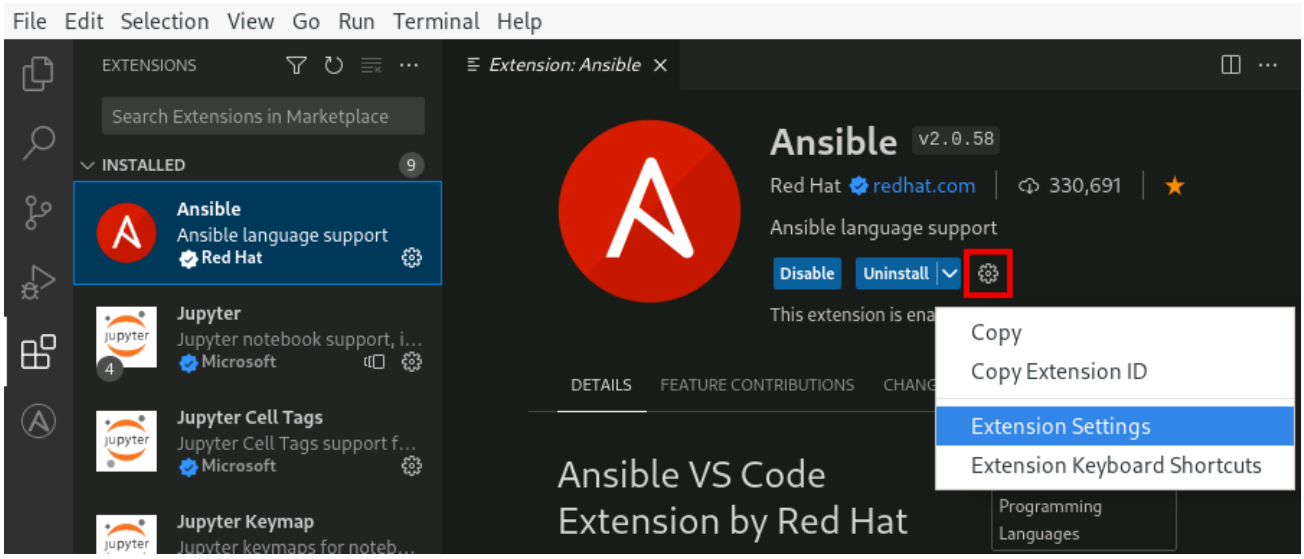


Figure 34. Configure the Ansible extension in VS Code

Hands-on activities in the classroom environment configure the following settings for the Ansible extension:

- Enable validation
- Enable Ansible lint
- Enable an automation execution environment

Configuring Settings

In addition to extension settings, you can update general settings. Go to **File › Auto Save** to automatically save file changes. Go to **File › Preferences › Settings** to update settings, such as font size, tab size, and how the editor renders white space.

Starting the Git Workflow in VS Code

Many hands-on activities in this course interact with remote Git repositories. Those activities typically involve the following actions:

- Cloning a Git repository
- Creating a branch
- Staging files for a commit
- Committing staged files to the Git repository
- Synchronizing changes

Cloning a Git Repository

To clone a Git repository in VS Code, click the **Source Control** icon in the Activity Bar or select **View › Source Control**.

Click **Clone Repository** and then enter the repository URL.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

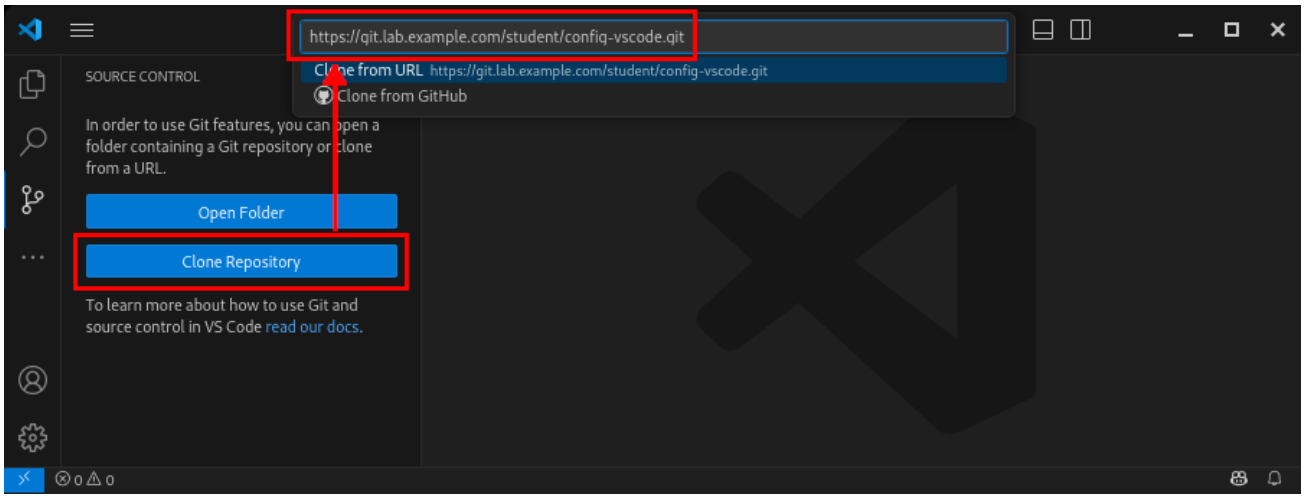


Figure 35. Cloning a repository in VS Code

VS Code prompts you to select a repository destination. If necessary, then you can create a directory, such as the `/home/student/git-repos` directory.

Creating a Branch

The status bar displays information about the state of the repository, including the name of the current branch. To create a branch, click the current branch name, such as `main`, enter the name of the new branch, and press **Enter**.

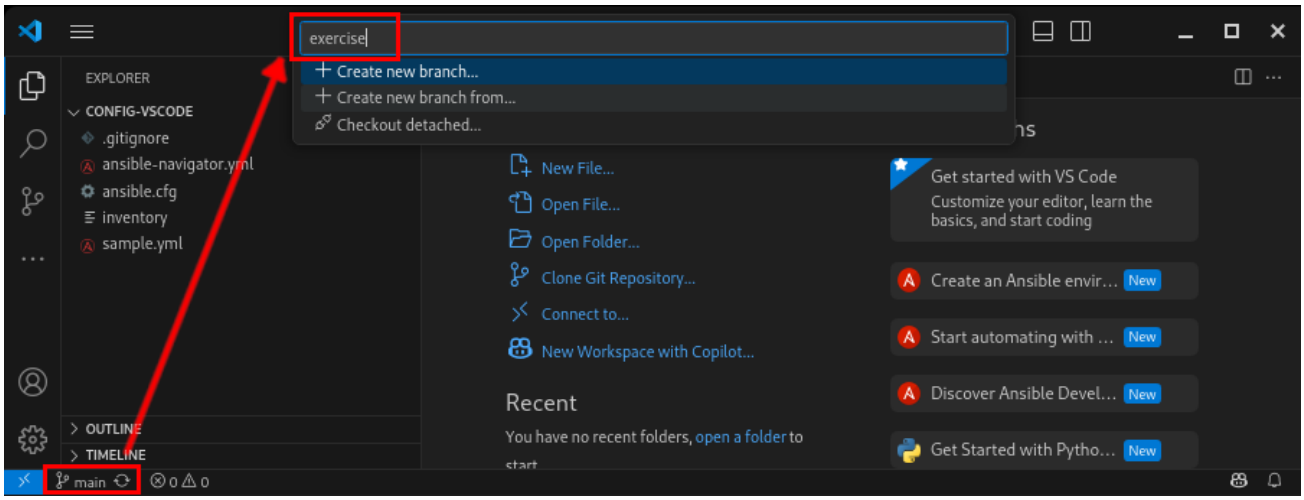


Figure 36. Creating a repository branch in VS Code

The status bar updates to display the current branch name. To switch between existing branches, click the current branch name and then select the desired branch.

Staging Files for a Commit

As you work, you create files and modify existing files in the working tree. This changes the working tree to a *dirty* state, which means that it contains modifications which have not been committed to the current branch.

To view staged files in VS Code, click the **Source Control** icon in the Activity Bar.

To include a file in the next commit, click the plus (+) symbol next to the name of the file in the Source Control view to add the file to the Staged Changes section. Click the minus (-) symbol to remove changed files from the Staged

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Changes section.

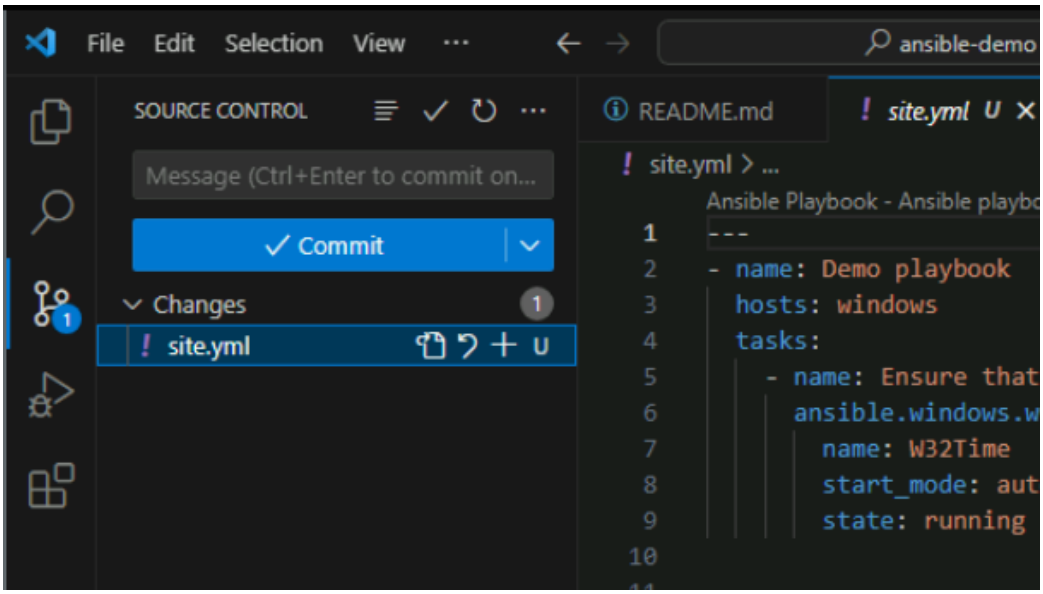


Figure 37. Staging a file

Committing Staged Files to the Git Repository

If you stage a file in the Source Control view, then VS Code opens a text entry line above the **Commit** button where you can enter a commit message describing your changes.

To commit your staged changes, type a commit message in the message box and then click **Commit**. This saves your changes to the local Git repository.

Commit messages do not have to be long, but they should be meaningful so that they are useful.

Synchronizing Changes

One common way to coordinate work with Git is for all developers to push their work to the same, shared, remote repository. This is also how you provide your Ansible automation code to automation controller.

You *push* to upload commits you made in your local repository to the remote repository. To push your changes to the remote repository using VS Code, click the More Actions menu (...) for your Git repository under **Source Control Repositories** in the Side Bar, and then select **Push**, or choose **Push** from its Pull, Push submenu.

To pull changes from the remote repository when using VS Code, click the More Actions menu (...) and then select **Pull**, or choose **Pull** from the Pull, Push submenu.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

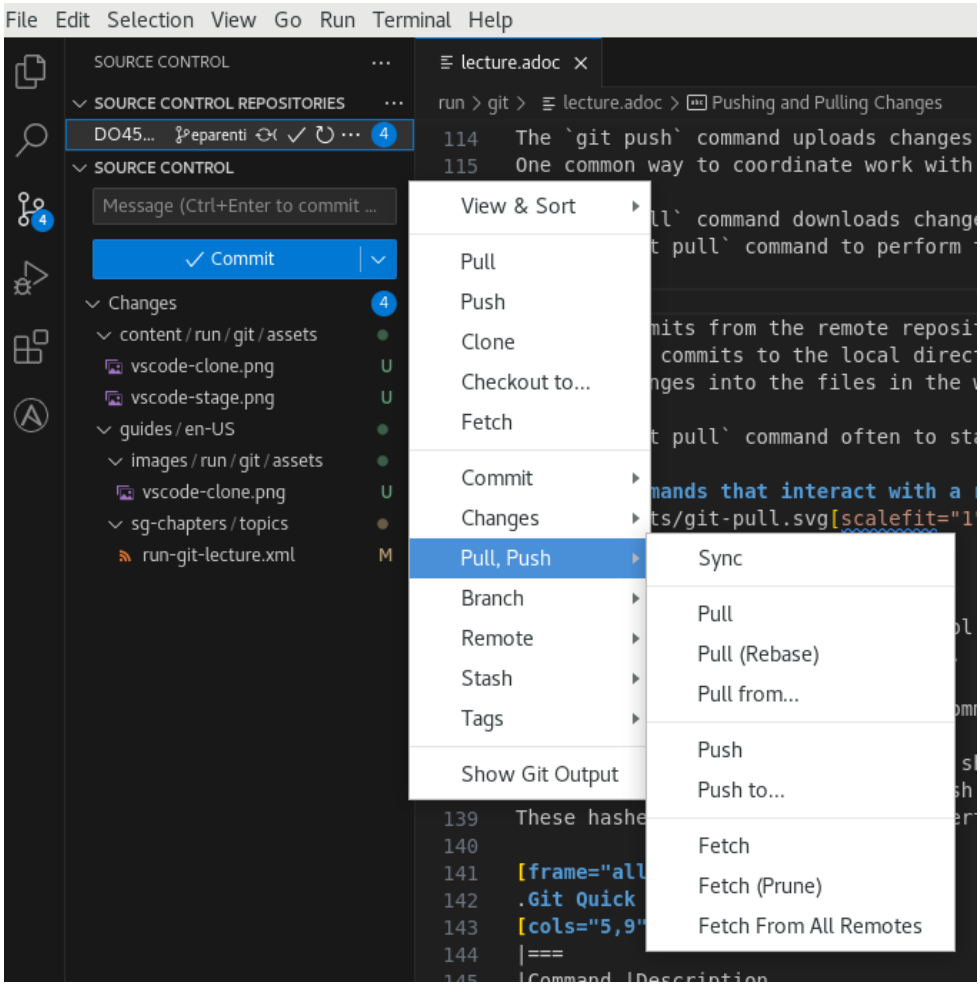


Figure 38. Synchronizing local changes with a remote repository

Important

There is also an action on the Status Bar, in the lower-left corner of the VS Code application window, that can be used to "synchronize" changes. Clicking it pulls changes from the remote repository, then pushes your changes to the remote repository. This "Synchronize Changes" (or **Source Control Sync**) action is demonstrated in more detail later in this section.

References

Automation Content Navigator Creator Guide

Visual Studio Code

Using Git Source Control in VS Code

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

4.6. Guided Exercise
Developing Content in Visual Studio Code

Perform essential playbook development tasks in Visual Studio Code.

Outcomes

- Perform common Git actions in Visual Studio Code.
- Create files and directories in Visual Studio Code.

Prerequisites

As the `student` user on the `workstation` machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
[student@workstation ~]$ lab start config-vscode
```

Instructions

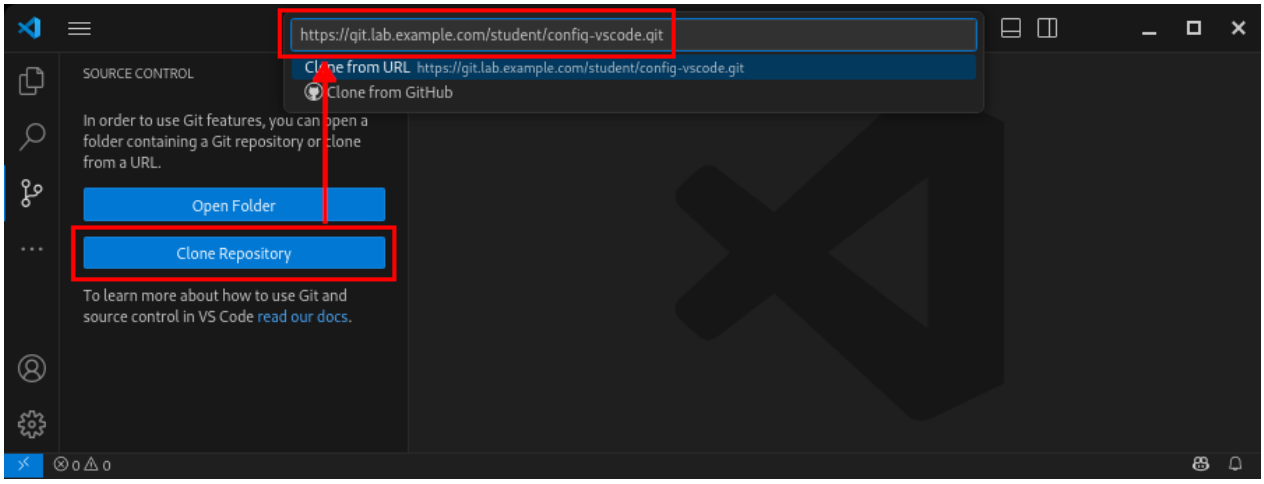
1. Clone the `https://git.lab.example.com/student/config-vscode.git` repository to the `/home/student/git-repos` directory and then create the `exercise` branch.

   1.1. To open VS Code, click **Activities** in the top bar and either type `code` in the search field and press `Enter`, or click the **Visual Studio Code** icon in the favorites list.

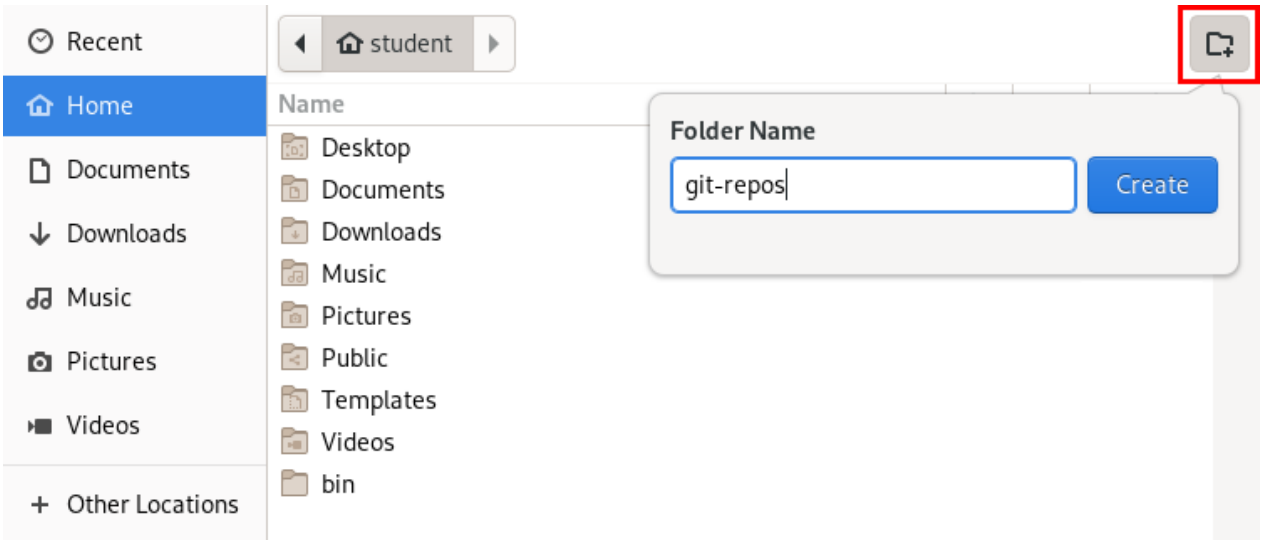
   1.2. If prompted, then enter `student` for the `Default keyring` password.

   1.3. Click the **Source Control** icon in the Activity Bar to display the Source Control panel.

   1.4. Click **Clone Repository**, enter `https://git.lab.example.com/student/config-vscode.git` as the repository URL, and then press `Enter`.



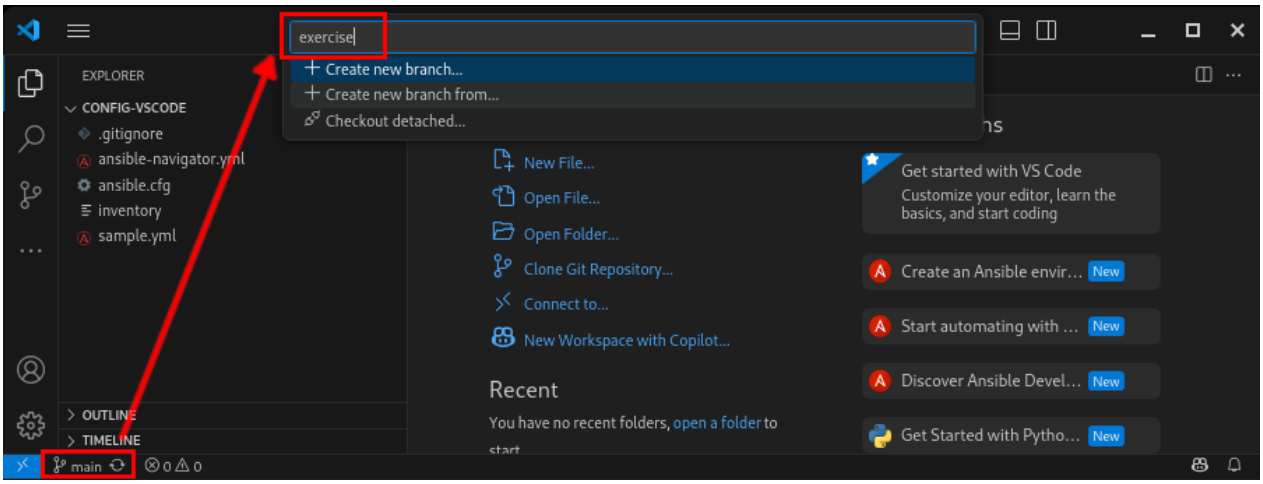
   1.5. If the `/home/student/git-repos` directory does not exist, then create the directory and select it as the repository destination.



1.6. Click **Open** to open the files from the cloned repository.

1.7. If prompted, select `Trust the authors of all files in the parent folder 'git-repos'` and then click **Yes, I trust the authors**.

1.8. Create the branch by clicking **main** in the Status Bar, enter `exercise` for the new branch name, and then press `Enter`.

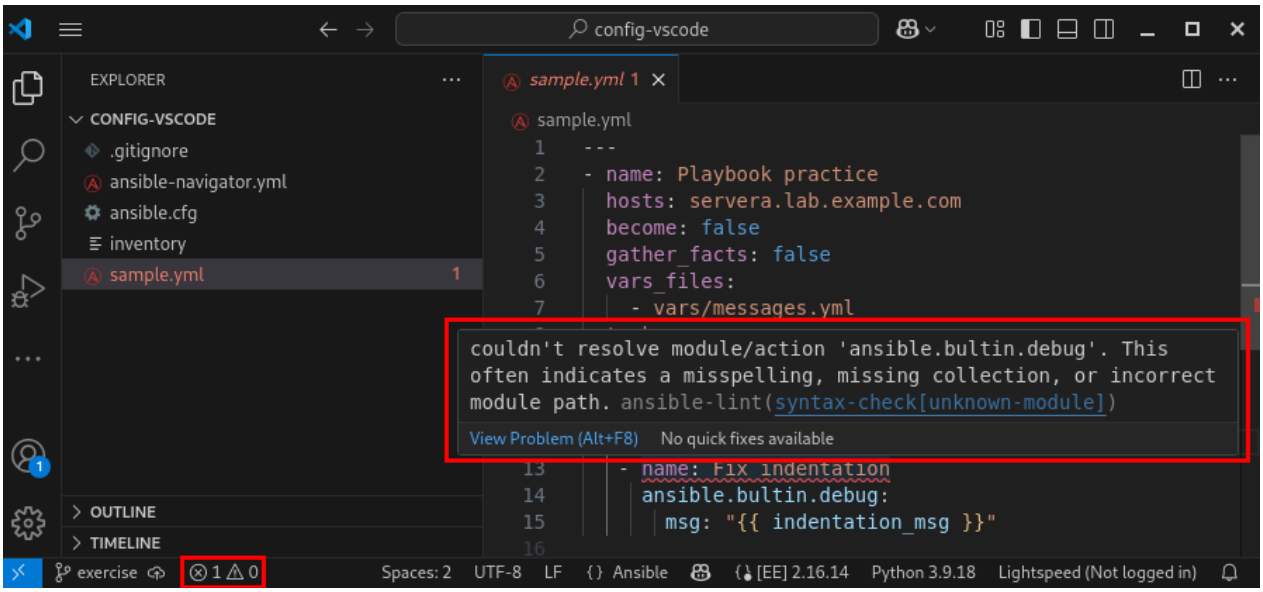


2. Update the `sample.yml` playbook to fix the misspelling identified by Ansible lint.

2.1. Click the `sample.yml` playbook in the Explorer view of the Side Bar.

2.2. As soon as you open the file, VS Code performs Ansible lint checks and identifies an error. You can see this error listed in the Status Bar. Hover your mouse over the squiggly line in the second task.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use



Ansible lint indicates that the `ansible.bultin.debug` module contains a mistake and is likely misspelled.

2.3. Update the module name used in the second task to correct the spelling mistake. The second task now contains the following content:

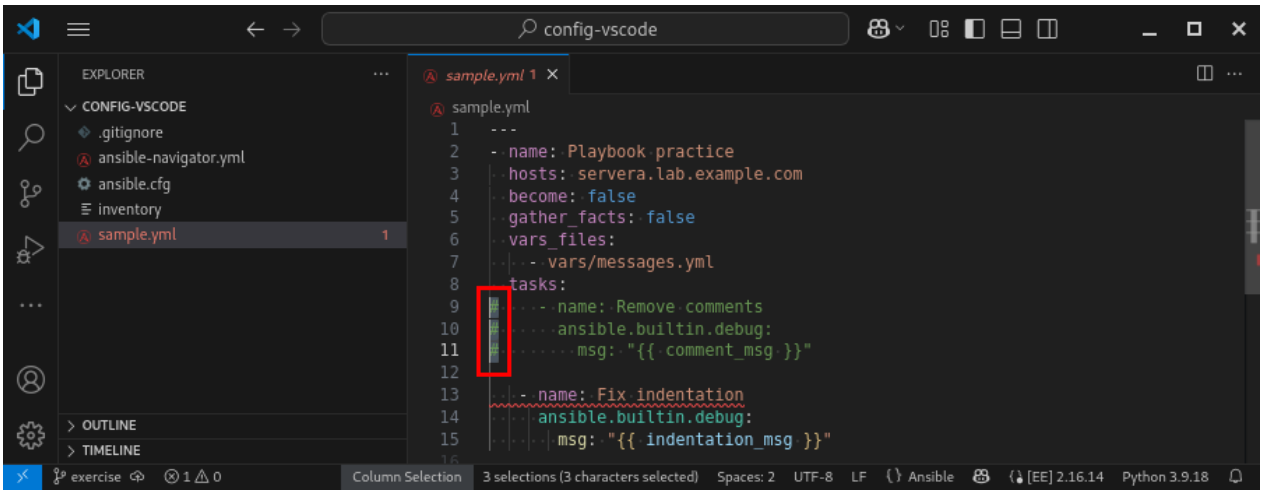
```
...output omitted...
   - name: Fix indentation
     ansible.builtin.debug:
       msg: "{{ indentation_msg }}"
```

3. Update the `sample.yml` playbook to remove the number sign (#) from each line in the first task.

3.1. Although you could remove the number signs from each individual line, column selection mode in VS Code can make this action easier. Click Selection. If **Column Selection Mode** is not enabled (it does not have a checkmark next to it), then click **Column Selection Mode**.

3.2. Select the number signs:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use



3.3. Press `Delete` to remove the highlighted number signs in one action.

3.4. Click **Selection > Column Selection Mode** to disable column selection mode.

4. Update the `sample.yml` playbook to fix the indentation of the playbook tasks. Notice that the indentation of the second task does not match the indentation of the first task.

4.1. Highlight the lines in the second task and then press `Tab` to increase the indentation level of each highlighted line.

> **Note**
> For some playbooks, you might need to press `Tab` multiple times until you achieve the correct indentation. You can also press `Shift` + `Tab` to reduce the indentation level of each highlighted line.

4.2. Click **File > Save**.

5. Create the `/home/student/git-repos/config-vscode/vars/messages.yml` file. The `sample.yml` playbook uses this file.

5.1. In the Explorer view of the Side Bar, hover your mouse over **CONFIG-VSCODE**, click the **New Folder** icon, enter `vars` as the directory name, and then press `Enter`.

5.2. Right-click the **vars** directory, select **New File**, enter `messages.yml` as the file name, and then press `Enter`.

5.3. Ensure that the `messages.yml` file consists of the following content and then save the file. The `sample.yml` playbook uses both the `comment_msg` and `indentation_msg` variables:

```
---
comment_msg: Comments removed from the first task.
indentation_msg: Indentation fixed for the second task.
```

6. Run the `sample.yml` playbook.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

6.1. Click **View > Terminal**.

6.2. Run the following command from the integrated VS Code terminal:

```
[student@workstation config-vscode]$ ansible-navigator run sample.yml

PLAY [Playbook practice]
*************************************************************

TASK [Remove comments]
*************************************************************
ok: [servera.lab.example.com] => {
    "msg": "Comments removed from the first task."
}

TASK [Fix indentation]
*************************************************************
ok: [servera.lab.example.com] => {
    "msg": "Indentation fixed for the second task."
}

PLAY RECAP
*************************************************************************
servera.lab.example.com    : ok=2    changed=0    unreachable=0    failed=0
...
```

7. Commit the changes that you made in this exercise and then synchronize those changes with the remote Git repository.

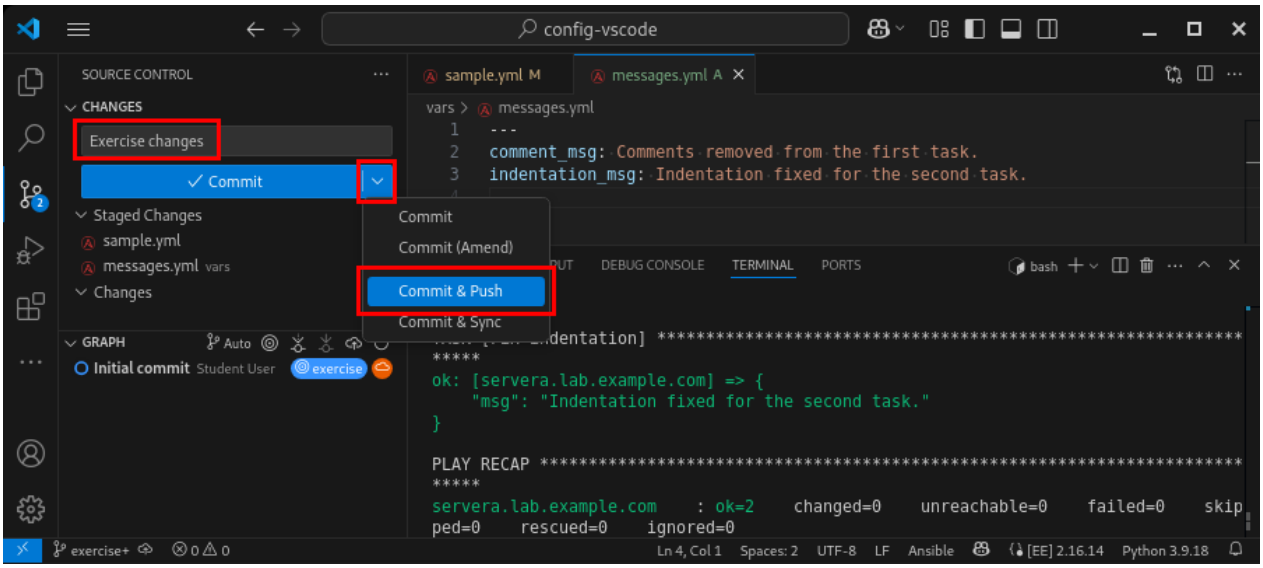
7.1. Click the **Source Control** icon in the Activity Bar to display the Source Control panel.

7.2. Hover your mouse over the `sample.yml` file and then click **Stage changes** (the + symbol). The modified file is now staged for the next commit.

7.3. Hover your mouse over the `messages.yml` file and then click **Stage changes** (the + symbol). The new file is now staged for the next commit.

7.4. Type `Exercise changes` in the message field above the **Commit** button and then click **Commit > Commit & Push**.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use



7.5. Click **Ok** to agree to publish the `exercise` branch, and then enter `Stud3nt123` as the password.

7.6. Click **File > Close Folder**.

Finish

On the `workstation` machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish config-vscode
```

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

4.7. Lab
Working with Ansible Configuration Settings

Configure automation content navigator to use a specific image by default and then use it to examine the current Ansible configuration.

Outcomes

- Configure automation content navigator by using an `ansible-navigator.yml` file.
- Configure Ansible by using an `ansible.cfg` file.
- Use the `ansible-navigator` command to review your current Ansible configuration.

Prerequisites

As the `student` user on the `workstation` machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
[student@workstation ~]$ lab start config-review
```

Instructions

1. Change to the `/home/student/config-review` directory and create the `ansible-navigator.yml` file. Configure that file to use the `aap.lab.example.com/ee-supported-rhel8:latest` image by default and set the pull policy to `missing`.

   > **Note**
   > You might use the `ansible-navigator settings` command (with options) to generate an initial automation content navigator settings file.

2. Configure the automation content navigator settings file to disable creating playbook artifacts.

3. Use automation content navigator to determine how to specify the maximum number of forks Ansible uses to execute tasks on target hosts. Identify the name of the variable and to which section the variable belongs. Modify the `~/config-review/ansible.cfg` Ansible configuration file to set the maximum number of forks to 15. Use automation content navigator to verify the change.

4. Use the `ansible-navigator` command with the `--ask-become-pass` option to run the `install-web.yml` playbook. When prompted, enter `student` as the privilege escalation password. Verify that the playbook runs successfully.

Evaluation

As the `student` user on the `workstation` machine, use the `lab` command to grade your work. Correct any reported failures and rerun the command until successful.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation ~]$ lab grade config-review
```

Finish

As the `student` user on the `workstation` machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish config-review
```

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Solution

Configure automation content navigator to use a specific image by default and then use it to examine the current Ansible configuration.

Outcomes

- Configure automation content navigator by using an `ansible-navigator.yml` file.
- Configure Ansible by using an `ansible.cfg` file.
- Use the `ansible-navigator` command to review your current Ansible configuration.

Prerequisites

As the `student` user on the `workstation` machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
[student@workstation ~]$ lab start config-review
```

Instructions

1. Change to the `/home/student/config-review` directory and create the `ansible-navigator.yml` file. Configure that file to use the `aap.lab.example.com/ee-supported-rhel8:latest` image by default and set the pull policy to `missing`.

> **Note**
> You might use the `ansible-navigator settings` command (with options) to generate an initial automation content navigator settings file.

   1.1. Change to the `/home/student/config-review` directory:

```
[student@workstation ~]$ cd ~/config-review
[student@workstation config-review]$
```

   1.2. Run the following `ansible-navigator settings` command to generate an automation content navigator settings file. Include the `--eei` option to specify the automation execution environment image and the `--pp` option to specify the pull policy. Redirect the command output to a file, such as `sample.yml`, but do not redirect the output to the `ansible-navigator.yml` file:

```
[student@workstation config-review]$ ansible-navigator settings --effective
\
--eei aap.lab.example.com/ee-supported-rhel8:latest --pp missing \
--mode stdout > sample.yml
```

1.3. Move the file generated in the previous step to the `ansible-navigator.yml` file:

```
[student@workstation config-review]$ mv sample.yml ansible-navigator.yml
```

2. Configure the automation content navigator settings file to disable creating playbook artifacts.

2.1. Edit the `~/config-review/ansible-navigator.yml` automation content navigator settings file. In the `playbook-artifact` section, change the value of the `enable` directive to `false`.

```
...output omitted...
  playbook-artifact:
    enable: false
...output omitted...
```

> **Note**
> You must disable creating playbook artifacts so that Ansible can prompt for a privilege escalation password. Because the project and playbook do not define a remote user, Ansible attempts to connect as the user who runs the playbook and the `student` user must authenticate to run `sudo` commands.

3. Use automation content navigator to determine how to specify the maximum number of forks Ansible uses to execute tasks on target hosts. Identify the name of the variable and to which section the variable belongs. Modify the `~/config-review/ansible.cfg` Ansible configuration file to set the maximum number of forks to 15. Use automation content navigator to verify the change.

3.1. Run the `ansible-navigator config` command and specify the `interactive` mode:

```
[student@workstation config-review]$ ansible-navigator config \
--mode interactive

    Name                            Default Source  Current
  0|Action warnings                 True    default True
  1|Agnostic become prompt          True    default True
  2|Ansible connection path         True    default None
...output omitted...
```

3.2. Type `:f forks` and then press `Enter`.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
  Name                          Default         Source      Current
0|Default forks                 True            default     5
```

3.3. Press `0` to inspect the `Default forks` configuration:

```
Default forks (current/default: 5)
 0|---
 1|current_config_file: /home/student/config-review/ansible.cfg
 2|current_value: 5
 3|default: true
 4|default_value: 5
 5|description: Maximum number of forks Ansible will use to execute tasks
...
 6|env:
 7|- name: ANSIBLE_FORKS
 8|ini:
 9|- key: forks  (1)
10|  section: defaults  (2)
11|name: Default forks
12|option: DEFAULT_FORKS
13|source: default
14|type: integer
15|via: default
```

(1) The `forks` key matches the name of the setting in the `ansible.cfg` file.

(2) The `defaults` section matches the name of the section that the variable belongs to in the `ansible.cfg` file.

Exit from `ansible-navigator` by pressing Esc twice.

3.4. Edit the provided `~/config-review/ansible.cfg` Ansible settings file and append the following `forks` line to the `[defaults]` section:

```
[defaults]
inventory = inventory
forks = 15
```

3.5. Use automation content navigator to verify the change:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation config-review]$ ansible-navigator config \
--mode interactive

    Name                            Default Source  Current
  0|Action warnings                 True    default True
  1|Agnostic become prompt          True    default True
  2|Ansible connection path         True    default None
...output omitted...
```

3.6. Type `:f forks` and then press Enter.

```
  Name            Default  Source
Current
0|Default forks   False    /home/student/config-review/ansible.cfg      15
```

Exit from `ansible-navigator` by pressing Esc.

4. Use the `ansible-navigator` command with the `--ask-become-pass` option to run the `install-web.yml` playbook. When prompted, enter `student` as the privilege escalation password. Verify that the playbook runs successfully.

4.1. Run the `install-web.yml` playbook and prompt for the privilege escalation password:

```
[student@workstation config-review]$ ansible-navigator run install-web.yml \
--ask-become-pass
BECOME password: student

PLAY [Install web server]
*************************************************

TASK [Install httpd package]
*************************************************
changed: [servera.lab.example.com]

PLAY RECAP
*********************************************************************
servera.lab.example.com    : ok=1    changed=1    unreachable=0    failed=0
...
```

Evaluation

As the `student` user on the `workstation` machine, use the `lab` command to grade your work. Correct any reported failures and rerun the command until successful.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation ~]$ lab grade config-review
```

Finish

As the `student` user on the `workstation` machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish config-review
```

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

4.8. Summary

- The `ansible-navigator config` command helps you explore and analyze the current configuration used by the `ansible-navigator run` command.
- Automation content navigator can use only `ansible.cfg` files that it has access to from inside the automation execution environment. These files include the `/etc/ansible/ansible.cfg` file in the execution environment container and the `ansible.cfg` file in your project directory.
- You can configure automation content navigator settings by using the file defined by the `ANSIBLE_NAVIGATOR_CONFIG` environment variable, the `ansible-navigator.yml` configuration file in your current directory, or the `~/.ansible-navigator.yml` configuration file in your home directory.
- The `ansible-navigator settings --sample` command generates a sample `ansible-navigator.yml` configuration file.
- The `ansible-navigator settings --effective` command generates a configuration file that reflects your current effective configuration, including options specified on the command line, values of environment variables, and defaults.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Chapter 5.
Managing Inventories

Goal

Manage inventories by using advanced features of Ansible.

Sections

- Managing Dynamic Inventories (and Guided Exercise)
- Writing YAML Inventory Files (and Guided Exercise)
- Managing Variables (and Guided Exercise)

Lab

- Managing Inventories

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

5.1. Managing Dynamic Inventories

Objectives

- Describe what dynamic inventories are, and install and use an existing script or plug-in as an Ansible dynamic inventory source.

Generating Inventories Dynamically

Static inventory files, which you have worked with so far, are convenient for managing small infrastructures. However, they require manual updates and can become a maintenance burden, especially in environments where hosts are created and destroyed on a regular basis. This can be inconvenient or challenging, especially when an organization wants to run the playbooks against hosts that are dynamically created in a virtual or cloud computing environment.

To automate inventory management in such dynamic environments, Ansible supports dynamically generated inventories.

Dynamic inventories are scripts or plug-ins that query external sources, such as cloud provider APIs, Cobbler, LDAP directories, or a configuration management database (CMDB), to build an up-to-date list of hosts and host groups. This approach is ideal for large or constantly changing infrastructures.

Ansible supports two methods for generating inventories dynamically:

- Inventory plug-ins
- Inventory scripts

Inventory Plug-ins

An inventory plug-in is a piece of Python code that generates an inventory object from a dynamic source. Ansible ships with inventory plug-ins for a number of external inventory sources, including:

- Amazon Web Services EC2
- Amazon Web Services RDS
- KubeVirt
- Microsoft Active Directory
- Red Hat Insights
- Terraform

Ansible provides inventory plug-ins through Ansible Content Collections. Third-party content collections provide additional inventory plug-ins for specific products.

Developing inventory plug-ins is beyond the scope of this course.

Using Inventory Plug-ins

For most inventory plug-ins you must prepare a configuration file in YAML format. This configuration file usually provides the connection parameters that the plug-in requires to access the external source.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

For example, the following `satellite.yml` configuration file provides the connection parameters to access a Red Hat Satellite Server installation:

```
plugin: redhat.satellite.foreman   (1)
url: https://satellite.example.com (2)
user: ansibleinventory
password: Sup3r53cr3t
host_filters: 'organization="Development"' (3)
```

1. The required `plugin` keyword indicates which plug-in to use. You must provide its fully qualified collection name (FQCN).
2. The connection parameters are specific to the plug-in. This example provides the Red Hat Satellite Server URL, the username, and the password to use to access the API endpoint.
3. Some plug-ins provide mechanisms to filter the list of hosts. This example uses the `host_filters` parameter to only retrieve the hosts in the `Development` Red Hat Satellite organization.

Every inventory plug-in has documentation with examples. You can access that documentation with the `ansible-navigator doc` command. Use the `--type inventory` option to look up documentation for inventory plug-ins.

Use the `--list` option to list all the inventory plug-ins available in your automation execution environment:

```
[user@host ~]$ ansible-navigator doc --mode stdout --type inventory --list
...output omitted...
amazon.aws.aws_ec2                 EC2 inventory source
amazon.aws.aws_rds                 RDS instance inventory source
ansible.builtin.advanced_host_list Parses a host List with ranges
ansible.builtin.auto               Loads and executes an inventory plugin spe...
ansible.builtin.constructed        Uses Jinja2 to construct vars and groups b...
ansible.builtin.generator          Uses Jinja2 to construct hosts and groups ...
ansible.builtin.host_list          Parses a host list string
...output omitted...
```

Notice that the command only lists the plug-ins from installed Ansible Content Collections. By default, those are the content collections installed in the automation execution environment, or the collections in the directories you define in your `ansible.cfg` configuration file by setting the `collections_path` directive.

To access the documentation for a specific plug-in, use the `ansible-navigator doc` command and provide the FQCN for the plug-in as an argument:

```
[user@host ~]$ ansible-navigator doc --mode stdout --type inventory \
redhat.satellite.foreman
```

To run a playbook that targets hosts in that dynamic inventory, use the `ansible-navigator run` command with the `-i` option to provide the plug-in configuration file:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[user@host ~]$ ansible-navigator run -i satellite.yml my_playbook.yml
```

Remember that you also use that same option for static inventory files. Because Ansible uses the `-i` option for different inventory formats, it parses the file you provide in the following order by default:

1. If the file is executable, then Ansible uses it as an inventory script, as described in the following section.
2. Otherwise, Ansible tries to parse the given file as a configuration file for inventory plug-ins.
3. If that fails, then Ansible uses the file as a static inventory file.

You can control that order by setting the `enable_plugins` directive under the `[inventory]` section in your `ansible.cfg` configuration file.

Developing Inventory Scripts

An inventory script is a program that collects information from an external source and returns the inventory in JSON format. You can write the custom program in any programming language as long as it returns inventory information in JSON format.

> **Important**
> Red Hat recommends developing inventory plug-ins instead of inventory scripts.
>
> Use inventory scripts when you already have legacy scripts that you want to reuse or when you do not want to use Python for your development.

The `ansible-navigator inventory` command is a helpful tool for learning how to write Ansible inventories in JSON format. This command takes an inventory file and returns it in JSON format.

To display the contents of an inventory file in JSON format, use the `--list` option. You can also add the `-i` option to specify the location of the inventory file to process instead of the default inventory set by the current Ansible configuration.

The following example displays the inventory file in its original INI format and then uses the `ansible-navigator inventory` command to process that file and output it in JSON format:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[user@host ~]$ cat inventory
monitor.example.com

[webservers]
web1.example.com
web2.example.com

[databases]
db1.example.com
db2.example.com

[user@host ~]$ ansible-navigator inventory -i inventory --list --mode stdout
{
    "_meta": {
        "hostvars": {}
    },
    "all": {
        "children": [
            "databases",
            "ungrouped",
            "webservers"
        ]
    },
    "databases": {
        "hosts": [
            "db1.example.com",
            "db2.example.com"
        ]
    },
    "ungrouped": {
        "hosts": [
            "monitor.example.com"
        ]
    },
    "webservers": {
        "hosts": [
            "web1.example.com",
            "web2.example.com"
        ]
    }
}
```

To develop your own dynamic inventory script, refer to `Inventory scripts` in the *Ansible Developer Guide*.

Start the script with an appropriate interpreter line (for example, `#!/usr/bin/python`) and make sure that it is executable so that Ansible can run it.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

The script must support the `--list` and `--host` *`managed-host`* options.

When called with the `--list` option, the script must print a JSON dictionary of all the hosts and groups in the inventory.

In its simplest form, a group is a list of managed hosts. In the following example, the `webservers` group includes the `web1.example.com` and `web2.example.com` hosts. The `databases` group includes the `db1.example.com` and `db2.example.com` hosts.

```
[user@host ~]$ ./inventoryscript --list
{
  "webservers": [ "web1.example.com", "web2.example.com" ],
  "databases": [ "db1.example.com", "db2.example.com" ]
}
```

Alternatively, each group value can be a JSON dictionary containing a list of managed hosts, a list of child groups, and group variables that might be set. The following example shows the JSON output for a more complex dynamic inventory. The `boston` group has two child groups, `backup` and `ipa`, three managed hosts, and a group variable set (`example_host: false`).

```
{
    "boston": {
        "children": [
            "backup",
            "ipa"
        ],
        "vars": {
            "example_host": false
        },
        "hosts": [
            "server1.example.com",
            "server2.example.com",
            "server3.example.com"
        ]
    },
    "backup": [
        "server4.example.com"
    ],
    "ipa": [
        "server5.example.com"
    ]
}
```

The script must also support the `--host` *`managed-host`* option. When you run the script with that option, it must print a JSON dictionary consisting of the variables associated with the host, or an empty JSON dictionary if there are

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

no variables for that host.

```
[user@host ~]$ ./inventoryscript --host server5.example.com
{
    "ntpserver": "ntp.example.com",
    "dnsserver": "dns.example.com"
}
```

Note

For each host in the inventory, Ansible calls the script with the `--host` option to retrieve the host variables. When the inventory is long, calling the script for each host can consume a lot of time and system resources.

To improve performance, you can directly provide the variables for all the hosts when the script is called with the `--list` option. To do so, the script must return a top-level element called `_meta`, which lists all the hosts and their variables. In that case, Ansible does not make the `--host` calls.

See the Tuning the External Inventory Script documentation for more information.

Using Inventory Scripts

You use dynamic inventory scripts just like static inventory text files. Specify the location of the inventory script either in the `ansible.cfg` file or by using the `-i` option.

If the inventory file is executable, then Ansible treats it as a dynamic inventory program and attempts to run it to generate the inventory. If the file is not executable, then Ansible treats it as a static inventory.

```
[user@host ~]$ ansible-navigator run -i inventoryscript my_playbook.yml
```

Important

The Python modules required to run a script locally might not be included in the default automation execution environment. To share your dynamic inventory scripts with other developers, and to use those dynamic inventory scripts in automation controller, you might need to build a custom automation execution environment that includes the necessary Python dependencies required by your scripts.

Managing Multiple Inventories

Ansible supports the use of multiple inventories in the same run. When the inventory path is a directory whether specified by the `-i` option, the `inventory` parameter, or another method, Ansible loads all inventory files in that directory. Both static and dynamic sources are combined to determine the inventory. The executable files within that directory are used to retrieve dynamic inventories, and the other files are used as static inventories or configuration files for inventory plug-ins.

Inventory files should not depend on other inventory files or scripts in order to resolve. If a group is specified as a child of another group in a static inventory file, then the group needs a placeholder entry. This is true even if all members of

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

that group come from a dynamic inventory. Consider the `cloud-east` group in the following example:

```
[cloud-east]

[servers]
test.example.com

[servers:children]
cloud-east
```

Adding `[cloud-east]`, even without any hosts or groups, ensures that no matter the order in which inventory files are parsed, they are all internally consistent.

Important

The order in which inventory files are parsed is not specified by the documentation. Currently, when multiple inventory files exist, they are parsed in alphabetical order. If one inventory source depends on information from another inventory source, then the order in which they are loaded determines if the inventory file works as expected or produces an error. Therefore, it is important to make sure that all files are self-consistent to avoid unexpected errors.

Ansible ignores files in an inventory directory if they end with certain suffixes. This can be controlled with the `inventory_ignore_extensions` directive in the Ansible configuration file. That directive defaults to `.pyc`, `.pyo`, `.swp`, `.bak`, `~`, `.rpm`, `.md`, `.txt`, `.rst`, `.orig`, `.ini`, `.cfg`, and `.retry`.

References

Ansible Documentation: Working with Dynamic Inventory

Ansible Documentation: Inventory Plugins

Ansible Documentation: Developing Dynamic Inventory

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

5.2. Guided Exercise
Managing Dynamic Inventories

Install custom scripts that dynamically generate a list of inventory hosts.

Outcomes

- Install and use existing dynamic inventory scripts.

Prerequisites

As the `student` user on the `workstation` machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
[student@workstation ~]$ lab start inventory-dynamic
```

Instructions

1. On the `workstation` machine, change to the `/home/student/dynamic` directory.

   View the contents of the `ansible.cfg` Ansible configuration file. The configuration file sets the inventory location to `inventory` in the current directory.

   ```
   [student@workstation ~]$ cd ~/dynamic/
   [student@workstation dynamic]$ cat ansible.cfg
   [defaults]
   inventory = inventory
   ```

2. Create the `/home/student/dynamic/inventory` directory:

   ```
   [student@workstation dynamic]$ mkdir inventory
   ```

3. Use the `tree` command to list the files in the current directory:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation dynamic]$ tree
.
├── ansible-navigator.yml
├── ansible.cfg
├── hosts (1)
├── inventory
├── inventorya.py (2)
└── inventoryw.py (3)

1 directory, 5 files
```

(1) The `hosts` static inventory file defines the `servers` group, which is a parent group of the `webservers` group.

(2) The `inventorya.py` dynamic inventory script defines the `webservers` group, which includes the `servera.lab.example.com` host.

(3) The `inventoryw.py` dynamic inventory script defines the `workstation.lab.example.com` host.

4. Move the Python scripts and the `hosts` file into the `inventory` directory:

```
[student@workstation dynamic]$ mv *.py hosts inventory/
```

5. Use the following `ansible-navigator inventory` command to list hosts and host groups. Use the `inventorya.py` script as the inventory source and add the `--graph` option to display the result as a graph. The command fails with a permission error on the `inventorya.py` file:

```
[student@workstation dynamic]$ ansible-navigator inventory \
-i inventory/inventorya.py --graph
[WARNING]:  * Failed to parse /home/student/dynamic/inventory/inventorya.py with
script plugin: problem running
/home/student/dynamic/inventory/inventorya.py --list ([Errno 13] Permission
denied:
'/home/student/dynamic/inventory/inventorya.py')
[WARNING]:  * Failed to parse /home/student/dynamic/inventory/inventorya.py with
ini plugin:
/home/student/dynamic/inventory/inventorya.py:3: Expected key=value host
variable assignment, got: subprocess
[WARNING]: Unable to parse /home/student/dynamic/inventory/inventorya.py as an
inventory source
[WARNING]: No inventory was parsed, only implicit localhost is available
@all:
  |--@ungrouped:
```

Ansible tries different plug-ins in a specific order to process inventory files. Ansible first tries to process the file as an inventory script and then tries to parse it as a static inventory file in INI format, which also fails.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

6. Inspect the current mode of the `inventorya.py` script. Change the mode of all Python scripts to be executable:

```
[student@workstation dynamic]$ ls -l inventory/inventorya.py
-rw-r--r--. 1 student student 640 Mar  5 00:14 inventory/inventorya.py
[student@workstation dynamic]$ chmod 755 inventory/*.py
```

7. Run the `inventorya.py` script with the `--list` option. The script displays the hosts in the `webservers` group:

```
[student@workstation dynamic]$ inventory/inventorya.py --list
{"webservers": {"hosts": ["servera.lab.example.com"], "vars": {}}}
```

8. Run the `inventoryw.py` script with the `--list` option. The script displays the `workstation.lab.example.com` host:

```
[student@workstation dynamic]$ inventory/inventoryw.py --list
{"all": {"hosts": ["workstation.lab.example.com"], "vars": {}}}
```

9. Display the content of the `inventory/hosts` file. Notice that the file references the `webservers` group that the dynamic inventory defines:

```
[student@workstation dynamic]$ cat inventory/hosts
[servers:children]
webservers
```

10. Use the following `ansible-navigator inventory` command to list the hosts and host groups in the inventory. Add the `--graph` option to display the result as a graph. The command produces a warning about the `webservers` group being undefined:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation dynamic]$ ansible-navigator inventory --graph
...output omitted...
[WARNING]:  * Failed to parse /home/student/dynamic/inventory/hosts with ini
plugin:
/home/student/dynamic/inventory/hosts:2: Section [servers:children] includes
undefined group: webservers
[WARNING]: Unable to parse /home/student/dynamic/inventory/hosts as an inventory
source
@all:
  |--@ungrouped:
  |  |--workstation.lab.example.com
  |--@servers:
  |--@webservers:
  |  |--servera.lab.example.com
```

> **Important**
> When multiple inventory files exist in a directory, Ansible attempts to process the files in alphabetical order, with numbers sorting before letters.

In this example, Ansible processes the `inventory/hosts` file before processing the `inventory/inventory_*.py` files. The `inventory/hosts` file defines the `webservers` host group as a child of the `servers` host group, but the `webservers` host group has not been defined yet.

To avoid this problem, a static inventory file should define a placeholder entry for any host group that it references.

11. Edit the `inventory/hosts` file and add an empty `[webservers]` section:

```
[webservers]

[servers:children]
webservers
```

12. Rerun the `ansible-navigator inventory` command to list the hosts and host groups in the inventory. Add the `--graph` option to display the result as a graph:

```
[student@workstation dynamic]$ ansible-navigator inventory --graph
@all:
  |--@ungrouped:
  |  |--workstation.lab.example.com
  |--@servers:
  |  |--@webservers:
  |  |  |--servera.lab.example.com
```


For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Finish

On the `workstation` machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish inventory-dynamic
```

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

5.3. Writing YAML Inventory Files

Objectives

- Write static inventory files in YAML format.

Inventory Plug-ins

Ansible uses plug-ins to support different formats of inventory files. Plug-ins are pieces of code that enhance the functionality of Ansible. By writing inventory plug-ins, Ansible developers can quickly support new formats and methods of generating inventory data. For example, Ansible manages static inventory files in INI format and dynamic inventory scripts with plug-ins.

Most inventory plug-ins are enabled by default. You can enable additional plug-ins by setting the `enable_plugins` directive under the `[inventory]` section in your `ansible.cfg` configuration file.

```
[inventory]
enable_plugins = host_list, script, auto, yaml, ini, toml
```

The preceding example shows the default list. When Ansible parses inventory sources, it tries the plug-ins in the order they appear in the `enable_plugins` directive.

Additional plug-ins shipped with Ansible include `advanced_host_list`, `constructed`, and `generator`. These additional plug-ins are covered by the `auto` plug-in.

The `script` plug-in provides support for dynamic inventory scripts. The `ini` plug-in provides support for the INI format. The other plug-ins get inventory information from files in other formats or other sources. Details are available at https://docs.ansible.com/ansible/latest/collections/ansible/builtin/index.html#inventory-plugins.

Some Ansible Content Collections come with additional inventory plug-ins. For example, the `amazon.aws` content collection provides the `aws_ec2` inventory plug-in to retrieve inventory hosts from Amazon Web Services EC2. Specify the fully qualified collection name (FQCN), `amazon.aws.aws_ec2`, in the `enable_plugins` list.

Writing YAML Static Inventory Files

You can use the `yaml` inventory plug-in to write static inventory files in YAML format.

The following example shows a static inventory file in INI format:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[lb_servers]
proxy.example.com

[web_servers]
web1.example.com
web2.example.com

[backend_server_pool]
appsrv-[a:e].example.com
```

This static inventory has three groups:

- The `lb_servers` group includes the `proxy.example.com` host.
- The `web_servers` group includes the `web1.example.com` and `web2.example.com` hosts.
- The `backend_server_pool` group uses an alphabetical range to include five hosts: `appsrv-a.example.com`, `appsrv-b.example.com`, `appsrv-c.example.com`, `appsrv-d.example.com`, and `appsrv-e.example.com`.

The following example defines the same inventory in YAML format:

```
lb_servers:
  hosts:
    proxy.example.com:
web_servers:
  hosts:
    web1.example.com:
    web2.example.com:
backend_server_pool:
  hosts:
    appsrv-[a:e].example.com:
```

YAML inventories use blocks to organize related configuration items. Each block begins with the name of a group followed by a colon. Everything indented below the group name belongs to that group.

If indented below a group name, then the `hosts` keyword starts a block of hostnames. All server names indented below the `hosts` keyword belong to this group. These servers themselves form their own groups, and must therefore end in a colon.

To define nested groups, use the `children` keyword in a group block. This keyword starts a list of subgroups that are members of this group. Those member subgroups can have their own `hosts` and `children` blocks.

One advantage that YAML format has over INI format is that it organizes both the lists of servers and the list of nested groups in the same place in the static inventory file.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Important

The `all` group implicitly exists at the top level and includes the rest of the inventory as its children. You can explicitly list it in your YAML inventory file, but it is not required.

```
all:
  children:
    lb_servers:
      hosts:
        proxy.example.com:
    web_servers:
      hosts:
        web1.example.com:
        web2.example.com:
    backend_server_pool:
      hosts:
        appsrv-[a:e].example.com:
```

Static inventories in INI format often include hosts that are not members of any group:

```
notinagroup.example.com

[mailservers]
mail1.example.com
```

Ansible automatically puts those hosts in the special `ungrouped` group. In YAML, explicitly assign them to the `ungrouped` group:

```
all:
  children:
    mailservers:
      hosts:
        mail1.example.com:
    ungrouped:
      hosts:
        notinagroup.example.com:
```

Setting Inventory Variables

You can set inventory variables directly in an inventory file in YAML format, just like you can in an inventory file in INI format.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

> **Note**
> It is often a good practice to avoid storing variables in static inventory files.
>
> Experienced Ansible developers prefer to use static inventory files only to list the hosts that Ansible manages, and to organize them in groups. Typically, the variables and their values are stored in the `host_vars` or `group_vars` files for the inventory.
>
> However, you might want to define variables such as `ansible_port` or `ansible_connection` in the same file as the inventory itself, to keep this information in one place. If you set variables in too many places, then it is harder to remember where a particular variable is defined.

In a group block, use the `vars` keyword to set group variables directly in a YAML inventory file.

The following static inventory file in INI format sets the `smtp_relay` variable for all the hosts in `monitoring` group:

```
[monitoring]
watcher.example.com

[monitoring:vars]
smtp_relay=smtp.example.com
```

The equivalent static inventory file in YAML format is as follows:

```
monitoring:
  hosts:
    watcher.example.com:
  vars:
    smtp_relay: smtp.example.com
```

In YAML, to set a variable for a specific host instead of a group, indent the variable under that host. The following static inventory file in INI format sets the `ansible_connection` variable for the `localhost` host:

```
[databases]
localhost ansible_connection=local
maria1.example.com
maria2.example.com
```

The equivalent static inventory file in YAML format is as follows:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
databases:
  hosts:
    localhost:
      ansible_connection: local
    maria1.example.com:
    maria2.example.com:
```

Converting a Static Inventory File in INI Format to YAML Format

You can use the `ansible-navigator inventory` command to help you convert an inventory file in INI format into YAML format. However, this is not the intended purpose of this command. The command is designed to display the entire configured inventory as Ansible sees it, and results can differ from those reported by the original inventory file. The `ansible-navigator inventory` command parses and tests the format of the inventory files, but it does not attempt to validate whether a hostname in the inventory exists.

> **Note**
> Remember that a name in an inventory file is not necessarily a valid hostname, but is used by the playbook to refer to a specific managed host. The name of that managed host could have an `ansible_host` variable set that points to the real name or IP address of the host. The name in the inventory file can be a simplified alias for the purposes of the playbook.

The following example shows a static inventory file in INI format, called `origin_inventory`, which you want to convert to YAML format. Notice that the `web1.example.com` host belongs to two groups: `web_servers` and `dc1`:

```
[lb_servers]
proxy.example.com

[web_servers]
web1.example.com
web2.example.com

[web_servers:vars]
http_port=8080

[backend_server_pool]
appsrv-[a:e].example.com

[dc1]
web1.example.com
appsrv-e.example.com
```

Use the following `ansible-navigator inventory` command to convert the `origin_inventory` file to YAML format:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[user@host ~]$ ansible-navigator inventory -i origin_inventory --list --yaml \
--mode stdout
all:
  children:
    backend_server_pool:
      hosts:
        appsrv-a.example.com: {}
        appsrv-b.example.com: {}
        appsrv-c.example.com: {}
        appsrv-d.example.com: {}
        appsrv-e.example.com: {}
    dc1:
      hosts:
        appsrv-e.example.com: {}
        web1.example.com:
          http_port: 8080
    lb_servers:
      hosts:
        proxy.example.com: {}
    ungrouped: {}
    web_servers:
      hosts:
        web1.example.com: {}
        web2.example.com:
          http_port: 8080
```

Notice that the command converts the `http_port` group variable to a host variable for the two members of the `web_servers` group.

Also, for the `web1.example.com` host, the variable is only set for the first occurrence of that host in the inventory; that is, when the host is declared in the `dc1` group. This is incorrect because this is not how the original inventory works. If you create a playbook that targets the `web_servers` group, then Ansible does not set the `http_port` variable for the `web1.example.com` host as it would with the original inventory.

It is therefore important to carefully review and fix the resulting YAML file. In this example, you would define the `http_port` variable in the `web_servers` group. You could also define the hosts in the `backend_server_pool` using a range:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
all:
  children:
    backend_server_pool:
      hosts:
        appsrv-[a:e].example.com: {}
    dc1:
      hosts:
        appsrv-e.example.com: {}
        web1.example.com: {}
    lb_servers:
      hosts:
        proxy.example.com: {}
    ungrouped: {}
    web_servers:
      hosts:
        web1.example.com: {}
        web2.example.com: {}
      vars:
        http_port: 8080
```

> **Note**
> Some group or host lines in the YAML output end with braces ({}). Those braces indicate that the group does not have any members or group variables, or that the host has no host variables. They do not need to be included.

Using the `ansible-navigator inventory` command to convert large inventory files can save time, but you must use it with caution. It works better when your original static inventory file does not directly declare inventory variables, but gets them from external files in the `host_vars` and `group_vars` directories.

Troubleshooting YAML Files

If you have trouble with static inventory files in YAML format, then consider the following points about YAML syntax.

Protecting a Colon Followed by a Space

Remember that in an unquoted string, a colon followed by a space causes an error. YAML interprets this as starting a new element in a dictionary.

The following example causes an error because the second colon is not protected:

```
title: Ansible: Best Practices
```

The following strings are correct because they are protected by quotes, or because they do not have a space after the colons:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
title: 'Ansible: Best Practices'
double: "Ansible: Best Practices"
fine: Not:a:problem
```

The Block Scalar syntax is designed to manage multiline values, but also escapes colons in content. Use the Literal Block Scalar (|) or Folded Block Scalar followed by a Chomp Indicator (-,+) to remove or keep the trailing newline character:

```
title: >-
  Ansible: Best Practices
```

Protecting a Variable That Starts a Value

Ansible performs variable replacement with the `{{ variable }}` syntax, but anything beginning with an opening brace ({) in YAML is interpreted as the start of a dictionary. Thus, you must enclose the variable placeholder with double quotes: `name: "{{ variable }} rest of the value"`.

In general, when using any of the reserved characters, {} > , | * & ! % # ` @ , you should use double quotes around the value.

Knowing the Difference Between a String and a Boolean or Float

Booleans and floating point numbers used as values for a variable must not be quoted. Quoted values are treated as strings. The following example sets the `active` variable to the Boolean value `true`, and sets the `default_answer` variable to a string value of `true`:

```
active: true
default_answer: "true"
```

The following example sets the `temperature` variable to a floating point value, and sets the `version` variable to a string:

```
temperature: 36.5
version: "2.0"
```

References

Ansible Documentation: Inventory Plugins

Ansible Documentation: How to Build Your Inventory

Ansible Documentation: YAML Syntax

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

5.4. Guided Exercise
Writing YAML Inventory Files

Convert an inventory file from INI format to YAML format.

Outcomes

- Replace an Ansible inventory file in INI format with its YAML equivalent.

Prerequisites

As the `student` user on the `workstation` machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
[student@workstation ~]$ lab start inventory-yaml
```

Instructions

1. The `lab` command provides an inventory file in INI format in the `/home/student/yaml` directory. Change to that directory, create a copy of the `inventory` file and save it as `inventory.yml`.

   1.1. Change to the `/home/student/yaml` directory:

   ```
   [student@workstation ~]$ cd ~/yaml/
   [student@workstation yaml]$
   ```

   1.2. Copy the existing static `inventory` file to `inventory.yml`:

   ```
   [student@workstation yaml]$ cp inventory inventory.yml
   ```

2. Edit the `inventory.yml` file to convert it to YAML format.

   2.1. Convert the `active_web_servers`, `inactive_web_servers`, and `all_servers` groups to YAML format:

   - Remove the brackets that surround the group names and end each group name with a colon.
   - Beneath each group name, add the `hosts:` keyword on a line indented by two spaces relative to the group name.
   - Indent the hosts in each group by four spaces relative to the group name and add a colon at the end of each hostname.

   The exact number of spaces does not matter as long as you keep a consistent indentation:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
active_web_servers:
  hosts:
    server[a:b].lab.example.com:

inactive_web_servers:
  hosts:
    serverc.lab.example.com:

[web_servers:children]
active_web_servers
inactive_web_servers

all_servers:
  hosts:
    serverd.lab.example.com:

[all_servers:children]
web_servers
```

2.2. Convert the definition of the child groups of the `web_servers` group. The file in INI format lists the child groups under the `[web_servers:children]` section.

- Beneath the group name, add the `children:` keyword on a line indented by two spaces relative to the group name.
- Add each child group on a line indented by four spaces relative to the group name and add a colon at the end of each group name.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
active_web_servers:
  hosts:
    server[a:b].lab.example.com:

inactive_web_servers:
  hosts:
    serverc.lab.example.com:

web_servers:
  children:
    active_web_servers:
    inactive_web_servers:

all_servers:
  hosts:
    serverd.lab.example.com:

[all_servers:children]
web_servers
```

2.3. Finally, convert the `all_servers` group to YAML format. Notice that this group includes an individual server as well as a group.

The resulting `inventory.yml` file should consist of the following content:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
active_web_servers:
  hosts:
    server[a:b].lab.example.com:

inactive_web_servers:
  hosts:
    serverc.lab.example.com:

web_servers:
  children:
    active_web_servers:
    inactive_web_servers:

all_servers:
  hosts:
    serverd.lab.example.com:
  children:
    web_servers:
```

2.4. *(Optional)* Instead of manually converting an INI-formatted inventory file to YAML format, you can use the `ansible-navigator inventory` command. This example redirects the command output to the `inventory2.yml` file:

```
[student@workstation yaml]$ ansible-navigator inventory -i inventory \
--list --yaml > inventory2.yml
```

The resulting `inventory2.yml` file contains the following content:

```
all:
  children:
    all_servers:
      children:
        web_servers:
          children:
            active_web_servers:
              hosts:
                servera.lab.example.com: {}
                serverb.lab.example.com: {}
            inactive_web_servers:
              hosts:
                serverc.lab.example.com: {}
      hosts:
        serverd.lab.example.com: {}
```

3. To test your inventory in YAML format, use the `ping.yml` playbook that the exercise provides. That playbook

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

targets the `all_servers` group and uses the `ping` module to test the connection to all the servers.

Use the `ansible-navigator` command to run the `ping.yml` playbook. Add the `-i` option to provide your inventory file in YAML format:

```
[student@workstation yaml]$ ansible-navigator run ping.yml -i inventory.yml

PLAY [Pinging all servers] *****************************************************

TASK [Ensure the hosts are reachable] ******************************************
ok: [servera.lab.example.com]
ok: [serverb.lab.example.com]
ok: [serverc.lab.example.com]
ok: [serverd.lab.example.com]
...output omitted...
```

Finish

On the `workstation` machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish inventory-yaml
```

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

5.5. Managing Variables

Objectives

- Structure host and group variables by using multiple files per host or group, and use special variables to override the host, port, or remote user that Ansible uses for a specific host.

The Basic Principles of Variables

You can use variables to write flexible and reusable tasks, roles, and playbooks. You can also use them to specify differences in configuration between different systems. You can set variables in many places, including:

- In the `defaults/main.yml` and `vars/main.yml` files for a role
- In the inventory file, either as a host variable or a group variable
- In a variable file in the `group_vars` or `host_vars` subdirectories of the playbook or inventory
- In a play, role, or task

As you define and manage variables in a project, plan to follow these principles:

Keep it simple

Even though you can define Ansible variables in many ways, try to define them by using only a couple of different methods and in only a few places.

Do not repeat yourself

If a set of systems has a common configuration, then organize the systems into a group and set variables for that group in a file in the `group_vars` directory. This way, you do not have to define the same settings for each host. When you have to modify a variable for that group of systems, you can update the variable in one location.

Organize variables in small, readable files

If you have a large project with many host groups and variables, then split the variable definitions into multiple files. To make it easier to find particular variables, group related variables into a file and give the file a meaningful name.

Remember that you can use subdirectories instead of files in the `host_vars` and `group_vars` directories. For example, you could have a `group_vars/webserver` directory for the `webserver` group, and that directory could contain a file called `firewall.yml`, which contains only variables related to firewall configuration. The `group_vars/webserver` directory could also contain additional variable files for configuring servers in the `webserver` group.

Variable Merging and Precedence

When you define the same variable in multiple ways, Ansible uses precedence rules to choose a value for that variable. After processing all the variable definitions, Ansible generates a set of merged variables for each host at the start of

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

each task.

When Ansible merges variables, if there are two definitions for the same variable in different places, then it uses the value that has the highest precedence.

Precedence order is covered in the *Variable Precedence: Where Should I Put a Variable?* documentation at https://docs.ansible.com/ansible/latest/playbook_guide/playbooks_variables.html#variable-precedence-where-should-i-put-a-variable. The following discussion breaks this down into more detail, from the lowest to the highest precedence.

Determining Command-line Option Precedence

Options that you pass to `ansible-navigator run` on the command line, other than `--extra-vars` (or `-e`) for extra variables, have the lowest precedence. For example, you can set the remote user with the `--user` (or `-u`) option to override the configuration file, but setting `ansible_user` at a higher precedence overrides it.

Determining Role Default Precedence

Variables set by a role in the *`rolename`*`/defaults/main.yml` file have a low precedence so that they are easy to override. These variables provide some reasonable default values and users often override them to configure the role.

Determining Host and Group Variable Precedence

You can set variables for inventory hosts and inventory groups in multiple places. You can set the variables relative to the location of your inventory or your playbook, by collecting host facts, or by reading facts from a cache.

The precedence order for these variables is as follows, from lowest to highest:

Group variables (in ascending order of precedence)

- Set *directly* in an inventory file or by a dynamic inventory script
- For `all` set in the *inventory* `group_vars/all` file or subdirectory
- For `all` set in the *playbook* `group_vars/all` file or subdirectory
- For other groups set in the *inventory* `group_vars` subdirectory
- For other groups set in the *playbook* `group_vars` subdirectory

Host variables (in ascending order of precedence)

- Set *directly* in an inventory file or by a dynamic inventory script
- Set in the *inventory* `host_vars` subdirectory
- Set in the *playbook* `host_vars` subdirectory

Host facts and cached facts

Both the variables coming from gathering host facts and those that read data from the cache, have the highest precedence in this category of host and group variables.

The biggest source of confusion here is related to the fact that the `group_vars` and `host_vars` subdirectories can exist not only in your project directory, which is where your playbook is located, but in the directory where your inventory file is located. These two locations might not be the same directory.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

For the `group_vars` and `host_vars` subdirectories relative to the inventory, the following is true:

- If you have `group_vars` and `host_vars` subdirectories in the same directory as your playbook, then Ansible automatically includes those variables for managed hosts and groups of managed hosts.
- If you use a flat inventory file in a directory other than the one the playbook is in, then Ansible also automatically includes the `group_vars` and `host_vars` directories in the inventory file directory.
- If you use an inventory directory in your project directory, and that inventory directory contains multiple inventory files, then Ansible includes the `group_vars` and `host_vars` subdirectories of your inventory directory.

> **Important**
> The variables that Ansible includes from the playbook directory override the variables from the inventory directory if there is a conflict.

Determining Play Variable Precedence

The next category of variables is those that you set in the playbook as part of a play, task, role parameter, or that you include or import. These have higher precedence than host or group variables, role defaults, and command-line options other than `-e`.

The precedence order for these variables is as follows, from lowest to highest:

- Set by the `vars` section of a play
- Set by prompting the user with a `vars_prompt` section in a play
- Set from a list of external files by the `vars_files` section of a play
- Set by a role *`rolename`*`/vars/main.yml` file
- Set for the current `block` with a `vars` section of that block
- Set for the current `task` with a `vars` section of that task
- Loaded dynamically with the `ansible.builtin.include_vars` module
- Set for a specific host by using either the `ansible.builtin.set_fact` module or by using `register` to record the result of task execution on a host
- Parameters set for a role in the playbook when loaded by the `role` section of a play or by using the `ansible.builtin.include_role` module
- Set by a `vars` section on tasks included with the `ansible.builtin.include_tasks` module

Notice that the normal `vars` section in a play has the lowest precedence in this category. There are many ways to override those settings if necessary. Variables set here override host-specific and group-specific settings in general.

Using `vars_prompt` is not a recommended practice. To operate correctly, you need to configure `ansible-navigator` to allow interaction with the `ansible-navigator run` command when it is running. The `vars_prompt` directive is also not compatible with automation controller.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

The `vars_files` directive is useful for organizing large lists of variables that are not host- or group-specific. These variables are organized by function into separate files. It can also help you separate sensitive variables into a separate file that you can encrypt with Ansible Vault. You can keep these sensitive variables separate from variables that are not sensitive, and which do not require encryption.

You can set variables that apply only to a specific block or task. These values override the play variables and inventory variables. You should use these variables sparingly, because they can make the playbook more complex:

```
- name: Task with a local variable definition
  vars:
    taskvariable: task
  ansible.builtin.debug:
    var: taskvariable
```

Notice that variables that you load by using `ansible.builtin.include_vars` have a high precedence, and can override variables set for roles and specific blocks and tasks. Often, you might want to use `vars_files` instead if you do not want to override those values with your external variable files.

The `ansible.builtin.set_fact` module and the `register` directive both set host-specific information, either a fact or the results of task execution on that host. Note that `ansible.buuiltin.set_fact` sets a high precedence value for that variable for the rest of the playbook run. If you cache that fact, then Ansible stores it in the fact cache at normal fact precedence (lower than play variables).

Determining the Precedence of Extra Variables

Extra variables that you set by using the `-e` option of the `ansible-navigator run` command always have the highest precedence. This is useful so that you can override the global setting for a variable in your playbook from the command line without editing any of the Ansible project files. For example:

```
[user@host ~]$ ansible-navigator run motd-setup.yml \
-e motd_message='Modified by Ansible'
```

Separating Variables from Your Inventory

If you are managing your inventory as a static file, then you can define variables in the inventory, either for a specific managed node or for a group of managed hosts. However, this is not a recommended practice. As your environment grows in both size and variety, the inventory file becomes large and difficult to read. A better approach is to move variable definitions from the inventory file into separate variable files.

Assigning Variables to Inventory Groups

You can create a subdirectory named `group_vars` in the same directory as your Ansible project. In the `group_vars` subdirectory, create a file for each group in your inventory and then add variables specific to that group of managed hosts:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[user@host project]$ tree -F group_vars
group_vars/
├── db_servers.yml
├── lb_servers.yml
└── web_servers.yml
```

The above organizational structure makes it easier to locate configuration variables for any of the `db_servers`, `lb_servers`, or `web_servers` inventory groups. That structure is sufficient when each inventory group does not contain too many variable definitions. As playbook complexity increases, however, even these files can become long and difficult to understand.

An even better approach for large, diverse environments is to create subdirectories for each inventory group of managed hosts under the `group_vars` subdirectory, such as the `groups_vars/db_servers` subdirectory, in your project directory. Ansible parses any YAML or JSON files in these subdirectories and associates the variables with an inventory group based on the parent directory.

If you use this organizational structure for variables, then place group variables with a common theme into the same file, and use a file name that indicates that common theme. A project organized according to this convention might appear as follows:

```
[user@host project3]$ tree -F group_vars
group_vars/
├── all/
│   └── common.yml
├── db_servers/
│   ├── mysql.yml
│   └── firewall.yml
├── lb_servers/
│   ├── firewall.yml
│   ├── haproxy.yml
│   └── ssl.yml
└── web_servers/
    ├── firewall.yml
    ├── webapp.yml
    └── apache.yml
```

With this organizational structure for a project, you can quickly see the types of variables that are defined for each host group.

Ansible merges all the variables present in files in directories under the `group_vars` directory with the rest of the variables. Separating variables into files grouped by functionality makes the whole playbook easier to understand and maintain.

Using Special Inventory Variables

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

A number of variables are available that you can use to change how Ansible connects to a host listed in the inventory. Some of these variables are most useful as host-specific variables, but others might be relevant to all hosts in an inventory group.

`ansible_connection`

The connection plug-in to use to access the managed host. By default, Ansible uses the `ssh` plug-in for all hosts except `localhost`, for which Ansible uses the `local` plug-in.

`ansible_host`

The actual IP address or fully qualified domain name to use when connecting to the managed host, instead of using the name from the inventory file. By default, this variable has the same value as the inventory hostname.

`ansible_port`

The port that Ansible uses to connect to the managed host. For the `ssh` connection plug-in, the default value is 22.

`ansible_user`

The user that Ansible uses to connect to the managed host. The default Ansible behavior is to connect to the managed host by using the same username as the user running the Ansible Playbook on the control node.

`ansible_become_user`

After Ansible has connected to the managed host, it switches to this user using `ansible_become_method`, which is `sudo` by default. You might need to provide authentication credentials in some way.

`ansible_python_interpreter`

The path to the Python executable that Ansible should use on the managed host. For Ansible 2.8 and later, this defaults to `auto`, which automatically selects a Python interpreter on the host running the playbook depending on what operating system it is running. Consequently, this setting is less likely to be required compared to earlier versions of Ansible.

Configuring Human Readable Inventory Hostnames

When Ansible executes a task on the remote host, the output displays the inventory hostname. Because you can specify alternative connection properties by using the preceding special inventory variables, you can assign meaningful names to inventory hosts. When you assign inventory hosts with meaningful names, you are better able to understand playbook output and diagnose playbook errors.

Consider a playbook that uses the following YAML inventory file:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
web_servers:
  hosts:
    server100.example.com:
    server101.example.com:
    server102.example.com:
lb_servers:
  hosts:
    server103.example.com:
```

With the preceding inventory file, the Ansible output references these names. Consider the following hypothetical output from a playbook that uses this inventory file:

```
[user@host project]$ ansible-navigator run site.yml
...output omitted...
PLAY RECAP *********************************************************************
server100.example.com  : ok=4    changed=0    unreachable=0    failed=0  ...
server101.example.com  : ok=4    changed=0    unreachable=0    failed=0  ...
server102.example.com  : ok=4    changed=0    unreachable=0    failed=0  ...
server103.example.com  : ok=3    changed=0    unreachable=0    failed=1  ...
```

From the preceding output, you cannot easily know that the failed host is a load balancer.

To improve that output, use an inventory file that contains descriptive names and define the necessary special inventory variables:

```
web_servers:
  hosts:
    webserver_1:
      ansible_host: server100.example.com
    webserver_2:
      ansible_host: server101.example.com
    webserver_3:
      ansible_host: server102.example.com
lb_servers:
  hosts:
    loadbalancer:
      ansible_host: server103.example.com
```

Now the output from the playbook provides descriptive names:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[user@host project]$ ansible-navigator run site.yml
...output omitted...
PLAY RECAP *********************************************************************
loadbalancer    : ok=3    changed=0    unreachable=0    failed=1  ...
webserver_1     : ok=4    changed=0    unreachable=0    failed=0  ...
webserver_2     : ok=4    changed=0    unreachable=0    failed=0  ...
webserver_3     : ok=4    changed=0    unreachable=0    failed=0  ...
```

In some situations you might want to use an arbitrary hostname in your playbook that you map to a real IP address or hostname with the `ansible_host` directive. For example:

- You might want Ansible to connect to that host by using a specific IP address that is different from the one that resolves in DNS. For example, there might be a particular management address that is not public, or the machine might have multiple addresses in DNS but one is on the same network as the control node.
- You might be provisioning cloud systems that have arbitrary names, but you want to refer to those systems in your playbook with names that make sense based on the roles that they play. If you use a dynamic inventory, then your dynamic inventory source might assign host variables automatically based on the intended role of each system.
- You might be referring to the machine by a short name in the playbook, but you need to refer to it by a fully qualified domain name in the inventory to properly connect to it.

Identifying the Current Host by Using Variables

When a play is running, you can use a number of variables and facts to identify the name of the current managed host executing a task:

`inventory_hostname`
: The name of the managed host currently being processed, taken from the inventory.

`ansible_host`
: The actual IP address or hostname that was used to connect to the managed host, as previously discussed.

`ansible_facts['hostname']`
: The short (unqualified) hostname collected as a fact from the managed host.

`ansible_facts['fqdn']`
: The fully qualified domain name (FQDN) collected as a fact from the managed host.

One final variable that can be useful is `ansible_play_hosts`, which is a list of all the hosts that have not yet failed during the current play, and therefore are going to be used for the tasks remaining in the play.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

References

Ansible Documentation: Variable Precedence: Where Should I Put a Variable?

Ansible Documentation: How to Build Your Inventory

Ansible Documentation: Special Variables

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

5.6. Guided Exercise
Managing Variables for Inventory Hosts and Groups

Set up directories that contain variable files for inventory host groups, and override the name used in an inventory file with a different name or IP address.

Outcomes

- Improve project maintainability by moving variables from playbooks to group variable directories for each inventory host group.

Prerequisites

As the `student` user on the `workstation` machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
[student@workstation ~]$ lab start inventory-vars
```

Instructions

1. Get familiar with the Ansible project. Identify the variables declared in each playbook. Review the inventory file.

   1.1. Change to the `/home/student/vars` directory:

   ```
   [student@workstation ~]$ cd ~/vars/
   [student@workstation vars]$
   ```

   1.2. List the files in the project directory:

   ```
   [student@workstation vars]$ ls
   ansible-navigator.yml  deploy_apache.yml   deploy_webapp.yml  roles
   ansible.cfg            deploy_haproxy.yml  inventory.yml      site.yml
   ```

   1.3. Review the `deploy_apache.yml` playbook. This playbook targets the `web_servers` host group in its single play. The `firewall_rules` variable defines the firewall configuration for the web servers.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
---
- name: Ensuring Apache HTTP Server is deployed
  hosts: web_servers
  become: true
  gather_facts: false
  tasks:
    - name: Ensure Apache HTTP Server is installed and started
      ansible.builtin.include_role:
        name: apache

    - name: Ensure the firewall ports are opened
      ansible.builtin.include_role:
        name: firewall
      vars:
        firewall_rules:
          # Allow HTTP requests from the load_balancer
          - zone: internal
            service: http
          - zone: internal
            source: "172.25.250.10"
```

1.4. Review the `deploy_haproxy.yml` playbook. This playbook targets the `lb_servers` host group in its single play and declares two variables.

- `haproxy_appservers` provides the HAProxy configuration.
- `firewall_rules` defines the firewall configuration.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
---
- name: Ensuring HAProxy is deployed
  hosts: lb_servers
  become: true
  gather_facts: false
  tasks:
    - name: Ensure HAProxy is installed and configured
      ansible.builtin.include_role:
        name: haproxy
      vars:
        haproxy_appservers:
          - name: serverb.lab.example.com
            ip: 172.25.250.11
            port: 80
          - name: serverc.lab.example.com
            ip: 172.25.250.12
            port: 80

    - name: Ensure the firewall ports are opened
      ansible.builtin.include_role:
        name: firewall
      vars:
        firewall_rules:
          - port: 80/tcp
```

1.5. Review the `deploy_webapp.yml` playbook. This playbook targets the `web_servers` host group in its single play. The `webapp_version` variable defines the version for the web application.

```
---
- name: Ensuring web content is deployed
  hosts: web_servers
  become: true
  gather_facts: false
  tasks:
    - name: Ensure web content is deployed
      ansible.builtin.include_role:
        name: webapp
      vars:
        webapp_version: v1.0
```

1.6. Review the `site.yml` playbook. This playbook imports the deploy playbooks in the project directory.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
---
- name: Deploy HAProxy
  ansible.builtin.import_playbook: deploy_haproxy.yml

- name: Deploy Web Server
  ansible.builtin.import_playbook: deploy_apache.yml

- name: Deploy Web App
  ansible.builtin.import_playbook: deploy_webapp.yml
```

1.7. Review the `inventory.yml` file, which is the inventory of the project.

The inventory declares the `servera.lab.example.com` host as member of the `lb_servers` host group, and the `serverb.lab.example.com` and `serverc.lab.example.com` hosts as members of the `web_servers` host group.

```
lb_servers:
  hosts:
    servera.lab.example.com:

web_servers:
  hosts:
    server[b:c].lab.example.com:
```

2. Create a directory structure to hold the variables for the host groups. Copy the playbook variables to files under the created directory structure (you remove the existing playbook variables in a later step).

2.1. Create the `group_vars/lb_servers` and `group_vars/web_servers` directories to hold the variable files:

```
[student@workstation vars]$ mkdir -p group_vars/lb_servers
[student@workstation vars]$ mkdir -p group_vars/web_servers
```

2.2. Create the `group_vars/lb_servers/vars.yml` variable file.

Copy and paste the `haproxy_appservers` and `firewall_rules` variable definitions from the `deploy_haproxy.yml` playbook:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
---
haproxy_appservers:
  - name: serverb.lab.example.com
    ip: 172.25.250.11
    port: 80
  - name: serverc.lab.example.com
    ip: 172.25.250.12
    port: 80

firewall_rules:
  - port: 80/tcp
```

2.3. Create the `group_vars/web_servers/vars.yml` variable file.

Copy and paste the `firewall_rules` variable definition from the `deploy_apache.yml` playbook and the `webapp_version` variable definition from the `deploy_webapp.yml` playbook:

```
---
firewall_rules:
  # Allow HTTP requests from the load_balancer
  - zone: internal
    service: http
  - zone: internal
    source: "172.25.250.10"

webapp_version: v1.0
```

2.4. Verify the directories and files in the `group_vars` directory by using the `tree` command:

```
[student@workstation vars]$ tree group_vars/
group_vars/
├── lb_servers
│   └── vars.yml
└── web_servers
    └── vars.yml

2 directories, 2 files
```

With this organizational structure, you can quickly find the variables defined for each host group.

3. Update the playbooks by removing the variables already defined.

3.1. Edit the `deploy_apache.yml` playbook to remove the `firewall_rules` variable. The resulting file should consist of the following content:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
---
- name: Ensuring Apache HTTP Server is deployed
  hosts: web_servers
  become: true
  gather_facts: false
  tasks:
    - name: Ensure Apache HTTP Server is installed and started
      ansible.builtin.include_role:
        name: apache

    - name: Ensure the firewall ports are opened
      ansible.builtin.include_role:
        name: firewall
```

3.2. Edit the `deploy_haproxy.yml` playbook to remove the `haproxy_appservers` and `firewall_rules` variables. The resulting file should consist of the following content:

```
---
- name: Ensuring HAProxy is deployed
  hosts: lb_servers
  become: true
  gather_facts: false
  tasks:
    - name: Ensure HAProxy is installed and configured
      ansible.builtin.include_role:
        name: haproxy

    - name: Ensure the firewall ports are opened
      ansible.builtin.include_role:
        name: firewall
```

3.3. Edit the `deploy_webapp.yml` playbook to remove the `webapp_version` variable. The resulting file should consist of the following content:

```
---
- name: Ensuring web content is deployed
  hosts: web_servers
  become: true
  gather_facts: false
  tasks:
    - name: Ensure web content is deployed
      ansible.builtin.include_role:
        name: webapp
```

4. Update the `inventory.yml` file so that playbook output displays `load_balancer` for any tasks that target the

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

`servera.lab.example.com` host:

```
lb_servers:
  hosts:
    load_balancer:
      ansible_host: servera.lab.example.com

web_servers:
  hosts:
    server[b:c].lab.example.com:
```

5. Run the `site.yml` playbook and confirm that the web deployment was successful.

   5.1. Use the `ansible-navigator run` command to run the `site.yml` playbook:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation vars]$ ansible-navigator run site.yml

PLAY [Ensuring HAProxy is deployed]
*******************************************

TASK [Ensure HAProxy is installed and configured]
*****************************

TASK [firewall : Ensure the firewalld package is installed]
*******************
ok: [load_balancer]

TASK [firewall : Ensure the firewall is configured]
**************************
changed: [load_balancer] => (item={'port': '80/tcp'})

TASK [haproxy : Ensure the HAProxy packages are installed]
********************
changed: [load_balancer]

TASK [haproxy : Ensure HAProxy is started and enabled]
************************
changed: [load_balancer]

TASK [haproxy : Ensure HAProxy configuration is set]
*************************
changed: [load_balancer]

...output omitted...
```

5.2. Use the `curl` command to confirm that the HAProxy server distributes the HTTP requests between the two Apache HTTP servers:

```
[student@workstation vars]$ curl servera
This is serverb.lab.example.com. (version v1.0)
[student@workstation vars]$ curl servera
This is serverc.lab.example.com. (version v1.0)
```

Finish

On the `workstation` machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish inventory-vars
```


For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

5.7. Lab
Managing Inventories

Convert an inventory from INI format to YAML format and apply recommended practices to variable structuring.

Outcomes

- Organize playbook variables in a directory structure.
- Write an inventory file in YAML format.
- Assign meaningful hostnames for remote hosts in an inventory.

Prerequisites

As the `student` user on the `workstation` machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available.

This command initializes the remote Git repository at `https://git.lab.example.com/student/inventory-review.git`. The Git repository contains a `site.yml` playbook that configures a front-end load balancer and a pool of back-end web servers. The back-end server pool is partitioned into two groups, A and B. Each pool partition deploys an independent version of the web application. You can push changes to this repository by using `Stud3nt123` as the Git password.

```
[student@workstation ~]$ lab start inventory-review
```

Instructions

1. Clone the `https://git.lab.example.com/student/inventory-review.git` repository into the `/home/student/git-repos` directory and then create a branch named `exercise`.

2. Review the inventory of the project. Run the `site.yml` playbook and confirm that the web deployment is successful. Requests for web pages sent to `servera` should produce the same web application version regardless of which back-end web server is used.

3. Configure the web servers in each host group to report different versions for the web application, as follows:
   - Create a directory for variable files for the `a_web_servers` host group. Create a variable file in that directory called `webapp.yml`. That file must set the value of the `webapp_version` variable to `v1.1a`.
   - Create another directory for the `b_web_servers` host group. It should also contain a variable file called `webapp.yml`. That second variable file must set the value of the `webapp_version` variable to `v1.1b`.

4. Run the `deploy_webapp.yml` playbook to update the version information displayed by the web application. Confirm that requests for web pages sent to `servera` produce two different versions of the web application depending on the host group the web server is in.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

5. Create a YAML-formatted inventory file named `inventory.yml` using the data from the INI-formatted `inventory` file. If you generate the `inventory.yml` file by running the `ansible-navigator inventory` command, then remove the variables that are added for each host. Do not remove the original `inventory` file. Run the `site.yml` playbook using the new `inventory.yml` file.

6. You have decided to change your YAML inventory to use inventory hostnames that are mapped to their purposes, rather than using actual hostnames. Modify the `inventory.yml` file as follows:

   - In the `lb_servers` host group, change `servera.lab.example.com` to `loadbalancer_1`.
   - In the `a_web_servers` host group, change `serverb.lab.example.com` to `backend_a1`.
   - In the `b_web_servers` host group, change `serverc.lab.example.com` to `backend_b1`.

   Verify that the `site.yml` playbook runs without errors when you use the updated `inventory.yml` inventory file.

7. Push your changes to the `exercise` branch of the remote Git repository.

Evaluation

As the `student` user on the `workstation` machine, use the `lab` command to grade your work. Correct any reported failures and rerun the command until successful.

```
[student@workstation ~]$ lab grade inventory-review
```

Finish

As the `student` user on the `workstation` machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish inventory-review
```

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Solution

Convert an inventory from INI format to YAML format and apply recommended practices to variable structuring.

Outcomes

- Organize playbook variables in a directory structure.
- Write an inventory file in YAML format.
- Assign meaningful hostnames for remote hosts in an inventory.

Prerequisites

As the `student` user on the `workstation` machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available.

This command initializes the remote Git repository at `https://git.lab.example.com/student/inventory-review.git`. The Git repository contains a `site.yml` playbook that configures a front-end load balancer and a pool of back-end web servers. The back-end server pool is partitioned into two groups, A and B. Each pool partition deploys an independent version of the web application. You can push changes to this repository by using `Stud3nt123` as the Git password.

```
[student@workstation ~]$ lab start inventory-review
```

Instructions

1. Clone the `https://git.lab.example.com/student/inventory-review.git` repository into the `/home/student/git-repos` directory and then create a branch named `exercise`.

   1.1. From a terminal, create the `/home/student/git-repos` directory if it does not already exist, and then change into it:

   ```
   [student@workstation ~]$ mkdir -p ~/git-repos/
   [student@workstation ~]$ cd ~/git-repos/
   [student@workstation git-repos]$
   ```

   1.2. Clone the `https://git.lab.example.com/student/inventory-review.git` repository and then change to the cloned repository:

```
[student@workstation git-repos]$ git clone \
https://git.lab.example.com/student/inventory-review.git
Cloning into 'inventory-review'...
...output omitted...
[student@workstation git-repos]$ cd inventory-review
[student@workstation inventory-review]$
```

1.3. Create the `exercise` branch and check it out:

```
[student@workstation inventory-review]$ git checkout -b exercise
Switched to a new branch 'exercise'
```

2. Review the inventory of the project. Run the `site.yml` playbook and confirm that the web deployment is successful. Requests for web pages sent to `servera` should produce the same web application version regardless of which back-end web server is used.

2.1. Review the `inventory` file, which is the inventory of the project. Notice that the `web_servers` host group is composed of the `a_web_servers` and `b_web_servers` host groups:

```
[lb_servers]
servera.lab.example.com

[web_servers]

[web_servers:children]
a_web_servers
b_web_servers

# Group "A" of back-end web servers
[a_web_servers]
serverb.lab.example.com

# Group "B" of back-end web servers
[b_web_servers]
serverc.lab.example.com
```

2.2. Use the `ansible-navigator run` command to run the `site.yml` playbook:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation inventory-review]$ ansible-navigator run site.yml
...output omitted...
PLAY RECAP
*********************************************************************
servera.lab.example.com    : ok=7    changed=6  ...  failed=0    skipped=4
...
serverb.lab.example.com    : ok=9    changed=7  ...  failed=0    skipped=2
...
serverc.lab.example.com    : ok=9    changed=7  ...  failed=0    skipped=2
...
```

2.3. Use the `curl` command to make two web requests to `servera`. Notice that the same version of the web application is deployed to both back-end web servers:

```
[student@workstation inventory-review]$ curl servera; curl servera
Hello from serverb.lab.example.com. (version v1.1)
Hello from serverc.lab.example.com. (version v1.1)
```

3. Configure the web servers in each host group to report different versions for the web application, as follows:

- Create a directory for variable files for the `a_web_servers` host group. Create a variable file in that directory called `webapp.yml`. That file must set the value of the `webapp_version` variable to `v1.1a`.
- Create another directory for the `b_web_servers` host group. It should also contain a variable file called `webapp.yml`. That second variable file must set the value of the `webapp_version` variable to `v1.1b`.

3.1. Create a directory structure to hold the variables for the `a_web_servers` and `b_web_servers` hosts groups:

```
[student@workstation inventory-review]$ mkdir -p group_vars/a_web_servers
[student@workstation inventory-review]$ mkdir -p group_vars/b_web_servers
```

3.2. Create the `group_vars/a_web_servers/webapp.yml` file with the following content:

```
---
webapp_version: v1.1a
```

3.3. Create the `group_vars/b_web_servers/webapp.yml` file with the following content:

```
---
webapp_version: v1.1b
```

4. Run the `deploy_webapp.yml` playbook to update the version information displayed by the web application. Confirm that requests for web pages sent to `servera` produce two different versions of the web application depending on the host group the web server is in.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

4.1. Use the `ansible-navigator run` command to run the `deploy_webapp.yml` playbook:

```
[student@workstation inventory-review]$ ansible-navigator run
deploy_webapp.yml

PLAY [Deploying Web App]
*****************************************************

TASK [Ensure web content is deployed]
***************************************

TASK [webapp : Ensure stub web content is deployed]
***************************
changed: [serverc.lab.example.com]
changed: [serverb.lab.example.com]

PLAY RECAP
*********************************************************************
serverb.lab.example.com    : ok=1    changed=1  ...  failed=0    skipped=0
...
serverc.lab.example.com    : ok=1    changed=1  ...  failed=0    skipped=0
...
```

4.2. Verify that requests to `servera` produce different versions of the web application:

```
[student@workstation inventory-review]$ curl servera; curl servera
Hello from serverb.lab.example.com. (version v1.1a)
Hello from serverc.lab.example.com. (version v1.1b)
```

5. Create a YAML-formatted inventory file named `inventory.yml` using the data from the INI-formatted `inventory` file. If you generate the `inventory.yml` file by running the `ansible-navigator inventory` command, then remove the variables that are added for each host. Do not remove the original `inventory` file. Run the `site.yml` playbook using the new `inventory.yml` file.

5.1. Create a YAML-formatted inventory file named `inventory.yml` using the data from the INI-formatted `inventory` file:

```
[student@workstation inventory-review]$ ansible-navigator inventory \
-i inventory --list --yaml > inventory.yml
```

5.2. Ensure that the `inventory.yml` file does not contain host variables. Either of the following two examples for the `inventory.yml` file are acceptable. Trim the file to match one or the other example.

- Example 1: This example includes the `all` group:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
all:
  children:
    lb_servers:
      hosts:
        servera.lab.example.com:
    web_servers:
      children:
        a_web_servers:
          hosts:
            serverb.lab.example.com:
        b_web_servers:
          hosts:
            serverc.lab.example.com:
```

- Example 2: This example more closely matches the existing `inventory` file:

```
lb_servers:
  hosts:
    servera.lab.example.com:
web_servers:
  children:
    a_web_servers:
      hosts:
        serverb.lab.example.com:
    b_web_servers:
      hosts:
        serverc.lab.example.com:
```

5.3. Use the `ansible-navigator` command to run the `site.yml` playbook. Add the `-i` option to provide your inventory file in YAML format:

```
[student@workstation inventory-review]$ ansible-navigator run site.yml \
-i inventory.yml
...output omitted...
PLAY RECAP
*********************************************************************
servera.lab.example.com    : ok=5    changed=0   ...   failed=0    skipped=4
...
serverb.lab.example.com    : ok=8    changed=0   ...   failed=0    skipped=2
...
serverc.lab.example.com    : ok=8    changed=0   ...   failed=0    skipped=2
...
```

6. You have decided to change your YAML inventory to use inventory hostnames that are mapped to their purposes, rather than using actual hostnames. Modify the `inventory.yml` file as follows:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

- In the `lb_servers` host group, change `servera.lab.example.com` to `loadbalancer_1`.
- In the `a_web_servers` host group, change `serverb.lab.example.com` to `backend_a1`.
- In the `b_web_servers` host group, change `serverc.lab.example.com` to `backend_b1`.

Verify that the `site.yml` playbook runs without errors when you use the updated `inventory.yml` inventory file.

6.1. Modify the `inventory.yml` file so that each server follows its naming convention:

```
lb_servers:
  hosts:
    loadbalancer_1:
      ansible_host: servera.lab.example.com

web_servers:
  children:
    a_web_servers:
      hosts:
        backend_a1:
          ansible_host: serverb.lab.example.com
    b_web_servers:
      hosts:
        backend_b1:
          ansible_host: serverc.lab.example.com
```

6.2. Use the `ansible-navigator run` command to run the `site.yml` playbook. Notice that the playbook output contains the inventory hostnames that you defined:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation inventory-review]$ ansible-navigator run site.yml \
-i inventory.yml

...output omitted...

TASK [haproxy : Ensure haproxy packages are present]
*************************
ok: [loadbalancer_1]

TASK [haproxy : Ensure haproxy is started and enabled]
***********************
ok: [loadbalancer_1]

TASK [haproxy : Ensure haproxy configuration is set]
*************************
ok: [loadbalancer_1]

...output omitted...

TASK [apache : Ensure httpd packages are installed]
**************************
ok: [backend_b1]
ok: [backend_a1]

TASK [apache : Ensure SELinux allows httpd connections to a remote database]
**
ok: [backend_b1]
ok: [backend_a1]

TASK [apache : Ensure httpd service is started and enabled]
*******************
ok: [backend_b1]
ok: [backend_a1]

...output omitted...

PLAY RECAP
*********************************************************************
backend_a1                 : ok=8    changed=1   ...  failed=0    skipped=2
...
backend_b1                 : ok=8    changed=1   ...  failed=0    skipped=2
...
loadbalancer_1             : ok=5    changed=0   ...  failed=0    skipped=4
...
```

6.3. Use the `curl` command to make two web requests to `servera`. Verify that the output of the requests

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

contains the backend_a1 and backend_b1 inventory hostnames:

```
[student@workstation inventory-review]$ curl servera; curl servera
Hello from backend_a1. (version v1.1a)
Hello from backend_b1. (version v1.1b)
```

7. Push your changes to the exercise branch of the remote Git repository.

   7.1. Display changes made to the local Git repository:

```
[student@workstation inventory-review]$ git status
On branch exercise
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        group_vars/a_web_servers/
        group_vars/b_web_servers/
        inventory.yml

nothing added to commit but untracked files present (use "git add" to track)
```

   7.2. Use the git add command to stage all the modified files for the next commit:

```
[student@workstation inventory-review]$ git add .
```

   7.3. Commit the staged files to the local Git repository:

```
[student@workstation inventory-review]$ git commit -m 'Changes for exercise'
...output omitted...
```

   7.4. Push the changes from the local Git repository to the exercise branch on the remote Git repository. If prompted, use Stud3nt123 as the Git password:

```
[student@workstation inventory-review]$ git push -u origin exercise
Password for 'https://student@git.lab.example.com': Stud3nt123
...output omitted...
```

   7.5. Verify that your changes have been pushed to the exercise branch:

```
[student@workstation inventory-review]$ git status
On branch exercise
Your branch is up to date with 'origin/exercise'.

nothing to commit, working tree clean
```

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Evaluation

As the `student` user on the `workstation` machine, use the `lab` command to grade your work. Correct any reported failures and rerun the command until successful.

```
[student@workstation ~]$ lab grade inventory-review
```

Finish

As the `student` user on the `workstation` machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish inventory-review
```

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

5.8. Summary

- Inventory plug-ins can dynamically obtain lists of hosts and host groups from sources such as cloud, virtualization, and systems management platforms.
- Dynamic inventory scripts can generate host and host group information from sources external to Ansible.
- The `ansible-navigator inventory` command can help convert an inventory file in INI format into YAML format.
- Host and host group variables organized in `host_vars` and `group_vars` directories can improve project maintainability.
- Special inventory variables control connections to hosts and can make playbook output more meaningful.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Chapter 6.
Managing Task Execution

Goal

Control and optimize the execution of tasks by Ansible Playbooks.

Sections

- Controlling Privilege Escalation (and Guided Exercise)
- Controlling Task Execution (and Guided Exercise)
- Running Selected Tasks (and Guided Exercise)
- Optimizing Execution for Speed (and Guided Exercise)

Lab

- Managing Task Execution

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

6.1. Controlling Privilege Escalation

Objectives

- Control automatic privilege escalation at the play, role, task, or block level.

Privilege Escalation Strategies

Ansible Playbooks can achieve privilege escalation at many levels. Ansible uses directives or connection variables for the privilege escalation level you intend to control. For plays, roles, blocks, and tasks you need to use the privilege escalation directives. The directives are `become`, `become_user`, `become_method`, and `become_ask_password`.

Privilege Escalation by Configuration

If you have the `become` Boolean set to `true` (or `yes`) in the `[privilege_escalation]` section of your Ansible configuration file, then all plays in your playbook use privilege escalation by default. Those plays use the current value of `become_method` to change to the user specified by the `become_user` directive when running on the managed hosts.

You can override the configuration file and specify privilege escalation settings when you run the playbook by using command-line options. The following table compares configuration directives and command-line options:

Table 4. Privilege Escalation Directives and Options

Configuration directive	Command-line option
`become`	`--become` or `-b`
`become_user`	`--become-user=`*`BECOME_USER`*
`become_method`	`--become-method=`*`BECOME_METHOD`*
`become_ask_pass`	`--ask-become-pass` or `-K`

Important

If the Ansible configuration file specifies `become: false`, but the command line includes the `-b` option, then Ansible ignores the configuration file and uses privilege escalation by default.

You can get additional information about the precedence rules in *Controlling How Ansible Behaves: Precedence Rules* at https://docs.ansible.com/ansible/11/reference_appendices/general_precedence.html#general-precedence-rules.

Defining Privilege Escalation in Plays

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

When you write a playbook, you might need privilege escalation for some plays but not for all plays. You can specify which plays should use privilege escalation. If the play does not specify privilege escalation settings, then the default settings from the configuration file or the command line are used.

The following playbook contains three plays. The first play uses `become: true` to use privilege escalation, regardless of what the configuration file or command-line options specify. The second play uses `become: false` to not use privilege escalation, even if the configuration file or command-line options are specified. The third play does not have a become directive and uses privilege escalation based on the default settings from the Ansible configuration file or the command line. The `ansible_user_id` variable displays the name of the user on the managed host running the current play.

```
---
- name: Become the user "manager"
  hosts: webservers
  become: true
  tasks:
    - name: Show the user used by this play
      ansible.builtin.debug:
        var: ansible_user_id

- name: Do not use privilege escalation
  hosts: webservers
  become: false
  tasks:
    - name: Show the user used by this play
      ansible.builtin.debug:
        var: ansible_user_id

- name: Use privilege escalation based on defaults
  hosts: webservers
  tasks:
    - name: Show the user used by this play
      ansible.builtin.debug:
        var: ansible_user_id
```

To ensure that privilege escalation is used correctly by your play, specify the setting explicitly in the playbook. Specifying the privilege escalation method or the privileged user in configuration settings or inventory variables might be necessary depending on the task or the hosts in question.

Privilege Escalation in Tasks

You can enable or disable privilege escalation for specific tasks in a play. To do this, add the appropriate become directive to the task:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
---
- name: Play with two tasks, one uses privilege escalation
  hosts: all
  become: false
  tasks:
    - name: This task requires privilege escalation
      become: true
      ansible.builtin.dnf:
        name: perl
        state: present

    - name: This task does not need privilege escalation
      ansible.builtin.shell: perl -v
      register: perlcheck
      failed_when: perlcheck['rc'] != 0
```

In the preceding example, the play has privilege escalation disabled by default, but the first task uses privilege escalation.

Grouping Privilege Escalation Tasks with Blocks

If you have a subset of tasks in your play that require (or do not require) privilege escalation, you can set `become` on a block. All tasks in the block share the same privilege escalation setting, which overrides the setting made at the play level. The following examples illustrate things you can do with this mechanism:

- Enable privilege escalation for a subset of tasks in your play
- Disable privilege escalation for a subset of tasks in your play
- Add the `become_user` directive to use privilege escalation to perform a subset of tasks as a specific user (such as a database user) instead of as `root`

To use this option, add the necessary directives to the `block` dictionary. In the following example, the playbook contains one play that has privilege escalation turned off by default. The play consists of a block (containing two tasks) and a separate task. The tasks in the block use privilege escalation, even though it is off by default for the play. The final task does not use privilege escalation because it is outside of the block.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
---
- name: Deploy web services
  hosts: webservers
  become: false
  tasks:
    - name: Tasks that need privilege escalation
      become: true
      block:
        - name: Ensure httpd is installed
          ansible.builtin.dnf:
            name: httpd
            state: present

        - name: Start and enable web service
          ansible.builtin.service:
            name: httpd
            state: started
            enabled: true

    - name: Test website from itself
      ansible.builtin.uri:
        url: http://{{ ansible_host }}
        return_content: true
      register: webpage
      failed_when: webpage['status'] != 200
```

Applying Privilege Escalation in Roles

You can use one of the following methods to enable privilege escalation for a role:

- Enable privilege escalation for only the role tasks that require it.
- Enable privilege escalation for the entire role.
- Enable privilege escalation for the entire play that calls the role.

The following example enables privilege escalation for the entire role:

```
- name: Example play with one role
  hosts: demo
  roles:
    - role: role-name
      become: true
```

Listing Privilege Escalation with Connection Variables

You can use connection variables to configure privilege escalation. Apply these connection variables as inventory variables on groups or individual hosts. The following table compares playbook directives with connection variable names:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Table 5. Privilege Escalation Directives and Connection Variables

Playbook directive	Connection variable
become	ansible_become
become_user	ansible_become_user
become_method	ansible_become_method

You can use host-based or group-based connection variables, such as `ansible_become`, `ansible_become_user`, and `ansible_become_method`, to override privilege escalation settings for individual hosts or groups that you originally set as configuration or playbook directives, such as `become`, `become_user`, and `become_method`.

If necessary, you can also use the `ansible_become_pass` connection variable to provide a privilege escalation password. Setting this variable in inventory files, group variable files, or host variable files poses a security risk if you store those files in version control. You might consider configuring your managed hosts to not require a privilege escalation password, or you might use the `-K` option to prompt for the privilege escalation password when you run your playbooks.

Important

Connection variables override the privilege escalation settings in the configuration file, as well as in plays, tasks, blocks, and roles.

Be cautious when using connection variables to configure privilege escalation because of their high precedence compared to the configuration and playbook directives.

The following example demonstrates configuring privilege escalation variables in a YAML inventory for a group:

```
webservers:
  hosts:
    servera.lab.example.com:
    serverb.lab.example.com:
  vars:
    ansible_become: true
```

You can set connection variables for privilege escalation at the host level. Doing this on a per-host basis is tedious, but can be helpful if one host in a larger group needs specific connection options. The following example demonstrates configuring privilege escalation by using connection variables at the host level:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
webservers:
  hosts:
    servera.lab.example.com:
      ansible_become: true
    serverb.lab.example.com:
```

You can set connection variables in the playbook; for example, on the play itself. This overrides the inventory variables (through normal variable precedence) and the setting of any become directives.

```
---
- name: Example play using connection variables
  hosts: webservers
  vars:
    ansible_become: true
  tasks:
    - name: Play will use privilege escalation even if inventory says no
      ansible.builtin.dnf:
        name: perl
        state: present
```

Choosing Privilege Escalation Approaches

Ansible provides many ways to control privilege escalation. Consider the following competing needs when choosing how to control privilege escalation:

- Run tasks with the least privilege when possible. This avoids inadvertent compromises and damage to managed hosts through playbook errors.
- Keep your playbook simple. This makes it easier for you to understand the playbook and whether it is running with elevated privileges.

Some users run their playbooks with privilege escalation always turned on. This approach is simple, and often the tasks that you are performing must run as `root`. Tasks that *do not* need to run as `root` also run with elevated privileges, increasing risk.

Some users connect to the `root` account to avoid privilege escalation. This is generally not a recommended practice. Anyone running the playbook must have `root` credentials on the managed hosts. This can make it hard to determine which administrator performed which playbook run.

A recommended practice is to control which plays or tasks need privilege escalation selectively.

For example, if the `apache` user can start the `httpd` server, there is no need to run the task as the `root` user.

Ideally, you should configure privilege escalation in as simple a way as possible, and it should be clear to you if privilege escalation is needed for a task. For example, you could use `become: true` to enable privilege escalation for a play and then use `become: false` to selectively disable privilege escalation for tasks that do not require it.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Alternatively, you could group tasks that need privilege escalation into one play and tasks that do not into another play (if compatible with the workflow). Setting privilege escalation in the configuration file might be necessary if a playbook requires privilege escalation, but you cannot edit that playbook for some reason.

Generally, if the playbook requires privilege escalation, it makes more sense to state that requirement. The biggest challenge is when different hosts that use a playbook require other privilege escalation methods to function correctly. In that case, it can make sense to set inventory variables such as `ansible_become_method` for those hosts or their group when enabling privilege escalation with `become` in the playbook.

> **References**
> Understanding Privilege Escalation - Ansible Documentation
>
> Blocks - Ansible Documentation

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

6.2. Guided Exercise
Controlling Privilege Escalation

Configure a playbook to escalate privileges only for specific plays, roles, tasks, or blocks that might need those privileges to operate correctly.

Outcomes

- Select the appropriate privilege escalation method for plays and tasks.

Prerequisites

As the `student` user on the `workstation` machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
[student@workstation ~]$ lab start task-escalation
```

Instructions

1. Clone the `https://git.lab.example.com/student/task-escalation.git` repository into the `/home/student/git-repos` directory and then create a branch for this exercise.

   1.1. From a terminal, create the `/home/student/git-repos` directory if it does not already exist, and then change into it:

   ```
   [student@workstation ~]$ mkdir -p ~/git-repos/
   [student@workstation ~]$ cd ~/git-repos/
   [student@workstation git-repos]$
   ```

   1.2. Clone the `https://git.lab.example.com/student/task-escalation.git` repository and then change directory to the cloned repository:

   ```
   [student@workstation git-repos]$ git clone \
   https://git.lab.example.com/student/task-escalation.git
   Cloning into 'task-escalation'...
   ...output omitted...
   [student@workstation git-repos]$ cd task-escalation
   [student@workstation task-escalation]$
   ```

   1.3. Create the `exercise` branch and check it out:

   ```
   [student@workstation task-escalation]$ git checkout -b exercise
   Switched to a new branch 'exercise'
   ```

2. Review the global privilege escalation setting for the project.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

2.1. Display the contents of the project `ansible.cfg` file to check for privilege escalation settings. Notice that the `ansible.cfg` file does not contain a `[privilege_escalation]` section.

```
[defaults]
inventory = ./inventory
remote_user = devops
collections_path = ./collections:/usr/share/ansible/collections
```

2.2. Query for privilege escalation settings by using the `ansible-navigator config` command:

```
[student@workstation task-escalation]$ ansible-navigator config -m
interactive
...output omitted...
```

2.3. Type `:f become` and then press **Enter** to filter for privilege escalation settings:

```
  Name                          Default   Source    Current
0|Agnostic become prompt        True      default   True
1|Become plugin path            True      default
['/home/runner/.ansible...
2|Default become                True      default   False
3|Default become ask pass       True      default   False
4|Default become exe            True      default   None
5|Default become flags          True      default   None
6|Default become method         True      default   sudo
7|Default become user           True      default   root
```

2.4. Press `2` to display details for the `Default become` setting. Notice that the current value for the setting is set to `false`, which means that no tasks run with escalated privileges. You can change the setting by defining the become key in the `[privilege_escalation]` section of the `/home/student/git-repos/task-escalation/ansible.cfg` file.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
Default become (current/default: False)
 0|---
 1|current_config_file: /home/student/git-repos/task-escalation/ansible.cfg
 2|current_value: false
 3|default: true
 4|default_value: false
 5|description: Toggles the use of privilege escalation, allowing
 6|  you to 'become' another user after login.
 7|env:
 8|- name: ANSIBLE_BECOME
 9|ini:
10|- key: become
11|  section: privilege_escalation
12|name: Default become
13|option: DEFAULT_BECOME
14|source: default
15|type: boolean
16|via: default
```

Exit from `ansible-navigator` by pressing `Esc` twice, or type `:q` and then press `Enter`.

3. Attempt to run the `intranet.yml` playbook, and then examine the playbook to determine the tasks that require escalation.

   3.1. Attempt to run the `intranet.yml` playbook. Notice that it results in a failure.

```
[student@workstation task-escalation]$ ansible-navigator run intranet.yml

PLAY [Enable intranet services]
*********************************************

TASK [Install software]
*********************************************************
fatal: [servera.lab.example.com]: FAILED! => {"ansible_facts":
{"discovered_interpreter_python": "/usr/bin/python3"}, "changed": false,
"msg": "This command has to be run under the root user.", "results": []}

PLAY RECAP
*********************************************************************
servera.lab.example.com    : ok=0    changed=0    unreachable=0    failed=1
...

Please review the log for errors.
```

   3.2. View the `intranet.yml` playbook to identify tasks that require privilege escalation:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
---
- name: Enable intranet services
  hosts: servera.lab.example.com
  gather_facts: false
  tasks:
    - name: Install software
      ansible.builtin.dnf:
        name:
          - httpd
          - firewalld
        state: present

    - name: Create sample html page
      ansible.builtin.copy:
        content: >
          Welcome to the example.com intranet!
        dest: /var/www/html/index.html
        owner: root
        group: root
        mode: "0644"

    - name: Start and enable firewalld
      ansible.builtin.service:
        name: firewalld
        state: started
        enabled: true

    - name: Open firewall access
      ansible.posix.firewalld:
        service: http
        immediate: true
        permanent: true
        state: enabled

    - name: Start and enable httpd
      ansible.builtin.service:
        name: httpd
        state: started
        enabled: true

- name: Test intranet web server
  hosts: localhost
  gather_facts: false
  tasks:
    - name: Connect to intranet web server
      ansible.builtin.uri:
```

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
        url: http://servera.lab.example.com
        return_content: true
        status_code: 200
```

The tasks from the first play need privilege escalation to run, but the one task from the second play does not. You can use any of the following methods to satisfy those requirements:

- Wrap sequential tasks that need escalation with a block, and set privilege escalation for the whole block.
- Add `become: true` to the first play. This setting causes every task in that play to run with privilege escalation unless otherwise specified.

4. Add the privilege escalation to tasks in the first play of the `intranet.yml` playbook by using a block:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
- name: Enable intranet services
  hosts: servera.lab.example.com
  gather_facts: false
  tasks:
    - name: Tasks that require privilege escalation
      become: true
      block:
        - name: Install software
          ansible.builtin.dnf:
            name:
              - httpd
              - firewalld
            state: present

        - name: Create sample html page
          ansible.builtin.copy:
            content: >
              Welcome to the example.com intranet!
            dest: /var/www/html/index.html
            owner: root
            group: root
            mode: "0644"

        - name: Start and enable firewalld
          ansible.builtin.service:
            name: firewalld
            state: started
            enabled: true

        - name: Open firewall access
          ansible.posix.firewalld:
            service: http
            immediate: true
            permanent: true
            state: enabled

        - name: Start and enable httpd
          ansible.builtin.service:
            name: httpd
            state: started
            enabled: true

- name: Test intranet web server
  hosts: localhost
  gather_facts: false
  tasks:
```

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
    - name: Connect to intranet web server
      ansible.builtin.uri:
        url: http://servera.lab.example.com
        return_content: true
        status_code: 200
```

5. Use the `ansible-navigator` command to run the playbook:

```
[student@workstation task-escalation]$ ansible-navigator run intranet.yml

PLAY [Enable intranet services] ************************************************

TASK [Install software] ********************************************************
changed: [servera.lab.example.com]

...output omitted...

PLAY [Test intranet web server] ************************************************

TASK [Connect to intranet web server] ******************************************
ok: [localhost]

PLAY RECAP *********************************************************************
localhost                  : ok=1    changed=0    unreachable=0    failed=0  ...
servera.lab.example.com    : ok=5    changed=4    unreachable=0    failed=0  ...
```

6. Test that the playbook ran correctly:

```
[student@workstation task-escalation]$ curl servera.lab.example.com
Welcome to the example.com intranet!
```

7. Remove the block from the first play and add the `become: true` statement. This statement enables privilege escalation for every task in the first play.

   7.1. Instead of removing the block manually, use the `git` command to restore the original file:

```
[student@workstation task-escalation]$ git restore intranet.yml
```

   7.2. Add privilege escalation to tasks in the first play of the `intranet.yml` playbook by using a `become: true` statement. The second play does not require privilege escalation and inherits `become: false` from the default privilege escalation settings:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
---
- name: Enable intranet services
  hosts: servera.lab.example.com
  become: true
  gather_facts: false
  tasks:
    - name: Install software
      ansible.builtin.dnf:
        name:
          - httpd
          - firewalld
        state: present

...output omitted...

- name: Test intranet web server
  hosts: localhost
  gather_facts: false
  tasks:
    - name: Connect to intranet web server
      ansible.builtin.uri:
        url: http://servera.lab.example.com
        return_content: true
        status_code: 200
```

8. Use the `ansible-navigator` command to run the playbook. The playbook succeeds.

```
[student@workstation task-escalation]$ ansible-navigator run intranet.yml
...output omitted...
PLAY RECAP *********************************************************************
localhost                  : ok=1    changed=0    unreachable=0    failed=0  ...
servera.lab.example.com    : ok=5    changed=0    unreachable=0    failed=0  ...
```

9. Test that the playbook ran correctly:

```
[student@workstation task-escalation]$ curl servera.lab.example.com
Welcome to the example.com intranet!
```

Finish

On the `workstation` machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish task-escalation
```


For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

6.3. Controlling Task Execution

Objectives

- Configure tasks that can run before roles or after normal handlers, and simultaneously notify multiple handlers.

Controlling the Order of Execution

In a play, Ansible always runs the tasks from roles, called by the `roles` statement, before the tasks that you define under the `tasks` section. The following playbook contains both a `roles` and a `tasks` section:

```
---
- name: Ensure Apache is deployed
  hosts: www1.example.com
  gather_facts: false

  tasks:
    - name: Open the firewall
      ansible.posix.firewalld:
        service: http
        state: enabled
        immediate: true
        permanent: true

  roles:
    - role: apache
```

When you run this playbook, Ansible runs the tasks from the `apache` role before it runs the `Open the firewall` task, even though the `tasks` section is defined first:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[user@host ~]$ ansible-navigator run deploy_apache.yml

PLAY [Ensure Apache is deployed] ***************************************

TASK [apache : Ensure httpd packages are installed] ********************
changed: [www1.example.com]

TASK [apache : Ensure httpd service is started and enabled] ************
changed: [www1.example.com]

TASK [Open the firewall] ***********************************************
changed: [www1.example.com]

PLAY RECAP *************************************************************
www1.example.com    : ok=3    changed=3    unreachable=0  ...
```

For readability, it is a good practice to write the `tasks` section after the `roles` section, so that the order of the playbook matches the order of execution.

> **Note**
>
> This order comes from the fact that each play listed in the playbook is a YAML dictionary of key-value pairs. The order of the top-level directives (`name`, `hosts`, `tasks`, `roles`, and so on) is arbitrary, but Ansible handles them in a standardized order when it parses and runs the play.
>
> It is a good practice to write your plays in a consistent order starting with the name of the play, but Ansible does not require this. However, deviating from this convention can make it harder to read your playbook, and therefore Red Hat recommends changing the order.

Importing or Including Roles as a Task

In recent versions of Ansible, you can include or import roles as a task instead of by using the `roles` section of the play. The advantage of this approach is that you can run a set of tasks, import or include a role, and then run more tasks. A potential disadvantage is that it might be less clear which roles your playbook uses without closely inspecting the playbook.

Use the `ansible.builtin.include_role` module to dynamically include a role, and use the `ansible.builtin.import_role` module to statically import a role.

The following playbook demonstrates how to include a role by using a task with the `ansible.builtin.include_role` module:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
---
- name: Executing a role as a task
  hosts: remote.example.com

  tasks:
    - name: A normal task
      ansible.builtin.debug:
        msg: 'first task'

    - name: A task to include role2 here
      ansible.builtin.include_role:
        name: role2

    - name: Another normal task
      ansible.builtin.debug:
        msg: 'second task'
```

When you use the `ansible.builtin.import_role` module, the `ansible-navigator run` command starts by parsing and inserting the role in the play before starting the execution. Ansible detects and reports syntax errors immediately, and does not start executing the playbook if it detects errors.

When you use the `ansible.builtin.include_role` module, however, Ansible parses and inserts the role in the play when it reaches the `ansible.builtin.include_role` task, during the play execution. If Ansible detects syntax errors in the role, then execution of the playbook is aborted.

If you use the `when` directive with the `ansible.builtin.include_role` module, then Ansible does not parse the role if the condition in the `when` directive is false.

Defining Pre- and Post-tasks

You might want a play that runs some tasks, and the handlers they notify, before your roles. You might also want to run tasks in the play after your normal tasks and handlers run. Two directives are available that you can use instead of `tasks` to do this:

- `pre_tasks` is a `tasks` section that runs before the `roles` section.
- `post_tasks` is a `tasks` section that runs after the `tasks` section and after any handlers notified by the `tasks` section.

The following playbook provides an example with `pre_tasks`, `roles`, `tasks`, `post_tasks`, and `handlers` sections. It is unusual for a play to contain all these sections.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
---
- name: Deploying New Application Version
  hosts: webservers

  pre_tasks:
    # Stop monitoring the web server to avoid sending false alarms
    # while the service is updating.
    - name: Disabling Nagios for this host
      community.general.nagios:
        action: disable_alerts
        host: "{{ inventory_hostname }}"
        services: http
      delegate_to: nagios-srv

  roles:
    - role: deploy-content

  tasks:
    - name: Restarting memcached
      ansible.builtin.service:
        name: memcached
        state: restarted
      notify: Notifying the support team

    # Confirm that the application is fully operational
    # after the update.
    - name: Validating the new deployment
      ansible.builtin.uri:
        url: http://{{ inventory_hostname }}/healthz
        return_content: true
      register: result
      failed_when: "'OK' not in result['content']"

  post_tasks:
    - name: Enabling Nagios for this host
      community.general.nagios:
        action: enable_alerts
        host: "{{ inventory_hostname }}"
        services: http
      delegate_to: nagios-srv

  handlers:
    # Send a message to the support team through Slack
    # every time the memcached service is restarted.
    - name: Notifying the support team
      community.general.slack:
```

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
        token: G922VJP25/D923DW937/3Ffe373sfhRE6y52Fg3rvf5GlK
        msg: |
          Memcached on {{ inventory_hostname }} restarted
      delegate_to: localhost
      become: false
```

Reviewing the Order of Execution

Ansible runs the play sections in the following order:

- `pre_tasks`
- Handlers that are notified in the `pre_tasks` section
- `roles`
- `tasks`
- Handlers that are notified in the `roles` and `tasks` sections
- `post_tasks`
- Handlers that are notified in the `post_tasks` section

The order of these sections in a play does not modify the order of execution. For example, if you write the `tasks` section before the `roles` section, Ansible still runs the roles before the tasks in the `tasks` section. For readability, however, it is a good practice to organize your play in the order of execution: `pre_tasks`, `roles`, `tasks`, and `post_tasks`. You usually define the handlers at the end of the play.

Ansible runs and flushes notified handlers at several points during a run: after the `pre_tasks` section, after the `roles` and `tasks` sections, and after the `post_tasks` section. This means that a handler can run more than one time, at different times during play execution, if notified in multiple sections.

To immediately run any handlers that have been notified by a particular task in the play, add a task that uses the `ansible.builtin.meta` module with the `flush_handlers` parameter. This enables you to define specific points during task execution when all notified handlers are run.

In the following example, the play runs the `Restart api server` handler, if notified, after deploying a new configuration file and before using the application API. Without this call to the `ansible.builtin.meta` module, the play only calls the handler after running all the tasks. If the handler has not run before the last task that uses the API, then that task might fail because the configuration file was updated, but the application has not yet reread the new configuration.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
---
- name: Updating the application configuration and cache
  hosts: app_servers

  tasks:
    - name: Deploying the configuration file
      ansible.builtin.template:
        src: api-server.cfg.j2
        dest: /etc/api-server.cfg
      notify: Restart api server

    - name: Running all notified handlers
      ansible.builtin.meta: flush_handlers

    - name: Asking the API server to rebuild its internal cache
      ansible.builtin.uri:
        url: https://{{ inventory_hostname }}/rest/api/2/cache/
        method: POST
        force_basic_auth: true
        user: admin
        password: redhat
        body_format: json
        body:
          type: data
          delay: 0
        status_code: 201

  handlers:
    - name: Restart api server
      ansible.builtin.service:
        name: api-server
        state: restarted
        enabled: true
```

Remember that in a play, handlers have global scope. A play can notify handlers defined in roles. One role can notify a handler defined by another role or by the play.

Ansible always runs handlers that have been notified in the order they are listed in the `handlers` section of the play, and not in the order in which they were notified.

Listening to Handlers

In addition to being notified by tasks, a handler can also *subscribe* to a specific notification, and run when that notification is triggered. This means that one notification can trigger multiple handlers.

By default, a handler runs when a notification string matches the handler name. However, because each handler must have a unique name, the only way to trigger multiple handlers at the same time is if each one subscribes to the same

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

notification name.

The following example shows a task that notifies `My handlers` when it changes, which is whenever it runs, because it has `changed_when: true` set. The task notifies the `My handlers` handler and any handler that lists `My handlers` in a `listen` directive.

```
---
- name: Testing the listen directive
  hosts: localhost
  gather_facts: false
  become: false

  tasks:
    - name: Trigger handlers
      ansible.builtin.debug:
        msg: Trigger the handlers
      notify: My handlers
      changed_when: true

  handlers:
    # Listening to the "My handlers" event
    - name: Listening to a notification
      ansible.builtin.debug:
        msg: First handler was notified
      listen: My handlers

    # As an example, this handler is also triggered because
    # its name matches the notification, but no two handlers
    # can have the same name.
    - name: My handlers
      ansible.builtin.debug:
        msg: Second handler was notified
```

> **Note**
> In the preceding playbook, it would be better if both handlers had unique names (not `My handlers`) and used `listen: My handlers`, because the playbook would be easier to read.

The `listen` directive is particularly helpful when used with roles. Roles use notifications to trigger their handlers. A role can document that it notifies a certain handler when an event occurs. Other roles or a play might be able to use this notification to run additional handlers defined outside the role.

For example, a role might notify one of its handlers when a service needs restarting. In your playbook, you can define a handler that listens to that notification and performs additional tasks when Ansible restarts the service, such as sending a message to a monitoring tool, or restarting a dependent service.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

As another example, you can create a role that ensures the validity of a given SSL certificate and notifies a handler that renews the certificate if it has expired. In a playbook, you can call this role to verify your Apache HTTP Server certificate validity. You can add a handler that listens for that SSL certificate renewal notification and that restarts the `httpd` service.

Notifying Handlers

To summarize, a task can notify multiple handlers in at least two ways:

- It can notify a list of handlers individually by name.
- It can notify one name for which multiple handlers are configured to listen.

If the handlers are reused as a set by multiple tasks, then the easiest way to configure your playbook is to use the second approach with the `listen` directive. With this approach, you need only change the handlers to mark whether they are included in the set or not, and to ensure that they are listed in the correct order.

This approach is useful when a task needs to notify a lot of handlers. Instead of notifying each handler by name, the task only sends one notification. Ansible triggers all the handlers listening to that notification. This way, you can add or remove handlers without updating the task.

In the first approach, you must find and edit every affected task to add the handler to the list. You must also ensure that the handlers are listed in the correct order in the `handlers` section.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Note

It is an error to have a task send a notification with no handler matching that notification:

```
[user@host ~]$ cat no_handler_error.yml
---
- name: Testing notification with no handler
  hosts: localhost
  gather_facts: false
  become: false

  tasks:
    - name: Task that changes
      ansible.builtin.debug:
        msg: Trigger a non-existent handler
      changed_when: true
      notify: Restart service

[user@host ~]$ ansible-navigator run -m stdout no_handler_error.yml

PLAY [Testing notification with no handler] ******************************

TASK [Task that changes] *************************************************
ERROR! The requested handler 'Restart service' was not found in either the
main handlers list nor in the listening handlers list
Please review the log for errors.
```

Controlling the Order of Host Execution

Ansible determines which hosts to manage for a play based on the `hosts` directive for the play. By default, Ansible runs the play against hosts in the order in which they are listed in the inventory. You can change that order on a play-by-play basis by using the `order` directive.

The following playbook alphabetically sorts the hosts in the `web_servers` group before running the task:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
---
- name: Testing host order
  hosts: web_servers
  order: sorted

  tasks:
    - name: Creating a test file in /tmp
      ansible.builtin.copy:
        content: 'This is a sample file'
        dest: /tmp/test.out
```

The `order` directive accepts the following values:

- `inventory` (default): Sorts the hosts based on the inventory order.
- `reverse_inventory`: Sorts the hosts based on the reverse of the inventory order.
- `sorted`: Sorts the hosts in alphabetical order. Numbers sort before letters.
- `reverse_sorted`: Sorts the hosts in reverse alphabetical order.
- `shuffle`: Randomizes the host list every time you run the play.

Note

Because Ansible normally runs each task in parallel on several hosts, the output of the `ansible-navigator` command might not reflect the expected order; the output shows the task completion order rather than the execution order.

For example, assume that the inventory is configured so that hosts are listed in alphabetical order, and plays are run on managed hosts in inventory order (the default behavior). You could still see the following output:

```
...output omitted...
TASK [Creating a test file in /tmp] *********************************
changed: [www2.example.com]
changed: [www3.example.com]
changed: [www1.example.com]
...output omitted...
```

Notice that the task on `www1.example.com` completed last.

References

Roles - Ansible Documentation

Handlers: Running Operations On Change - Ansible Documentation

Re-using Ansible Artifacts - Ansible Documentation

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

6.4. Guided Exercise
Controlling Task Execution

Use `pre_tasks` and `post_tasks` sections to control whether tasks run before or after roles, and `listen` directives to notify multiple handlers at the same time.

Outcomes

- Control the execution order of tasks.
- Trigger handlers using the `listen` directive.

Prerequisites

As the `student` user on the `workstation` machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
[student@workstation ~]$ lab start task-exec
```

Instructions

1. Review the provided `/home/student/task-exec/deploy.yml` playbook.

   1.1. From a terminal, change to the `/home/student/task-exec` directory:

   ```
   [student@workstation ~]$ cd ~/task-exec
   [student@workstation task-exec]$
   ```

   1.2. The `deploy.yml` playbook consists of one play. This play uses a `pre_tasks` section to include tasks that configure an Apache HTTPD web server, and then a `roles` section to run a role that allows access through the firewall.

   ```
   ---
   - name: Implementing Handlers
     hosts: web_servers
     become: true

     pre_tasks:
       - name: Configure Apache
         ansible.builtin.include_tasks: apache.yml

     roles:
       - role: firewall
   ```

2. Edit the `deploy.yml` playbook to add tasks that notify handlers.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

2.1. Use the `ansible.builtin.set_fact` module to assign the static values of `80/tcp` for the `web_port` variable and `public` for the `web_zone` variable. Use `changed_when: true` to guarantee that the task always notifies the `Display variables` handler.

The content for the `pre_tasks` section should read as follows:

```
  pre_tasks:
    - name: Configure Apache
      ansible.builtin.include_tasks: apache.yml

    - name: Set web_port and web_zone
      ansible.builtin.set_fact:
        web_port: 80/tcp
        web_zone: public
      changed_when: true
      notify: Display variables
```

2.2. After the `roles` section, add the `post_tasks` section with a task to copy the `index.html` file to the `/var/www/html` directory. Use the `ansible.builtin.copy` module to copy the file. Notify the `Verify connectivity` handler if the task changes the managed host.

The content for the `post_tasks` section should read as follows:

```
  roles:
    - role: firewall

  post_tasks:
    - name: Copy web content
      ansible.builtin.copy:
        src: index.html
        dest: /var/www/html/
        owner: root
        group: root
        mode: "0644"
      notify: Verify connectivity
```

> **Note**
> You could define the `Copy web content` task under a `tasks` section because Ansible also runs the `tasks` section after the `roles` section. However, to mirror the `pre_tasks` section, define the task under the `post_tasks` section.

3. Add the handlers for the play. Two handlers must listen for the `Display variables` notification to show the values for the `web_port` and the `web_zone` variables when they are defined. The same notification must trigger both handlers.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

In addition, add a handler named `Verify connectivity` that verifies that the web content at `http://{{ ansible_facts['fqdn'] }}` can be retrieved at the end of the first play if the play changed the content. This confirms that the play worked.

> **Important**
> Make sure that you create this `handlers` section in the `deploy.yml` playbook at the end of the play.

3.1. Create the `handlers` section at the end of the play, and include two similar handlers using the `ansible.builtin.debug` module to print the value of the `web_port` and the `web_zone` variables. Give the handlers an arbitrary string as a name, and use the `listen` directive for the `Display variables` notification.

The `handlers` section is highlighted and should consist of the following content:

```
  post_tasks:
    - name: Copy web content
      ansible.builtin.copy:
        src: index.html
        dest: /var/www/html/
        owner: root
        group: root
        mode: "0644"
      notify: Verify connectivity

  handlers:
    - name: Show web_port (configured in pre_tasks)
      ansible.builtin.debug:
        var: web_port
      listen: Display variables

    - name: Show web_zone (configured in pre_tasks)
      ansible.builtin.debug:
        var: web_zone
      listen: Display variables
```

3.2. Add the handler for the `Verify connectivity` notification. Use the `ansible.builtin.uri` module to verify that the web content is available:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
  handlers:
    - name: Show web_port (configured in pre_tasks)
      ansible.builtin.debug:
        var: web_port
      listen: Display variables

    - name: Show web_zone (configured in pre_tasks)
      ansible.builtin.debug:
        var: web_zone
      listen: Display variables

    - name: Verify connectivity
      ansible.builtin.uri:
        url: http://{{ ansible_facts['fqdn'] }}
        status_code: 200
```

The complete `deploy.yml` file should consist of the following content:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
---
- name: Implementing Handlers
  hosts: web_servers
  become: true

  pre_tasks:
    - name: Configure Apache
      ansible.builtin.include_tasks: apache.yml

    - name: Set web_port and web_zone
      ansible.builtin.set_fact:
        web_port: 80/tcp
        web_zone: public
      changed_when: true
      notify: Display variables

  roles:
    - role: firewall

  post_tasks:
    - name: Copy web content
      ansible.builtin.copy:
        src: index.html
        dest: /var/www/html/
        owner: root
        group: root
        mode: "0644"
      notify: Verify connectivity

  handlers:
    - name: Show web_port (configured in pre_tasks)
      ansible.builtin.debug:
        var: web_port
      listen: Display variables

    - name: Show web_zone (configured in pre_tasks)
      ansible.builtin.debug:
        var: web_zone
      listen: Display variables

    - name: Verify connectivity
      ansible.builtin.uri:
        url: http://{{ ansible_facts['fqdn'] }}
        status_code: 200
```

4. Run the `deploy.yml` Ansible Playbook to verify your work and confirm that Ansible triggers your handlers. Pay

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

attention to the order of execution of the handlers.

In particular, you should see that the handlers that are subscribed to the `Display variables` notification, called in the `pre_tasks` section, run before the `Verify connectivity` handler, which is called in the `post_tasks` sections.

4.1. Run the `deploy.yml` playbook:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation task-exec]$ ansible-navigator run deploy.yml

PLAY [Implementing Handlers]
*************************************************

TASK [Gathering Facts]
*************************************************************
ok: [serverc.lab.example.com]

TASK [Configure Apache]
*************************************************************
included: /home/student/task-exec/apache.yml for serverc.lab.example.com

TASK [Instal apache]
*************************************************************
changed: [serverc.lab.example.com]

TASK [Start apache]
*************************************************************
changed: [serverc.lab.example.com]

TASK [Set web_port and web_zone]
*********************************************
changed: [serverc.lab.example.com]

RUNNING HANDLER [Show web_port (configured in pre_tasks)]
**********************
ok: [serverc.lab.example.com] => {
    "web_port": "80/tcp"
}

RUNNING HANDLER [Show web_zone (configured in pre_tasks)]
**********************
ok: [serverc.lab.example.com] => {
    "web_zone": "public"
}

TASK [firewall : Ensure Firewall Port Configuration]
***************************
changed: [serverc.lab.example.com] => (item={'port': '80/tcp', 'zone':
'public'})

RUNNING HANDLER [firewall : Reload firewalld]
*********************************
changed: [serverc.lab.example.com]
```

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
TASK [Copy web content]
*************************************************************
changed: [serverc.lab.example.com]

RUNNING HANDLER [Verify connectivity]
*****************************************
ok: [serverc.lab.example.com]

PLAY RECAP
*************************************************************************
serverc.lab.example.com    : ok=11   changed=6    unreachable=0    failed=0
...
```

4.2. Use the `curl` command to verify that the playbook successfully updated the host:

```
[student@workstation task-exec]$ curl serverc.lab.example.com
This is the index file.
```

5. Update two tasks in the `revert.yml` playbook to notify handlers and add a new handler to the playbook.

5.1. Add a `notify: Package check` statement to the `Uninstall Apache` task. Add a `notify: Post handler` statement to the `Remove index.html file` task. After you modify the `revert.yml` file, the content of the playbook should read as follows:

```
...output omitted...
  tasks:
    - name: Uninstall Apache
      ansible.builtin.dnf:
        name: httpd
        state: absent
        autoremove: true
      notify: Package check

  post_tasks:
    - name: Remove index.html file
      ansible.builtin.file:
        path: /var/www/html/index.html
        state: absent
      notify: Post handler
...output omitted...
```

5.2. In the `revert.yml` playbook, create a handler named `Post handler` that uses the `ansible.builtin.debug` module to print `The index.html file was deleted` to `stdout`.

The complete `revert.yml` file should consist of the following content:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
---
- name: Revert web servers
  hosts: web_servers
  become: true
  gather_facts: false

  roles:
    - role: firewall
      vars:
        firewall_rules:
          - port: 80/tcp
            zone: public
            state: disabled

  tasks:
    - name: Uninstall Apache
      ansible.builtin.dnf:
        name: httpd
        state: absent
        autoremove: true
      notify: Package check

  post_tasks:
    - name: Remove index.html file
      ansible.builtin.file:
        path: /var/www/html/index.html
        state: absent
      notify: Post handler

  handlers:
    - name: Gather package facts
      ansible.builtin.package_facts:
        manager: auto
      listen: Package check

    - name: Check for the httpd package
      when: "'httpd' not in ansible_facts['packages']"
      ansible.builtin.debug:
        msg: httpd is not installed!
      listen: Package check

    - name: Post handler
      ansible.builtin.debug:
        msg: The index.html file was deleted.
```

6. Run the `revert.yml` Ansible Playbook to verify your work and confirm that Ansible triggers your handlers. Pay

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

attention to the order of execution of the handlers.

6.1. Run the `revert.yml` playbook:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation task-exec]$ ansible-navigator run revert.yml

PLAY [Revert web servers]
*****************************************************

TASK [firewall : Ensure Firewall Port Configuration]
**************************
changed: [serverc.lab.example.com] => (item={'port': '80/tcp', 'zone':
'public', 'state': 'disabled'})

TASK [Uninstall Apache]
*************************************************************
changed: [serverc.lab.example.com]

RUNNING HANDLER [firewall : Reload firewalld]
*********************************
changed: [serverc.lab.example.com]

RUNNING HANDLER [Gather package facts]
*****************************************
ok: [serverc.lab.example.com]

RUNNING HANDLER [Check for the httpd package]
*********************************
ok: [serverc.lab.example.com] => {
    "msg": "httpd is not installed!"
}

TASK [Remove index.html file]
************************************************
changed: [serverc.lab.example.com]

RUNNING HANDLER [Post handler]
************************************************
ok: [serverc.lab.example.com] => {
    "msg": "The index.html file was deleted."
}

PLAY RECAP
*********************************************************************
serverc.lab.example.com    : ok=7    changed=4    unreachable=0    failed=0
...
```

6.2. Use the `curl` command to verify that the playbook successfully updated the host:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation task-exec]$ curl serverc.lab.example.com
curl: (7) Failed to connect to serverc.lab.example.com port 80: No route to
host
```

Finish

On the `workstation` machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish task-exec
```

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

6.5. Running Selected Tasks

Objectives

- Label tasks with tags, run only tasks labeled with specific tags, or start playbook execution at a specific task.

Tagging Ansible Resources

When working with a large or complex playbook, you might consider running only a subset of its plays or tasks. You can apply *tags* to specific resources that you want to skip or run. A tag is a text label on an Ansible resource, such as a play or task. To tag a resource, use the `tags` keyword on the resource, followed by a list of the tags to apply.

When running a playbook with `ansible-navigator`, use the `--tags` option to filter the playbook and run only specific tagged plays or tasks. Tags are available for the following resources:

Entire plays

Use the `tags` directive at the play level to tag all tasks in the play:

```
---
- name: Set up web services
  hosts: webservers
  tags: setup
...output omitted...
```

Individual tasks

Use the `tags` directive on individual tasks:

```
- name: Install software
  tags: install
  ansible.builtin.dnf:
    name: "{{ packages }}"
    state: present
```

Included task files

Use the `tags` directive to tag all the tasks loaded by the `ansible.builtin.import_tasks` module:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
- name: Import common web services tasks
  tags:
    - webproxy
    - webserver
  ansible.builtin.import_tasks: common.yml
```

Roles in the `roles` section

Use the `tags` directive to tag all tasks in a role. In this example, the `databases` role has two tags, `production` and `staging`:

```
  roles:
    - { role: databases, tags: ['production', 'staging'] }
```

Blocks of tasks

Use the `tags` directive to tag all tasks in a block. In this example, you group all `httpd`-related tasks under the `webserver` tag:

```
- name: Install packages and start httpd service
  tags: webserver
  block:
    - name: Install software
      ansible.builtin.dnf:
        name:
          - httpd
          - php
          - git
          - php-mysqlnd
        state: present

    - name: Allow httpd connections to a remote database
      ansible.posix.seboolean:
        name: httpd_can_network_connect_db
        state: true
        persistent: true

    - name: Start and enable service
      ansible.builtin.service:
        name: httpd
        state: started
        enabled: true
```

Important

How Ansible applies tags to roles and task files depends on whether the role or task file was *imported* or *included*.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

> When you tag an *imported* resource, Ansible applies the tag to all tasks in the imported role or task file. Imported resources could include a role that uses the `roles` directive, or a task that uses the `ansible.builtin.import_role` or `ansible.builtin.import_tasks` modules.
>
> When you tag an *included* resource, Ansible applies the tag to the task itself, and only runs tasks in the included role or task file if those tasks also use the same tag, or the `always` tag. Included resources could include a task that uses the `ansible.builtin.include_role` or `ansible.builtin.include_tasks` modules.
>
> If you plan to include a role or a task file, then you must also tag the tasks in the role or task file.

Managing Tagged Resources

Use the `ansible-navigator` command to run tasks with a specific tag, using the `--tags` option, or skip tasks with a specific tag, using the `--skip-tags` option.

Running Tasks with Specific Tags

The following playbook contains two tasks. The first task is tagged with the `webserver` tag. The second task does not have any associated tags.

```
---
- name: Example play using tagging
  hosts:
    - servera.lab.example.com
    - serverb.lab.example.com

  tasks:
    - name: Install httpd
      tags: webserver
      ansible.builtin.dnf:
        name: httpd
        state: latest

    - name: Install postfix
      ansible.builtin.dnf:
        name: postfix
        state: latest
```

Add the `--tags webserver` option to only run the first task:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[user@host ~]$ ansible-navigator run main.yml --tags webserver

PLAY [Example play using tagging] ********************************************

TASK [Gathering Facts] *******************************************************
ok: [serverb.lab.example.com]
ok: [servera.lab.example.com]

TASK [Install httpd] *********************************************************
ok: [servera.lab.example.com]
ok: [serverb.lab.example.com]

PLAY RECAP *******************************************************************
servera.lab.example.com    : ok=2    changed=1    unreachable=0    failed=0 ...
serverb.lab.example.com    : ok=2    changed=0    unreachable=0    failed=0 ...
```

Because the `--tags webserver` option was specified, the playbook ran only the task tagged with the `webserver` tag.

Skipping Tasks with Specific Tags

You can use the `--skip-tags` option to skip tasks with a specific tag and only run the tasks without that tag.

```
[user@host ~]$ ansible-navigator run main.yml --skip-tags webserver

PLAY [Example play using tagging] ********************************************

TASK [Gathering Facts] *******************************************************
ok: [serverb.lab.example.com]
ok: [servera.lab.example.com]

TASK [Install postfix] *******************************************************
ok: [serverb.lab.example.com]
ok: [servera.lab.example.com]

PLAY RECAP *******************************************************************
servera.lab.example.com    : ok=2    changed=1    unreachable=0    failed=0 ...
serverb.lab.example.com    : ok=2    changed=1    unreachable=0    failed=0 ...
```

Combining Tags to Run Multiple Tasks

You can associate multiple tags with the `--tags` and `--skip-tags` options using a comma-separated list.

```
[user@host ~]$ ansible-navigator run main.yml --tags install,setup
```

Listing Tags in a Playbook

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

To list all tags that exist in a playbook, pass the `--list-tags` option to the `ansible-navigator` command. For example:

```
[user@host examples]$ ansible-navigator run playbook.yml --list-tags

playbook: playbook.yml

  play #1 (webservers): Setup web services      TAGS: [setup]
      TASK TAGS: [setup]

  play #2 (webservers): Teardown web services   TAGS: [teardown]
      TASK TAGS: [teardown]
```

Assigning Special Tags

Ansible has a special tag that can be assigned in a playbook: `always`. A resource tagged `always` runs every time, even if it does not match the list of tags passed to the `--tags` option. The only exception is when it is explicitly skipped, using the `--skip-tags always` option.

A task that you tag with the `never` tag does not run, unless you run the playbook with the `--tags` option set to `never` or to one of the other tags associated with the task.

Ansible supports three additional special tags:

`tagged`
: Runs any resource with an explicit tag.

`untagged`
: Runs any resource that does not have an explicit tag, and excludes all tagged resources.

`all`
: Includes all tasks in the play, whether they have a tag or not. This is the default behavior of Ansible.

> **References**
> Tags - Ansible Documentation

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

6.6. Guided Exercise
Running Selected Tasks

Use tags to run only specific tasks in a playbook.

Outcomes

- Tag specific tasks in a playbook to run or skip them.

Prerequisites

As the `student` user on the `workstation` machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
[student@workstation ~]$ lab start task-tags
```

Instructions

1. Clone the `https://git.lab.example.com/student/task-tags.git` repository to the `/home/student/git-repos` directory and then create a branch for this exercise.

   1.1. From a terminal, create the `/home/student/git-repos` directory if it does not already exist, and then change into it:

   ```
   [student@workstation ~]$ mkdir -p ~/git-repos/
   [student@workstation ~]$ cd ~/git-repos/
   [student@workstation git-repos]$
   ```

   1.2. Clone the `https://git.lab.example.com/student/task-tags.git` repository and then change directory to the cloned repository:

   ```
   [student@workstation git-repos]$ git clone \
   https://git.lab.example.com/student/task-tags.git
   Cloning into 'task-tags'...
   ...output omitted...
   [student@workstation git-repos]$ cd task-tags
   [student@workstation task-tags]$
   ```

   1.3. Create the `exercise` branch and check it out:

   ```
   [student@workstation task-tags]$ git checkout -b exercise
   Switched to a new branch 'exercise'
   ```

2. Create a playbook to test the successful deployment of the web application.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

2.1. Create the `test_webapp.yml` playbook.

- Add a task that queries the load balance server for each host in the `web_servers` inventory group. Register the output of that task.
- Add a second task that displays the content of each web server query.
- Make sure to add the `tests` tag to each of the tasks.

The file should consist of the following content:

```
---
- name: Web application smoke test
  hosts: lb_servers
  gather_facts: true
  tasks:
    - name: Query web content
      tags: tests
      ansible.builtin.uri:
        url: http://{{ ansible_facts['fqdn'] }}
        return_content: true
      register: test_url
      loop: "{{ groups['web_servers'] }}"

    - name: Show web content
      tags: tests
      ansible.builtin.debug:
        var: test_url['results'] | map(attribute='content')
```

Note

The second task displays the `content` key (or attribute) for each result item in the registered `test_url` variable. Filters, such as the `map` filter, are covered later in the course.

To display everything captured in the `test_url` variable, you could add the following task to the play:

```
    - name: Show test_url
      ansible.builtin.debug:
        var: test_url
```

2.2. Add the new playbook to the `site.yml` file. The file should consist of the following content:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
---
- name: Deploy HAProxy
  ansible.builtin.import_playbook: deploy_haproxy.yml

- name: Deploy Web Server
  ansible.builtin.import_playbook: deploy_apache.yml

- name: Deploy Web App
  ansible.builtin.import_playbook: deploy_webapp.yml

- name: Test Web App
  ansible.builtin.import_playbook: test_webapp.yml
```

3. Use the `site.yml` playbook to only test the web application deployment.

   After the test fails, run the playbook to configure the environment, skipping all tests.

   3.1. Use the `tests` tag to test the web application before it is set up:

```
[student@workstation task-tags]$ ansible-navigator run site.yml --tags tests
...output omitted...
TASK [Query web content]
*********************************************************
failed: [servera.lab.example.com] (item=serverb.lab.example.com) =>
{"ansible_facts": {"discovered_interpreter_python": "/usr/bin/python3"},
"ansible_loop_var": "item", "changed": false, "content": "", "elapsed": 0,
"item": "serverb.lab.example.com", "msg": "Status code was -1 and not [200]:
Request failed: <urlopen error [Errno 111] Connection refused>",
"redirected": false, "status": -1, "url": "http://servera.lab.example.com"}
failed: [servera.lab.example.com] (item=serverc.lab.example.com) =>
{"ansible_loop_var": "item", "changed": false, "content": "", "elapsed": 0,
"item": "serverc.lab.example.com", "msg": "Status code was -1 and not [200]:
Request failed: <urlopen error [Errno 111] Connection refused>",
"redirected": false, "status": -1, "url": "http://servera.lab.example.com"}

PLAY RECAP
*********************************************************************
servera.lab.example.com    : ok=1    changed=0    unreachable=0    failed=1
...

Please review the log for errors.
```

   This command only ran the tasks tagged with the `tests` tag; because the server was not set up, the test failed.

   3.2. Run the playbook, skipping the tests to set up the web application:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation task-tags]$ ansible-navigator run site.yml \
--skip-tags tests
...output omitted...
PLAY [Web application smoke test]
*********************************************

TASK [Gathering Facts]
*********************************************************
ok: [servera.lab.example.com]

PLAY RECAP
*********************************************************************
servera.lab.example.com    : ok=8    changed=6    unreachable=0    failed=0
...
serverb.lab.example.com    : ok=7    changed=7    unreachable=0    failed=0
...
serverc.lab.example.com    : ok=7    changed=7    unreachable=0    failed=0
...
```

3.3. Use the `tests` tag again to test the web application. Now that the web application has been deployed, the test succeeds.

```
[student@workstation task-tags]$ ansible-navigator run site.yml --tags tests
...output omitted...
TASK [Query web content]
*****************************************************
ok: [servera.lab.example.com] => (item=serverb.lab.example.com)
ok: [servera.lab.example.com] => (item=serverc.lab.example.com)

TASK [Show web content]
******************************************************
ok: [servera.lab.example.com] => {
    "test_url['results'] | map(attribute='content')": [
        "Hello from serverb.lab.example.com. (version v1.0)\n",
        "Hello from serverc.lab.example.com. (version v1.0)\n"
    ]
}

PLAY RECAP
*********************************************************************
servera.lab.example.com    : ok=3    changed=0    unreachable=0    failed=0
...
```

Finish

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

On the `workstation` machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish task-tags
```

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

6.7. Optimizing Execution for Speed

Objectives

- Optimize your playbook to run more efficiently, and use callback plug-ins to profile and analyze which tasks consume the most time.

Optimizing Playbook Execution

You can optimize your Ansible Playbooks in a number of ways. Writing efficient playbooks becomes increasingly important as the number of hosts you manage increases.

Optimizing the Infrastructure

Each release of Red Hat Ansible Automation Platform adds enhancements and improvements. Using the latest version might help increase the speed of your playbooks as the core components of Ansible and especially the modules provided with it are optimized over time.

An architectural optimization that you can make is to keep your control node close to the managed nodes from a networking perspective. Ansible relies heavily on network communication and the transfer of data. High latency connections or low bandwidth between the control node and its managed hosts degrade the execution time of playbooks.

Disabling Fact Gathering

Each play has a hidden task that runs first, using the `ansible.builtin.setup` module to collect facts from each host. Those facts provide information about each managed host, and you can use the `ansible_facts` variable in your plays to use that information.

Collecting the facts from each remote host takes time. If you do not use those facts in your play, then you can skip the fact gathering task by setting the `gather_facts` directive to `false`.

The following example disables fact gathering:

```
---
- name: Demonstrate disabling the facts gathering
  hosts: web_servers
  gather_facts: false
...output omitted...
```

Playbooks often use the `ansible_facts['hostname']`, `ansible_hostname`, `ansible_facts['nodename']`, or `ansible_nodename` variables to refer to the host currently being processed. Those variables come from the fact gathering task, but you can usually replace them with the `inventory_hostname` and `inventory_hostname_short` special variables if your inventory uses actual hostnames.

Even if you disable fact gathering, you can choose to collect facts manually at any point in a play by running the `ansible.builtin.setup` module as a task, and the collected facts are then available for subsequent plays in the

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

playbook.

Reusing Gathered Facts with Fact Caching

Ansible uses *cache plug-ins* to store gathered facts or inventory source data gathered by a play. You can take advantage of the fact cache to limit how many times you need to gather facts by reusing facts gathered earlier.

Fact caching is always enabled. You can only use one cache plug-in at a time. The `memory` cache plug-in is enabled by default if you do not change the `ansible-navigator` configuration. This plug-in caches facts gathered during the current Ansible run.

You can take advantage of this to improve performance for playbooks that contain multiple plays. The first play can gather facts for all the hosts for which you need facts in the playbook. Subsequent plays can then disable fact gathering and use the cached facts from the first play, improving performance.

The following playbook contains two simple plays and illustrates how this works:

```
---
- name: The first play gather facts for all hosts
  hosts: all
  gather_facts: true
  tasks: []

- name: The second play does not gather facts
  hosts: all
  gather_facts: false
  tasks:
    - name: Show ansible_facts for each host
      ansible.builtin.debug:
        var: ansible_facts
```

Note

Another way to use fact caching is to use `smart` gathering. You can set the following option for the `gathering` key in the `ansible.cfg` file to enable `smart` gathering:

```
[defaults]
gathering=smart
```

When enabled, `smart` gathering gathers facts from each new host in a playbook run, but if the same host is used across multiple plays, then the host is not contacted for fact gathering again in the run.

If you enable `smart` gathering, then removing both `gather_facts` lines in the previous playbook sample produces the same output. The first play gathers facts for all hosts, but the second play does not gather facts because they have already been gathered.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Fact caching also works on automation controller by default. In addition, when you edit a job template in automation controller, you can select the **Enable fact storage** checkbox. This changes the fact caching plug-in to one that stores facts gathered by jobs launched by that template, so that they can be reused between *multiple* playbook runs. (You need to periodically run a job that gathers facts to update the automation controller fact storage if you use this feature.)

If that checkbox is *not* selected, automation controller uses the default `memory` fact caching plug-in, which only caches facts during a particular job run.

> **Important**
> If facts might change from play to play in the same playbook, the disadvantage of relying on fact caching with the `memory` plug-in is that you do not get updated facts for the later plays. You can, of course, gather facts for any play that might be affected, or run the `ansible.builtin.setup` module manually as a task in a play that might be affected.

Limiting Fact Gathering

You can also selectively limit fact gathering if you disable automatic fact gathering and run the `ansible.builtin.setup` module as an explicit task with its `gather_subset` option. This is generally quicker than gathering all available facts.

Some possible subsets are `all`, `min`, `hardware`, `network`, `virtual`, `ohai`, and `facter`. If you exclude the `all` subset, then you still get the `min` subset.

For example, if you only want to retrieve facts in the `network` subset, you can include it but exclude `all` and `min`:

```
- name: A play that gathers some facts
  hosts: all
  gather_facts: false
  tasks:
    - name: Collect only network-related facts
      ansible.builtin.setup:
        gather_subset:
          - '!all'
          - '!min'
          - network
```

Increasing Parallelism

When Ansible is running a play, it runs the first task on every host in the current batch, and then runs the second task on every host in the current batch, and so on until the play completes. The `forks` parameter controls how many connections Ansible can have active at the same time. By default, this is set to 5, which means that even if there are 100 hosts to process on the current task, Ansible only communicates with them in groups of five. After it has communicated with all 100 hosts, Ansible moves to the next task.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

By increasing the `forks` value, Ansible runs each task simultaneously on more hosts, and the playbook usually completes in less time. For example, if you set `forks` to `100`, Ansible can attempt to open connections to all 100 hosts in the previous example simultaneously. This places more load on the control node, which still needs enough time to communicate with each of the hosts.

You can specify the number of forks to use in the Ansible configuration file by using the `-f` option with the `ansible-navigator` command, or in the `Forks` field of a job template in automation controller.

The following example shows the `forks` parameter set to `100` under the `[defaults]` section of the `ansible.cfg` configuration file:

```
[defaults]
forks=100
```

Because the `forks` value specifies how many worker processes Ansible starts, a number that is too high might slow down your control node and your network. Try first with a conservative value, such as `20` or `50`, and increase that number, each time monitoring your system resources.

Avoiding Loops with the Package Manager Modules

Some modules accept a list of items to work on and do not require the use of a loop. This approach can increase efficiency, because the module is only called one time.

The modules for managing operating system packages work this way. The following example uses the `ansible.builtin.dnf` module to install several packages in a single transaction, which is the most efficient way to install a group of packages:

```
---
- name: Install the packages on the web servers
  hosts: web_servers
  become: true
  gather_facts: false
  tasks:
    - name: Ensure the packages are installed
      ansible.builtin.dnf:
        name:
          - httpd
          - mod_ssl
          - httpd-tools
          - mariadb-server
          - mariadb
          - php
          - php-mysqlnd
        state: present
```

The preceding playbook would be equivalent to running the following command from the shell prompt:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[root@host ~]# dnf install httpd mod_ssl httpd-tools \
mariadb-server mariadb php php-mysqlnd
```

The following example is *not* efficient. It uses a loop to install the packages one at a time.

```
---
- name: Install the packages on the web servers
  hosts: web_servers
  become: true
  gather_facts: false
  tasks:
    - name: Ensure the packages are installed
      ansible.builtin.dnf:
        name: "{{ item }}"
        state: present
      loop:
        - httpd
        - mod_ssl
        - httpd-tools
        - mariadb-server
        - mariadb
        - php
        - php-mysqlnd
```

The second example is equivalent to running multiple `dnf` commands, as follows:

```
[root@host ~]# dnf install httpd
[root@host ~]# dnf install mod_ssl
[root@host ~]# dnf install httpd-tools
[root@host ~]# dnf install mariadb-server
[root@host ~]# dnf install mariadb
[root@host ~]# dnf install php
[root@host ~]# dnf install php-mysqlnd
```

The second example is slower and less efficient because Ansible runs the `ansible.builtin.dnf` module seven times, starting a process for the module seven times, and doing dependency resolution seven times.

Not all Ansible modules accept a list for the `name` parameter. For example, the `ansible.builtin.service` module only accepts a single value for its `name` parameter, and you need a loop to operate on multiple items.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
- name: Starting the services on the web servers
  hosts: web_servers
  become: true
  gather_facts: false
  tasks:
    - name: Ensure the services are started
      ansible.builtin.service:
        name: "{{ item }}"
        state: started
        enabled: true
      loop:
        - httpd
        - mariadb
```

Use the `ansible-navigator doc` command to get information about what types of values different module arguments can accept:

```
[user@host ~]$ ansible-navigator doc ansible.builtin.dnf -m stdout
...output omitted...
- name
...output omitted...
        url or a local path to an rpm file. To operate on several packages this
        can accept a comma separated string of packages or a list of packages.
...output omitted...

[user@host ~]$ ansible-navigator doc ansible.builtin.service -m stdout
...output omitted...
= name
        Name of the service.
        type: str
...output omitted...
```

Efficiently Copying Files to Managed Hosts

The `ansible.builtin.copy` module recursively copies files and directories to managed hosts. When the directory is large, with many files, the copy can take a long time. If you run the playbook multiple times, subsequent copies take less time because the module only copies the files that are different.

However, it is generally more efficient to use the `ansible.posix.synchronize` module to copy large numbers of files to managed hosts. This module uses `rsync` in the background and is usually faster than the `ansible.builtin.copy` module. By setting the `delete` option to `true`, the module can also remove files on the target that no longer exist on the source.

The following playbook uses the `ansible.posix.synchronize` module to recursively copy the `web_content` directory to the web servers:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
---
- name: Deploy the web content on the web servers
  hosts: web_servers
  become: true
  gather_facts: false
  tasks:
    - name: Ensure web content is updated
      ansible.posix.synchronize:
        src: web_content/
        dest: /var/www/html
```

Using Templates

The `ansible.builtin.lineinfile` module inserts or removes lines in a file, such as configuration directives in a configuration file. The following playbook updates the Apache HTTP Server configuration file by replacing several lines:

```
---
- name: Configure the Apache HTTP Server
  hosts: web_servers
  become: true
  gather_facts: false
  tasks:
    - name: Ensure proper configuration of the Apache HTTP Server
      ansible.builtin.lineinfile:
        dest: /etc/httpd/conf/httpd.conf
        regexp: "{{ item['regexp'] }}"
        line: "{{ item['line'] }}"
        state: present
      loop:
        - regexp: '^Listen 80$'
          line: 'Listen 8181'
        - regexp: '^ServerAdmin root@localhost'
          line: 'ServerAdmin support@example.com'
        - regexp: '^DocumentRoot "/var/www/html"'
          line: 'DocumentRoot "/var/www/web"'
        - regexp: '^<Directory "/var/www/html">'
          line: '<Directory "/var/www/web">'
```

When used with a loop, the `ansible.builtin.lineinfile` module is inefficient (and can be error-prone). In this situation, use either the `ansible.builtin.template` or the `ansible.builtin.copy` module instead:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
---
- name: Configure the Apache HTTP Server
  hosts: web_servers
  become: true
  gather_facts: false
  tasks:
    - name: Ensure proper configuration of the Apache HTTP Server
      ansible.builtin.template:
        src: templates/httpd.conf.j2
        dest: /etc/httpd/conf/httpd.conf
```

The `httpd.conf.j2` template file in the previous example is the customized version of the `httpd.conf` file.

Enabling Pipelining

To run a task on a remote node, Ansible performs several SSH operations to copy the module and all its data to the remote node and to run the module. To increase the performance of your playbook, you can activate the pipelining feature. With pipelining, Ansible establishes fewer SSH connections.

To activate pipelining, set the ANSIBLE_PIPELINING environment variable to `true` in the `execution-environment` section of the `ansible-navigator.yml` configuration file:

```
---
ansible-navigator:
  ansible:
    config: ./ansible.cfg

  execution-environment:
    image: ee-supported-rhel8:latest
    pull-policy: missing
    environment-variables:
      set:
        ANSIBLE_PIPELINING: true
```

Ansible does not use pipelining by default because the feature requires that the `requiretty` sudo option on all the remote nodes be disabled. On Red Hat Enterprise Linux 8, that sudo option is disabled by default, but it might be active on other systems.

To disable the option, use the `visudo` command to edit the `/etc/sudoers` file on your managed nodes and disable the `requiretty` option:

```
[root@host ~]# visudo
...output omitted...
Defaults !requiretty
```

Profiling Playbook Execution with Callback Plug-ins

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Callback plug-ins extend Ansible by adjusting how it responds to various events. Some of these plug-ins modify the output of the command-line tools, such as the `ansible-navigator` command, to provide additional information. For example, the `ansible.posix.timer` plug-in shows the playbook execution time in the output of the `ansible-navigator` command.

> **Important**
> Automation controller logs some information about jobs (playbook runs), which it extracts from the output of `ansible-navigator`. Because some callback plug-ins modify this output, you should use them with caution or avoid using them entirely, especially if you run the playbook with automation controller.

Ansible Automation Platform ships with a collection of callback plug-ins that you can enable in the `ansible.cfg` file by using the `callbacks_enabled` directive:

```
[defaults]
callbacks_enabled=ansible.posix.timer, ansible.posix.profile_tasks
```

Use the `ansible-navigator doc -t callback -l -m stdout` command to list the available callback plug-ins:

```
[user@host ~]$ ansible-navigator doc -t callback -l -m stdout
amazon.aws.aws_resource_actions summarizes all "resource:actions" completed
ansible.builtin.default         default Ansible screen output
ansible.builtin.junit           write playbook output to a JUnit file
ansible.builtin.minimal         minimal Ansible screen output
ansible.builtin.oneline         oneline Ansible screen output
ansible.builtin.tree            Save host events to files
ansible.legacy.awx_display      Playbook event dispatcher for ansible-runner
ansible.posix.cgroup_perf_recap Profiles system activity of tasks and full
execution using cgroups
ansible.posix.debug             formatted stdout/stderr display
ansible.posix.json              Ansible screen output as JSON
ansible.posix.jsonl             Ansible screen output as JSONL (lines in json
format)
ansible.posix.profile_roles     adds timing information to roles
ansible.posix.profile_tasks     adds time information to tasks
ansible.posix.skippy            Ansible screen output that ignores skipped status
ansible.posix.timer             Adds time to play stats
redhat.rhv.stdout               Output the log of ansible
redhat.satellite.foreman        Sends events to Foreman
```

Run the `ansible-navigator doc -t callback` *`plug-in-name`* `-m stdout` command to access the documentation for a specific plug-in:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[user@host ~]$ ansible-navigator doc -t callback \
ansible.posix.cgroup_perf_recap -m stdout
> ANSIBLE.POSIX.CGROUP_PERF_RECAP
(/usr/share/ansible/collections/ansible_collections/ansib>

        This is an ansible callback plugin utilizes cgroups to profile
        system activity of ansible and individual tasks, and display a recap
        at the end of the playbook execution

OPTIONS (= is mandatory):

= control_group
        Name of cgroups control group
        set_via:
          env:
          - name: CGROUP_CONTROL_GROUP
          ini:
          - key: control_group
            section: callback_cgroup_perf_recap

...output omitted...
```

Timing Tasks and Roles

You can use the `ansible.posix.timer`, `ansible.posix.profile_tasks`, and `ansible.posix.profile_roles` callback plug-ins to help identify slow tasks and roles.

The `ansible.posix.timer` plug-in displays the duration of playbook execution.

The `ansible.posix.profile_tasks` plug-in displays the start time of each task, and the time spent on each task, sorted in descending order, at the end of the playbook execution.

The `ansible.posix.profile_roles` plug-in displays the time spent on each role at the end of the output, sorted in descending order.

To activate these plug-ins, add or update the `callbacks_enabled` directive in the `ansible.cfg` file. Ensure that all the plug-ins are on a single line, as follows:

```
[defaults]
callbacks_enabled=ansible.posix.timer, ansible.posix.profile_tasks,
ansible.posix.profile_roles
```

You do not have to enable all three plug-ins; select the ones that you need.

The following example shows the output of the `ansible-navigator` command when you activate the three plug-ins:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[user@host ~]$ ansible-navigator run deploy_webservers.yml
...output omitted...
PLAY RECAP *********************************************************************
www1.example.com    : ok=9    changed=7    unreachable=0 ...
www2.example.com    : ok=10   changed=9    unreachable=0 ...
www3.example.com    : ok=10   changed=9    unreachable=0 ...

Playbook run took 0 days, 0 hours, 0 minutes, 21 seconds
Wednesday 30 April 2025  19:12:31 +0000 (0:00:00.858)       0:00:21.325 *******
===============================================================================
apache : Ensure httpd packages are installed --------------------------- 7.14s
haproxy : Ensure haproxy packages are present -------------------------- 1.78s
apache : Ensure SELinux allows httpd connections to a remote database --- 1.72s
Gathering Facts -------------------------------------------------------- 3.89s
haproxy : Ensure haproxy configuration is set -------------------------- 1.12s
haproxy : Ensure haproxy is started and enabled ------------------------ 0.92s
webapp : Ensure stub web content is deployed --------------------------- 0.86s
firewall : Ensure Firewall Sources Configuration ----------------------- 0.85s
firewall : Ensure Firewall Port Configuration -------------------------- 0.84s
firewall : Ensure Firewall Service Configuration ----------------------- 0.79s
apache : Ensure httpd service is started and enabled ------------------- 0.71s
firewall : Firewall Port Configuration --------------------------------- 0.70s
Wednesday 08 September 2021  19:12:31 +0000 (0:00:00.867)       0:00:21.333 ***
===============================================================================
apache ----------------------------------------------------------------- 9.57s
gather_facts ----------------------------------------------------------- 3.89s
haproxy ---------------------------------------------------------------- 3.81s
firewall --------------------------------------------------------------- 3.18s
webapp ----------------------------------------------------------------- 0.86s
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
total ------------------------------------------------------------------ 21.31s
```

References

`ssh_config`(5) and `sudoers`(5) man pages

ansible.builtin.setup module - Gathers Facts about Remote Hosts

Ansible Configuration Settings - Ansible Documentation

Callback Plugins - Ansible Documentation

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

6.8. Guided Exercise
Optimizing Execution for Speed

Optimize a playbook for more efficient execution, analyzing the playbook's performance with callback plug-ins.

Outcomes

- Activate Ansible callback plug-ins in the configuration file.
- Identify the playbook tasks that take the most time to complete.
- Optimize a playbook.

Prerequisites

As the `student` user on the `workstation` machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
[student@workstation ~]$ lab start task-speed
```

Instructions

1. Change to the `/home/student/task-speed` directory and review the `deploy_webservers.yml` playbook:

```
[student@workstation ~]$ cd ~/task-speed
[student@workstation task-speed]$
```

   1.1. Review the contents of the `deploy_webservers.yml` playbook:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
---
- name: Deploy the web servers
  hosts: web_servers
  become: true
  tasks:
    - name: Install software
      ansible.builtin.dnf:
        name: "{{ item }}"
        state: present
      loop:
        - httpd
        - mod_ssl
        - httpd-tools
        - mariadb-server
        - mariadb
        - php
        - php-mysqlnd

    - name: Start and enable services
      ansible.builtin.service:
        name: "{{ item }}"
        state: started
        enabled: true
      loop:
        - httpd
        - mariadb

    - name: Populate web content
      ansible.builtin.copy:
        src: web_content/
        dest: /var/www/html
```

The playbook installs packages, starts services, and recursively copies a local directory to the managed nodes.

2. Activate the `ansible.posix.timer` and the `ansible.posix.profile_tasks` callback plug-ins, and then run the `deploy_webservers.yml` playbook.

   2.1. Edit the `ansible.cfg` file to add the `callbacks_enabled` directive with the two callback plug-ins that you need to activate:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[defaults]
inventory=inventory.yml
remote_user=devops
callbacks_enabled=ansible.posix.timer, ansible.posix.profile_tasks
```

2.2. Run the `deploy_webservers.yml` playbook, and take note of the total elapsed time:

```
[student@workstation task-speed]$ ansible-navigator run
deploy_webservers.yml
...output omitted...
PLAY RECAP
*********************************************************************
serverb.lab.example.com    : ok=4    changed=3    unreachable=0    failed=0
...
serverc.lab.example.com    : ok=4    changed=3    unreachable=0    failed=0
...

Playbook run took 0 days, 0 hours, 1 minutes, 44 seconds
Wednesday 30 April 2025  13:24:38 +0000 (0:00:42.475)       0:01:44.618
*******
===============================================================================
Install software ------------------------------------------------------
50.68s
Populate web content --------------------------------------------------
42.48s
Start and enable services ---------------------------------------------
9.10s
Gathering Facts -------------------------------------------------------
2.33s
```

The playbook and task execution times are probably different on your system. However, package installations and web content deployment should be the most time-consuming tasks.

3. Optimize the `deploy_webservers.yml` playbook. Disable fact gathering, because the playbook does not use facts. Also remove the loop in the package installation task and replace the `ansible.builtin.copy` module with the `ansible.posix.synchronize` module.

If you make a mistake, then you can restore the original file from the `deploy_webservers.yml.backup` file.

3.1. Edit the `deploy_webservers.yml` playbook. Add the `gather_facts` directive and set it to `false`. Do not close the file yet.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
---
- name: Deploy the web servers
  hosts: web_servers
  become: true
  gather_facts: false
...output omitted...
```

3.2. Remove the loop from the package installation task. To do so, move the list of packages under the `name` parameter and remove the `loop` directive:

```
...output omitted...
    - name: Install software
      ansible.builtin.dnf:
        name:
          - httpd
          - mod_ssl
          - httpd-tools
          - mariadb-server
          - mariadb
          - php
          - php-mysqlnd
        state: present

...output omitted...
```

The next task uses the `ansible.builtin.service` module to start the `httpd` and `mariadb` services. This task also uses a loop. However, in contrast to the `ansible.builtin.dnf` module, the `ansible.builtin.service` module does not accept a list of services in its `name` parameter. Leave this task as it is.

3.3. In the last task, replace the `ansible.builtin.copy` module with the `ansible.posix.synchronize` module, which is more efficient when deploying large directories:

```
...output omitted...
    - name: Populate web content
      ansible.posix.synchronize:
        src: web_content/
        dest: /var/www/html
```

Save and close the file when done.

4. Run the `clean.yml` playbook to clean up the managed hosts. When done, run the optimized `deploy_webservers.yml` playbook and compare the execution time with its first execution.

4.1. Run the `clean.yml` playbook:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation task-speed]$ ansible-navigator run clean.yml
...output omitted...
```

4.2. Run the `deploy_webservers.yml` playbook. Notice that the fact gathering task is absent and the playbook and task execution times should be reduced:

```
[student@workstation task-speed]$ ansible-navigator run
deploy_webservers.yml
...output omitted...
PLAY RECAP
*********************************************************************
serverb.lab.example.com    : ok=3    changed=3    unreachable=0    failed=0
...
serverc.lab.example.com    : ok=3    changed=3    unreachable=0    failed=0
...

Playbook run took 0 days, 0 hours, 0 minutes, 35 seconds
Wednesday 30 April 2025  13:47:42 +0000 (0:00:01.323)       0:00:35.647
*******
===============================================================================
Install software ------------------------------------------------------
30.08s
Start and enable services ---------------------------------------------
4.23s
Populate web content --------------------------------------------------
1.32s
```

> **Note**
> You can safely ignore the deprecation warning that Ansible generates for the `Populate web content` task.

Finish

On the `workstation` machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish task-speed
```

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

6.9. Lab
Managing Task Execution

Optimize the execution of plays and tasks in a playbook by selectively escalating privileges, configuring when tasks run, optimizing plays and tasks for speed, and adding tags to specify that selected parts of the play should run.

Outcomes

- Change privilege escalation to a more secure configuration.
- Add task hooks and handlers to alter task behavior.
- Tag tasks to control their execution.

Prerequisites

As the `student` user on the `workstation` machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available.

This command initializes the remote Git repository that you need for this lab. When you are ready to push your changes to the remote repository, use `Stud3nt123` as the Git password.

```
[student@workstation ~]$ lab start task-review
```

Instructions

1. Clone the `https://git.lab.example.com/student/task-review.git` repository to the `/home/student/git-repos` directory and then create the `exercise` branch.

2. Modify the `ansible.cfg` file so that privilege escalation is disabled by default.

   Because all the tasks in the `firewall`, `haproxy`, `apache`, and `webapp` roles require privilege escalation, enable privilege escalation at the play level in the `deploy_apache.yml`, `deploy_haproxy.yml`, and `deploy_webapp.yml` playbooks.

3. In the `deploy_haproxy.yml` playbook, add a task that uses the `ansible.builtin.copy` module. That task must run before any other task in the play and must write the text `Playbook site.yml ready to start` to the `/tmp/site.ready` file on the servers in the `lb_servers` group.

4. Add a handler to the `haproxy` role that writes the message `Reloaded` to the `/tmp/haproxy.status` file. Notify the handler if the task that uses the `ansible.builtin.template` module in the `haproxy` role reports that the state of the system changed. Use `Update status` as the name of the handler.

5. Add the `apache_installer` tag to the `Install software` task in the `apache` role.

6. Enable the `ansible.posix.timer` and the `ansible.posix.profile_tasks` callback plug-ins for playbooks in the project.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

7. Run the `site.yml` playbook and analyze the output to find the task that uses the most time.

8. Refactor the task that takes the most time to complete so that it is more efficient. When done, run the `cleanup_apache.yml` playbook to clean up the web servers so that you can compare the results. To verify the changes, run the `site.yml` playbook with an appropriate tag so that you only run the modified task.

9. Commit and push your changes to the remote Git repository.

Evaluation

As the `student` user on the `workstation` machine, use the `lab` command to grade your work. Correct any reported failures and rerun the command until successful.

```
[student@workstation ~]$ lab grade task-review
```

Finish

As the `student` user on the `workstation` machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish task-review
```

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Solution

Optimize the execution of plays and tasks in a playbook by selectively escalating privileges, configuring when tasks run, optimizing plays and tasks for speed, and adding tags to specify that selected parts of the play should run.

Outcomes

- Change privilege escalation to a more secure configuration.
- Add task hooks and handlers to alter task behavior.
- Tag tasks to control their execution.

Prerequisites

As the `student` user on the `workstation` machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available.

This command initializes the remote Git repository that you need for this lab. When you are ready to push your changes to the remote repository, use `Stud3nt123` as the Git password.

```
[student@workstation ~]$ lab start task-review
```

Instructions

1. Clone the `https://git.lab.example.com/student/task-review.git` repository to the `/home/student/git-repos` directory and then create the `exercise` branch.

   1.1. From a terminal, create the `/home/student/git-repos` directory if it does not already exist, and then change into it:

   ```
   [student@workstation ~]$ mkdir -p ~/git-repos/
   [student@workstation ~]$ cd ~/git-repos/
   [student@workstation git-repos]$
   ```

   1.2. Clone the `https://git.lab.example.com/student/task-review.git` repository and then change directory to the cloned repository:

   ```
   [student@workstation git-repos]$ git clone \
   https://git.lab.example.com/student/task-review.git
   Cloning into 'task-review'...
   ...output omitted...
   [student@workstation git-repos]$ cd task-review/
   [student@workstation task-review]$
   ```

   1.3. Create the `exercise` branch and check it out:

```
[student@workstation task-review]$ git checkout -b exercise
Switched to a new branch 'exercise'
```

2. Modify the `ansible.cfg` file so that privilege escalation is disabled by default.

   Because all the tasks in the `firewall`, `haproxy`, `apache`, and `webapp` roles require privilege escalation, enable privilege escalation at the play level in the `deploy_apache.yml`, `deploy_haproxy.yml`, and `deploy_webapp.yml` playbooks.

   2.1. Edit the `ansible.cfg` file to remove the `become=true` entry from the `privilege_escalation` block, use `become=false` in the `privilege_escalation` block, or remove the entire `[privilege_escalation]` block from the `ansible.cfg` file. If you choose the last option, then the file contains the following content:

```
[defaults]
inventory=inventory.yml
remote_user=devops
```

   2.2. Enable privilege escalation for the play in the `deploy_apache.yml` playbook. Add the `become: true` line to the play:

```
---
- name: Ensure Apache is deployed
  hosts: web_servers
  become: true
  force_handlers: true
  gather_facts: false

  roles:
    # The "apache" role has a dependency on the "firewall" role.
    # The "firewall" role requires the "firewall_rules" variable.
    - role: apache
```

   2.3. Enable privilege escalation for the play in the `deploy_haproxy.yml` playbook. Add the `become: true` line to the play:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
---
- name: Ensure HAProxy is deployed
  hosts: lb_servers
  become: true
  force_handlers: true
  gather_facts: false

  roles:
    # The "haproxy" role has a dependency on the "firewall" role.
    # The "firewall" role requires the "firewall_rules" variable.
    - role: haproxy
```

2.4. Enable privilege escalation for the play in the `deploy_webapp.yml` playbook. Add the `become: true` line to the play:

```
---
- name: Ensure Web App is deployed
  hosts: web_servers
  become: true
  gather_facts: false
  vars:
    webapp_version: v1.0

  roles:
    - role: webapp
```

3. In the `deploy_haproxy.yml` playbook, add a task that uses the `ansible.builtin.copy` module. That task must run before any other task in the play and must write the text `Playbook site.yml ready to start` to the `/tmp/site.ready` file on the servers in the `lb_servers` group.

3.1. Edit the `deploy_haproxy.yml` file. Add a `pre_tasks` block to the play, consisting of the following content. The task uses the `ansible.builtin.copy` module to create the `/tmp/site.ready` file containing the specified text.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
---
- name: Ensure HAProxy is deployed
  hosts: lb_servers
  become: true
  force_handlers: true
  gather_facts: false

  pre_tasks:
    - name: Set a status message
      ansible.builtin.copy:
        content: |
          Playbook site.yml ready to start
        dest: /tmp/site.ready
        owner: root
        group: root
        mode: "0644"

  roles:
    # The "haproxy" role has a dependency on the "firewall" role.
    # The "firewall" role requires the "firewall_rules" variable.
    - role: haproxy
```

4. Add a handler to the haproxy role that writes the message Reloaded to the /tmp/haproxy.status file. Notify the handler if the task that uses the ansible.builtin.template module in the haproxy role reports that the state of the system changed. Use Update status as the name of the handler.

   4.1. Edit the roles/haproxy/handlers/main.yml file and add the handler that copies the text to the file. The modified file contains the following content:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
---
# handlers file for haproxy

- name: Restart haproxy
  ansible.builtin.service:
    name: haproxy
    state: restarted

- name: Reload haproxy
  ansible.builtin.service:
    name: haproxy
    state: reloaded

- name: Update status
  ansible.builtin.copy:
    content: |
      Reloaded
    dest: /tmp/haproxy.status
    owner: root
    group: root
    mode: "0644"
```

4.2. Edit the `roles/haproxy/tasks/main.yml` file and modify the `Configure haproxy` task so that it notifies the `Update status` handler if the task reports that it changed the current system. Because the modified task notifies two handlers, the existing `Reload haproxy` handler must be moved to a separate line. The modified file contains the following content:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
---
# tasks file for haproxy

- name: Install software
  ansible.builtin.dnf:
    name:
      - haproxy
      - socat
    state: present

- name: Start and enable the service
  ansible.builtin.service:
    name: haproxy
    state: started
    enabled: true

- name: Configure haproxy
  ansible.builtin.template:
    src: haproxy.cfg.j2
    dest: /etc/haproxy/haproxy.cfg
    owner: root
    group: root
    mode: 0644
  notify:
    - Reload haproxy
    - Update status
```

5. Add the `apache_installer` tag to the `Install software` task in the apache role.

   5.1. Edit the `roles/apache/tasks/main.yml` file and add the `apache_installer` tag to the `Install software` task. The modified file contains the following content:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
---
# tasks file for apache

- name: Install software
  tags: apache_installer
  ansible.builtin.dnf:
    name: "{{ item }}"
    state: present
  loop:
    - httpd
    - php
    - git
    - php-mysqlnd

- name: Allow httpd connections to a remote database
  ansible.posix.seboolean:
    name: httpd_can_network_connect_db
    state: true
    persistent: true

- name: Start and enable the service
  ansible.builtin.service:
    name: httpd
    state: started
    enabled: true
```

6. Enable the `ansible.posix.timer` and the `ansible.posix.profile_tasks` callback plug-ins for playbooks in the project.

   6.1. Edit the `ansible.cfg` configuration file and add the plug-ins to the `callbacks_enabled` directive. The modified file contains the following content:

```
[defaults]
inventory=inventory
remote_user=devops
callbacks_enabled=ansible.posix.timer,ansible.posix.profile_tasks
```

7. Run the `site.yml` playbook and analyze the output to find the task that uses the most time.

   7.1. Run the `site.yml` playbook. Your output looks similar to the following:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation task-review]$ ansible-navigator run site.yml
...output omitted...
PLAY RECAP
*********************************************************************
servera.lab.example.com    : ok=8    changed=8    unreachable=0    failed=0
...
serverb.lab.example.com    : ok=6    changed=6    unreachable=0    failed=0
...
serverc.lab.example.com    : ok=6    changed=6    unreachable=0    failed=0
...

Playbook run took 0 days, 0 hours, 0 minutes, 57 seconds
Wednesday 30 April 2025  19:51:09 +0000 (0:00:01.236)       0:00:57.696
*******
===============================================================================
===
apache : Install software ------------------------------------------------
26.79s
haproxy : Install software -----------------------------------------------
9.52s
apache : Allow httpd connections to a remote database --------------------
3.38s
Set a status message -----------------------------------------------------
2.13s
haproxy : Start and enable the service -----------------------------------
2.11s
haproxy : Reload haproxy -------------------------------------------------
2.08s
...output omitted...
```

7.2. The previous output sorted the tasks based on how long it took for each task to complete. The `Install software` task in the apache role took the most time.

```
apache : Install software ------------------------------------------------
26.79s
```

8. Refactor the task that takes the most time to complete so that it is more efficient. When done, run the `cleanup_apache.yml` playbook to clean up the web servers so that you can compare the results. To verify the changes, run the `site.yml` playbook with an appropriate tag so that you only run the modified task.

8.1. Identify the file containing the `Install software` task for the apache role. The `-R` option performs a recursive search and the `-l` option restricts the output to only show the file name. The output display the path to the `roles/apache/tasks/main.yml` file:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation task-review]$ grep -Rl 'Install software' roles/apache/
roles/apache/tasks/main.yml
```

8.2. Edit the `roles/apache/tasks/main.yml` file to remove the loop from the `Install software` task and instead install all the packages in one transaction. The modified file contains the following content:

```
---
# tasks file for apache

- name: Install software
  tags: apache_installer
  ansible.builtin.dnf:
    name:
      - httpd
      - php
      - git
      - php-mysqlnd
    state: present

- name: Allow httpd connections to a remote database
  ansible.posix.seboolean:
    name: httpd_can_network_connect_db
    state: true
    persistent: true

- name: Start and enable the service
  ansible.builtin.service:
    name: httpd
    state: started
    enabled: true
```

8.3. Run the `cleanup_apache.yml` playbook to clean up the web servers:

```
[student@workstation task-review]$ ansible-navigator run cleanup_apache.yml
...output omitted...
```

8.4. Run the `site.yml` playbook and specify the `apache_installer` tag. Verify that the `Install software` task takes less time to complete:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation task-review]$ ansible-navigator run site.yml \
--tags apache_installer
...output omitted...
PLAY RECAP
*********************************************************************
serverb.lab.example.com    : ok=1    changed=1    unreachable=0    failed=0
...
serverc.lab.example.com    : ok=1    changed=1    unreachable=0    failed=0
...

Playbook run took 0 days, 0 hours, 0 minutes, 13 seconds
Wednesday 30 April 2025  20:04:21 +0000 (0:00:13.582)       0:00:13.618
*******
===============================================================================
apache : Install software ---------------------------------------------
13.58s
```

9. Commit and push your changes to the remote Git repository.

   9.1. Add the changed files, commit the changes, and push them to the Git repository. If prompted, use Stud3nt123 as the Git password:

```
[student@workstation task-review]$ git add .
[student@workstation task-review]$ git commit -m "Lab updates"
...output omitted...
[student@workstation task-review]$ git push -u origin exercise
Password for 'https://student@git.lab.example.com': Stud3nt123
...output omitted...
```

Evaluation

As the student user on the workstation machine, use the lab command to grade your work. Correct any reported failures and rerun the command until successful.

```
[student@workstation ~]$ lab grade task-review
```

Finish

As the student user on the workstation machine, use the lab command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish task-review
```


For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

6.10. Summary

- You can configure privilege escalation at the play, role, block, or task level.
- Privilege escalation uses the `become`, `become_user`, `become_method`, and `become_ask_pass` directives.
- Ansible runs the play sections in the following order: `pre_tasks`, `roles`, `tasks`, and `post_tasks`.
- Handlers run in the same order that they appear in the `handlers` section of the play.
- You can use tags to select or skip tasks during play execution.
- You can speed up playbook execution by disabling fact gathering.
- You can use the `timer`, `profile_tasks`, and `profile_roles` callback plug-ins to profile playbooks.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Chapter 7.
Transforming Data with Filters and Plug-ins

Goal

Populate, manipulate, and manage data in variables using filters and plug-ins.

Sections

- Processing Variables Using Filters (and Guided Exercise)
- Templating External Data Using Lookups (and Guided Exercise)
- Implementing Advanced Loops (and Guided Exercise)
- Using Filters to Work with Network Addresses (and Guided Exercise)

Lab

- Transforming Data with Filters and Plug-ins

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

7.1. Processing Variables Using Filters

Objectives

- Format, parse, and define the values of variables using filters.

Ansible Filters

Ansible applies variable values to playbooks and templates by using Jinja2 expressions. For example, the following Jinja2 expression replaces the name of the variable inside the double braces with its value:

```
{{ variable }}
```

Jinja2 expressions also support filters. Filters are used to modify or process the value from the variable that is placed in the playbook or template. Some filters are provided by the Jinja2 language; others are included with Red Hat Ansible Automation Platform as plug-ins. You can also create custom filters, but that is beyond the scope of this course. Filters can be extremely useful for preparing data for use in your playbook or template.

You can use the `ansible-navigator doc` command to list and provide documentation on most filters:

```
[user@host ~]$ ansible-navigator doc -t filter -l -m stdout
ansible.builtin.b64decode                Decode a base64 string
ansible.builtin.b64encode                Encode a string as base64
ansible.builtin.basename                 get a path's base name
ansible.builtin.bool                     cast into a boolean
...output omitted...

[user@host ~]$ ansible-navigator doc -t filter ansible.builtin.basename \
-m stdout
> ANSIBLE.BUILTIN.BASENAME    (/usr/lib/python3.11/site-packages/ansible/plug...

        Returns the last name component of a path, what is left in the string
        that is not 'dirname'.
...output omitted...
```

To use a filter in a Jinja2 expression, add a pipe (|) character after the variable name or value and then write the filter or filter expression. You can specify multiple filters in a pipeline, separating each filter or filter expression with a pipe character:

```
{{ variable | filter }}
```

Providing Default Variable Values

You can use the `default` filter to ignore an undefined variable or to provide a value to an undefined variable. Both use cases prevent a playbook from generating an error. The following example of an error message indicates that a

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

task uses the `shell` variable, but that the variable is undefined:

```
FAILED! => {"msg": "The task includes an option with an undefined variable.
The error was: 'dict object' has no attribute 'shell'
```

You can use the `default` filter to prevent this error.

The following generic task uses the `ansible.builtin.user` module to manage users defined in the `user_list` variable. Each user in the list must define the `name` key. Additionally, each user in the list can optionally define the `groups`, `system`, `shell`, `state`, and `remove` keys.

Use the `default` filter with each optional key. Consult the module documentation to identify the keys required by the module and the default values for keys.

```
- name: Manage user
  ansible.builtin.user:
    name: "{{ item['name'] }}"
    groups: "{{ item['groups'] | default(omit) }}" (1)
    system: "{{ item['system'] | default(false) }}" (2)
    shell: "{{ item['shell'] | default('/bin/bash') }}" (3)
    state: "{{ item['state'] | default('present') }}" (4)
    remove: "{{ item['remove'] | default(false) }}" (5)
  loop: "{{ user_list }}"
```

(1) If the `groups` key is not defined for the `item` variable, then do not generate an error and do not provide a value for the key.
(2) If the `system` key is not defined for the `item` variable, then use the `false` Boolean value for the key. Pass a value of `true` to create a system user.
(3) If the `shell` key is not defined for the `item` variable, then use the `/bin/bash` value for the key. You might pass a value of `/sbin/nologin` if you create a system user.
(4) If the `state` key is not defined for the `item` variable, then use the `present` value for the key. Pass a value of `absent` to ensure that the user does not exist.
(5) If the `remove` key is not defined for the `item` variable, then use the `false` Boolean value for the key. Passing a value of `true` when removing a user is the equivalent of the `userdel -r` command.

Whenever you use a module, you must decide which module keys to use in the task. You can manually specify values for the keys or you can configure the keys to set their values based on variables passed to the task. Because undefined variables produce errors, you can use the `default` filter to either ignore an undefined variable or supply a value for the undefined variable.

Typically, the `default` filter only provides a value if a variable is not defined. In some situations, you might want to provide a value if the variable passed to the `default` filter is an empty string or evaluates to the `false` Boolean value.

You can do this by adding `true` to the `default` filter. Consider the following playbook:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
---
- name: Default filter examples
  hosts: localhost
  tasks:
    - name: Default filter examples
      vars:
        pattern: "some text"
      ansible.builtin.debug:
        msg: "{{ item }}"
      loop:
        - "{{ pattern | regex_search('test') | default('MESSAGE') }}" (1)
        - "{{ pattern | regex_search('test') | default('MESSAGE', true) }}" (2)
        - "{{ pattern | bool | default('MESSAGE') }}" (3)
        - "{{ pattern | bool | default('MESSAGE', true) }}" (4)
```

(1) Because the regular expression is not found in the variable, the `regex_search` filter returns an empty string. The `default` filter is not used.
(2) Although the `regex_search` filter returns an empty string, the `default` filter is used because it includes `true`.
(3) Because the string evaluates to the `false` Boolean, the `default` filter is not used.
(4) Although the string evaluates to the `false` Boolean, the `default` filter is used because it includes `true`.

Where appropriate, you can use the `default` filter before passing a variable to additional filters.

Variable Types

To understand filters, you need to know how Ansible handles variable values.

Ansible stores runtime data in variables. The YAML structure or the content of the value defines the exact type of data. The following table lists some value types:

Type	Description
Strings	A sequence of characters
Numbers	A numeric value
Booleans	True or false values
Dates	ISO-8601 calendar date
Null	The variable becomes undefined

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Type	Description
Lists or Arrays	A sorted collection of values
Dictionaries	A collection of key-value pairs

Sometimes, you might first need to convert the value to an integer with the `int` filter, or to a float with the `float` filter. For example, the following Jinja2 expression increments the current hour value, which is collected as a fact and stored as a string, not an integer:

```
- name: Date
  ansible.builtin.debug:
    msg: "{{ ( ansible_facts['date_time']['hour'] | int ) + 1 }}"
```

Various filters are available that can perform mathematical operations on numbers, such as `log`, `pow`, `root`, `abs`, and `round`. The `root` filter, for example, takes the square root of the variable or value:

```
- name: Square root
  ansible.builtin.debug:
    msg: "{{ 1764 | root }}"
```

Manipulating Lists

Many filters are available to analyze and manipulate lists.

If the list consists of numbers, you can use the `max`, `min`, or `sum` filters to find the largest number, the smallest number, or the sum of all list items:

```
- name: Sum
  ansible.builtin.debug:
    msg: "{{ [2, 4, 6, 8, 10, 12] | sum }}"
```

Extracting List Elements

You can obtain information about the content of lists, such as the first or last elements, or the length of a list:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
- name: All three of these assertions are true
  ansible.builtin.assert:
    that:
      - "{{ [ 2, 4, 6, 8, 10, 12 ] | length }} is eq( 6 )"
      - "{{ [ 2, 4, 6, 8, 10, 12 ] | first }} is eq( 2 )"
      - "{{ [ 2, 4, 6, 8, 10, 12 ] | last }} is eq( 12 )"
```

The `random` filter returns a random element from the list:

```
- name: Random
  ansible.builtin.debug:
    msg: "{{ ['Douglas', 'Marvin', 'Arthur'] | random }}"
```

Modifying the Order of List Elements

You can use any of the following methods to reorder a list. The `sort` filter returns a list that is sorted by the natural order of its elements. The `reverse` filter returns a list where the order is the opposite of the original order. The `shuffle` filter returns a list with the same elements, but in a random order.

```
- name: reversing and sorting lists
  ansible.builtin.assert:
    that:
      - "{{ [ 2, 4, 6, 8, 10 ] | reverse }} is eq( [ 10, 8, 6, 4, 2] )"
      - "{{ [ 4, 8, 10, 6, 2 ] | sort }} is eq( [ 2, 4, 6, 8, 10 ] )"
```

Merging Lists

Sometimes it is useful to merge several lists into a single list to simplify iteration. The `flatten` filter recursively takes any inner list in the input list value, and adds the inner values to the outer list.

```
- name: Flatten turns nested lists on the left to list on the right
  ansible.builtin.assert:
    that:
      - "{{ [ 2, [4, [6, 8]], 10 ] | flatten }} is eq( [ 2, 4, 6, 8, 10] )"
```

Use the `flatten` filter to merge list values that come from iterating a parent list.

Operating on Lists as Sets

Use the `unique` filter to ensure that a list has no duplicate elements. This filter is useful if you are operating on a list of facts that you have collected, such as usernames or hostnames that might have duplicate entries.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
- name: The 'unique' filter leaves unique elements
  ansible.builtin.assert:
    that:
      - "{{ [ 1, 1, 2, 2, 2, 3, 4, 4 ] | unique }} is eq( [ 1, 2, 3, 4 ] )"
```

If two lists have no duplicate elements, then you can use set theory operations on them:

- The `union` filter returns a set with elements from both input sets.
- The `intersect` filter returns a set with elements common to both sets.
- The `difference` filter returns a set with elements from the first set that are not present in the second set.

```
- name: The 'difference' filter provides elements not in specified set
  ansible.builtin.assert:
    that:
      - "{{ [2, 4, 6, 8, 10] | difference([2, 4, 6, 16]) }} is eq( [8, 10] )"
```

Manipulating Dictionaries

Unlike lists, dictionaries are not ordered in any way, but rather are just a collection of key-value pairs. You can use filters to construct dictionaries and you can convert those dictionaries into lists, and vice versa.

Joining Dictionaries

Use the `combine` filter to join two dictionaries. Entries from the second dictionary have higher priority than entries from the first dictionary, as seen in the following task:

```
- name: The 'combine' filter combines two dictionaries into one
  vars:
    expected:
      A: 1
      B: 4
      C: 5
  ansible.builtin.assert:
    that:
      - "{{ {'A':1,'B':2} | combine({'B':4,'C':5}) }} is eq( expected )"
```

Converting Dictionaries

Use the `dict2items` filter to convert a dictionary to a list. Use the `items2dict` filter to convert a list to a dictionary.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
- name: converting between dictionaries and lists
  vars:
    characters_dict:
      Douglas: Human
      Marvin: Robot
      Arthur: Human
    characters_items:
      - key: Douglas
        value: Human
      - key: Marvin
        value: Robot
      - key: Arthur
        value: Human
  ansible.builtin.assert:
    that:
      - "{{ characters_dict | dict2items }} is eq( characters_items )"
      - "{{ characters_items | items2dict }} is eq( characters_dict )"
```

Hashing, Encoding, and Manipulating Strings

Various filters are available to manipulate the text of a value. You can compute checksums, create password hashes, and convert text to and from Base64 encoding, as used by a number of applications.

Hashing Strings and Passwords

The hash filter returns the hash value of the input string, using the provided hashing algorithm.

```
- name: the string's SHA-1 hash
  vars:
    expected: '8bae3f7d0a461488ced07b3e10ab80d018eb1d8c'
  ansible.builtin.assert:
    that:
      - "'{{ 'Arthur' | hash('sha1') }}' is eq( expected )"
```

Use the password_hash filter to generate password hashes:

```
- name: Password Hash
  ansible.builtin.debug:
    msg: "{{ 'secret_password' | password_hash('sha512') }}"
```

Encoding Strings

Use the b64encode filter to translate binary data to Base64, or to translate Base64 encoded data back to binary data with the b64decode filter:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
- name: Base64 encoding and decoding of values
  ansible.builtin.assert:
    that:
      - "'{{ 'âÉïôú' | b64encode }}' is eq( 'w6LDicOvw7TDug==' )"
      - "'{{ 'w6LDicOvw7TDug==' | b64decode }}' is eq( 'âÉïôú' )"
```

Before sending strings to the underlying shell, and to avoid parsing or code injection issues, it is a good practice to sanitize the string by using the `quote` filter:

```
- name: Put quotes around 'my_string'
  vars:
    my_string: test
  ansible.builtin.shell: echo {{ my_string | quote }}
```

Formatting Text

Use the `lower`, `upper`, or `capitalize` filters to enforce the case of an input string:

```
- name: Change case of characters
  ansible.builtin.assert:
    that:
      - "'{{ 'Marvin' | lower }}' is eq( 'marvin' )"
      - "'{{ 'Marvin' | upper }}' is eq( 'MARVIN' )"
      - "'{{ 'marvin' | capitalize }}' is eq( 'Marvin' )"
```

Replacing Text

Use the `replace` filter to replace all occurrences of a substring inside the input string:

```
- name: Replace 'ar' with asterisks
  ansible.builtin.assert:
    that:
      - "'{{ 'marvin, arthur' | replace('ar','**') }}' is eq( 'm**vin, **thur' )"
```

Use the `regex_search` and `regex_replace` filters to perform more complex searches and replacements by using regular expressions:

```
- name: Test results of regex search and search-and-replace
  ansible.builtin.assert:
    that:
      - "'{{ 'marvin, arthur' | regex_search('ar\\S*r') }}' is eq( 'arthur' )"
      - "'{{ 'arthur up' | regex_replace('ar(\\S*)r','\\1mb') }}' is eq( 'thumb up'
)"
```

Manipulating Data Structures

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Many data structures used by Ansible are in JSON format. JSON and YAML are closely related, and Ansible data structures can be processed as JSON. Likewise, many APIs that Ansible Playbooks might interact with consume or provide information in JSON format. Because this format is widely used, JSON filters are particularly useful.

Data Structure Queries

You can combine `selectattr` with the `map` filter to extract information from Ansible data structures.

Use the `selectattr` filter to select a sequence of objects based on attributes of the objects in the list. Use the `map` filter to turn a list of dictionaries into a simple list based on a given attribute.

> **Note**
> Although the `community.general` collection provides the `json_query` filter, you can usually achieve the same functionality using the `selectattr` and `map` filters. Red Hat does not support the `community.general` collection.

Consider the following playbook, which queries the `/api/v2/execution_environments/` API endpoint and displays the ID for the `Control Plane Execution Environment` automation controller resource:

```
---
- name: Query automation controller EEs
  hosts: localhost
  gather_facts: false
  tasks:
    - name: Query EEs
      vars:
        username_password: "admin:redhat"
      ansible.builtin.uri:
        url: https://aap.lab.example.com/api/controller/v2/execution_environments/
        method: GET
        headers:
          Authorization: Basic {{ username_password | string | b64encode }}
        validate_certs: false
      register: query_results

    - name: Show execution environment ID
      ansible.builtin.debug:
        msg: "{{ query_results['json']['results'] | selectattr('name', '==',
'Control Plane Execution Environment') | map(attribute='id') | first }}"
```

The `query_results['json']['results']` variable is a list containing entries for each execution environment resource. Because each entry defines a `name` key, you can select the entry for the `Control Plane Execution Environment` resource by using the `selectattr` filter. After selecting the correct entry, you can use the `map` filter to display the value of any key in that entry, such as the value of the `id` key. Because the `selectattr` filter combined with the `map` filter often creates a list consisting of one item, you can use the `first` filter to select that item.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

The playbook might produce the following output to indicate that the ID for the `Control Plane Execution Environment` resource is 5:

```
PLAY [Query automation controller EEs] ***************************************

TASK [Query projects] ********************************************************
ok: [localhost]

TASK [Show execution environment ID] *****************************************
ok: [localhost] => {
    "msg": "5"
}

PLAY RECAP *******************************************************************
localhost                  : ok=2    changed=0    unreachable=0    failed=0  ...
```

Similarly, you might list all the execution environment names by sending the `query_results['json']['results']` variable to the `map` filter:

```
    - name: Show execution environment names
      ansible.builtin.debug:
        msg: "{{ query_results['json']['results'] | map(attribute='name') }}"
```

Adding this task to the previous playbook generates this additional output:

```
TASK [Show execution environment names] **************************************
ok: [localhost] => {
    "msg": [
        "Control Plane Execution Environment",
        "Default execution environment",
        "Minimal execution environment",
        "Hub Default execution environment",
        "Hub Minimal execution environment"
    ]
}
```

You can use the `selectattr` and `map` filters with both JSON and YAML data structures.

Parsing and Encoding Data Structures

Transforming data structures to and from text is useful for debugging and communication. Data structures serialize to JSON or YAML format with the `to_json` and `to_yaml` filters. Use `to_nice_json` and `to_nice_yaml` filters to obtain a formatted, human-readable output:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
- name: Convert between JSON and YAML format
  vars:
    hosts:
      - name: bastion
        ip:
          - 172.25.250.254
          - 172.25.252.1
    hosts_json: '[{"name": "bastion", "ip": ["172.25.250.254", "172.25.252.1"]}]'
  ansible.builtin.assert:
    that:
      - "'{{ hosts | to_json }}' is eq( hosts_json )"
```

Other filters are introduced in the following sections and chapters that are suited to specific scenarios. Review the official Ansible and Jinja2 documentation to discover more useful filters to meet your needs.

References

Using Filters to Manipulate Data

Template Designer Documentation: Filters

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

7.2. Guided Exercise
Processing Variables Using Filters

Use filters to reformat the values of variables for use in subsequent tasks.

Outcomes

- Use filters to process and manipulate variables in a playbook or role.

Prerequisites

As the `student` user on the `workstation` machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
[student@workstation ~]$ lab start data-filters
```

Instructions

In this exercise, you deploy an HAProxy load balancer on `servera` to distribute the incoming web requests between `serverb` and `serverc`. On those two back-end servers, you deploy Apache HTTP and some initial content. The playbooks you use to do this use filters to set default values and to process and reuse the data stored in existing variables.

1. Change to the `/home/student/git-repos` directory, clone the project repository, and then create a branch for this exercise. In this exercise, you do not change the `deploy_haproxy.yml` playbook or the `haproxy` role. Run the `deploy_haproxy.yml` playbook to deploy the load balancer.

   1.1. Create the `/home/student/git-repos` directory if it does not exist, and then change into it:

   ```
   [student@workstation ~]$ mkdir -p ~/git-repos
   [student@workstation ~]$ cd ~/git-repos
   [student@workstation git-repos]$
   ```

   1.2. Clone the `https://git.lab.example.com/student/data-filters.git` repository and then change to the cloned repository:

   ```
   [student@workstation git-repos]$ git clone \
   https://git.lab.example.com/student/data-filters.git
   Cloning into 'data-filters'...
   ...output omitted...
   [student@workstation git-repos]$ cd data-filters
   [student@workstation data-filters]$
   ```

   1.3. Create the `exercise` branch and check it out:

```
[student@workstation data-filters]$ git checkout -b exercise
Switched to a new branch 'exercise'
```

1.4. Run the `deploy_haproxy.yml` playbook to deploy the load balancer. Because you have not deployed the web servers, requests to `servera` result in a 503 HTTP status code:

```
[student@workstation data-filters]$ ansible-navigator run deploy_haproxy.yml

PLAY [Ensure HAProxy is deployed]
*****************************************
...output omitted...
PLAY RECAP
*********************************************************************
loadbalancer_01   : ok=6    changed=5    unreachable=0    ...

[student@workstation data-filters]$ curl servera
<html><body><h1>503 Service Unavailable</h1>
...output omitted...
```

2. Carefully review the task list and variables for the apache role.

2.1. Review the `roles/apache/tasks/main.yml` task file for the apache role:

```
...output omitted...
- name: Calculate the package list
  ansible.builtin.set_fact:
    # TODO: Combine the apache_base_packages and
    # apache_optional_packages variables into one list.
    apache_package_list: >-
      {{ apache_base_packages }}

- name: Ensure httpd packages are installed
  ansible.builtin.dnf:
    name: "{{ apache_package_list }}"
    state: present
    #   TODO: omit the 'enablerepo' directive
    #   below if the apache_enablerepos_list is empty;
    #   otherwise use the list as the value for the
    #   'enablerepo' directive.
    #enablerepo: "{{ apache_enablerepos_list }}"
...output omitted...
```

The `apache_package_list` variable currently matches the value of the `apache_base_packages` variable. In a later step, you update the `apache_package_list` variable so that it includes packages from both the `apache_base_packages` variable and the `apache_optional_packages` variable.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

2.2. View the `roles/apache/vars/main.yml` file. Variables defined in this file are not overridden by inventory host variables or host group variables. This file defines the `apache_base_packages` variable as a list that contains one package, `httpd`:

```
apache_base_packages:
  - httpd
```

2.3. View the `roles/apache/defaults/main.yml` file. This file defines the `apache_optional_packages` variable as an empty list:

```
...output omitted...
apache_optional_packages: []
...output omitted...
```

This file also defines the `apache_enabledrepos_list` variable. Although the variable currently defines an empty list, the role temporarily enables any Yum repository ID defined in the list:

```
...output omitted...
apache_enablerepos_list: []
```

3. Update the `roles/apache/tasks/main.yml` file to add a filter to the Jinja2 expression that defines the `apache_package_list` variable.

Define the `apache_optional_packages` variable for the `web_servers` host group to contain the following values: `git`, `php`, and `php-mysqlnd`.

3.1. Edit the Jinja2 expression in the first task of the `apache` role that defines the `apache_package_list` variable. Add the `union` filter to create a single list from the lists stored in the `apache_base_packages` and `apache_optional_packages` variables.

When you finish, remove the comments from this task. The first task in the `roles/apache/tasks/main.yml` file should consist of the following content:

```
- name: Calculate the package list
  ansible.builtin.set_fact:
    apache_package_list: >-
      {{ apache_base_packages | union(apache_optional_packages) }}
```

Save the file.

3.2. Define the `apache_optional_packages` variable in a new file named `group_vars/web_servers/apache.yml`. The variable defines three packages: `git`, `php`, and `php-mysqlnd`. This overrides the role's default value for this variable.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

The `group_vars/web_servers/apache.yml` file should consist of the following content:

```
---
apache_optional_packages:
  - git
  - php
  - php-mysqlnd
```

Save the file.

4. Remove the comments from the `enablerepo` directive in the second task from the `apache` role. Edit the directive's Jinja2 expression to use the `default` filter to omit this directive if the variable evaluates to a Boolean value of `false`. Remove the TODO comment section for the second task, and save the task file. The second task in the `roles/apache/tasks/main.yml` file should consist of the following content:

```
- name: Ensure httpd packages are installed
  ansible.builtin.dnf:
    name: "{{ apache_package_list }}"
    state: present
    enablerepo: "{{ apache_enablerepos_list | default(omit, true) }}"
```

The completed `roles/apache/tasks/main.yml` file should consist of the following content:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
---
# tasks file for apache

- name: Calculate the package list
  ansible.builtin.set_fact:
    apache_package_list: >-
      {{ apache_base_packages | union(apache_optional_packages) }}

- name: Ensure httpd packages are installed
  ansible.builtin.dnf:
    name: "{{ apache_package_list }}"
    state: present
    enablerepo: "{{ apache_enablerepos_list | default(omit, true) }}"

- name: Ensure SELinux allows httpd connections to a remote database
  ansible.posix.seboolean:
    name: httpd_can_network_connect_db
    state: true
    persistent: true

- name: Ensure httpd service is started and enabled
  ansible.builtin.service:
    name: httpd
    state: started
    enabled: true
```

5. Run the `deploy_apache.yml` playbook and add the `-v` option. Verify that the optional packages are included in the `apache_package_list` fact:

```
[student@workstation data-filters]$ ansible-navigator run deploy_apache.yml -v
...output omitted...
TASK [apache : Calculate the package list] *********************************
ok: [webserver_01] => {"ansible_facts": {"apache_package_list": ["php", "httpd",
"git", "php-mysqlnd"]}, "changed": false}
...output omitted...
```

6. Review the task list and variable definitions for the `webapp` role. The `webapp` role ensures that the correct web application content exists on each host. When correctly implemented, the role removes any content from the root web directory that does not belong to the web application.

   Edit the three tasks in the `webapp` role that have a `TODO` comment. Use filters to implement the functionality indicated in each comment.

   6.1. Review the tasks in the `roles/webapp/tasks/main.yml` file:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
---
# tasks file for webapp

- name: Ensure stub web content is deployed
  ansible.builtin.copy:
    content: >
      {{ webapp_message }} (version {{ webapp_version }})
    dest: "{{ webapp_content_root_dir }}/index.html"
    owner: root
    group: root
    mode: "0644"

- name: Find deployed webapp files
  ansible.builtin.find:
    paths: "{{ webapp_content_root_dir }}"
    recurse: true
  register: webapp_find_files

- name: Compute webapp file list
  ansible.builtin.set_fact:
    # TODO: Use the map filter to extract
    # the 'path' attribute of each entry
    # in the 'webapp_find_files'
    # variable 'files' list.
    webapp_deployed_files: []

- name: Compute relative webapp file list
  ansible.builtin.set_fact:
    # TODO: Use the 'map' filter, along with
    # the 'relpath' filter, to create the
    # 'webapp_rel_deployed_files' variable
    # from the 'webapp_deployed_files' variable.
    #
    # Files in the 'webapp_rel_deployed_files'
    # variable should have a path relative to
    # the 'webapp_content_root_dir' variable.
    webapp_rel_deployed_files: []

- name: Remove Extraneous Files
  ansible.builtin.file:
    path: "{{ webapp_content_root_dir }}/{{ item }}"
    state: absent
  # TODO: Loop over a list of files
  # that are in the 'webapp_rel_deployed_files'
  # list, but not in the 'webapp_file_list' list.
```

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
  # Use the difference filter.
  loop: []
```

The `webapp_file_list` variable is defined in the `roles/webapp/vars/main.yml` file. This variable defines a file manifest for the web application. For this version of the web application, the only file in the application is `index.html`.

The `webapp_content_root_dir` variable defines the directory location of web application content on each web server. The default value is `/var/www/html`.

6.2. Replace the empty list for the `webapp_deployed_files` variable in the third task of the `roles/webapp/tasks/main.yml` file with a Jinja2 expression. Start with the `webapp_find_files['files']` variable and apply the `map` filter, followed by the `list` filter. Provide the `map` filter with an argument of `attribute='path'` to retrieve the `path` attribute from each entry in the list.

After you remove the TODO comments, the third task should consist of the following content:

```
- name: Compute the webapp file list
  ansible.builtin.set_fact:
    webapp_deployed_files: >-
      {{ webapp_find_files['files'] |
      map(attribute='path') |
      list }}
```

> **Note**
> You might define the `webapp_deployed_files` variable definition as a single line of text. This example uses `>-` to break the definition of the variable over multiple lines.

6.3. Replace the empty list for the `webapp_rel_deployed_files` variable in the fourth task of the `roles/webapp/tasks/main.yml` file with a Jinja2 expression. Start with the `webapp_deployed_files` variable and apply the `map` filter, followed by the `list` filter.

The first argument to the `map` function is the `relpath` string, which executes the `relpath` function on each item of the `webapp_deployed_files` list. The second argument to the `map` function is the `webapp_content_root_dir` variable. This variable is passed as an argument to the `relpath` function.

After you remove the TODO comments, the fourth task should consist of the following content:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
- name: Compute the relative webapp file list
  ansible.builtin.set_fact:
    webapp_rel_deployed_files: >-
      {{ webapp_deployed_files |
      map('relpath', webapp_content_root_dir) |
      list }}
```

6.4. Replace the empty loop list in the fifth task in the `roles/webapp/tasks/main.yml` file with a Jinja2 expression. Start with the `webapp_rel_deployed_files` variable and apply the `difference` filter. Provide the `webapp_file_list` variable as an argument to the `difference` filter.

After you remove the TODO comments, the fifth task should consist of the following content:

```
- name: Remove Extraneous Files
  ansible.builtin.file:
    path: "{{ webapp_content_root_dir }}/{{ item }}"
    state: absent
  loop: "{{ webapp_rel_deployed_files | difference(webapp_file_list) }}"
```

Save the changes to the `roles/webapp/tasks/main.yml` file.

The completed `roles/webapp/tasks/main.yml` file should consist of the following content:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
---
# tasks file for webapp

- name: Ensure stub web content is deployed
  ansible.builtin.copy:
    content: >
      {{ webapp_message }} (version {{ webapp_version }})
    dest: "{{ webapp_content_root_dir }}/index.html"
    owner: root
    group: root
    mode: "0644"

- name: Find deployed webapp files
  ansible.builtin.find:
    paths: "{{ webapp_content_root_dir }}"
    recurse: true
  register: webapp_find_files

- name: Compute the webapp file list
  ansible.builtin.set_fact:
    webapp_deployed_files: >-
      {{ webapp_find_files['files'] |
      map(attribute='path') |
      list }}

- name: Compute the relative webapp file list
  ansible.builtin.set_fact:
    webapp_rel_deployed_files: >-
      {{ webapp_deployed_files |
      map('relpath', webapp_content_root_dir) |
      list }}

- name: Remove Extraneous Files
  ansible.builtin.file:
    path: "{{ webapp_content_root_dir }}/{{ item }}"
    state: absent
  loop: "{{ webapp_rel_deployed_files | difference(webapp_file_list) }}"
```

7. Run the `deploy_webapp.yml` playbook with the `-v` option. Verify that the playbook identifies a non application file on `webserver_01` and removes it:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation data-filters]$ ansible-navigator run deploy_webapp.yml -v
...output omitted...
TASK [webapp : Compute the webapp file list] ********************************
ok: [webserver_01] => {"ansible_facts": {"webapp_deployed_files":
[“/var/www/html/test.html”, "/var/www/html/index.html"]}, "changed": false}
ok: [webserver_02] => {"ansible_facts": {"webapp_deployed_files":
["/var/www/html/index.html"]}, "changed": false}

TASK [webapp : Compute the relative webapp file list] ***********************
ok: [webserver_01] => {"ansible_facts": {"webapp_rel_deployed_files":
[“test.html”, "index.html"]}, "changed": false}
ok: [webserver_02] => {"ansible_facts": {"webapp_rel_deployed_files":
["index.html"]}, "changed": false}

TASK [webapp : Remove Extraneous Files] *************************************
changed: [webserver_01] => (item=test.html) => {"ansible_loop_var": "item",
"changed": true, "item": "test.html", "path": "/var/www/html/test.html",
"state": "absent"}
skipping: [webserver_02] => {"changed": false, "skipped_reason": "No items in
the list"}

PLAY RECAP ******************************************************************
webserver_01   : ok=5  changed=2  unreachable=0  failed=0    skipped=0  ...
webserver_02   : ok=4  changed=1  unreachable=0  failed=0    skipped=1  ...
```

The `webserver_01` host initially has two files deployed in the `/var/www/html` directory: `test.html` and `index.html`. Someone with access to the host might have installed a temporary test page on the web server.

The playbook removes any web server files from the web content root directory that are not part of the actual web application.

8. Verify that the load balancer is now servicing requests correctly:

```
[student@workstation data-filters]$ curl servera; curl servera
Hello from webserver_01. (version v1.1)
Hello from webserver_02. (version v1.1)
```

Finish

On the `workstation` machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish data-filters
```

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

7.3. Templating External Data Using Lookups

Objectives

- Populate variables with data from external sources using lookup plug-ins.

Lookup Plug-ins

A lookup plug-in is an Ansible extension to the Jinja2 templating language. These plug-ins enable Ansible to use data from external sources, such as files and the shell environment.

Calling Lookup Plug-ins

You can call lookup plug-ins with one of two Jinja2 template functions: `lookup` or `query`. Both methods have a syntax that is similar to filters. Specify the name of the function, and in parentheses the name of the lookup plug-in that you want to call and any arguments that the plug-in needs.

For example, the following variable definition uses the `ansible.builtin.file` lookup plug-in to put the contents of the `/etc/hosts` file into the `file_content` Ansible variable:

```
vars:
  file_content: >-
    {{ lookup('ansible.builtin.file', '/etc/hosts') }}
```

You can include more than one file name to the `ansible.builtin.file` plug-in. When called with the `lookup` function, the contents of each file are separated by a comma in the resulting value.

Consider the following Jinja2 template expression:

```
vars:
  file_content: >-
    {{ lookup('ansible.builtin.file', '/etc/hosts', '/etc/issue') }}
```

This expression results in the following structure (line-wrapped):

```
file_content: "127.0.0.1\tlocalhost localhost.localdomain localhost4
localhost4.localdomain4\n::1\tlocalhost localhost.localdomain localhost6
localhost6.localdomain6`,`\\S\nKernel \\r on an \\m"
```

In Ansible 2.5 and later, you can use the `query` function, or its abbreviation q, instead of `lookup` to call lookup plug-ins. The difference between the two is that instead of the returned values being comma-separated, `query` always returns a list, which can be easier to parse and work with.

You can use the following `query` function to replace the preceding example:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
vars:
  file_content: >-
    {{ query('ansible.builtin.file', '/etc/hosts', '/etc/issue') }}
```

The query call returns the following data structure (line-wrapped):

```
file_content:
  - "127.0.0.1\tlocalhost localhost.localdomain localhost4
localhost4.localdomain4\n::1\tlocalhost localhost.localdomain localhost6
localhost6.localdomain6",
  - "\\S\nKernel \\r on an \\m"
```

Selecting Lookup Plug-ins

The default Ansible installation provides many plug-ins. Use the `ansible-navigator doc --mode stdout -t lookup -l` command to obtain the complete list of installed lookup plug-ins. For details about the purpose of specific plug-ins and how to use them, run the `ansible-navigator doc --mode stdout -t lookup` *`PLUGIN_NAME`* command.

Reading the Contents of Files

Ansible uses the `ansible.builtin.file` plug-in to load the contents of a local file into a variable. If you provide a relative path, the plug-in looks for files in the playbook's `files` directory.

The following example reads the contents of the user's public key file and uses the `ansible.posix.authorized_key` module to add the authorized key to the managed host. The example uses a loop and the ~ operator to join strings in the template to look up the `files/fred.key.pub` and `files/naoko.key.pub` files:

```
- name: Add authorized keys
  hosts: all
  vars:
    users:
      - fred
      - naoko
  tasks:
    - name: Add authorized keys
      ansible.posix.authorized_key:
        user: "{{ item }}"
        key: "{{ lookup('ansible.builtin.file', item ~ '.key.pub') }}"
      loop: "{{ users }}"
```

> **Note**
> In the preceding example, the public key files are included in the Ansible project's `files` subdirectory, so that they are available inside the automation execution environment.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

The `ansible.builtin.file` plug-in can parse YAML and JSON files into properly structured data by using the `from_yaml` or `from_json` filters:

```
my_yaml: >-
  {{ lookup('ansible.builtin.file', 'path/to/my.yaml') | from_yaml }}
```

Important

The `ansible.builtin.file` plug-in reads files that are in the *automation execution environment*, not on the control node or managed host.

Automation execution environments read files that are *inside the automation execution environment container*, not those on the host on which you ran the `ansible-navigator` command.

If you use the `ansible-playbook` command, or the `ansible-navigator` command with the `--ee false` option, then the automation execution environment is the same as the control node.

Note

If you need the content of a file that is not in the automation execution environment or your project directory, then you can use the `ansible.builtin.slurp` module instead of a lookup plug-in.

The following example displays the content of the `/home/user/.ssh/id_rsa.pub` file on the managed host:

```
- name: Collect file content
  ansible.builtin.slurp:
    src: /home/user/.ssh/id_rsa.pub
  register: slurped_file

- name: Print returned information
  ansible.builtin.debug:
    var: slurped_file['content'] | b64decode
```

Applying Data with a Template

Like the `ansible.builtin.file` plug-in, the `ansible.builtin.template` plug-in returns the contents of files. The difference is that the `ansible.builtin.template` plug-in expects the file contents to be a Jinja2 template, and it evaluates that template before applying the contents. If you pass a relative path to the template file, the plug-in looks for it in the `templates` directory.

The following example processes the `templates/my.template.j2` template:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
{{ lookup('ansible.builtin.template', 'my.template.j2') }}
```

For example, assume that the template contains the following content:

```
Hello {{ name }}.
```

The following task displays the string "Hello class":

```
- name: Print "Hello class"
  vars:
    name: class
  ansible.builtin.debug:
    msg: "{{ lookup('ansible.builtin.template', 'my.template.j2') }}"
```

The `ansible.builtin.template` plug-in also accepts some extra parameters, such as defining the start and end marking sequences. In case the output string is a YAML value, the `convert_data` option parses the string to provide structured data.

> **Important**
> Do not confuse the `ansible.builtin.template` lookup plug-in with the `ansible.builtin.template` module.

Reading Command Output in the Execution Environment

The `ansible.builtin.pipe` and `ansible.builtin.lines` plug-ins both run a command in the automation execution environment and return the output. The `ansible.builtin.pipe` plug-in returns the raw output generated by the command; the `ansible.builtin.lines` plug-in splits the output of that command into lines.

For example, consider the following Jinja2 expression:

```
{{ query('ansible.builtin.pipe', 'ls files') }}
```

This expression returns the raw output of the `ls` command as a string. If you use the `ansible.builtin.lines` plug-in, then the expression is as follows:

```
{{ query('ansible.builtin.lines', 'ls files') }}
```

This command results in a list with each line of output returned by `ls` as a list item.

You can use this functionality to retrieve the first line (or any specific line) of output from a set of commands:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
- name: Prints the first line of some files
  ansible.builtin.debug:
    msg: "{{ item[0] }}"
  loop:
    - "{{ query('ansible.builtin.lines', 'cat files/my.file') }}"
    - "{{ query('ansible.builtin.lines', 'cat files/my.file.2') }}"
```

Note that this example might not be the most efficient way to do this particular task, given the existence of the head command.

Getting Content from a URL

Similar to the way the `ansible.builtin.file` plug-in gets content from a file, the `ansible.builtin.url` plug-in gets content from a URL:

```
{{ lookup('ansible.builtin.url', 'https://my.site.com/my.file') }}
```

Many options are available for controlling authentication, selecting web proxies, or splitting the returned content into lines.

However, one advantage of using the `ansible.builtin.url` plug-in is that you can use the returned data as values in your variables, possibly processing it first with a filter.

Getting Information from the Kubernetes API

The `kubernetes.core.k8s` plug-in provides full access to the Kubernetes API through the `openshift` Python module. To fetch a Kubernetes object, you must provide the object type by using the `kind` option. Providing additional object details, such as the `namespace` or `label_selector` option, helps to filter the results:

```
{{ lookup('kubernetes.core.k8s', kind='Deployment', namespace='ns',
resource_name='my_res') }}
```

> **Note**
> Be aware that the `kubernetes.core.k8s` plug-in is a *lookup* plug-in. Its primary purpose is to extract information from the Kubernetes cluster, not to update it. Use the `kubernetes.core.k8s` *module* to manage a Kubernetes cluster.

Using Custom Lookup Plug-ins

Plug-ins are Python scripts. You can develop custom plug-ins and use them in your playbooks. For Ansible to find your lookup plug-ins, copy them into one of the following locations:

- In a custom Ansible Content Collection, use the `./plugins/lookup` directory.
- In a custom role, use the `./filter_plugins` directory.
- In an Ansible project, use the `./lookup_plugins` directory in the same directory as the playbooks.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

The Ansible team recommends using collections to distribute custom plug-ins. Plug-in development is beyond the scope of this course.

Handling Lookup Errors

Most Ansible plug-ins are designed to abort the Ansible Playbook in the event of a failure. However, the `lookup` function delegates execution to other plug-ins that might not need to abort the Ansible Playbook. For example, the `ansible.builtin.file` plug-in might not need to abort the Ansible Playbook if the file is not found, but playbook execution might need to continue after recovery.

To adapt to different plug-in needs, the `ansible.builtin.lookup` plug-in accepts the `errors` option:

```
{{ lookup('ansible.builtin.file', 'my.file', errors='warn') }}
```

The default value for the `errors` option is `strict`. This means that the `ansible.builtin.lookup` plug-in raises a fatal error if the underlying script fails. If the `errors` option has the value `warn`, then the `ansible.builtin.lookup` plug-in logs a warning when the underlying script fails and returns an empty string (or an empty list). If the `errors` option has the value `ignore`, then the `ansible.builtin.lookup` plug-in silently ignores the error and returns an empty string or list.

References

Collection Index - Ansible.Builtin - Lookup Plugins

Using Ansible - Lookup plugins

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

7.4. Guided Exercise
Templating External Data Using Lookups

Use the `lookup` and `query` functions to template data from external sources into playbooks and deployed template files.

Outcomes

- Identify different sources of information in Ansible Playbooks.
- Use existing lookup plug-ins to retrieve information.

Prerequisites

As the `student` user on the `workstation` machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
[student@workstation ~]$ lab start data-lookups
```

Instructions

1. Clone the `https://git.lab.example.com/student/data-lookups.git` repository into the `/home/student/git-repos` directory and then create a branch for this exercise.

   1.1. From a terminal, create the `/home/student/git-repos` directory if it does not already exist, and then change into it:

   ```
   [student@workstation ~]$ mkdir -p ~/git-repos/
   [student@workstation ~]$ cd ~/git-repos/
   [student@workstation git-repos]$
   ```

   1.2. Clone the `https://git.lab.example.com/student/data-lookups.git` repository and then change directory to the cloned repository:

   ```
   [student@workstation git-repos]$ git clone \
   https://git.lab.example.com/student/data-lookups.git
   Cloning into 'data-lookups'...
   ...output omitted...
   [student@workstation git-repos]$ cd data-lookups
   [student@workstation data-lookups]$
   ```

   1.3. Create the `exercise` branch:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation data-lookups]$ git checkout -b exercise
Switched to a new branch 'exercise'
```

2. Populate the `site.yml` file with tasks to create groups using YAML-formatted data from the `groups.yml` file.

   2.1. Review the `groups.yml` file:

```
---
- name: devs
  members:
    - user1
    - user2
- name: ops
  members:
    - user3
```

   2.2. You can read the contents of a YAML-formatted file in several ways. In the `Load group information` task in the `site.yml` playbook, you can use the `ansible.builtin.file` lookup plug-in to read the file, and the `from_yaml` filter to load its data into the `user_groups` fact as structured YAML so that Ansible can parse it later:

```
{{ lookup('ansible.builtin.file', 'groups.yml') | from_yaml }}
```

   The completed task should contain the following content:

```
    - name: Load group information
      ansible.builtin.set_fact:
        user_groups: >-
          {{ lookup('ansible.builtin.file', 'groups.yml') | from_yaml }}
```

   2.3. Complete the `Create groups` task to create each group specified by the YAML data in the `user_groups` fact. The task must use the `ansible.builtin.group` module, looping through the `user_groups` fact. Use `item['name']` to specify the group name of the group to create for each iteration of the loop. The completed task should consist of the following content:

```
    - name: Create groups
      ansible.builtin.group:
        name: "{{ item['name'] }}"
        state: present
      loop: "{{ user_groups }}"
```

3. Complete the `Create users` task.

   3.1. Use the `ansible.builtin.lines` lookup plug-in to read user information from the `users.txt` file.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
"{{ query('ansible.builtin.lines', 'cat users.txt') }}"
```

The `ansible.builtin.lines` plug-in reads the output of `cat users.txt` as individual lines. The `ansible.builtin.query` function ensures that the data is a list.

Configure the task to loop through the users in this list.

Use the `loop` keyword to configure the task to loop through the users in this list:

```
    - name: Create users
      ansible.builtin.debug:
        msg: "To be done"
      loop: "{{ query('ansible.builtin.lines', 'cat users.txt') }}"
```

3.2. When you create the users, create a random password as well. The `ansible.builtin.password` lookup plug-in generates random passwords and optionally stores the passwords in a local file.

The previous step added a loop on the list of usernames from `users.txt`; you can use the same `item` variable to generate the associated file for each user.

```
{{ lookup('ansible.builtin.password', 'credentials/' ~ item ~ ' length=9')
}}
```

Store the password into a task variable to make it easier to use. Ensure that you include the space after the first single quote in the `' length=9'` argument. The task should now consist of the following content:

```
    - name: Create users
      vars:
        password_plain: >-
          {{ lookup('ansible.builtin.password',
          'credentials/' ~ item ~ ' length=9') }}
      ansible.builtin.debug:
        msg: "To be done"
      loop: "{{ query('ansible.builtin.lines', 'cat users.txt') }}"
```

3.3. Replace the `ansible.builtin.debug` module with the `ansible.builtin.user` module.

You need to hash the new password before you can use it. You also need to set `update_password: on_create` for the module.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

> **Important**
>
> The `update_password: on_create` option only sets the password if the task creates a user on the managed host. If the user already exists, but the password specified by the playbook is different from the one already set for the user on the system, then Ansible does **not** change the user's password.
>
> This helps ensure that Ansible does not change an existing user's password if that user has already changed it, but you run the playbook a second time.

```
- name: Create users
  vars:
    password_plain: >-
      {{ lookup('ansible.builtin.password',
      'credentials/' ~ item ~ ' length=9') }}
  ansible.builtin.user:
    name: "{{ item }}"
    password: "{{ password_plain | password_hash('sha512') }}"
    update_password: on_create
    state: present
  loop: "{{ query('ansible.builtin.lines', 'cat users.txt') }}"
```

3.4. The last task uses the `subelements` filter to populate the members of each group. However, no lookup plug-ins are used. Confirm that the completed play consists of the following content:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
---
- name: Populate users and groups
  hosts: all
  become: true
  gather_facts: false
  tasks:

    - name: Load group information
      ansible.builtin.set_fact:
        user_groups: >-
          {{ lookup('ansible.builtin.file', 'groups.yml') | from_yaml }}

    - name: Create groups
      ansible.builtin.group:
        name: "{{ item['name'] }}"
        state: present
      loop: "{{ user_groups }}"

    - name: Create users
      vars:
        password_plain: >-
          {{ lookup('ansible.builtin.password',
          'credentials/' ~ item ~ ' length=9') }}
      ansible.builtin.user:
        name: "{{ item }}"
        password: "{{ password_plain | password_hash('sha512') }}"
        update_password: on_create
        state: present
      loop: "{{ query('ansible.builtin.lines', 'cat users.txt') }}"

    - name: Add users to groups
      ansible.builtin.user:
        name: '{{ item[1] }}'
        groups: "{{ item[0]['name'] }}"
        append: true
      loop: "{{ user_groups | subelements('members', 'skip_missing=true')
}}"
```

4. Run the playbook and verify that the users, groups, and group members are configured correctly.

   4.1. Use the `ansible-navigator run` command to run the playbook:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation data-lookups]$ ansible-navigator run site.yml

PLAY [Populate users and groups]
*********************************************

TASK [Load group information]
***************************************************
ok: [serverf.lab.example.com]

TASK [Create groups]
*************************************************************
changed: [serverf.lab.example.com] => (item={'name': 'devs', 'members':
['user1', 'user2']})
changed: [serverf.lab.example.com] => (item={'name': 'ops', 'members':
['user3']})

TASK [Create users]
*************************************************************
[DEPRECATION WARNING]: Encryption using the Python crypt module is
deprecated. The Python crypt module is deprecated and will be removed from
Python 3.13. Install the passlib library for continued encryption
functionality. This feature will be removed in version 2.17. Deprecation
warnings can be disabled by setting deprecation_warnings=False in
ansible.cfg.
changed: [serverf.lab.example.com] => (item=user1)
changed: [serverf.lab.example.com] => (item=user2)
changed: [serverf.lab.example.com] => (item=user3)

TASK [Add users to groups]
***************************************************
changed: [serverf.lab.example.com] => (item=[{'name': 'devs', 'members':
['user1', 'user2']}, 'user1'])
changed: [serverf.lab.example.com] => (item=[{'name': 'devs', 'members':
['user1', 'user2']}, 'user2'])
changed: [serverf.lab.example.com] => (item=[{'name': 'ops', 'members':
['user3']}, 'user3'])

PLAY RECAP
*********************************************************************
serverf.lab.example.com    : ok=4    changed=3    unreachable=0    failed=0
...
```

Note

You can safely ignore the deprecation warning that Ansible generates for the `Create users` task.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

4.2. Verify that the users have been created on the `serverf` server. The user and group ID numbers in your environment might be different from the ID numbers displayed in the following output:

```
[student@workstation data-lookups]$ ssh serverf "tail -n3 /etc/passwd"
user1:x:1002:1004::/home/user1:/bin/bash
user2:x:1003:1005::/home/user2:/bin/bash
user3:x:1004:1006::/home/user3:/bin/bash
```

4.3. Verify that the groups exist and have the relevant members. The group ID numbers in your environment might be different from the ID numbers displayed in the following output:

```
[student@workstation data-lookups]$ ssh serverf "tail -n5 /etc/group"
devs:x:1002:user1,user2
ops:x:1003:user3
user1:x:1004:
user2:x:1005:
user3:x:1006:
```

Finish

On the `workstation` machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish data-lookups
```

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

7.5. Implementing Advanced Loops

Objectives

- Implement loops using structures other than simple lists using lookup plug-ins and filters.

Comparing Loops and Lookup Plug-ins

Using loops to iterate over tasks can help simplify your Ansible Playbooks. The `loop` keyword loops over a flat list of items. When used with lookup plug-ins, you can construct more complex data in your lists for your loops.

The `loop` keyword was introduced in Ansible 2.5. Before that, task iteration was implemented by using keywords that started with `with_` and ended with the name of a lookup plug-in. The equivalent to `loop` in this syntax is `with_list`, and is designed for iteration over a simple flat list. For simple lists, `loop` is the best syntax to use.

For example, the following three syntaxes have the same results. The first of these is preferred:

```
- name: Using loop
  ansible.builtin.debug:
    msg: "{{ item }}"
  loop: "{{ mylist }}"

- name: Using with_list
  ansible.builtin.debug:
    msg: "{{ item }}"
  with_list: "{{ mylist }}"

- name: Using lookup plugin
  ansible.builtin.debug:
    msg: "{{ item }}"
  loop: "{{ lookup('ansible.builtin.list', mylist) }}"
```

You can refactor a `with_*` iteration task to use the `loop` keyword by using an appropriate combination of lookup plug-ins and filters to match the functionality.

Using the `loop` keyword in place of `with_*` loops has the following benefits:

- No need to memorize or find a `with_*` style keyword to suit your iteration scenario. Instead, use plug-ins and filters to adapt a `loop` keyword task to your use case.
- Focus on learning the plug-ins and filters that are available in Ansible, which have broader applicability beyond iteration.
- Provides command-line access to the lookup plug-in documentation, with the `ansible-navigator doc --mode stdout -t lookup -l` command. This documentation helps you to discover lookup plug-ins and to design and use custom iteration scenarios.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Important

Guidance from upstream Ansible on when to use `loop` or `with_` loops has been evolving since Ansible 2.5. Red Hat recommends that you use the `loop` keyword instead of the `with_*` loops, but some use cases exist where the earlier syntax might be better. According to the documentation, the `with_*` syntax is not deprecated, and it should be valid for the foreseeable future. The syntax of `loop` might continue to evolve in future releases of Ansible.

The upstream guidance for Ansible 6 includes the following tips:

- The `loop` keyword requires a list, and does not accept a string. If you are having problems, then remember the difference between `lookup` and `query`.
- Any use of `with_*` that is discussed in `"Migrating from with_X to loop"` can be safely converted. These mostly use filters.
- If you need to use a lookup plug-in to convert a `with_*` construct to use `loop`, then it might be clearer to keep using the `with_*` syntax.

Example Iteration Scenarios

The following examples show some ways to construct more complex loops using Jinja2 expressions, filters, lookup plug-ins, and the `with_*` syntax.

Iterating over a List of Lists

The `with_items` keyword provides a way to iterate over complex lists. For example, consider a hypothetical play with the following task:

```
- name: Remove build files
  ansible.builtin.file:
    path: "{{ item }}"
    state: absent
  with_items:
    - "{{ app_a_tmp_files }}"
    - "{{ app_b_tmp_files }}"
    - "{{ app_c_tmp_files }}"
```

The `app_a_tmp_files`, `app_b_tmp_files`, and `app_c_tmp_files` variables each contain a list of temporary files. The `with_items` keyword combines these three lists into a single list containing the entries from all three lists. It automatically performs one-level flattening of its list.

To refactor a `with_items` task to use the `loop` keyword, use the `ansible.builtin.flatten` filter. This filter recursively searches for embedded lists, and creates a single list from discovered values.

The `ansible.builtin.flatten` filter accepts a `levels` argument, which specifies an integer number of levels to search for embedded lists. A `levels=1` argument specifies that values are obtained by only descending into one

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

additional list for each item in the initial list. This is the same one-level flattening that `with_items` does implicitly.

To refactor a `with_items` task to use the `loop` keyword, you must also use the `ansible.builtin.flatten(levels=1)` filter:

```
- name: Remove build files
  ansible.builtin.file:
    path: "{{ item }}"
    state: absent
  loop: "{{ list_of_lists | ansible.builtin.flatten(levels=1) }}"
  vars:
    list_of_lists:
      - "{{ app_a_tmp_files }}"
      - "{{ app_b_tmp_files }}"
      - "{{ app_c_tmp_files }}"
```

Important

Because the `loop` keyword does not perform this implicit one-level flattening, it is not exactly equivalent to `with_items`. However, provided that the list passed to the loop is a simple list, both methods behave identically. The distinction only matters if you have nested lists.

Iterating over Nested Lists

Data from variable files, Ansible facts, and external services are often a composition of simpler data structures, such as lists and dictionaries. Consider the following `users` variable:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
users:
  - name: paul
    password: "{{ paul_pass }}"
    authorized:
      - keys/paul_key1.pub
      - keys/paul_key2.pub
    mysql:
      hosts:
        - "%"
        - "127.0.0.1"
        - "::1"
        - "localhost"
   groups:
      - wheel

  - name: john
    password: "{{ john_pass }}"
    authorized:
      - keys/john_key.pub
    mysql:
      password: other-mysql-password
      hosts:
         - "utility"
    groups:
      - wheel
      - devops
```

The `users` variable is a list. Each entry in the list is a dictionary with the `name`, `password`, `authorized`, `mysql`, and `groups` keys. The `name` and `password` keys define simple strings, and the `authorized` and `groups` keys define lists. The `mysql` key references another dictionary that contains MySQL-related metadata for each user.

Similar to the `ansible.builtin.flatten` filter, the `ansible.builtin.subelements` filter creates a single list from a list with nested lists. The filter processes a list of dictionaries, and each dictionary contains a key that refers to a list. To use the `ansible.builtin.subelements` filter, you must provide the name of a key on each dictionary that corresponds to a list.

To illustrate, consider again the previous `users` variable definition. The `ansible.builtin.subelements` filter enables iteration through all users and their authorized key files defined in the variable:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
- name: Set authorized ssh key
  ansible.posix.authorized_key:
    user: "{{ item[0]['name'] }}"
    key: "{{ lookup('ansible.builtin.file', item[1]) }}"
  loop: "{{ users | ansible.builtin.subelements('authorized') }}"
```

The `ansible.builtin.subelements` filter creates a list from the `users` variable data. Each item in the list is itself a two-element list. The first element contains a reference to each user. The second element contains a reference to a single entry from the `authorized` list for that user.

Iterating over a Dictionary

You often encounter data that is organized as a set of key-value pairs, commonly referred to in the Ansible community as a dictionary, instead of being organized as a list. For example, consider the following definition of a `users` variable:

```
users:
  demo1:
    name: Demo User 1
    mail: demo1@example.com
  demo2:
    name: Demo User 2
    mail: demo2@example.com
...output omitted...
  demo200:
    name: Demo User 200
    mail: demo200@example.com
```

Before Ansible 2.5, you had to use the `with_dict` keyword to iterate through the key-value pairs of this dictionary. For each iteration, the `item` variable has two attributes: `key` and `value`. The `key` attribute contains the value of one of the dictionary keys, and the `value` attribute contains the data associated with the dictionary key:

```
- name: Iterate over Users
  ansible.builtin.user:
    name: "{{ item['key'] }}"
    comment: "{{ item['value']['name'] }}"
    state: present
  with_dict: "{{ users }}"
```

Alternatively, you can use the `ansible.builtin.dict2items` filter to transform a dictionary into a list, and this is probably easier to understand. The items in this list are structured the same as items produced by the `with_dict` keyword:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
- name: Iterate over Users
  ansible.builtin.user:
    name: "{{ item['key'] }}"
    comment: "{{ item['value']['name'] }}"
    state: present
  loop: "{{ users | ansible.builtin.dict2items }}"
```

Iterating over a File Globbing Pattern

You can construct a loop that iterates over a list of files that match a provided file globbing pattern with the `ansible.builtin.fileglob` lookup plug-in.

To illustrate, consider the following play:

```
- name: Test
  hosts: localhost
  gather_facts: false
  tasks:
    - name: Test fileglob lookup plugin
      ansible.builtin.debug:
        msg: "{{ lookup('ansible.builtin.fileglob', '~/.bash*') }}"
```

The output from the `ansible.builtin.fileglob` lookup plug-in is a string of comma-separated files, indicated by the double quotation marks around the `msg` variable's data:

```
PLAY [Test] *********************************************************************

TASK [Test fileglob lookup plugin] **********************************************
ok: [localhost] => {
    "msg":
"/home/student/.bash_logout,/home/student/.bash_profile,/home/student/.bashrc,/home
/student/.bash_history"
}

PLAY RECAP **********************************************************************
localhost  : ok=1    changed=0    unreachable=0    failed=0    ...
```

To force a lookup plug-in to return a list of values instead of a string of comma-separated values, use the `query` keyword instead of the `lookup` keyword. Consider the following modification of the previous playbook example:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
- name: Test
  hosts: localhost
  gather_facts: false
  tasks:
    - name: Test fileglob lookup plugin
      ansible.builtin.debug:
        msg: "{{ query('ansible.builtin.fileglob', '~/.bash*') }}"
```

The output of this modified playbook indicates that the `msg` keyword references a list of files, because the data is inside brackets:

```
PLAY [Test] ********************************************************************

TASK [Test fileglob lookup plugin] *********************************************
ok: [localhost] => {
    "msg": [
        "/home/student/.bash_logout",
        "/home/student/.bash_profile",
        "/home/student/.bashrc",
        "/home/student/.bash_history"
    ]
}

PLAY RECAP *********************************************************************
localhost  : ok=1    changed=0    unreachable=0    failed=0    ...
```

To use the data from this lookup plug-in in a loop, ensure that the processed data is returned as a list. Both tasks in the following play iterate over the files matching the `~/.bash*` globbing pattern:

```
- name: Both tasks have the same result
  hosts: localhost
  gather_facts: false
  tasks:

    - name: Iteration Option One
      ansible.builtin.debug:
        msg: "{{ item }}"
      loop: "{{ query('ansible.builtin.fileglob', '~/.bash*') }}"

    - name: Iteration Option Two
      ansible.builtin.debug:
        msg: "{{ item }}"
      with_fileglob: "~/.bash*"
```

The `with_fileglob` task is preferred in this case because it is cleaner.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Retrying a Task

Often you do not want a play to run until a specific condition is met. The `until` directive implements a special kind of loop to specify that kind of condition.

For example, during a blue-green deployment, you must wait until the status of the `blue` host is good before continuing the deployment on the `green` host:

```
- name: Perform smoke test
  ansible.builtin.uri:
    url: "https://{{ blue }}/status"
    return_content: true
  register: smoke_test
  until: "'STATUS_OK' in smoke_test['content']"
  retries: 12
  delay: 10
```

The `blue` web server status page is queried every 10 seconds, and fails after 12 attempts if no queries are successful.

> **References**
> Loops - Ansible Documentation

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

7.6. Guided Exercise
Implementing Advanced Loops

Write tasks that use advanced looping techniques that illustrate some important use cases for complex loops.

Outcomes

- Use filters and lookup plug-ins to iterate over complex data structures.

Prerequisites

As the `student` user on the `workstation` machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
[student@workstation ~]$ lab start data-loops
```

Instructions

1. Use a loop with the `ansible.builtin.dict2items` filter to transform the `rgroups` variable into a list.

   1.1. Change to the `/home/student/data-loops` directory:

   ```
   [student@workstation ~]$ cd ~/data-loops/
   [student@workstation data-loops]$
   ```

   1.2. Review the structure of the `rgroups` variable section defined in the `group_vars/all/my_vars.yml` file:

   ```
   ### Group variables ###
   rgroups:                    (1)
     accounts:                 (2)
       name: accounts          (3)
       gid: 7001
     development:
       name: development
       gid: 7002
   ...output omitted...
   ```

   (1) The `rgroups` variable is a dictionary.

   (2) Each key in the dictionary represents a group name, and the value for each key is another dictionary containing metadata for that group.

   (3) Each group metadata dictionary contains two keys: `name` and `gid`.

   1.3. Review the `add-groups.yml` playbook. This playbook adds a group to the targeted host.

```
---
- name: First task
  hosts: servera.lab.example.com
  gather_facts: false
  become: true

  tasks:
    - name: Add groups
      ansible.builtin.group:
        name: accounts
        gid: 7001
        state: present
```

1.4. Modify the playbook to use the `loop` keyword with the `ansible.builtin.dict2items` filter for your loop.

The `add-groups.yml` playbook should consist of the following content:

```
---
- name: First task
  hosts: servera.lab.example.com
  gather_facts: false
  become: true

  tasks:
    - name: Add groups
      ansible.builtin.group:
        name: "{{ item['key'] }}"
        gid: "{{ item['value']['gid'] }}"
        state: present
      loop: "{{ rgroups | ansible.builtin.dict2items }}"
```

1.5. Run the `add-groups.yml` playbook to add the groups:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation data-loops]$ ansible-navigator run add-groups.yml

PLAY [First task]
*****************************************************************

TASK [Add groups]
*****************************************************************
changed: [servera.lab.example.com] => (item={'key': 'accounts', 'value':
{'name': 'accounts', 'gid': 7001}})
changed: [servera.lab.example.com] => (item={'key': 'development', 'value':
{'name': 'development', 'gid': 7002}})
changed: [servera.lab.example.com] => (item={'key': 'manufacturing',
'value': {'name': 'manufacturing', 'gid': 7003}})
changed: [servera.lab.example.com] => (item={'key': 'marketing', 'value':
{'name': 'marketing', 'gid': 7004}})

PLAY RECAP
*****************************************************************************
servera.lab.example.com    : ok=1    changed=1    unreachable=0    failed=0
skipped=0    rescued=0    ignored=0
```

2. Use a nested loop with the `ansible.builtin.subelements` filter to transform the `my_users` and `my_groups` variables into a list.

   2.1. Review the structure of the `my_users` variable section defined in the `group_vars/all/my_vars.yml` file:

```
...output omitted...
### User variables ###
my_users:                    (1)
  john:                      (2)
    name: john
    my_groups:               (3)
      - accounts
      - development
  jane:
    name: jane
    my_groups:
      - manufacturing
      - marketing
...output omitted...
```

   (1) The `my_users` variable is a dictionary.

   (2) Each key in the dictionary is itself a dictionary, with keys of `name` and `my_groups`.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

③ The `my_groups` key references a list of groups. For example, the user `john` becomes a member of the `accounts` and `development` groups.

2.2. Review the `add-users.yml` playbook. This playbook adds a user and puts it in the correct group.

```
---
- name: Second task
  hosts: servera.lab.example.com
  gather_facts: false
  become: true

  tasks:
    - name: Add users and put them in the right groups
      ansible.builtin.user:
        name: john
        append: true
        groups: accounts
        state: present
```

2.3. Modify the playbook to use the `loop` keyword with the `ansible.builtin.subelements` filter for your loop.

The `add-users.yml` playbook should consist of the following content:

```
---
- name: Second task
  hosts: servera.lab.example.com
  gather_facts: false
  become: true

  tasks:
    - name: Add users and put them in the right groups
      ansible.builtin.user:
        name: "{{ item[0]['name'] }}"
        append: true
        groups: "{{ item[1] }}"
        state: present
      loop: "{{ my_users | ansible.builtin.subelements('my_groups') }}"
```

2.4. Run the `add-users.yml` playbook to add the users:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation data-loops]$ ansible-navigator run add-users.yml

PLAY [Second task]
*************************************************************

TASK [Add users and put them in the right groups]
*****************************
changed: [servera.lab.example.com] => (item=[{'name': 'john', 'my_groups':
['accounts', 'development']}, 'accounts'])
changed: [servera.lab.example.com] => (item=[{'name': 'john', 'my_groups':
['accounts', 'development']}, 'development'])
changed: [servera.lab.example.com] => (item=[{'name': 'jane', 'my_groups':
['manufacturing', 'marketing']}, 'manufacturing'])
changed: [servera.lab.example.com] => (item=[{'name': 'jane', 'my_groups':
['manufacturing', 'marketing']}, 'marketing'])
changed: [servera.lab.example.com] => (item=[{'name': 'tom', 'my_groups':
['accounts', 'marketing']}, 'accounts'])
changed: [servera.lab.example.com] => (item=[{'name': 'tom', 'my_groups':
['accounts', 'marketing']}, 'marketing'])
changed: [servera.lab.example.com] => (item=[{'name': 'lisa', 'my_groups':
['development', 'manufacturing']}, 'development'])
changed: [servera.lab.example.com] => (item=[{'name': 'lisa', 'my_groups':
['development', 'manufacturing']}, 'manufacturing'])

PLAY RECAP
*********************************************************************
servera.lab.example.com    : ok=1    changed=1    unreachable=0    failed=0
skipped=0    rescued=0    ignored=0
```

3. Use a loop over the `public_key_lists` variable defined in the `group_vars/all/my_vars.yml` file. Use the `ansible.builtin.map` filter, followed by the `ansible.builtin.flatten` filter, to generate a simple list of SSH public key files for all developers. Update the task's `key` keyword to contain the file content of each iteration item.

   3.1. Review the structure of the `public_key_lists` variable section defined in the `group_vars/all/my_vars.yml` file:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
...output omitted...
### Public key variables ###
public_key_lists:          (1)
  - username: john         (2)
    public_keys:           (3)
      - pubkeys/john/id_rsa.pub
      - pubkeys/john/laptop_rsa.pub
  - username: jane
    public_keys:
      - pubkeys/jane/id_rsa.pub
...output omitted...
```

(1) The `public_key_lists` variable is a list.

(2) Each entry in the list is a dictionary with the `public_keys` and `username` keys.

(3) The `public_keys` key references a list of public key files for the given user. For example, the `john` user has two public key files: `pubkeys/john/id_rsa.pub` and `pubkeys/john/laptop_rsa.pub`. The file names are relative to the location of the playbook.

3.2. Review the `add-keys.yml` playbook. This playbook sets up authorized keys for the generic `developer` account:

```
---
- name: Third task
  hosts: servera.lab.example.com
  gather_facts: false
  become: true

  tasks:
    - name: Set up authorized keys
      ansible.posix.authorized_key:
        user: developer
        state: present
        key: pubkeys/john/id_rsa.pub
```

3.3. Modify the playbook to perform the following tasks:

- Add the Jinja2 `map` filter to the end of the expression to extract the `public_keys` attribute. This change creates a list of lists; each entry in the list is a list of SSH public key files for a particular user.
- Add the `ansible.builtin.flatten` filter after the `map` filter to create a single list of files.
- Use the `ansible.builtin.file` lookup plug-in to configure the `key` keyword with the file content of each iteration item.

The `add-keys.yml` playbook should consist of the following content:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
---
- name: Third task
  hosts: servera.lab.example.com
  gather_facts: false
  become: true

  tasks:
    - name: Set up authorized keys
      ansible.posix.authorized_key:
        user: developer
        state: present
        key: "{{ lookup('ansible.builtin.file', item) }}"
      loop: >-
        {{ public_key_lists | map(attribute='public_keys') |
        ansible.builtin.flatten }}
```

3.4. Run the `add-keys.yml` playbook to add the keys:

```
[student@workstation data-loops]$ ansible-navigator run add-keys.yml

PLAY [Third task]
*****************************************************************

TASK [Set up authorized keys]
*************************************************
changed: [servera.lab.example.com] => (item=pubkeys/john/id_rsa.pub)
changed: [servera.lab.example.com] => (item=pubkeys/john/laptop_rsa.pub)
changed: [servera.lab.example.com] => (item=pubkeys/jane/id_rsa.pub)
changed: [servera.lab.example.com] => (item=pubkeys/tom/id_rsa.pub)
changed: [servera.lab.example.com] => (item=pubkeys/lisa/id_rsa.pub)

PLAY RECAP
*********************************************************************
servera.lab.example.com    : ok=1    changed=1    unreachable=0    failed=0
skipped=0    rescued=0    ignored=0
```

Finish

On the `workstation` machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish data-loops
```

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

7.7. Using Filters to Work with Network Addresses

Objectives

- Use filters to inspect, validate, and manipulate variables containing networking information.

Gathering and Processing Networking Information

A number of filters and lookup plug-ins are available to collect and process network information for Ansible automation. These filters and plug-ins can be useful with fact gathering when you configure the network devices on your managed hosts.

The standard `ansible.builtin.setup` module, which automatically gathers facts at the start of many plays, collects a lot of network-related information from each managed host.

The `ansible_facts['interfaces']` fact is a list of all the network interface names on the system. You can use this list to examine the details of each of the network interfaces. For example, if `enp11s0` is an interface on the system, then it has a fact called `ansible_facts['enp11s0']`, which is a dictionary containing information about the interface such as the MAC address, IPv4 and IPv6 addresses, kernel module, and so on.

The following facts are useful to remember:

Table 6. Selected Networking Facts

Fact name	Description
`ansible_facts['dns']['nameservers']`	The DNS name servers used for name resolution by the managed host (included in the `min` subset).
`ansible_facts['domain']`	The domain for the managed host.
`ansible_facts['all_ipv4_addresses']`	All the IPv4 addresses configured on the managed host.
`ansible_facts['all_ipv6_addresses']`	All the IPv6 addresses configured on the managed host.
`ansible_facts['fqdn']`	The fully qualified domain name (DNS name) of the managed host.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Fact name	Description
ansible_facts['hostname']	The unqualified hostname; the string in the FQDN before the first period.

The following playbook shows an example of using network facts:

```
---
- name: Set up web servers
  hosts: web_servers
  become: true

  tasks:
    - name: Ensure httpd is installed
      ansible.builtin.dnf:
        name: httpd
        state: installed

    - name: Configure web server status page
      ansible.builtin.copy:
        content: >
          Server: {{ ansible_facts['fqdn'] }} at
          {{ ansible_facts['default_ipv4']['address'] }}
        dest: /var/www/html/status.html
        owner: apache
        group: apache
        mode: '0444'

    - name: Start and enable webserver
      ansible.builtin.service:
        name: httpd
        state: started
        enabled: true

    - name: Notify root of server provisioning
      community.general.mail:
        subject: >
          System {{ ansible_facts['hostname'] }} has
          been successfully provisioned.
```

Network Information Filters

You can validate and manipulate networking-related data stored in variables and facts with the `ansible.utils.ipaddr` filter. The `ansible.utils.ipaddr` filter can validate the syntax of IP addresses, filter

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

out bad data, convert from Variable Length Subnet Mask (VLSM) subnet masks to Classless Inter-Domain Routing (CIDR) subnet prefix notation, perform subnet math, or find the next usable address in a network range.

See the References section for comprehensive documentation for the `ansible.utils.ipaddr` filter. This section explores some use cases for the filter.

> **Note**
> The `ansible.utils.ipaddr` filter needs the `netaddr` Python module, provided by the `python3-netaddr` package in Red Hat Enterprise Linux 8 and later.
>
> This Python module is installed by default in the `ansible-automation-platform-25/ee-supported-rhel8` execution environment. Ensure that the Python module is installed on your control node if you are using the `ansible-playbook` command, or if you are using the `ansible-navigator` command without an automation execution environment.

Testing IP Addresses

In its simplest form, the `ansible.utils.ipaddr` filter with no arguments accepts a single value. If the value is an IP address, then the filter returns the IP address. If the value is not an IP address, then the filter returns `false`.

Given a list, the filter returns the valid IP addresses but not the invalid ones. If all the items are invalid, then the filter returns an empty list.

```
{{ my_hosts_list | ansible.utils.ipaddr }}
```

You can also pass arguments to the `ansible.utils.ipaddr` filter. These arguments can make the filter return different information based on the data input.

Filtering Data

The following subsections explore some options that you can use with the `ansible.utils.ipaddr` filter. To better understand how they work, each of the following examples shows the output that results when that filter is used with a variation on the following task:

```
    - name: Filter example
      ansible.builtin.debug:
        var: listips | ansible.utils.ipaddr
```

The `ipaddr` filter in the `var` line is replaced with a version that uses the appropriate option. For example, the following version of the line retrieves the network masks of the given addresses:

```
        var: listips | ansible.utils.ipaddr('netmask')
```

The `listips` variable used to generate the examples contains the following list:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
    listips:
      - 192.168.2.1
      - 10.0.0.128/25
      - 172.24.10.0/255.255.255.0
      - 172.24.10.0/255.255.255.255
      - ff02::1
      - ::1
      - 2001::1/64
      - 2001::/64
      - www.redhat.com
```

`host`

Outputs only valid individual IP addresses, in the correct CIDR prefix format for host addresses.

```
    "listips | ansible.utils.ipaddr('host')": [
        "192.168.2.1/32",
        "172.24.11.0/32",
        "ff02::1/128",
        "::1/128",
        "2001::1/64"
    ]
```

`net`

Filters out values that are not valid network specifications, converting the netmask into a CIDR prefix if necessary.

```
    "listips | ansible.utils.ipaddr('net')": [
        "10.0.0.128/25",
        "172.24.10.0/24",
        "2001::/64"
    ]
```

`private`

Displays input IP addresses or network ranges that are in ranges reserved by *Internet Assigned Numbers Authority (IANA)* to be private. This includes RFC 1918 address spaces and IPv6 site-local addresses at a minimum.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
    "listips | ansible.utils.ipaddr('private')": [
        "192.168.2.1",
        "10.0.0.128/25",
        "172.24.10.0/255.255.255.0",
        "172.24.10.0/255.255.255.255",
        "::1",
        "2001::1/64",
        "2001::/64
    ]
```

`public`

Outputs addresses and networks in public (globally routable) address spaces, according to IANA. (This example has no addresses in the public space.)

```
    "listips | ansible.utils.ipaddr('public')": []
```

The `ansible.utils.ipwrap` filter puts brackets around the address part of items that appear to be IPv6 addresses. The filter passes all other values unchanged. This is useful in templates for certain configuration files that expect this syntax for IPv6 addresses.

```
    "listips | ansible.utils.ipwrap": [
        "192.168.2.1",
        "10.0.0.128/25",
        "172.24.10.0/255.255.255.0",
        "172.24.10.0/255.255.255.255",
        "[ff02::1]",
        "[::1]",
        "[2001::1]/64",
        "[2001::]/64",
        "www.redhat.com"
    ]
```

Note

Most of the filters in the preceding examples remove the `www.redhat.com` list item, which is not an IP address. The `ansible.utils.ipwrap` filter does not do this, but you could use the `ansible.utils.ipaddr` filter first to handle this scenario:

```
"{{ listips | ansible.utils.ipaddr | ansible.utils.ipwrap }}"
```

Manipulating IP Addresses

You can use a number of the `ansible.utils.ipaddr` options to format networking data from an IPv4 or IPv6 address.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

For example:

- To return the address part, `192.0.2.1`:

```
"{{ '192.0.2.1/24' | ansible.utils.ipaddr('address') }}"
```

- To return the variable-length subnet mask, `255.255.255.0`:

```
"{{ '192.0.2.1/24' | ansible.utils.ipaddr('netmask') }}"
```

- To return the CIDR prefix, 24:

```
"{{ '192.0.2.1/24' | ansible.utils.ipaddr('prefix') }}"
```

- To return the network's broadcast address, `192.0.2.255`:

```
"{{ '192.0.2.1/24' | ansible.utils.ipaddr('broadcast') }}"
```

- To return the network's network address, `192.0.2.0`:

```
"{{ '192.0.2.1/24' | ansible.utils.ipaddr('network') }}"
```

- To return the IP address in DNS pointer record (PTR) format, `1.2.0.192.in-addr.arpa.`:

```
"{{ '192.0.2.1/24' | ansible.utils.ipaddr('revdns') }}"
```

This correctly converts IPv6 addresses as well.

If you pass a list of items to `ansible.utils.ipaddr`, it attempts to convert the items that appear to be valid addresses and filter out items that are not.

> **Important**
> The `broadcast` option should not be used with an IPv6 address/prefix because that protocol does not assign a broadcast address. Other options should work with IPv6 addresses as expected.

Reformatting or Calculating Network Information

You can display an IP address on a given network by passing an integer to the `ansible.utils.ipaddr` filter. The following returns the fifth address (`192.0.2.5/24`) on the `192.0.2.0/24` network:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
"{{ '192.0.2.0/24' | ansible.utils.ipaddr(5) }}"
```

You can convert the network specification to a range of usable addresses for hosts with the `range_usable` option. The following example returns the `192.0.2.1-192.0.2.254` address range:

```
"{{ '192.0.2.0/24' | ansible.utils.ipaddr('range_usable') }}"
```

If you pass a network specification to the `ansible.utils.network_in_usable` filter, it determines whether an IP address is usable for hosts on that network. The following example returns the Boolean value `true`:

```
"{{ '192.0.2.0/24' | ansible.utils.network_in_usable('192.0.2.5') }}"
```

If you replace `192.0.2.5` with `192.0.2.255` (the reserved broadcast address) or `192.0.3.2` (not on the network), the filter returns the Boolean value `false`.

You can also find the next host on a network. The following example returns the `192.0.2.6/24` address:

```
"{{ '192.0.2.5/24' | ansible.utils.ipaddr('next_usable') }}"
```

When the end of the network is reached, the filter returns an empty value.

Network administrators can use the `ansible.utils.cidr_merge` filter to merge a list of subnets and individual hosts into the minimal representation. For example, consider the following list:

```
    mynetworks:
      - 10.0.1.0/24
      - 10.0.2.0/24
      - 10.0.3.0/24
      - 10.0.100.0/24
      - 2001:db8::/64
      - 2001:db8:0:1::/64
```

You can use the `"{{ mynetworks | ansible.utils.cidr_merge }}"` expression to return the following list:

```
    "mynetworks | ansible.utils.cidr_merge": [
        "10.0.1.0/24",
        "10.0.2.0/23",
        "10.0.100.0/24",
        "2001:db8::/63"
    ]
```

More filters, as well as more options for the preceding filters, are available in the `ansible.utils` collection. Find more information about using these filters in the References section.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

References

Using the ipaddr Filter - Ansible Documentation

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

7.8. Guided Exercise
Using Filters to Work with Network Addresses

Use filters and plug-ins to manipulate and validate data stored in variables that contain networking-related information.

Outcomes

- Use the `ansible.utils.ipaddr` filter to derive networking information from IP address data retrieved from gathered facts.

Prerequisites

As the `student` user on the `workstation` machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
[student@workstation ~]$ lab start data-netfilters
```

Instructions

1. Get familiar with the Ansible project. Review and run the `default_ipv4.yml` playbook.

   1.1. Change to the `/home/student/data-netfilters directory` and list the files in that project directory:

   ```
   [student@workstation ~]$ cd ~/data-netfilters/
   [student@workstation data-netfilters]$ ls
   ansible-navigator.yml  ansible.cfg  default_ipv4.yml  filtering_data.yml
   inventory  site.yml
   ```

   1.2. Review the `default_ipv4.yml` playbook:

   ```
   ---
   - name: Display the default ipv4 address facts
     hosts: servera.lab.example.com
     become: false

     tasks:
       - name: Print the default ipv4 address facts
         ansible.builtin.debug:
           var: ansible_facts['default_ipv4']
   ```

   1.3. Run the `default_ipv4.yml` playbook to list the `ansible_facts['default_ipv4']` facts:

```
[student@workstation data-netfilters]$ ansible-navigator run
default_ipv4.yml

PLAY [Display the default ipv4 address facts]
*********************************

TASK [Gathering Facts]
************************************************************
ok: [servera.lab.example.com]

TASK [Print the default ipv4 address facts]
***********************************
ok: [servera.lab.example.com] => {
    "ansible_facts['default_ipv4']": {
        "address": "172.25.250.10",
        "alias": "eth0",
        "broadcast": "172.25.250.255",
        "gateway": "172.25.250.254",
        "interface": "eth0",
        "macaddress": "52:54:00:00:fa:0a",
        "mtu": 1500,
        "netmask": "255.255.255.0",
        "network": "172.25.250.0",
        "prefix": "24",
        "type": "ether"
    }
}

PLAY RECAP
*********************************************************************
servera.lab.example.com    : ok=2    changed=0    unreachable=0    failed=0
...
```

2. The `filtering_data.yml` playbook contains incomplete tasks that use the `ansible.utils.ipaddr` filter with various options. Edit the `filtering_data.yml` playbook, replacing any occurrences of `#FIXME#` in a fact definition with the expected value.

   The `listips` variable that is used to generate the examples contains the following list:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
  listips:
    - 192.168.2.1
    - 10.0.0.128/25
    - 172.24.10.0/255.255.255.0
    - 172.24.10.0/255.255.255.255
    - ff02::1
    - ::1
    - 2001::1/64
    - 2001::/64
    - www.redhat.com
```

2.1. Review the first task, which uses `ansible.utils.ipaddr` without any options to output the items in the list that are valid IP addresses or networks. You do not need to change anything in that task.

```
- name: Display all addresses
  ansible.builtin.debug:
    var: listips | ansible.utils.ipaddr
```

2.2. Use the `host` option with the `ansible.utils.ipaddr` filter to output only valid individual IP addresses, in the correct Classless Inter-Domain Routing (CIDR) prefix format for host addresses:

```
- name: Use host
  ansible.builtin.debug:
    var: listips | ansible.utils.ipaddr('host')
```

2.3. Use the `net` option to output only values that are valid network specifications, converting the netmask to CIDR prefix format if necessary:

```
- name: Use net
  ansible.builtin.debug:
    var: listips | ansible.utils.ipaddr('net')
```

2.4. Use the `ansible.utils.ipwrap` filter to put brackets around the address part of IPv6 addresses:

```
- name: Use ipwrap
  ansible.builtin.debug:
    var: listips | ansible.utils.ipwrap
```

2.5. Most of the filters in the preceding examples removed the `www.redhat.com` list item, which is not an IP address, except for the `ansible.utils.ipwrap` filter. The following example uses the `ansible.utils.ipaddr` filter, followed by the `ansible.utils.ipwrap` filter, to exclude the `www.redhat.com` list item:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
- name: Use ipaddr and ipwrap
  ansible.builtin.debug:
    var: listips | ansible.utils.ipaddr | ansible.utils.ipwrap
```

2.6. The completed `filtering_data.yml` playbook should consist of the following content:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
# Complete each task by setting the fact as the expected value.
# Replace #FIXME# by the appropriate filter usage.

- name: First play for netfilter exercise
  hosts: servera.lab.example.com
  become: false
  vars:
    listips:
      - 192.168.2.1
      - 10.0.0.128/25
      - 172.24.10.0/255.255.255.0
      - 172.24.10.0/255.255.255.255
      - ff02::1
      - ::1
      - 2001::1/64
      - 2001::/64
      - www.redhat.com

  tasks:
    - name: Display all addresses
      ansible.builtin.debug:
        var: listips | ansible.utils.ipaddr

    - name: Use host
      ansible.builtin.debug:
        var: listips | ansible.utils.ipaddr('host')

    - name: Use net
      ansible.builtin.debug:
        var: listips | ansible.utils.ipaddr('net')

    - name: Use ipwrap
      ansible.builtin.debug:
        var: listips | ansible.utils.ipwrap

    - name: Use ipaddr and ipwrap
      ansible.builtin.debug:
        var: listips | ansible.utils.ipaddr | ansible.utils.ipwrap
```

2.7. Run the `filtering_data.yml` playbook and verify the output of each task:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation data-netfilters]$ ansible-navigator run
filtering_data.yml

PLAY [First play for netfilter exercise]
************************************

TASK [Gathering Facts]
*********************************************************
ok: [servera.lab.example.com]

TASK [Display all addresses]
*****************************************************
ok: [servera.lab.example.com] => {
    "listips | ansible.utils.ipaddr": [
        "192.168.2.1",
        "10.0.0.128/25",
        "172.24.10.0/24",
        "172.24.10.0/32",
        "ff02::1",
        "::1",
        "2001::1/64",
        "2001::/64"
    ]
}

TASK [Use host]
***********************************************************************
ok: [servera.lab.example.com] => {
    "listips | ansible.utils.ipaddr('host')": [
        "192.168.2.1/32",
        "172.24.10.0/32",
        "ff02::1/128",
        "::1/128",
        "2001::1/64"
    ]
}

TASK [Use net]
************************************************************************
ok: [servera.lab.example.com] => {
    "listips | ansible.utils.ipaddr('net')": [
        "10.0.0.128/25",
        "172.24.10.0/24",
        "2001::/64"
    ]
}
```


For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
TASK [Use ipwrap]
*****************************************************************
ok: [servera.lab.example.com] => {
    "listips | ansible.utils.ipwrap": [
        "192.168.2.1",
        "10.0.0.128/25",
        "172.24.10.0/255.255.255.0",
        "172.24.10.0/255.255.255.255",
        "[ff02::1]",
        "[::1]",
        "[2001::1]/64",
        "[2001::]/64",
        "www.redhat.com"
    ]
}

TASK [Use ipaddr and ipwrap]
*************************************************
ok: [servera.lab.example.com] => {
    "listips | ansible.utils.ipaddr | ansible.utils.ipwrap": [
        "192.168.2.1",
        "10.0.0.128/25",
        "172.24.10.0/24",
        "172.24.10.0/32",
        "[ff02::1]",
        "[::1]",
        "[2001::1]/64",
        "[2001::]/64"
    ]
}

PLAY RECAP
*********************************************************************
servera.lab.example.com    : ok=6    changed=0    unreachable=0    failed=0
...
```

3. Open the `site.yml` playbook. The `tasks` section of the play in that playbook has two parts. The tasks in the first part use the `ansible.builtin.set_fact` module to set variables that later tasks use with the `ansible.utils.ipaddr` filter. Replace any occurrence of `#FIXME#` in a fact definition with the value specified by the following substeps. The tasks in the second part use the `ansible.builtin.debug` module to print the information contained in the facts that you set.

   3.1. Use the `ansible.utils.ipaddr` filter to verify that the `ipv4_addr` variable contains a properly formatted IP address:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
    - name: Set facts derived from ipv4_addr and ipv4_subnet
      ansible.builtin.set_fact:
        server_address: "{{ ipv4_addr | ansible.utils.ipaddr }}"
```

3.2. Use the `revdns` option with `ansible.utils.ipaddr` filter to retrieve the IP address in DNS pointer record (PTR) format:

```
    - name: Set facts derived from ipv4_addr and ipv4_subnet
      ansible.builtin.set_fact:
        server_address: "{{ ipv4_addr | ansible.utils.ipaddr }}"
        ptr_record: "{{ ipv4_addr | ansible.utils.ipaddr('revdns') }}"
```

3.3. Use the `network/prefix` option to retrieve the network address and CIDR prefix:

```
    - name: Set facts derived from net_mask
      ansible.builtin.set_fact:
        cidr: "{{ net_mask | ansible.utils.ipaddr('network/prefix') }}"
```

3.4. Use the `broadcast` option to retrieve the broadcast address:

```
    - name: Set facts derived from net_mask
      ansible.builtin.set_fact:
        cidr: "{{ net_mask | ansible.utils.ipaddr('network/prefix') }}"
        broadcast: "{{ net_mask | ansible.utils.ipaddr('broadcast') }}"
```

3.5. Use the `cidr` variable, which you set in an earlier task, to provide the network and its CIDR prefix as an option to the `ansible.utils.ipaddr` filter so that the list it returns only includes IP addresses that are on that network:

```
- name: Print addresses on {{ cidr }} from this task's list
  ansible.builtin.debug:
    msg: "{{ item | ansible.utils.ipaddr(cidr) }}"
  loop:
    - 172.16.0.1
    - 172.25.250.4
    - 192.168.122.20
    - 192.0.2.55
```

> **Important**
>
> In this example, `cidr` must *not* be in quotes because it is a variable defined earlier in the play that must be expanded, not a literal option for the filter.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

3.6. Verify that the completed `site.yml` playbook consists of the following content:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
# Complete each task by setting the fact as the expected value.
# Replace #FIXME# by the appropriate filter usage.
# Tasks make use of the gathered fact 'default_ipv4', and its keys
'address', and 'netmask'.

- name: Second play for netfilter exercise
  hosts: servera.lab.example.com
  become: false

  tasks:
    - name: Set address and network facts
      ansible.builtin.set_fact:
        ipv4_addr: "{{ ansible_facts['default_ipv4']['address'] }}"
        ipv4_subnet: "{{ ansible_facts['default_ipv4']['netmask'] }}"

    - name: Set facts derived from ipv4_addr and ipv4_subnet
      ansible.builtin.set_fact:
        server_address: "{{ ipv4_addr | ansible.utils.ipaddr }}"
        ptr_record: "{{ ipv4_addr | ansible.utils.ipaddr('revdns') }}"
        net_mask: "{{ ipv4_addr }}/{{ ipv4_subnet }}"

    - name: Set facts derived from net_mask
      ansible.builtin.set_fact:
        cidr: "{{ net_mask | ansible.utils.ipaddr('network/prefix') }}"
        broadcast: "{{ net_mask | ansible.utils.ipaddr('broadcast') }}"

    - name: Display server_address
      ansible.builtin.debug:
        var: server_address

    - name: Display PTR record form for server_address
      ansible.builtin.debug:
        var: ptr_record

    - name: Display address and netmask in VLSM notation
      ansible.builtin.debug:
        var: net_mask

    - name: Display address and prefix in CIDR notation
      ansible.builtin.debug:
        var: cidr

    - name: Display broadcast address for {{ cidr }}
      ansible.builtin.debug:
        var: broadcast
```

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
    - name: Print addresses on {{ cidr }} from this task's list
      ansible.builtin.debug:
        msg: "{{ item | ansible.utils.ipaddr(cidr) }}"
      loop:
        - 172.16.0.1
        - 172.25.250.4
        - 192.168.122.20
        - 192.0.2.55
```

3.7. Run the `site.yml` playbook and verify the output of each task:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation data-netfilters]$ ansible-navigator run site.yml

PLAY [Second play for netfilter exercise]
**************************************

TASK [Gathering Facts]
*********************************************************
ok: [servera.lab.example.com]

TASK [Set address and network facts]
******************************************
ok: [servera.lab.example.com]

TASK [Set facts derived from ipv4_addr and ipv4_subnet]
************************
ok: [servera.lab.example.com]

TASK [Set facts derived from net_mask]
***************************************
ok: [servera.lab.example.com]

TASK [Display server_address]
************************************************
ok: [servera.lab.example.com] => {
    "server_address": "172.25.250.10
}

TASK [Display PTR record form for server_address]
*******************************
ok: [servera.lab.example.com] => {
    "ptr_record": "10.250.25.172.in-addr.arpa."
}

TASK [Display address and netmask in VLSM notation]
****************************
ok: [servera.lab.example.com] => {
    "net_mask": "172.25.250.10/255.255.255.0"
}

TASK [Display address and prefix in CIDR notation]
****************************
ok: [servera.lab.example.com] => {
    "cidr": "172.25.250.0/24"
}

TASK [Display broadcast address for 172.25.250.0/24]
```

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
**************************
ok: [servera.lab.example.com] => {
    "broadcast": "172.25.250.255"
}

TASK [Print addresses on 172.25.250.0/24 from this task's list]
***************
ok: [servera.lab.example.com] => (item=172.16.0.1) => {
    "msg": ""
}
ok: [servera.lab.example.com] => (item=172.25.250.4) => {
    "msg": "172.25.250.4"
}
ok: [servera.lab.example.com] => (item=192.168.122.20) => {
    "msg": ""
}
ok: [servera.lab.example.com] => (item=192.0.2.55) => {
    "msg": ""
}

PLAY RECAP
*********************************************************************
servera.lab.example.com    : ok=10   changed=0    unreachable=0    failed=0
...
```

Finish

On the `workstation` machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish data-netfilters
```

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

7.9. Lab
Transforming Data with Filters and Plug-ins

Use filters and plug-ins to transform data in variables and facts in order to use that data for other tasks in the same play.

Outcomes

- Use filters and lookup plug-ins to manipulate variables in playbook and role tasks.

Prerequisites

As the `student` user on the `workstation` machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available.

This command ensures that the remote Git repository `https://git.lab.example.com/student/data-review.git` is initialized. The Git repository contains playbooks that configure a front-end load balancer and a pool of back-end web servers. You can push changes to this repository using `Stud3nt123` as the Git password.

```
[student@workstation ~]$ lab start data-review
```

Instructions

1. Clone the `https://git.lab.example.com/student/data-review.git` repository to the `/home/student/git-repos` directory and then create the `exercise` branch.

2. Examine the `firewall` role in the `~/git-repos/data-review` project directory.

   The `roles/firewall/tasks/main.yml` tasks file contains three tasks that use the `ansible.posix.firewalld` module to configure firewall rules defined by the `firewall_rules` variable.

   Edit the tasks file so that it uses filters to set default values for variables in each of the three tasks if they are not set, as follows:

   - For the `state` option, if `item['state']` is not set, then set it to `enabled` by default.
   - For the `zone` option, if `item['zone']` is not set, then omit the `zone` option.
   - In the `Ensure Firewall Port Configuration` task, in the `port` option to the `firewalld` module, if `item['protocol']` is not set then set it to `tcp`. Also use the `lower` filter to ensure that the value of the `item['protocol']` option is in lowercase.

3. Test the changes you made to the `roles/firewall/tasks/main.yml` file in the preceding step by running the `test_firewall_role.yml` playbook.

   If you completed the preceding step successfully, then the playbook runs without errors.

4. Examine the `deploy_apache.yml` playbook. It calls the `apache` role to ensure that Apache HTTP Server is deployed, which itself calls the `firewall` role. The playbook also defines the `firewall_rules` variable to make sure that the `http` service for `firewalld` and the IPv4 address of the load balancer (`172.25.250.10`) are both enabled in the `internal` zone for `firewalld`.

   Edit the playbook. You need to change the value of the `firewall_rules` variable in the `deploy_apache.yml` playbook to a Jinja2 expression that uses the `template` lookup plug-in to dynamically generate the variable's setting from the `apache_firewall_rules.yml.j2` Jinja2 template. You must use the `from_yaml` filter in the Jinja2 expression to convert the resulting value from a text string into a YAML data structure that can be interpreted by Ansible.

5. Examine the `templates/apache_firewall_rules.yml.j2` template that your playbook now uses to set the `firewall_rules` variable.

   For the first rule, the `apache_port` variable is set by files in the `group_vars` directory.

   The template contains a Jinja2 `for` loop that creates a rule for each host in the `lb_servers` group, setting the source IP address from the value of the `load_balancer_ip_addr` variable.

   This is not the best solution. It requires you to set `load_balancer_ip_addr` as a host variable for each load balancer in the `lb_servers` group. To remove this manual maintenance, use gathered facts from these hosts to set that value instead.

   Edit the `templates/apache_firewall_rules.yml.j2` template to replace the `load_balancer_ip_addr` host variable in the Jinja2 `for` loop with the `hostvars[server]['ansible_facts']['default_ipv4']['address']` fact.

6. Run the `site.yml` playbook to test your changes. If you successfully completed the preceding steps, then the playbook runs without errors.

7. Verify that web browser requests from `workstation` to the load balancer on `servera` succeed, and that direct requests from `workstation` to either TCP port 80 or TCP port 8008 on the back-end web servers (`serverb` and `serverc`) are denied.

8. Commit and push your changes to the remote Git repository. If prompted, use `Stud3nt123` as the Git password.

Evaluation

As the `student` user on the `workstation` machine, use the `lab` command to grade your work. Correct any reported failures and rerun the command until successful.

```
[student@workstation ~]$ lab grade data-review
```

Finish

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

As the `student` user on the `workstation` machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish data-review
```

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Solution

Use filters and plug-ins to transform data in variables and facts in order to use that data for other tasks in the same play.

Outcomes

- Use filters and lookup plug-ins to manipulate variables in playbook and role tasks.

Prerequisites

As the `student` user on the `workstation` machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available.

This command ensures that the remote Git repository `https://git.lab.example.com/student/data-review.git` is initialized. The Git repository contains playbooks that configure a front-end load balancer and a pool of back-end web servers. You can push changes to this repository using `Stud3nt123` as the Git password.

```
[student@workstation ~]$ lab start data-review
```

Instructions

1. Clone the `https://git.lab.example.com/student/data-review.git` repository to the `/home/student/git-repos` directory and then create the `exercise` branch.

   1.1. From a terminal, create the `/home/student/git-repos` directory if it does not exist, and then change into it:

   ```
   [student@workstation ~]$ mkdir -p ~/git-repos/
   [student@workstation ~]$ cd ~/git-repos/
   [student@workstation git-repos]$
   ```

   1.2. Clone the `https://git.lab.example.com/student/data-review.git` repository and then change to the cloned repository:

   ```
   [student@workstation git-repos]$ git clone \
   https://git.lab.example.com/student/data-review.git
   Cloning into 'data-review'...
   ...output omitted...
   [student@workstation git-repos]$ cd data-review
   [student@workstation data-review]$
   ```

   1.3. Create the exercise branch and check it out:

```
[student@workstation data-review]$ git checkout -b exercise
Switched to a new branch 'exercise'
```

2. Examine the `firewall` role in the `~/git-repos/data-review` project directory.

   The `roles/firewall/tasks/main.yml` tasks file contains three tasks that use the `ansible.posix.firewalld` module to configure firewall rules defined by the `firewall_rules` variable.

   Edit the tasks file so that it uses filters to set default values for variables in each of the three tasks if they are not set, as follows:

   - For the `state` option, if `item['state']` is not set, then set it to `enabled` by default.
   - For the `zone` option, if `item['zone']` is not set, then omit the `zone` option.
   - In the `Ensure Firewall Port Configuration` task, in the `port` option to the `firewalld` module, if `item['protocol']` is not set then set it to `tcp`. Also use the `lower` filter to ensure that the value of the `item['protocol']` option is in lowercase.

   2.1. Edit the `roles/firewall/tasks/main.yml` file. For the `state` option of all three `firewalld` tasks, add the `default('enabled')` filter to the `item['state']` Jinja2 expression.

   The resulting file contains the following content:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
---
# tasks file for firewall

- name: Ensure Firewall Sources Configuration
  ansible.posix.firewalld:
    source: "{{ item['source'] }}"
    zone: "{{ item['zone'] }}"
    immediate: true
    permanent: true
    state: "{{ item['state'] | default('enabled') }}"
  loop: "{{ firewall_rules }}"
  when: item['source'] is defined

- name: Ensure Firewall Service Configuration
  ansible.posix.firewalld:
    service: "{{ item['service'] }}"
    zone: "{{ item['zone'] }}"
    immediate: true
    permanent: true
    state: "{{ item['state'] | default('enabled') }}"
  loop: "{{ firewall_rules }}"
  when: item['service'] is defined

- name: Ensure Firewall Port Configuration
  ansible.posix.firewalld:
    port: "{{ item['port'] }}/{{ item['protocol'] }}"
    zone: "{{ item['zone'] }}"
    immediate: true
    permanent: true
    state: "{{ item['state'] | default('enabled') }}"
  loop: "{{ firewall_rules }}"
  when: item['port'] is defined
```

2.2. Continue editing the tasks file. For the `zone` option of all three `firewalld` tasks, add the `default(omit)` filter to the `item['zone']` Jinja2 expression.

> **Note**
>
> If desired, you can use the `default(omit, true)` filter instead. Adding `true` to the `default` filter means that the filter is also used if the variable is either an empty string or is set to the `false` Boolean value.
>
> The `default` filter is always used if the variable is undefined.

After this step, the task file contains the following content:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
- name: Ensure Firewall Sources Configuration
  ansible.posix.firewalld:
    source: "{{ item['source'] }}"
    zone: "{{ item['zone'] | default(omit) }}"
    immediate: true
    permanent: true
    state: "{{ item['state'] | default('enabled') }}"
  loop: "{{ firewall_rules }}"
  when: item['source'] is defined

- name: Ensure Firewall Service Configuration
  ansible.posix.firewalld:
    service: "{{ item['service'] }}"
    zone: "{{ item['zone'] | default(omit) }}"
    immediate: true
    permanent: true
    state: "{{ item['state'] | default('enabled') }}"
  loop: "{{ firewall_rules }}"
  when: item['service'] is defined

- name: Ensure Firewall Port Configuration
  ansible.posix.firewalld:
    port: "{{ item['port'] }}/{{ item['protocol'] }}"
    zone: "{{ item['zone'] | default(omit) }}"
    immediate: true
    permanent: true
    state: "{{ item['state'] | default('enabled') }}"
  loop: "{{ firewall_rules }}"
  when: item['port'] is defined
```

2.3. Edit the task file again. In the third `firewalld` task, `Ensure Firewall Port Configuration`, replace `{{ item['protocol'] }}` with `{{ item['protocol'] | default('tcp') | lower }}`. Save your work.

The completed task file contains the following content:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
- name: Ensure Firewall Sources Configuration
  ansible.posix.firewalld:
    source: "{{ item['source'] }}"
    zone: "{{ item['zone'] | default(omit) }}"
    immediate: true
    permanent: true
    state: "{{ item['state'] | default('enabled') }}"
  loop: "{{ firewall_rules }}"
  when: item['source'] is defined

- name: Ensure Firewall Service Configuration
  ansible.posix.firewalld:
    service: "{{ item['service'] }}"
    zone: "{{ item['zone'] | default(omit) }}"
    immediate: true
    permanent: true
    state: "{{ item['state'] | default('enabled') }}"
  loop: "{{ firewall_rules }}"
  when: item['service'] is defined

- name: Ensure Firewall Port Configuration
  ansible.posix.firewalld:
    port: "{{ item['port'] }}/{{ item['protocol'] | default('tcp') | lower
}}"
    zone: "{{ item['zone'] | default(omit) }}"
    immediate: true
    permanent: true
    state: "{{ item['state'] | default('enabled') }}"
  loop: "{{ firewall_rules }}"
  when: item['port'] is defined
```

3. Test the changes you made to the `roles/firewall/tasks/main.yml` file in the preceding step by running the `test_firewall_role.yml` playbook.

   If you completed the preceding step successfully, then the playbook runs without errors.

   Run the `test_firewall_role.yml` playbook to test your changes:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation data-review]$ ansible-navigator run \
test_firewall_role.yml

PLAY [Test Firewall Role] ******************************************************
...output omitted...

PLAY RECAP *********************************************************************
serverb.lab.example.com  : ok=4  changed=2  unreachable=0  failed=0  ...
serverc.lab.example.com  : ok=4  changed=2  unreachable=0  failed=0  ...
```

4. Examine the `deploy_apache.yml` playbook. It calls the `apache` role to ensure that Apache HTTP Server is deployed, which itself calls the `firewall` role. The playbook also defines the `firewall_rules` variable to make sure that the `http` service for `firewalld` and the IPv4 address of the load balancer (`172.25.250.10`) are both enabled in the `internal` zone for `firewalld`.

   Edit the playbook. You need to change the value of the `firewall_rules` variable in the `deploy_apache.yml` playbook to a Jinja2 expression that uses the `template` lookup plug-in to dynamically generate the variable's setting from the `apache_firewall_rules.yml.j2` Jinja2 template. You must use the `from_yaml` filter in the Jinja2 expression to convert the resulting value from a text string into a YAML data structure that can be interpreted by Ansible.

   One correct solution results in the following content in the `deploy_apache.yml` Ansible Playbook:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
---
- name: Ensure Apache is deployed
  hosts: web_servers
  become: true
  force_handlers: true
  gather_facts: false

  roles:
    # Use the apache_firewall_rules.yml.j2 template to
    # generate the firewall rules.
    # Remove the two current list items from the firewall_rules
    # variable definition and set its value with a Jinja2
    # template of the form:
    # firewall_rules: "{{ plugin | filter }}"
    - role: apache
      firewall_rules: "{{ lookup('ansible.builtin.template',
'apache_firewall_rules.yml.j2') | from_yaml }}"
```

> **Note**
>
> In the preceding example, ensure you enter the expression for the `firewall_rules` variable on a single line.
>
> If you use linting rules that indicate that lines should not exceed 80 characters, then you might define the Jinja2 expression over multiple lines by defining the `firewall_rules` variable using a greater than symbol (`>`). For example:
>
> ```
>       firewall_rules: >
>         {{ lookup('ansible.builtin.template',
>  'apache_firewall_rules.yml.j2')
>         | from_yaml }}
> ```

5. Examine the `templates/apache_firewall_rules.yml.j2` template that your playbook now uses to set the `firewall_rules` variable.

   For the first rule, the `apache_port` variable is set by files in the `group_vars` directory.

   The template contains a Jinja2 `for` loop that creates a rule for each host in the `lb_servers` group, setting the source IP address from the value of the `load_balancer_ip_addr` variable.

   This is not the best solution. It requires you to set `load_balancer_ip_addr` as a host variable for each load balancer in the `lb_servers` group. To remove this manual maintenance, use gathered facts from these hosts to set that value instead.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Edit the `templates/apache_firewall_rules.yml.j2` template to replace the `load_balancer_ip_addr` host variable in the Jinja2 `for` loop with the `hostvars[server]['ansible_facts']['default_ipv4']['address']` fact.

The completed `templates/apache_firewall_rules.yml.j2` template should contain the following content:

```
- port: {{ apache_port }}
  protocol: TCP
  zone: internal
{% for server in groups['lb_servers'] %}
- zone: internal
  source: {{ hostvars[server]['ansible_facts']['default_ipv4']['address'] }}
{% endfor %}
```

Save the template.

6. Run the `site.yml` playbook to test your changes. If you successfully completed the preceding steps, then the playbook runs without errors.

```
[student@workstation data-review]$ ansible-navigator run site.yml
...output omitted...
```

7. Verify that web browser requests from `workstation` to the load balancer on `servera` succeed, and that direct requests from `workstation` to either TCP port 80 or TCP port 8008 on the back-end web servers (`serverb` and `serverc`) are denied.

```
[student@workstation data-review]$ curl servera; curl servera
This is serverb. (version v1.0)
This is serverc. (version v1.0)
[student@workstation data-review]$ curl serverb; curl serverb:8008
curl: (7) Failed to connect to serverb port 80: No route to host
curl: (7) Failed to connect to serverb port 8008: No route to host
[student@workstation data-review]$ curl serverc; curl serverc:8008
curl: (7) Failed to connect to serverc port 80: No route to host
curl: (7) Failed to connect to serverc port 8008: No route to host
```

8. Commit and push your changes to the remote Git repository. If prompted, use `Stud3nt123` as the Git password.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation data-review]$ git status
On branch exercise
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   deploy_apache.yml
        modified:   roles/firewall/tasks/main.yml
        modified:   templates/apache_firewall_rules.yml.j2

no changes added to commit (use "git add" and/or "git commit -a")

[student@workstation data-review]$ git add deploy_apache.yml \
roles/firewall/tasks/main.yml \
templates/apache_firewall_rules.yml.j2

[student@workstation data-review]$ git commit \
-m "Added Filters and Plug-ins"
...output omitted...

[student@workstation data-review]$ git push -u origin exercise
Password for 'https://student@git.lab.example.com': Stud3nt123
...output omitted...
```

Evaluation

As the `student` user on the `workstation` machine, use the `lab` command to grade your work. Correct any reported failures and rerun the command until successful.

```
[student@workstation ~]$ lab grade data-review
```

Finish

As the `student` user on the `workstation` machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish data-review
```

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

7.10. Summary

- You can use filters in Jinja2 expressions and templates to format, transform, and parse data.
- You can apply multiple filters sequentially in the same Jinja2 expression.
- Some filters are native to Jinja2 and some are provided by Ansible or by Ansible Content Collections.
- Lookup plug-ins get data from external sources for plays.
- Lookup plug-ins and filters run in the automation execution environment, not on the managed host.
- You can combine filters and lookup plug-ins to implement sophisticated loops.
- Ansible provides many supplementary filters for processing data related to networking, such as the `ansible.utils.ipaddr` filter.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Chapter 8.
Coordinating Rolling Updates

Goal

Use advanced features of Ansible to manage rolling updates in order to minimize downtime, and to ensure the maintainability and simplicity of Ansible Playbooks.

Sections

- Delegating Tasks and Facts (and Guided Exercise)
- Configuring Parallelism (and Guided Exercise)
- Managing Rolling Updates (and Guided Exercise)

Lab

- Coordinating Rolling Updates

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

8.1. Delegating Tasks and Facts

Objectives

- Run a task for a managed host on a different host, and control whether facts gathered by that task are delegated to the managed host or to the other host.

Delegating Tasks

Sometimes, when Ansible is running a play to ensure the correct configuration of a system, it might need to perform one or more tasks on another host on behalf of the managed host. For example, you might need to perform any of the following tasks:

- Log in to a network device to change a DHCP configuration.
- Make sure that certain groups exist in an Active Directory domain.
- Communicate with the API of a service by using tools that are not available on the managed host.

In a play, you can *delegate* a task to run on another host instead of the current managed host.

Use the `delegate_to` directive to delegate the action to a host. This directive points Ansible to the host that executes the task in place of the corresponding target. The host that you delegate the task to does not need to be listed in the Ansible inventory.

The following example runs the `uname -a` command on each host in the play, and then runs the `uname -a` command on `host.lab.example.com` on behalf of each host in the play:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
---
- name: Delegation Example
  hosts: demo.lab.example.com
  become: false

  tasks:
    - name: Get system information
      ansible.builtin.command: uname -a
      register: managed_host

    - name: Display demo system information
      ansible.builtin.debug:
        var: managed_host

    - name: Get system information
      ansible.builtin.command: uname -a
      delegate_to: host.lab.example.com
      register: delegated

    - name: Display localhost system information
      ansible.builtin.debug:
        var: delegated
```

In the following example, the first task is delegated to each of the HAProxy load balancers in the `lbservers` host group. Each load balancer removes the managed host from the pool of available web servers. The second task, which is not delegated, then stops the web server on the managed host. Both tasks run for each host in the play:

```
    - name: Remove the server from HAProxy
      community.general.haproxy:
        state: disabled
        host: "{{ ansible_facts['fqdn'] }}"
        socket: /var/lib/haproxy/stats
      delegate_to: "{{ item }}"
      loop: "{{ groups['lbservers'] }}"

    - name: Make sure Apache HTTPD is stopped
      ansible.builtin.service:
        name: httpd
        state: stopped
```

You might also delegate tasks to another host when you want to verify access to a service on the managed host that Ansible is currently processing. For example, you might want to ensure that communication from a test client can pass through a host-based firewall running on that host:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
    - name: Access the web service
      ansible.builtin.uri:
        url: http://demo.example.com
        timeout: 5
      delegate_to: test-client.example.com
```

Delegating to the Automation Execution Environment

One of the most common places you might delegate a task is to the automation execution environment, or `localhost` environment. For example, if your machine cannot reach a managed host, but the automation execution environment can reach the host, then you can delegate the task as follows:

```
    - name: Get information about controller instance
      ansible.builtin.uri:
        url: https://aap.example.com/api/controller/v2/ping/
        method: GET
        validate_certs: false
        return_content: true
      delegate_to: localhost
      register: controller_ping
```

Important

The introduction of automation execution environments in Red Hat Ansible Automation Platform 2 changes how delegation to `localhost` works. With the `ansible-navigator` command, if you delegate a task to run on `localhost`, then the module runs in the automation execution environment container. In earlier versions that used the `ansible-playbook` command, the module ran directly on the control node, with various consequences.

Privilege escalation might fail if the `sudo` command is not present. The supported automation execution environments provided by Red Hat require the `sudo` command.

If you are reusing a playbook that assumes it can access resources or commands on the control node, those resources might not be available inside the automation execution environment container.

Delegating Facts

In an earlier example, the `ansible_facts['fqdn']` fact was used in a task that was delegated to each of the load balancers. In that task, the fully qualified domain name (FQDN) for the managed host is used, not that of the current load balancer.

When you delegate a task, Ansible uses the host variables and facts for the managed host (the current `inventory_hostname`) for which the task is running. For example, if a task is running for the `demo` host, but has been delegated to the `client` host, then the variables and facts for the `demo` host are used. Usually, even when the task is delegated to another host, it is correct to use the variables and facts for the managed host.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

However, sometimes you might want to assign the facts collected by a delegated task to the host to which the task was delegated. To change this setting, set the `delegate_facts` directive to `true`:

```
- name: Delegate Fact Example
  hosts: localhost
  gather_facts: false
  tasks:
    - name: Set a fact in delegated task on demo
      ansible.builtin.set_fact:
        myfact: Where am I set?
      delegate_to: demo.lab.example.com
      delegate_facts: true

    - name: Display the facts from demo.lab.example.com
      ansible.builtin.debug:
        msg: "{{ hostvars['demo.lab.example.com']['myfact'] }}"
```

The preceding play runs for `localhost`, but the first task is delegated to `demo.lab.example.com`. Using the `delegate_facts` directive on that task instructs Ansible to gather facts into the `hostvars['demo.lab.example.com']` namespace for the delegated host, instead of the default `hostvars['localhost']` namespace for the current managed host.

> **References**
> Controlling Where Tasks Run: Delegation and Local Actions - Ansible Documentation

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

8.2. Guided Exercise
Delegating Tasks and Facts

Run a playbook that includes a task that is delegated to another host.

Outcomes

- Delegate a task to run on another host.

Prerequisites

As the `student` user on the `workstation` machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
[student@workstation ~]$ lab start update-delegation
```

Instructions

1. Clone the `https://git.lab.example.com/student/update-delegation.git` repository to the `/home/student/git-repos` directory and then create a branch for this exercise.

   1.1. From a terminal, create the `/home/student/git-repos` directory if it does not already exist, and then change into it:

   ```
   [student@workstation ~]$ mkdir -p ~/git-repos/
   [student@workstation ~]$ cd ~/git-repos/
   [student@workstation git-repos]$
   ```

   1.2. Clone the `https://git.lab.example.com/student/update-delegation.git` repository and then change to the cloned repository:

   ```
   [student@workstation git-repos]$ git clone \
   https://git.lab.example.com/student/update-delegation.git
   Cloning into 'update-delegation'...
   ...output omitted...
   [student@workstation git-repos]$ cd update-delegation
   [student@workstation update-delegation]$
   ```

   1.3. Create the `exercise` branch and check it out:

   ```
   [student@workstation update-delegation]$ git checkout -b exercise
   Switched to a new branch 'exercise'
   ```

2. Get familiar with the Ansible project. Review the inventory and the playbook in the project directory. Complete the playbook to perform the following actions:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

- Query the time on each server in the `web_servers` host group.
- Store the results in the `/tmp/times.txt` file on the `workstation` machine.

2.1. List the files in the project directory. Identify the inventory and the playbook files:

```
[student@workstation update-delegation]$ ls
ansible-navigator.yml  ansible.cfg  inventory.yml query_times.yml
```

2.2. Display the contents of the `inventory.yml` file. Notice that the `web_servers` host group contains servers from `servera.lab.example.com` to `serverd.lab.example.com`:

```
---
web_servers:
  hosts:
    server[a:d].lab.example.com:
```

2.3. Add a task to the `query_times.yml` playbook to create a file on the `workstation` machine for each host in the `web_servers` host group. Each file must contain two lines of text and the file name must use the `ansible_facts['hostname']` fact.

The first line in that file must contain the name of a web server. The second line must contain the time on the managed host.

For both lines, use facts gathered from the `web_servers` host group.

Create each file on the `workstation` machine by delegating this task:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
---
- name: Query server times and store them locally
  hosts: web_servers
  gather_facts: true
  become: true
  tasks:
    - name: Save server times to workstation
      ansible.builtin.copy:
        content: |
          {{ ansible_facts['fqdn'] }}
          {{ '%c' | ansible.builtin.strftime }}
        dest: /tmp/times-{{ ansible_facts['hostname'] }}.txt
        owner: root
        group: root
        mode: "0644"
      delegate_to: workstation
```

> **Note**
>
> You can use the `ansible.builtin.strftime` filter to display time in a particular format by passing a format to the filter. Consult the `date(1)` man page for available formats.

3. Run the `query_times.yml` playbook to gather the server times. Confirm that Ansible stored the information on the `workstation` machine.

   3.1. Use the `ansible-navigator run` command to run the `query_times.yml` playbook:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation update-delegation]$ ansible-navigator run
query_times.yml

PLAY [Query server times and store them locally]
*******************************

TASK [Gathering Facts]
*********************************************************
ok: [servera.lab.example.com]
ok: [serverb.lab.example.com]
ok: [serverc.lab.example.com]
ok: [serverd.lab.example.com]

TASK [Save server times to workstation]
************************************
changed: [servera.lab.example.com -> workstation]
changed: [serverb.lab.example.com -> workstation]
changed: [serverc.lab.example.com -> workstation]
changed: [serverd.lab.example.com -> workstation]

PLAY RECAP
*********************************************************************
servera.lab.example.com    : ok=2    changed=1    unreachable=0    failed=0
...
serverb.lab.example.com    : ok=2    changed=1    unreachable=0    failed=0
...
serverc.lab.example.com    : ok=2    changed=1    unreachable=0    failed=0
...
serverd.lab.example.com    : ok=2    changed=1    unreachable=0    failed=0
...
```

3.2. Verify that the playbook worked correctly by listing the created files and then displaying the content of each file:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation update-delegation]$ ls -1 /tmp/times-*.txt
/tmp/times-servera.txt
/tmp/times-serverb.txt
/tmp/times-serverc.txt
/tmp/times-serverd.txt

[student@workstation update-delegation]$ cat /tmp/times-*.txt
servera.lab.example.com
Wed May 15 02:16:11 2025
serverb.lab.example.com
Wed May 15 02:16:11 2025
serverc.lab.example.com
Wed May 15 02:16:11 2025
serverd.lab.example.com
Wed May 15 02:16:10 2025
```

4. Push your changes to the `exercise` branch of the remote Git repository.

   4.1. Stage the modified `query_times.yml` playbook and commit to the local Git repository:

```
[student@workstation update-delegation]$ git add query_times.yml
[student@workstation update-delegation]$ git commit -m 'Changes for
exercise'
...output omitted...
```

   4.2. Push the changes to the `exercise` branch on the remote Git repository. If prompted, use `Stud3nt123` as the Git password:

```
[student@workstation update-delegation]$ git push -u origin exercise
Password for 'https://student@git.lab.example.com': Stud3nt123
...output omitted...
```

Finish

On the `workstation` machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish update-delegation
```


For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

8.3. Configuring Parallelism

Objectives

- Tune the number of simultaneous connections that Ansible opens to managed hosts and how Ansible processes groups of managed hosts through the play's tasks.

Configure Parallelism in Ansible Using Forks

When Ansible processes a playbook, it runs each play in order. After determining the list of hosts for the play, Ansible runs through each task in order. Normally, all hosts must complete a task before any host starts the next task in the play.

In theory, Ansible could connect to all hosts in the play simultaneously for each task. This approach works fine for small lists of hosts. If the play targets hundreds of hosts, then this approach can put a heavy load on the control node and the automation execution environment.

The maximum number of simultaneous connections that Ansible makes is controlled by the `forks` parameter in the Ansible configuration file. This parameter is set to 5 by default, which you can verify by using one of the following methods:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[user@host demo]$ ansible-navigator config dump -m stdout
...output omitted...
DEFAULT_FORCE_HANDLERS(default) = False
DEFAULT_FORKS(default) = 5
DEFAULT_GATHERING(default) = implicit
...output omitted...
```

```
[user@host demo]$ ansible-navigator config list -m stdout
...output omitted...
DEFAULT_FORKS:
  default: 5
  description: Maximum number of forks Ansible will use to execute tasks on target
    hosts.
  env:
  - name: ANSIBLE_FORKS
  ini:
  - key: forks
    section: defaults
  name: Number of task forks
  type: integer
...output omitted...
```

Note

You can search the output of the previous two commands. To search for the default `forks` parameter, type `/DEFAULT_FORKS` and press `Enter`.

For example, assume that Ansible is configured with the default value of five forks and that the play has ten managed hosts. Ansible runs the first task in the play on the first five managed hosts, followed by a second round of execution of the first task on the next five managed hosts. After running the first task on all the managed hosts, Ansible runs the next task across all the managed hosts in groups of five hosts at a time. Ansible does this with each task in turn until the play finishes.

The default setting for `forks` is conservative. If a play that is running in your automation execution environment manages Linux hosts, most tasks run on the managed hosts, and the automation execution environment has less load. In this case, you can usually set the `forks` parameter to a much higher value, possibly closer to `100`, and realize performance improvements.

If your plays run a lot of code in the automation execution environment, then you should raise the fork limit judiciously. If you use Ansible to manage network routers and switches, then most of those modules run in the automation execution environment and not on the network device. Because of the increased load this places on the automation execution environment, the number of forks should be significantly lower compared to a play running in an execution environment that manages only Linux hosts.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

You can override the default setting for the `forks` option from the command line. The `ansible-navigator run` command offers the `-f` or `--forks` option to specify the number of forks.

Running Batches of Hosts Through the Entire Play

Normally, when Ansible runs a play, it ensures that all managed hosts complete each task before it starts the next task. Any notified handlers are run after all managed hosts have completed all tasks.

However, running all tasks on all hosts can lead to undesirable behavior. For example, if a play updates a cluster of load-balanced web servers, it might need to take each web server out of service during the update. If all the servers are updated in the same play, they could all be out of service simultaneously.

One way to avoid this problem is to use the `serial` keyword to run the hosts through the play in batches. The play processes each batch of hosts in sequence.

In the following example, Ansible runs the play on two managed hosts simultaneously until all managed hosts have been updated. Ansible first runs the tasks in the play on the first two managed hosts. If any of those hosts notifies a handler, then Ansible runs the handler. When the play is complete on those two managed hosts, Ansible repeats the process on the next two managed hosts. Ansible continues to run the play in this way until all managed hosts are updated.

```
---
- name: Rolling update
  hosts: webservers
  serial: 2
  tasks:
    - name: Latest apache httpd package is installed
      ansible.builtin.dnf:
        name: httpd
        state: latest
      notify: Restart apache

  handlers:
    - name: Restart apache
      ansible.builtin.service:
        name: httpd
        state: restarted
```

Suppose that the `webservers` group in the previous example contains five web servers behind a load balancer. With the `serial` parameter set to 2, the play runs on up to two web servers simultaneously. Thus, a majority of the five web servers are always available.

In contrast, in the absence of the `serial` keyword, the play and resulting handler would run across all five web servers at the same time. This approach would probably lead to a service outage because web services on all the web servers would be restarted at the same time.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Important

For certain purposes, each batch of hosts counts as if it was a full play running on a subset of hosts. This means that if an entire batch fails, then the play fails, which causes the entire playbook run to fail.

In the previous scenario with the `serial` parameter set to 2, if something is wrong and the play fails for the first two processed hosts, then the playbook aborts and the play does not run on the remaining three hosts. This is a useful feature because only a subset of the servers would be unavailable, leaving the service degraded rather than down.

The `serial` keyword can also be specified as a percentage. This percentage is applied to the total number of hosts in the play to determine the rolling update batch size. Regardless of the percentage, the number of hosts per pass is always one or greater.

References

Setting the Batch Size with Serial - Controlling Playbook Execution: Strategies and More - Ansible Documentation

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

8.4. Guided Exercise
Configuring Parallelism

Explore the effects of different `serial` and `forks` directives on how Ansible processes a play.

Outcomes

- Tune parallel and serial executions of a playbook across multiple managed hosts.

Prerequisites

As the `student` user on the `workstation` machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
[student@workstation ~]$ lab start update-parallelism
```

Instructions

1. Clone the `https://git.lab.example.com/student/update-parallelism.git` repository to the `/home/student/git-repos` directory and then create a branch for this exercise.

   1.1. Create the `/home/student/git-repos` directory if it does not already exist, and then change into it:

   ```
   [student@workstation ~]$ mkdir -p ~/git-repos
   [student@workstation ~]$ cd ~/git-repos
   [student@workstation git-repos]$
   ```

   1.2. Clone the `https://git.lab.example.com/student/update-parallelism.git` repository and then change directory to the cloned repository:

   ```
   [student@workstation git-repos]$ git clone \
   https://git.lab.example.com/student/update-parallelism.git
   Cloning into 'update-parallelism'...
   ...output omitted...
   [student@workstation git-repos]$ cd update-parallelism
   [student@workstation update-parallelism]$
   ```

   1.3. Create the `exercise` branch and check it out:

   ```
   [student@workstation update-parallelism]$ git checkout -b exercise
   Switched to a new branch 'exercise'
   ```

2. Examine the contents of the project directory to become familiar with the project files.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

2.1. Examine the contents of the `ansible.cfg` file. Note that the inventory file is set to `inventory`, the `forks` parameter is set to 4, and the `ansible.posix.timer` and `ansible.posix.profile_tasks` callback plug-ins are enabled:

```
[defaults]
inventory=inventory
remote_user=devops
forks=4
callbacks_enabled=ansible.posix.timer,ansible.posix.profile_tasks
```

2.2. Examine the contents of the `inventory` file. Note that it contains a host group, `webservers`, which contains four hosts:

```
[webservers]
servera.lab.example.com
serverb.lab.example.com
serverc.lab.example.com
serverd.lab.example.com
```

2.3. Examine the contents of the `update_httpd.yml` playbook. This playbook runs against the `webservers` host group, ensuring that the latest `httpd` package is installed and that the `httpd` service is enabled and started.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
---
- name: Update the web server
  hosts: webservers
  become: true
  gather_facts: false

  tasks:
    - name: Latest httpd package installed
      ansible.builtin.dnf:
        name: httpd
        state: latest
      notify: Restart httpd

  handlers:
    - name: Restart httpd
      ansible.builtin.service:
        name: httpd
        enabled: true
        state: restarted
```

2.4. Examine the contents of the `remove_httpd.yml` playbook. This playbook runs against the `webservers` host group, ensuring that the `httpd` service is disabled and stopped and that the `httpd` package is not installed.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
---
- name: Remove the web server
  hosts: webservers
  become: true
  gather_facts: false

  tasks:
    - name: Query service facts
      ansible.builtin.service_facts:

    - name: Disable httpd service
      ansible.builtin.service:
        name: httpd
        enabled: false
        state: stopped
      when: ansible_facts['services']['httpd.service'] is defined

    - name: Remove httpd package
      ansible.builtin.dnf:
        name: httpd
        state: absent
        autoremove: true
```

3. Run the `update_httpd.yml` playbook and then the `remove_httpd.yml` playbook.

   3.1. Run the `update_httpd.yml` playbook and note that Ansible performs each task on all four hosts at the same time. Also, note the output from the callback plug-ins to determine how long it takes the `ansible-navigator` command to run the playbook:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation update-parallelism]$ ansible-navigator run
update_httpd.yml

PLAY [Update the web server]
*************************************************

...output omitted...

PLAY RECAP
*********************************************************************
servera.lab.example.com    : ok=2    changed=2    unreachable=0    failed=0
...
serverb.lab.example.com    : ok=2    changed=2    unreachable=0    failed=0
...
serverc.lab.example.com    : ok=2    changed=2    unreachable=0    failed=0
...
serverd.lab.example.com    : ok=2    changed=2    unreachable=0    failed=0
...
Playbook run took 0 days, 0 hours, 0 minutes, 8 seconds
Thursday 15 May 2025  10:34:30 +0000 (0:00:01.342)       0:00:08.421
**********
=========================================================================
===
Latest httpd package installed ----------------------------------------
7.06s
Restart httpd ---------------------------------------------------------
1.34s
```

3.2. Run the `remove_httpd.yml` playbook to stop and disable the `httpd` service and to remove the `httpd` package:

```
[student@workstation update-parallelism]$ ansible-navigator run
remove_httpd.yml
...output omitted...
```

4. Change the value of the `forks` parameter to 1 in the `ansible.cfg` file, and then time the execution of the `update_httpd.yml` playbook again.

4.1. Change the value of the `forks` parameter to 1 in the `ansible.cfg` file:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[defaults]
inventory=inventory
remote_user=devops
forks=1
callbacks_enabled=ansible.posix.timer,ansible.posix.profile_tasks
```

4.2. Run the `update_httpd.yml` playbook again. Watch the playbook as it runs. This time Ansible performs each task on one host at a time. Notice how decreasing the number of forks causes the playbook execution to take longer:

```
[student@workstation update-parallelism]$ ansible-navigator run
update_httpd.yml

PLAY [Update the web server]
*************************************************

...output omitted...

PLAY RECAP
*********************************************************************
servera.lab.example.com    : ok=2    changed=2    unreachable=0    failed=0
...
serverb.lab.example.com    : ok=2    changed=2    unreachable=0    failed=0
...
serverc.lab.example.com    : ok=2    changed=2    unreachable=0    failed=0
...
serverd.lab.example.com    : ok=2    changed=2    unreachable=0    failed=0
...
Playbook run took 0 days, 0 hours, 0 minutes, 23 seconds
Thursday 15 May 2025  10:42:36 +0000 (0:00:04.449)       0:00:23.075
**********
=========================================================================
===
Latest httpd package installed ----------------------------------------
18.61s
Restart httpd ---------------------------------------------------------
4.45s
```

4.3. Run the `remove_httpd.yml` playbook to stop and disable the `httpd` service and to remove the `httpd` package:

```
[student@workstation update-parallelism]$ ansible-navigator run
remove_httpd.yml
...output omitted...
```

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

5. Add the `serial` parameter to the play in the `update_httpd.yml` playbooks so that the play only runs on two hosts simultaneously. The beginning of the playbook should consist of the following content:

```
---
- name: Update the web server
  hosts: webservers
  become: true
  gather_facts: false
  serial: 2
...output omitted...
```

6. Set the value of the `forks` parameter to 2 in the `ansible.cfg` file. The `forks` parameter must be at least equal to the value for of the `serial` parameter; otherwise, it restricts execution speed.

```
[defaults]
inventory=inventory
remote_user=devops
forks=2
callbacks_enabled=ansible.posix.timer,ansible.posix.profile_tasks
```

7. Run the `update_httpd.yml` playbook and then the `remove_httpd.yml` playbook.

   7.1. Run the `update_httpd.yml` playbook and note that Ansible runs through the entire play on two hosts before rerunning the play on the two remaining hosts:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation update-parallelism]$ ansible-navigator run
update_httpd.yml

PLAY [Update the web server]
*************************************************

TASK [Latest httpd package installed]
******************************************
Thursday 15 May 2025  10:48:28 +0000 (0:00:00.012)       0:00:00.012
**********
changed: [servera.lab.example.com]
changed: [serverb.lab.example.com]

...output omitted...

TASK [Latest httpd package installed]
******************************************
Thursday 15 May 2025  10:48:34 +0000 (0:00:01.280)       0:00:06.262
**********
changed: [serverd.lab.example.com]
changed: [serverc.lab.example.com]

...output omitted...
```

7.2. Run the `remove_httpd.yml` playbook to stop and disable the `httpd` service and to remove the `httpd` package:

```
[student@workstation update-parallelism]$ ansible-navigator run
remove_httpd.yml
...output omitted...
```

8. Set the value of the `forks` parameter back to 4 in the `ansible.cfg` file:

```
[defaults]
inventory=inventory
remote_user=devops
forks=4
callbacks_enabled=ansible.posix.timer,ansible.posix.profile_tasks
```

9. Set the `serial` parameter in the `update_httpd.yml` playbook to 3. The beginning of the playbook should consist of the following content:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
---
- name: Update the web server
  hosts: webservers
  become: true
  gather_facts: false
  serial: 3
...output omitted...
```

10. Run the `update_httpd.yml` playbook again, and watch as it runs. Note that Ansible runs through the entire play on three hosts, and then runs the play on the one remaining host:

```
[student@workstation update-parallelism]$ ansible-navigator run update_httpd.yml

PLAY [Update the web server] ***************************************************

TASK [Latest httpd package installed] ******************************************
Thursday 15 May 2025  10:51:58 +0000 (0:00:00.013)       0:00:00.013 **********
changed: [servera.lab.example.com]
changed: [serverc.lab.example.com]
changed: [serverb.lab.example.com]

...output omitted...

TASK [Latest httpd package installed] ******************************************
Thursday 15 May 2025  10:52:04 +0000 (0:00:01.487)       0:00:06.849 **********
changed: [serverd.lab.example.com]

...output omitted...
```

Finish

On the `workstation` machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish update-parallelism
```

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

8.5. Managing Rolling Updates

Objectives

- Tune the behavior of the `serial` directive when batching hosts for execution, abort the play if it fails for too many hosts, and create tasks that run only once for each batch or for all hosts in the inventory.

Overview

Ansible has several features that enable *rolling updates*, a strategy that staggers deployments to batches of servers. With this strategy, infrastructure deployment can complete with zero downtime.

Ansible can halt the deployment and limit the errors to servers in a particular batch when an unforeseen problem occurs. With tests and monitoring in place, you can configure playbook tasks to perform the following actions:

- Roll back configuration for hosts in the affected batch.
- Quarantine affected hosts to enable analysis of the failed deployment.
- Send notifications of the deployment to stakeholders.

Controlling Batch Size

By default, Ansible runs a play by running one task for all hosts before running the next task. Imagine that your play has a task that cannot succeed due to some error. If your play runs on all hosts in parallel, they all fail and abort the play when Ansible reaches that task, and no host completes the play. As a result, it is likely that none of the hosts are working correctly, which could lead to an outage.

To avoid this, you could process *some* hosts through the play before starting the next batch of hosts. If too many hosts fail, then you could have Ansible abort the entire play before all your hosts attempt to run it.

The following sections describe how to configure your play to abort if too many hosts in a batch fail.

Setting a Fixed Batch Size

To process hosts through a play in batches, use the `serial` directive in your play. The `serial` directive specifies how many hosts should be in each batch. Ansible processes each batch of hosts through the play before starting the next batch. If all hosts in the current batch fail, the entire play is aborted, and Ansible does not start the next batch.

Consider the following lines in the first part of a play:

```
---
- name: Update Webservers
  hosts: web_servers
  serial: 2
```

In this example, the `serial` directive instructs Ansible to process hosts in the `web_servers` host group in batches of two hosts. The play is repeated with a new batch if the play runs without error.

This process continues until all the hosts in the play are processed. If the total number of hosts in the play is not divisible by the batch size, then the last batch might contain fewer hosts than the indicated value of the `serial` directive. In the previous example, the last batch contains one host if the total number of web servers is an odd value.

Remember, if you use an integer with the `serial` directive, then as the number of hosts in the play increases, the number of batches needed to complete the play also increases. With a `serial` value of 2, a host group with 200 hosts requires 100 batches to complete, but a host group with 20 hosts requires ten batches.

Setting Batch Size as a Percentage

You can also specify a percentage for the value of the `serial` directive:

```
---
- name: Update Webservers
  hosts: web_servers
  serial: 25%
```

The play runs on the percentage of the hosts that you specify.

Ansible applies the percentage to the total number of hosts in the host group. If the resulting value is not an integer number of hosts, then the value is truncated (rounded down) to the nearest integer. The batch size cannot be zero hosts. If the truncated value is zero, Ansible changes the batch size to one host.

The following table illustrates the use of a percentage and the resulting size and number of batches:

Description	Host group 1	Host group 2	Host group 3
Total number of hosts	3	13	19
`serial` value	25%	25%	25%
Exact number of hosts	0.75	3.25	4.75
Rounded down	0	3	4
Resulting batch size	1	3	4
Number of batches	3	5	5
Size of last batch	1	1	3

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Setting Batch Sizes to Change During the Play

You can change the batch size as the play runs. For example, you could test a play on a batch of one host; if that host fails, the entire play aborts. However, if the play is successful on one host, you could increase the batch size to 10 percent of your hosts, then 50 percent of the managed hosts, and then the remainder.

You can gradually change the batch size by setting the `serial` directive to a list of values. This list can contain any combination of integers and percentages and resets the size of each batch in sequence. If the value is a percentage, then Ansible computes the batch size based on the total size of the host group and not the size of the group of unprocessed hosts.

Consider the following example:

```
---
- name: Update Webservers
  hosts: web_servers
  serial:
    - 1
    - 10%
    - 100%
```

The first batch contains a single host.

The second batch contains 10% of the total hosts in the `web_servers` host group. Ansible computes the actual value according to the previously discussed rules.

The third batch contains all the remaining unprocessed hosts in the play. This setting enables Ansible to process all the remaining hosts efficiently.

If unprocessed hosts remain after the last batch corresponding to the previous `serial` directive entry, the last batch repeats until all hosts are processed. Consider the following play, which runs against a `web_servers` host group with 100 hosts:

```
---
- name: Update Webservers
  hosts: web_servers
  serial:
    - 1
    - 10%
    - 25%
```

The first batch contains one host and the second batch contains ten hosts (10% of 100). The third batch processes 25 hosts (25% of 100), leaving 64 unprocessed hosts (1 + 10 + 25 processed). Ansible continues executing the play in batch sizes of 25 hosts (25% of 100) until fewer than 25 unprocessed hosts remain. In this example, the final batch processes the remaining 14 hosts (1 + 10 + 25 + 25 + 25 + 14 = 100).

Aborting the Play

By default, Ansible tries to get as many hosts to complete a play as possible. If a task fails for a host, the host is dropped from the play, but Ansible continues to run the remaining tasks for other hosts. The play only stops if all hosts fail.

However, if you use the `serial` directive to organize hosts into batches, then if all hosts in the current batch fail, Ansible stops the play for *all remaining hosts*, not just the remaining hosts in the current batch. If all hosts in a batch fail, the play aborts and the next batch does not start.

Ansible keeps a list of the active servers for each batch in the `ansible_play_batch` variable. Ansible removes a host that fails a task from the `ansible_play_batch` list, and updates this list after every task.

Consider the following hypothetical playbook, which runs against a `web_servers` host group with 100 hosts:

```
---
- name: Update Webservers
  hosts: web_servers
  tasks:
    - name: Step One
      ansible.builtin.command: /usr/bin/some_command

    - name: Step Two
      ansible.builtin.command: /usr/bin/some_other_command
```

If 99 web servers fail the first task, but one host succeeds, Ansible continues to the second task. When Ansible runs the second task, Ansible only runs that task for the one host that previously succeeded.

If you use the `serial` directive, playbook execution continues, provided that hosts remain in the current batch with no failures. Consider this modification to the hypothetical playbook:

```
---
- name: Update Webservers
  hosts: web_servers
  serial: 2
  tasks:
    - name: Step One
      ansible.builtin.command: /usr/bin/some_command

    - name: Step Two
      ansible.builtin.command: /usr/bin/some_other_command
```

If the first batch of two contains a host that succeeds and a host that fails, then the batch completes, and Ansible moves on to the second batch of two. If both hosts in the second batch fail on a task in the play, Ansible aborts the entire play and does not start any more batches.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

In this scenario, running the playbook can produce the following states:

- One host completes the play.
- Three hosts are in an error state.
- The rest of the hosts remain unaltered.

Specifying Failure Tolerance

By default, Ansible only halts play execution when all hosts in a batch fail. However, you might want a play to abort if more than a certain percentage of hosts in the inventory fail, even if no entire batch fails. You can also "fail fast" and abort the play if any tasks fail.

You can add the `max_fail_percentage` directive to a play to alter the default Ansible failure behavior. When the number of failed hosts in a batch exceeds this percentage, Ansible halts playbook execution.

Consider the following hypothetical playbook that runs against the `web_servers` host group, which contains 100 hosts:

```
---
- name: Update Webservers
  hosts: web_servers
  max_fail_percentage: 30%
  serial:
    - 2
    - 10%
    - 100%
  tasks:
    - name: Step One
      ansible.builtin.command: /usr/bin/some_command

    - name: Step Two
      ansible.builtin.command: /usr/bin/some_other_command
```

The first batch contains two hosts. Because 30% of 2 is 0.6, a single host failure causes execution to stop.

If both hosts in the first batch succeed, then Ansible continues with the second batch of 10 hosts. Because 30% of 10 is 3, more than three host failures must occur for Ansible to stop playbook execution. If three or fewer hosts experience errors in the second batch, Ansible continues with the third batch.

To implement a "fail fast" strategy, set the `max_fail_percentage` to zero.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Important

The Ansible failure behavior can be summarized as follows:

- If the `serial` directive and the `max_fail_percentage` values are not defined, all hosts are run through the play in one batch. If all hosts fail, then the play fails.
- If the `serial` directive is defined, then hosts are run through the play in multiple batches, and the play fails if all hosts in any one batch fail.
- If the `max_fail_percentage` directive is defined, the play fails if more than that percentage of hosts in a batch fail.

If a play fails, Ansible aborts all remaining plays in the playbook.

Throttling Task Execution

Throttling allows you to limit the number of workers for a particular task. This is especially helpful when performing CPU-heavy tasks or when interacting with APIs that have strict rate limits. To use throttling, add the `throttle` keyword and specify the number of workers as the parameter. This can be applied at either the task or block level:

```
- name: Run CPU-intensive command one host at a time
  ansible.builtin.command: /path/to/cpu_intensive_command
  throttle: 1
```

Note

Note that throttling can only reduce concurrency; it cannot exceed limits already set by the `forks` or `serial` directives.

Controlling Host Execution Order

Use the `order` keyword to define the sequence in which hosts are targeted:

`inventory`
: (default): Sorts the hosts based on the inventory order.

`reverse_inventory`
: Sorts the hosts based on the reverse of the inventory order.

`sorted`
: Sorts the hosts in alphabetical order. Numbers sort before letters.

`reverse_sorted`
: Sorts the hosts in reverse alphabetical order.

`shuffle`
: Randomizes the host list every time you run the play.

Running a Task Once

In certain scenarios, you might only need to run a task once for an entire batch of hosts, rather than once for each host in the batch. To accommodate this scenario, add the `run_once` directive to the task with a Boolean `true` (or `yes`) value.

Consider the following hypothetical task:

```
- name: Reactivate Hosts
  ansible.builtin.command: /sbin/activate.sh {{ active_hosts_string }}
  run_once: true
  delegate_to: monitor.example.com
  vars:
    active_hosts_string: "{{ ansible_play_batch | join(' ')}}"
```

This task runs once and runs on the `monitor.example.com` host. The task uses the `active_hosts_string` variable to pass a list of active hosts as command-line arguments to an activation script. The variable contains only those hosts in the current batch that have succeeded for all previous tasks.

Important

Setting the `run_once: true` directive causes a task to run once for each batch. If you only need to run a task once for all hosts in a play, and the play has multiple batches, then you can add the following conditional statement to the task:

```
when: inventory_hostname == ansible_play_hosts[0]
```

This conditional statement only runs the task for the first host in the play.

References

Controlling Playbook Execution: Strategies and More - Ansible Documentation

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

8.6. Guided Exercise
Managing Rolling Updates

Run a playbook that uses unequal batch sizes with the `serial` directive so that the play aborts if too many hosts fail.

Outcomes

- Control the update process of an existing HAProxy cluster by using the `serial` directive to set the batch size.

Prerequisites

As the `student` user on the `workstation` machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
[student@workstation ~]$ lab start update-management
```

Instructions

1. Get familiar with the Ansible project. The `site.yml` playbook configures a front-end load balancer and a pool of back-end web servers. Identify the message displayed on each web server.

   1.1. Change to the `/home/student/management` directory and list the files in that directory:

   ```
   [student@workstation ~]$ cd ~/management
   [student@workstation management]$ ls
   ansible-navigator.yml  deploy_apache.yml   group_vars         roles
   ansible.cfg            deploy_haproxy.yml  inventory          site.yml
   collections            deploy_webapp.yml   issue_requests.sh
   update_webapp.yml
   ```

   1.2. Review the `site.yml` playbook. This playbook imports the `deploy_*.yml` playbooks in the project directory.

   ```
   ---
   - name: Deploy HAProxy
     ansible.builtin.import_playbook: deploy_haproxy.yml

   - name: Deploy Web Server
     ansible.builtin.import_playbook: deploy_apache.yml

   - name: Deploy Web App
     ansible.builtin.import_playbook: deploy_webapp.yml
   ```

   1.3. Review the `deploy_webapp.yml` playbook. This playbook targets the `web_servers` host group in its single play and uses the webapp role to ensure that the web content is deployed:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
---
- name: Deploying Web App
  hosts: web_servers
  become: true
  gather_facts: true
  tasks:
    - name: Ensure web content is deployed
      ansible.builtin.include_role:
        name: webapp
```

1.4. Review the `roles/webapp/tasks/main.yml` file. The `index.html` file for each web server is created using the `index.html.j2` template:

```
---
# tasks file for webapp

- name: Ensure directory exists
  ansible.builtin.file:
    path: "{{ webapp_content_dir }}"
    state: directory
    owner: root
    group: root
    mode: "0755"
    setype: _default

- name: Ensure web content is deployed
  ansible.builtin.template:
    src: index.html.j2
    dest: "{{ webapp_content_dir }}/index.html"
    owner: root
    group: root
    mode: "0644"
    setype: _default
```

1.5. Display the content of the `roles/webapp/templates/index.html.j2` file. Each web server displays its hostname and the path to the `index.html` file:

```
{{ ansible_facts['hostname'] }}: {{ webapp_content_dir }}/index.html
--
```

1.6. Review the content of the inventory file. Notice that the `web_servers` host group contains servers from `serverb.lab.example.com` to `serverf.lab.example.com`:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[lb_servers]
servera.lab.example.com

[web_servers]
server[b:f].lab.example.com
```

2. Run the `site.yml` playbook and confirm that the web deployment is successful.

   2.1. Use the `ansible-navigator run` command to run the `site.yml` playbook:

```
[student@workstation management]$ ansible-navigator run site.yml

...output omitted...

PLAY RECAP
*********************************************************************
servera.lab.example.com   : ok=7    changed=6  ...  failed=0    skipped=4
...
serverb.lab.example.com   : ok=14   changed=8  ...  failed=0    skipped=2
...
serverc.lab.example.com   : ok=14   changed=8  ...  failed=0    skipped=2
...
serverd.lab.example.com   : ok=14   changed=8  ...  failed=0    skipped=2
...
servere.lab.example.com   : ok=14   changed=8  ...  failed=0    skipped=2
...
serverf.lab.example.com   : ok=14   changed=8  ...  failed=0    skipped=2
...
```

   2.2. Use the `curl` command to make five web requests to `servera`:

```
[student@workstation management]$ for x in {1..5}; do curl servera; done
serverb: /var/www/html/index.html
--
serverc: /var/www/html/index.html
--
serverd: /var/www/html/index.html
--
servere: /var/www/html/index.html
--
serverf: /var/www/html/index.html
--
```

   The load balancer distributes requests to the five back-end web servers that serve content from the `/var/www/html` directory.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

> **Note**
> The order of the servers might be different from the order displayed throughout this example.

3. Install the content collection specified in the `collections/requirements.yml` file into the `collections` directory.

   3.1. Review the `collections/requirements.yml` file. The project requires the `community.general` collection.

   ```
   ---
   collections:
     - name: community.general
   ```

   3.2. Go to `https://aap.lab.example.com` and log in to Ansible Automation Platform as `student` using `student` as the password.

   3.3. Go to **Automation Content > API Token** and then click **Generate token**.

   3.4. Copy the generated token.

   3.5. Edit the `~/management/ansible.cfg` file to paste the token as the value for the `token` parameter:

   ```
   [defaults]
   inventory = inventory
   remote_user = devops
   collections_path = ./collections:/usr/share/ansible/collections

   [galaxy]
   server_list = community

   [galaxy_server.community]
   url=https://aap.lab.example.com/api/galaxy/content/community/
   token=28c2dbdad1ac0414b99f574aacd3287f185c38d6
   ```

   Your token is different from the one displayed in this example.

   3.6. Use the `ansible-galaxy collection install` command to install the collection:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation management]$ ansible-galaxy collection \
install -r collections/requirements.yml -p collections/
...output omitted...
community.general:10.3.0 was installed successfully
[student@workstation management]$
```

4. The `update_webapp.yml` playbook performs a rolling update of the web application hosted on the back-end web servers. Review the playbook.

   To prevent your hosts from becoming inoperable when running the `update_webapp.yml` playbook, use the `serial` directive to update the playbook as follows:

   - Ansible must run the first play of the `update_webapp.yml` playbook on a single host. If the run fails, then the play must abort.
   - If the run succeeds, then Ansible must run the play on 25% of the hosts.
   - If the run on 25% of the hosts succeeds, then Ansible must run the play on the remaining hosts.

   4.1. Review the top section of the `update_webapp.yml` playbook:

   ```
   ---
   - name: Update web servers to use a new document root
     hosts: web_servers
     become: true
     force_handlers: true
     vars:
       webapp_content_dir: /srv/web/app/html

   ...output omitted...
   ```

   The `update_webapp.yml` playbook runs against the hosts in the `web_servers` host group. The `webapp_content_dir` variable specifies the root directory for Apache web documents. If not specified, the `webapp_content_dir` variable defaults to the `/var/www/html` directory.

   4.2. Review the `pre_tasks` section of the play:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
...output omitted...

  pre_tasks:
    - name: Remove web server from service during the update
      community.general.haproxy:
        state: disabled
        backend: app
        host: "{{ inventory_hostname }}"
      delegate_to: "{{ item }}"
      loop: "{{ groups['lb_servers'] }}"

...output omitted...
```

Before the playbook updates the web application on each server, the play uses the `community.general.haproxy` module to disable the server in all load balancers. This task protects external clients from errors discovered after application deployment.

4.3. Review the `tasks` section of the play:

```
...output omitted...

  tasks:
    - name: Ensure Apache HTTP Server is installed and started
      ansible.builtin.include_role:
        name: apache

    - name: Ensure web content is deployed
      ansible.builtin.include_role:
        name: webapp

...output omitted...
```

The play includes the `apache` and `webapp` roles.

The `apache` role modifies the `/etc/httpd/conf/httpd.conf` configuration file to use the directory specified by the `webapp_content_dir` variable. The `webapp` role deploys content to the directory that is specified by the `webapp_content_dir` variable.

4.4. Review the `post_tasks` section of the play:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
...output omitted...

  post_tasks:
    - name: Smoke Test - Ensure HTTP 200 OK
      ansible.builtin.uri:
        url: http://{{ inventory_hostname }}:{{ apache_port }}
        status_code: 200
      delegate_to: "{{ groups['lb_servers'][0] }}"
      become: false

    - name: Enable healthy server in load balancers
      community.general.haproxy:
        state: enabled
        backend: app
        host: "{{ inventory_hostname }}"
      delegate_to: "{{ item }}"
      loop: "{{ groups['lb_servers'] }}"
```

After the playbook deploys the web application, a smoke test ensures that each back-end web server responds with a 200 HTTP status code. The firewall rules on each web server only enable web requests from a load balancer, so all smoke tests are delegated to a load balancer.

If the smoke test fails for a server, then further processing of that server halts, and the web server is not re-enabled in the load balancer. The second task uses the `community.general.haproxy` module to enable the server in the load balancer when the smoke test passes.

4.5. Edit the `update_webapp.yml` playbook. Add the `serial` directive to the play so that the play runs on a single host, then on 25% of the hosts, and then on the remaining hosts:

```
---
- name: Update web servers to use a new document root
  hosts: web_servers
  become: true
  force_handlers: true
  vars:
    webapp_content_dir: /srv/web/app/html
  serial:
    - 1
    - 25%
    - 100%

  pre_tasks:
...output omitted...
```

5. Use the `update_webapp.yml` playbook to deploy the web application to the `/srv/web/app/html` directory.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Send another five requests to the load balancer and analyze the result.

5.1. Use the `ansible-navigator run` command to run the `update_webapp.yml` playbook. The playbook starts running against the `serverb.lab.example.com` hosts, and fails in the smoke test task:

```
[student@workstation management]$ ansible-navigator run update_webapp.yml

...output omitted...

TASK [Smoke Test - Ensure HTTP 200 OK]
*************************************
fatal: [serverb.lab.example.com -> servera.lab.example.com]: FAILED! =>
{"accept_ranges": "bytes", "changed": false, "connection": "close",
"content_length": "5909", "content_type": "text/html; charset=UTF-8",
"date": "Wed, 21 May 2025 01:41:09 GMT", "elapsed": 0, "etag": "\"1715-
5c91ee59c9780\"", "last_modified": "Mon, 09 Aug 2021 11:43:42 GMT", "msg":
"Status code was 403 and not [200]: HTTP Error 403: Forbidden",
"redirected": false, "server": "Apache/2.4.57 (Red Hat Enterprise Linux)",
"status": 403, "url": "http://serverb.lab.example.com:80"}

PLAY RECAP
*********************************************************************
serverb.lab.example.com   : ok=11    changed=5  ...  failed=1    skipped=1
...
```

5.2. Use the `curl` command to make five web requests to `servera`. Notice how the load balancer only redirects requests to four of the original five web servers:

```
[student@workstation management]$ for x in {1..5}; do curl servera; done
serverd: /var/www/html/index.html
--
servere: /var/www/html/index.html
--
serverf: /var/www/html/index.html
--
serverc: /var/www/html/index.html
--
serverd: /var/www/html/index.html
--
```

The `update_webapp.yml` playbook first removed one of the servers from the load-balancing pool (`serverb` in this example). After applying changes to the web server, the playbook ran a task to validate that the web server could still respond successfully to web requests. Because the web server did not respond successfully, the playbook did not add the server back to the load-balancing pool.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

> **Note**
> In this example, each web server displays a custom message. A real-world example would deploy the same content to each web server. Users accessing the load balancer would be unaware that one or more web servers were removed from the load-balancing pool.

6. Web servers often return a 403 HTTP status code because of either a regular file permission problem or an SELinux access problem. Identify the problem.

   6.1. Connect to `serverb` as the `student` user:

```
[student@workstation management]$ ssh serverb
...output omitted...
[student@serverb ~]$
```

   6.2. Use the `ls` command to display the permissions and ownership for the `/srv/web/app/html/index.html` file. Regular file permissions indicate that the `index.html` file is readable by everyone, so the file permissions are not the problem.

```
[student@serverb ~]$ ls -l /srv/web/app/html/index.html
-rw-r--r--. 1 root root 41 May 21 19:26 /srv/web/app/html/index.html
```

   6.3. Use the `ls` command to display SELinux context information for the `/srv/web/app/html/index.html` file. The `var_t` SELinux context type is not appropriate for web content. SELinux prevents the `httpd` process from accessing files with an incorrect context type.

```
[student@serverb ~]$ ls -Z /srv/web/app/html/index.html
system_u:object_r:var_t:s0 /srv/web/app/html/index.html
```

   6.4. By default, the `/var/www/html` directory uses a context type allowed by SELinux. Identify the context type for the `/var/www/html` directory:

```
[student@serverb ~]$ ls -Zd /var/www/html
system_u:object_r:httpd_sys_content_t:s0 /var/www/html
```

   6.5. Identify the context type for the `/var/www/html/index.html` file:

```
[student@serverb ~]$ ls -Z /var/www/html/index.html
system_u:object_r:httpd_sys_content_t:s0 /var/www/html/index.html
```

   6.6. Log out of the `serverb` machine:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@serverb ~]$ logout
Connection to serverb closed.
[student@workstation management]$
```

7. Update the `update_webapp.yml` playbook to resolve the issue. Run the `update_webapp.yml` playbook again. This time, the playbook succeeds.

   As the playbook runs, monitor the output of the `issue_requests.sh` script in another terminal tab or window. The output shows the back-end servers gradually switching over to using the new web document root directory.

   7.1. *(Optional)* Verify that the `apache` and `webapp` roles set the default SELinux context on the path given by the `webapp_content_dir` variable.

   Review the `roles/apache/tasks/main.yml` file:

```
...output omitted...

- name: Ensure directory exists
  ansible.builtin.file:
    path: "{{ webapp_content_dir }}"
    state: directory
    owner: root
    group: root
    mode: "0755"
    setype: _default
```

   Review the `roles/webapp/tasks/main.yml` file:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
---
# tasks file for webapp

- name: Ensure directory exists
  ansible.builtin.file:
    path: "{{ webapp_content_dir }}"
    state: directory
    owner: root
    group: root
    mode: "0755"
    setype: _default

- name: Ensure web content is deployed
  ansible.builtin.template:
    src: index.html.j2
    dest: "{{ webapp_content_dir }}/index.html"
    owner: root
    group: root
    mode: "0644"
    setype: _default
```

7.2. Modify the `update_webapp.yml` playbook. Add a task to the `pre-tasks` section of the play. Use the `redhat.rhel_system_roles.selinux` role to create the `httpd_sys_content_t` SELinux type for all files under the `/srv/web/app/html` directory:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
...output omitted...
  pre_tasks:
    - name: Remove web server from service during the update
      community.general.haproxy:
        state: disabled
        backend: app
        host: "{{ inventory_hostname }}"
      delegate_to: "{{ item }}"
      loop: "{{ groups['lb_servers'] }}"

    - name: Ensure SELinux context is applied
      ansible.builtin.include_role:
        name: redhat.rhel_system_roles.selinux
      vars:
        selinux_fcontexts:
          - target: "{{ webapp_content_dir }}(/.*)?"
            setype: httpd_sys_content_t
            state: present

  tasks:
...output omitted...
```

7.3. Run the `issue_requests.sh` script in a separate terminal tab or window and monitor the output during the execution of the `update_webapp.yml` playbook. The script sends a web request to the `servera.lab.example.com` load balancer every two seconds. Output is displayed in the terminal and logged to the `curl_output.log` file.

```
[student@workstation management]$ ./issue_requests.sh
serverc: /var/www/html/index.html
--
serverd: /var/www/html/index.html
--
servere: /var/www/html/index.html
--
serverf: /var/www/html/index.html
--
...output omitted...
```

Initially, responses cycle through the four hosts remaining in the load-balancing pool.

Leave the `issue_requests.sh` script running and return to the previous terminal tab or window.

7.4. Use the `ansible-navigator run` command to run the `update_webapp.yml` playbook:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation update-management]$ ansible-navigator run
update_webapp.yml
...output omitted...
```

7.5. Monitor the output of the `issue_requests.sh` script.

Eventually, the smoke test passes for the new application, and the `serverb` server is returned to service with an updated web document root directory:

```
...output omitted...
servere: /var/www/html/index.html
--
serverf: /var/www/html/index.html
--
serverb: /srv/web/app/html/index.html
--
serverd: /var/www/html/index.html
--
...output omitted...
```

The playbook processes the next batch, which only contains the `serverc` server.

The playbook removes `serverc` from service:

```
...output omitted...
serverb: /srv/web/app/html/index.html
--
serverd: /var/www/html/index.html
--
servere: /var/www/html/index.html
--
serverf: /var/www/html/index.html
--
serverb: /srv/web/app/html/index.html
--
...output omitted...
```

Eventually, the smoke test passes for the `serverc` server and the server is put back into service:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
...output omitted...
serverb: /srv/web/app/html/index.html
--
serverc: /srv/web/app/html/index.html
--
serverd: /var/www/html/index.html
--
servere: /var/www/html/index.html
...output omitted...
```

The last batch processes all remaining web servers. The playbook first disables all three of these servers, leaving only the `serverb` and `serverc` servers to handle requests:

```
...output omitted...
serverb: /srv/web/app/html/index.html
--
serverc: /srv/web/app/html/index.html
--
serverb: /srv/web/app/html/index.html
--
serverc: /srv/web/app/html/index.html
--
...output omitted...
```

Eventually, each server passes the smoke test and is put back into service:

```
...output omitted...
serverb: /srv/web/app/html/index.html
--
serverc: /srv/web/app/html/index.html
--
serverd: /srv/web/app/html/index.html
--
servere: /srv/web/app/html/index.html
--
serverf: /srv/web/app/html/index.html
--
serverb: /srv/web/app/html/index.html
--
...output omitted...
```

7.6. Press `Ctrl` + `C` to stop the `issue_requests.sh` script.

7.7. Return to the terminal tab or window that is running the `ansible-navigator` command. The `update_webapp.yml` playbook completed successfully.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
...output omitted...

PLAY RECAP
*********************************************************************
serverb.lab.example.com   : ok=20   changed=4  ...  failed=0   skipped=17
...
serverc.lab.example.com   : ok=21   changed=7  ...  failed=0   skipped=17
...
serverd.lab.example.com   : ok=21   changed=7  ...  failed=0   skipped=17
...
servere.lab.example.com   : ok=21   changed=7  ...  failed=0   skipped=17
...
serverf.lab.example.com   : ok=21   changed=7  ...  failed=0   skipped=17
...
```

Finish

On the `workstation` machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish update-management
```

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

8.7. Lab
Coordinating Rolling Updates

Modify a playbook to implement rolling updates.

Outcomes

- Delegate tasks to other hosts.
- Implement deployment batches with the `serial` keyword.
- Limit failures with the `max_fail_percentage` keyword.

Prerequisites

As the `student` user on the `workstation` machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available.

This command initializes the remote Git repository at `https://git.lab.example.com/student/update-review.git`. The Git repository contains playbooks that configure a front-end load balancer and a pool of back-end web servers. You can push changes to this repository by using Stud3nt123 as the Git password.

```
[student@workstation ~]$ lab start update-review
```

Instructions

1. Clone the `https://git.lab.example.com/student/update-review.git` repository into the `/home/student/git-repos` directory, and then create the `exercise` branch for this exercise. Run the `site.yml` playbook to deploy the web application infrastructure.

2. The `update_webapp.yml` playbook requires collections that are defined in the `collections/requirements.yml` file.

   Obtain a token from `https://aap.lab.example.com` by logging in as `student` using `student` as the password. Use this token to update the `token` line in the `ansible.cfg` file. Use the `ansible-galaxy` command to install the collections. The collections must be available to the execution environment, so they must be installed in the `/home/student/git-repos/update-review/collections/` directory.

3. Add a `pre_tasks` section to the play in the `update_webapp.yml` playbook. Add a task to the section that disables the web servers on the HAProxy load balancer. Without this task, external clients might experience problems with the web application due to unforeseen deployment issues.

   Configure the new task as follows:

   - Use the `community.general.haproxy` module.
   - Disable the host in the app back end by using the `inventory_hostname` variable.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

- Delegate each action to the load balancer. The `{{ groups['lb_servers'][0] }}` Jinja2 expression provides the name of this load balancer.

4. In the `update_webapp.yml` playbook, the first task in the `post_tasks` section is a "smoke test". That task verifies that the web application on each web server responds to requests that originate from that same server. Because the current task tests the web server from the web server itself, the task does not test any network-related functions.

   Using delegation, modify the smoke test task so that each request to a web server originates from the load balancer. Use the `inventory_hostname` variable to connect to each web server.

5. Add a task to the `post_tasks` section after the smoke test to re-enable each web server on the HAProxy load balancer. This task is similar in structure to the `community.general.haproxy` task in the `pre_tasks` section, except that it sets the state to `enabled` instead of `disabled`.

6. Configure the play in the `update_webapp.yml` playbook to run through its tasks in batches to mitigate the effects of unforeseen deployment errors. Ensure that the playbook uses no more than three batches to complete the upgrade of all web server hosts. Set the first batch to consist of 5% of the hosts, the second batch 35% of the hosts, and the last batch to consist of all remaining hosts.

   Add an appropriate keyword to the play to ensure that playbook execution stops if any host fails a task during the upgrade.

7. Run the `update_webapp.yml` playbook to perform the rolling update.

8. Commit and push your changes to the remote Git repository.

Evaluation

As the `student` user on the `workstation` machine, use the `lab` command to grade your work. Correct any reported failures. Make sure to commit and push any additional changes to the remote Git repository and then rerun the command until successful.

```
[student@workstation ~]$ lab grade update-review
```

Finish

As the `student` user on the `workstation` machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish update-review
```

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Solution

Modify a playbook to implement rolling updates.

Outcomes

- Delegate tasks to other hosts.
- Implement deployment batches with the `serial` keyword.
- Limit failures with the `max_fail_percentage` keyword.

Prerequisites

As the `student` user on the `workstation` machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available.

This command initializes the remote Git repository at `https://git.lab.example.com/student/update-review.git`. The Git repository contains playbooks that configure a front-end load balancer and a pool of back-end web servers. You can push changes to this repository by using `Stud3nt123` as the Git password.

```
[student@workstation ~]$ lab start update-review
```

Instructions

1. Clone the `https://git.lab.example.com/student/update-review.git` repository into the `/home/student/git-repos` directory, and then create the `exercise` branch for this exercise. Run the `site.yml` playbook to deploy the web application infrastructure.

   1.1. From a terminal, create the directory `/home/student/git-repos` if it does not exist, and then change into it:

   ```
   [student@workstation ~]$ mkdir -p ~/git-repos/
   [student@workstation ~]$ cd ~/git-repos/
   [student@workstation git-repos]$
   ```

   1.2. Clone the repository and then change to the cloned repository:

   ```
   [student@workstation git-repos]$ git clone \
   https://git.lab.example.com/student/update-review.git
   Cloning into 'update-review'...
   ...output omitted...
   [student@workstation git-repos]$ cd update-review
   [student@workstation update-review]$
   ```

   1.3. Create the `exercise` branch:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation update-review]$ git checkout -b exercise
Switched to a new branch 'exercise'
```

1.4. Use the `ansible-navigator run` command to run the `site.yml` playbook:

```
[student@workstation update-review]$ ansible-navigator run site.yml

PLAY [Gather web_server facts]
*************************************************

TASK [Gathering Facts]
*********************************************************
ok: [serverc.lab.example.com]
ok: [serverf.lab.example.com]
ok: [serverd.lab.example.com]
ok: [servere.lab.example.com]
ok: [serverb.lab.example.com]

PLAY [Ensure HAProxy is deployed]
**********************************************

...output omitted...

PLAY RECAP
*********************************************************************
servera.lab.example.com    : ok=6    changed=5    unreachable=0    failed=0
...
serverb.lab.example.com    : ok=11   changed=7    unreachable=0    failed=0
...
serverc.lab.example.com    : ok=11   changed=7    unreachable=0    failed=0
...
serverd.lab.example.com    : ok=11   changed=7    unreachable=0    failed=0
...
servere.lab.example.com    : ok=11   changed=7    unreachable=0    failed=0
...
serverf.lab.example.com    : ok=11   changed=7    unreachable=0    failed=0
...
```

2. The `update_webapp.yml` playbook requires collections that are defined in the `collections/requirements.yml` file.

   Obtain a token from `https://aap.lab.example.com` by logging in as `student` using `student` as the password. Use this token to update the `token` line in the `ansible.cfg` file. Use the `ansible-galaxy` command to install the collections. The collections must be available to the execution environment, so they must be installed in the `/home/student/git-repos/update-review/collections/` directory.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

2.1. Go to `https://aap.lab.example.com` and log in to Ansible Automation Platform as `student` using `student` as the password.

2.2. Go to **Automation Content > API Token** and then click **Generate token**. Copy the API token.

2.3. Update the `token` line in the `ansible.cfg` file by using the copied token. Your token is probably different from the one that is displayed in this example.

```
[defaults]
inventory=inventory
remote_user=devops
collections_path = ./collections:/usr/share/ansible/collections

[galaxy]
server_list = community

[galaxy_server.community]
url=https://aap.lab.example.com/api/galaxy/content/community/
token=8b11d83ed58853059658fccf73a0a92ac2fb07ff
```

2.4. Use the `ansible-galaxy` command to install the `community.general` content collection into the `collections/` directory:

```
[student@workstation update-review]$ ansible-galaxy collection install \
-r collections/requirements.yml -p collections/
...output omitted...
community.general:10.3.0 was installed successfully
```

3. Add a `pre_tasks` section to the play in the `update_webapp.yml` playbook. Add a task to the section that disables the web servers on the HAProxy load balancer. Without this task, external clients might experience problems with the web application due to unforeseen deployment issues.

Configure the new task as follows:

- Use the `community.general.haproxy` module.
- Disable the host in the app back end by using the `inventory_hostname` variable.
- Delegate each action to the load balancer. The `{{ groups['lb_servers'][0] }}` Jinja2 expression provides the name of this load balancer.

3.1. Edit the play in the `update_webapp.yml` playbook by adding a `pre_tasks` section containing a task that uses the `community.general.haproxy` module to disable the web server on the load balancer:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
...output omitted...
  pre_tasks:
    - name: Remove web server from service during the update
      community.general.haproxy:
        state: disabled
        backend: app
        host: "{{ inventory_hostname }}"
      delegate_to: "{{ groups['lb_servers'][0] }}"

  roles:
    - role: webapp
...output omitted...
```

4. In the `update_webapp.yml` playbook, the first task in the `post_tasks` section is a "smoke test". That task verifies that the web application on each web server responds to requests that originate from that same server. Because the current task tests the web server from the web server itself, the task does not test any network-related functions.

   Using delegation, modify the smoke test task so that each request to a web server originates from the load balancer. Use the `inventory_hostname` variable to connect to each web server.

   4.1. Add a `delegate_to` directive to the smoke test task that runs on the load balancer. Change the testing URL to point to the current value of the `inventory_hostname` variable:

   ```
   ...output omitted...
       - name: Smoke Test - Ensure HTTP 200 OK
         ansible.builtin.uri:
           url: http://{{ inventory_hostname }}:{{ apache_port }}
           status_code: 200
         become: false
         delegate_to: "{{ groups['lb_servers'][0] }}"
   ...output omitted...
   ```

5. Add a task to the `post_tasks` section after the smoke test to re-enable each web server on the HAProxy load balancer. This task is similar in structure to the `community.general.haproxy` task in the `pre_tasks` section, except that it sets the state to `enabled` instead of `disabled`.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
...output omitted...
  post_tasks:
    - name: Smoke Test - Ensure HTTP 200 OK
      ansible.builtin.uri:
        url: http://localhost:{{ apache_port }}
        status_code: 200
      become: false

    - name: Enable healthy server in load balancers
      community.general.haproxy:
        state: enabled
        backend: app
        host: "{{ inventory_hostname }}"
      delegate_to: "{{ groups['lb_servers'][0] }}"
```

6. Configure the play in the `update_webapp.yml` playbook to run through its tasks in batches to mitigate the effects of unforeseen deployment errors. Ensure that the playbook uses no more than three batches to complete the upgrade of all web server hosts. Set the first batch to consist of 5% of the hosts, the second batch 35% of the hosts, and the last batch to consist of all remaining hosts.

   Add an appropriate keyword to the play to ensure that playbook execution stops if any host fails a task during the upgrade.

   6.1. Add the `max_fail_percentage` directive and set the value to `0` to stop execution of the play at any failure. Add the `serial` directive and set the value to a list of three elements: 5%, 35%, and 100% to ensure that all servers are updated in the last batch. The beginning of the play should appear as follows:

   ```
   - name: Upgrade Web Application
     hosts: web_servers
     become: true
     vars:
       webapp_version: v1.1
     max_fail_percentage: 0
     serial:
       - 5%
       - 35%
       - 100%
   ...output omitted...
   ```

   6.2. The completed `update_webapp.yml` playbook should appear as follows:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
---
- name: Upgrade Web Application
  hosts: web_servers
  become: true
  vars:
    webapp_version: v1.1
  max_fail_percentage: 0
  serial:
    - 5%
    - 35%
    - 100%

  pre_tasks:
    - name: Remove web server from service during the update
      community.general.haproxy:
        state: disabled
        backend: app
        host: "{{ inventory_hostname }}"
      delegate_to: "{{ groups['lb_servers'][0] }}"

  roles:
    - role: webapp

  post_tasks:
    - name: Smoke Test - Ensure HTTP 200 OK
      ansible.builtin.uri:
        url: http://{{ inventory_hostname }}:{{ apache_port }}
        status_code: 200
      become: false
      delegate_to: "{{ groups['lb_servers'][0] }}"

    - name: Enable healthy server in load balancers
      community.general.haproxy:
        state: enabled
        backend: app
        host: "{{ inventory_hostname }}"
      delegate_to: "{{ groups['lb_servers'][0] }}"
```

7. Run the `update_webapp.yml` playbook to perform the rolling update.

   7.1. Use the `ansible-navigator` command to run the `update_webapp.yml` playbook:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation update-review]$ ansible-navigator run update_webapp.yml
...output omitted...

PLAY RECAP
*********************************************************************
serverb.lab.example.com    : ok=6    changed=3    unreachable=0    failed=0
...
serverc.lab.example.com    : ok=6    changed=3    unreachable=0    failed=0
...
serverd.lab.example.com    : ok=6    changed=3    unreachable=0    failed=0
...
servere.lab.example.com    : ok=6    changed=3    unreachable=0    failed=0
...
serverf.lab.example.com    : ok=6    changed=3    unreachable=0    failed=0
...
```

8. Commit and push your changes to the remote Git repository.

   8.1. Add and commit the changed files, and push them to the Git repository. If prompted, use `Stud3nt123` as the password.

```
[student@workstation update-review]$ git add .
[student@workstation update-review]$ git commit -m "Rolling updates"
...output omitted...
[student@workstation update-review]$ git push -u origin exercise
Password for 'https://student@git.lab.example.com': Stud3nt123
...output omitted...
```

Evaluation

As the `student` user on the `workstation` machine, use the `lab` command to grade your work. Correct any reported failures. Make sure to commit and push any additional changes to the remote Git repository and then rerun the command until successful.

```
[student@workstation ~]$ lab grade update-review
```

Finish

As the `student` user on the `workstation` machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish update-review
```


For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

8.8. Summary

- The `delegate_to` keyword can delegate a task to run on a different host.
- The `forks` parameter in the Ansible configuration file specifies the maximum number of parallel connections to managed hosts.
- The `serial` keyword configures Ansible to run hosts through a play in multiple batches.
- The `max_fail_percentage` keyword configures Ansible to abort the play if more than a certain percentage of hosts in the current batch fail.
- The `run_once` keyword specifies that a task runs once for a batch rather than once for each host in the batch.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Chapter 9.
Creating Ansible Content Collections and Automation Execution Environments

Goal

Write your own Ansible Content Collections, publish them, embed them in a custom automation execution environment, and run them in playbooks by using automation controller.

Sections

- Writing Ansible Content Collections (and Guided Exercise)
- Building a Custom Automation Execution Environment (and Guided Exercise)
- Validating a Custom Automation Execution Environment (and Guided Exercise)
- Using Custom Ansible Content Collections and Automation Execution Environments in Automation Controller (and Guided Exercise)

Lab

- Creating Ansible Content Collections and Automation Execution Environments

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

9.1. Writing Ansible Content Collections

Objectives

- Create content collections and distribute them for reuse.

Developing Ansible Content Collections

Red Hat, its partners, and the Ansible community provide Ansible Content Collections for many software products and technologies. However, customers often have automation needs that no existing collection covers. For example, you might have to pilot an application for which there is no available collection, or your company might develop internal tools that you need to automate. In all those situations, you should consider developing collections.

> **Note**
> Ansible 2.8 and earlier do not support collections. With those versions, you develop and distribute individual roles and provide modules and plug-ins inside those roles or alongside your playbooks.
>
> Red Hat still supports that architecture but recommends moving your roles, modules, plug-ins, and filters inside collections. Automation hub and private automation hub do not support individual roles.

Selecting a Namespace for Collections

Ansible organizes collections in namespaces. The namespace is the first part of a collection name. For example, the namespace of the `amazon.aws` collection is `amazon`.

Consider where you intend to publish your collections when choosing a namespace:

- If you plan to publish your collections to Ansible Galaxy, then use your Ansible Galaxy username as the namespace.
- If you publish your collections to private automation hub, then ask the platform administrators to create a namespace for you. Note that you might be able to create a namespace yourself if your user account has the required permissions.
- If you are a Red Hat partner and publish your collections to automation hub, then use the namespace that Red Hat has assigned for your company.

Creating the Collection Directory Structure

Use the `ansible-galaxy collection init` command to create the directory structure for your new collection. Specify the name of the collection, including the namespace, as the argument to the command:

```
[user@host demo]$ ansible-galaxy collection init \
my_namespace.my_collection
- Collection my_namespace.my_collection was created successfully
```

This creates a basic directory structure with all required metadata files for your collection.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[user@host demo]$ tree my_namespace/
my_namespace/
└── my_collection
    ├── README.md
    ├── docs
    ├── galaxy.yml
    ├── meta
    │   └── runtime.yml
    ├── plugins
    │   └── README.md
    └── roles

4 directories, 4 files
```

You can also configure a collection that can be directly installed from a Git repository:

- Create a Git repository that matches the name of your desired collection, such as *namespace/collection*.
- Clone the repository and change to the repository directory.
- Initialize the collection with the `ansible-galaxy collection init` command.
- Move everything from the *namespace/collection* directory to top level of the repository.
- Recursively remove the *namespace* directory.

Adding Content to a Collection

Organize your custom modules, plug-ins, and filters in subdirectories under the `plugins` directory. For modules, create the `plugins/modules` subdirectory and then copy the Python scripts for your modules into that subdirectory. For inventory plug-ins, create and then use the `plugins/inventory` subdirectory. Similarly, the other plug-in types have dedicated subdirectories. Developing modules, plug-ins, and filters is beyond the scope of this course.

If you have existing roles, then you can move their directory structure under the `roles` directory. To create a role for the collection, use the `ansible-galaxy role init` command from inside the `roles` directory:

```
[user@host demo]$ cd my_namespace/my_collection/roles/
[user@host roles]$ ansible-galaxy role init my_role
- Role my_role was created successfully
[user@host roles]$ ls my_role/
README.md  defaults  files  handlers  meta  tasks  templates  tests  vars
```

Updating Collection Metadata

The `galaxy.yml` file at the root of the collection directory provides information for Ansible to build and publish the collection. The file includes comments that describe each parameter.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Important
After you publish a collection, you must update the `version` parameter in the `galaxy.yml` file before building and publishing another version of the collection.

You can define additional metadata for the collection in the `meta/runtime.yml` file. If your collection requires a specific version of Ansible, then add the `requires_ansible` parameter to the file. The following example specifies that the collection works with Ansible 2.15.0 or later:

```
---
requires_ansible: ">=2.15.0"
```

Important
The `meta/runtime.yml` file and the `requires_ansible` parameter are mandatory if you plan to publish your collection to Ansible Galaxy, automation hub, or private automation hub. Otherwise, the validation process that those platforms run rejects the collection.

Although the default value of `2.9.10` works, private automation hub produces a warning if the value is less than `2.15.0`.

Declaring Collection Dependencies

Some collections depend on other collections to work correctly. For example, a role in your new collection might call modules from other collections. Declare the required collections by using the `dependencies` parameter in your `galaxy.yml` file.

The `dependencies` parameter is a dictionary of collections. For each entry, the key is the fully qualified collection name (FQCN), and the value is the collection version (use `>=1.0.0` when you do not need a specific version). The following example lists the `community.general` and `ansible.posix` collections as requirements:

```
...output omitted...
dependencies:
  community.general: '>=1.0.0'
  ansible.posix: '>=1.0.0'
...output omitted...
```

You do not need to declare the `ansible.builtin` collection.

Declaring Python Dependencies

More complex collections might provide modules, plug-ins, and filters that depend on additional Python libraries. If your collection has Python dependencies, then create a `requirements.txt` file at the root of your collection. Each line of the file declares a Python package and might optionally specify a version for the package. The following example shows the content of the `requirements.txt` file for the `amazon.aws` collection:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
# When updating the minimal requirements please also update
# - tests/unit/constraints.txt
# - tests/integration/constraints.txt
# - tests/integration/targets/setup_botocore_pip
botocore>=1.29.0
boto3>=1.26.0
```

Declaring System Package Dependencies

Some collections also require system packages in the automation execution environment. For example, the `ansible.posix` collection requires the `rsync` RPM package. At the root of your collection, create the `bindep.txt` file and list the RPM packages, one per line. The following example shows the content of the `bindep.txt` file for the `ansible.posix` collection:

```
rsync [platform:redhat]
```

The `bindep` tool, which processes the `bindep.txt` file, can manage packages for several Linux distributions. Different distributions might have other package names for the same software. The `[platform:redhat]` directive targets Linux distributions similar to Red Hat. The `[platform:rpm]` directive targets Linux distributions that use the RPM packaging system.

More information about `bindep` is available in the references at the end of this section.

> **Note**
>
> The `ansible-galaxy collection install` command installs collection dependencies defined in the `galaxy.yml` file, but does not install Python dependencies defined in the `requirements.txt` file or the system packages defined in the `bindep.txt` file.
>
> You should still define Python dependencies and system packages for your collection because the `ansible-builder` command uses the `requirements.txt` and `bindep.txt` files when it creates a custom automation execution environment.
>
> If you are using the `ansible-playbook` command, or the `ansible-navigator` command with the `--ee false` option, then you must install the Python and system package dependencies on your local system before you can use the collection.

Validating and Testing Collections

The `ansible-navigator lint` command is a tool that checks Ansible Playbooks, roles, and collections for recommended practices and potential issues. It helps enforce coding standards by running a series of linting rules against your automation content.

By catching common errors early, such as incorrect module usage or risky practices, Ansible lint improves the reliability and maintainability of your Ansible code. It is especially useful in collaborative environments or CI pipelines where consistent automation quality matters.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Use the following Ansible lint command to check a directory that contains a collection:

```
[user@host my_collection]$ ansible-navigator lint $(pwd)
...output omitted...
```

When you publish your collection to Ansible Galaxy, automation hub, or private automation hub, those platforms automatically run some tests.

Private automation hub uses Ansible lint to evaluate each uploaded collection and saves each report as an import log.

The following output shows an import log for the collection named `my_namespace.my_collection`. Although the import log indicates that the collection uploaded successfully, you might address the issues identified by Ansible lint:

```
Importing with galaxy-importer 0.4.25
Getting doc strings via ansible-doc
Finding content inside collection
Loading role my_role
No changelog found. Add a CHANGELOG.rst or CHANGELOG.md file in the collection root
or docs/ dir, or a changelogs/changelog.(yml/yaml) file. (1)
Linting collection via ansible-lint...
meta/runtime.yml:1: meta-runtime[unsupported-version]: 'requires_ansible' key must
refer to a currently supported version such as: >=2.15.0, >=2.16.0, >=2.17.0 (2)
roles/my_role/meta/main.yml:1: meta-incorrect: Should change default metadata:
license (3)
roles/my_role/meta/main.yml:1: schema[meta]: $.galaxy_info.min_ansible_version 2.1
is not of type 'string'. See
https://docs.ansible.com/ansible/latest/playbook_guide/playbooks_reuse_roles.html#u
sing-role-dependencies (4)
roles/my_role/tasks/main.yml:3: risky-file-permissions File permissions unset or
incorrect. (5)
...ansible-lint run complete
Collection loading complete

Done
```

1. The collection does not include a change log file.
2. The collection uses the default value of `>=2.9.10` for the `requires_ansible` key in the `metadata/runtime.yml` file. The message indicates alternative values.
3. The role uses the default value of `license (GPL-2.0-or-later, MIT, etc)` for the `license` key in the `meta/main.yml` file.
4. The role uses the default value of `2.1` for the `min_ansible_version` key. You can resolve this issue by quoting the value of the key, such as `"2.1"`, in the `meta/main.yml` file.
5. Ansible lint might identify additional areas for improvement.

For collections that Red Hat partners develop, automation hub runs more thorough tests by using the `ansible-test` tool. You might install and use that tool before publishing your collections because some platforms reject

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

contributions that do not pass the required tests.

At a minimum, you should build your collection, install your collection from the `.tar.gz` file, and then develop a playbook that uses the collection. Your test playbook should demonstrate that the collection works as expected.

Building and Publishing Collections

You can use the `ansible-galaxy collection build` command to build a collection. If successful, then the command produces a `.tar.gz` archive file:

```
[user@host my_namespace]$ ansible-galaxy collection build
Created collection for my_namespace.my_collection at
/home/user/demo/my_namespace/my_collection/my_namespace-my_collection-1.0.0.tar.gz
```

The name of the archive file includes the version of the collection as defined in the collection `galaxy.yml` file.

You can use the resulting archive file to install and test the collection locally, share the collection with team members, or publish it to a distribution platform such as Ansible Galaxy or private automation hub.

For additional security, automation content creators can sign their collections. More information about collections and content signing is available in the references at the end of this section.

Publishing a Collection to Private Automation Hub

Use the following procedure to publish a collection to your private automation hub:

- Log in to the Ansible Automation Platform unified UI.
- From the unified UI, go to **Automation Content > Namespaces** and click the **Create namespace**.
- Enter the name of your namespace and then click **Create namespace**.
- Click the **Collections** tab and then click **Upload collection**.
- Browse to the location of the `.tar.gz` file and then click **Upload collection**.

Approving or Rejecting an Uploaded Collection

Before users can download your collection from private automation hub, an administrator must review and approve your collection:

- From the unified UI, go to **Automation Content > Collection Approvals** to list the collections waiting for approval.
- View the import log of the uploaded collection.
- Either click **Approve and sign collection** (the thumbs up icon) to approve the collection or click **Reject collection** (the thumbs down icon) to reject the collection.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

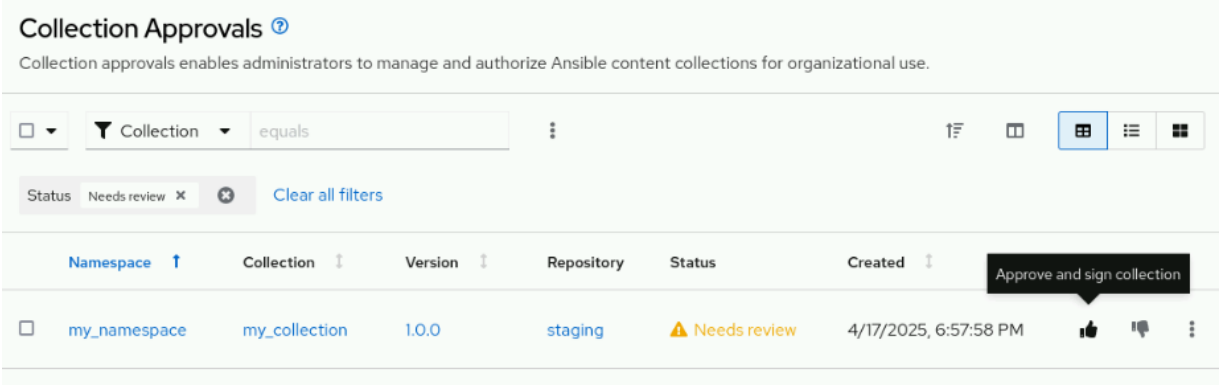


Figure 39. Approving a collection

You can download approved private automation hub collections from the `published` repository.

References

Developing Collections - Ansible Documentation

Collection Galaxy Metadata Structure - Ansible Documentation

PIP Documentation v25.0.1 Requirements File Format

Ansible Lint Documentation: Using commands

Collections and Content Signing in Private Automation Hub

Introduction to `bindep`

Generating Changelogs and Porting Guide Entries in a Collection

Distributing Collections

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

9.2. Guided Exercise
Writing Ansible Content Collections

Create a content collection and publish it to a private automation hub.

Outcomes

- Create a collection directory structure.
- Add content to the collection.
- Build the collection.
- Publish the collection.

Prerequisites

As the `student` user on the `workstation` machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
[student@workstation ~]$ lab start create-writing
```

Instructions

1. Initialize the structure of a collection so that the collection can be installed from a Git repository.

   1.1. From a terminal, create the `/home/student/git-repos` directory if it does not already exist, and then change into it:

   ```
   [student@workstation ~]$ mkdir -p ~/git-repos/
   [student@workstation ~]$ cd ~/git-repos/
   [student@workstation git-repos]$
   ```

   1.2. Clone the `https://git.lab.example.com/student/student.test.git` repository and then change to the cloned repository:

   ```
   [student@workstation git-repos]$ git clone \
   https://git.lab.example.com/student/student.test.git
   Cloning into 'student.test'...
   ...output omitted...
   [student@workstation git-repos]$ cd student.test
   [student@workstation student.test]$
   ```

   1.3. Initialize the default directories and files for a collection:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation student.test]$ ansible-galaxy collection \
init student.test
- Collection student.test was created successfully
```

1.4. The `ansible-galaxy collection init` command created the `student` directory for the collection namespace and the `test` subdirectory for the collection name. Although this directory structure matches the directory structure of an installed collection, this directory structure does not work for a collection hosted in a Git repository.

Move everything from the created collection to the current directory and then remove the `student` directory:

```
[student@workstation student.test]$ mv student/test/* .
[student@workstation student.test]$ rm -rf student/
```

> **Important**
> Notice that `student.test` in the previously cloned Git repository (`https://git.lab.example.com/student/student.test.git`) matches the desired collection name.

2. Review the initialized collection.

2.1. Use the `tree` command to list the files and directories in the collection:

```
[student@workstation student.test]$ tree
.
├── README.md
├── docs
├── galaxy.yml
├── meta
│   └── runtime.yml
├── plugins
│   └── README.md
└── roles

4 directories, 4 files
```

2.2. Review the `galaxy.yml` file for collection metadata, such as `namespace`, `name`, and `version`. The following command displays the first ten lines that do not start with the # character and are not blank lines:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation student.test]$ grep -Ev "^#|^$" galaxy.yml | head
namespace: student
name: test
version: 1.0.0
readme: README.md
authors:
- your name <example@domain.com>
description: your collection description
license:
- GPL-2.0-or-later
license_file: ''
```

Important

If you previously completed this exercise, then you must edit the `galaxy.yml` file to increment the collection version to a number greater than the previous version, such as to `1.0.1`. Failure to update the version might result in an error similar to the following when you attempt to install the collection:

```
ERROR! Failed to download collection tar from 'published' due to the
following unforeseen error: Mismatch artifact hash with downloaded file
```

2.3. Update the `meta/runtime.yml` file to specify that the collection requires Ansible 2.15.0 or later to work. Remove the comment and space from the `requires_ansible` line and change the version to `2.15.0`:

```
---
# Collections must specify a minimum required ansible version to upload
# to galaxy
requires_ansible: '>=2.15.0'
...output omitted...
```

3. Populate the collection with some provided files and then build the collection.

3.1. Create the `plugin/modules` directory and then copy the `/home/student/create-writing/myping.py` file to that directory. The `myping.py` file is essentially a copy of the `ansible.builtin.ping` module.

```
[student@workstation student.test]$ mkdir plugins/modules
[student@workstation student.test]$ cp ~/create-writing/myping.py \
plugins/modules/
```

3.2. Copy the `/home/student/create-writing/reset_firewall` role to the collection `roles` directory. This role searches for files in the `/etc/firewalld/zones` and `/etc/firewalld/services` directories

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

and removes any files found there. Those files take precedence over the files in their respective `/usr/lib/firewalld` directories.

```
[student@workstation student.test]$ cp -r ~/create-writing/reset_firewall \
roles/
```

3.3. Use the `ansible-galaxy` command to build the collection:

```
[student@workstation student.test]$ ansible-galaxy collection build
Created collection for student.test at /home/student/git-
repos/student.test/student-test-1.0.0.tar.gz
```

4. Make the local collection available to the remote Git repository.

4.1. Stage all changes made to the local repository:

```
[student@workstation student.test]$ git add .
```

4.2. Check the status of the local repository. Notice that the collection archive was not added. The project `.gitignore` file ignores files that match the `student-test-*.tar.gz` pattern.

```
[student@workstation student.test]$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   README.md
        new file:   galaxy.yml
        new file:   meta/runtime.yml
        new file:   plugins/README.md
        new file:   plugins/modules/myping.py
        new file:   roles/reset_firewall/README.md
        new file:   roles/reset_firewall/handlers/main.yml
        new file:   roles/reset_firewall/meta/main.yml
        new file:   roles/reset_firewall/tasks/main.yml
```

4.3. Commit and then push the changes to the remote repository. If prompted, use `Stud3nt123` as the password:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation student.test]$ git commit -m "Add student.test
collection"
...output omitted...

[student@workstation student.test]$ git push
Password for 'https://student@git.lab.example.com': Stud3nt123
...output omitted...
```

5. Create the `student` namespace in private automation hub and then upload and approve the new collection.

   5.1. Go to `https://aap.lab.example.com` and log in to Ansible Automation Platform as `admin` using `redhat` as the password.

   5.2. From the unified UI, go to **Automation Content > Namespaces** and then click **Create namespace**.

   5.3. Enter `student` as the name of the namespace and then click **Create namespace**.

   5.4. Click the **Collections** tab and then click **Upload collection**.

   5.5. Click **Browse**. Select the most recent version of the archive in the `/home/student/git-repos/student.test` directory, such as the `student-test-1.0.0.tar.gz` file, and then click **Upload collection**.

> **Note**
> The name of the archive file might be different if you updated the `galaxy.yml` file to increment the collection version.

   5.6. From the **Collection Approvals** page, click **Approve and sign collection** (the thumbs up icon) to approve the `student.test` collection.

   Confirm your decision, click **Approve collections**, and then close the confirmation window.

   5.7. Go to **Automation Content > Collections** and enter `test` in the search field.

   Verify that the unified UI displays the `test` automation content collection. Notice that the collection exists in the `published` repository.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

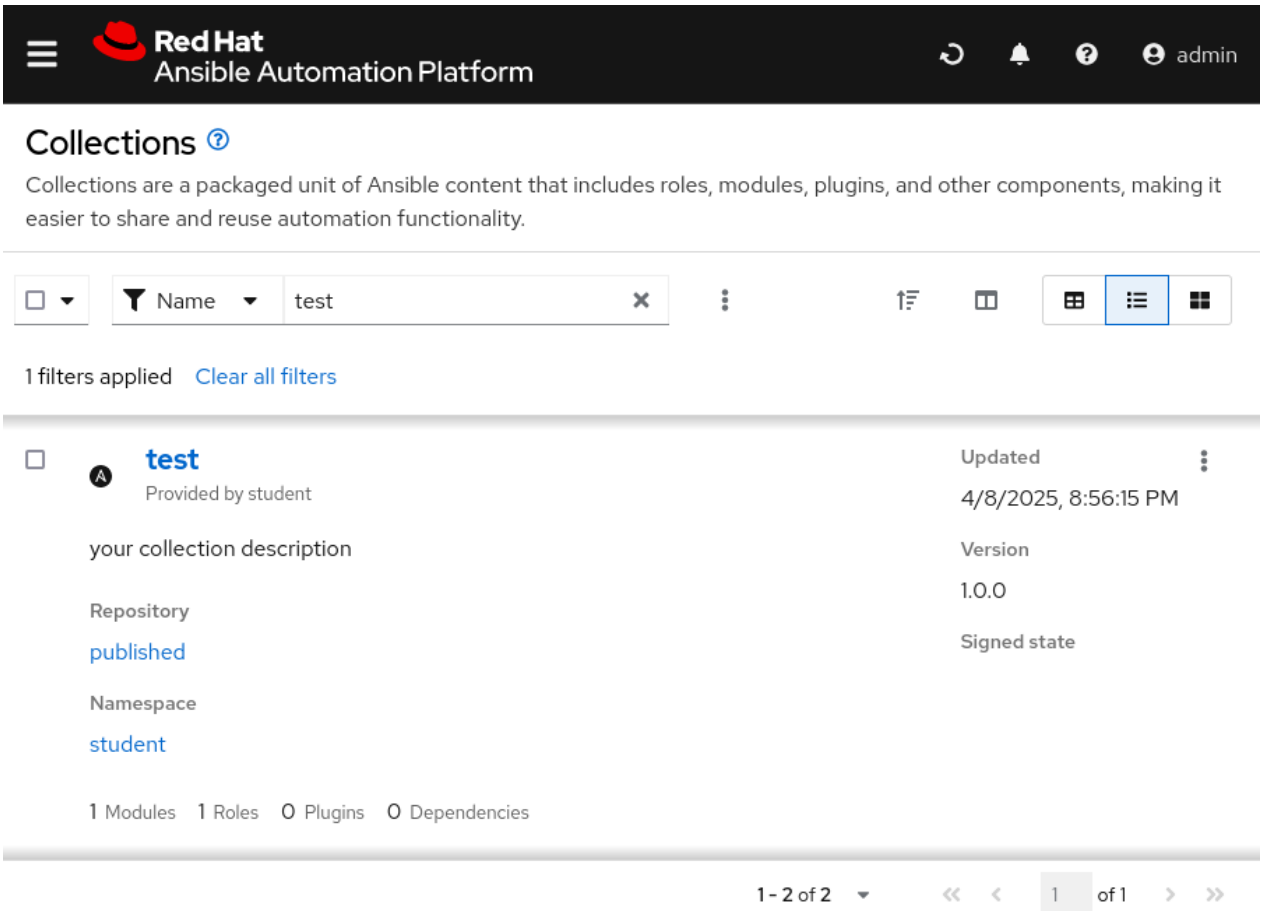


Figure 40. Verify published collection

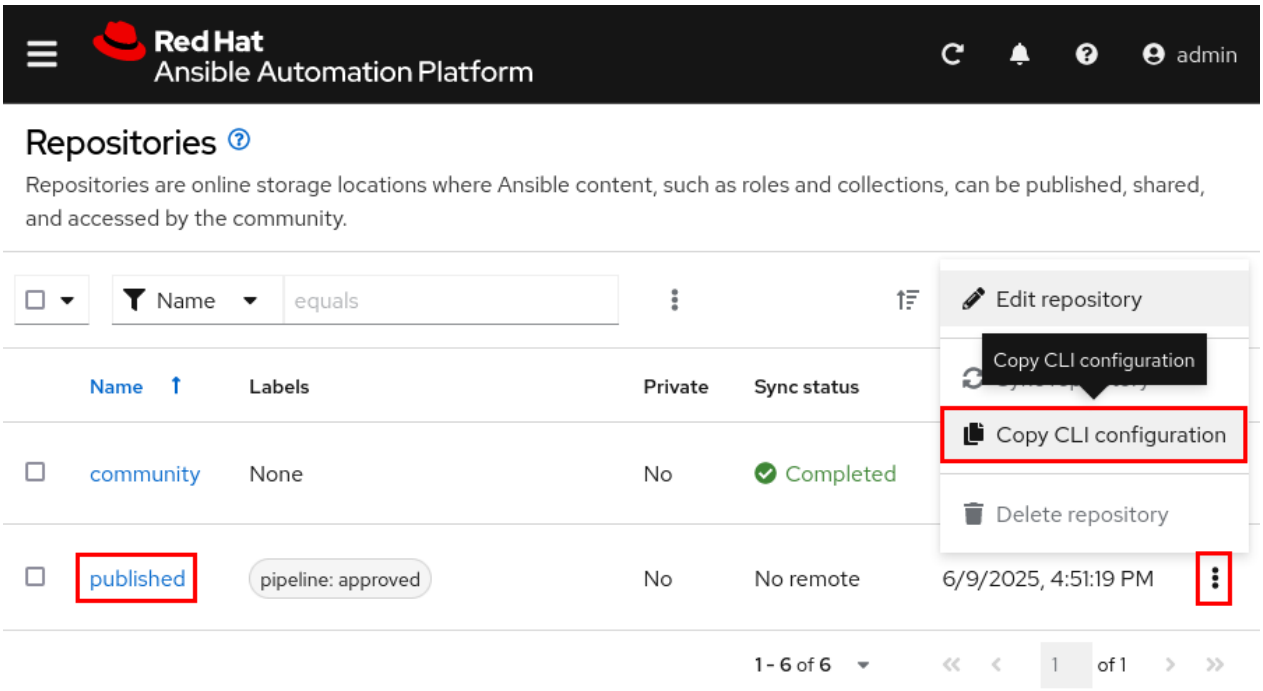
6. Use the `ansible-galaxy` command to install the `student.test` collection from private automation hub.

   6.1. Log out of the unified UI, and then log back in as `student` using `student` as the password.

   6.2. Go to **Automation Content > Repositories**.

   From the `published` row, click the menu icon at the end of the row and select **Copy CLI configuration**.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use



Do not close your web browser window.

6.3. Change to the `/home/student/create-writing` directory:

```
[student@workstation student.test]$ cd ~/create-writing/
[student@workstation create-writing]$
```

6.4. Modify the `ansible.cfg` file to append the lines that you copied to the clipboard in the preceding step. The modified `ansible.cfg` file now contains the following content:

```
[defaults]
inventory = ./inventory
remote_user = devops
collections_path = ./collections:/usr/share/ansible/collections

[galaxy]
server_list = published

[galaxy_server.published]
url=http://localhost:5001/api/galaxy/
token=<put your token here>
```

6.5. Update the `ansible.cfg` file to use the correct URL for the classroom private automation hub. The modified `ansible.cfg` file now contains the following content:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[defaults]
inventory = ./inventory
remote_user = devops
collections_path = ./collections:/usr/share/ansible/collections

[galaxy]
server_list = published

[galaxy_server.published]
url=https://aap.lab.example.com/api/galaxy/
token=<put your token here>
```

6.6. Return to the unified UI. Go to **Automation Content > API Token** and then click **Generate token**. Copy the API token.

6.7. Using the copied token, update the `token` line in the `ansible.cfg` file. Your token is different from the one displayed in this example. Save and close the file when done:

```
[defaults]
inventory = ./inventory
remote_user = devops
collections_path = ./collections:/usr/share/ansible/collections

[galaxy]
server_list = published

[galaxy_server.published]
url=https://aap.lab.example.com/api/galaxy/
token=24e23ebe16403c8f0832fb8ed5f44b07e49e5d6e
```

6.8. The project includes a `collections/requirements.yml` file that references the `student.test` collection. Review the file to confirm that the contents are as follows:

```
---
collections:
  - name: student.test
```

6.9. Install the `student.test` collection using the `ansible-galaxy` command. The collection is installed in the project `collections` directory. The entire project directory, including the `collections` directory, is loaded into the automation execution environment.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation create-writing]$ ansible-galaxy collection \
install -r collections/requirements.yml -p collections/
...output omitted...
student.test:1.0.0 was installed successfully
```

7. Test the `student.test` collection by adding one or more firewall rules to `serverd.lab.example.com` and then run the `test_collection.yml` playbook to ensure that the `student.test.reset_firewall` role removes the added rules.

   7.1. Review the `allow_firewall_svc.yml` playbook. This playbook allows immediate and persistent access through the firewall for the service defined by the `firewall_svc` variable.

```
---
- name: Set firewall rules
  hosts: serverd.lab.example.com
  become: true
  gather_facts: false
  vars:
    fierwall_svc: http
  tasks:
    - name: Allow firewall access
      ansible.posix.firewalld:
        service: "{{ fierwall_svc }}"
        state: enabled
        immediate: true
        permanent: true
        zone: public
```

   7.2. Run the `allow_firewall_svc.yml` playbook:

```
[student@workstation create-writing]$ ansible-navigator run \
allow_firewall_svc.yml
...output omitted...
```

   7.3. Review the `query_firewall.yml` playbook:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
- name: Query and display allowed firewall services for active zones
  hosts: serverd.lab.example.com
  become: true
  gather_facts: false
  tasks:
    - name: Query active firewall zones
      ansible.posix.firewalld_info:
        active_zones: true
      register: active_zone_info

    - name: Show enabled services in public zone
      ansible.builtin.debug:
        var: active_zone_info['firewalld_info']['zones']['public']
['services']
```

7.4. Run the `query_firewall.yml` playbook to display a list of services allowed through the `public` zone:

```
[student@workstation create-writing]$ ansible-navigator run
query_firewall.yml
...output omitted...
TASK [Show enabled services in public zone]
***********************************
ok: [serverd.lab.example.com] => {
    "active_zone_info['firewalld_info']['zones']['public']['services']": [
        "ssh",
        "dhcpv6-client",
        "cockpit",
        "http"
    ]
}
...output omitted...
```

7.5. Review the `test_collection.yml` playbook:

```
---
- name: Test custom collection
  hosts: serverd.lab.example.com
  become: true
  gather_facts: false
  roles:
    - role: student.test.reset_firewall
```

7.6. Run the `test_collection.yml` playbook to test the `student.test.reset_firewall` role:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation create-writing]$ ansible-navigator run
test_collection.yml
...output omitted...
TASK [student.test.reset_firewall : Remove files in /etc/firewalld/zones]
******
changed: [serverd.lab.example.com] => (item=/etc/firewalld/zones/public.xml)
...output omitted...
RUNNING HANDLER [student.test.reset_firewall : Restart firewalld]
**************
changed: [serverd.lab.example.com]
...output omitted...
```

7.7. Run the `query_firewall.yml` playbook to verify that the `student.test.reset_firewall` role removed the added firewall service. The displayed services are the default services allowed by the `public` zone.

```
[student@workstation create-writing]$ ansible-navigator run
query_firewall.yml
...output omitted...
TASK [Show enabled services in public zone]
**********************************
ok: [serverd.lab.example.com] => {
    "active_zone_info['firewalld_info']['zones']['public']['services']": [
        "ssh",
        "dhcpv6-client",
        "cockpit"
    ]
}
...output omitted...
```

Finish

On the `workstation` machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish create-writing
```


For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

9.3. Building a Custom Automation Execution Environment

Objectives

- Build a custom automation execution environment image by using the `ansible-builder` command.

Deciding When to Create a Custom Automation Execution Environment

Red Hat provides several Ansible automation execution environments that suit the needs of many users. These automation execution environments include the most common Ansible Content Collections.

You can use the `ansible-navigator images` command to inspect a container image and view the collections, Python packages, and operating system packages it includes.

```
 Image: ee-supported-rhel8:latest Description
0|Image information               Information collected from image inspection
1|General information             OS and python version information
2|Ansible version and collections Information about ansible and ansible collect
3|Python packages                  Information about python and python packages
4|Operating system packages       Information about operating system packages
5|Everything                      All image information
```

The automation execution environments that Red Hat provides might be sufficient for your playbooks. In that case, you do not need to create custom automation execution environments.

Sometimes, you might need an Ansible Content Collection that is not included in one of the existing automation execution environments. Often, you can install these extra collections in your Ansible project and then use an existing automation execution environment without having to build a new one.

On the other hand, you should consider creating a custom automation execution environment in the following situations:

- You often reuse specific collections that are not included in the existing automation execution environments. It is more efficient to embed those collections in a custom automation execution environment than to install them from the project `requirements.yml` file every time you run a playbook in that project. This is especially true if you have many projects that use that collection.
- The collection you want to use requires Python packages or software that is not included in an existing automation execution environment.
- You need to use a collection that conflicts with another collection in your existing automation execution environments.

Preparing for a New Automation Execution Environment

You use the `ansible-builder` command to create the container images used for automation execution environments. The `ansible-builder` RPM package provides that command.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[root@host ~]# dnf install ansible-builder
```

Create a working directory to prepare the files that you need to build the automation execution environment container image. The `ansible-builder` command searches this working directory for its `execution-environment.yml` configuration file, which it uses to determine how to build the container image.

This is an example `execution-environment.yml` file:

```
---
version: 3 (1)

images: (2)
  base_image:
    name: aap.lab.example.com/ee-minimal-rhel8:latest

dependencies: (3)
  galaxy: requirements.yml (4)
  python: requirements.txt (5)
  system: bindep.txt (6)

options: (7)
  package_manager_path: /usr/bin/microdnf

additional_build_files: (8)
  - src: ansible.cfg
    dest: conf.d/
  - src: private_automation_hub-ca.pem
    dest: certs/

additional_build_steps: (9)
  prepend_galaxy: (10)
    - COPY _build/conf.d/ansible.cfg /home/runner/.ansible.cfg
    - COPY _build/certs/private_automation_hub-ca.pem /etc/pki/ca-
trust/source/anchors
    - RUN update-ca-trust
```

(1) The `version` parameter specifies the automation execution environment definition version to use. Ansible Automation Platform 2.5 uses version 3 of the definition. If you do not specify a version, then Ansible Builder defaults to using version 1.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

(2) The `images` parameter specifies the base image to use when building the custom automation execution environment. This parameter requires that you specify at least the source image and tag.

> **Important**
> You are not allowed to publicly distribute container images that you build on top of the automation execution environment images that Red Hat provides with Red Hat Ansible Automation Platform.
>
> To distribute your images on a public platform, such as Quay.io, use the public `quay.io/ansible/ansible-runner:stable-2.12-latest` base image instead. Alternatively, you can publish your `execution-environment.yml` file and let users build the container image themselves.
>
> Red Hat only supports container images that you build on top of the Red Hat images.

(3) The `dependencies` parameter can have multiple attributes to include content collections, roles, Python packages, and system packages in the custom automation execution environment. You can define these dependencies inline, or you can provide requirement files.

(4) The `galaxy` parameter points to the file that lists the collections to install inside the automation execution environment. Ansible users usually set the file name to `requirements.yml`.

(5) The `python` parameter points to the file that lists the Python packages to install. Some collections include a `requirements.txt` file. If you install such a collection, then you do not need to list its Python dependencies. Python developers usually set the file name to `requirements.txt`.

(6) The `system` parameter points to the file that lists the RPM packages to install. Some collections include a `bindep.txt` file.

(7) The `options` parameter has available attributes that can affect the builder runtime functionality, such as setting the path to the package manager, changing the default working directory for new processes to start, and changing the ENTRYPOINT and CMD directives of the custom automation execution environment. The `package_manager_path` key is set the `/usr/bin/dnf` by default, which is not present in the automation execution environment images that Red Hat provides with Red Hat Ansible Automation Platform.

(8) The `additional_build_files` parameter specifies files to add to the build `context` directory. This parameter is useful for providing an `ansible.cfg` file or certificate authority certificates to the custom automation execution environment. The destination for each file must be a directory and the location is relative to the `context/_build/` directory.

(9) The `additional_build_steps` parameter specifies custom build commands inserted directly into the `Containerfile` file for the container runtime. This parameter can be used to copy an `ansible.cfg` file from the build `context` directory into the custom automation execution environment so that it can access automation hub or private automation hub.

(10) The `prepend_galaxy` key for the `additional_build_steps` parameter specifies that the build commands be inserted before building the galaxy image. Some other valid keys include `prepend_base`, which inserts build commands before the base image is built, and `append_final`, which insets build commands after the final image is built. Review the references at the end of this section for a list of all the available keys.

The local `ee_schema.py` file, provided by the `ansible-builder` RPM, contains information about the different Ansible Builder schema versions. Run the following command to identify the location of that file on your system. You might find that file useful when you create your own `execution-environment.yml` files.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[user@host ~]$ rpm -ql ansible-builder | grep ee_schema.py
/usr/lib/python3.11/site-packages/ansible_builder/ee_schema.py
```

Declaring the Ansible Content Collections to Install

To install collections inside an automation execution environment, create a `requirements.yml` file and list the required collections.

The following example is a minimal `requirements.yml` file that instructs the `ansible-builder` command to install the `community.aws` and `community.general` collections from Ansible Galaxy:

```
---
collections:
  - community.aws
  - community.general
```

The following example is a more complex `requirements.yml` file:

```
---
collections:
  - name: redhat.insights (1)
    version: 1.3.0 (2)
    source: https://console.redhat.com/api/automation-hub/ (3)
  - name: ansible.posix
    source: https://aap.lab.example.com/pulp_ansible/galaxy/rh-certified/
```

(1) The name of the collection.
(2) The version of the collection. If not specified, then the `ansible-builder` command selects the latest version.
(3) The distribution platform that provides the collection. If the distribution platform is automation hub or private automation hub, then you must create an `ansible.cfg` file and populate it with the token used to access that server. Then you must copy that file to `/home/runner/.ansible.cfg` or to `/etc/ansible/ansible.cfg` in the container image.

The following example `ansible.cfg` file provides the parameters that the `ansible-builder` command needs to authenticate to automation hub and to private automation hub (at `aap.lab.example.com` in this example):

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[galaxy]
server_list = automation_hub, my_hub, galaxy

[galaxy_server.automation_hub]
url=https://console.redhat.com/api/automation-hub/
auth_url=https://sso.redhat.com/auth/realms/redhat-external/protocol/openid-
connect/token
token=eyJh...0Nwk

[galaxy_server.my_hub]
url=https://aap.lab.example.com/api/galaxy/content/rh-certified/
token=e8e4...b0c2

[galaxy_server.galaxy]
url=https://galaxy.ansible.com/
```

Declaring Python Packages

In the `requirements.txt` file, list the Python packages that `ansible-builder` must install inside the automation execution environment.

The following example `requirements.txt` file lists some required Python packages. For each package, you can provide a specific version to install.

```
sh==2.2.2
jsonschema>=3.2.0,<4.0.1
textfsm
ttp
xmltodict
dnspython
```

Some collections include a `requirements.txt` file. In that case, you do not have to duplicate that file in your `requirements.txt` file.

Declaring RPM Packages

In the `bindep.txt` file, list the RPM packages that `ansible-builder` must install inside the automation execution environment.

The following `bindep.txt` file lists the RPM packages required by the preceding examples:

```
rsync [platform:rpm]
kubernetes-client [platform:rpm]
```

The `bindep` tool, which processes the `bindep.txt` file, can manage packages for several Linux distributions. Different distributions might have other package names for the same software. In the preceding file, the

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

`[platform:rpm]` directive targets all the Linux distributions that use the RPM packaging system.

Some collections include a `bindep.txt` file. In that case, you do not have to duplicate that file in your `bindep.txt` file.

If the system on which you run the build process has a valid subscription, then the `ansible-builder` command has access to the same RPM packages as that system. However, if you do not register your system with Red Hat, then the build process only has access to the Red Hat Universal Base Images (UBI) repositories. These repositories are publicly available but only provide a limited set of RPM packages. The packages you define in the `bindep.txt` file might not be available in these repositories.

Working in Disconnected Environments

When used as a base image, the `ee-minimal-rhel8` and `ee-supported-rhel8` automation execution environments (and their RHEL 9 equivalents) default to using https://pypi.org for Python dependencies and UBI repositories for RPM dependencies. You can override this behavior if you work in a disconnected environment.

Specifying a Python Package Index Server

You can configure your `execution-environment.yml` file to point to a different Python Package Index (PyPi) server. To do this, add list items to the `prepend_base` key of the `additional_build_steps` parameter.

```
additional_build_steps:
  prepend_base: (1)
    - ENV PIP_INDEX_URL="http://workstation.lab.example.com:8000/simple/" (2)
    - ENV PIP_TRUSTED_HOST="workstation.lab.example.com:8000" (3)
```

(1) The `base` stage is the first stage of the build process. Steps added here are applied to other build stages.
(2) Set the `PIP_INDEX_URL` variable to the location of an alternative PyPi server.
(3) Set the `PIP_TRUSTED_HOST` variable to indicate that you trust the alternative server.

Specifying an RPM Repository

You can configure your `execution-environment.yml` file to point to a different RPM repository. Copy your custom repository file to the build environment, disable any existing repositories, and then copy your custom repository file to the `/etc/yum.repos.d` directory:

```
additional_build_files:
  - src: custom.repo (1)
    dest: ./

additional_build_steps:
  prepend_base:
    - RUN sed -i "s/^enabled.*$/enabled = 0/" /etc/yum.repos.d/*.repo (2)
    - COPY _build/custom.repo /etc/yum.repos.d/custom.repo (3)
```

(1) Add your custom repository file to the build environment. This file can be any file ending with the `.repo` extension.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

2. You might disable existing repositories so that the build process only uses your custom repository.
3. Copy your custom repository file to the `/etc/yum.repos.d` directory.

> **Important**
> If the system on which you run the build process has a valid subscription (and you do not want to use Red Hat repositories), then you might need to add a build step to the `prepend_base` key with content similar to the following step. Setting the PKGMGR_OPTS variable in your `execution-environment.yml` file overrides the PKGMGR_OPTS variable used by scripts in the `context/_build/scripts` directory.
>
> ```
> - ENV PKGMGR_OPTS="--nodocs --setopt install_weak_deps=0 --disablerepo=rhel-8-
> for-x86_64-baseos-rpms --disablerepo=rhel-8-for-x86_64-appstream-rpms"
> ```

Building a New Automation Execution Environment

The execution environment builder automatically pulls the base and the builder images if they are not already available locally. If you use images from a container registry that requires authentication, then you must authenticate before starting the build process.

The following example shows how to log in to the Red Hat container registry that hosts the default images, `ee-minimal-rhel8` and `ansible-builder-rhel8`:

```
[user@host demo]$ podman login registry.redhat.io
```

After you have prepared your configuration files and authenticated to the container registry, use the `ansible-builder build` command to create your automation execution environment container image.

Use the `--tag` (or `-t`) option to provide a name for the container image. The name has two parts: a name and an optional tag. For example, use the `--tag demo:v1.0` option to name the container demo and give it the `v1.0` tag. The tag defaults to `latest` if you do not specify it. For example, using the `--tag demo` option is the equivalent of using `--tag demo:latest`. Optionally, use the `--verbosity 3` option to increase the verbosity of the output.

A successful build creates a container image.

```
[user@host demo]$ ansible-builder build --tag ee-demo:v1.0
Running command:
  podman build -f context/Containerfile -t ee-demo:v1.0 context
Complete! The build context can be found at: /home/user/demo/context
```

Use the `ansible-navigator images` command to display the automation execution environment images:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
  Image                Tag     Execution environment     Created        Size
0|ee-demo              1.0     True                      2 hours ago    744 MB
1|ee-minimal-rhel8     latest  True                      4 months ago   341 MB
2|ee-supported-rhel8   latest  True                      4 months ago   2.24 GB

^b/PgUp page up  ^f/PgDn page down  ↑↓ scroll  esc back  [0-9] goto  :help help
```

Note

You can also use the `podman images` command to display local container images.

```
[user@host demo]$ podman images
REPOSITORY                          TAG     IMAGE ID      CREATED      SIZE
localhost/ee-demo                   v1.0    9e1e281437a2  2 hours ago  744 MB
...output omitted...
```

Interacting with the Build Process

The inclusion of the `additional_build_files` and `additional_build_steps` parameters in the version 3 schema of execution environment builder should provide sufficient customization for most use cases.

The execution environment builder operates in two stages. You can run both stages by running the `ansible-builder build` command or you can run the first stage separately by running the `ansible-builder create` command.

1. Both commands create the `context/` directory in the current directory.
   - Both commands create the `context/Containerfile` file, the equivalent of a `Dockerfile` file, which contains instructions for the `podman build` command. If you define the `additional_build_steps` parameter, then these commands add those instructions to the appropriate locations in the `Containerfile` file.
   - Both commands create the `context/_build/` directory and then copy your `requirements.yml`, `requirements.txt`, and `bindep.txt` files into that directory so that the build process can access them. If you define the `additional_build_files` parameter, then additional files might be copied to the `context/_build/` directory, such as an `ansible.cfg` file.
2. The `ansible-builder build` command then runs the `podman build` command, which constructs the resulting automation execution environment container image.

To prepare for customizing the build process, use the `ansible-builder create` command to perform the first stage:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[user@host demo]$ ls
ansible.cfg bindep.txt execution-environment.yml requirements.txt requirements.yml
[user@host demo]$ ansible-builder create
Complete! The build context can be found at: /home/user/demo/context
```

After running the `ansible-builder create` command, you can list the content of the `context/` directory. If files do not exist where you expect them to be, then you can modify the `execution-environment.yml` configuration file and run the command again.

```
[user@host demo]$ tree
.
├── ansible.cfg
├── bindep.txt
├── context
│   ├── Containerfile
│   ├── _build
│   │   ├── ansible.cfg
│   │   ├── bindep.txt
│   │   ├── requirements.txt
│   │   └── requirements.yml
├── execution-environment.yml
├── requirements.txt
└── requirements.yml

2 directories, 10 files
```

Modifying the Container Configuration File

If the `context/Containerfile` file does not contain the correct instructions, consider modifying the `additional_build_steps` parameter in the `execution-environment.yml` file and then run the `ansible-builder create` command again.

> **Important**
> Manually editing the `context/Containerfile` file is an option, but the `ansible-builder create` command always overwrites that file, which means that any manual customizations are lost.

If you must modify the `context/Containerfile`, consider adding that file to version control so that you do not lose your customizations.

Building the Execution Environment Image

You can run both stages of the build process by running the `ansible-navigator build` command. That command displays an equivalent `podman build` command that you can use to only run the second stage of the build process.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[user@host demo]$ ansible-navigator build -t ee-demo:v2.0
Running command:
  podman build -f context/Containerfile -t ee-demo:v2.0 context (1)
...output omitted...
```

(1) Use the equivalent `podman build` command if you manually updated the `context/Containerfile` file.

After the build succeeds, verify that the local container image exists:

```
[user@host demo]$ podman images
REPOSITORY                      TAG     IMAGE ID      CREATED         SIZE
localhost/ee-demo               v2.0    ba77b0d0a59d  2 minutes ago   308 MB
...output omitted...
```

Inspecting Automation Execution Environments

After creating a new automation execution environment, you can use the `ansible-navigator images` command to interactively inspect the automation execution environment container image. You can also use the `ansible-navigator collections` command to specifically view the collections inside the image.

When using the `ansible-navigator collections` command, you must specify the local image with the `--eei` option and set the `--pp` option to either `never` or `missing`. The command fails if it unsuccessfully attempts to download the local automation execution environment.

```
[user@host demo]$ ansible-navigator collections --eei localhost/ee-demo:v2.0 \
--pp never
```

References

Using Ansible Builder

Knowledgebase: Universal Base Images (UBI): Images, Repositories, Packages, and Source Code

Introduction - Ansible Builder

Execution Environment Definition

Introduction to `bindep`

Building Custom EEs With ansible-builder

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

9.4. Guided Exercise
Building a Custom Automation Execution Environment

Create a custom automation execution environment that provides a specific Ansible Content Collection and its supporting software.

Outcomes

- Create the configuration files needed for a custom automation execution environment.
- Build a new automation execution environment.
- Verify that the new automation execution environment contains the specified Ansible Content Collection.

Prerequisites

As the `student` user on the `workstation` machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
[student@workstation ~]$ lab start create-custom
```

Instructions

1. Review some files in the `exercise.motd` custom Ansible Content Collection. This collection contains a role that updates the Message of the Day file that is displayed when users log in.

   1.1. From a terminal, create the `/home/student/git-repos` directory if it does not already exist, and then change into it:

   ```
   [student@workstation ~]$ mkdir -p ~/git-repos/
   [student@workstation ~]$ cd ~/git-repos/
   [student@workstation git-repos]$
   ```

   1.2. Clone the `https://git.lab.example.com/student/exercise.motd.git` repository and then change to the cloned repository:

   ```
   [student@workstation git-repos]$ git clone \
   https://git.lab.example.com/student/exercise.motd.git
   Cloning into 'exercise.motd'...
   ...output omitted...
   [student@workstation git-repos]$ cd exercise.motd
   [student@workstation exercise.motd]$
   ```

   1.3. Use the `tree` command to list the files in the working directory:

```
[student@workstation exercise.motd]$ tree
.
├── README.md
├── galaxy.yml
├── meta
│   └── runtime.yml
├── plugins
│   └── README.md
└── roles
    └── banner
        ├── README.md
        ├── defaults
        │   └── main.yml
        ├── meta
        │   └── main.yml
        ├── tasks
        │   └── main.yml
        ├── templates
        │   └── motd.j2
        └── vars
            └── main.yml

9 directories, 10 files
```

1.4. Display the content of the `roles/banner/templates/motd.j2` template file. The `banner` role uses this template to create the `/etc/motd.d/banner` file.

```
================================================================
================================================================
==                                                            ==
{% for LINE in formatted_lines %}
{{ LINE }}
{% endfor %}
==                                                            ==
================================================================
================================================================
```

1.5. Display the content of the `roles/banner/tasks/main.yml` task file in the `banner` role:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
---
# tasks file for banner
- name: Create /etc/motd.d
  ansible.builtin.file:
    path: /etc/motd.d
    state: directory
    owner: root
    group: root
    mode: "0755"

- name: Set formatted_lines
  vars:
    left_pad: >-
      {{ (width - (item | length)) // 2 }}
    right_pad: >-
      {{ left_pad | int + ((width - (item | length)) % 2) }}
    formatted_line: >-
      =={% for x in range((left_pad | int) -2) %} {% endfor %}
      {{ item }}
      {% for y in range((right_pad | int) -2) %} {% endfor %}==
    line_item:
      - "{{ formatted_line }}"
  ansible.builtin.set_fact:
    formatted_lines: >-
      {{ formatted_lines + line_item }}
  loop: "{{ motd_msg | trim | split(line_break) }}"

- name: Create /etc/motd.d/banner
  ansible.builtin.template:
    src: motd.j2
    dest: /etc/motd.d/banner
    owner: root
    group: root
    mode: "0644"
```

The `banner` role uses the `width` and `motd_msg` variables to align a custom message in the center of the `/etc/motd.d/banner` file. When using the role, users might override the default value of the `motd_msg` variable.

2. Create the configuration files for the new automation execution environment.

> **Note**
> The `/home/student/create-custom/solutions` directory contains configuration files that you can use for comparison.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

2.1. Change to the `~/create-custom` directory:

```
[student@workstation exercise.motd]$ cd ~/create-custom/
[student@workstation create-custom]$
```

2.2. Create the `execution-environment.yml` file with the following content:

```
---
version: 3

images:
  base_image:
    name: aap.lab.example.com/ee-minimal-rhel8:latest

dependencies:
  galaxy: requirements.yml
  python: requirements.txt
  system: bindep.txt

options:
  package_manager_path: /usr/bin/microdnf

additional_build_files:
  - src: ee.repo
    dest: config/
  - src: classroom-ca.pem
    dest: certs/

additional_build_steps:
  prepend_base:
    - ENV PIP_INDEX_URL="http://workstation.lab.example.com:8000/simple/"
    - ENV PIP_TRUSTED_HOST="workstation.lab.example.com:8000"
    - RUN sed -i "s/^enabled.*$/enabled = 0/" /etc/yum.repos.d/*.repo
    - COPY _build/config/ee.repo /etc/yum.repos.d/ee.repo
    - COPY _build/certs/classroom-ca.pem /etc/pki/ca-trust/source/anchors
    - RUN update-ca-trust
```

The `lab` command downloaded the `aap.lab.example.com/ee-minimal-rhel8` container image to the `workstation` machine. This container image can be used as a base image for additional customizations.

2.3. Create the `requirements.yml` file with the following content:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
---
collections:
  - name: https://git.lab.example.com/student/exercise.motd
    type: git
```

> **Important**
> Because the required collection is available from a public Git repository in the classroom environment, you do not need to provide a custom `ansible.cfg` file to the build process.

2.4. Create the `requirements.txt` file for the Python package requirements with the following content:

```
# Python dependencies
funmotd
```

2.5. Create the `bindep.txt` file to specify the system packages to install:

```
# System-level dependencies
hostname
```

2.6. Copy the classroom certificate authority certificate to the current directory:

```
[student@workstation create-custom]$ cp /etc/pki/tls/certs/classroom-ca.pem
.
```

3. Create the custom automation execution environment.

3.1. Verify that `ansible-builder` is installed:

```
[student@workstation create-custom]$ rpm -q ansible-builder
ansible-builder-3.1.0-1.el9ap.noarch
```

> **Note**
> If `ansible-builder` is not installed, install the `ansible-builder` package. The password for the `student` user is `student`.
>
> ```
> [student@workstation create-custom]$ sudo dnf install ansible-builder
> ```

3.2. Use the `ansible-builder build` command to build the custom automation execution environment. Label the resulting container image with the `ee-motd-minimal:1.0` tag. The `ansible-builder build` command might take a few minutes to complete.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation create-custom]$ ansible-builder build \
-t ee-motd-minimal:1.0
Running command:
  podman build -f context/Containerfile -t ee-motd-minimal:1.0 context
Complete! The build context can be found at: /home/student/create-
custom/context
```

Important

The `ansible-builder build` command displays an associated `podman build` command. Use the `podman build` command if you make customizations to the `context/Containerfile` file that cannot be made by modifying the `execution-environment.yml` file.

You can add the `-v` option, such as `-v3`, to display the output of the build steps.

3.3. Use the `podman images` command to list local container images. The image ID for your container might be different from the one displayed.

```
[student@workstation create-custom]$ podman images localhost/ee-motd-minimal
REPOSITORY                  TAG         IMAGE ID      CREATED         SIZE
localhost/ee-motd-minimal   1.0         44ebd37ea04c  2 minutes ago   353 MB
```

4. Use the `ansible-navigator images` command to examine the `ee-motd-minimal` container image.

4.1. Verify that `ansible-navigator` is installed:

```
[student@workstation create-custom]$ rpm -q ansible-navigator
ansible-navigator-24.9.0-1.el9ap.noarch
```

Note

If the `ansible-navigator` package is not installed, install it:

```
[student@workstation create-custom]$ sudo dnf install ansible-navigator
```

4.2. Use `ansible-navigator` command to verify that the custom automation execution environment exists locally. Specify the name of the automation execution environment image as `localhost/ee-motd-minimal` with the tag `1.0`, and use `never` as the image pull policy:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation create-custom]$ ansible-navigator images --pp never \
--eei localhost/ee-motd-minimal:1.0 -m interactive
...output omitted...
```

4.3. Locate the `ee-motd-minimal` image and type the number associated with that line:

```
  Image                  Tag       Execution environment   ...
0|ee-minimal-rhel8       latest    True                    ...
1|ee-motd-minimal        1.0       True                    ...
2|ee-supported-rhel8     latest    True                    ...
```

4.4. Type 2 to select the `Ansible version and collections` line:

```
  Image: ee-motd-minimal:1.0 (primary)  Description
0|Image information                     Information collected from image
insp...
1|General information                   OS and python version information
2|Ansible version and collections       Information about ansible and
ansible...
3|Python packages                       Information about python and python
p...
4|Operating system packages             Information about operating system
pa...
5|Everything                            All image information
```

4.5. The resulting output indicates that the automation execution environment contains the `exercise.motd` content collection.

```
Image: ee-motd-minimal:1.0 (primary) (Information about ansible and ansible
...
0|---
1|ansible:
2|  collections:
3|    details:
4|      exercise.motd: 1.0.0
5|  version:
6|    details: ansible [core 2.16.14]
```

4.6. Press Esc to go back one level, and then type 3 to select the `Python packages` line:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
  Image: ee-motd-minimal:1.0 (primary)  Description
0|Image information                     Information collected from image
insp...
1|General information                   OS and python version information
2|Ansible version and collections       Information about ansible and
ansible...
3|Python packages                       Information about python and python
p...
4|Operating system packages             Information about operating system
pa...
5|Everything                            All image information
```

4.7. Verify that the `funmotd` package is in the list of installed Python packages:

```
   Name             Version   Summary
 0|ansible-compat   24.9.0    Ansible compatibility goodies
 1|ansible-core     2.16.14   Radically simple IT automation
...output omitted...
18|funmotd          1.1       TV Show and Movie Quotes MOTD for Terminal
...output omitted...
```

4.8. Press Esc to go back one level, and then type 4 to select the `Operating system packages` line:

```
  Image: ee-motd-minimal:1.0 (primary)  Description
0|Image information                     Information collected from image
insp...
1|General information                   OS and python version information
2|Ansible version and collections       Information about ansible and
ansible...
3|Python packages                       Information about python and python
p...
4|Operating system packages             Information about operating system
pa...
5|Everything                            All image information
```

4.9. Verify that the `hostname` RPM package you specified as a dependency in the `bindep.txt` file is included in the container image:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
...output omitted...
 38│grep       3.1      Pattern matching utilities
 39│gzip       1.9      The GNU data compression program
 40│hostname   3.20     Utility to set/show the host name or domain name
 41│info       6.5      A stand-alone TTY-based reader for GNU texinfo
docu...
 42│json-c     0.13.1   JSON implementation in C
...output omitted...
```

Type `:q` and press `Enter` to exit the `ansible-navigator` command when finished.

Finish

On the `workstation` machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish create-custom
```

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

9.5. Validating a Custom Automation Execution Environment

Objectives

- Validate that a custom automation execution environment works as expected by testing it with the `ansible-navigator` command, and then distribute the automation execution environment for reuse.

Testing Automation Execution Environments Locally

After the `ansible-builder` command successfully creates an automation execution environment, it is important to validate that it works correctly.

To ensure that the automation execution environment uses its internal collections, you can remove locally installed collections from your Ansible project directory. Use the `ansible-galaxy` command to list installed collections:

```
[user@host myproject]$ ansible-galaxy collection list

# /home/user/myproject/collections/ansible_collections
Collection         Version
-----------------  -------
ansible.controller 4.6.3
community.crypto   2.24.0
community.general  10.3.0
```

In addition to showing the installed collections, the output displays where the collections are installed. Recursively removing the `ansible_collections` directory removes the locally installed collections.

```
[user@host myproject]$ rm -rf collections/ansible_collections
```

Running a Test Playbook

If your new automation execution environment uses custom collections, then create a playbook that tests one or more of those collections. The test might use a simple playbook to test a role or a module included in the collection.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
---
- name: Testing a custom Ansible Content Collection
  hosts: all
  roles:
    - role: mynamespace.mycollection.my_role

  tasks:
    - name: Ensure my custom module works
      mynamespace.mycollection.my_module:
```

Important

When using automation execution environments, `localhost` is the container itself rather than the machine from which you run the `ansible-navigator` command. Playbooks originally written for the `ansible-playbook` command that target `localhost` might not work as intended.

If you want plays or tasks to target your local machine, then define your local machine in the inventory file and modify the plays and tasks to point to that machine name.

For playbooks that you intend to run locally as well as on automation controller, you can use `hosts: all` to target all inventory hosts. Then you can create an inventory that only contains the desired hosts.

Use the `ansible-navigator` command to run your test playbook using the new automation execution environment.

- Use the `-i` option to specify an inventory file or directory.
- Use the `--eei` option to specify the automation execution environment container image.
- If the container image exists locally, then use the `--pp never` option. Without that option, and if the image tag is `latest`, then the `ansible-navigator` command attempts to pull the container image from a container registry, such as the Red Hat Ecosystem Catalog or Docker Hub.
- If the playbook requires privilege escalation and does not use `become: true`, then add the `-b` option to enable privilege escalation.
- Optionally add the `-m stdout` option to redirect the playbook output to your terminal window. Alternatively, run the playbook in the default interactive mode:

```
[user@host myproject]$ ansible-navigator run test_playbook.yml -i inventory \
--eei localhost/ee-demo:v2.0 --pp never -b
...output omitted...
```

If the playbook succeeds, then you can share the automation execution environment. If the playbook does not succeed, then investigate any reported issues.

Providing Authentication Credentials

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

When Ansible runs a play on a managed host, it generally must provide authentication credentials to that host to log in and make changes.

For Linux-based managed hosts, one common solution is to install the user account specified by the Ansible `remote_user` directive with the public SSH key that matches a private SSH key available to the automation execution environment.

Automation Controller Machine Credentials

Automation controller can store your private SSH key in a *machine credential*, a special resource that can be updated but not read from the unified UI. If you use password-based SSH authentication, it can instead store an SSH username and password. You can provide the machine credential in the playbook's job template so that the credential is automatically passed to the automation execution environment when automation controller runs the playbook. The machine credential can also store a password used for privilege escalation.

Automation Content Navigator Authentication

If you connect to managed hosts using SSH keys, then automation content navigator can use those keys.

You can also use a username and password for password-based SSH authentication, but by default, `ansible-navigator` does not prompt you for passwords or other interactive input. This means that normally you cannot use options that prompt you for such things as a connection password (`--ask-pass`), a privilege escalation password (`--ask-become-pass`), or vault passwords (`--ask-vault-pass`).

To enable the `ansible-navigator` command to support interactive prompts for passwords, you can either use the `--pae false` and `-m stdout` options, or you can edit your `ansible-navigator.yml` configuration file and add the following sections:

```
---
ansible-navigator:
  mode: stdout
  playbook-artifact:
    enable: false
```

Sharing an Automation Execution Environment from Private Automation Hub

The `ansible-builder` command creates automation execution environment container images on your local system. To share these images with automation controller or with other team members, push the images to a container registry, such as Quay.io or private automation hub.

> **Important**
> You are not allowed to publicly distribute container images that you build on top of the automation execution environment images that Red Hat provides.

Push a container image to private automation hub by using a procedure such as the following. In this example, private automation hub is at `hub.example.com`.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

- List your local container images:

```
[user@host ~]$ podman images
REPOSITORY                    TAG     IMAGE ID      CREATED         SIZE
localhost/ee-demo             v2.0    ba77b0d0a59d  2 minutes ago   308 MB
...output omitted...
```

- Use the `podman tag` command to tag the local container image for your private automation hub. The image name on private automation hub does not have to be the same as the local image. Also, private automation hub organizes the images in namespaces. You can use those namespaces to group your images.

```
[user@host ~]$ podman tag localhost/ee-demo:v2.0 \
hub.example.com/mynamespace/ee-demo:v2.0
```

Note

The `podman tag` command uses the `latest` tag if you do not specify a tag. A tag is arbitrary text and can specify a version, a date, an environment, and so on. Examples include `v1.0`, `20250323`, and `devel`.

A container image can have multiple tags, such as `latest`, `v1.0`, and `best`. Add each tag individually.

An individual tag, such as `latest`, can only apply to one container image at a time. Adding a tag name to an image removes that tag name from any other container image with the same name.

Using a tag other than `latest` makes it easier for automation controller to use a specific version of the automation execution environment.

- Use the `podman login` command to log in to private automation hub:

```
[user@host ~]$ podman login hub.example.com
```

- Use the `podman push` command to push the container image to private automation hub. This command also creates the namespace if it does not exist.

```
[user@host ~]$ podman push hub.example.com/mynamespace/ee-demo:v2.0
```

References

`podman-login`(1), `podman-tag`(1), and `podman-push`(1) man pages

Ansible Navigator Documentation

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

9.6. Guided Exercise
Validating a Custom Automation Execution Environment

Validate that a custom automation execution environment works as expected and then publish it to private automation hub.

Outcomes

- Use the `ansible-navigator` command to run a playbook using a local automation execution environment.
- Upload an automation execution environment to private automation hub.

Prerequisites

As the `student` user on the `workstation` machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
[student@workstation ~]$ lab start create-validate
```

Instructions

1. Review the inventory file and the `~/create-validate/motd-test.yml` playbook, which uses the `exercise.motd.banner` role.

   1.1. Change to the `~/create-validate` directory:

   ```
   [student@workstation ~]$ cd ~/create-validate/
   [student@workstation create-validate]$
   ```

   1.2. Review the `inventory` file:

   ```
   serverc.lab.example.com
   serverd.lab.example.com
   ```

   1.3. Review the contents of the `motd-test.yml` playbook:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
---
- name: Test banner role from exercise.motd collection
  hosts: all
  become: true
  gather_facts: true
  tasks:
    - name: Include the exercise.motd.banner role
      ansible.builtin.include_role:
        name: exercise.motd.banner
```

The playbook calls the `banner` role from the `exercise.motd` content collection. Notice that the single play targets all inventory hosts.

2. Display the existing messages of the day for the `serverc` machine.

   2.1. Log in to `serverc`:

```
[student@workstation create-validate]$ ssh serverc
...output omitted...
Activate the web console with: systemctl enable --now cockpit.socket (1)

Register this system with Red Hat Insights: insights-client --register (2)
Create an account or view all your systems at https://red.ht/insights-
dashboard
```

   (1) The `/etc/motd.d/cockpit` file contains the message to activate the web console.

   (2) The `/etc/motd.d/insights-client` file contains the Red Hat Insights message.

   2.2. Log out of `serverc`:

```
[student@serverc ~]$ logout
Connection to serverc closed.
[student@workstation create-validate]$
```

3. Run the `motd-test.yml` playbook. Try the default automation execution environment selected by `ansible-navigator` first, and when that fails, test the custom `ee-motd-minimal` automation execution environment that includes the `exercise.motd` collection. (This custom automation execution environment is the same as the one you created in the preceding guided exercise, but has been prepared by the `lab` command.)

   3.1. Attempt to run the `motd-test.yml` playbook using the `ansible-navigator` command. The attempt fails because the `exercise.motd.banner` role from the custom `exercise.motd` collection is not installed on the `workstation` machine and is not available in the `aap.lab.example.com/ee-supported-rhel8` automation execution environment.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation create-validate]$ ansible-navigator run motd-test.yml
...output omitted...
ERROR! the role 'exercise.motd.banner' was not found in
/home/student/create-
validate/roles:/home/runner/.ansible/roles:/usr/share/ansible/roles:/etc/ans
ible/roles:/home/student/create-validate

The error appears to be in '/home/student/create-validate/motd-test.yml':
line 9, column 15, but may
be elsewhere in the file depending on the exact syntax problem.

The offending line appears to be:

      ansible.builtin.include_role:
        name: exercise.motd.banner
              ^ here
...output omitted...
```

3.2. List the local container images:

```
[student@workstation create-validate]$ podman images
REPOSITORY                               TAG         IMAGE ID      ...
localhost/ee-motd-minimal                1.0         61402d9fffd5  ...
...output omitted...
```

> **Note**
> Your machine might display additional images, and the image ID for the `localhost/ee-motd-minimal` container image might be different.

3.3. Use the `ansible-navigator` command with the following options to run the `motd-test.yml` playbook:

Option	Purpose
`--eei localhost/ee-motd-minimal:1.0`	Select the custom container image for the automation execution environment.
`--pp never`	Do not attempt to download the container image from a container image repository.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation create-validate]$ ansible-navigator run motd-test.yml \
--eei localhost/ee-motd-minimal:1.0 --pp never

PLAY [Test banner role from exercise.motd collection]
*************************

TASK [Gathering Facts]
***********************************************************
ok: [serverd.lab.example.com]
ok: [serverc.lab.example.com]

TASK [Include the exercise.motd.banner role]
***********************************

TASK [exercise.motd.banner : Create /etc/motd.d]
*****************************
ok: [serverc.lab.example.com]
ok: [serverd.lab.example.com]

TASK [exercise.motd.banner : Set formatted_lines]
*****************************
ok: [serverc.lab.example.com] => (item=This system is managed by Ansible.)
ok: [serverd.lab.example.com] => (item=This system is managed by Ansible.)

TASK [exercise.motd.banner : Create /etc/motd.d/banner]
************************
changed: [serverc.lab.example.com]
changed: [serverd.lab.example.com]

PLAY RECAP
*********************************************************************
serverc.lab.example.com    : ok=4    changed=1    unreachable=0    failed=0
...
serverd.lab.example.com    : ok=4    changed=1    unreachable=0    failed=0
...
```

4. Verify that the playbook run configured the `serverc` machine to include an additional message of the day (MOTD) message.

   4.1. Log in to `serverc`:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation create-validate]$ ssh serverc
================================================================
================================================================
==                                                            ==
==              This system is managed by Ansible.            ==
==                                                            ==
================================================================
================================================================
...output omitted...
```

In addition to the previous web console and Red Hat Insights messages, there is a new message indicating that the system is managed by Ansible. This new message comes from the `/etc/motd.d/banner` file.

4.2. Log out of `serverc`:

```
[student@serverc ~]$ logout
Connection to serverc closed.
[student@workstation create-validate]$
```

5. Publish the tested `localhost/ee-motd-minimal` automation execution environment to the container registry on the classroom private automation hub.

5.1. Use the `podman tag` command to add the `aap.lab.example.com/ee-motd-minimal:1.0` label to the `localhost/ee-motd-minimal:1.0` image. The command does not produce any output.

```
[student@workstation create-validate]$ podman tag \
localhost/ee-motd-minimal:1.0 aap.lab.example.com/ee-motd-minimal:1.0
```

5.2. Use the `podman login` command to log in to the container registry at `aap.lab.example.com`:

```
[student@workstation create-validate]$ podman login aap.lab.example.com
Username: admin
Password: redhat
Login Succeeded!
```

5.3. Push the `aap.lab.example.com/ee-motd-minimal:1.0` container image to private automation hub:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation create-validate]$ podman push \
aap.lab.example.com/ee-motd-minimal:1.0
...output omitted...
Writing manifest to image destination
```

> **Note**
> In some cases, the entire container image is not pushed to private automation hub. If the `podman push` command produces an error, then try rerunning the command.

5.4. Go to `https://aap.lab.example.com` and log in to Ansible Automation Platform as `admin` using `redhat` as the password.

5.5. From the unified UI, go to **Automation Content > Execution Environments**. Notice that the page lists the `ee-motd-minimal` automation execution environment.

6. Download and test the automation execution environment.

6.1. Display the existing MOTD messages for the `servere` machine:

```
[student@workstation create-validate]$ ssh servere
Activate the web console with: systemctl enable --now cockpit.socket

Register this system with Red Hat Insights: insights-client --register
Create an account or view all your systems at https://red.ht/insights-
dashboard
```

6.2. Log out of `servere`:

```
[student@servere ~]$ logout
Connection to servere closed.
[student@workstation create-validate]$
```

6.3. Append `servere.lab.example.com` to the existing `inventory` file so that the file contains the following content:

```
serverc.lab.example.com
serverd.lab.example.com
servere.lab.example.com
```

6.4. Rerun the `motd-test.yml` playbook using the `ansible-navigator run` command and use the following options:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Option	Purpose
`--eei aap.lab.example.com/ee-motd-minimal:1.0`	Select the custom container image for the automation execution environment.
`--pp always`	Always download the container image from the container image repository.
`-e motd_msg`	Override the default value of the `motd_msg` variable with a custom message.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation create-validate]$ ansible-navigator run motd-test.yml \
--eei aap.lab.example.com/ee-motd-minimal:1.0 --pp always \
-e motd_msg="'Testing the banner role.'"
------------------------------------------------------------------------------
Execution environment image and pull policy overview
------------------------------------------------------------------------------
Execution environment image name:     aap.lab.example.com/ee-motd-
minimal:1.0
Execution environment image tag:      1.0
Execution environment pull arguments: None
Execution environment pull policy:    always
Execution environment pull needed:    True
------------------------------------------------------------------------------
Updating the execution environment
------------------------------------------------------------------------------
Running the command: podman pull aap.lab.example.com/ee-motd-minimal:1.0
...output omitted...

TASK [exercise.motd.banner : Create /etc/motd.d/banner]
************************
changed: [serverc.lab.example.com]
changed: [serverd.lab.example.com]
changed: [servere.lab.example.com]

PLAY RECAP
*********************************************************************
serverc.lab.example.com    : ok=4    changed=1    unreachable=0    failed=0
...
serverd.lab.example.com    : ok=4    changed=1    unreachable=0    failed=0
...
servere.lab.example.com    : ok=4    changed=1    unreachable=0    failed=0
...
```

6.5. Verify that the `servere` machine now contains the additional MOTD message:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation create-validate]$ ssh servere
================================================================
================================================================
==                                                            ==
==                  Testing the banner role.                  ==
==                                                            ==
================================================================
================================================================
...output omitted...
```

6.6. Log out of `servere`:

```
[student@servere ~]$ logout
Connection to servere closed.
[student@workstation create-validate]$
```

Finish

On the `workstation` machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish create-validate
```

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

9.7. Using Custom Ansible Content Collections and Automation Execution Environments in Automation Controller

Objectives

- Run a playbook in automation controller that uses an Ansible Content Collection in the Ansible project or provided by a specific automation execution environment.

Using Custom Collections with Existing Automation Execution Environments

You might want to run an Ansible Playbook that requires an Ansible Content Collection that is not included in an existing automation execution environment. If so, then automation controller can automatically download the required collections into the project. The project files, including the added collections, are mounted into the automation execution environment before automation controller runs the playbook.

To configure automation controller to download the required collections, you need to do two things:

- Create a `collections/requirements.yml` file in your Ansible project, which specifies the additional collections that your playbook needs.
- Configure automation controller with the authentication credentials it needs to access the automation hub or private automation hub that stores those collections.

Preparing Ansible Projects for Automation Controller

When you use the `ansible-navigator run` command to run a playbook that requires Ansible Content Collections that are not in the automation execution environment, you must manually install those collections. Usually, the project provides a `collections/requirements.yml` file that you can process with the `ansible-galaxy collection install -r collections/requirements.yml` command to do this.

Automation controller does not require you to manually run this command. Instead, it automatically runs the following commands to install collections and roles when you synchronize a project:

- `ansible-galaxy collection install -r collections/requirements.yml`
- `ansible-galaxy role install -r roles/requirements.yml`

The `collections/requirements.yml` file that you provide in your Ansible project might point to collections available on automation hub or private automation hub.

For example, the following `collections/requirements.yml` file lists two collections. Automation hub hosts the `redhat.insights` collection, and private automation hub at `aap.example.com` hosts the `ansible.posix` collection.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
---
collections:
  - name: redhat.insights
    source: https://console.redhat.com/api/automation-hub/content/published/

  - name: ansible.posix
    source: https://aap.example.com/pulp_ansible/galaxy/rh-certified/
```

Storing Authentication Credentials for Collections

Both automation hub and private automation hub require login credentials. You must configure automation controller to store this information in `Ansible Galaxy/Automation Hub API Token` credentials.

Because collection repositories, such as the `rht-certified`, `validated`, and `community` repositories, have unique URLs, you must create a credential for each repository. Use the following procedure to create each `Ansible Galaxy/Automation Hub API Token` credential:

- Log in to the unified UI of your Ansible Automation Platform.
- Go to **Automation Execution › Infrastructure › Credentials** and click **Create credential**.
- Select the **Ansible Galaxy/Automation Hub API Token** credential type and then specify a name for the credential, associate the credential with an organization, specify the galaxy server URL, and paste your API token. Some repositories might also include an authentication URL.

  When finished, click **Create credential**.

> **Note**
>
> For automation hub, you can obtain the relevant URLs and your token at https://console.redhat.com/ansible/automation-hub/token.
>
> For private automation hub, you can obtain the relevant URLs by going to **Automation Content › Repositories** and then clicking the link for each required repository. The **Details** tab for each repository displays the repository URL.
>
> For private automation hub, you can generate an API token by going to **Automation Content › API Token** and then clicking **Generate token**.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

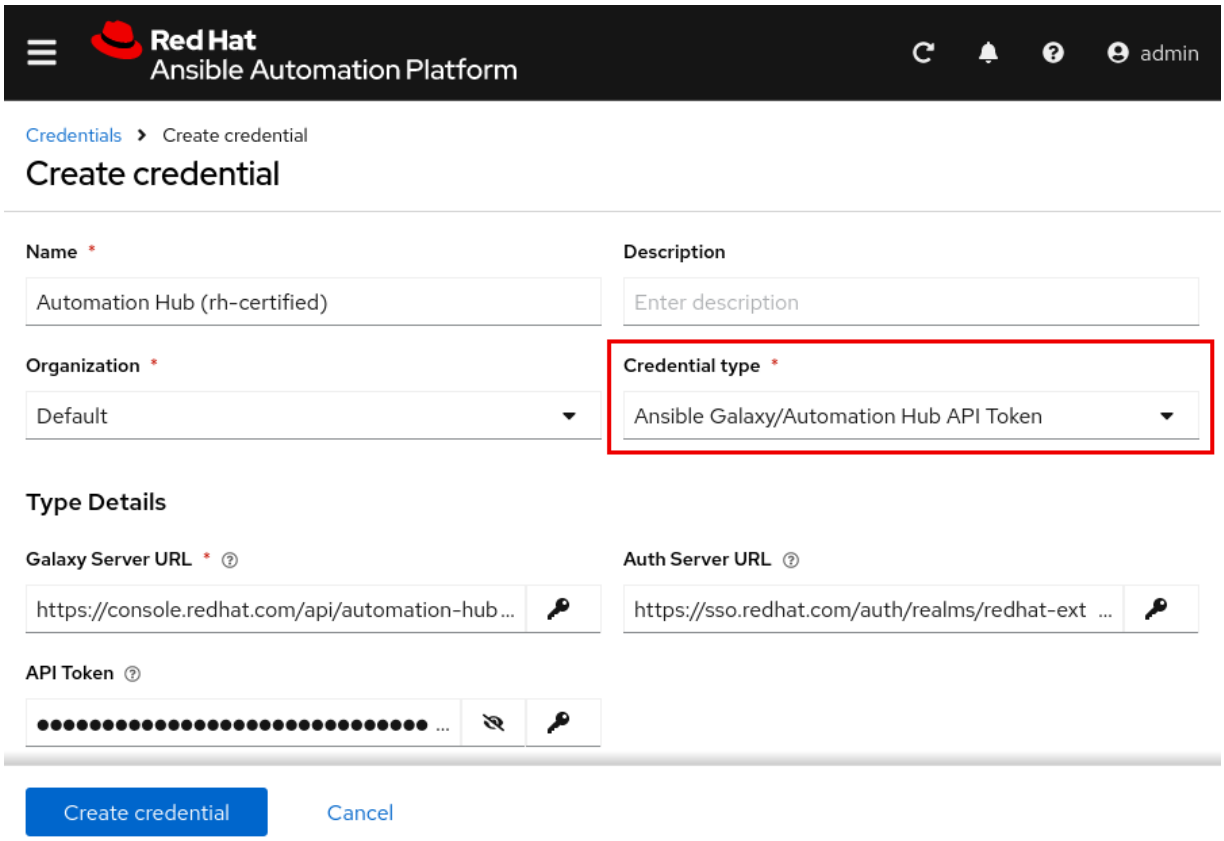


Figure 41. Creating a credential for accessing automation hub

> **Important**
>
> Because generating a new API token deletes your previous token, you might consider the following procedure:
>
> 1. Create all required `Ansible Galaxy/Automation Hub API Token` credentials (without entering a value for the API token).
> 2. Generate a new API token.
> 3. Update all `Ansible Galaxy/Automation Hub API Token` credentials with the new API token.
>
> Remember to update those credentials any time you generate a new API token.

After you create the required `Ansible Galaxy/Automation Hub API Token` credentials, add the credentials to each organization that should be able to use them:

- Go to **Access Management > Organizations**, select an organization, and then click **Edit organization**.
- Add one or more credentials in the **Galaxy Credentials** field and then click **Next**.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

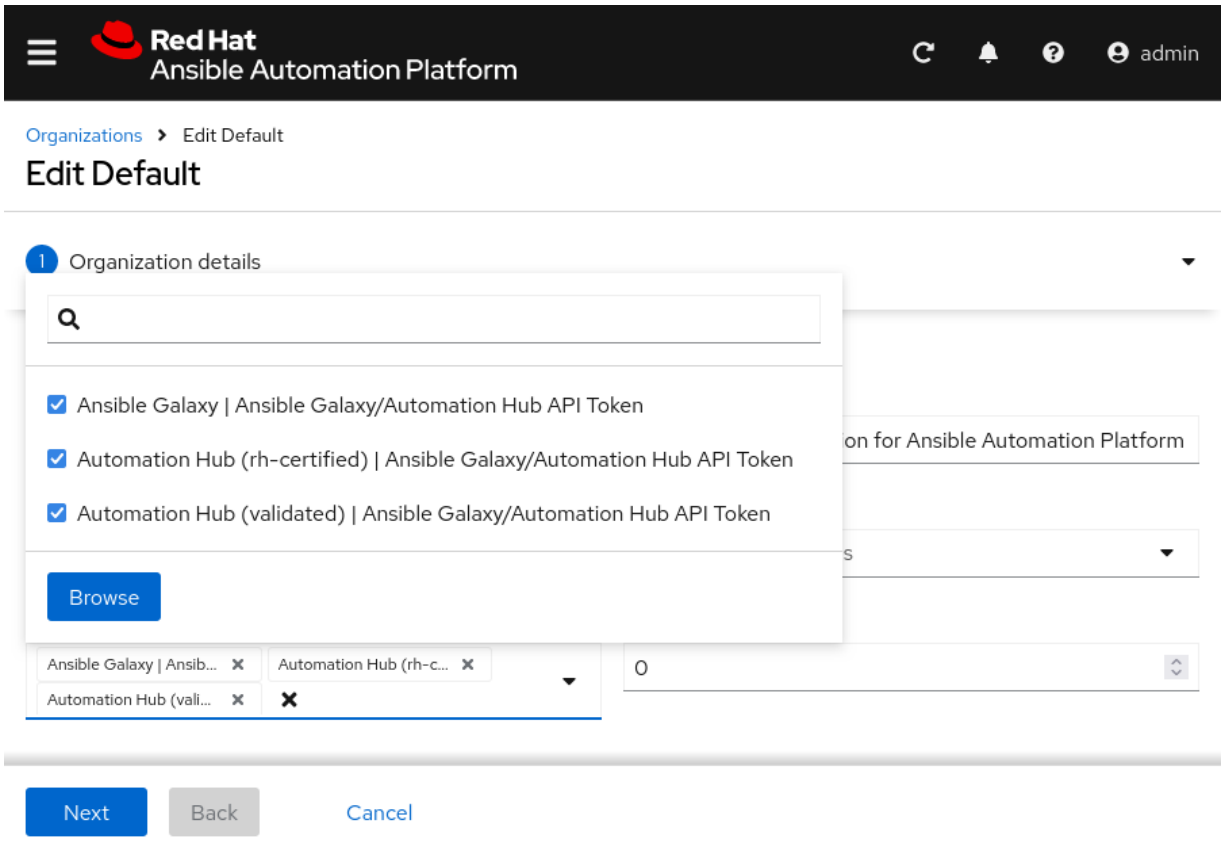


Figure 42. Associating credentials with organizations

- Drag the credentials to adjust the order in which you want to search the repositories for collections (the repository associated with the top credential is searched first). Click **Next**.
- Click **Finish** to associate the credentials.

Using Custom Automation Execution Environments with Automation Controller

Automation controller uses automation execution environments for both control plane tasks and execution plane tasks. By default, automation controller already includes supported automation execution environments from Red Hat.

You can use your own custom automation execution environments as well. You can configure automation controller to automatically download the container images for these automation execution environments.

Use the following procedure to add automation execution environments to automation controller:

- If the desired automation execution environment does not exist in your private automation hub, then configure automation controller with a credential that it can use to authenticate to the container registry.
- Configure automation controller with information about what container images to pull from where, and how often to pull them.
- *(Optional)* Specify a default automation execution environment for automation controller to use when running playbooks from a particular project.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

- *(Optional)* Specify an automation execution environment to use when running a playbook from a particular template, if you do not want the template to use the project's default automation execution environment.

Storing Container Registry Credentials

Automation controller automatically downloads the container images for its automation execution environments from a container registry. This could be a container registry built into automation hub, or one provided by a service such as Quay.

Many container registries require authentication for access. You need to configure automation controller with a credential to store this authentication information.

Use the following procedure to create a credential for a container registry:

- Log in to the unified UI of your Ansible Automation Platform.
- Go to **Automation Execution > Infrastructure > Credentials** and click **Create credential**.
- Select the **Container Registry** credential type and then provide a name for the credential, the URL of the container registry, your username, and your password or token.
- Click **Create credential**.

Configuring Automation Execution Environments

Use the following procedure to add automation execution environments to automation controller:

- Go to **Automation Execution > Execution Environments** and then click **Create execution environment**.
- Complete the form and then click **Create execution environment**.

> **Important**
> If you do not specify a tag for the image, then the tag defaults to `latest`.
>
> Using the `latest` tag might be problematic. Suppose that a developer creates an updated version of an automation execution environment but forgets to assign a tag to it. If the developer pushes an image without a tag to the container registry, then that image becomes the new `latest` image. Automation controller might start using that image, which can have severe consequences if the image does not work correctly.

Configuring the Default Automation Execution Environment for a Project

When you create a project in automation controller, you can specify a default automation execution environment for playbooks from that project to use.

Use the following procedure to define the default automation execution environment for your project:

- Go to **Automation Execution > Projects** and then click **Create project**.
- Complete the form to configure your project and select the automation execution environment to use by default in the **Execution environment** field.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

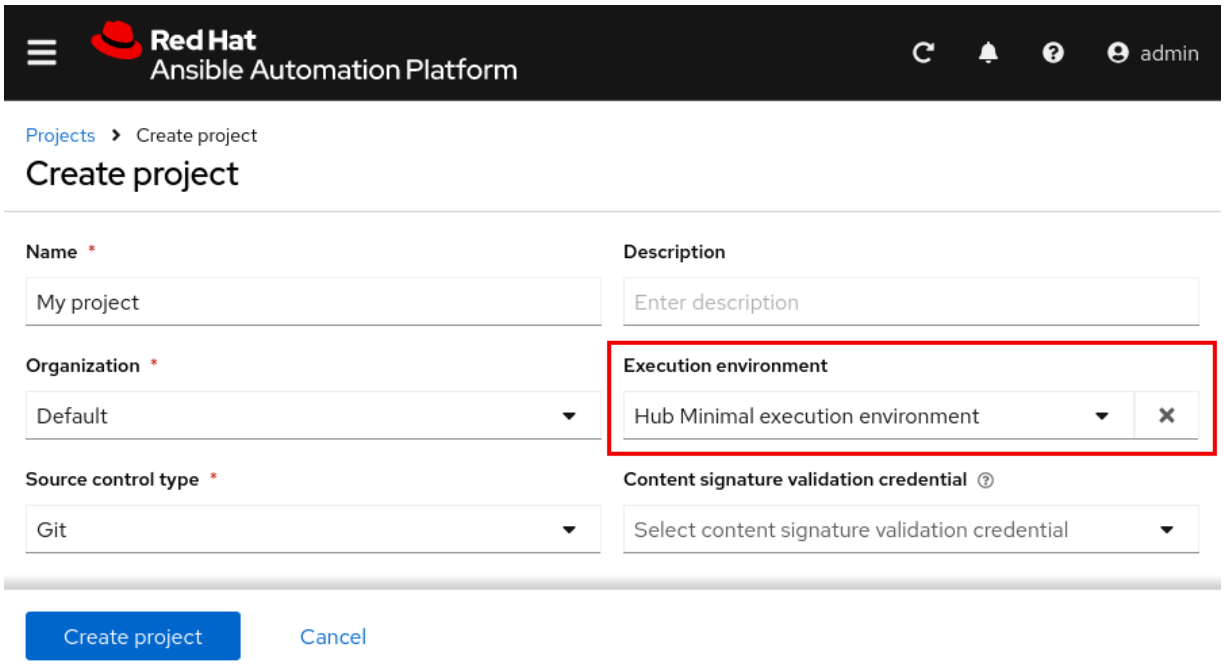


Figure 43. Selecting an automation execution environment for a project

- Click **Create project**.

You can also update existing projects to use a specific automation execution environment by default.

Specifying an Automation Execution Environment in a Job Template

A job template specifies all the parameters that automation controller needs to run a playbook, including the project containing the playbook, the inventory to use, and machine credentials needed to access the managed nodes.

When you set up a job template to run a playbook, you can also select a specific automation execution environment to use when running the playbook. Defining an automation execution environment in a job template overrides the default automation execution environment specified by the playbook's project.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

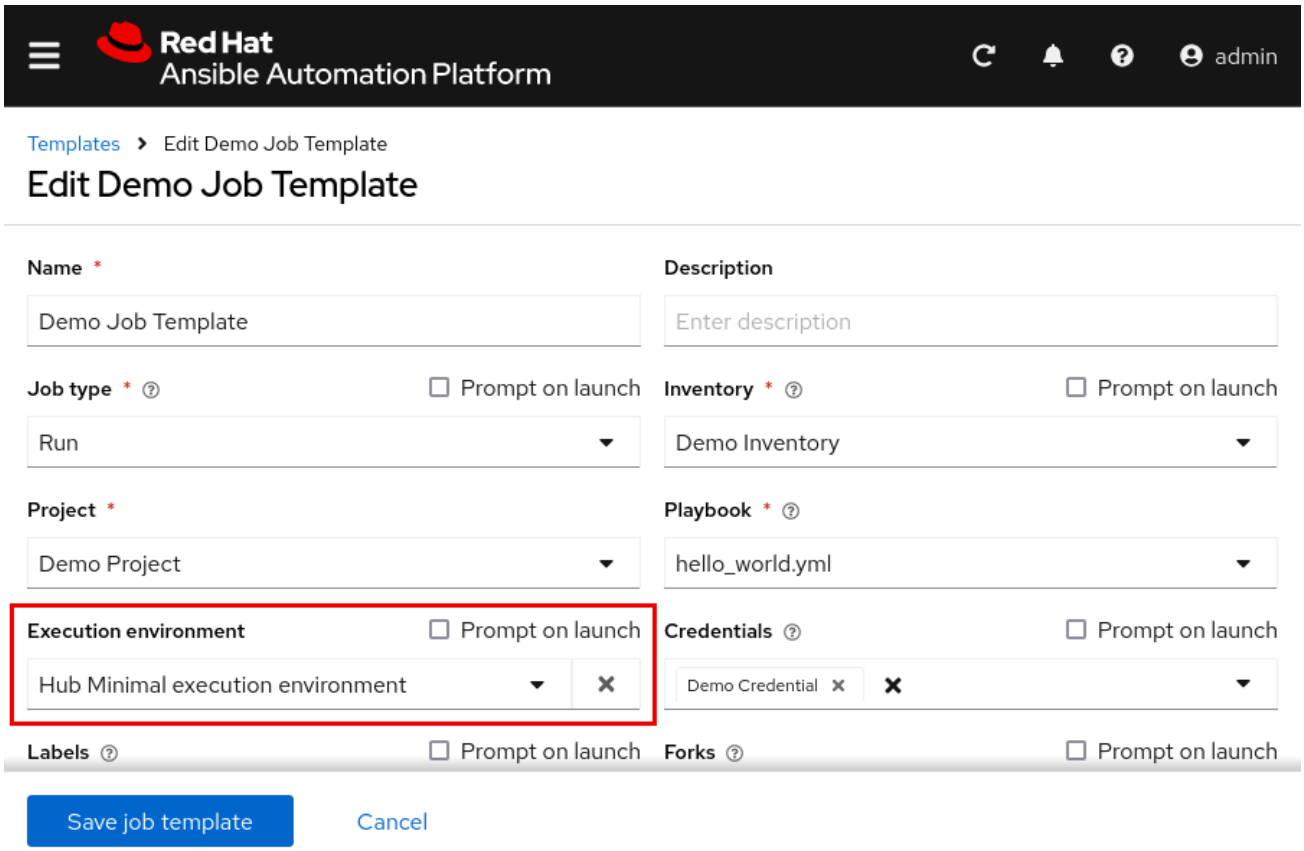


Figure 44. Selecting an automation execution environment in a job template

If you leave the **Execution environment** field empty, then automation controller uses the default automation execution environment configured for the project, the organization, or the global default.

After creating the template, click **Launch template** to start a job (run the playbook). The first time you launch a job that uses a new automation execution environment, the job might take longer than later runs. Automation controller must download the automation execution environment's container image if it is not present.

References

Getting Started with Ansible Automation Platform

Creating and Using Execution Environments

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

9.8. Guided Exercise
Using Custom Ansible Content Collections and Automation Execution Environments in Automation Controller

Run a playbook in automation controller that retrieves an Ansible Content Collection by using a requirements file and that uses content in a custom Ansible Content Collection.

Outcomes

- Run a playbook on automation controller that uses a customized collection stored on a private automation hub.

Prerequisites

As the `student` user on the `workstation` machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
[student@workstation ~]$ lab start create-use
```

Instructions

1. Publish the provided Ansible Content Collection on private automation hub.

   1.1. Go to `https://aap.lab.example.com` and log in to Ansible Automation Platform as `admin` using `redhat` as the password.

   1.2. From the unified UI, go to **Automation Content > Namespaces** and then click **Create namespace**.

   > **Note**
   > You might need to increase the display resolution on the `workstation` machine, decrease the web browser zoom, or both, to see the **Create namespace** button.

   1.3. Enter `exercise` as the name of the namespace and then click **Create namespace**.

   1.4. Click the **Collections** tab and then click **Upload collection**.

   1.5. Click **Browse**. Select the most recent version of the archive in the `/home/student/git-repos/exercise.motd` directory, such as the `exercise-motd-1.0.0.tar.gz` file, click **Open**, and then click **Upload collection**.

   1.6. From the **Collection Approvals** page, click **Approve and sign collection** (the thumbs up icon) to approve the `exercise.motd` collection.

   Confirm your decision, click **Approve collections**, and then close the confirmation window.

2. Create an automation controller credential that you can use to install collections from the `published` repository.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

2.1. From the unified UI, go to **Automation Content > Repositories** and then click the link for the `published` repository.

2.2. Click the **Copy to clipboard** icon to copy the repository URL.

2.3. Go to **Automation Execution > Infrastructure > Credentials** and then click **Create credential**.

2.4. Use the following information for the credential. You can paste the previously copied URL into the **Galaxy Server URL** field. When finished, click **Create credential**:

Field	Value
Name	`Published collections`
Organization	`Default`
Credential type	`Ansible Galaxy/Automation Hub API Token`
Galaxy Server URL	`https://aap.lab.example.com/pulp_ansible/galaxy/published/`
Auth Server URL	*(empty)*
API Token	*(empty)*

> **Note**
> Although not required for this exercise, you might perform the same steps to create credentials that you can use to install collections from the `rh-certified`, `validated`, and `community` repositories available from your private automation hub.

2.5. Go to **Automation Content > API Token** and click **Generate token**.

2.6. Copy the token.

2.7. Go to **Automation Execution > Infrastructure > Credentials**. For each `Ansible Galaxy/Automation Hub API Token` credential that you created, click **Edit credential** for that credential, paste the copied token into the **API Token** field, and then click **Save credential**.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

> **Important**
> Because generating an API token replaces your previous token, you must update each `Ansible Galaxy/Automation Hub API Token` credential that requires the token.

3. Update the `Default` organization so that it can use custom `Ansible Galaxy/Automation Hub API Token` credentials.

   3.1. Go to **Access Management > Organizations** and click the **Edit organization** icon for the `Default` organization.

   3.2. Click the menu for the **Galaxy credentials** field, select `Published collections | Ansible Galaxy/Automation Hub API Token`, and then click **Next**.

   3.3. Drag `Published collections` to the top of the list and then click **Next**.

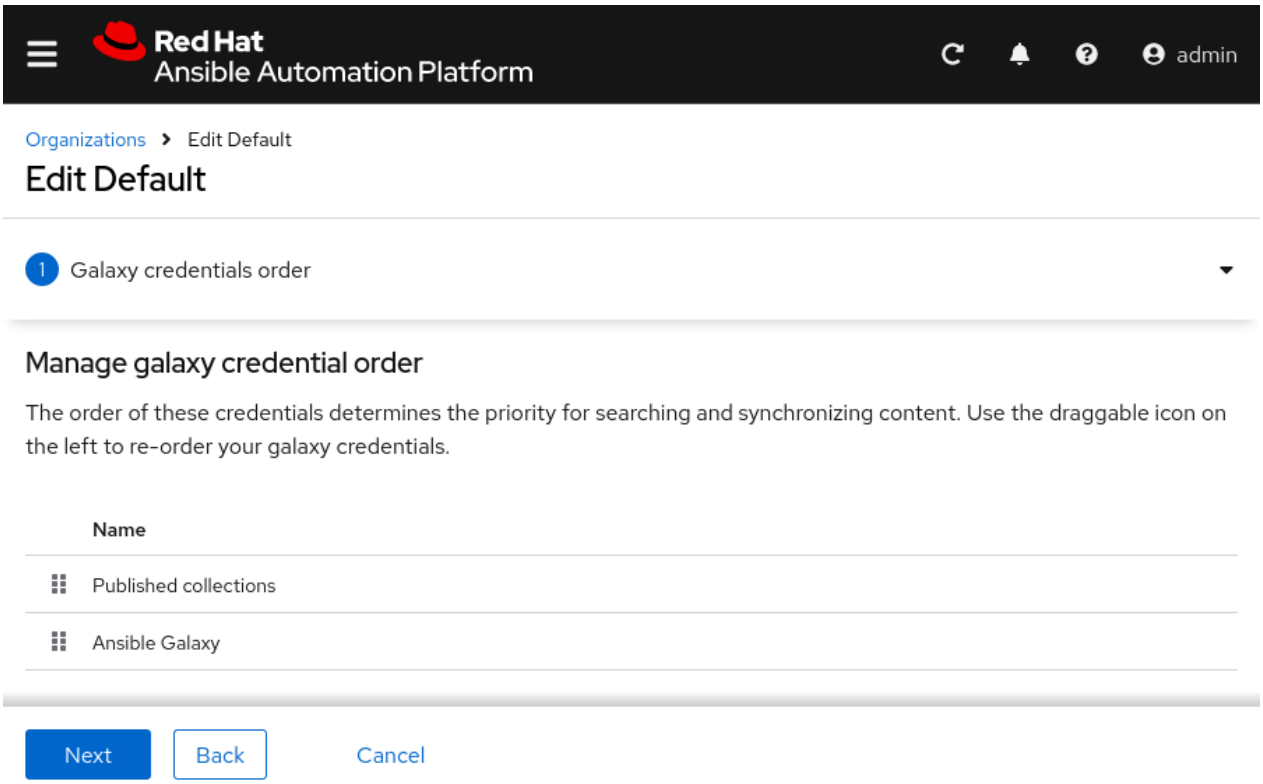


Figure 45. Adjust credential order

   3.4. Click **Finish**.

4. Create an automation controller project.

   4.1. From the unified UI, go to **Automation Execution > Projects** and then click **Create project**.

   4.2. Use the following information for the project and then click **Create project**:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Field	Value
Name	Install Collection
Organization	Default
Source control type	Git
Source control URL	https://git.lab.example.com/student/use-collection.git

> **Note**
> This project contains a collection requirements file that automation controller uses to install the exercise.motd collection.

4.3. Wait for the synchronization to complete.

> **Note**
> If the synchronization fails, then ensure that you entered the correct source control URL and that the uploaded motd collection has been approved. Also ensure that the Published collections credential uses a valid API token and that the Default organization can use that credential. Attempt the project synchronization again.

4.4. Go to **Automation Execution > Jobs** and then click link for the Install Collection source control update job. Notice that the job installed collections found in the collections/requirements.yml file:

```
...output omitted...
TASK [Fetch galaxy collections from collections/requirements.(yml/yaml)]
*******
changed: [localhost]
...output omitted...
```

> **Note**
> You can safely ignore the deprecation warning about the ANSIBLE_COLLECTIONS_PATHS option.

5. Display the existing messages of the day for the servera and serverb machines. The machines do not display custom messages of the day.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

5.1. Log in to `servera` to display the messages of the day, and then log out:

```
[student@workstation ~]$ ssh servera
Activate the web console with: systemctl enable --now cockpit.socket

Register this system with Red Hat Insights: insights-client --register
Create an account or view all your systems at https://red.ht/insights-
dashboard
...output omitted...

[student@servera ~]$ logout
Connection to servera closed.
[student@workstation ~]$
```

5.2. Log in to `serverb` to display the messages of the day, and then log out:

```
[student@workstation ~]$ ssh serverb
Activate the web console with: systemctl enable --now cockpit.socket

Register this system with Red Hat Insights: insights-client --register
Create an account or view all your systems at https://red.ht/insights-
dashboard
...output omitted...

[student@serverb ~]$ logout
Connection to serverb closed.
[student@workstation ~]$
```

6. Create a job template to run the `motd-test.yml` playbook from the `Install Collection` project. This project installs the required collection from a `requirements.yml` file.

6.1. From the unified UI, go to **Automation Execution > Templates** and then click **Create template > Create job template**.

6.2. Create a job template using the following information and then click **Create job template**:

Field	Value
Name	MOTD Test 1
Description	Test using installed collection
Inventory	Common Hosts

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Field	Value
Project	Install Collection
Playbook	motd-test.yml
Execution environment	Hub Minimal execution environment
Credentials	Developers | Machine
Limit	servera.lab.example.com

> **Note**
> To select the credential, you can either select the credential and then click outside the menu, or you can click **Browse**, select the credential, and then click **Confirm**.

6.3. Click **Launch template** to launch the `MOTD Test 1` job template. The job succeeds, and the output indicates that `servera.lab.example.com` was changed.

```
...output omitted...
TASK [exercise.motd.banner : Create /etc/motd.d/banner]
************************
changed: [servera.lab.example.com]
...output omitted...
```

6.4. Log in to `servera` to display the messages of the day, and then log out. The system displays a message indicating that it is managed by Ansible.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation ~]$ ssh servera
================================================================
================================================================
==                                                            ==
==              This system is managed by Ansible.            ==
==                                                            ==
================================================================
================================================================
...output omitted...

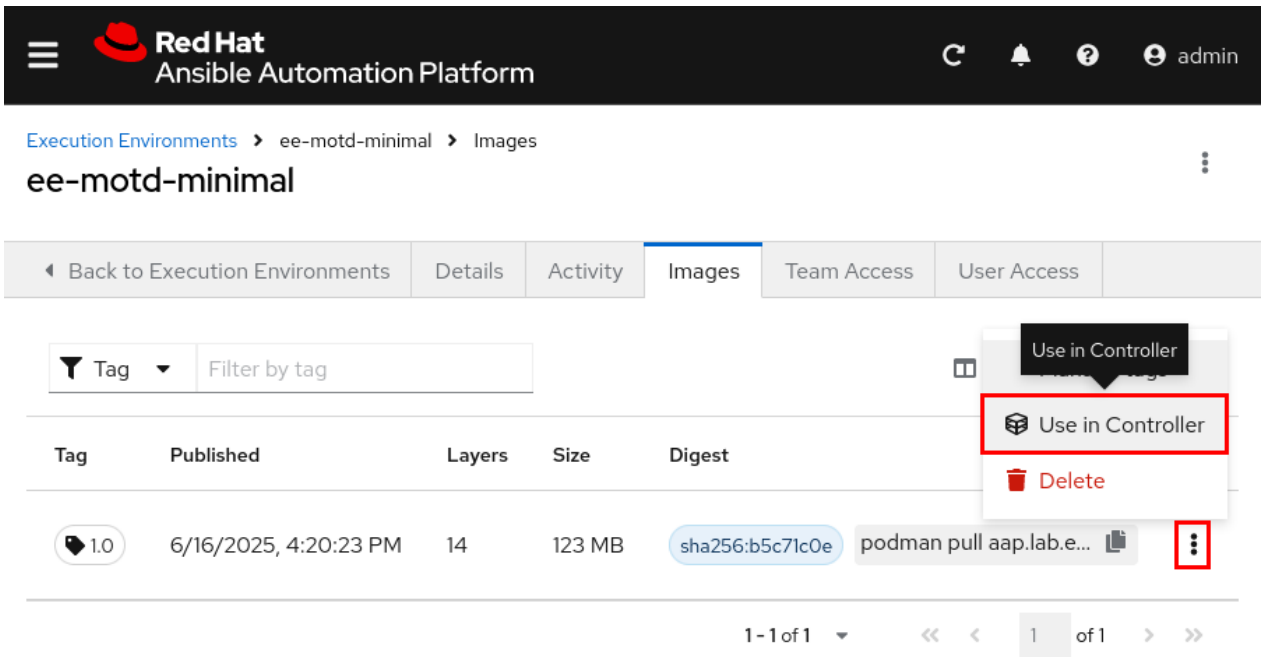
[student@servera ~]$ logout
Connection to servera closed.
[student@workstation ~]$
```

7. Make an automation execution environment available to automation controller.

   7.1. Go to **Automation Content > Execution Environments** and click the `ee-motd-minimal` link.

   7.2. Click the **Images** tab.

   7.3. On the row that contains the `1.0` tag, click the menu icon and select **Use in Controller**.



   7.4. Create the automation execution environment resource using the following information, and then click **Create execution environment**:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Field	Value
Name	MOTD
Image	aap.lab.example.com/ee-motd-minimal:1.0
Pull	Only pull the image if not present before running
Organization	Default

8. Create an automation controller project that does not automatically install the `exercise.motd` collection.

   8.1. Go to **Automation Execution > Projects** and then click **Create project**.

   8.2. Create a project resource using the following information, and then click **Create project**:

Field	Value
Name	Use Custom Automation Execution Environment
Organization	Default
Source control type	Git
Source control URL	https://git.lab.example.com/student/use-ee.git

   8.3. Wait for the synchronization to complete.

> **Note**
> If the synchronization fails, then ensure that you entered the correct source control URL and attempt the project synchronization again.

9. Create a job template to run the `motd-test.yml` playbook from the `Use Custom Automation Execution Environment` project. Because the `Use Custom Automation Execution Environment` project does not automatically install the `exercise.motd` collection, you must select an automation execution environment that includes that collection.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

9.1. Go to **Automation Execution > Templates** and then click **Create template > Create job template**.

9.2. Create a job template using the following information and then click **Create job template**:

Field	Value
Name	`MOTD Test 2`
Inventory	`Common Hosts`
Project	`Use Custom Automation Execution Environment`
Playbook	`motd-test.yml`
Execution environment	`MOTD`
Credentials	`Developers | Machine`
Limit	`serverb.lab.example.com`
Extra variables	`motd_msg: My test message`

> **Note**
> The job template selects a specific automation execution environment and only uses one host from the `Common Hosts` inventory resource. The job template overrides the value of the `motd_msg` variable set in the `exercise.motd.banner` role.

9.3. Click **Launch template** to launch the `MOTD Test 2` job template. The job succeeds, and the output indicates that `serverb.lab.example.com` was changed.

```
...output omitted...
TASK [exercise.motd.banner : Create /etc/motd.d/banner]
************************
changed: [serverb.lab.example.com]
...output omitted...
```

9.4. Log in to `serverb` to display the messages of the day, and then log out. The system displays a message using the value of the `motd_msg` variable configured in the `MOTD Test 2` job template.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation ~]$ ssh serverb
================================================================
================================================================
==                                                            ==
==                        My test message                     ==
==                                                            ==
================================================================
================================================================
...output omitted...

[student@serverb ~]$ logout
Connection to serverb closed.
[student@workstation ~]$
```

Finish

On the `workstation` machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish create-use
```

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

9.9. Lab
Creating Ansible Content Collections and Automation Execution Environments

Create an Ansible Content Collection, upload it to private automation hub, create an automation execution environment that includes the collection, and then use that automation execution environment to run a playbook from automation controller.

Outcomes

- Create an Ansible Content Collection that includes roles and a plug-in.
- Create a custom automation execution environment.
- Upload an Ansible Content Collection and an automation execution environment to private automation hub.
- Configure automation controller to use a custom automation execution environment from private automation hub.

Prerequisites

As the `student` user on the `workstation` machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
[student@workstation ~]$ lab start create-review
```

Instructions

You can access the unified UI of Ansible Automation Platform at `https://aap.lab.example.com` as `admin` using `redhat` as the password.

1. Create an Ansible Content Collection named `training.web` in the `/home/student/create-review` directory.
    - Add the `apache` and `firewall` roles, available from the `/home/student/create-review/roles` directory, to the collection.
    - Extract the provided `/home/student/create-review/lookup.tgz` archive file to the collection `plugins` directory. This lookup plug-in creates a QR (Quick Response) code in PNG (Portable Network Graphics) format.
    - Define the requirements and dependencies for the collection:
        - Ansible version 2.15.0 or later
        - The `ansible.posix` collection (version 1.0.0 or later)
        - The `pillow` and `qrcode` Python packages
    - Build the collection and then copy the resulting `.tar.gz` file to the `/home/student/create-review` directory.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

> **Important**
>
> If you previously completed this exercise, then you must edit the collection `galaxy.yml` file to increment the collection version to a number greater than the previous version, such as to `1.0.1`. Failure to update the version might result in an error similar to the following when you attempt to install the collection:
>
> ```
> ERROR! Failed to download collection tar from 'published' due to the
> following unforeseen error: Mismatch artifact hash with downloaded file
> ```

2. Publish the `training.web` collection so that the collection can be downloaded and installed from your private automation hub.

3. Prepare the `/home/student/ee-build` directory with the files required to build a custom automation execution environment.
   - Update the provided `ansible.cfg` file so that the `ansible-builder` command can install the required collections from your private automation hub. Ensure that the file can be used to install collections from the `rh-certified` and `published` collection repositories.
   - Create the `requirements.yml` file to define the `training.web` collection requirement.
   - Copy the `/etc/pki/tls/certs/classroom-ca.pem` certificate authority file to the `/home/student/ee-build` directory.

4. In the `/home/student/ee-build` directory, update the provided `execution-environment.yml` file with the following information:
   - Use the `aap.lab.example.com/ee-minimal-rhel8:latest` container as the base image.
   - Install collections defined in the `requirements.yml` file.

     > **Important**
     >
     > To install collections from your private automation hub, the build process must trust that server's certificates. Use the `classroom-ca.pem` file to update the certificate authority trust of the container image.
     >
     > Because the `ansible.cfg` file contains your API token and identifies the required collection repositories, you must make that file available to the build process.

   - Define `/usr/bin/microdnf` as the executable to install software.

5. Build the custom automation execution environment container image and use `aap.lab.example.com/ee-training-rhel8:1.0` as the tag. Push the container image to the `aap.lab.example.com` container registry. You can log in to the container registry as `admin` using `redhat` as the password.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

6. Make the `aap.lab.example.com/ee-training-rhel8:1.0` automation execution environment that you uploaded to private automation hub available to automation controller. Use the following settings for the automation execution environment:

Field	Value
Name	`Training Review`
Image	`aap.lab.example.com/ee-training-rhel8:1.0`
Pull	`Always pull container before running`
Organization	`Default`

7. Create an automation controller job template that uses the following settings and then launch a job from that job template. If the job succeeds, then navigating to `http://serverd.lab.example.com` displays a QR code image.

Field	Value
Name	`Deploy QR Codes`
Inventory	`Development`
Project	`QR Codes`
Playbook	`test.yml`
Execution environment	`Training Review`
Credentials	`Developers | Machine`

> **Note**
> The `lab` command created the `Development` inventory, the `QR Codes` project, and the `Developers` machine credential.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Evaluation

As the `student` user on the `workstation` machine, use the `lab` command to grade your work. Correct any reported failures and rerun the command until successful.

```
[student@workstation ~]$ lab grade create-review
```

Finish

As the `student` user on the `workstation` machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish create-review
```

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Solution

Create an Ansible Content Collection, upload it to private automation hub, create an automation execution environment that includes the collection, and then use that automation execution environment to run a playbook from automation controller.

Outcomes

- Create an Ansible Content Collection that includes roles and a plug-in.
- Create a custom automation execution environment.
- Upload an Ansible Content Collection and an automation execution environment to private automation hub.
- Configure automation controller to use a custom automation execution environment from private automation hub.

Prerequisites

As the `student` user on the `workstation` machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
[student@workstation ~]$ lab start create-review
```

Instructions

You can access the unified UI of Ansible Automation Platform at `https://aap.lab.example.com` as `admin` using `redhat` as the password.

1. Create an Ansible Content Collection named `training.web` in the `/home/student/create-review` directory.
    - Add the `apache` and `firewall` roles, available from the `/home/student/create-review/roles` directory, to the collection.
    - Extract the provided `/home/student/create-review/lookup.tgz` archive file to the collection `plugins` directory. This lookup plug-in creates a QR (Quick Response) code in PNG (Portable Network Graphics) format.
    - Define the requirements and dependencies for the collection:
        - Ansible version 2.15.0 or later
        - The `ansible.posix` collection (version 1.0.0 or later)
        - The `pillow` and `qrcode` Python packages
    - Build the collection and then copy the resulting `.tar.gz` file to the `/home/student/create-review` directory.

Important

If you previously completed this exercise, then you must edit the collection `galaxy.yml` file to increment the collection version to a number greater than the previous version, such as to `1.0.1`. Failure to update the version might result in an error similar to the following when you attempt to install the collection:

```
ERROR! Failed to download collection tar from 'published' due to the
following unforeseen error: Mismatch artifact hash with downloaded file
```

1.1. Change to the `/home/student/create-review` directory and initialize the `training.web` collection:

```
[student@workstation ~]$ cd ~/create-review/
[student@workstation create-review]$ ansible-galaxy collection init
training.web
- Collection training.web was created successfully
```

1.2. Copy the provided roles to the `~/create-review/training/web/roles` directory:

```
[student@workstation create-review]$ ls roles/
apache  firewall
[student@workstation create-review]$ cp -r roles/* training/web/roles/
```

1.3. Extract the `lookup.tgz` file containing the lookup plug-in into the `~/create-review/training/web/plugins` directory:

```
[student@workstation create-review]$ tar xf lookup.tgz -C
training/web/plugins/
[student@workstation create-review]$ tree training/web/plugins/
training/web/plugins/
├── README.md
└── lookup
    └── qrcode.py

1 directory, 2 files
```

1.4. Update the `~/create-review/training/web/meta/runtime.yml` file to specify that your collection requires Ansible version 2.15.0 or later:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
---
# Collections must specify a minimum required ansible version to upload
# to galaxy
requires_ansible: '>=2.15.0'
...output omitted...
```

1.5. Edit the ~/create-review/training/web/galaxy.yml file to declare the ansible.posix collection as a required dependency of your collection. If necessary, increment the collection version as well:

```
...output omitted...
version: 1.0.0
...output omitted...
dependencies:
  ansible.posix: '>=1.0.0'
...output omitted...
```

1.6. Create the ~/create-review/training/web/requirements.txt file with the following content to declare the Python package dependencies:

```
pillow
qrcode
```

1.7. Change to the ~/create-review/training/web directory and then build the collection:

```
[student@workstation create-review]$ cd training/web/
[student@workstation web]$ ansible-galaxy collection build
Created collection for training.web at /home/student/create-
review/training/web/training-web-1.0.0.tar.gz
```

1.8. Copy the generated archive file to the /home/student/create-review directory:

```
[student@workstation web]$ cp training-web-1.0.0.tar.gz ~/create-review/
```

> **Note**
> If you previously completed the exercise and had to increment the collection version, then the archive file has a different version number.

2. Publish the training.web collection so that the collection can be downloaded and installed from your private automation hub.

2.1. Go to https://aap.lab.example.com and log in to Ansible Automation Platform as admin using redhat as the password.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

2.2. From the unified UI, go to **Automation Content > Namespaces** and then click **Create namespace**.

> **Note**
> You might need to increase the display resolution on the `workstation` machine, decrease the web browser zoom, or both, to see the **Create namespace** button.

2.3. Enter `training` as the name of the namespace and then click **Create namespace**.

2.4. Click the **Collections** tab and then click **Upload collection**.

2.5. Click **Browse**. Select the most recent version of the archive in the `/home/student/create-review` directory, such as the `training-web-1.0.0.tar.gz` file, click **Open**, and then click **Upload collection**.

2.6. From the **Collection Approvals** page, click **Approve and sign collection** (the thumbs up icon) to approve the `training.web` collection.

Confirm your decision, click **Approve collections**, and then close the confirmation window.

2.7. Confirm that the collection is published. Go to **Automation Content > Collections** and enter web in the search field.

Click the web link and then click the **Contents** tab. Confirm that the page lists the two roles, `apache` and `firewall`, and the `qrcode` lookup plug-in.

Click the **Dependencies** tab and confirm that the page lists the `ansible.posix >=1.0.0` dependency.

3. Prepare the `/home/student/ee-build` directory with the files required to build a custom automation execution environment.

- Update the provided `ansible.cfg` file so that the `ansible-builder` command can install the required collections from your private automation hub. Ensure that the file can be used to install collections from the `rh-certified` and `published` collection repositories.
- Create the `requirements.yml` file to define the `training.web` collection requirement.
- Copy the `/etc/pki/tls/certs/classroom-ca.pem` certificate authority file to the `/home/student/ee-build` directory.

3.1. Change to the `~/ee-build` directory:

```
[student@workstation web]$ cd ~/ee-build
[student@workstation ee-build]$
```

3.2. Display the content of the `ansible.cfg` file:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[galaxy]
server_list = rh-certified, published

[galaxy_server.rh-certified]
url=http://localhost:5001/api/galaxy/content/rh-certified/
token=<put your token here>

[galaxy_server.published]
url=http://localhost:5001/api/galaxy/
token=<put your token here>
```

Notice that the file contains entries for the `rh-certified` and `published` collection repositories. You must update the repository URL and add your API token for both repositories.

3.3. Update each repository URL to change `http://localhost:5001` to `https://aap.lab.example.com`. The modified `ansible.cfg` file now contains the following content:

```
[galaxy]
server_list = rh-certified, published

[galaxy_server.rh-certified]
url=https://aap.lab.example.com/api/galaxy/content/rh-certified/
token=<put your token here>

[galaxy_server.published]
url=https://aap.lab.example.com/api/galaxy/
token=<put your token here>
```

3.4. From the unified UI, go to **Automation Content > API Token** and click **Generate token**. Copy the API token.

3.5. Using the copied token, update both `token` lines in the `ansible.cfg` file. Your token is different from the one displayed in this example. Save and close the file when done:

```
[galaxy]
server_list = rh-certified, published

[galaxy_server.rh-certified]
url=https://aap.lab.example.com/api/galaxy/content/rh-certified/
token=10a190dd23f3f2e99c7cefa1e7b8b7afbc9c4b2d

[galaxy_server.published]
url=https://aap.lab.example.com/api/galaxy/
token=10a190dd23f3f2e99c7cefa1e7b8b7afbc9c4b2d
```

3.6. Create the `requirements.yml` file to define the `training.web` collection requirement:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
---
collections:
  - name: training.web
```

3.7. Copy the `/etc/pki/tls/certs/classroom-ca.pem` certificate authority file to the current directory:

```
[student@workstation ee-build]$ cp /etc/pki/tls/certs/classroom-ca.pem .
```

4. In the `/home/student/ee-build` directory, update the provided `execution-environment.yml` file with the following information:

- Use the `aap.lab.example.com/ee-minimal-rhel8:latest` container as the base image.
- Install collections defined in the `requirements.yml` file.

> **Important**
>
> To install collections from your private automation hub, the build process must trust that server's certificates. Use the `classroom-ca.pem` file to update the certificate authority trust of the container image.
>
> Because the `ansible.cfg` file contains your API token and identifies the required collection repositories, you must make that file available to the build process.

- Define `/usr/bin/microdnf` as the executable to install software.

4.1. Update the `execution-environment.yml` file to specify the base image:

```
---
version: 3

images:
  base_image:
    name: aap.lab.example.com/ee-minimal-rhel8:latest
...output omitted...
```

4.2. Update the `execution-environment.yml` file to install the collection dependencies that are defined in the `requirements.yml` file:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
...output omitted...
dependencies:
  galaxy: requirements.yml
...output omitted...
```

4.3. Because the `dnf` RPM package is not installed in the base image, you must use the `/usr/bin/microdnf` executable to install software. Configure the `package_manager_path` key to use `/usr/bin/microdnf`:

```
...output omitted...
options:
  package_manager_path: /usr/bin/microdnf
...output omitted...
```

4.4. Update the `execution-environment.yml` file to copy the `ansible.cfg` and `classroom-ca.pem` files to the `context/_build` directory:

```
...output omitted...
additional_build_files:
  - src: ee.repo
    dest: ./
  - src: ansible.cfg
    dest: ./
  - src: classroom-ca.pem
    dest: ./
...output omitted...
```

> **Important**
>
> The locations of the source and destination are not the same. The source location is relative to the `execution-environment.yml` file.
>
> The destination must be a directory and the location is relative to the `context/_build` directory. The `ansible-builder` command creates the `context/_build` directory when you use either the `create` or `build` subcommands.

4.5. Update the `execution-environment.yml` file to copy the `ansible.cfg` file to `/etc/ansible/ansible.cfg`:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
...output omitted...
additional_build_steps:
  prepend_base:
    - ENV PIP_INDEX_URL="http://workstation.lab.example.com:8000/simple/"
    - ENV PIP_TRUSTED_HOST="workstation.lab.example.com:8000"
    - RUN sed -i "s/^enabled.*$/enabled = 0/" /etc/yum.repos.d/*.repo
    - COPY _build/ee.repo /etc/yum.repos.d/ee.repo
    - COPY _build/ansible.cfg /etc/ansible/ansible.cfg
```

4.6. Update the `execution-environment.yml` file with steps so that the container image trusts the private automation hub certificates:

```
...output omitted...
additional_build_steps:
  prepend_base:
    - ENV PIP_INDEX_URL="http://workstation.lab.example.com:8000/simple/"
    - ENV PIP_TRUSTED_HOST="workstation.lab.example.com:8000"
    - RUN sed -i "s/^enabled.*$/enabled = 0/" /etc/yum.repos.d/*.repo
    - COPY _build/ee.repo /etc/yum.repos.d/ee.repo
    - COPY _build/ansible.cfg /etc/ansible/ansible.cfg
    - COPY _build/classroom-ca.pem /etc/pki/ca-trust/source/anchors
    - RUN update-ca-trust
```

4.7. Ensure that the completed `execution-environment.yml` file contains the following content:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
---
version: 3

images:
  base_image:
    name: aap.lab.example.com/ee-minimal-rhel8:latest

dependencies:
  galaxy: requirements.yml

options:
  package_manager_path: /usr/bin/microdnf

additional_build_files:
  - src: ee.repo
    dest: ./
  - src: ansible.cfg
    dest: ./
  - src: classroom-ca.pem
    dest: ./

additional_build_steps:
  prepend_base:
    - ENV PIP_INDEX_URL="http://workstation.lab.example.com:8000/simple/"
    - ENV PIP_TRUSTED_HOST="workstation.lab.example.com:8000"
    - RUN sed -i "s/^enabled.*$/enabled = 0/" /etc/yum.repos.d/*.repo
    - COPY _build/ee.repo /etc/yum.repos.d/ee.repo
    - COPY _build/ansible.cfg /etc/ansible/ansible.cfg
    - COPY _build/classroom-ca.pem /etc/pki/ca-trust/source/anchors
    - RUN update-ca-trust
```

5. Build the custom automation execution environment container image and use `aap.lab.example.com/ee-training-rhel8:1.0` as the tag. Push the container image to the `aap.lab.example.com` container registry. You can log in to the container registry as `admin` using `redhat` as the password.

   5.1. Log in to the container registry at `aap.lab.example.com` as `admin` using `redhat` as the password:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation ee-build]$ podman login aap.lab.example.com
Username: admin
Password: redhat
Login Succeeded!
```

> **Note**
> This step allows you to download the base container image and to later push the custom container image to the image registry.

5.2. *(Optional)* Create and display the `context` directory. Notice that the `ansible-builder create` command creates the `context/_build` directory and copies files to that directory:

```
[student@workstation ee-build]$ ansible-builder create
Complete! The build context can be found at: /home/student/ee-build/context

[student@workstation ee-build]$ tree -L 3
.
├── ansible.cfg
├── classroom-ca.pem
├── context
│   ├── Containerfile
│   └── _build
│       ├── ansible.cfg
│       ├── classroom-ca.pem
│       ├── ee.repo
│       ├── requirements.yml
│       └── scripts
├── ee.repo
├── execution-environment.yml
└── requirements.yml

3 directories, 10 files
```

5.3. Build the custom automation execution environment. Add the `-t` option to specify the `aap.lab.example.com/ee-training-rhel8:1.0` tag. You can also add the `-v3` option if you want to display the output of the build steps.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
[student@workstation ee-build]$ ansible-builder build \
-t aap.lab.example.com/ee-training-rhel8:1.0
Running command:
  podman build -f context/Containerfile -t aap.lab.example.com/ee-training-
rhel8:1.0 context
Complete! The build context can be found at: /home/student/ee-build/context
```

5.4. Confirm that the new container image is available locally. Your container image ID is different from the one displayed in the following output.

```
[student@workstation ee-build]$ podman images \
aap.lab.example.com/ee-training-rhel8:1.0
REPOSITORY                                TAG  IMAGE ID      ...
aap.lab.example.com/ee-training-rhel8  1.0  c418503ae3ac  ...
```

5.5. Push the container image to your private automation hub:

```
[student@workstation ee-build]$ podman push \
aap.lab.example.com/ee-training-rhel8:1.0
...output omitted...
```

6. Make the `aap.lab.example.com/ee-training-rhel8:1.0` automation execution environment that you uploaded to private automation hub available to automation controller. Use the following settings for the automation execution environment:

Field	Value
Name	Training Review
Image	aap.lab.example.com/ee-training-rhel8:1.0
Pull	Always pull container before running
Organization	Default

6.1. From the unified UI, go to **Automation Execution › Infrastructure › Execution Environments** and click **Create execution environment**.

6.2. Create the automation execution environment with the settings in the preceding table. When finished, click **Create execution environment**.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

7. Create an automation controller job template that uses the following settings and then launch a job from that job template. If the job succeeds, then navigating to `http://serverd.lab.example.com` displays a QR code image.

Field	Value
Name	`Deploy QR Codes`
Inventory	`Development`
Project	`QR Codes`
Playbook	`test.yml`
Execution environment	`Training Review`
Credentials	`Developers | Machine`

> **Note**
> The `lab` command created the `Development` inventory, the `QR Codes` project, and the `Developers` machine credential.

7.1. Go to **Automation Execution > Templates** and then click **Create template > Create job template**.

7.2. Create the job template with the settings in the preceding table. When finished, click **Create job template**.

7.3. Click **Launch template** to start a job from the `Deploy QR Codes` job template. Wait for the job to complete.

7.4. Click the **Details** tab for the job. The job displays the **Success** status.

7.5. Confirm the successful execution of the job by opening a web browser and navigating to `http://serverd.lab.example.com`. If the job completed correctly, then the web page displays a QR code image.

Evaluation

As the `student` user on the `workstation` machine, use the `lab` command to grade your work. Correct any reported failures and rerun the command until successful.

```
[student@workstation ~]$ lab grade create-review
```

Finish

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

As the `student` user on the `workstation` machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish create-review
```

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

9.10. Summary

- Use `ansible-galaxy collection init` to create the base directory structure for a new Ansible Content Collection.
- Dependencies for a collection are specified in three files: other collections under the `dependencies` parameter of the `galaxy.yml` file, Python packages in the `requirements.txt` file, and system packages in the `bindep.txt` file.
- Use `ansible-builder` to build custom automation execution environments, tailored to your collections dependencies.
- Define required collections, Python packages, and system packages in the `execution-environment.yml` file.
- The `podman images`, `podman tag`, and `podman push` commands list, tag, and push automation execution environment images to a container registry.
- Collections and roles listed in the `collections/requirements.yml` file for a project can be automatically pulled, but stored credentials must be configured for access to automation hub.
- Automation controller can use and automatically pull custom automation execution environments, but you need to configure credentials for the container registry and specify the location of the automation execution environment first.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Chapter 10.
Comprehensive Review

Goal

This lesson tests your knowledge acquired in the *Developing Advanced Automation with Red Hat Ansible Automation Platform* course.

Objectives

- Validate your knowledge acquired in the *Developing Advanced Automation with Red Hat Ansible Automation Platform* course.

Sections

- Summary of Learning Objectives
- Managing Inventory Variables to Use with Automation Content Navigator (Lab)
- Optimizing a Playbook for Large-scale Use (Lab)
- Creating and Using Ansible Content Collections and Automation Execution Environments (Lab)

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

10.1. Summary of Learning Objectives

Objectives

- After completing this section, you should have reviewed and refreshed the knowledge and skills that you learned in *Developing Advanced Automation with Red Hat Ansible Automation Platform Automation (AU374)*.

Reviewing Developing Advanced Automation with Red Hat Ansible Automation Platform Automation (AU374)

Before beginning this comprehensive review, you should be comfortable with the topics covered in each chapter. Do not hesitate to ask the instructor for extra guidance or clarification on these topics.

Chapter 1 - Developing Playbooks with Ansible Automation Platform 2

Develop Ansible Playbooks with Red Hat Ansible Automation Platform 2 following recommended practices.

- Review the architecture of Red Hat Ansible Automation Platform 2 and how its new features help with Ansible automation development.
- Install automation content navigator and use it to run an existing playbook with a supported automation execution environment.
- Create and manage Ansible Playbooks in a Git repository, following recommended practices.
- Demonstrate recommended practices for developing and maintaining effective Ansible automation solutions.

Chapter 2 - Managing Ansible Content Collections and Automation Execution Environments

Run playbooks that use Ansible Content Collections not included in `ansible-core`, either by using an existing automation execution environment or by downloading the required collections.

- Describe how Ansible Content Collections are used to distribute modules and plug-ins, and create plays that use content from them.
- Search automation hub for Ansible Content Collections, and install them from the command line by name or by using a `requirements.yml` file.
- Identify the automation execution environments provided by Red Hat and select the correct one for your use case.
- Run a playbook that needs an Ansible Content Collection in a particular automation execution environment, and another playbook that needs an Ansible Content Collection that is not provided by an automation execution environment.

Chapter 3 - Running Playbooks with Automation Controller

Understand what automation controller is and demonstrate how to use it to run playbooks that you developed with automation content navigator.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

- Describe the architecture and use cases of the automation controller component of Red Hat Ansible Automation Platform.
- Navigate and describe the Automation Execution component of the unified UI, and successfully launch a job using a job template, project, credential, and inventory.

Chapter 4 - Working with Ansible Configuration Settings

Examine and adjust the configuration of Ansible and automation content navigator to simplify development and to troubleshoot issues.

- Browse the current Ansible configuration by using automation content navigator.
- Change configuration settings for automation content navigator with its configuration file, and determine where the configuration file is located.
- Use Visual Studio Code to develop and run automation content and synchronize changes with a remote Git repository.

Chapter 5 - Managing Inventories

Manage inventories by using advanced features of Ansible.

- Describe what dynamic inventories are, and install and use an existing script or plug-in as an Ansible dynamic inventory source.
- Write static inventory files in YAML format.
- Structure host and group variables by using multiple files per host or group, and use special variables to override the host, port, or remote user that Ansible uses for a specific host.

Chapter 6 - Managing Task Execution

Control and optimize the execution of tasks in Ansible Playbooks.

- Control automatic privilege escalation at the play, role, task, or block level.
- Configure tasks that can run before roles or after normal handlers, and simultaneously notify multiple handlers.
- Label tasks with tags, run only those tasks labeled with specific tags, or start playbook execution at a specific task.
- Optimize your playbook to run more efficiently, and use callback plug-ins to profile and analyze which tasks consume the most time.

Chapter 7 - Transforming Data with Filters and Plug-ins

Populate, manipulate, and manage data in variables using filters and plug-ins.

- Format, parse, and define the values of variables using filters.
- Populate variables with data from external sources using lookup plug-ins.
- Implement loops using structures other than simple lists using lookup plug-ins and filters.
- Use filters to inspect, validate, and manipulate variables containing networking information.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Chapter 8 - Coordinating Rolling Updates

Use advanced features of Ansible to manage rolling updates in order to minimize downtime, and to ensure the maintainability and simplicity of Ansible Playbooks.

- Run a task for a managed host on a different host, and control whether facts gathered by that task are delegated to the managed host or to the other host.
- Tune the number of simultaneous connections that Ansible opens to managed hosts and how Ansible processes groups of managed hosts through the play's tasks.
- Tune the behavior of the `serial` directive when batching hosts for execution, abort the play if it fails for too many hosts, and create tasks that run only once for each batch or for all hosts in the inventory.

Chapter 9 - Creating Ansible Content Collections and Automation Execution Environments

- Create content collections and distribute them for reuse.
- Build a custom automation execution environment image by using the `ansible-builder` command.
- Validate that a custom automation execution environment works as expected by testing it with the `ansible-navigator` command, and then distribute the automation execution environment for reuse.
- Run a playbook in automation controller that uses an Ansible Content Collection in the Ansible project or provided by a specific automation execution environment.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

10.2. Lab
Managing Inventory Variables to Use with Automation Content Navigator

Manage playbook and inventory host group variables, configure automation content navigator settings, and push your changes to a new branch using Git.

Outcomes

- Use Git to clone a repository, create a branch, ignore files, and push changes to a branch.
- Configure automation content navigator settings.
- Manage inventory hosts and host groups.
- Define variables for inventory host groups.

> **Note**
> The allocated time for this activity is 30 minutes. If you need additional time to complete the task, then you must revisit the course content again, or practice more.

Prerequisites

As the `student` user on the `workstation` machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available.

This command initializes the remote Git repository that you need for this lab. When you are ready to push your changes to the remote repository, use `Stud3nt123` as the Git password.

```
[student@workstation ~]$ lab start compreview-cr1
```

Specifications

- Update an existing project that is stored in a Git repository:
  - The repository is stored at `https://git.lab.example.com/student/compreview-cr1.git`.
  - The repository must be cloned into the `/home/student/git-repos` directory.
  - All your work must take place within the `student/inventory-edits` branch.
  - Git must be configured to not track or commit files with the `.log` file extension, or files in the project's `collections/ansible_collections` directory.
  - Update the project `ansible.cfg` configuration file so that you can install collections from private automation hub. You can obtain an API token from `https://aap.lab.example.com` by logging in as `student` with `student` as the password. Install the collections defined in the project `collections/requirements.yml` file.
- Configure Ansible and automation content navigator as follows:

  - Automation content navigator must use the `aap.lab.example.com/ee-supported-rhel8:latest` image as the default automation execution environment, must only pull the image when it is missing, and must not generate any playbook artifact files.
  - Ensure that any dynamic inventory scripts are executable in the directory defined by the project `ansible.cfg` file.
  - Add the db host group to the project `inventory/inventory-static.yml` file with the `serverd.lab.example.com` host as the only member of the host group.

- Move playbook variables to inventory host group variables:
  - The project `haproxy.yml` playbook defines the `lb_service` and `firewall_services` variables. To make this playbook more flexible, move these variables out of the playbook, and into the `group_vars/haproxy/vars.yml` file.

- Configure inventory host group variables:
  - The project `webapp.yml` playbook requires that the `web_service` and `firewall_services` variables be defined. Create the `group_vars/web/vars.yml` file to define these variables for the web group with the following values:

Variable	Value
`web_service`	`httpd`
`firewall_services`	`http` and `https`

  - The project `db.yml` playbook runs plays against the db group. This playbook requires that the `db_service` and `db_port` variables be defined. Create the `group_vars/db/vars.yml` file to define these variables for the db group with the following values:

Variable	Value
`db_service`	`mariadb`
`db_port`	`3306/tcp`

- After all the variables are defined, use automation content navigator to run the following playbooks:
  - `haproxy.yml`

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

  - webapp.yml
  - db.yml

- Verify that web browser requests from `workstation` to the load balancer on `servera` succeed. The web application switches between displaying the following two messages:

```
This is serverb and the database is up.
This is serverc and the database is up.
```

- After you confirm that the web application is running, commit and push your changes to the `student/inventory-edits` remote branch.

Evaluation

As the `student` user on the `workstation` machine, use the `lab` command to grade your work. Correct any reported failures. Make sure to commit and push any additional changes to the remote Git repository and then rerun the command until successful.

```
[student@workstation ~]$ lab grade compreview-cr1
```

Finish

As the `student` user on the `workstation` machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish compreview-cr1
```

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Solution

Manage playbook and inventory host group variables, configure automation content navigator settings, and push your changes to a new branch using Git.

Outcomes

- Use Git to clone a repository, create a branch, ignore files, and push changes to a branch.
- Configure automation content navigator settings.
- Manage inventory hosts and host groups.
- Define variables for inventory host groups.

> **Note**
> The allocated time for this activity is 30 minutes. If you need additional time to complete the task, then you must revisit the course content again, or practice more.

Prerequisites

As the `student` user on the `workstation` machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available.

This command initializes the remote Git repository that you need for this lab. When you are ready to push your changes to the remote repository, use `Stud3nt123` as the Git password.

```
[student@workstation ~]$ lab start compreview-cr1
```

Specifications

- Update an existing project that is stored in a Git repository:
  - The repository is stored at `https://git.lab.example.com/student/compreview-cr1.git`.
  - The repository must be cloned into the `/home/student/git-repos` directory.
  - All your work must take place within the `student/inventory-edits` branch.
  - Git must be configured to not track or commit files with the `.log` file extension, or files in the project's `collections/ansible_collections` directory.
  - Update the project `ansible.cfg` configuration file so that you can install collections from private automation hub. You can obtain an API token from `https://aap.lab.example.com` by logging in as `student` with `student` as the password. Install the collections defined in the project `collections/requirements.yml` file.
- Configure Ansible and automation content navigator as follows:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

  - Automation content navigator must use the `aap.lab.example.com/ee-supported-rhel8:latest` image as the default automation execution environment, must only pull the image when it is missing, and must not generate any playbook artifact files.
  - Ensure that any dynamic inventory scripts are executable in the directory defined by the project `ansible.cfg` file.
  - Add the db host group to the project `inventory/inventory-static.yml` file with the `serverd.lab.example.com` host as the only member of the host group.

- Move playbook variables to inventory host group variables:
  - The project `haproxy.yml` playbook defines the `lb_service` and `firewall_services` variables. To make this playbook more flexible, move these variables out of the playbook, and into the `group_vars/haproxy/vars.yml` file.

- Configure inventory host group variables:
  - The project `webapp.yml` playbook requires that the `web_service` and `firewall_services` variables be defined. Create the `group_vars/web/vars.yml` file to define these variables for the web group with the following values:

Variable	Value
`web_service`	`httpd`
`firewall_services`	`http` and `https`

  - The project `db.yml` playbook runs plays against the db group. This playbook requires that the `db_service` and `db_port` variables be defined. Create the `group_vars/db/vars.yml` file to define these variables for the db group with the following values:

Variable	Value
`db_service`	`mariadb`
`db_port`	`3306/tcp`

- After all the variables are defined, use automation content navigator to run the following playbooks:
  - `haproxy.yml`

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

  - webapp.yml
  - db.yml

- Verify that web browser requests from `workstation` to the load balancer on `servera` succeed. The web application switches between displaying the following two messages:

```
This is serverb and the database is up.
This is serverc and the database is up.
```

- After you confirm that the web application is running, commit and push your changes to the `student/inventory-edits` remote branch.

Solution

1. On the `workstation` machine, clone the `https://git.lab.example.com/student/compreview-cr1.git` repository into the `/home/student/git-repos` directory, and create a branch named `student/inventory-edits`.

2. Configure Git to ignore any files with the `.log` file extension and the `collections/ansible_collections` directory.

3. Configure the project so that it can install collections from private automation hub. Go to `https://aap.lab.example.com` and obtain a token as the `student` user. Add the `token` into the `ansible.cfg` file. Use the `ansible-galaxy` command to install the `community.mysql` content collection into the `collections` directory.

4. Configure automation content navigator to use the `aap.lab.example.com/ee-supported-rhel8:latest` image as the default automation execution environment, and to pull the image only when it is missing. Automation content navigator should not generate playbook artifacts.

> **Note**
> If you prefer, you can use the `ansible-navigator settings` command (with options) to generate an initial automation content navigator settings file.

5. Verify the existing hosts and host groups available to the project, and resolve any inventory issues. Add the db host group to the `inventory/inventory-static.yml` file with the `serverd.lab.example.com` host as the only member of the host group.

6. Move the variables from the `haproxy.yml` playbook into `group_vars/haproxy/vars.yml` to make it more flexible. Create the `group_vars/web/vars.yml` file and add the `web_service` and `firewall_services` variables to it. Create the `group_vars/db/vars.yml` file and add the `db_service` and `db_port` variables to it.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

7. Use automation content navigator to run the playbooks. Run the `haproxy.yml` playbook, the `webapp.yml` playbook, and then the `db.yml` playbook.

8. Verify that the web application is reachable on the network.

9. Push your changes to the `student/inventory-edits` branch.

Evaluation

As the `student` user on the `workstation` machine, use the `lab` command to grade your work. Correct any reported failures. Make sure to commit and push any additional changes to the remote Git repository and then rerun the command until successful.

```
[student@workstation ~]$ lab grade compreview-cr1
```

Finish

As the `student` user on the `workstation` machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish compreview-cr1
```

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

10.3. Lab
Optimizing a Playbook for Large-scale Use

Use filters and lookup plug-ins to get and format values for variables, modify a play to implement rolling updates, and optimize the execution of plays and tasks in a playbook.

Outcomes

- Use filters and lookup plug-ins to manipulate variables in play and role tasks.
- Delegate tasks to other hosts, run hosts through plays in batches by using the `serial` keyword, and limit failures with the `max_fail_percentage` keyword.
- Set privilege escalation per play or per task.
- Add tags to target specific tasks.

> **Note**
> The allocated time for this activity is 30 minutes. If you need additional time to complete the task, then you must revisit the course content again, or practice more.

Prerequisites

As the `student` user on the `workstation` machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available.

This command ensures that the remote `https://git.lab.example.com/student/compreview-cr2.git` repository is initialized. The Git repository contains playbooks that configure a front-end load balancer and a pool of back-end web servers. You can push changes to this repository using `Stud3nt123` as the Git password.

```
[student@workstation ~]$ lab start compreview-cr2
```

Specifications

- Create the `/home/student/git-repos` directory if it does not exist. Clone the `https://git.lab.example.com/student/compreview-cr2.git` repository into the `/home/student/git-repos` directory. Create the `exercise` branch for this exercise to store your changes.
- Configure the project's `ansible.cfg` file so that privilege escalation is not performed by default. Edit the plays in the `deploy_apache.yml`, `deploy_haproxy.yml`, and `deploy_webapp.yml` playbooks so that privilege escalation is enabled at the play level. Privilege escalation is not required for fact gathering tasks.
- Edit the `roles/firewall/tasks/main.yml` tasks file so that it uses filters to set default values for variables in each of the three tasks if they are not set, as follows:
  - For the `state` option, if `item['state']` is not set, then set it to `enabled` by default.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

  - For the `zone` option, if `item['zone']` is not set, then omit the `zone` option.
  - In the `Ensure Firewall Port Configuration` task, in the `port` option to the `firewalld` module, if `item['protocol']` is not set, then set it to `tcp`. Use the `lower` filter to ensure that its value is in lowercase.
  - You can test your changes to the `roles/firewall/tasks/main.yml` file by running the `test_firewall_role.yml` playbook.

- Edit the `deploy_apache.yml` playbook. Change the value of the `firewall_rules` variable to a Jinja2 expression that uses the `template` lookup plug-in to dynamically generate the variable's setting from the `apache_firewall_rules.yml.j2` Jinja2 template. Use the `from_yaml` filter in the Jinja2 expression to convert the resulting value from a text string into a YAML data structure that Ansible can interpret.

- Edit the `templates/apache_firewall_rules.yml.j2` template to replace the `load_balancer_ip_addr` host variable in the Jinja2 `for` loop with the `hostvars[server]['ansible_facts']['default_ipv4']['address']` fact.

- In the `apache` role, refactor the task that took the longest time to complete to make it more efficient.
  - Enable the `timer` and `profile_tasks` callback plug-ins for the project.
  - Using the `ee-supported-rhel8:latest` automation execution environment, run the `site.yml` playbook and analyze the output to identify the task that takes the longest to complete.
  - Refactor that time-expensive task.
  - Add the `apache_installer` tag to the refactored task.
  - To verify your work, run the `cleanup_apache.yml` playbook to clean up the web servers, and then rerun the `site.yml` playbook, but limit the execution to the task with the `apache_installer` tag.

- The `update_webapp.yml` playbook performs a rolling update of the web content on the back-end web servers. It is not yet functional. Edit the `update_webapp.yml` playbook as follows:
  - Add a `pre_tasks` section to the play. In that section, add a task that uses the `community.general.haproxy` module to disable the web servers on the HAProxy load balancer. Disable the host by using the `inventory_hostname` variable in the app back end. Delegate the task to the load balancer. The `{{ groups['lb_servers'][0] }}` Jinja2 expression provides the name of this load balancer.
  - Add a task at the end of the `post_tasks` section to re-enable the web servers.
  - Configure the play in the playbook to run in batches. The first batch must contain 5% of the hosts, the second batch 35% of the hosts, and the final batch all remaining hosts in the play.
  - Set `max_fail_percentage` on the play with a value that ensures that playbook execution stops if any host fails a task during the execution of the play.
  - To verify your work, run the `update_webapp.yml` playbook.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

- To verify the correct deployment of the load balancer and the web servers, run the `curl servera` command several times from the `workstation` machine. The HAProxy server that you installed on the `servera` machine dispatches the requests between the back-end web servers that are installed on the `serverb` to `serverf` machines. The command must return the name of a different back-end web server each time you run it.

```
[student@workstation compreview-cr2]$ curl servera
This is serverd. (version v1.1)
[student@workstation compreview-cr2]$ curl servera
This is servere. (version v1.1)
[student@workstation compreview-cr2]$ curl servera
This is serverf. (version v1.1)
```

- Commit and push your changes to the `exercise` branch. If prompted, use `Stud3nt123` as the password.

Evaluation

As the `student` user on the `workstation` machine, use the `lab` command to grade your work. Correct any reported failures. Make sure to commit and push any additional changes to the remote Git repository and then rerun the command until successful.

```
[student@workstation ~]$ lab grade compreview-cr2
```

Finish

As the `student` user on the `workstation` machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish compreview-cr2
```

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Solution

Use filters and lookup plug-ins to get and format values for variables, modify a play to implement rolling updates, and optimize the execution of plays and tasks in a playbook.

Outcomes

- Use filters and lookup plug-ins to manipulate variables in play and role tasks.
- Delegate tasks to other hosts, run hosts through plays in batches by using the `serial` keyword, and limit failures with the `max_fail_percentage` keyword.
- Set privilege escalation per play or per task.
- Add tags to target specific tasks.

> **Note**
> The allocated time for this activity is 30 minutes. If you need additional time to complete the task, then you must revisit the course content again, or practice more.

Prerequisites

As the `student` user on the `workstation` machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available.

This command ensures that the remote `https://git.lab.example.com/student/compreview-cr2.git` repository is initialized. The Git repository contains playbooks that configure a front-end load balancer and a pool of back-end web servers. You can push changes to this repository using `Stud3nt123` as the Git password.

```
[student@workstation ~]$ lab start compreview-cr2
```

Specifications

- Create the `/home/student/git-repos` directory if it does not exist. Clone the `https://git.lab.example.com/student/compreview-cr2.git` repository into the `/home/student/git-repos` directory. Create the `exercise` branch for this exercise to store your changes.
- Configure the project's `ansible.cfg` file so that privilege escalation is not performed by default. Edit the plays in the `deploy_apache.yml`, `deploy_haproxy.yml`, and `deploy_webapp.yml` playbooks so that privilege escalation is enabled at the play level. Privilege escalation is not required for fact gathering tasks.
- Edit the `roles/firewall/tasks/main.yml` tasks file so that it uses filters to set default values for variables in each of the three tasks if they are not set, as follows:
  - For the `state` option, if `item['state']` is not set, then set it to `enabled` by default.
  - For the `zone` option, if `item['zone']` is not set, then omit the `zone` option.

  - In the `Ensure Firewall Port Configuration` task, in the `port` option to the `firewalld` module, if `item['protocol']` is not set, then set it to `tcp`. Use the `lower` filter to ensure that its value is in lowercase.
  - You can test your changes to the `roles/firewall/tasks/main.yml` file by running the `test_firewall_role.yml` playbook.

- Edit the `deploy_apache.yml` playbook. Change the value of the `firewall_rules` variable to a Jinja2 expression that uses the `template` lookup plug-in to dynamically generate the variable's setting from the `apache_firewall_rules.yml.j2` Jinja2 template. Use the `from_yaml` filter in the Jinja2 expression to convert the resulting value from a text string into a YAML data structure that Ansible can interpret.

- Edit the `templates/apache_firewall_rules.yml.j2` template to replace the `load_balancer_ip_addr` host variable in the Jinja2 `for` loop with the `hostvars[server]['ansible_facts']['default_ipv4']['address']` fact.

- In the `apache` role, refactor the task that took the longest time to complete to make it more efficient.
  - Enable the `timer` and `profile_tasks` callback plug-ins for the project.
  - Using the `ee-supported-rhel8:latest` automation execution environment, run the `site.yml` playbook and analyze the output to identify the task that takes the longest to complete.
  - Refactor that time-expensive task.
  - Add the `apache_installer` tag to the refactored task.
  - To verify your work, run the `cleanup_apache.yml` playbook to clean up the web servers, and then rerun the `site.yml` playbook, but limit the execution to the task with the `apache_installer` tag.

- The `update_webapp.yml` playbook performs a rolling update of the web content on the back-end web servers. It is not yet functional. Edit the `update_webapp.yml` playbook as follows:
  - Add a `pre_tasks` section to the play. In that section, add a task that uses the `community.general.haproxy` module to disable the web servers on the HAProxy load balancer. Disable the host by using the `inventory_hostname` variable in the app back end. Delegate the task to the load balancer. The `{{ groups['lb_servers'][0] }}` Jinja2 expression provides the name of this load balancer.
  - Add a task at the end of the `post_tasks` section to re-enable the web servers.
  - Configure the play in the playbook to run in batches. The first batch must contain 5% of the hosts, the second batch 35% of the hosts, and the final batch all remaining hosts in the play.
  - Set `max_fail_percentage` on the play with a value that ensures that playbook execution stops if any host fails a task during the execution of the play.
  - To verify your work, run the `update_webapp.yml` playbook.

- To verify the correct deployment of the load balancer and the web servers, run the `curl servera` command several times from the `workstation` machine. The HAProxy server that you installed on the `servera`

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

machine dispatches the requests between the back-end web servers that are installed on the `serverb` to `serverf` machines. The command must return the name of a different back-end web server each time you run it.

```
[student@workstation compreview-cr2]$ curl servera
This is serverd. (version v1.1)
[student@workstation compreview-cr2]$ curl servera
This is servere. (version v1.1)
[student@workstation compreview-cr2]$ curl servera
This is serverf. (version v1.1)
```

- Commit and push your changes to the `exercise` branch. If prompted, use `Stud3nt123` as the password.

Solution

1. Clone the `https://git.lab.example.com/student/compreview-cr2.git` repository into the `/home/student/git-repos` directory and then create and switch to the `exercise` branch.

2. Modify the `ansible.cfg` file so that privilege escalation is not performed by default. Enable privilege escalation for the `Ensure Apache is deployed` play in the `deploy_apache.yml` playbook. Enable privilege escalation for the second play in the `deploy_haproxy.yml` playbook. Enable privilege escalation for the play in the `deploy_webapp.yml` playbook.

3. Edit each of the tasks in the `roles/firewall/tasks/main.yml` tasks file to use filters to set default values for specific variables if they do not have a value set.
   - For the `state` option, if `item['state']` is not set, then set it to `enabled` by default.
   - For the `zone` option, if `item['zone']` is not set, then omit the `zone` option.
   - In the `Ensure Firewall Port Configuration` task, in the `port` option to the `firewalld` module, if `item['protocol']` is not set, then set it to `tcp`. Use the `lower` filter to ensure that its value is in lowercase.

   Test the changes that you made to the `roles/firewall/tasks/main.yml` file by running the `test_firewall_role.yml` playbook.

4. Examine the `deploy_apache.yml` playbook. The `8008/tcp` port for `firewalld` and the IPv4 address of the load balancer (`172.25.250.10`) must be enabled in the `internal` zone for `firewalld`. Change the value of the `firewall_rules` variable to a Jinja2 expression that uses the `template` lookup plug-in to dynamically generate the variable's setting from the `apache_firewall_rules.yml.j2` Jinja2 template. Use the `from_yaml` filter in the Jinja2 expression to convert the resulting value from a text string into a YAML data structure that Ansible can interpret.

5. Examine the `templates/apache_firewall_rules.yml.j2` template that your playbook now uses to set the `firewall_rules` variable. Edit the `templates/apache_firewall_rules.yml.j2` file to replace the

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

`load_balancer_ip_addr` host variable in the Jinja2 `for` loop with the `hostvars[server]['ansible_facts']['default_ipv4']['address']` fact.

6. Enable the `timer` and `profile_tasks` callback plug-ins for the project.

7. Run the `site.yml` playbook and analyze the output to find the task that uses the most time.

8. Refactor the task that took the longest time to complete to make it more efficient. Identify the file that contains the `Ensure httpd packages are installed` task. Edit the task file to remove the loop from the `dnf` task. Add the `apache_installer` tag to the task. Run the `cleanup_apache.yml` playbook to clean up the web servers. Run the `site.yml` playbook with the `apache_installer` tag. Verify that the `Ensure httpd packages are installed` task takes less time to complete.

9. Add a `pre_tasks` section and task to the play in the `update_webapp.yml` playbook. This task should disable the web servers on the HAProxy load balancer. Without this task, external clients might experience problems due to unforeseen deployment issues with the web application.

   Configure the new task as follows:

   - Use the `community.general.haproxy` module.
   - Disable the host by using the `inventory_hostname` variable in the app back end.
   - Delegate each action to the load balancer. The `{{ groups['lb_servers'][0] }}` Jinja2 expression provides the name of this load balancer.

10. In the play in the `update_webapp.yml` playbook, add a task to the `post_tasks` section after the smoke test to re-enable each web server in the HAProxy load balancer. This task is similar in structure to the `community.general.haproxy` task in the `pre_tasks` section, but uses the `state: enabled` directive.

11. Configure the play in the `update_webapp.yml` playbook to run in batches. This change mitigates the effects of unforeseen deployment errors.

    - Add the `max_fail_percentage` directive to the play and set its value to `0`. This setting ensures that playbook execution stops if any host fails a task during the upgrade.
    - Add the `serial` directive and set the value to a list of three items, 5%, 35%, and 100%. This setting ensures that the playbook uses no more than three batches to complete the upgrade of all web server hosts.

12. Run the `update_webapp.yml` playbook to perform a rolling update. Notice that Ansible first updates the `serverb.lab.example.com` machine, which is the only machine in the first batch (5% of 5 machines). Then Ansible updates the `serverc.lab.example.com` machine, which is the only machine in the second batch (35% of 5 machines). Finally, Ansible updates the remaining machines (100% of the machines in the last batch).

13. Verify that web browser requests from `workstation` to the load balancer on `servera` succeed.

14. Add the changed files, commit the changes, and push them to the Git repository.

Evaluation

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

As the student user on the workstation machine, use the lab command to grade your work. Correct any reported failures. Make sure to commit and push any additional changes to the remote Git repository and then rerun the command until successful.

```
[student@workstation ~]$ lab grade compreview-cr2
```

Finish

As the student user on the workstation machine, use the lab command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish compreview-cr2
```

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

10.4. Lab
Creating and Using Ansible Content Collections and Automation Execution Environments

Create an Ansible Content Collection and an automation execution environment, and develop a playbook to test the automation execution environment from automation controller.

Outcomes

- Create an Ansible Content Collection that includes roles.
- Create a custom automation execution environment.
- Upload an Ansible Content Collection and an automation execution environment to private automation hub.
- Configure automation controller to use a custom automation execution environment from private automation hub.
- Create automation controller resources such as credentials, projects, and inventories.
- Create and launch a new job template using a specific automation execution environment.

> **Note**
> The allocated time for this activity is 30 minutes. If you need additional time to complete the task, then you must revisit the course content again, or practice more.

Prerequisites

As the `student` user on the `workstation` machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
[student@workstation ~]$ lab start compreview-cr3
```

Specifications

You can access the Ansible Automation Platform unified UI at `https://aap.lab.example.com`. Log in as `admin` using `redhat` as the password.

- Create an Ansible Content Collection as follows:
  - Name the collection `review.system`.
  - Initialize a directory for the collection on `workstation` in the `/home/student/compreview-cr3` directory.
  - The collection must require Ansible version `>=2.15.0`.
  - Add the `iscsi_target_create` role to the collection. Copy the role from the `/home/student/compreview-cr3/roles` directory to the correct location in the collection's

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

directory structure.

- The `iscsi_target_create` role requires the `ansible.posix` collection version 1.0.0 or later. Add that dependency to the collection.
- After you build the `review.system` collection, copy the resulting compressed tar archive to the `/home/student/compreview-cr3` directory.
- On your private automation hub, create a namespace for the collection, upload the collection, and then approve the collection.

> **Important**
> If you previously completed this exercise, then you must edit the collection `galaxy.yml` file to increment the collection version to a number greater than the previous version, such as to `1.0.1`. Failure to update the version might result in an error similar to the following when you attempt to install the collection:
>
> ```
> ERROR! Failed to download collection tar from 'published' due to the
> following unforeseen error: Mismatch artifact hash with downloaded file
> ```

- Create a custom automation execution environment as follows:
  - Use the `/home/student/ee-custom` directory to store the configuration files needed to build a custom automation execution environment.
  - Use `aap.lab.example.com/ee-minimal-rhel8:latest` as the base container image.
  - The build environment must be configured to retrieve Ansible Content Collections from the `rh-certified` and `published` repositories of your private automation hub. Ensure that an `ansible.cfg` file containing the appropriate settings is available to the build process.
  - The automation execution environment must include the `review.system` and `ansible.posix` Ansible Content Collections.
  - To install collections from your private automation hub, the build process must trust that server's certificate. Copy the `/etc/pki/tls/certs/classroom-ca.pem` file to the `/home/student/ee-custom` directory and then use that file to update the certificate authority trust of the container image.
  - Define `/usr/bin/microdnf` as the executable to install software.
  - Configure the build process so that it can install RPM and Python packages in a disconnected environment. Add the following build steps to the `prepend_base` key of the `additional_build_steps` parameter to use an alternative Python package index server and to disable existing RPM repositories. You must also ensure that the build process uses the `/home/student/ee-custom/ee.repo` file to install RPM packages.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
- ENV PIP_INDEX_URL="http://workstation.lab.example.com:8000/simple/"
- ENV PIP_TRUSTED_HOST="workstation.lab.example.com:8000"
- RUN sed -i "s/^enabled.*$/enabled = 0/" /etc/yum.repos.d/*.repo
```

  - The automation execution environment must be tagged as `aap.lab.example.com/system/ee-review-rhel8:1.0`.

- Prepare an Ansible Playbook that tests your `review.system` Ansible Content Collection and custom automation execution environment, as follows:
  - A partially completed playbook named `demo.yml` is available in the Git repository at `https://git.lab.example.com/student/iscsi.git`. Clone that repository to the `/home/student/git-repos` directory on `workstation`.
  - Create the `review3` branch to store your modifications.
  - In your branch, edit the `demo.yml` playbook. The play in that playbook must call the `review.system.iscsi_target_create` role and must set `sdb` as the value of the `iscsi_target_create_disk` variable.

> **Important**
> The `/dev/sdb` disk on the `serverc` machine was created for this exercise. Do not use any other drive than this one, or you risk losing access to `serverc`.

  - Commit changes to the `demo.yml` playbook to your local repository and then push your new branch to the remote repository. Use `Stud3nt123` as the Git password.

- Configure your automation controller with the following resources. Create a job template to run your playbook to test your Ansible Content Collection and custom automation execution environment.
  - Create a machine credential that enables access to the managed node:

Field	Value
Name	`DevOps`
Organization	`Default`
Credential type	`Machine`
Username	`devops`

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Field	Value
Password	*(empty)*
SCM Private Key	Content of the `/home/student/.ssh/lab_rsa` file.

> **Note**
> If you choose to browse for the file, then right-click anywhere in the directory navigation area and select **Show Hidden Files**. Viewing hidden files enables you to view the `.ssh` directory in the `/home/student` directory.

– Create a source control credential that enables access to the Git repository:

Field	Value
Name	`GitLab`
Organization	`Default`
Credential type	`Source Control`
Username	`student`
SCM Private Key	Content of the `/home/student/.ssh/gitlab_rsa` file.

– Create an inventory resource named `Review 3` that belongs to the `Default` organization.

– Add the `serverc.lab.example.com` managed node to the `Review 3` inventory.

– Create a project resource. Use the following settings:

Field	Value
Name	`iSCSI`
Organization	`Default`

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Field	Value
Source control type	Git
Source control URL	git@git.lab.example.com:student/iscsi.git
Source control credential	GitLab
Options	Allow branch override *(selected)*

– Create an automation execution environment resource for your custom automation execution environment. Use the following settings:

Field	Value
Name	Review 3
Image	aap.lab.example.com/system/ee-review-rhel8:1.0
Pull	Always pull the container before running.
Organization	Default

– Create a job template resource that you can use to run your playbook on the `serverc.lab.example.com` managed node. Use the following settings:

Field	Value
Name	Configure iSCSI
Inventory	Review 3
Project	iSCSI

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Field	Value
Playbook	`demo.yml`
Source control branch	`review3`
Execution environment	`Review 3`
Credentials	`DevOps	Machine`

- When everything is configured, start a job from the `Configure iSCSI` job template resource. To confirm that the job is successful, you can verify that the `/etc/target/saveconfig.json` file is created on the `serverc.lab.example.com` machine.

Evaluation

As the `student` user on the `workstation` machine, use the `lab` command to grade your work. Correct any reported failures. Make sure to commit and push any additional changes to the remote Git repository and then rerun the command until successful.

```
[student@workstation ~]$ lab grade compreview-cr3
```

Finish

As the `student` user on the `workstation` machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish compreview-cr3
```

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Solution

Create an Ansible Content Collection and an automation execution environment, and develop a playbook to test the automation execution environment from automation controller.

Outcomes

- Create an Ansible Content Collection that includes roles.
- Create a custom automation execution environment.
- Upload an Ansible Content Collection and an automation execution environment to private automation hub.
- Configure automation controller to use a custom automation execution environment from private automation hub.
- Create automation controller resources such as credentials, projects, and inventories.
- Create and launch a new job template using a specific automation execution environment.

> **Note**
> The allocated time for this activity is 30 minutes. If you need additional time to complete the task, then you must revisit the course content again, or practice more.

Prerequisites

As the `student` user on the `workstation` machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
[student@workstation ~]$ lab start compreview-cr3
```

Specifications

You can access the Ansible Automation Platform unified UI at `https://aap.lab.example.com`. Log in as `admin` using `redhat` as the password.

- Create an Ansible Content Collection as follows:
  - Name the collection `review.system`.
  - Initialize a directory for the collection on `workstation` in the `/home/student/compreview-cr3` directory.
  - The collection must require Ansible version `>=2.15.0`.
  - Add the `iscsi_target_create` role to the collection. Copy the role from the `/home/student/compreview-cr3/roles` directory to the correct location in the collection's directory structure.

  - The `iscsi_target_create` role requires the `ansible.posix` collection version 1.0.0 or later. Add that dependency to the collection.
  - After you build the `review.system` collection, copy the resulting compressed tar archive to the `/home/student/compreview-cr3` directory.
  - On your private automation hub, create a namespace for the collection, upload the collection, and then approve the collection.

> **Important**
> If you previously completed this exercise, then you must edit the collection `galaxy.yml` file to increment the collection version to a number greater than the previous version, such as to `1.0.1`. Failure to update the version might result in an error similar to the following when you attempt to install the collection:
>
> ```
> ERROR! Failed to download collection tar from 'published' due to the
> following unforeseen error: Mismatch artifact hash with downloaded file
> ```

- Create a custom automation execution environment as follows:
  - Use the `/home/student/ee-custom` directory to store the configuration files needed to build a custom automation execution environment.
  - Use `aap.lab.example.com/ee-minimal-rhel8:latest` as the base container image.
  - The build environment must be configured to retrieve Ansible Content Collections from the `rh-certified` and `published` repositories of your private automation hub. Ensure that an `ansible.cfg` file containing the appropriate settings is available to the build process.
  - The automation execution environment must include the `review.system` and `ansible.posix` Ansible Content Collections.
  - To install collections from your private automation hub, the build process must trust that server's certificate. Copy the `/etc/pki/tls/certs/classroom-ca.pem` file to the `/home/student/ee-custom` directory and then use that file to update the certificate authority trust of the container image.
  - Define `/usr/bin/microdnf` as the executable to install software.
  - Configure the build process so that it can install RPM and Python packages in a disconnected environment. Add the following build steps to the `prepend_base` key of the `additional_build_steps` parameter to use an alternative Python package index server and to disable existing RPM repositories. You must also ensure that the build process uses the `/home/student/ee-custom/ee.repo` file to install RPM packages.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
- ENV PIP_INDEX_URL="http://workstation.lab.example.com:8000/simple/"
- ENV PIP_TRUSTED_HOST="workstation.lab.example.com:8000"
- RUN sed -i "s/^enabled.*$/enabled = 0/" /etc/yum.repos.d/*.repo
```

  - The automation execution environment must be tagged as `aap.lab.example.com/system/ee-review-rhel8:1.0`.

- Prepare an Ansible Playbook that tests your `review.system` Ansible Content Collection and custom automation execution environment, as follows:
  - A partially completed playbook named `demo.yml` is available in the Git repository at `https://git.lab.example.com/student/iscsi.git`. Clone that repository to the `/home/student/git-repos` directory on `workstation`.
  - Create the `review3` branch to store your modifications.
  - In your branch, edit the `demo.yml` playbook. The play in that playbook must call the `review.system.iscsi_target_create` role and must set `sdb` as the value of the `iscsi_target_create_disk` variable.

> **Important**
> The `/dev/sdb` disk on the `serverc` machine was created for this exercise. Do not use any other drive than this one, or you risk losing access to `serverc`.

  - Commit changes to the `demo.yml` playbook to your local repository and then push your new branch to the remote repository. Use `Stud3nt123` as the Git password.

- Configure your automation controller with the following resources. Create a job template to run your playbook to test your Ansible Content Collection and custom automation execution environment.
  - Create a machine credential that enables access to the managed node:

Field	Value
Name	`DevOps`
Organization	`Default`
Credential type	`Machine`
Username	`devops`

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Field	Value
Password	*(empty)*
SCM Private Key	Content of the `/home/student/.ssh/lab_rsa` file.

> **Note**
> If you choose to browse for the file, then right-click anywhere in the directory navigation area and select **Show Hidden Files**. Viewing hidden files enables you to view the `.ssh` directory in the `/home/student` directory.

– Create a source control credential that enables access to the Git repository:

Field	Value
Name	`GitLab`
Organization	`Default`
Credential type	`Source Control`
Username	`student`
SCM Private Key	Content of the `/home/student/.ssh/gitlab_rsa` file.

– Create an inventory resource named `Review 3` that belongs to the `Default` organization.

– Add the `serverc.lab.example.com` managed node to the `Review 3` inventory.

– Create a project resource. Use the following settings:

Field	Value
Name	`iSCSI`
Organization	`Default`

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Field	Value
Source control type	`Git`
Source control URL	`git@git.lab.example.com:student/iscsi.git`
Source control credential	`GitLab`
Options	`Allow branch override` *(selected)*

– Create an automation execution environment resource for your custom automation execution environment. Use the following settings:

Field	Value
Name	`Review 3`
Image	`aap.lab.example.com/system/ee-review-rhel8:1.0`
Pull	`Always pull the container before running.`
Organization	`Default`

– Create a job template resource that you can use to run your playbook on the `serverc.lab.example.com` managed node. Use the following settings:

Field	Value
Name	`Configure iSCSI`
Inventory	`Review 3`
Project	`iSCSI`

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

Field	Value
Playbook	demo.yml
Source control branch	review3
Execution environment	Review 3
Credentials	DevOps | Machine

- When everything is configured, start a job from the Configure iSCSI job template resource. To confirm that the job is successful, you can verify that the /etc/target/saveconfig.json file is created on the serverc.lab.example.com machine.

Solution

You can access the Ansible Automation Platform unified UI at https://aap.lab.example.com. Log in as admin using redhat as the password.

1. Create an Ansible Content Collection named review.system in the /home/student/compreview-cr3 directory. Copy the provided iscsi_target_create role to the ~/compreview-cr3/review/system/roles directory.

   Update the ~/compreview-cr3/review/system/meta/runtime.yml file to specify that your collection requires Ansible version 2.15.0 or later. Edit the ~/compreview-cr3/review/system/galaxy.yml file to declare the ansible.posix collection as a required dependency of your collection.

   Change to the ~/compreview-cr3/review/system directory and then build the collection. Copy the generated archive file to the ~/compreview-cr3 directory.

2. Publish the review.system collection so that the collection can be downloaded and installed from your private automation hub.

   Use review as the name of the namespace. Select the most recent version of the archive in the /home/student/compreview-cr3 directory, such as the review-system-1.0.0.tar.gz file, and then upload it. Approve the review.system collection. Confirm that ansible.posix >=1.0.0 is listed as a dependency.

3. Prepare the /home/student/ee-custom directory with the files required to build a custom automation execution environment.

   Leave the creation of the execution-environment.yml file for a later step. From the unified UI, go to **Automation Content > Repositories**. This page has the parameters that you need for creating the

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

`ansible.cfg` file.

Copy the CLI configuration for the `rh-certified` repository. Create the `~/ee-custom/ansible.cfg` file with the lines that you copied to the clipboard. Repeat the procedure with the `published` row.

Update the `ansible.cfg` file so that the first `[galaxy]` section lists both repositories, `rh-certified` and `published`. Update the `ansible.cfg` file so that both `url` lines point to the correct URL for the classroom private automation hub.

Generate a token and update the `ansible.cfg` file. Create the `~/ee-custom/requirements.yml` file and configure it to install the `review.system` collection into the automation execution environment. Copy the `/etc/pki/tls/certs/classroom-ca.pem` certificate authority file to the current directory.

4. Create the `~/ee-custom/execution-environment.yml` file with the necessary instructions for building an automation execution environment. Add the following parameters to the `~/ee-custom/execution-environment.yml` file:
   - `version`
   - `images`
   - `dependencies`
   - `options`
   - `additional_build_files`
   - `additional_build_steps`

   - Use version 3 of the Ansible Builder schema.
   - Use the `aap.lab.example.com/ee-minimal-rhel8:latest` image as the base image.
   - Edit the `dependencies` parameter to define the collection requirements.
   - Edit the `options` parameter to define the package manager path of `/usr/bin/microdnf`.
   - Edit the `additional_build_files` parameter to copy the `ee.repo`, `ansible.cfg`, and `classroom-ca.pem` files to the `context/_build` directory.
   - Add the `prepend_base` key to the `additional_build_steps` parameter with the following steps. These steps specify an alternative Python package index server, disable existing RPM repositories, and add the `ee.repo` file to the `/etc/yum.repos.d` directory:

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

```
...output omitted...

additional_build_steps:
  prepend_base:
    - ENV PIP_INDEX_URL="http://workstation.lab.example.com:8000/simple/"
    - ENV PIP_TRUSTED_HOST="workstation.lab.example.com:8000"
    - RUN sed -i "s/^enabled.*$/enabled = 0/" /etc/yum.repos.d/*.repo
    - COPY _build/ee.repo /etc/yum.repos.d/ee.repo
```

- Add steps to the `prepend_base` key to copy the `ansible.cfg` file to the `/etc/ansible` directory and to ensure that the container image trusts the private automation hub certificate:

```
...output omitted...

additional_build_steps:
  prepend_base:
    - ENV PIP_INDEX_URL="http://workstation.lab.example.com:8000/simple/"
    - ENV PIP_TRUSTED_HOST="workstation.lab.example.com:8000"
    - RUN sed -i "s/^enabled.*$/enabled = 0/" /etc/yum.repos.d/*.repo
    - COPY _build/ee.repo /etc/yum.repos.d/ee.repo
    - COPY _build/ansible.cfg /etc/ansible/ansible.cfg
    - COPY _build/classroom-ca.pem /etc/pki/ca-trust/source/anchors
    - RUN update-ca-trust
```

5. Build the automation execution environment and then push the container image to the `aap.lab.example.com` container registry. Add the `-t` option to specify the tag. Confirm that the new container image is available locally. Push the container image to your private automation hub.

6. Clone the `https://git.lab.example.com/student/iscsi.git` repository into the `/home/student/git-repos` directory and then create the `review3` branch. The `demo.yml` playbook must call the `review.system.iscsi_target_create` role. Set the `iscsi_target_create_disk` role variable to `sdb`. When done, commit and push your changes.

> **Important**
> The `/dev/sdb` disk on the `serverc` machine was created for this exercise. Do not use any other drive than this one, or you risk losing access to `serverc`.

Commit and then push your changes.

7. Create the credentials, inventory, project, and automation execution environment resources in Ansible Automation Platform.

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

8. Create the `Configure iSCSI` job template and start a job from it. Verify that the `demo.yml` playbook correctly configures the `serverc.lab.example.com` managed node.

   Confirm the successful execution of the job by verifying that the role created the `/etc/target/saveconfig.json` file on the managed node.

Evaluation

As the `student` user on the `workstation` machine, use the `lab` command to grade your work. Correct any reported failures. Make sure to commit and push any additional changes to the remote Git repository and then rerun the command until successful.

```
[student@workstation ~]$ lab grade compreview-cr3
```

Finish

As the `student` user on the `workstation` machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish compreview-cr3
```

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use

10.5. Summary

In this comprehensive review, you practiced what you learned in the *Developing Advanced Automation with Red Hat Ansible Automation Platform (AU374)* course.

Next Steps

Upon successful completion of all the exercise tasks in the allocated time, you might consider taking the *Developing Advanced Automation with Red Hat Ansible Automation Platform (EX374)* exam.

Another excellent next step would be to learn more about Red Hat Enterprise Linux Automation by taking the *Managing Enterprise Automation with Red Hat Ansible Automation Platform (DO467)* course.

References

Red Hat Certified Specialist in Developing Automation with Ansible Automation Platform exam

Managing Enterprise Automation with Red Hat Ansible Automation Platform

For use by Diego Gomez-Cantero diego.gomez-cantero@t-systems.com diego.gomez-cantero@t-systems.com Copyright © 2025 Red Hat, Inc. | For use